

APUNTES

Programación de Servicios y Procesos

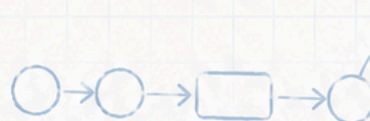
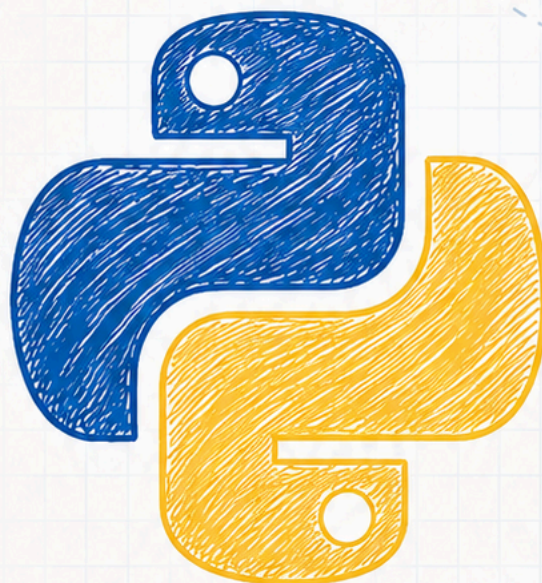
(Con Python)

```
1 import time
2 import socket
3 from multiprocessing import Process
4
5 def trabajo(nombre):
6     print(f"Iniciando {nombre}...")
7     time.sleep(2)
8     print(f"{nombre} finalizado.")
9
10 if __name__ == "__main__":
11     p = Process(target=trabajo, args=("Proceso",))
12     p.start()
12     p.join()
```

importaciones

definición de función

ejecución del proceso



Por
Sergi García Barea

licencia
CC BY SA



Contents

- [Inicio](#)
 - [📁 Unidades](#)
 - [📄 Boletines de ejercicios](#)
 - [📄 Licencia](#)
- [U01 — Procesos y Subprocess](#)
 - [🎯 Objetivo de la unidad](#)
 - [📖 Mapa de la unidad](#)
 - [📄 Boletines de la unidad](#)
 - [✅ Criterios de evaluación cubiertos \(RA1\)](#)
 - [📖 ¿Por dónde empiezo?](#)
- [01 — Qué es un proceso](#)
 - [📄 La idea en una frase](#)
 - [🧠 La burbuja de memoria](#)
 - [👤 El PID, el carnet de identidad](#)
 - [📋 Características de un proceso](#)
 - [🔍 La analogía de la cocina](#)
 - [🗣️ Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [02 — Estados de un proceso](#)
 - [📄 La idea en una frase](#)
 - [🔄 El ciclo de vida de un proceso](#)
 - [🚦 Las transiciones, una a una](#)
 - [☕ El ejemplo de la cafetería](#)
 - [🗣️ Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [03 — Paralela vs Distribuida](#)
 - [📄 La idea en una frase](#)
 - [👥 La tabla de los tres conceptos](#)
 - [🔍 Paralela: cuatro sartenes](#)
 - [🍽️ Distribuida: varios restaurantes](#)

-  [CPUs vs máquinas](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [04 — subprocess.run\(\)](#)
 -  [La idea en una frase](#)
 -  [El primer lanzamiento](#)
 -  [Los parámetros importantes](#)
 -  [Timeout: no esperes para siempre](#)
 -  [Un ejemplo con error controlado](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — subprocess.Popen\(\)](#)
 -  [La idea en una frase](#)
 -  [El primer Popen: abrir el bloc de notas](#)
 -  [Los métodos de Popen](#)
 -  [Lanzar varios procesos a la vez](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [06 — Comunicación con procesos](#)
 -  [La idea en una frase](#)
 -  [Conectar los tubos](#)
 -  [Enviar y recibir con communicate\(\)](#)
 -  [El mismo ejemplo completo, con sentido](#)
 -  [¿Cuándo usar cada herramienta?](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — Compatibilidad Windows / Linux](#)
 -  [La idea en una frase](#)
 -  [La tabla de comandos equivalentes](#)

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) El truco del shell](#)
- [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Lo que nunca cambia](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) Mini-chequeo](#)
- [!\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) Vocabulario rápido](#)
- [08 — Procesos en la práctica](#)
 - [!\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) La idea en una frase](#)
 - [!\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) Sé el código: abrir bloc de notas y calculadora](#)
 - [!\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) El ring de los conceptos — run\(\) vs Popen\(\)](#)
 - [!\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) Aprieta el lápiz](#)
 - [!\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 - [!\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) Sé el proceso](#)
 - [!\[\]\(b3461332979d340882fda628798720af_img.jpg\) Fireside Chat: Proceso vs Hilo](#)
 - [!\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) ¿Quién soy?](#)
 - [!\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) CONRAD VS EL MUNDO: “maté el proceso y no volvió”](#)
 - [!\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) Laboratorio de tortura: el lanzador con cierre automático](#)
 - [!\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) Logros de esta unidad](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) Atrévete a pensar](#)
 - [!\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) Crucigrama de bits](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) Entrevista de trabajo](#)
 - [!\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) No hay preguntas tontas](#)
 - [!\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) Poscréditos](#)
 - [!\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) Criterios de evaluación cubiertos \(RA1\)](#)
- [U02 — Hilos Fundamentos](#)
 - [!\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) Objetivo de la unidad](#)
 - [!\[\]\(4909cf475d986aa46b298b5fdca4ee07_img.jpg\) Mapa de la unidad](#)
 - [!\[\]\(a7d5e2c96349c1d63a82cd31458462fc_img.jpg\) Boletines de la unidad](#)
 - [!\[\]\(e28368bd9a276ed4dec00f0d27ffd39f_img.jpg\) Criterios de evaluación cubiertos \(RA2\)](#)
 - [!\[\]\(8085f10a37332aa3c492c2f85ed04dd1_img.jpg\) ¿Por dónde empiezo?](#)
- [01 — De proceso a hilo](#)

- [!\[\]\(71ac35c616fd8bfda805d579390e24d8_img.jpg\) La idea en una frase](#)
- [!\[\]\(b10a8b91056068472be58f587e00cb47_img.jpg\) ¿Qué es un hilo?](#)
- [!\[\]\(26a0aa65ffdf9b4c0922ec277970eeda_img.jpg\) Hilos vs Procesos](#)
- [!\[\]\(94aeee9c39a3a3d10654831c4bdd6b76_img.jpg\) La analogía de los chefs en una cocina](#)
- [!\[\]\(3e6c1aedeeaa8d5deb59d3ee4ab46da3_img.jpg\) Mini-chequeo](#)
- [!\[\]\(c902edf397a6ca641da2827a7619fb31_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(2eeb38d109c7620c04b72105577a1616_img.jpg\) Vocabulario rápido](#)
- [02 — Tu primer hilo](#)
 - [!\[\]\(9b13254820f9ffd91316055c68d8eb60_img.jpg\) La idea en una frase](#)
 - [!\[\]\(77cc4955267260b8e40fe850d4fd81f6_img.jpg\) Crear y lanzar un hilo](#)
 - [!\[\]\(34437df9eac3d056fab1af3d28d2b5ea_img.jpg\) El hilo principal vs los hilos secundarios](#)
 - [!\[\]\(422361fdcd577a4c2437f2e7efadccc8_img.jpg\) join\(\) — esperar a que termine](#)
 - [!\[\]\(484ab55bb87d294f9e51b0e2984d6d54_img.jpg\) Propiedades de un hilo](#)
 - [!\[\]\(0a023bb142905d93595b969588773668_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(5c387e5a75a789fa99a236343bca2d24_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(38bb85a21bd8aa529f78d3d8fa76b623_img.jpg\) Vocabulario rápido](#)
- [03 — Hilos con argumentos](#)
 - [!\[\]\(30e403e066c2b42c8cfd719e90afa03d_img.jpg\) La idea en una frase](#)
 - [!\[\]\(9ee64f081dfe5b318c871ded3b6c135a_img.jpg\) Pasar argumentos con args](#)
 - [!\[\]\(2e220993f164ea4f53759f06a2669171_img.jpg\) kwargs: argumentos con nombre](#)
 - [!\[\]\(f6f44624d5f5317708919f7a59aa59da_img.jpg\) Nombrar hilos: .name = "hilo-" + str\(n\)](#)
 - [!\[\]\(bab06d899f209ed3e1f8b1dd4347c39c_img.jpg\) Varios hilos a la vez](#)
 - [!\[\]\(d4ea5d8f03144c71af65d26ea0032b36_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(b848c3c83ae0e94e44846dbcec03a5f1_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(c077fcd639e908c7e6f0fb528e56aa0c_img.jpg\) Vocabulario rápido](#)
- [04 — Hilos daemon](#)
 - [!\[\]\(1d456d5e1c40bf1928b0b5d3828f1782_img.jpg\) La idea en una frase](#)
 - [!\[\]\(97c7af8063946a06d20e0721fdd81bc9_img.jpg\) ¿Qué es un hilo daemon?](#)
 - [!\[\]\(af00bd745e76224b66a8951c0fa75d1c_img.jpg\) Be the code, my friend — daemon en acción](#)
 - [!\[\]\(d1b72fa7e16b412af217cb0149f8a763_img.jpg\) El ring de los conceptos — Hilo normal vs Hilo daemon](#)
 - [!\[\]\(771ae2bf4438d1a4551cf1295d248035_img.jpg\) join\(\) para esperar a un daemon \(si acaso lo necesitas\)](#)
 - [!\[\]\(f86a93b388be2108085fba3a1da48ed9_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(129a49758668ae121809f74fc7cd9ce1_img.jpg\) Resumen en 3 frases](#)

- [!\[\]\(694fcb4611893e9db5249daba48abfc1_img.jpg\) Vocabulario rápido](#)
- [05 — Timer](#)
 - [!\[\]\(8ec8d5dc48934930a762fecf6ecbe179_img.jpg\) La idea en una frase](#)
 - [!\[\]\(c34a15e67573dae8fbb88f4cbfb0f2e9_img.jpg\) El ejemplo mínimo](#)
 - [!\[\]\(41f06fdeabb4e5a71d06fe8f32a46127_img.jpg\) Cancelar un Timer](#)
 - [!\[\]\(18eb66208e65404cce5042d73cf0a851_img.jpg\) Pasar argumentos al Timer](#)
 - [!\[\]\(14a9d4de9e6699d41b68e8807e2d5f76_img.jpg\) Usos típicos del Timer](#)
 - [!\[\]\(415790129e00c225ba52b81c8addfb14_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(fa8e43d6f5da9cf596808674ced6c198_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(2b564e327fe9708ac2f9320a9ae84c76_img.jpg\) Vocabulario rápido](#)
- [06 — El GIL](#)
 - [!\[\]\(484cd33a03c33977d2fcf6bb9cc02435_img.jpg\) La idea en una frase](#)
 - [!\[\]\(c885083f23ac65632c3cf77b16f7a193_img.jpg\) ¿Qué es el GIL?](#)
 - [!\[\]\(20f0f5805f0a60636883bdd13c58dc31_img.jpg\) CPU-bound: los hilos NO sirven de nada](#)
 - [!\[\]\(90f7aa1b3da6a942e186a7e3fdaaf44b_img.jpg\) I/O-bound: los hilos SÍ aceleran](#)
 - [!\[\]\(b2b96e5d9b571004907039c0a8a75b54_img.jpg\) La tabla que hay que saberse](#)
 - [!\[\]\(6285cf242551105b2a630c622e0c55ae_img.jpg\) Cuando sirven de verdad los hilos](#)
 - [!\[\]\(ea61eebb01e76a95ff1c56cf17f53608_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(bef5c8d7250866df258df26efebb2d6b_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(1855a3a8d9a41a3f4f1dc8d294fd804e_img.jpg\) Vocabulario rápido](#)
- [07 — Estados del hilo](#)
 - [!\[\]\(4bff6b8888da539b37a899aef567f1be_img.jpg\) La idea en una frase](#)
 - [!\[\]\(1a1b0261b18044c47100576c25c435f9_img.jpg\) El diagrama del ciclo de vida](#)
 - [!\[\]\(7b1db38756a02e50672c39a699367588_img.jpg\) Los estados, uno a uno](#)
 - [!\[\]\(5092e0d5a46a35d887ea84e10efda211_img.jpg\) El estado BLOQUEADO, el más importante](#)
 - [!\[\]\(222624bfbcd41cb4b8e992f2fb4d1d5d_img.jpg\) ¿Cuándo termina un hilo?](#)
 - [!\[\]\(90ca981cfd6ce35151faff84419b8399_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(4cdd93c76f3bb2c7ca3858ab7830b87e_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(cad3a662fcdef3adde1636c9b3b708d8_img.jpg\) Vocabulario rápido](#)
- [08 — Hilos en la práctica](#)
 - [!\[\]\(45e5f22e5e91d2d2349088139ffd9439_img.jpg\) La idea en una frase](#)
 - [!\[\]\(23345d74f4e8714038e154b94c68c7b8_img.jpg\) Be the code, my friend — El hilo viajero](#)
 - [!\[\]\(e04c62507b16f6eaa81ce61047658f95_img.jpg\) El ring de los conceptos — Hilo vs Proceso](#)

-  [Aprieta el lápiz](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 -  [Sé el hilo](#)
 -  [Fireside Chat: Hilo vs Proceso](#)
 -  [¿Quién soy?](#)
 -  [CONRAD VS EL MUNDO: “el programa termina antes que mis hilos”](#)
 -  [Laboratorio de tortura: la sala de descargas](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a pensar](#)
 -  [Crucigrama de bits](#)
 -  [Entrevista de trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Poscréditos](#)
 -  [Criterios de evaluación cubiertos \(RA2\)](#)
- [U03 — Sincronización entre Hilos](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA2\)](#)
 -  [¿Por dónde empiezo?](#)
- [01 — Condición de carrera](#)
 -  [La idea en una frase](#)
 -  [¿Por qué falla? El += no es atómico](#)
 -  [La analogía: dos cajeros y una caja](#)
 -  [Dónde aparece la condición de carrera](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [02 — Lock](#)
 -  [La idea en una frase](#)

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) Métodos de Lock](#)
- [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Siempre with lock:](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) Be the code: contador con y sin Lock](#)
- [!\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) Mini-chequeo](#)
- [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) Vocabulario rápido](#)
- [03 — RLock](#)
 - [!\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) La idea en una frase](#)
 - [!\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) ¿Cuándo lo necesitas? Funciones que se llaman entre sí](#)
 - [!\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) Lock vs RLock](#)
 - [!\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) Vocabulario rápido](#)
- [04 — Semaphore](#)
 - [!\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) La idea en una frase](#)
 - [!\[\]\(b3461332979d340882fda628798720af_img.jpg\) Semáforo con timeout](#)
 - [!\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) ¿Para qué sirve un semáforo?](#)
 - [!\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) Vocabulario rápido](#)
- [05 — Barrier](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) La idea en una frase](#)
 - [!\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) Sincronizar fases de un trabajo](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) Barrier frente al resto](#)
 - [!\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) Vocabulario rápido](#)
- [06 — Condition](#)
 - [!\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) La idea en una frase](#)
 - [!\[\]\(4909cf475d986aa46b298b5fdca4ee07_img.jpg\) Métodos de Condition](#)
 - [!\[\]\(a7d5e2c96349c1d63a82cd31458462fc_img.jpg\) La regla de oro: while, no if](#)
 - [!\[\]\(e28368bd9a276ed4dec00f0d27ffd39f_img.jpg\) ¿Cuándo usarla?](#)
 - [!\[\]\(8085f10a37332aa3c492c2f85ed04dd1_img.jpg\) Mini-chequeo](#)

- [!\[\]\(83eb2aa26b610eb6a9dca7cf4702d681_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(94dfacbf937cdd7da4837a6fcd8fc785_img.jpg\) Vocabulario rápido](#)
- [07 — Productor-Consumidor](#)
 - [!\[\]\(dae8c3c5fa7c80febd6526a5e8a853bf_img.jpg\) La idea en una frase](#)
 - [!\[\]\(8f38ab9775d1331a4e1fd6648d0a83f1_img.jpg\) Paso a paso](#)
 - [!\[\]\(5e48b3241d711ef916255d822ab3415f_img.jpg\) Con varios productores y consumidores](#)
 - [!\[\]\(c4c3604751fde0df44855086d7798e30_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(a16ee0329f478adb49fd7876490e96fe_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(a8a494e883c31c6f4a74584cc935a790_img.jpg\) Vocabulario rápido](#)
- [08 — Buenas prácticas](#)
 - [!\[\]\(8a50a875cd94cd57f6a7348a4d34b45f_img.jpg\) La idea en una frase](#)
 - [!\[\]\(7ddc0498d10972f0f157498fccf65370_img.jpg\) El deadlock: todos esperan al Lock](#)
 - [!\[\]\(1481f3a2b8d8d64dbbf1eb2242b57620_img.jpg\) Cómo evitar deadlocks: las 3 reglas](#)
 - [!\[\]\(1d12b5523dd7503f9f6b056602d4b0bb_img.jpg\) Preguntas rápidas \(y sus respuestas\)](#)
 - [!\[\]\(a3cf9896eece75d50b7dee194989c9bc_img.jpg\) El ring de los conceptos — Lock vs Semáforo vs Barrera](#)
 - [!\[\]\(26147612c3694ee09d020cac0faae7d9_img.jpg\) Aprieta el lápiz](#)
 - [!\[\]\(46af714d66844046465576855b12c56a_img.jpg\) Pool Puzzle de sincronización](#)
 - [!\[\]\(e123f1fe9eaf288ea9a688d32daa4ea4_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(780c9e9bdcf3e80b97a0a12adfab01dc_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(dd4f403c0f3886141896591186a0332a_img.jpg\) Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 - [!\[\]\(eba5717f9532900568c94b25862a3240_img.jpg\) Sé el lock](#)
 - [!\[\]\(8787ffe874068f6bc6ac6dfc8c73a8ea_img.jpg\) Fireside Chat: Lock vs Semaphore](#)
 - [!\[\]\(4bf65409525017dedd6ddb75688acc31_img.jpg\) ¿Quién soy?](#)
 - [!\[\]\(83155363cfc680b4fdc84409779c43c2_img.jpg\) CONRAD VS EL MUNDO: “olvidé el release\(\) y todos esperan al lock”](#)
 - [!\[\]\(7df4dc75718f5564bb085b99af0d0d10_img.jpg\) Laboratorio de tortura: la caja común de la cooperativa](#)
 - [!\[\]\(61d1cd78c9c864118ea1680747722675_img.jpg\) Logros de esta unidad](#)
 - [!\[\]\(f4289423d97bfe35ba5b95f76bb18c25_img.jpg\) Atrévete a pensar](#)
 - [!\[\]\(1645549eff763b7d16d331a24e8e7eb6_img.jpg\) Crucigrama de bits](#)
 - [!\[\]\(aa239fdf73afc4446284be0d9db1da52_img.jpg\) Entrevista de trabajo](#)
 - [!\[\]\(3fb3888bb2558834f9b320e257f20107_img.jpg\) No hay preguntas tontas](#)
 - [!\[\]\(505bbbf5324ed2982a8a1c174c3ae8c2_img.jpg\) Poscréditos](#)
 - [!\[\]\(d8271eb151bc5f54ccebd8d9719d01a6_img.jpg\) Criterios de evaluación cubiertos \(RA2\)](#)

- [U04 — Sockets TCP](#)
 - [🎯 Objetivo de la unidad](#)
 - [📖 Mapa de la unidad](#)
 - [📄 Boletines de la unidad](#)
 - [✅ Criterios de evaluación cubiertos \(RA3\)](#)
 - [🏠 ¿Por dónde empiezo?](#)
- [01 — Qué es un socket](#)
 - [🗉 La idea en una frase](#)
 - [🖨 IP y puerto: el “quién” y el “dónde”](#)
 - [🔄 Crear un socket en Python](#)
 - ☎ [La analogía del teléfono, al detalle](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦊 Vocabulario rápido](#)
- [02 — Cliente TCP](#)
 - [🗉 La idea en una frase](#)
 - [🔄 Ciclo de vida del cliente](#)
 - [🔄 El cliente mínimo resuelto](#)
 - [🧩 Cada método, al detalle](#)
 - [🎭 Be the code, my friend — Cliente TCP](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦊 Vocabulario rápido](#)
- [03 — Servidor TCP](#)
 - [🗉 La idea en una frase](#)
 - [🔄 Ciclo de vida del servidor](#)
 - [🔄 El servidor mínimo resuelto](#)
 - [🧩 Cada método, al detalle](#)
 - [📄 El bucle de atención](#)
 - [🧩 Pool Puzzle — El ciclo de vida del socket](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦊 Vocabulario rápido](#)

- [04 — Ciclo de vida de la conexión](#)
 - [📖 La idea en una frase](#)
 - [🧱 TCP: orientado a conexión](#)
 - [🤝 Three-way handshake \(establecer conexión\)](#)
 - [👋 Cierre de la conexión: la despedida](#)
 - [📷 El ciclo completo, en una mirada](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [05 — Errores y gestión](#)
 - [📖 La idea en una frase](#)
 - [🔒 La función conectar seguro](#)
 - [🧩 El mapa de excepciones](#)
 - [🕒 Non-blocking y timeouts](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [06 — SO_REUSEADDR](#)
 - [📖 La idea en una frase](#)
 - [😞 El problema: “Address already in use”](#)
 - [🔧 La solución: SO_REUSEADDR](#)
 - [🧠 ¿Por qué existe TIME_WAIT en primer lugar?](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [07 — Protocolos sobre TCP](#)
 - [📖 La idea en una frase](#)
 - [🌐 HTTP desde cero: el protocolo que mueve la web](#)
 - [🔢 El orden de los bytes \(byte ordering\)](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [08 — Servidor eco completo](#)

-  [La idea en una frase](#)
-  [El servidor eco](#)
-  [El cliente que lo prueba](#)
-  [Be the code, my friend — Mano a mano TCP](#)
-  [Mini-chequeo](#)
-  [Aprieta el lápiz](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 -  [Sé el socket](#)
 -  [Fireside Chat: Servidor vs Cliente](#)
 -  [¿Quién soy?](#)
 -  [CONRAD VS EL MUNDO: “conexión reiniciada por el host”](#)
 -  [Laboratorio de tortura: cliente y servidor TCP](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a pensar](#)
 -  [Crucigrama de bits](#)
 -  [Entrevista de trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Poscréditos](#)
 -  [Criterios de evaluación cubiertos \(RA3\)](#)
- [U05 — Sockets UDP y Protocolos](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA3\)](#)
 -  [¿Por dónde empiezo?](#)
- [01 — TCP vs UDP](#)
 -  [La idea en una frase](#)
 -  [La analogía: carta certificada contra avión de papel](#)
 -  [TCP vs UDP cara a cara](#)
 -  [El socket en código](#)
 -  [Fiabilidad vs velocidad: el verdadero dilema](#)

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) Mini-chequeo](#)
- [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) Vocabulario rápido](#)
- [02 — Cliente UDP](#)
 - [!\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) La idea en una frase](#)
 - [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) Enviar sin conectarse: sendto\(\)](#)
 - [!\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) Sendto contra send: la diferencia visual](#)
 - [!\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) Be the code: cliente UDP paso a paso](#)
 - [!\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) Vocabulario rápido](#)
- [03 — Servidor UDP](#)
 - [!\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) La idea en una frase](#)
 - [!\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) El servidor más pequeño del mundo](#)
 - [!\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) Sin accept\(\), sin listen\(\): la diferencia con TCP](#)
 - [!\[\]\(b3461332979d340882fda628798720af_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) Vocabulario rápido](#)
- [04 — Datagramas y pérdida](#)
 - [!\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) La idea en una frase](#)
 - [!\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) El datagrama: cada envío es un mundo aparte](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) Los tres males de UDP](#)
 - [!\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) ¿Por qué alguien elegiría esto?](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) Vocabulario rápido](#)
- [05 — HTTP desde cero](#)
 - [!\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) La idea en una frase](#)
 - [!\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) La petición HTTP](#)
 - [!\[\]\(4909cf475d986aa46b298b5fdca4ee07_img.jpg\) La respuesta HTTP](#)
 - [!\[\]\(a7d5e2c96349c1d63a82cd31458462fc_img.jpg\) Be the code, my friend: Cliente HTTP manual](#)
 - [!\[\]\(e28368bd9a276ed4dec00f0d27ffd39f_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(8085f10a37332aa3c492c2f85ed04dd1_img.jpg\) Resumen en 3 frases](#)

- [!\[\]\(83eb2aa26b610eb6a9dca7cf4702d681_img.jpg\) Vocabulario rápido](#)
- [06 — NTP y servidores de tiempo](#)
 - [!\[\]\(94dfacbf937cdd7da4837a6fcd8fc785_img.jpg\) La idea en una frase](#)
 - [!\[\]\(dae8c3c5fa7c80febd6526a5e8a853bf_img.jpg\) Por qué tu reloj no necesita ser atómico](#)
 - [!\[\]\(8f38ab9775d1331a4e1fd6648d0a83f1_img.jpg\) Cliente NTP manual en Python](#)
 - [!\[\]\(5e48b3241d711ef916255d822ab3415f_img.jpg\) SNTP: la versión ligera para dispositivos](#)
 - [!\[\]\(c4c3604751fde0df44855086d7798e30_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(a16ee0329f478adb49fd7876490e96fe_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(a8a494e883c31c6f4a74584cc935a790_img.jpg\) Vocabulario rápido](#)
- [07 — Cuándo usar cada protocolo](#)
 - [!\[\]\(8a50a875cd94cd57f6a7348a4d34b45f_img.jpg\) La idea en una frase](#)
 - [!\[\]\(7ddc0498d10972f0f157498fccf65370_img.jpg\) El criterio: velocidad o fiabilidad](#)
 - [!\[\]\(1481f3a2b8d8d64dbbf1eb2242b57620_img.jpg\) Casos reales: la tabla que se repite en las entrevistas](#)
 - [!\[\]\(1d12b5523dd7503f9f6b056602d4b0bb_img.jpg\) Y si quieres ambas cosas: QUIC](#)
 - [!\[\]\(a3cf9896eece75d50b7dee194989c9bc_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(26147612c3694ee09d020cac0faae7d9_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(46af714d66844046465576855b12c56a_img.jpg\) Vocabulario rápido](#)
- [08 — Práctica eco UDP](#)
 - [!\[\]\(e123f1fe9eaf288ea9a688d32daa4ea4_img.jpg\) La idea en una frase](#)
 - [!\[\]\(780c9e9bdcf3e80b97a0a12adfab01dc_img.jpg\) El servidor eco](#)
 - [!\[\]\(dd4f403c0f3886141896591186a0332a_img.jpg\) El cliente que lo prueba](#)
 - [!\[\]\(eba5717f9532900568c94b25862a3240_img.jpg\) Be the code: eco UDP paso a paso](#)
 - [!\[\]\(8787ffe874068f6bc6ac6dfc8c73a8ea_img.jpg\) Aprieta el lápiz](#)
 - [!\[\]\(4bf65409525017dedd6ddb75688acc31_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(83155363cfc680b4fdc84409779c43c2_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(7df4dc75718f5564bb085b99af0d0d10_img.jpg\) Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 - [!\[\]\(61d1cd78c9c864118ea1680747722675_img.jpg\) Sé el datagrama](#)
 - [!\[\]\(f4289423d97bfe35ba5b95f76bb18c25_img.jpg\) Fireside Chat: TCP vs UDP](#)
 - [!\[\]\(1645549eff763b7d16d331a24e8e7eb6_img.jpg\) ¿Quién soy?](#)
 - [!\[\]\(aa239fdf73afc4446284be0d9db1da52_img.jpg\) CONRAD VS EL MUNDO: “perdí paquetes y mi stream se corta”](#)
 - [!\[\]\(3fb3888bb2558834f9b320e257f20107_img.jpg\) Laboratorio de tortura: eco UDP en dos terminales](#)
 - [!\[\]\(505bbbf5324ed2982a8a1c174c3ae8c2_img.jpg\) Logros de esta unidad](#)

-  [Atrévete a pensar](#)
-  [Crucigrama de bits](#)
-  [Entrevista de trabajo](#)
-  [No hay preguntas tontas](#)
-  [Poscréditos](#)
-  [Criterios de evaluación cubiertos \(RA3\)](#)
- [U06 — APIs REST y HTTP](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA4a-b\)](#)
 -  [¿Por dónde empiezo?](#)
- [01 — Web y HTTP](#)
 -  [La idea en una frase](#)
 -  [¿Qué es una API?](#)
 -  [La web en tres piezas](#)
 -  [Petición y respuesta](#)
 -  [La analogía del pedido a domicilio](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [02 — Métodos HTTP](#)
 -  [La idea en una frase](#)
 -  [Los verbos de HTTP](#)
 -  [¿Qué es la idempotencia?](#)
 -  [Métodos en Python](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [03 — Principios REST](#)
 -  [La idea en una frase](#)
 -  [Recursos: las cosas, no las acciones](#)
 -  [Los cuatro pilares de REST](#)

-  [Sin estado \(stateless\)](#)
-  [¿Qué significa “RESTful”?](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [04 — Códigos de estado](#)
 -  [La idea en una frase](#)
 -  [Las familias de códigos](#)
 -  [La tabla completa](#)
 -  [Códigos en Python](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — JSON](#)
 -  [La idea en una frase](#)
 -  [El formato](#)
 -  [De JSON a Python y viceversa](#)
 -  [JSON dentro de requests](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [06 — requests: el GET](#)
 -  [La idea en una frase](#)
 -  [El GET más básico](#)
 -  [La respuesta y sus propiedades](#)
 -  [Parámetros de consulta](#)
 -  [Cabeceras personalizadas](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — requests: POST, PUT y DELETE](#)
 -  [La idea en una frase](#)
 -  [POST: crear un recurso](#)

-  [PUT y PATCH: actualizar](#)
-  [DELETE: borrar](#)
-  [Gestión de errores: status_code y raise_for_status](#)
-  [Pool Puzzle — Petición API con errores](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [08 — Práctica: mini cliente de API](#)
 -  [La idea en una frase](#)
 -  [El mini cliente de API](#)
 -  [Sé el código: una petición GET, milisegundo a milisegundo](#)
 -  [Aprieta el lápiz](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 -  [Sé la petición HTTP](#)
 -  [Fireside Chat: GET vs POST vs PUT vs DELETE](#)
 -  [¿Quién soy?](#)
 -  [CONRAD VS EL MUNDO: “404 que nadie entiende”](#)
 -  [Laboratorio de tortura: mini cliente contra una API pública](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a pensar](#)
 -  [Crucigrama de bits](#)
 -  [Entrevista de trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Poscréditos](#)
 -  [Criterios de evaluación cubiertos \(RA4a-b\)](#)
- [U07 — APIs Comerciales](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA4a-b\)](#)

- [!\[\]\(694fcb4611893e9db5249daba48abfc1_img.jpg\) ¿Por dónde empiezo?](#)
- [01 — API Keys](#)
 - [!\[\]\(8ec8d5dc48934930a762fecf6ecbe179_img.jpg\) La idea en una frase](#)
 - [!\[\]\(c34a15e67573dae8fbb88f4cbfb0f2e9_img.jpg\) ¿Qué es una API key?](#)
 - [!\[\]\(41f06fdeabb4e5a71d06fe8f32a46127_img.jpg\) ¿Cómo se obtiene?](#)
 - [!\[\]\(18eb66208e65404cce5042d73cf0a851_img.jpg\) ¿Dónde va la clave?](#)
 - [!\[\]\(14a9d4de9e6699d41b68e8807e2d5f76_img.jpg\) La analogía del llavero de acceso](#)
 - [!\[\]\(415790129e00c225ba52b81c8addfb14_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(fa8e43d6f5da9cf596808674ced6c198_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(2b564e327fe9708ac2f9320a9ae84c76_img.jpg\) Vocabulario rápido](#)
- [02 — Variables de entorno](#)
 - [!\[\]\(484cd33a03c33977d2fcf6bb9cc02435_img.jpg\) La idea en una frase](#)
 - [!\[\]\(c885083f23ac65632c3cf77b16f7a193_img.jpg\) El archivo .env](#)
 - [!\[\]\(20f0f5805f0a60636883bdd13c58dc31_img.jpg\) Cargarlo con python-dotenv](#)
 - [!\[\]\(90f7aa1b3da6a942e186a7e3fdaaf44b_img.jpg\) No subir claves al repo](#)
 - [!\[\]\(b2b96e5d9b571004907039c0a8a75b54_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(6285cf242551105b2a630c622e0c55ae_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(ea61eebb01e76a95ff1c56cf17f53608_img.jpg\) Vocabulario rápido](#)
- [03 — OpenWeatherMap](#)
 - [!\[\]\(bef5c8d7250866df258df26efebb2d6b_img.jpg\) La idea en una frase](#)
 - [!\[\]\(1855a3a8d9a41a3f4f1dc8d294fd804e_img.jpg\) Registro y API key](#)
 - [!\[\]\(4bff6b8888da539b37a899aef567f1be_img.jpg\) La URL y sus parámetros](#)
 - [!\[\]\(1a1b0261b18044c47100576c25c435f9_img.jpg\) Llamada y parseo](#)
 - [!\[\]\(7b1db38756a02e50672c39a699367588_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(5092e0d5a46a35d887ea84e10efda211_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(222624bfbcd41cb4b8e992f2fb4d1d5d_img.jpg\) Vocabulario rápido](#)
- [04 — OpenAI](#)
 - [!\[\]\(90ca981cfd6ce35151faff84419b8399_img.jpg\) La idea en una frase](#)
 - [!\[\]\(4cdd93c76f3bb2c7ca3858ab7830b87e_img.jpg\) Registro y clave](#)
 - [!\[\]\(cad3a662fcdef3adde1636c9b3b708d8_img.jpg\) Chat completions](#)
 - [!\[\]\(45e5f22e5e91d2d2349088139ffd9439_img.jpg\) Mensajes y roles](#)
 - [!\[\]\(23345d74f4e8714038e154b94c68c7b8_img.jpg\) Alternativa con httpx \(sin librería oficial\)](#)
 - [!\[\]\(e04c62507b16f6eaa81ce61047658f95_img.jpg\) Consumo y coste](#)

-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [05 — Rate limiting](#)
 -  [La idea en una frase](#)
 -  [¿Qué es el rate limit?](#)
 -  [El error 429](#)
 -  [Esperar antes de reintentar](#)
 -  [Reintentos inteligentes \(backoff\)](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [06 — Errores HTTP](#)
 -  [La idea en una frase](#)
 -  [Los códigos que verás](#)
 -  [raise_for_status\(\) y try/except](#)
 -  [Una llamada segura de verdad](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — Seguridad y buenas prácticas](#)
 -  [La idea en una frase](#)
 -  [Lo que nunca debes hacer](#)
 -  [El .gitignore y el commit](#)
 -  [Rotación de claves](#)
 -  [Checklist de buenas prácticas](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [08 — Práctica: APIs comerciales](#)
 -  [La idea en una frase](#)
 -  [El programa completo: tiempo + cerebro](#)
 -  [Sé el código: la consulta meteorológica](#)

-  [Herramientas para probar APIs](#)
-  [Aprieta el lápiz](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 -  [Sé la petición](#)
 -  [Fireside Chat: REST vs SOAP vs GraphQL](#)
 -  [¿Quién soy?](#)
 -  [CONRAD VS EL MUNDO: “clave subida a GitHub”](#)
 -  [Laboratorio de tortura: OpenWeatherMap + OpenAI](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a pensar](#)
 -  [Crucigrama de bits](#)
 -  [Entrevista de trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Poscréditos](#)
 -  [Criterios de evaluación cubiertos \(RA4a-b\)](#)
- [U08 — Hash y Cifrado Clásico](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA5\)](#)
 -  [¿Por dónde empiezo?](#)
- [01 — Principios de seguridad](#)
 -  [La idea en una frase](#)
 -  [Los seis principios que no puedes olvidar](#)
 -  [La analogía de las cerraduras de una casa](#)
 -  [La cita que debes saber](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [02 — Qué es un hash](#)

-  [La idea en una frase](#)
-  [La huella digital](#)
-  [Hash en Python: tres sabores](#)
-  [La analogía de la licuadora](#)
-  [Las cinco propiedades del hash](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [03 — MD5, SHA-1 y SHA-256](#)
 -  [La idea en una frase](#)
 -  [La tabla comparativa](#)
 -  [Hash de un archivo: verificar integridad](#)
 -  [SHA-512 en acción](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [04 — Hash de contraseñas](#)
 -  [La idea en una frase](#)
 -  [Be the code: registro y login con hash](#)
 -  [La traza paso a paso](#)
 -  [La regla de oro](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — Hash con sal](#)
 -  [La idea en una frase](#)
 -  [El problema: hashes idénticos](#)
 -  [La solución: salt](#)
 -  [Por qué la sal destroza las tablas rainbow](#)
 -  [Pool Puzzle — Verificación de contraseña con hash](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)

- [06 — Cifrado César](#)
 - [!\[\]\(2dc8cdc0c918df88cde61039ecf68682_img.jpg\) La idea en una frase](#)
 - [!\[\]\(793119bf0d613bd9b598fb8668922511_img.jpg\) La historia: Julio César y su desplazamiento de 3](#)
 - [!\[\]\(0a4819029e810ca9d2aba79260b63a4d_img.jpg\) Cifrar y descifrar en Python](#)
 - [!\[\]\(5b78a2fafd05db5e14d20573d68ef9b3_img.jpg\) Be the code: César paso a paso](#)
 - [!\[\]\(25fe2c0d7244c22c84de6bda963b471d_img.jpg\) Romperlo por fuerza bruta](#)
 - [!\[\]\(d4bd0dc972749ad3ba477eac47688a0b_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(5eab3de5002abb449199a3fc43c9f414_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(3f2384a64e2c0ffe3eae9a8107dd00c7_img.jpg\) Vocabulario rápido](#)
- [07 — Hash vs Cifrado](#)
 - [!\[\]\(0a4ab723df2c815236fb0c30cb14280f_img.jpg\) La idea en una frase](#)
 - [!\[\]\(a5e6025d913df625081ab04ab57538d0_img.jpg\) El ring de los conceptos: Hash vs Cifrado](#)
 - [!\[\]\(933db2af0bc51ccc9956c85daceec771_img.jpg\) La moraleja: cada uno a lo suyo](#)
 - [!\[\]\(aa37c4d902b06a4c60c80e22dbdcae63_img.jpg\) La escena que lo aclara todo](#)
 - [!\[\]\(305017e3492be9834d83526ded9a5546_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(ce93526cc7275dc99a5e28e42c2dec45_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(3d8621e1356074fde3f4532b15fe8afa_img.jpg\) Vocabulario rápido](#)
- [08 — Buenas prácticas y verificación](#)
 - [!\[\]\(871f3967dd06baf62bc0e930d9243f73_img.jpg\) La idea en una frase](#)
 - [!\[\]\(37c6dc81049e8a4c6dd88ffa288e134a_img.jpg\) Verificación de integridad: el checksum de las descargas](#)
 - [!\[\]\(4d2dd53ee1a5a6f5efa820e745814047_img.jpg\) ¿Cuándo hash y cuándo cifrado? La regla de oro](#)
 - [!\[\]\(632644865a98825d4ab6066f88459a98_img.jpg\) Guía rápida de buenas prácticas](#)
 - [!\[\]\(c5f5eef4efa8c0ad34aae60427d6ac5b_img.jpg\) Aprieta el lápiz](#)
 - [!\[\]\(d82eccc468be357cae6806191127e55d_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(29682f0a9343f6406dd572d30664137a_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(9072e497dc9469cd8fef36b3e671b928_img.jpg\) Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 - [!\[\]\(d5eb365f0ca476d7473041e988d395dc_img.jpg\) Sé el hash](#)
 - [!\[\]\(87e611d5e1961433c877458822e2b766_img.jpg\) Fireside Chat: Hash vs Cifrado](#)
 - [!\[\]\(8f5ae82295611d92d64aa46df97d9f7a_img.jpg\) ¿Quién soy?](#)
 - [!\[\]\(523047843acc9a1e0403ccd04a69c0cc_img.jpg\) CONRAD VS EL MUNDO: “guardé las contraseñas con MD5 y sin sal”](#)
 - [!\[\]\(68537e80d498309418ddde7ee99e37be_img.jpg\) Laboratorio de tortura: registro y login con sal](#)
 - [!\[\]\(eb37c866e2cac2929b775e1ab51b8671_img.jpg\) Logros de esta unidad](#)

- [!\[\]\(fd4127b9e2af37bd6ea0fa06afa8e6d8_img.jpg\) Atrévete a pensar](#)
- [!\[\]\(3278d6283d12f18012b5aa7d40747611_img.jpg\) Crucigrama de bits](#)
- [!\[\]\(bb96b32142ec45f72f12316beae3ef61_img.jpg\) Entrevista de trabajo](#)
- [!\[\]\(a75d0d7d1ac0ea815a0c027a77b137d5_img.jpg\) No hay preguntas tontas](#)
- [!\[\]\(adab168ecea4e2ae40993afba3fcd26e_img.jpg\) Poscréditos](#)
- [!\[\]\(e95ea1a0e297340f96c3d715d76d8c23_img.jpg\) Criterios de evaluación cubiertos \(RA5\)](#)
- [U09 — Cifrado Moderno](#)
 - [!\[\]\(0e0cdd76cf8fb3c858658faf72d7f324_img.jpg\) Objetivo de la unidad](#)
 - [!\[\]\(c7bdebfc2a52eab4f6683b0c1c0c28d2_img.jpg\) Mapa de la unidad](#)
 - [!\[\]\(8a5c092736ceaf7cfeed23b17ea5e6ff_img.jpg\) Boletines de la unidad](#)
 - [!\[\]\(dd896e8effa212be4a7431ac3e5bf510_img.jpg\) Criterios de evaluación cubiertos \(RA5\)](#)
 - [!\[\]\(1136f3a14b3ba5842784adf33e611db1_img.jpg\) ¿Por dónde empiezo?](#)
- [01 — Cifrado simétrico vs asimétrico](#)
 - [!\[\]\(d536ae100fbef949d5488b8fe23458f5_img.jpg\) La idea en una frase](#)
 - [!\[\]\(ca07eb06779edef206c13bdfc2c41b40_img.jpg\) Cifrado simétrico: una sola clave](#)
 - [!\[\]\(7e7914364faa22e4d6649c877c7e723a_img.jpg\) Cifrado asimétrico: el par público/privado](#)
 - [!\[\]\(8ca478e471a5a40f192eb324324796da_img.jpg\) La comparativa: AES vs RSA](#)
 - [!\[\]\(1ef1c1e934c0d3468708b1b65c673a8e_img.jpg\) ¿Cuándo uso cada uno?](#)
 - [!\[\]\(2feef24d2b62f527f17ec4a26c99892b_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(62fbabcb8f920179a05cac5e6f77b9ec_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(ea93f456ce60033911637cde1e339ebc_img.jpg\) Vocabulario rápido](#)
- [02 — AES](#)
 - [!\[\]\(5ee090fbd54350d5f2535fbbb0cd8843_img.jpg\) La idea en una frase](#)
 - [!\[\]\(4dd804f7f115c0312bcea2f969779146_img.jpg\) Componentes de AES](#)
 - [!\[\]\(0917fa5071e46a535e4715939ddc0471_img.jpg\) AES con pycryptodome](#)
 - [!\[\]\(27fec434af9954d953dfa4171448e1b3_img.jpg\) Be the code, my friend — AES paso a paso](#)
 - [!\[\]\(d145641c384340f63dd70fecc7aabb25_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(243bbedc054f34725b139b217207eb1a_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(f3c6d79887bd9f21070de580cbc98211_img.jpg\) Vocabulario rápido](#)
- [03 — Modos de AES](#)
 - [!\[\]\(2d1ece5ae055d1ae0a9970ac957b3b56_img.jpg\) La idea en una frase](#)
 - [!\[\]\(5b120c922ef28e4fc683ca8b94422cdb_img.jpg\) ¿Por qué hace falta un modo?](#)
 - [!\[\]\(3ecc71fc952378d9f4ffcad55785a56c_img.jpg\) La tabla de los modos](#)

-  [CBC: cada bloque depende del anterior](#)
-  [GCM y EAX: cifrar y autenticar a la vez](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [04 — RSA](#)
 -  [La idea en una frase](#)
 -  [Generar el par de claves](#)
 -  [Cifrar con la pública, descifrar con la privada](#)
 -  [El límite de RSA](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — Firmas digitales](#)
 -  [La idea en una frase](#)
 -  [Firmar \(con tu clave privada\)](#)
 -  [Verificar \(con la clave pública del firmante\)](#)
 -  [Be the code, my friend — Firma y verificación](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [06 — Cifrado híbrido](#)
 -  [La idea en una frase](#)
 -  [El intercambio, paso a paso](#)
 -  [Código completo](#)
 -  [Por qué HTTPS usa esto](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — RBAC y roles](#)
 -  [La idea en una frase](#)
 -  [Los actores](#)
 -  [La matriz de permisos](#)

-  [RBAC también protege la criptografía](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [08 — Práctica: sistema seguro](#)
 -  [La idea en una frase](#)
 -  [El sistema completo](#)
 -  [Be the code, my friend — El mensaje completo](#)
 -  [Aprieta el lápiz](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 -  [Sé la clave](#)
 -  [Fireside Chat: AES vs RSA](#)
 -  [¿Quién soy?](#)
 -  [CONRAD VS EL MUNDO: “cifré el mensaje pero no puedo descifrarlo”](#)
 -  [Laboratorio de tortura: mensaje cifrado y firmado](#)
 -  [Logros de esta unidad](#)
 -  [Atrévete a pensar](#)
 -  [Crucigrama de bits](#)
 -  [Entrevista de trabajo](#)
 -  [No hay preguntas tontas](#)
 -  [Poscréditos](#)
 -  [Criterios de evaluación cubiertos \(RA5\)](#)
- [U10 — Servidores Concurrentes](#)
 -  [Objetivo de la unidad](#)
 -  [Mapa de la unidad](#)
 -  [Boletines de la unidad](#)
 -  [Criterios de evaluación cubiertos \(RA4c-d\)](#)
 -  [¿Por dónde empiezo?](#)
- [01 — Servidor secuencial](#)
 -  [La idea en una frase](#)

-  [El servidor secuencial](#)
-  [La traza: secuencial vs concurrente](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [02 — El problema de la espera](#)
 -  [La idea en una frase](#)
 -  [Un cliente lento congela la cola](#)
 -  [La analogía: la ventanilla única](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [03 — Hilo por cliente](#)
 -  [La idea en una frase](#)
 -  [El servidor multihilo](#)
 -  [Be the code — Servidor multihilo en acción](#)
 -  [Ventajas y el primer cuidado](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [04 — ThreadPoolExecutor](#)
 -  [La idea en una frase](#)
 -  [El problema: miles de hilos](#)
 -  [El servidor con pool](#)
 -  [¿Qué tamaño de pool elegir?](#)
 -  [Pool Puzzle — Servidor con ThreadPool](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — Benchmark](#)
 -  [La idea en una frase](#)
 -  [El experimento](#)
 -  [Los resultados](#)

-  [El lanzador masivo de clientes](#)
-  [Cómo interpretar los resultados](#)
-  [Mini-chequeo](#)
-  [Resumen en 3 frases](#)
-  [Vocabulario rápido](#)
- [06 — Sincronización en servidores](#)
 -  [La idea en una frase](#)
 -  [La condición de carrera en el servidor](#)
 -  [El Lock en acción](#)
 -  [¿Y el contador de bytes?](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — Límites y buenas prácticas](#)
 -  [La idea en una frase](#)
 -  [¿Cuántos hilos puede tener un servidor?](#)
 -  [¿Qué pasa si un hilo se cuelga?](#)
 -  [¿ThreadPool o hilo por cliente en producción?](#)
 -  [¿Y si el servidor recibe 10.000 conexiones?](#)
 -  [¿Puedo mezclar hilos y asyncio?](#)
 -  [Guía rápida de buenas prácticas](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [08 — Servidor concurrente completo](#)
 -  [La idea en una frase](#)
 -  [El servidor: servidor.py](#)
 -  [El lanzador: lanzaclientes.py](#)
 -  [Cómo probarlos juntos](#)
 -  [Aprieta el lápiz](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)

- [09 — Cierre: consolida lo aprendido](#)
 - [★ Sé el servidor](#)
 - [🔥 Fireside Chat: Hilo por cliente vs ThreadPool](#)
 - [🕵️ ¿Quién soy?](#)
 - [🤖 CONRAD VS EL MUNDO: “un cliente lento bloquea el servidor”](#)
 - [⚡ Laboratorio de tortura: servidor concurrente con ThreadPool](#)
 - [🏆 Logros de esta unidad](#)
 - [🧠 Atrévete a pensar](#)
 - [🧩 Crucigrama de bits](#)
 - [💬 Entrevista de trabajo](#)
 - [🙋 No hay preguntas tontas](#)
 - [🎬 Poscréditos](#)
 - [✅ Criterios de evaluación cubiertos \(RA4c-d\)](#)
- [U11 — asyncio y Disponibilidad](#)
 - [🎯 Objetivo de la unidad](#)
 - [🗺️ Mapa de la unidad](#)
 - [📄 Boletines de la unidad](#)
 - [✅ Criterios de evaluación cubiertos \(RA4e-g\)](#)
 - [📖 ¿Por dónde empiezo?](#)
- [01 — Event Loop](#)
 - [🗣️ La idea en una frase](#)
 - [🚫 El problema de esperar](#)
 - [☕ La analogía de la cafetería](#)
 - [⚙️ Qué es el event loop, de verdad](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)
 - [🦎 Vocabulario rápido](#)
- [02 — Corrutinas](#)
 - [🗣️ La idea en una frase](#)
 - [🌀 `async def` y `await`](#)
 - [😬 Corrutina no es una función normal](#)
 - [🧠 Mini-chequeo](#)
 - [✅ Resumen en 3 frases](#)

-  [Vocabulario rápido](#)
- [03 — create_task y gather](#)
 -  [La idea en una frase](#)
 -  [asyncio.gather: todas a la vez](#)
 -  [asyncio.create_task: en segundo plano](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [04 — Timeouts](#)
 -  [La idea en una frase](#)
 -  [El problema del cuelgue](#)
 -  [wait_for con respaldo](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [05 — Heartbeat](#)
 -  [La idea en una frase](#)
 -  [El latido en asyncio](#)
 -  [Heartbeat con estado](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [06 — Backoff](#)
 -  [La idea en una frase](#)
 -  [El problema: reintentar sin piedad](#)
 -  [La secuencia del backoff](#)
 -  [Mini-chequeo](#)
 -  [Resumen en 3 frases](#)
 -  [Vocabulario rápido](#)
- [07 — Threads vs asyncio](#)
 -  [La idea en una frase](#)
 -  [El ring de los conceptos](#)
 -  [La tabla comparativa](#)

- [!\[\]\(2dc8cdc0c918df88cde61039ecf68682_img.jpg\) ¿Cuándo usar cada uno?](#)
- [!\[\]\(793119bf0d613bd9b598fb8668922511_img.jpg\) Mini-chequeo](#)
- [!\[\]\(0a4819029e810ca9d2aba79260b63a4d_img.jpg\) Resumen en 3 frases](#)
- [!\[\]\(5b78a2fafd05db5e14d20573d68ef9b3_img.jpg\) Vocabulario rápido](#)
- [08 — Disponibilidad y práctica](#)
 - [!\[\]\(25fe2c0d7244c22c84de6bda963b471d_img.jpg\) La idea en una frase](#)
 - [!\[\]\(d4bd0dc972749ad3ba477eac47688a0b_img.jpg\) El monitor de servicio](#)
 - [!\[\]\(5eab3de5002abb449199a3fc43c9f414_img.jpg\) Be the code: el event loop en un servidor](#)
 - [!\[\]\(3f2384a64e2c0ffe3eae9a8107dd00c7_img.jpg\) Aprieta el lápiz](#)
 - [!\[\]\(0a4ab723df2c815236fb0c30cb14280f_img.jpg\) Mini-chequeo](#)
 - [!\[\]\(a5e6025d913df625081ab04ab57538d0_img.jpg\) Resumen en 3 frases](#)
 - [!\[\]\(933db2af0bc51ccc9956c85daceec771_img.jpg\) Vocabulario rápido](#)
- [09 — Cierre: consolida lo aprendido](#)
 - [!\[\]\(aa37c4d902b06a4c60c80e22dbdcae63_img.jpg\) Sé la corrutina](#)
 - [!\[\]\(305017e3492be9834d83526ded9a5546_img.jpg\) Fireside Chat: hilos vs asyncio](#)
 - [!\[\]\(ce93526cc7275dc99a5e28e42c2dec45_img.jpg\) ¿Quién soy?](#)
 - [!\[\]\(3d8621e1356074fde3f4532b15fe8afa_img.jpg\) CONRAD VS EL MUNDO: “el servidor se cuelga si un cliente no responde”](#)
 - [!\[\]\(871f3967dd06baf62bc0e930d9243f73_img.jpg\) Laboratorio de tortura: el monitor de servicio definitivo](#)
 - [!\[\]\(37c6dc81049e8a4c6dd88ffa288e134a_img.jpg\) Logros de esta unidad](#)
 - [!\[\]\(4d2dd53ee1a5a6f5efa820e745814047_img.jpg\) Atrévete a pensar](#)
 - [!\[\]\(632644865a98825d4ab6066f88459a98_img.jpg\) Crucigrama de bits](#)
 - [!\[\]\(c5f5eef4efa8c0ad34aae60427d6ac5b_img.jpg\) Entrevista de trabajo](#)
 - [!\[\]\(d82eccc468be357cae6806191127e55d_img.jpg\) No hay preguntas tontas](#)
 - [!\[\]\(29682f0a9343f6406dd572d30664137a_img.jpg\) Poscréditos](#)
 - [!\[\]\(9072e497dc9469cd8fef36b3e671b928_img.jpg\) Criterios de evaluación cubiertos \(RA4e-g\)](#)
- [Boletín U01 — Inicial \(Resuelto\)](#)
 - [1. Listar el directorio actual](#)
 - [2. Saber el nombre del equipo](#)
 - [3. Abrir la calculadora](#)
 - [4. Mi primer PID](#)
 - [5. Versión de Python](#)
 - [6. Abrir el bloc de notas y esperar](#)
 - [7. Ordena el ciclo de vida](#)

- [8. Ping con timeout](#)
- [Boletín U01 — Inicial](#)
 - [1. Listar el directorio actual](#)
 - [2. Saber el nombre del equipo](#)
 - [3. Abrir la calculadora](#)
 - [4. Mi primer PID](#)
 - [5. Versión de Python](#)
 - [6. Abrir el bloc de notas y esperar](#)
 - [7. Ordena el ciclo de vida](#)
 - [8. Ping con timeout](#)
- [Boletín U01 — Avanzado \(Resuelto\)](#)
 - [1. Filtrar ipconfig](#)
 - [2. Comprobar si un programa existe](#)
 - [3. Abrir el navegador](#)
 - [4. Lanzador múltiple](#)
 - [5. Captura de ping](#)
 - [6. PID del padre](#)
 - [7. Comunicación en dos direcciones](#)
 - [8. Timeout con reintentos](#)
 - [9. Mini monitor de procesos](#)
- [Boletín U01 — Avanzado](#)
 - [1. Filtrar ipconfig](#)
 - [2. Comprobar si un programa existe](#)
 - [3. Abrir el navegador](#)
 - [4. Lanzador múltiple](#)
 - [5. Captura de ping](#)
 - [6. PID del padre](#)
 - [7. Comunicación en dos direcciones](#)
 - [8. Timeout con reintentos](#)
 - [9. Mini monitor de procesos](#)
- [Boletín U02 — Inicial \(Resuelto\)](#)
 - [1. Hilo que cuenta](#)
 - [2. Múltiples hilos con retardo](#)

- [3. Hilo con argumento](#)
- [4. Hilo que saluda](#)
- [5. Dos hilos en paralelo](#)
- [6. Hilo con nombre](#)
- [7. Esperando con join\(\)](#)
- [8. Varios hilos con nombre y argumentos](#)
- [Boletín U02 — Inicial](#)
 - [1. Hilo que cuenta](#)
 - [2. Múltiples hilos con retardo](#)
 - [3. Hilo con argumento](#)
 - [4. Hilo que saluda](#)
 - [5. Dos hilos en paralelo](#)
 - [6. Hilo con nombre](#)
 - [7. Esperando con join\(\)](#)
 - [8. Varios hilos con nombre y argumentos](#)
- [Boletín U02 — Avanzado \(Resuelto\)](#)
 - [1. Join con timeout](#)
 - [2. Listar hilos activos](#)
 - [3. Identificar el hilo actual](#)
 - [4. 🎯 Carrera de mensajes](#)
 - [5. 🔍 Simular descarga con progreso](#)
 - [6. 🧩 Detener hilo con bandera](#)
 - [7. 🎭 Hilo que devuelve un valor](#)
 - [8. ⌚ Timer con repetición](#)
 - [9. 🏊 Pool de hilos manual](#)
 - [🎁 Bonus: tres clásicos resueltos](#)
- [Boletín U02 — Avanzado](#)
 - [1. Join con timeout](#)
 - [2. Listar hilos activos](#)
 - [3. Identificar el hilo actual](#)
 - [4. 🎯 Carrera de mensajes](#)
 - [5. 🔍 Simular descarga con progreso](#)
 - [6. 🧩 Detener hilo con bandera](#)

- [7. 🎭 Hilo que devuelve un valor](#)
- [8. ⌚ Timer con repetición](#)
- [9. 🏊 Pool de hilos manual](#)
- [Boletín U03 — Inicial \(Resuelto\)](#)
 - [1. Lock protege una lista](#)
 - [2. RLock básico](#)
 - [3. Semáforo con timeout](#)
 - [4. Carrera sin Lock](#)
 - [5. Contador protegido con Lock](#)
 - [6. Sección crítica con sleep](#)
 - [7. Semáforo simple](#)
 - [8. Barrera de 3](#)
- [Boletín U03 — Inicial](#)
 - [1. Lock protege una lista](#)
 - [2. RLock básico](#)
 - [3. Semáforo con timeout](#)
 - [4. Carrera sin Lock](#)
 - [5. Contador protegido con Lock](#)
 - [6. Sección crítica con sleep](#)
 - [7. Semáforo simple](#)
 - [8. Barrera de 3](#)
- [Boletín U03 — Avanzado \(Resuelto\)](#)
 - [1. Condition con notify_all](#)
 - [2. Deadlock entre dos hilos](#)
 - [3. Semáforo con recursos limitados](#)
 - [4. 🎯 Banco con cuentas compartidas](#)
 - [5. 🔍 Semáforo con timeout](#)
 - [6. 🧩 Barrera con fases múltiples](#)
 - [7. 🎭 Condition con timeout](#)
 - [8. ⌚ Deadlock provocado y solución](#)
 - [9. 🏊 Productor-Consumidor múltiple](#)
- [Boletín U03 — Avanzado](#)
 - [1. Condition con notify_all](#)

- [2. Deadlock entre dos hilos](#)
- [3. Semáforo con recursos limitados](#)
- [4. 🎯 Banco con cuentas compartidas](#)
- [5. 🔍 Semáforo con timeout](#)
- [6. 🧩 Barrera con fases múltiples](#)
- [7. 🤖 Condition con timeout](#)
- [8. ⌚ Deadlock provocado y solución](#)
- [9. 🏭 Productor-Consumidor múltiple](#)
- [Boletín U04 — Inicial \(Resuelto\)](#)
 - [1. Servidor saludo](#)
 - [2. Cliente que envía y recibe](#)
 - [3. Servidor contador](#)
 - [4. Servidor eco](#)
 - [5. Cliente mínimo](#)
 - [6. Servidor mínimo](#)
 - [7. IP y puerto](#)
 - [8. Servidor hora](#)
- [Boletín U04 — Inicial](#)
 - [1. Servidor saludo](#)
 - [2. Cliente que envía y recibe](#)
 - [3. Servidor contador](#)
 - [4. Servidor eco](#)
 - [5. Cliente mínimo](#)
 - [6. Servidor mínimo](#)
 - [7. IP y puerto](#)
 - [8. Servidor hora](#)
- [Boletín U04 — Avanzado \(Resuelto\)](#)
 - [1. Servidor que cuenta caracteres](#)
 - [2. Cliente interactivo](#)
 - [3. Servidor que registra IP](#)
 - [4. Servidor de mayúsculas](#)
 - [5. Cliente con reconexión](#)
 - [6. Servidor que gestiona múltiples conexiones \(sin hilos\)](#)

- [7. Calculadora remota](#)
- [8. Timeout personalizado](#)
- [9. Servidor de chat simple](#)
- [Boletín U04 — Avanzado](#)
 - [1. Servidor que cuenta caracteres](#)
 - [2. Cliente interactivo](#)
 - [3. Servidor que registra IP](#)
 - [4. Servidor de mayúsculas](#)
 - [5. Cliente con reconexión](#)
 - [6. Servidor que gestiona múltiples conexiones \(sin hilos\)](#)
 - [7. Calculadora remota](#)
 - [8. Timeout personalizado](#)
 - [9. Servidor de chat simple](#)
- [Boletín U05 — Inicial \(Resuelto\)](#)
 - [1. Cliente UDP con entrada de usuario](#)
 - [2. Servidor UDP con eco personalizado](#)
 - [3. Servidor UDP contador](#)
 - [4. Cliente UDP mínimo](#)
 - [5. Servidor UDP mínimo](#)
 - [6. Servidor UDP eco](#)
 - [7. TCP vs UDP: clasifica](#)
 - [8. Cliente HTTP manual](#)
- [Boletín U05 — Inicial](#)
 - [1. Cliente UDP con entrada de usuario](#)
 - [2. Servidor UDP con eco personalizado](#)
 - [3. Servidor UDP contador](#)
 - [4. Cliente UDP mínimo](#)
 - [5. Servidor UDP mínimo](#)
 - [6. Servidor UDP eco](#)
 - [7. TCP vs UDP: clasifica](#)
 - [8. Cliente HTTP manual](#)
- [Boletín U05 — Avanzado \(Resuelto\)](#)
 - [1. Cliente NTP manual](#)

- [2. Servidor UDP multimensaje](#)
- [3. Cliente HTTP con parseo de cabeceras](#)
- [4. Cliente HTTP manual a /ip](#)
- [5. Compara TCP y UDP](#)
- [6. Mini servidor web](#)
- [7. Ping UDP](#)
- [8. Broadcast UDP](#)
- [9. HTTP desde cero con parseo](#)
- [10. Mini navegador web](#)
- [11. Comparativa TCP vs UDP \(100 mensajes\)](#)
- [12. Servidor HTTP simple](#)
- [Boletín U05 — Avanzado](#)
 - [1. Cliente NTP manual](#)
 - [2. Servidor UDP multimensaje](#)
 - [3. Cliente HTTP con parseo de cabeceras](#)
 - [4. Cliente HTTP manual a /ip](#)
 - [5. Compara TCP y UDP](#)
 - [6. Mini servidor web](#)
 - [7. Ping UDP](#)
 - [8. Broadcast UDP](#)
 - [9. HTTP desde cero con parseo](#)
 - [10. Mini navegador web](#)
 - [11. Comparativa TCP vs UDP](#)
 - [12. Servidor HTTP simple](#)
- [Boletín U06 — Inicial \(Resuelto\)](#)
 - [1. GET a JSONPlaceholder](#)
 - [2. Título del post](#)
 - [3. Contar usuarios](#)
 - [4. GET a GitHub](#)
 - [5. Mostrar JSON](#)
 - [6. Nombre real](#)
 - [7. Métodos HTTP](#)
 - [8. Códigos de estado](#)

- [Boletín U06 — Inicial](#)
 - [1. GET a JSONPlaceholder](#)
 - [2. Título del post](#)
 - [3. Contar usuarios](#)
 - [4. GET a GitHub](#)
 - [5. Mostrar JSON](#)
 - [6. Nombre real](#)
 - [7. Métodos HTTP](#)
 - [8. Códigos de estado](#)
- [Boletín U06 — Avanzado \(Resuelto\)](#)
 - [1. PUT para actualizar](#)
 - [2. DELETE un recurso](#)
 - [3. HEAD request](#)
 - [4. 🎯 Info de usuario de GitHub](#)
 - [5. 🔍 Buscador de repositorios](#)
 - [6. 🧩 API con paginación](#)
 - [7. 🦄 POST y respuesta](#)
 - [8. ⌚ Tiempo de respuesta](#)
 - [9. 🏠 Cliente de API con caché](#)
- [Boletín U06 — Avanzado](#)
 - [1. PUT para actualizar](#)
 - [2. DELETE un recurso](#)
 - [3. HEAD request](#)
 - [4. 🎯 Info de usuario de GitHub](#)
 - [5. 🔍 Buscador de repositorios](#)
 - [6. 🧩 API con paginación](#)
 - [7. 🦄 POST y respuesta](#)
 - [8. ⌚ Tiempo de respuesta](#)
 - [9. 🏠 Cliente de API con caché](#)
- [Boletín U07 — Inicial \(Resuelto\)](#)
 - [1. Humedad actual](#)
 - [2. Descripción del clima](#)
 - [3. Temperatura mínima y máxima](#)

- [4. Temperatura y sensación térmica](#)
- [5. Error 401](#)
- [6. ¿Dónde va la API key?](#)
- [7. Clave desde el .env](#)
- [8. Protege el repo](#)
- [Boletín U07 — Inicial](#)
 - [1. Humedad actual](#)
 - [2. Descripción del clima](#)
 - [3. Temperatura mínima y máxima](#)
 - [4. Temperatura y sensación térmica](#)
 - [5. Error 401](#)
 - [6. ¿Dónde va la API key?](#)
 - [7. Clave desde el .env](#)
 - [8. Protege el repo](#)
- [Boletín U07 — Avanzado \(Resuelto\)](#)
 - [1. Clima con .env para dos ciudades](#)
 - [2. GPT con temperatura creativa](#)
 - [3. Capturar errores HTTP](#)
 - [4. Comprobación de claves](#)
 - [5. Chat con GPT y rol system](#)
 - [6. Timeout](#)
 - [7. 🎯 Pronóstico extendido](#)
 - [8. 🔍 Múltiples ciudades](#)
 - [9. 🧩 GPT: explicador automático](#)
 - [10. 🗨️ Conversación con contexto](#)
 - [11. 🕒 Monitor de uptime](#)
 - [12. 🛠️ Agregador de APIs](#)
- [Boletín U07 — Avanzado](#)
 - [1. Clima con .env para dos ciudades](#)
 - [2. GPT con temperatura creativa](#)
 - [3. Capturar errores HTTP](#)
 - [4. Comprobación de claves](#)
 - [5. Chat con GPT y rol system](#)

- [6. Timeout](#)
- [7. 🎯 Pronóstico extendido](#)
- [8. 🔍 Múltiples ciudades](#)
- [9. 🧩 GPT: explicador automático](#)
- [10. 🗨️ Conversación con contexto](#)
- [11. ⌚ Monitor de uptime](#)
- [12. 📡 Agregador de APIs](#)
- [Boletín U08 — Inicial \(Resuelto\)](#)
 - [1. SHA1 de tu nombre](#)
 - [2. MD5 de una frase](#)
 - [3. Compara “Hola” vs “hola”](#)
 - [4. Longitud de los hashes de “Hola mundo”](#)
 - [5. Hash de un archivo simple](#)
 - [6. Cifrado César sencillo](#)
 - [7. Descifrar un César dado](#)
 - [8. Determinismo del hash](#)
- [Boletín U08 — Inicial](#)
 - [1. SHA1 de tu nombre](#)
 - [2. MD5 de una frase](#)
 - [3. Compara “Hola” vs “hola”](#)
 - [4. Longitud de los hashes de “Hola mundo”](#)
 - [5. Hash de un archivo simple](#)
 - [6. Cifrado César sencillo](#)
 - [7. Descifrar un César dado](#)
 - [8. Determinismo del hash](#)
- [Boletín U08 — Avanzado \(Resuelto\)](#)
 - [1. Hash de un archivo de texto](#)
 - [2. Cifrado César con espacios](#)
 - [3. Descifrado con mayúsculas y minúsculas](#)
 - [4. Verificador de integridad](#)
 - [5. Fuerza bruta César](#)
 - [6. Hash con salt](#)
 - [7. Comparación gráfica de avalancha](#)

- [8. Velocidad de hashes](#)
- [9. Mini gestor de contraseñas](#)
- [Boletín U08 — Avanzado](#)
 - [1. Hash de un archivo de texto](#)
 - [2. Cifrado César con espacios](#)
 - [3. Descifrado con mayúsculas y minúsculas](#)
 - [4. Verificador de integridad](#)
 - [5. Fuerza bruta César](#)
 - [6. Hash con salt](#)
 - [7. Comparación gráfica de avalancha](#)
 - [8. Velocidad de hashes](#)
 - [9. Mini gestor de contraseñas](#)
- [Boletín U09 — Inicial \(Resuelto\)](#)
 - [1. AES modo ECB](#)
 - [2. Nonce y tag](#)
 - [3. RSA: exportar clave](#)
 - [4. AES: cifrar mensaje](#)
 - [5. AES: descifrar](#)
 - [6. RSA: generar claves](#)
 - [7. Simétrico vs asimétrico](#)
 - [8. AES: cifrar y descifrar completo](#)
- [Boletín U09 — Inicial](#)
 - [1. AES modo ECB](#)
 - [2. Nonce y tag](#)
 - [3. RSA: exportar clave](#)
 - [4. AES: cifrar mensaje](#)
 - [5. AES: descifrar](#)
 - [6. RSA: generar claves](#)
 - [7. Simétrico vs asimétrico](#)
 - [8. AES: cifrar y descifrar completo](#)
- [Boletín U09 — Avanzado \(Resuelto\)](#)
 - [1. Cifrado híbrido simplificado](#)
 - [2. Firma alterada](#)

- [3. RBAC con permisos cifrado](#)
- [4. Cifrar archivo completo](#)
- [5. RSA: cifrar mensajes largos](#)
- [6. Intercambio de claves simulado](#)
- [7. Firma con verificación de integridad](#)
- [8. RSA vs AES benchmark](#)
- [9. Sistema de cifrado de extremo a extremo](#)
- [Boletín U09 — Avanzado](#)
 - [1. Cifrado híbrido simplificado](#)
 - [2. Firma alterada](#)
 - [3. RBAC con permisos cifrado](#)
 - [4. Cifrar archivo completo](#)
 - [5. RSA: cifrar mensajes largos](#)
 - [6. Intercambio de claves simulado](#)
 - [7. Firma con verificación de integridad](#)
 - [8. RSA vs AES benchmark](#)
 - [9. Sistema de cifrado de extremo a extremo](#)
- [Boletín U10 — Inicial \(Resuelto\)](#)
 - [1. Servidor eco básico](#)
 - [2. Servidor eco con bucle](#)
 - [3. Cliente eco](#)
 - [4. Servidor secuencial](#)
 - [5. Servidor multihilo](#)
 - [6. Cliente con respuesta](#)
 - [7. Servidor con ThreadPoolExecutor](#)
 - [8. Lanzador de 5 clientes](#)
- [Boletín U10 — Inicial](#)
 - [1. Servidor eco básico](#)
 - [2. Servidor eco con bucle](#)
 - [3. Cliente eco](#)
 - [4. Servidor secuencial](#)
 - [5. Servidor multihilo](#)
 - [6. Cliente con respuesta](#)

- [7. Servidor con ThreadPoolExecutor](#)
- [8. Lanzador de 5 clientes](#)
- [Boletín U10 — Avanzado \(Resuelto\)](#)
 - [1. Servidor con timeout](#)
 - [2. Cliente con timeout](#)
 - [3. Contador de bytes totales](#)
 - [4. Servidor con límite de clientes](#)
 - [5. Prueba de carga](#)
 - [6. Servidor con cola de espera](#)
 - [7. Estado del servidor](#)
 - [8. Heartbeat en servidor](#)
 - [9. Balanceador de carga simple](#)
- [Boletín U10 — Avanzado](#)
 - [1. Servidor con timeout](#)
 - [2. Cliente con timeout](#)
 - [3. Contador de bytes totales](#)
 - [4. Servidor con límite de clientes](#)
 - [5. Prueba de carga](#)
 - [6. Servidor con cola de espera](#)
 - [7. Estado del servidor](#)
 - [8. Heartbeat en servidor](#)
 - [9. Balanceador de carga simple](#)
- [Boletín U11 — Inicial \(Resuelto\)](#)
 - [1. Mensaje diferido](#)
 - [2. Saludo y despedida](#)
 - [3. Temporizador](#)
 - [4. Corrutina que devuelve](#)
 - [5. Tres tareas a la vez](#)
 - [6. Tarea en segundo plano](#)
 - [7. Heartbeat básico](#)
 - [8. Timeout básico](#)
- [Boletín U11 — Inicial](#)
 - [1. Mensaje diferido](#)

- [2. Saludo y despedida](#)
- [3. Temporizador](#)
- [4. Corrutina que devuelve](#)
- [5. Tres tareas a la vez](#)
- [6. Tarea en segundo plano](#)
- [7. Heartbeat básico](#)
- [8. Timeout básico](#)
- [Boletín U11 — Avanzado \(Resuelto\)](#)
 - [1. Backoff exponencial](#)
 - [2. Timeout con respaldo](#)
 - [3. Dos heartbeats](#)
 - [4. 🎯 Web scraper asíncrono](#)
 - [5. 🔍 Servidor asyncio con heartbeat](#)
 - [6. 🕒 Monitorización de servidores](#)
 - [7. 🧩 Semáforo asyncio](#)
 - [8. 🐼 Timeout con fallback](#)
 - [9. 🕒 Chat asíncrono](#)
- [Boletín U11 — Avanzado](#)
 - [1. Backoff exponencial](#)
 - [2. Timeout con respaldo](#)
 - [3. Dos heartbeats](#)
 - [4. 🎯 Web scraper asíncrono](#)
 - [5. 🔍 Servidor asyncio con heartbeat](#)
 - [6. 🕒 Monitorización de servidores](#)
 - [7. 🧩 Semáforo asyncio](#)
 - [8. 🐼 Timeout con fallback](#)
 - [9. 🕒 Chat asíncrono](#)

Inicio

APUNTES

Programación de Servicios y Procesos

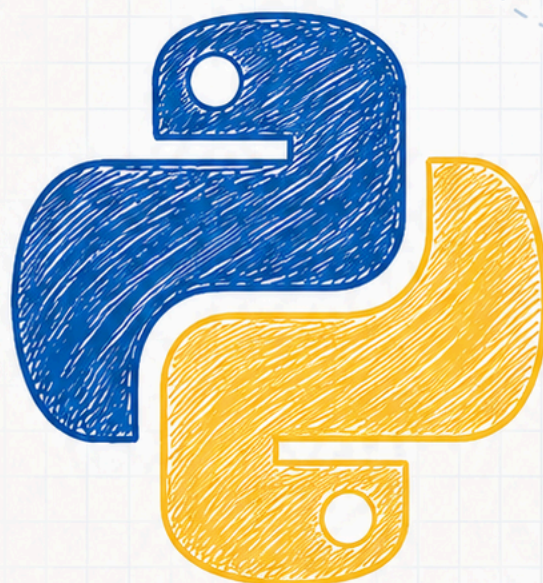
(Con Python)

```
1 import time
2 import socket
3 from multiprocessing import Process
4
5 def trabajo(nombre):
6     print(f"Iniciando {nombre}...")
7     time.sleep(2)
8     print(f"{nombre} finalizado.")
9
10 if __name__ == "__main__":
11     p = Process(target=trabajo, args=("Proceso",))
12     p.start()
13     p.join()
```

importaciones

definición de función

ejecución del proceso



Por

Sergi García Barea

licencia
CC BY SA



[🚀 Empezar por la Unidad 1](#)[🐙 Ver en GitHub](#)

Sergi Garcia Barea — CC BY-SA 4.0

↓ ¿Prefieres estudiar sin conexión? Descarga el curso completo

[PDF](#)[EPUB](#)

Unidades

UNIDAD 1

RA1

Procesos y Subprocess

Procesos, `subprocess.run`, `subprocess.Popen`, comunicación entre procesos, paralela vs distribuida.

[👉 Ver unidad 👈](#)

UNIDAD 2

RA2

Hilos Fundamentos

Crear hilos con `threading.Thread`, `join`, `daemon`, `Timer`, GIL, estados del hilo.

[👉 Ver unidad 👈](#)

UNIDAD 3

RA2

Sincronización entre Hilos

`Lock`, `Semaphore`, `Barrier`, `Condition`, `RLock`, productor-consumidor.

[👉 Ver unidad 👈](#)

UNIDAD 4

RA3

Sockets TCP

TCP cliente/servidor, `socket`, `bind`, `listen`, `accept`, errores, `SO_REUSEADDR`.

[👉 Ver unidad 👈](#)

UNIDAD 5

RA3

Sockets UDP y Protocolos

UDP, HTTP desde cero, NTP, TCP vs UDP, protocolos de aplicación.

[👉 Ver unidad 👈](#)

UNIDAD 6

RA4a-b

APIs REST y HTTP

REST, métodos HTTP, `requests`, JSON, códigos de estado, consumo de APIs.

[👉 Ver unidad 👈](#)

UNIDAD 7

RA4a-b

UNIDAD 8

RA5

APIs Comerciales

OpenWeatherMap, OpenAI, `python-dotenv`, `rate limit`, gestión de errores, autenticación.

👉 Ver unidad 👈

Hash y Cifrado Clásico

Hash, MD5, SHA, Cifrado César, principios de seguridad, esteganografía básica.

👉 Ver unidad 👈

UNIDAD 9

RA5

Cifrado Moderno

AES, RSA, cifrado híbrido, firmas digitales, RBAC, `pycryptodome`.

👉 Ver unidad 👈

UNIDAD 10

RA4c-d

Servidores Concurrentes

ThreadPool, servidor multihilo, `concurrent.futures`, balanceo de carga básico.

👉 Ver unidad 👈

UNIDAD 11

RA4e-g

Asyncio y Disponibilidad

`asyncio`, `async / await`, heartbeat, reintentos con backoff, timeouts, alta disponibilidad.

👉 Ver unidad 👈



Boletines de ejercicios

Cada unidad tiene **4 boletines** (44 en total): inicial y avanzado, con y sin solución.

UNIDAD 1

✅ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

UNIDAD 2

✅ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

UNIDAD 3

✅ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

UNIDAD 4

✅ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

UNIDAD 5

UNIDAD 6

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

UNIDAD 7

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

UNIDAD 8

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

UNIDAD 9

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

UNIDAD 10

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver

UNIDAD 11

✓ Inicial resuelto ● Inicial por resolver
👉 Avanzado resuelto ★ Avanzado por resolver



Licencia



CC BY-SA 4.0 — *Sergi Garcia Barea*

[Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/)

Puedes compartir y adaptar el material para cualquier propósito, incluso comercial, siempre que reconozcas la autoría y compartas las modificaciones bajo la misma licencia.

U01 — Procesos y Subprocess

Los programas en marcha: procesos y subprocess 🚀

📖 **Ruta del viaje:** 🚀 **Procesos y Subprocess** → 🔄 Hilo → 🔒 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 📡 APIs comerciales → 🔑 Hash → 🧬 Cifrado → 🏢 Servidores → ⌚ asyncio

Un programa en el disco duro es un **muerto viviente**: no ocupa memoria, no consume CPU, no hace nada. Un **proceso** es ese mismo programa **vivo**, ejecutándose, ocupando memoria y consumiendo CPU. En esta primera unidad aprenderás a crear procesos con el módulo subprocess, a conocer sus estados, a diferenciar computación paralela de distribuida y a comunicar procesos entre sí con pipes y `communicate()`.

Todo el módulo PSP cuelga de esta unidad: los hilos de la [U02](#) son procesos ligeros, y los servidores y las APIs de las unidades siguientes serán procesos o hilos ejecutándose de fondo. Si no entiendes bien qué es un proceso, el resto del viaje se hace cuesta arriba. Empieza por aquí con calma.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un proceso, su PID y la burbuja de memoria que lo forma (código, estado, contador).
- Describir el ciclo de vida de un proceso: NUEVO, LISTO, EJECUCIÓN, BLOQUEADO y TERMINADO.
- Distinguir computación **paralela**, **distribuida** y **concurrente**, y saber cuándo usar multiprocessing.
- Lanzar y esperar programas con `subprocess.run()`, capturando salida y controlando timeout y errores.
- Lanzar procesos en segundo plano con `subprocess.Popen()` y gestionarlos con `wait`, `poll`, `terminate` y `kill`.

- Comunicar procesos pasando datos por `stdin` y leyendo su respuesta por `stdout` con `communicate()`.
- Adaptar tus programas a Windows y Linux cambiando solo el comando.
- Razonar sobre procesos paso a paso con la técnica “Sé el código”.



Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Qué es un proceso	La burbuja de memoria, el PID y las características de todo proceso	Todos
02 · Estados de un proceso	NUEVO, LISTO, EJECUCIÓN, BLOQUEADO, TERMINADO y sus transiciones	Todos
03 · Paralela vs Distribuida	La diferencia entre paralela, distribuida y concurrencia, con multiprocessing	Todos
04 · subprocess.run()	Lanzar y esperar: capturar salida, timeout y errores	Todos
05 · subprocess.Popen()	Lanzar y seguir: wait, poll, terminate, kill y el PID	Todos
06 · Comunicación con procesos	stdin, stdout y <code>communicate()</code> : pasar datos a un proceso	Todos
07 · Compatibilidad Windows / Linux	La tabla de comandos equivalentes y los trucos del shell	Todos
08 · Procesos en la práctica	Sé el código, el ring run vs Popen y los ejercicios del lápiz	Todos
09 · Cierre	Sé el proceso, Fireside, Laboratorio de tortura, Crucigrama...	Todos



Flujo de lectura: los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.



Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Inicial resuelto

● Inicial por resolver

👤 Avanzado resuelto

★ Avanzado por resolver

✓ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce las características y la gestión de los procesos en un sistema operativo.

CE	Criterio	Dónde se cubre
a)	Reconoce las características de los procesos	✓ Punto 1
b)	Distingue entre computación paralela y distribuida	✓ Punto 3
c)	Conoce los estados de un proceso	✓ Punto 2
d)	Identifica las diferencias clave entre proceso e hilo	→ U02
e)	Crea programas con procesos (subprocess)	✓ Puntos 4 y 5 + ⚡ Laboratorio (punto 9)
f)	Establece comunicación entre procesos	✓ Punto 6

RA1d (proceso vs hilo) se cubre en la **U02 · Hilos Fundamentos**. RA1g (análisis de ventajas de procesos frente a hilos) también en **U02**. RA1h (documentación) es transversal a todo el curso.

📖 ¿Por dónde empiezo?

¿Vienes de cero en programación de sistemas? Perfecto, esta es la primera unidad del módulo y no necesitas nada anterior: solo saber Python básico (funciones, listas, `print()`). Empieza por el [punto 1](#) y no te saltes los puntos 1 a 3: son la base conceptual de TODO el módulo. Los procesos reaparecen en cada unidad siguiente.

¿Ya sabes qué es un proceso y has jugado con la terminal? Puedes saltar a los [puntos 4 y 5](#), que son el corazón práctico de la unidad, y hacer el resto a ritmo ligero. Pero si dudas de qué

diferencia hay entre `run()` y `Popen()`, vuelve al [punto 5](#).

📌 Primer punto: [01 · Qué es un proceso](#)

▶▶ Al acabar la unidad, continúa en [U02 · Hilos Fundamentos](#).

01 — Qué es un proceso

La burbuja de memoria que vive en tu sistema 🌈

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 01 · Qué es un proceso

📌 La idea en una frase

Un **proceso** es un programa en ejecución con su propio espacio de memoria, recursos (archivos abiertos, sockets) y un identificador único llamado **PID**.

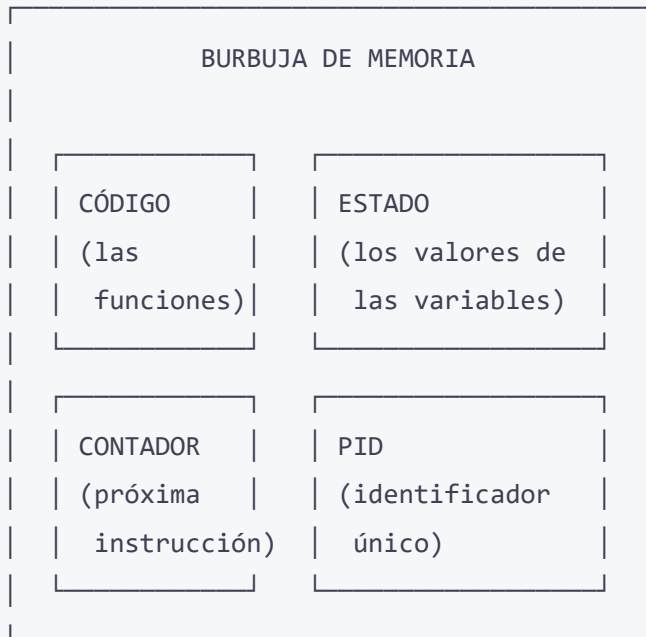
Un programa en el disco duro es un **muerto viviente**: no hace nada. Un proceso es ese mismo programa **vivo**, ocupando memoria, consumiendo CPU y respondiendo al teclado, a la red o al ratón. La diferencia no está en el código: está en que el sistema operativo lo ha cargado en memoria y lo está ejecutando.

```
import os
print(f"Este proceso se llama PID {os.getpid()}")
```

Cada vez que ejecutas ese programa, el sistema operativo crea un proceso nuevo con su propio PID.

🌈 La burbuja de memoria

Cada proceso vive en su propia **burbuja de memoria**: un espacio de direcciones aislado del resto del sistema. Dentro de esa burbuja viajan:



- **Código:** las instrucciones del programa.
- **Estado:** los valores actuales de las variables.
- **Contador de programa:** qué instrucción toca ejecutar ahora.
- **PID:** el carnet de identidad del proceso.

“Si un proceso se cuelga, los demás no se enteran. Cada uno vive en su burbuja de memoria.”



El PID, el carnet de identidad

Cada proceso tiene un **PID** (*Process Identifier*): un número único que asigna el sistema operativo en el momento de crearlo. Sirve para referirse a él, para matarlo, para vigilarlo.

```
import os

print(f"Mi PID es {os.getpid()}")
```

Para ver los procesos de tu sistema con sus PIDs:

- **Windows:** Administrador de tareas → pestaña “Detalles”, o `tasklist` en la terminal.
- **Linux / macOS:** `ps aux` o `top`.



Características de un proceso

Propiedad	Descripción
PID	Identificador único numérico
Memoria propia	Cada proceso tiene su espacio de direcciones aislado
Recursos	Archivos, sockets, manejadores
Contexto	Estado de la CPU, registros, contador de programa
Comunicación	Necesita mecanismos externos (pipes, sockets, archivos)

La última fila es la clave: los procesos **no comparten memoria por defecto**. Si quieren intercambiar datos necesitan un mecanismo externo: un archivo, un socket o un pipe. Lo verás en el [punto 6](#).

La analogía de la cocina

Un proceso es una **receta en marcha** en una cocina. La receta escrita en el libro es el **programa** (código muerto en el disco). Cuando un cocinero la coge, pone los ingredientes sobre su mesa (memoria), empieza a leerla por el paso 1 (contador de programa) y recibe su propio número de pedido (PID).

Cada cocinero con su receta y su mesa: si uno se quema, los demás siguen cocinando sin enterarse. Eso es el **aislamiento** de la burbuja de memoria. En el [punto 3](#) verás qué pasa cuando hay varias cocinas (varias CPUs) o varios restaurantes (varias máquinas).

Mini-chequeo

1. ¿Qué es el PID y para qué sirve?
2. ¿Qué contiene la burbuja de memoria de un proceso?
3. ¿Por qué dos procesos no pueden compartir una variable directamente?

>  Respuestas

Resumen en 3 frases

- Un proceso es un programa **en ejecución** con memoria, recursos y un PID propio.
- Su burbuja de memoria contiene código, estado, contador de programa y PID, aislada del resto.
- Los procesos no comparten memoria: se comunican con mecanismos externos.



Vocabulario rápido

Término	Idea general
Proceso	Programa en ejecución con memoria y recursos propios
PID	Identificador único del proceso
Burbuja de memoria	Espacio de direcciones aislado de cada proceso
Contador de programa	Qué instrucción toca ejecutar ahora
Contexto	Estado de la CPU, registros y contador del proceso



[Volver al índice de la unidad](#) · **Siguiente:** [02 · Estados de un proceso](#)

02 — Estados de un proceso

El ciclo de vida: de NUEVO a TERMINADO 

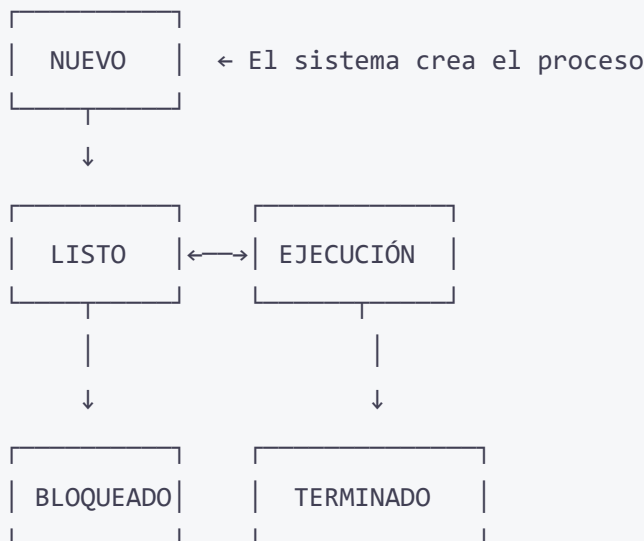
 Estás en:  U01 · Procesos y Subprocess → 02 · Estados de un proceso

La idea en una frase

Un proceso no está siempre “ejecutándose”: nace, espera su turno, usa la CPU, se bloquea esperando recursos y finalmente muere. Esos **cinco estados** forman su ciclo de vida.

El sistema operativo gestiona decenas o cientos de procesos a la vez con una sola CPU (o pocas). Para que todo parezca simultáneo, reparte la CPU en porciones de tiempo y cada proceso va pasando de estado según lo que le toca.

El ciclo de vida de un proceso



Estado	Qué significa
NUEVO	El proceso acaba de ser creado
LISTO	Preparado para ejecutar, esperando que la CPU esté libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones

Estado	Qué significa
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado

Las transiciones, una a una

- **NUEVO → LISTO**: el proceso ya está cargado y espera su turno de CPU.
- **LISTO → EJECUCIÓN**: el planificador (*scheduler*) le entrega la CPU.
- **EJECUCIÓN → LISTO**: el planificador se la quita porque llega otro con más prioridad o por **time slice** (cada proceso tiene una porción de tiempo).
- **EJECUCIÓN → BLOQUEADO**: el proceso pide una operación de E/S (leer un archivo, esperar un socket, `time.sleep()`) y no puede continuar hasta que llegue el dato.
- **BLOQUEADO → LISTO**: la E/S ha terminado y el proceso vuelve a la cola de preparados.
- **EJECUCIÓN → TERMINADO**: el proceso acaba (`return`, `sys.exit()`, o lo matan).

💡 **LISTO y BLOQUEADO no son lo mismo.** LISTO significa “puedo ejecutar, solo espero a que la CPU me toque”. BLOQUEADO significa “no puedo ejecutar aunque me dieran la CPU: estoy esperando un recurso”.

El ejemplo de la cafetería

Imagina una cola de gente en una cafetería con un único camarero:

- Entrás por la puerta → **NUEVO**.
- Te pones en la cola → **LISTO** (esperando al camarero).
- El camarero te atiende → **EJECUCIÓN**.
- Pides un café y el camarero se va a la máquina → **BLOQUEADO** (esperando el recurso “café”).
- El café está listo y vuelves a la cola → **LISTO**.
- El camarero te lo entrega → **EJECUCIÓN**.
- Pagas y te vas → **TERMINADO**.

El camarero (la CPU) alterna entre los clientes de la cola dando a cada uno unos segundos: por eso una sola CPU puede atender a muchos procesos “a la vez”.

Mini-chequeo

1. ¿Qué diferencia hay entre LISTO y BLOQUEADO?
2. ¿Qué transición ocurre cuando un proceso llama a `time.sleep(2)`?
3. Un proceso está ejecutándose y el planificador le quita la CPU porque llega otro con más prioridad. ¿A qué estado pasa?

>  Respuestas

Resumen en 3 frases

- Un proceso recorre cinco estados: **NUEVO**, **LISTO**, **EJECUCIÓN**, **BLOQUEADO** y **TERMINADO**.
- LISTO y BLOQUEADO son distintos: uno espera CPU y el otro espera un recurso.
- El planificador reparte la CPU y por eso un solo procesador parece ejecutar muchos procesos a la vez.

Vocabulario rápido

Término	Idea general
NUEVO	El proceso acaba de crearse
LISTO	Preparado, esperando que la CPU se libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado
Planificador (scheduler)	Decide qué proceso LISTO pasa a EJECUCIÓN

03 — Paralela vs Distribuida

CPUs, máquinas y la concurrencia de por medio 🧑🏻💻

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 03 · Paralela vs Distribuida

📌 La idea en una frase

Paralela es ejecutar varias tareas **a la vez** en varias CPUs; **distribuida** es ejecutarlas en **múltiples máquinas** conectadas por red; **concurrencia** es que varias tareas **avancen**, aunque se turnen en una sola CPU.

La diferencia está en el “dónde”: la paralela reparte trabajo entre los núcleos de tu máquina, la distribuida entre máquinas, y la concurrencia es una forma de *parecer* simultáneo aunque físicamente no lo sea.

🧑🏻💻 La tabla de los tres conceptos

Concepto	Qué significa
Paralela	Varias tareas ejecutándose a la vez en múltiples CPUs/núcleos
Distribuida	Varias tareas ejecutándose en múltiples máquinas conectadas por red
Concurrencia	Varias tareas avanzando (no necesariamente a la vez, pueden turnarse)

Ejemplos cotidianos

- **Paralela**: 4 hilos de cocina, cada uno friendo un huevo en su sartén (4 CPUs).
- **Distribuida**: 4 restaurantes en 4 ciudades, todos cocinando el mismo menú.
- **Concurrente**: 1 cocinero que va friendo huevos de 3 pedidos, alternando.

🔍 Paralela: cuatro sartenes

Imagina que tienes que freír 4 huevos. Con **una sola sartén** (una CPU), los fríes de uno en uno o alternando: eso es *concurrencia*. Con **4 sartenes** (4 CPUs), los fríes los 4 a la vez: eso es

paralelismo real.

En Python, el módulo multiprocessing crea procesos reales que el sistema operativo reparte entre los núcleos de tu CPU:

```
# Paralela con multiprocessing (varios CPUs)
from multiprocessing import Pool
def cuadrado(n):
    return n * n

with Pool(4) as p: # 4 procesos en paralelo
    print(p.map(cuadrado, [1, 2, 3, 4]))
```

Salida:

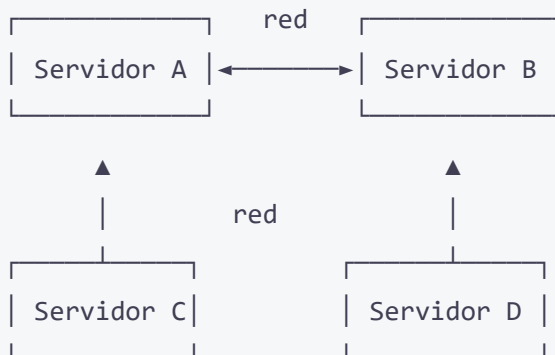
```
[1, 4, 9, 16]
```

Los 4 procesos se reparten la lista y cada uno calcula una parte. Si tu máquina tiene 4 núcleos, los 4 se ejecutan **a la vez**.



Distribuida: varios restaurantes

La computación distribuida lleva la idea al límite: no varias CPUs de la misma máquina, sino **máquinas completas conectadas por red**, cada una con su memoria y su CPU. Es el modelo de los clústeres, la web y los servicios en la nube.



Cada máquina ejecuta uno o varios procesos independientes. La distribución introduce un problema nuevo: **la comunicación por red** y **los fallos de máquina**. Eso lo verás en las unidades de sockets ([U04](#)) y de APIs REST ([U06](#)).



CPUs vs máquinas

Pregunta	Paralela	Distribuida
¿Dónde se ejecuta?	Múltiples CPUs/núcleos de una máquina	Múltiples máquinas conectadas por red
¿Comparten memoria?	Sí (misma máquina, memoria compartida)	No (cada máquina tiene la suya)
¿Comunicación?	Pipes, memoria compartida, locks	Sockets, HTTP, mensajería
¿Escala?	Hasta los núcleos de tu CPU	Hasta cientos de máquinas
Ejemplo en Python	<code>multiprocessing.Pool</code>	socket, APIs REST



El **GIL** de Python limita la concurrencia real de los hilos (lo verás en la [U02](#)); con **procesos** (`multiprocessing`) ese límite no existe porque cada proceso tiene su propio intérprete.



Mini-chequeo

1. ¿Cuál es la diferencia clave entre paralela y distribuida?
2. ¿Por qué el ejemplo del “cocinero alternando” es concurrencia y no paralelismo?
3. ¿Qué función de `multiprocessing` reparte una tarea entre varios procesos?

>  Respuestas



Resumen en 3 frases

- **Paralela** = varias CPUs a la vez; **distribuida** = varias máquinas por red; **concurrencia** = avanzar turnándose.
- `multiprocessing.Pool` reparte trabajo real entre los núcleos de tu CPU.
- La paralela comparte memoria y es interna a la máquina; la distribuida necesita red y comunicación por sockets/HTTP.



Vocabulario rápido

Término	Idea general
Paralela	Varias tareas a la vez en varias CPUs
Distribuida	Varias tareas en varias máquinas por red
Concurrencia	Varias tareas avanzando, turnándose
multiprocessing	Módulo Python que crea procesos reales
Pool	Grupo de procesos que reparte tareas

 [Volver al índice de la unidad](#) · Anterior: [02 · Estados de un proceso](#) · Siguiente: [04 · subprocess.run\(\)](#)

04 — subprocess.run()

Lanza un programa y espera a que termine 🕒

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 04 · subprocess.run()

La idea en una frase

`subprocess.run()` lanza un programa y **espera** a que termine, devolviéndote su salida y su código de retorno. Es la forma más sencilla de ejecutar otro programa desde Python.

Cuando necesitas el resultado de un comando (¿qué versión hay? ¿responde el ping? ¿qué dice este comando?), `run()` es tu herramienta: lanza, espera y captura.

El primer lanzamiento

```
import subprocess

# Comando simple
resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(f"Salida: {resultado.stdout}")
print(f"Código de retorno: {resultado.returncode}")
```

Salida (más o menos):

```
Salida: Python 3.11.4
Código de retorno: 0
```

- `subprocess.run(["python", "--version"])` recibe una **lista** con el ejecutable y sus argumentos. Cada elemento de la lista es un argumento aparte.
- `capture_output=True` captura la salida estándar (`stdout`) y la de errores (`stderr`).
- `text=True` devuelve cadenas en vez de bytes (mucho más cómodo para leer).
- `resultado.returncode` es el código de retorno: **0 = todo bien**, distinto de 0 = algo falló.



Los parámetros importantes

Parámetro	Qué hace
<code>capture_output=True</code>	Captura stdout y stderr
<code>text=True</code>	Devuelve strings en vez de bytes
<code>timeout=N</code>	Lanza excepción si tarda más de N segundos
<code>check=True</code>	Lanza excepción si el código de retorno no es 0

Resultado: stdout, stderr y returncode

Atributo	Qué contiene
<code>resultado.stdout</code>	La salida estándar del programa
<code>resultado.stderr</code>	La salida de errores
<code>resultado.returncode</code>	0 si fue bien, otro número si falló



Timeout: no esperes para siempre

Si un comando puede colgarse (un ping, un servidor), protege tu programa con `timeout`:

```
# Con timeout y control de errores
try:
    resultado = subprocess.run(
        ["ping", "google.com", "-n", "3"],
        capture_output=True,
        text=True,
        timeout=10
    )
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("❌ El ping tardó demasiado")
except subprocess.CalledProcessError:
    print("❌ Error en el comando")
```

- `subprocess.TimeoutExpired`: el comando superó los `timeout` segundos.
- `subprocess.CalledProcessError`: solo se lanza con `check=True` cuando el `returncode` no es 0.



Un ejemplo con error controlado

```
import subprocess

resultado = subprocess.run(["python", "--versio"], capture_output=True, text=True)
print(f"Código: {resultado.returncode}")
print(f"Error: {resultado.stderr}")
```

Salida:

```
Código: 2
Error: Unknown option: --versio
```

El código de retorno **2** (distinto de 0) te dice que el comando falló, y `stderr` te dice por qué. No hizo falta ninguna excepción: por defecto `run()` no lanza, solo devuelve el resultado.



Mini-chequeo

1. ¿Qué hace `capture_output=True` y `text=True`?

2. ¿Qué significa un returncode distinto de 0?
3. ¿Qué excepción lanza `run()` si el comando tarda más de `timeout`?

> Respuestas

Resumen en 3 frases

- `subprocess.run()` lanza un comando y **espera** su finalización.
- Captura la salida con `capture_output=True`, texto con `text=True` y controla el tiempo con `timeout`.
- El `returncode` te dice si fue bien (0) o mal (distinto de 0).

Vocabulario rápido

Término	Idea general
<code>subprocess.run</code>	Función que lanza un comando y espera
<code>capture_output</code>	Captura stdout y stderr
<code>text=True</code>	Devuelve strings en vez de bytes
<code>timeout</code>	Lanza <code>TimeoutExpired</code> si excede
<code>returncode</code>	Código de retorno (0 = bien)
<code>check=True</code>	Lanza <code>CalledProcessError</code> si <code>returncode</code> $\neq$ 0

 [Volver al índice de la unidad](#) · Anterior: [03 · Paralela vs Distribuida](#) · Siguiente: [05 · subprocess.Popen\(\)](#)

05 — subprocess.Popen()

Lanza un proceso y sigue tu camino 🚶

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 05 · subprocess.Popen()

📌 La idea en una frase

`subprocess.Popen()` lanza un proceso en **segundo plano** y devuelve el control inmediatamente: no espera a que termine. Tú sigues haciendo cosas mientras él vive.

`run()` espera; `Popen()` no. Eso convierte a `Popen` en la herramienta para abrir aplicaciones, lanzar varios programas a la vez y gestionarlos después con sus métodos.

🚀 El primer Popen: abrir el bloc de notas

```
import subprocess

# Lanzar el bloc de notas (no espera)
proceso = subprocess.Popen(["notepad.exe"])
print(f"Bloc de notas lanzado con PID {proceso.pid}")

# Podemos hacer otras cosas mientras el bloc de notas está abierto
print("Haciendo otras cosas...")

# Cuando queramos, esperamos a que termine
proceso.wait()
print("El bloc de notas se cerró")
```

Paso a paso:

1. `subprocess.Popen(["notepad.exe"])` lanza el bloc de notas y **vuelve al instante**.
2. `proceso.pid` te da el PID del proceso recién creado.
3. Python sigue ejecutando (`print("Haciendo otras cosas...")`) mientras el bloc de notas está abierto.
4. `proceso.wait()` **bloquea** a Python hasta que el bloc de notas se cierra.



Los métodos de Popen

Método de Popen	Qué hace
<code>proceso.wait()</code>	Espera a que termine (bloqueante)
<code>proceso.poll()</code>	Pregunta si ha terminado (no bloqueante)
<code>proceso.terminate()</code>	Envía señal de terminación
<code>proceso.kill()</code>	Mata el proceso forzosamente
<code>proceso.pid</code>	PID del proceso hijo

`wait()` vs `poll()`

- `wait()` **bloquea**: Python se queda parado hasta que el proceso termina.
- `poll()` **no bloquea**: devuelve `None` si el proceso sigue vivo, o su código de retorno si ya terminó.

```
import subprocess, time

proceso = subprocess.Popen(["notepad.exe"])

while proceso.poll() is None:
    print("El bloc de notas sigue abierto...")
    time.sleep(1)

print(f"Cerrado con código {proceso.poll()}")
```

`terminate()` vs `kill()`

- `terminate()` envía una señal de cierre **suave**: el proceso tiene oportunidad de guardar y salir.
- `kill()` lo mata **forzosamente**, sin darle opción. Úsalo solo cuando `terminate()` no baste.

```
proceso = subprocess.Popen(["calc.exe"])
print(f"Calculadora lanzada con PID {proceso.pid}")

proceso.terminate()    # cierre suave
proceso.kill()         # solo si el anterior no funcionó
```



Lanzar varios procesos a la vez

La gran ventaja de Popen es lanzar varios procesos y esperarlos después:

```
import subprocess, time

notepad = subprocess.Popen(["notepad.exe"])
calc = subprocess.Popen(["calc.exe"])
mspaint = subprocess.Popen(["mspaint.exe"])

print(f"PIDs: {notepad.pid}, {calc.pid}, {mspaint.pid}")

time.sleep(5)          # les damos 5 segundos de vida
notepad.terminate()
calc.terminate()
mspaint.kill()
print("Todos cerrados 🏁")
```

Los 3 procesos viven a la vez, cada uno con su PID, mientras Python hace otras cosas. Eso es imposible con `run()`, que espera a cada uno antes de lanzar el siguiente.



Mini-chequeo

1. ¿Qué diferencia hay entre `wait()` y `poll()`?
2. ¿Qué devuelve `proceso.poll()` mientras el proceso sigue vivo?
3. ¿Cuándo usarías `kill()` en lugar de `terminate()`?

>  Respuestas



Resumen en 3 frases

- Popen lanza un proceso en segundo plano y devuelve el control **al instante**.
- `wait()` bloquea, `poll()` pregunta, `terminate()` cierra suave y `kill()` mata a la fuerza.
- Con Popen puedes lanzar y gestionar **varios procesos a la vez**, cada uno con su PID.



Vocabulario rápido

Término	Idea general
Popen	Clase que lanza procesos en segundo plano
<code>wait()</code>	Espera bloqueante a que termine
<code>poll()</code>	Consulta no bloqueante del estado
<code>terminate()</code>	Señal de cierre suave
<code>kill()</code>	Muerte forzosa
<code>pid</code>	PID del proceso hijo

 [Volver al índice de la unidad](#) · Anterior: [04 · subprocess.run\(\)](#) · Siguiente: [06 · Comunicación con procesos](#)

06 — Comunicación con procesos

Pasa datos por stdin y lee la respuesta por stdout 📧

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 06 · Comunicación con procesos

📌 La idea en una frase

Un proceso hijo puede **leer de su stdin** y **escribir en su stdout**; con `communicate()` le pasas datos por un tubo y lees su respuesta por el otro.

Los procesos no comparten memoria ([punto 1](#)), pero el sistema operativo les presta **pipes** (tuberías): el `stdin`, `stdout` y `stderr` de cada proceso pueden conectarse a Python. Así un proceso “pregunta” y el otro “responde”.

🔌 Conectar los tubos

```
import subprocess

proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)
```

- `stdin=subprocess.PIPE`: Python podrá **escribir** en el `stdin` del hijo.
- `stdout=subprocess.PIPE`: Python podrá **leer** el `stdout` del hijo.
- `text=True`: los datos viajan como texto, no como bytes.

El proceso hijo ejecuta `print(input().upper())`: lee una línea de su `stdin`, la pasa a mayúsculas y la escribe en su `stdout`.

📧 Enviar y recibir con `communicate()`

```
salida, _ = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"
```

`communicate(input="hola mundo"):`

1. Escribe "hola mundo" en el stdin del hijo y cierra el tubo de entrada.
2. Espera a que el hijo termine.
3. Devuelve una tupla (stdout, stderr).

Después de `communicate()` el proceso ya terminó: no puedes volver a comunicarte con él.

El mismo ejemplo completo, con sentido

```
import subprocess

# Escribir en stdin y leer stdout
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

salida, error = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"

print(f"Error (None si todo bien): {error}")
# → Error (None si todo bien): None
```

Flujo de datos:

```
Python —stdin→ proceso hijo (lee input, pasa a mayúsculas)
proceso hijo —stdout→ Python (recibe "HOLA MUNDO")
```

¿Cuándo usar cada herramienta?

Situación	Herramienta
Necesitas el resultado de un comando y puedes esperar	<code>subprocess.run(..., capture_output=True)</code>
El proceso vive en segundo plano y no necesitas sus datos	<code>subprocess.Popen(...) + wait()</code>
Necesitas enviarle datos y/o leer su respuesta	<code>Popen(..., stdin=PIPE, stdout=PIPE) + communicate()</code>

⚠ **Cuidado con los deadlocks:** si el hijo escribe mucho en stdout y nadie lo lee, el pipe se llena y el hijo se bloquea esperando a que alguien lo vacíe. `communicate()` lee todo y te evita ese problema. Por eso con `run()` se usa `capture_output=True` y con `Popen`, `communicate()`.

Mini-chequeo

1. ¿Qué significa `stdin=subprocess.PIPE` y `stdout=subprocess.PIPE`?
2. ¿Qué devuelve `communicate(input="texto")`?
3. ¿Por qué `communicate()` evita los deadlocks con pipes?

>  Respuestas

Resumen en 3 frases

- Con `stdin=PIPE` y `stdout=PIPE` conectas tuberías entre Python y el proceso hijo.
- `communicate(input="...")` envía datos al hijo, espera su fin y devuelve (`stdout`, `stderr`).
- Los pipes permiten “preguntar” a un proceso y “leer” su respuesta sin compartir memoria.

Vocabulario rápido

Término	Idea general
Pipe	Tubería que conecta la salida de un proceso con la entrada de otro
stdin	Entrada estándar del proceso

Término	Idea general
stdout	Salida estándar del proceso
stderr	Salida de errores del proceso
PIPE	Constante que le dice a Popen “conecta esta tubería”
communicate()	Envía datos y recoge la salida del hijo

 [Volver al índice de la unidad](#) · Anterior: [05 · subprocess.Popen\(\)](#) · Siguiente: [07 · Compatibilidad Windows / Linux](#)

07 — Compatibilidad Windows / Linux

Un mismo código, dos mundos: la tabla de trucos  

 Estás en:  U01 · Procesos y Subprocess → 07 · Compatibilidad Windows / Linux

La idea en una frase

Los ejemplos de esta unidad están escritos para **Windows**, pero los conceptos (`subprocess.run`, `Popen`, `PID`, `communicate`) son **idénticos en todos los sistemas**: solo cambia el comando.

La API de `subprocess` es la misma en Windows, Linux y macOS. Lo único que cambia es qué ejecutable pones en la lista: `notepad.exe` no existe en Linux y `ls` no existe en Windows. Aprende a sustituir el comando y el código funciona igual.

La tabla de comandos equivalentes

Windows	Linux / macOS
<code>notepad.exe</code>	<code>gedit</code> , <code>nano</code> , <code>xed</code>
<code>calc.exe</code>	<code>gnome-calculator</code> , <code>bc</code>
<code>mspaint.exe</code>	<code>pinta</code> , <code>kolourpaint</code>
<code>ping -n 5 8.8.8.8</code>	<code>ping -c 5 8.8.8.8</code>
<code>ipconfig</code>	<code>ip addr</code> , <code>ifconfig</code>
<code>where python</code>	<code>which python</code>
<code>cmd /c mkdir</code>	<code>mkdir</code> (es ejecutable real)
<code>start http://...</code>	<code>xdg-open http://...</code>
<code>dir</code>	<code>ls</code>

Sustituye el comando correspondiente y el código funciona igual.

El truco del shell

Algunos comandos **no son ejecutables reales**: son palabras que interpreta el intérprete de comandos (shell). El ejemplo clásico es `mkdir`:

- En **Windows**, `mkdir` es un **comando interno de cmd**: no hay un archivo `mkdir.exe`, así que hay que invocarlo a través del shell con `cmd /c mkdir ....`
- En **Linux**, `mkdir` es un **ejecutable real** (`/usr/bin/mkdir`): se lanza directamente.

```
import subprocess

# En Windows: cmd /c ejecuta comandos internos del shell
resultado = subprocess.run(["cmd", "/c", "mkdir", "prueba_psp"],
                           capture_output=True, text=True)
print(f"Creada con código: {resultado.returncode}")
```

En Linux usarías directamente `["mkdir", "prueba_psp"]` porque `mkdir` es un ejecutable real.

Otro ejemplo: abrir el navegador con una URL.

- **Windows**: `start http://localhost:4321` (con `shell=True`, porque `start` es interno del shell).
- **Linux**: `xdg-open http://localhost:4321` (ejecutable real).

```
import subprocess

# Windows
subprocess.run(["start", "http://localhost:4321"], shell=True)

# Linux / macOS
# subprocess.run(["xdg-open", "http://localhost:4321"])
```

⚠ **shell=True con cuidado**: solo úsalo cuando el comando es interno del shell (como `start` o `dir`). Con `shell=True` y texto del usuario, un atacante podría inyectar comandos. Pasa siempre la lista de argumentos, no un string concatenado.

Lo que nunca cambia

Concepto	Windows	Linux
<code>subprocess.run(...)</code>	✓	✓
<code>subprocess.Popen(...)</code>	✓	✓
<code>capture_output=True, text=True</code>	✓	✓
<code>timeout, returncode</code>	✓	✓
<code>communicate(input=...)</code>	✓	✓
<code>proceso.wait(), poll(), terminate(), kill()</code>	✓	✓
<code>proceso.pid</code>	✓	✓

Aprende los conceptos con `notepad.exe` y `calc.exe` y, cuando toques Linux, solo cambia la lista de comandos de la tabla de arriba.



Mini-chequeo

1. ¿Qué comando de Windows equivale a `ls` en Linux?
2. ¿Por qué en Windows `mkdir` necesita `cmd /c`?
3. ¿Cuándo tienes que usar `shell=True`?

>  Respuestas



Resumen en 3 frases

- La API de `subprocess` es idéntica en Windows y Linux: solo cambian los comandos.
- Comandos internos del shell (como `mkdir` en Windows) necesitan `cmd /c` o `shell=True`.
- Sustituye el comando de la tabla y tu código funcionará en cualquier sistema.



Vocabulario rápido

Término	Idea general
Shell	Intérprete de comandos (cmd, bash)
Comando interno	Palabra que interpreta el shell (no es un ejecutable)
cmd /c	Ejecuta un comando interno de cmd en Windows
shell=True	Delega la ejecución en el shell (¡con cuidado!)
xdg-open	Abre una URL o archivo con la app por defecto en Linux

 [Volver al índice de la unidad](#) · **Anterior:** [06 · Comunicación con procesos](#) · **Siguiente:** [08 · Procesos en la práctica](#)

08 — Procesos en la práctica

Sé el código, el ring y los ejercicios del lápiz 

 Estás en:  U01 · Procesos y Subprocess → 08 · Procesos en la práctica

La idea en una frase

Aterrizas todo lo anterior: sé el código que abre dos aplicaciones a la vez, mira pelearse a `run()` y `Popen()` en el ring, y pon a prueba tus manos con los ejercicios del lápiz.

★ Sé el código: abrir bloc de notas y calculadora

“Sé el programa que abre dos aplicaciones a la vez. Traza cada paso.”

```
import subprocess, time

print("🚀 Abriendo bloc de notas...")
notepad = subprocess.Popen(["notepad.exe"])
print(f"    → PID: {notepad.pid}")

print("🚀 Abriendo calculadora...")
calc = subprocess.Popen(["calc.exe"])
print(f"    → PID: {calc.pid}")

print("\nAmbos programas están abiertos.")
print("El usuario puede escribir en el bloc de notas y usar la calculadora.")

# Mientras tanto, Python no está bloqueado
for i in range(5):
    print(f"    Python haciendo cosas... ({i+1}/5)")
    time.sleep(1)

print("\nCerrando programas...")
notepad.terminate()
calc.kill()
print("Hecho 🏁")
```

Traza paso a paso:

1. `subprocess.Popen(["notepad.exe"])`
 - El SO crea un nuevo proceso
 - Windows busca `notepad.exe` en el PATH
 - Lo carga en memoria
 - Asigna PID 12345
 - Python recibe el control inmediatamente
2. `print("PID: 12345")`
3. `subprocess.Popen(["calc.exe"])`
 - Mismo proceso, PID 12346
4. Python hace 5 cosas mientras ambos están abiertos
 - 3 procesos independientes: Python, notepad, calc
5. `notepad.terminate()`
 - Envía señal de cierre al bloc de notas
6. `calc.kill()`
 - Mata la calculadora forzosamente

Fíjate en el paso 4: **Python no se bloquea**. Mientras los dos programas están abiertos, el proceso de Python sigue imprimiendo. Tres procesos independientes conviven a la vez, cada uno con su PID, su memoria y su turno de CPU ([punto 1](#) y [punto 2](#)).



El ring de los conceptos — `run()` vs `Popen()`

Dos funciones de `subprocess` se citan en el cuadrilátero para resolver, de una vez, quién lanza mejor.

`run()`: — ¡Yo soy el rey de la simplicidad! Lanzas un comando, esperas, y ¡zas! tienes el resultado.

`Popen()`: — Sí, pero mientras tú esperas como un muñeco, yo puedo lanzar un proceso y seguir haciendo otras cosas. ¿Para qué esperar si no hace falta?

`run()`: — ¿Y si necesitas el resultado? Conmigo es directo: `result.stdout` y ya. Tú necesitas `communicate()`, más vueltas...

Popen(): — Pero imagina que quieres lanzar 3 programas a la vez. Conmigo puedes lanzarlos, irte a hacer café, y luego matarlos a todos. Tú tendrías que esperar a que termine cada uno antes de lanzar el siguiente.

run(): — Vale, vale... para procesos que viven en segundo plano, eres mejor. Pero para comandos rápidos y resultados inmediatos, soy más limpio.

Moraleja: `run()` para comandos rápidos que necesitan respuesta. `Popen()` para procesos que deben vivir en segundo plano.



Aprieta el lápiz

Pon a prueba lo aprendido. Intenta resolver cada ejercicio antes de mirar la solución.

1. PID del proceso

Crea un programa que imprima su propio PID y el PID del proceso padre.

>  Solución

2. Lanzador de apps

Usa `Popen` para abrir 3 programas distintos (bloc de notas, calculadora, navegador).

>  Solución

3. Comando con timeout

Lanza ping con timeout de 3 segundos. Captura el error si excede.

>  Solución

4. Comunicación bidireccional

Crea un proceso hijo que lea de `stdin` y devuelva el texto en mayúsculas.

>  Solución

5. Caza de procesos

Lanza 5 procesos notepad y luego mata todos menos uno.

>  Solución

Mini-chequeo

1. En el “Sé el código”, ¿cuántos procesos hay vivos mientras Python imprime sus 5 mensajes?
2. ¿Cuándo usarías `run()` y cuándo `Popen()`?
3. ¿Qué devuelve `os.getppid()`?

>  Respuestas

Resumen en 3 frases

- `Popen` lanza aplicaciones en segundo plano y Python sigue trabajando mientras viven.
- `run()` es para respuestas inmediatas; `Popen()` para procesos de larga vida ([ring](#)).
- Con `os.getpid()`, `os.getppid()`, `communicate()` y `terminate()` ya sabes jugar con procesos de verdad.

Vocabulario rápido

Término	Idea general
Sé el código	Técnica de traza mental: ejecutar el programa paso a paso en tu cabeza
<code>os.getpid()</code>	PID del proceso actual
<code>os.getppid()</code>	PID del proceso padre
<code>communicate()</code>	Enviar datos al hijo y leer su respuesta
<code>terminate()</code> / <code>kill()</code>	Cierre suave / muerte forzada

 [Volver al índice de la unidad](#) · Anterior: [07 · Compatibilidad Windows / Linux](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé el proceso, laboratorio con subprocess y el ring final de la unidad 🧠

📖 Estás en: 🚀 U01 · Procesos y Subprocess → 09 · Cierre

Has terminado la teoría: la burbuja de memoria y el PID, los cinco estados, paralela contra distribuida, `run()`, `Popen()`, los pipes con `communicate()` y la compatibilidad Windows/Linux. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el proceso

Eres un proceso de Python recién lanzado con `subprocess.Popen(["python", "calcula.py"])`. Acaban de asignarte un PID: 12345.

¿Qué pasa?

1. El sistema operativo crea tu proceso: asigna PID **12345**, reserva tu **burbuja de memoria** (código, estado, contador de programa).
2. Pasas al estado **NUEVO** y, en cuanto estás cargado, a **LISTO**: esperas tu turno de CPU en la cola del planificador.
3. La CPU te toca: pasas a **EJECUCIÓN**. Ejecutas las instrucciones de `calcula.py` una a una.
4. Llamas a `input()` para leer de `stdin`: te bloqueas (**BLOQUEADO**) esperando que el usuario escriba algo.
5. El usuario escribe y pulsa **ENTER**; la E/S termina y vuelves a **LISTO**.
6. La CPU te toca otra vez (**EJECUCIÓN**), terminas tu último `print()` y llegas a **TERMINADO**.
7. Tu padre espera con `proceso.wait()` o `poll()` y recoge tu código de retorno: **0**.

💡 **Ahora tú:** ¿y si tu padre lanza a la vez otros dos procesos (`calc.exe` y `notepad.exe`)? Los tres avanzáis turnándoos la CPU (conurrencia) y, si hay varios núcleos, alguno ejecutará de verdad a la vez (paralelismo). Eso es lo que viste en el [punto 3](#).

🔥 Fireside Chat: Proceso vs Hilo

Un proceso y un hilo se sientan junto a la chimenea a resolver, de una vez, quién es más ligero.

Proceso: — Yo soy la unidad completa: memoria propia, recursos, PID. Si me cuelgo, tú ni te enteras. Soy el [punto 1](#).

Hilo: — ¿Y para qué quieres toda esa burbuja? Yo vivo **dentro** de un proceso y comparto su memoria. Soy mucho más barato de crear.

Proceso: — Pero mis fallos no tumban a nadie más. Tú, si se te va un puntero, te llevas por delante al proceso entero.

Hilo: — Cierto, pero yo puedo compartir variables directamente. Tú necesitas pipes y `communicate()` para hablar con tu vecino.

Proceso: — Sí, pero en Python tengo otra ventaja: mi propio intérprete. Los hilos comparten el **GIL**, así que mi paralelismo con `multiprocessing` es real.

Hilo: — Vale, vale. Tú para aislamiento y paralelismo real; yo para tareas ligeras que comparten memoria. ¿Empate?

Proceso: — *sonríe* Empate... hasta la U02.

Moraleja: el proceso aísla y paraleliza de verdad; el hilo es ligero y comparte memoria. Los hilos son el plato principal de la [U02 · Hilos Fundamentos](#).



¿Quién soy?

1. Soy el identificador único que el SO asigna a cada proceso.
2. Soy el estado en el que el proceso espera un recurso (I/O, socket, sleep).
3. Soy el mecanismo por el que una sola CPU parece ejecutar varios procesos a la vez.
4. Soy el módulo de Python que reparte tareas en procesos paralelos reales.
5. Soy el tubo que conecta la salida de un proceso con la entrada de otro.
6. Soy el proceso que ya terminó pero que su padre no ha recogido todavía.

>  **Respuestas**



CONRAD VS EL MUNDO: “maté el proceso y no volvió”

CONRAD: — “Clásico: *‘he lanzado mi programa con subprocess.run y no me devuelve el control’*. ¡Pues claro! `run()` espera a que el comando termine, y si lanzas `notepad.exe`, el bloc de notas no se cierra nunca: te quedas colgado hasta que el usuario cierre la ventana. Si querías lanzar y seguir, eso es `Popen`, no `run()`.”

CONRAD: — “Y lo mejor: *‘puse `shell=True` para abrir el navegador y ahora un comando borra mis archivos’*. Pues sí: `shell=True` delega en el intérprete de comandos. Si le pasas un string con datos del usuario, es un agujero de seguridad. Lista de argumentos, nunca strings concatenados. O `cmd /c`, que para eso está.”

CONRAD: — “Y no me vengas con *‘¿será que mi proceso se ha convertido en zombie?’*. Míralo con `proceso.poll()`: si te devuelve el código de retorno, ya terminó y solo falta que lo recojas con `wait()` o `poll()`. Zombie sin recoger = entrada ocupada en la tabla de procesos. A diagnosticar.”

⚡ Laboratorio de tortura: el lanzador con cierre automático

Duración: 40 minutos **Herramienta:** Python 3 (`subprocess` y `time`, sin instalar nada)

Escenario: construye un “lanzador de aplicaciones” que abra tres programas (bloc de notas, calculadora y Paint), muestre sus PIDs, los deje vivir 5 segundos y luego cierre solo los que sigan abiertos.

Tareas paso a paso:

1. Define una función `lanzar_apps()` que cree los tres `Popen` y los guarde en una lista de tuplas (`nombre`, `proceso`).
2. Recorre la lista mostrando nombre y `proceso.pid`.
3. Espera 5 segundos con `time.sleep(5)` mientras las apps están abiertas.
4. Recorre otra vez la lista: si `proceso.poll()` is `None` (sigue viva), llámale `terminate()`; si ya terminó, imprime que se cerró sola.
5. Tras cada `terminate()`, recoge el código de retorno con `proceso.wait()` e imprímelo.
6. Envuelve la llamada en `if __name__ == "__main__":`.

Fallo intencionado: en el paso 5, en lugar de `wait()` tras `terminate()`, **borra esa línea** y deja el programa así. ¿Qué pasa? El padre lanza las apps, las mata, pero **nunca recoge su código de retorno**: los tres procesos terminados quedan como **zombies**, ocupando una entrada en la tabla de procesos hasta que el padre muere (o los recoge).

Pista 1: un proceso no desaparece al terminar: su código de retorno queda guardado hasta que el padre lo **recoge** con `wait()` o `poll()`. Sin esa llamada, la entrada en la tabla de procesos sigue ocupada: es el zombie de las [preguntas tontas](#).

Pista 2: si no ves el zombie a simple vista, añade `print(f"{nombre} → poll(): {proceso.poll()}")` justo después del `terminate()`. Con `wait()` devolverá el código de retorno y se limpia; sin él, el `poll()` te seguirá devolviendo un código “pendiente” en un proceso que ya murió.



Logros de esta unidad

Logro	Cómo conseguirlo
Process Whisperer	Explicar el ciclo de vida de un proceso: sus 5 estados y sus transiciones
Launcher	Abrir y gestionar varias aplicaciones a la vez con <code>Popen</code> , <code>terminate()</code> y <code>kill()</code>
Pipe Talker	Comunicar un proceso padre y un hijo con <code>communicate()</code>
Parallel Thinker	Distinguir paralela, distribuida y concurrencia, y saber cuándo usar <code>multiprocessing</code>



Atrévete a pensar

1. ¿Por qué un proceso no puede compartir memoria directamente con otro?
2. ¿Qué pasa si un proceso hijo muere y su padre nunca llama a `wait()` ni `poll()`?
3. ¿Cuándo usarías `multiprocessing` en lugar de `subprocess`?
4. ¿Por qué `shell=True` es peligroso si le pasas datos del usuario?
5. ¿Qué ventaja tiene `Popen` sobre `run()` para lanzar 3 aplicaciones a la vez?

🧩 Crucigrama de bits

Horizontal:

1. Identificador único del proceso (3 letras)
4. Estado en el que el proceso espera a que la CPU se libere (5 letras)
6. Señal de cierre suave de Popen (9 letras)
8. Proceso terminado sin recoger por su padre (6 letras)

Vertical:

2. Módulo Python que crea procesos en paralelo real (14 letras)
3. Estado del proceso mientras espera un recurso (9 letras)
5. Tubería que conecta la salida de un proceso con la entrada de otro (4 letras)
7. Método de Popen que pregunta sin bloquear (4 letras)

💬 Entrevista de trabajo

1. “¿Qué es un proceso y en qué se diferencia de un programa?”
2. “Explica el ciclo de vida de un proceso.”
3. “¿Qué diferencia hay entre computación paralela y distribuida?”
4. “¿Cómo lanzarías un programa externo desde Python y leerías su salida?”
5. “¿Qué diferencia hay entre `run()` y `Popen()`? ¿Cuándo usarías cada uno?”
6. “¿Cómo comunicarías dos procesos entre sí?”

💡 **Cómo encararlas:** la 4 y la 5 son las “preguntas reina”. Para la 4, recorre la cadena: `subprocess.run(..., capture_output=True, text=True) → resultado.stdout`; si el proceso necesita datos, `Popen(stdin=PIPE, stdout=PIPE) + communicate(input=...)`. Para la 5, repite la moraleja del [punto 8](#): `run()` para respuestas inmediatas, `Popen()` para segundo plano. Y para la 1 no olvides la **burbuja de memoria** y el **PID**. Si sabes contarlo fluido, ya eres medio programador de sistemas.

🙋 No hay preguntas tontas

? ¿Cuántos procesos puede tener mi sistema?

Depende de la **memoria RAM**. Cada proceso ocupa memoria. En Windows puedes verlos en el Administrador de tareas.

? ¿Qué pasa si un proceso hijo muere?

El proceso padre puede enterarse con `proceso.wait()` o `proceso.poll()`. Si no, el hijo se convierte en **zombie** (ocupa una entrada en la tabla de procesos).

? ¿Y si el padre muere antes que el hijo?

Los hijos se convierten en **huérfanos**. En Windows, el sistema los gestiona. En Linux, `init` los adopta.

? `run()` vs `Popen()` — ¿cuándo usar cada uno?

- `run()`: cuando necesitas el resultado y puedes esperar.
- `Popen()`: cuando el proceso debe vivir en segundo plano mientras tú haces otras cosas.

? ¿Puedo lanzar cualquier programa?

Sí, cualquier ejecutable. Pero el **PATH** debe incluirlo o debes dar la ruta completa.

Poscréditos

El bloc de notas y la calculadora se abren a la vez. Tres procesos independientes conviven: Python, notepad y calc.

El usuario escribe en el bloc de notas mientras la calculadora suma. La CPU alterna entre ellos: nadie se entera de que se turnan.

Notepad se cierra con un `terminate()` y su código de retorno vuela de vuelta a Python por el pipe.

Los procesos se comunican sin compartir memoria. Cada uno en su burbuja.

PRÓXIMAMENTE EN UO2: *Los procesos son pesados: cada uno su memoria, su burbuja, su PID. Pero ¿y si quieres que varias tareas compartan memoria y se turnen la CPU? Necesitas algo más ligero: los hilos.*



Criterios de evaluación cubiertos (RA1)

RA1: Reconoce las características y la gestión de los procesos en un sistema operativo.

CE	Criterio	Cubierto
a)	Reconoce las características de los procesos	✓ Burbuja de memoria y PID (punto 1) + ★ Sé el proceso
b)	Distingue entre computación paralela y distribuida	✓ Punto 3 + 🔥 Fireside (punto 9)
c)	Conoce los estados de un proceso	✓ Punto 2 + ★ Sé el proceso
d)	Identifica las diferencias clave entre proceso e hilo	→ U02 (🔥 Fireside lo anticipa)
e)	Crea programas con procesos (subprocess)	✓ Puntos 4-5-8 + ⚡ Laboratorio de tortura
f)	Establece comunicación entre procesos	✓ Punto 6 + ⚡ Laboratorio de tortura

RA1d (proceso vs hilo) se cubre en la **U02 · Hilos Fundamentos**. RA1g (análisis de ventajas de procesos frente a hilos) también en **U02**. RA1h (documentación) es transversal a todo el curso.

 [Volver al índice de la unidad](#) · Anterior: [U08 · Procesos en la práctica](#) · Siguiente: [U02 · Hilos Fundamentos](#)

U02 — Hilos Fundamentos

Multitarea dentro de un proceso: hilos en Python 🐍

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 **Hilos Fundamentos** → 🗝 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 📡 APIs comerciales → 🗝 Hash → 🧬 Cifrado → 🏗 Servidores → ⌚ asyncio

En la U01 lanzaste procesos completos con `subprocess`: cada uno con su propia memoria, su propio PID y su propio mundo. Ahora toca bajar un nivel y entrar *dentro* del proceso: un proceso puede tener varias tareas ejecutándose a la vez, compartiendo su memoria. Esas tareas son los **hilos** (threads): se crean en milisegundos, se comunican con variables compartidas y son la base de los servidores que atienden a muchos clientes a la vez.

Esta unidad es el primer contacto serio con los hilos en Python: los crearás con `threading.Thread`, los lanzarás con `start()`, los esperarás con `join()`, los convertirás en servidores de fondo con `daemon=True` y programarás avisos diferidos con `Timer`. También entenderás el famoso **GIL**, el candado que limita a los hilos de Python, y seguirás su ciclo de vida de punta a punta. Es la puerta de entrada a la sincronización del [TEMA 03 · Sincronización entre Hilos](#) y a los servidores concurrentes de la U10.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un hilo, en qué se diferencia de un proceso y por qué todos comparten la memoria del proceso que los contiene.
- Crear y lanzar hilos con `threading.Thread`, `start()` y `join()`, distinguiendo el hilo principal de los hilos secundarios.
- Pasar argumentos a un hilo con `args/kwargs` y nombrar cada hilo con `.name`.
- Explicar qué es un hilo **daemon**, por qué existe y cuándo usar uno en lugar de un hilo normal con `join()`.
- Usar `threading.Timer` para ejecutar una función una sola vez después de un retardo.

- Explicar el **GIL**: qué limita, por qué los hilos no aceleran el código CPU-bound y por qué sí aceleran el I/O-bound.
- Describir el ciclo de vida de un hilo: nuevo, ejecutable, en ejecución, bloqueado y terminado.
- Seguir la ejecución de varios hilos paso a paso (traza) sabiendo que el orden de salida nunca está garantizado.
- Diagnosticar cuándo los hilos sirven de verdad y cuándo conviene recurrir a multiprocessing.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · De proceso a hilo	Qué es un hilo y en qué se diferencia de un proceso	Todos
02 · Tu primer hilo	threading.Thread, start(), join() y el hilo principal	Todos
03 · Hilos con argumentos	args, kwargs y el nombre de cada hilo con .name	Todos
04 · Hilos daemon	Hilos de fondo que se sacrifican al salir del programa	Todos
05 · Timer	Ejecutar algo una sola vez tras un retardo	Todos
06 · El GIL	El candado de CPython: por qué los hilos no aceleran la CPU	Todos
07 · Estados del hilo	El ciclo de vida: nuevo, ejecutable, en ejecución, bloqueado, terminado	Todos
08 · Hilos en la práctica	Be the code, el ring Hilo vs Proceso y Aprieta el lápiz	Todos
09 · Cierre	Sé el hilo, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

✓ Criterios de evaluación cubiertos (RA2)

RA2 — Hilos (parcial): estructura, creación, esperas, daemon y GIL.

CE	Criterio	Dónde se cubre
RA2a	Identifica la estructura de un hilo	✓ Punto 1 + Punto 7
RA2b	Crea y lanza hilos con threading	✓ Puntos 2-3 + ⚡ Laboratorio (punto 9)
RA2e	Implementa esperas con join() y sleep()	✓ Puntos 2-4 + ⚡ Laboratorio (punto 9)
RA2f	Gestiona hilos daemon	✓ Punto 4 + ⚡ Laboratorio (punto 9)
RA2h	Conoce el GIL y sus limitaciones	✓ Punto 6 + Punto 8

Los criterios RA2c (Lock), RA2d (semáforos) y RA2g (condiciones de carrera) se cubren en el **TEMA 03 — Sincronización entre Hilos**.

¿Por dónde empiezo?

¿Vienes de la U01 y dominas procesos y subprocess? Perfecto, ese es el trampolín ideal: repasa la [U01 · Procesos y subprocess](#) para tener fresca la diferencia entre lanzar un programa entero y ejecutar una tarea dentro de él, y arranca en el [punto 1](#), que parte justo de ahí: de proceso a hilo.

¿Ya sabes lanzar hilos con start() y join() y solo necesitas el GIL o los estados? Puedes saltar a los [puntos 6](#) y [7](#). Pero si vienes de cero en multitarea, no te saltes los puntos 1 a 3: entender qué es un hilo y cómo se lanza es la base de todo el módulo.

📌 Primer punto: [01 · De proceso a hilo](#)

▶▶ Al acabar la unidad, continúa en [U03 · Sincronización entre Hilos](#).

01 — De proceso a hilo

Qué es un hilo y en qué se diferencia de un proceso 🧵

📖 Estás en: 🧵 U02 · Hilos Fundamentos → 01 · De proceso a hilo

📌 La idea en una frase

Un **hilo** es la unidad más pequeña de ejecución: una tarea que vive *dentro* de un proceso y comparte su memoria con los demás hilos de ese proceso.

En la U01 lanzaste procesos: programas completos con su propia memoria, su propio PID y su propio estado. Ahora entramos a un nivel más fino: un solo proceso puede tener **varios hilos ejecutándose a la vez**, todos trabajando con la misma memoria. Es como pasar de abrir varias casas (procesos) a repartir habitaciones dentro de una sola (hilos).

🧵 ¿Qué es un hilo?

Un **hilo** (thread) es la unidad más pequeña de ejecución que el sistema operativo puede gestionar. Un proceso puede tener múltiples hilos, todos compartiendo la misma memoria.

```
import threading

def saludar():
    print("¡Hola desde un hilo!")

hilo = threading.Thread(target=saludar)
hilo.start()
hilo.join()
```

Fíjate en las tres líneas mágicas que ya usarás toda la unidad:

1. `threading.Thread(target=saludar)` → **crea** el hilo (todavía no hace nada).
2. `hilo.start()` → **lo lanza**: el hilo empieza a ejecutar la función `saludar`.
3. `hilo.join()` → **espera** a que el hilo termine antes de seguir con el programa.

Características de un hilo

- **Comparten memoria** con otros hilos del mismo proceso (por eso se comunican tan rápido).
- Son **más ligeros que los procesos**: cuestan muchos menos recursos al crearlos.
- Se comunican mediante **variables compartidas** (con cuidado: eso es el TEMA 03).
- En Python, están **limitados por el GIL** para código CPU-bound (lo verás en el [punto 6](#)).

Hilos vs Procesos

Característica	Proceso	Hilo
Memoria	Aislada (cada uno la suya)	Compartida (todos en la misma)
Creación	Lenta (el SO debe copiar recursos)	Rápida
Comunicación	Pipes, sockets, archivos	Variables globales
Aislamiento	Alto (uno no afecta a otro)	Bajo (uno puede romper a todos)
Coste	Alto	Bajo

“Los procesos son como casas separadas. Los hilos son como habitaciones de la misma casa.”

La analogía de los chefs en una cocina

Imagina una cocina de restaurante con varios cocineros:

- **Procesos** = cocinas de restaurantes diferentes. Cada una tiene su propia cocina, sus propios fogones y sus propios ingredientes. Si un restaurante se quema, el de al lado ni se entera (aislamiento alto). Pero abrir un restaurante nuevo es caro y lento.
- **Hilos** = cocineros dentro de la **misma** cocina. Todos comparten la encimera, la nevera y los ingredientes (memoria compartida). Contratar a un cocinero más es barato y rápido. Pero si uno tira el aceite caliente, todos lo sufren (aislamiento bajo).

“Un hilo es como una tarea dentro de una casa. Todos los hilos comparten la misma casa (memoria), pero cada uno hace su propia cosa.”

Ese reparto de la misma nevera es lo que hace a los hilos tan rápidos para comunicarse... y tan peligrosos si dos tocan el mismo ingrediente a la vez. Ese peligro se llama **condición de carrera** y lo atacarás en el [TEMA 03 · Sincronización](#).



Mini-chequeo

1. ¿Dónde vive un hilo: dentro de un proceso o junto a él?
2. ¿Por qué un hilo es más barato de crear que un proceso?
3. ¿Cuál es el precio de compartir memoria entre hilos?

>  Respuestas



Resumen en 3 frases

- Un **hilo** es la unidad más pequeña de ejecución y vive dentro de un proceso compartiendo su memoria.
- Frente a los procesos, los hilos son más ligeros, se crean más rápido y se comunican con variables compartidas, pero con menos aislamiento.
- La memoria compartida es a la vez su gran ventaja y su gran riesgo: a eso le pondremos remedio en el TEMA 03.



Vocabulario rápido

Término	Idea general
Hilo (thread)	Unidad más pequeña de ejecución dentro de un proceso
Memoria compartida	La memoria del proceso, usada por todos sus hilos
threading.Thread	Clase de Python para crear un hilo
start()	Lanza el hilo: empieza a ejecutar su función
join()	Espera a que el hilo termine
GIL	Candado de CPython que limita los hilos para código CPU-bound

02 — Tu primer hilo

Crea, lanza y espera tu primer hilo con `start()` y `join()` 🚀

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 02 · Tu primer hilo

La idea en una frase

Crear un hilo es escribir `threading.Thread(target=funcion)`, lanzarlo con `.start()` y esperarlo con `.join()`.

Tres líneas. Eso es todo lo que necesitas para que una función se ejecute en paralelo con el resto del programa. La gracia (y la complicación) está en *cuándo* se ejecuta cada pieza.

Crear y lanzar un hilo

```
import threading

def saludar():
    print("¡Hola desde un hilo!")

hilo = threading.Thread(target=saludar)
hilo.start()
hilo.join()
```

Salida:

```
¡Hola desde un hilo!
```

Desglose de las tres líneas clave:

Línea	Qué hace
<code>threading.Thread(target=saludar)</code>	Crea el hilo. En este momento el hilo existe pero no hace nada (estado NUEVO, lo verás en el punto 7).

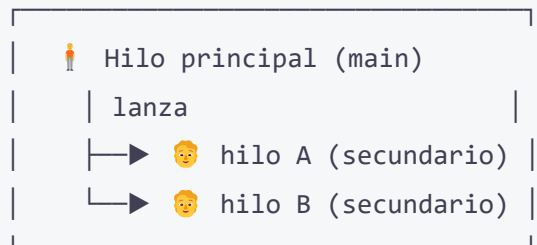
Línea	Qué hace
<code>hilo.start()</code>	Lanza el hilo: pasa a ejecutable y el sistema operativo decide cuándo ejecutar <code>saludar()</code> .
<code>hilo.join()</code>	Espera a que el hilo termine. Sin ella, el programa principal seguiría su camino y podría terminar antes que el hilo.

⚠ Un error típico de principiante: llamar a `saludar()` directamente (con paréntesis) en vez de pasar la función sin ellos. `target=saludar` pasa la *referencia*; `target=saludar()` ejecuta la función antes de crear el hilo y no lanza nada.

👑 El hilo principal vs los hilos secundarios

Cuando ejecutas `python programa.py`, tu código se ejecuta dentro de un hilo: el **hilo principal** (main thread). Todo lo que lanzas con `Thread()` son **hilos secundarios** que viven dentro del mismo proceso.

PROGRAMA (proceso)



El hilo principal **no espera** a los secundarios por arte de magia: hay que decírselo con `join()`. Sin `join()`, el principal puede llegar al final de su código y el programa terminará cuando termine... pero sin garantías sobre los secundarios.

🕒 `join()` — esperar a que termine

`join()` hace que el programa principal espere hasta que el hilo termine.

```
import threading, time

def trabajador(segundos):
    print(f"Trabajando durante {segundos}s...")
    time.sleep(segundos)
    print("Terminado")

h = threading.Thread(target=trabajador, args=(3,))
h.start()

print("Esperando al hilo...")
h.join() # Espera hasta que termine
print("El hilo terminó, continuamos")
```

Salida:

```
Trabajando durante 3s...
Esperando al hilo...
Terminado
El hilo terminó, continuamos
```

Fíjate en el orden: el `print("Esperando al hilo...")` del principal aparece *mientras* el hilo sigue dormido. El principal llega a `join()`, se queda bloqueado esperando, y recién cuando el hilo termina sigue con su última línea.

Sin `join()`, el programa principal seguiría y posiblemente terminaría antes que el hilo. Con `join()`, tienes la garantía de que el hilo ha acabado cuando tú continúas.

Propiedades de un hilo

Un hilo no es solo una función en ejecución: es un objeto con información útil.

```
import threading

def fn():
    print(f"Ejecutando {threading.current_thread().name}")

hilo = threading.Thread(target=fn, name="hilo-1")
hilo.start()
hilo.join()

print(hilo.name)          # "hilo-1"
print(hilo.ident)         # ID numérico del hilo
print(hilo.daemon)        # True/False (lo verás en el punto 4)
print(hilo.is_alive())    # True si sigue ejecutándose
```

Propiedad	Qué devuelve
<code>.name</code>	El nombre del hilo (por defecto algo como "Thread-1")
<code>.ident</code>	Un ID numérico único mientras el hilo vive
<code>.daemon</code>	True si es un hilo daemon, False si no
<code>.is_alive()</code>	True si el hilo aún se está ejecutando



Desde dentro del hilo puedes obtener tu propia información con `threading.current_thread()`, que devuelve el objeto Thread actual.



Mini-chequeo

1. ¿Qué pasa si creas un hilo y no llamas a `start()`?
2. ¿Qué pasa si llamas a `start()` dos veces sobre el mismo hilo?
3. ¿Para qué sirve `join()`? ¿Qué riesgo evitas?

>  Respuestas



Resumen en 3 frases

- Crear y lanzar un hilo es `Thread(target=fn)`, `.start()` y `.join()`: crear, lanzar, esperar.

- El **hilo principal** no espera a los secundarios por defecto; `join()` fuerza esa espera.
- Cada hilo es un objeto con `.name`, `.ident`, `.daemon` e `.is_alive()` que te cuenta su estado.



Vocabulario rápido

Término	Idea general
Hilo principal	El hilo que ejecuta <code>python programa.py</code> , el “main”
Hilo secundario	Un hilo lanzado desde otro con <code>Thread()</code>
<code>start()</code>	Lanza el hilo: de nuevo a ejecutable
<code>join()</code>	Bloquea el programa principal hasta que el hilo termina
<code>is_alive()</code>	Dice si el hilo todavía se está ejecutando
<code>current_thread()</code>	Devuelve el objeto <code>Thread</code> desde dentro del propio hilo

 [Volver al índice de la unidad](#) · Anterior: [01 · De proceso a hilo](#) · Siguiente: [03 · Hilos con argumentos](#)

03 — Hilos con argumentos

Pasa datos al hilo con args y kwargs, y ponle nombre a cada hilo 🖨️

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 03 · Hilos con argumentos

📌 La idea en una frase

Para que un hilo trabaje con datos propios le pasas argumentos en args= (tupla) o kwargs= (diccionario), y para distinguirlo le pones nombre con .name.

Sin argumentos, todos los hilos harían exactamente lo mismo. Con args y kwargs, cada hilo puede hacer su versión del trabajo. Y con .name, puedes saber en cada momento *quién* está haciendo qué.

🎒 Pasar argumentos con args

La función que ejecuta el hilo puede recibir argumentos: se los pasas en args= como una **tupla**, en el mismo orden que espera la función.

```
import threading

def trabajar(nombre, tarea):
    print(f"{nombre} empezando: {tarea}")
    # ... trabajo ...
    print(f"{nombre} terminó: {tarea}")

# Crear hilos con nombre y argumentos
hilo_a = threading.Thread(target=trabajar, args=("Ana", "lavar platos"))
hilo_b = threading.Thread(target=trabajar, args=("Bob", "fregar suelo"))

# Lanzarlos
hilo_a.start()
hilo_b.start()

# Esperar
hilo_a.join()
hilo_b.join()
```


Salida (orden no garantizado):

```
Ana empezando: lavar platos
Bob empezando: fregar suelo
Ana terminó: lavar platos
Bob terminó: fregar suelo
```

⚠ **Truco de la coma:** una tupla de un solo elemento necesita coma final: `args=("Ana",)`. Sin la coma, `("Ana")` es solo un string, no una tupla, y trabajar recibiría un solo argumento.

kwargs: argumentos con nombre

Si tu función usa parámetros con nombre, puedes pasarlos con `kwargs=` en forma de diccionario.

```
import threading

def preparar(plato, minutos):
    print(f"Preparando {plato}: tarda {minutos} min")

hilo = threading.Thread(
    target=preparar,
    kwargs={"plato": "paella", "minutos": 40}
)
hilo.start()
hilo.join()
```

Salida:

```
Preparando paella: tarda 40 min
```

Puedes mezclar ambos: `args=("Ana",)` y `kwargs={"tarea": "limpiar"}` si la función recibe (nombre, tarea=...). Lo importante es que el orden de args coincide con los parámetros posicionales, y kwargs con los de nombre.

Nombrar hilos: `.name = "hilo-" + str(n)`

Cuando lanzas varios hilos iguales, ¿cómo sabes cuál es cuál? Poniéndoles nombre. La forma clásica es generarlos con un nombre único dentro de un bucle:

```
import threading

def trabajar(n):
    print(f"🌱 {threading.current_thread().name}: trabajando ({n})")
    # ... trabajo ...
    print(f"🏴‍☠️ {threading.current_thread().name}: terminado ({n})")

hilos = []
for i in range(1, 4):
    h = threading.Thread(target=trabajar, args=(i,))
    h.name = "hilo-" + str(i)  # 📄 nombre único
    hilos.append(h)

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Salida (orden no garantizado):

```
🌱 hilo-1: trabajando (1)
🌱 hilo-2: trabajando (2)
🌱 hilo-3: trabajando (3)
🏴‍☠️ hilo-1: terminado (1)
🏴‍☠️ hilo-3: terminado (3)
🏴‍☠️ hilo-2: terminado (2)
```

Fíjate en los detalles:

- `h.name = "hilo-" + str(i)` asigna un **nombre único** a cada hilo antes de lanzarlo.
- Dentro de la función, `threading.current_thread().name` devuelve el nombre del hilo que está ejecutando.
- Los print **se entremezclan**: el orden exacto lo decide el scheduler del sistema operativo, no nosotros.

💡 También puedes poner el nombre directamente al crear: `threading.Thread(target=trabajar, args=(i,), name="hilo-" + str(i))`. El resultado es idéntico.



Varios hilos a la vez

Con una **lista por comprensión** puedes crear y lanzar N hilos en tres líneas. Es el patrón que usarás toda la unidad (y en los servidores concurrentes de la U10):

```
import threading

def tarea(n):
    print(f"{threading.current_thread().name} → tarea {n}")

hilos = [threading.Thread(target=tarea, args=(i,), name="hilo-" + str(i))
          for i in range(1, 5)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()

print("Todas las tareas terminadas")
```

Salida (orden no garantizado):

```
hilo-1 → tarea 1
hilo-3 → tarea 3
hilo-2 → tarea 2
hilo-4 → tarea 4
Todas las tareas terminadas
```

El mensaje final **siempre aparece el último** gracias a los dos bucles: primero lanzas todos, luego esperas a todos con `join()`.



Mini-chequeo

1. ¿Qué le pasas a `args=`? ¿Y a `kwargs=`?
2. ¿Por qué hace falta la coma en `args=("Ana",)`?
3. ¿Cómo sabes, dentro de la función del hilo, el nombre del hilo que te está ejecutando?

✓ Resumen en 3 frases

- Los argumentos del hilo se pasan con `args=` (tupla) o `kwargs=` (diccionario), en el orden o con el nombre que espera la función.
- Con `.name = "hilo-" + str(n)` cada hilo lleva un nombre único que puedes leer con `threading.current_thread().name`.
- El patrón lista por comprensión + `for h in hilos: h.start()` + `for h in hilos: h.join()` lanza N hilos a la vez y espera a todos.

Vocabulario rápido

Término	Idea general
<code>args</code>	Tupla de argumentos posicionales que recibe la función del hilo
<code>kwargs</code>	Diccionario de argumentos con nombre que recibe la función
<code>.name</code>	Nombre del hilo, único para distinguirlo
<code>current_thread()</code>	El objeto Thread del hilo que se está ejecutando
Lista por comprensión	Forma compacta de crear N hilos en una línea

 [Volver al índice de la unidad](#) · Anterior: [02 · Tu primer hilo](#) · Siguiente: [04 · Hilos daemon](#)

04 — Hilos daemon

Hilos de fondo que se sacrifican para que el programa pueda salir 🐙

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 04 · Hilos daemon

La idea en una frase

Un hilo **daemon** se ejecuta en segundo plano y **se mata automáticamente** cuando el programa principal termina. Los no-daemon, en cambio, impiden que el programa salga hasta terminar.

Imagina una alarma de incendios: la quieres encendida mientras el edificio está vivo, pero si el edificio desaparece, la alarma no tiene sentido y muere con él. Esa es la filosofía del hilo daemon.

¿Qué es un hilo daemon?

Un hilo **daemon** (diablo) es un hilo de fondo pensado para tareas auxiliares: monitorizar, limpiar, emitir un heartbeat, mostrar un reloj... La regla de oro:

- Un hilo **no daemon** (por defecto) impide que el programa termine hasta que él acabe.
- Un hilo **daemon** se corta en seco cuando el programa principal llega al final, sin esperar a nada.

```
import threading, time

def reloj():
    """Imprime la hora cada segundo (por siempre)"""
    while True:
        print(f"🕒 {time.strftime('%H:%M:%S')}")
        time.sleep(1)

hilo_reloj = threading.Thread(target=reloj, daemon=True)
hilo_reloj.start()

time.sleep(3) # El programa principal dura 3 segundos
print("Programa principal terminando...")
# Al terminar, el hilo daemon se mata solo
```

Salida:

```
🕒 14:35:22
🕒 14:35:23
🕒 14:35:24
Programa principal terminando...
```

El reloj es un `while True`: nunca terminaría. Si no fuera `daemon`, el programa se quedaría colgado para siempre esperándolo. Siendo `daemon`, el programa principal hace su vida, y al terminar **el hilo muere con él**.

Tipo	Comportamiento
<code>daemon=False</code> (defecto)	El programa espera a que termine
<code>daemon=True</code>	El programa lo mata al salir

Los hilos **no daemon** impiden que el programa termine. Los `daemon` se sacrifican para que el programa pueda salir.



Be the code, my friend — daemon en acción

Vamos a ser el código. El mismo trabajador de 5 pasos, primero sin `daemon` y luego con él.

SIN daemon — el programa espera:

```
import threading, time

def trabajador():
    for i in range(5):
        print(f"Trabajando... ({i+1}/5)")
        time.sleep(0.5)
    print("Trabajador terminó")

# SIN daemon — el programa espera
h1 = threading.Thread(target=trabajador)
h1.start()
h1.join()
print("Programa terminó (después del hilo)")
```

Salida:

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Trabajando... (4/5)
Trabajando... (5/5)
Trabajador terminó
Programa terminó (después del hilo)
```

El hilo hace sus 5 pasos completos y **recién después** el programa principal termina.

CON daemon — el hilo se mata al salir:

```
# CON daemon — el hilo se mata al salir
h2 = threading.Thread(target=trabajador, daemon=True)
h2.start()
time.sleep(1.2)
print("Programa terminó (el hilo daemon muere conmigo)")
```

Salida:

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Programa terminó (el hilo daemon muere conmigo)
```

El daemon solo llegó a la iteración 3. El programa principal terminó y lo mató. Fíjate: no hay `join()` aquí, porque no queremos esperarlo; justo lo contrario.

El ring de los conceptos — Hilo normal vs Hilo daemon

Hilo Normal: — Yo soy un hilo de verdad. El programa principal espera a que termine lo que tengo que hacer. Tengo responsabilidad.

Hilo Daemon: — ¡Qué aburrido! Yo soy libre. Mi única misión es servir en segundo plano. Cuando el programa principal termina, yo me muero con él, sin dramas.

Hilo Normal: — ¿Y si estás en medio de algo importante cuando el main termina? Pierdes datos, dejas cosas a medias...

Hilo Daemon: — Para eso existen los daemon bien hechos: tareas de monitorización, limpieza, heartbeat... cosas que da igual si se cortan. Si quieres garantía de finalización, usas un hilo normal con `join()`.

Hilo Normal: — Cierto. Al final, cada uno tiene su sitio. Yo para tareas críticas, tú para servicios auxiliares.

Moraleja: Usa hilos **daemon** para servicios de fondo prescindibles. Usa hilos **normales** con `join()` para tareas que deben completarse sí o sí.

`join()` para esperar a un daemon (si acaso lo necesitas)

Un daemon se mata solo al final... pero también puedes esperarlo explícitamente con `join()` si quieres que termine *antes* de que el programa siga. El `join()` funciona igual: bloquea hasta que el hilo termina. Con un daemon infinito (`while True`) eso significa esperar para siempre, así que solo tiene sentido con daemons finitos o con `join(timeout)`:


```
import threading, time

def aviso():
    time.sleep(2)
    print("Aviso listo")

h = threading.Thread(target=aviso, daemon=True)
h.start()
h.join(timeout=3)  # espera como mucho 3 segundos
print("Continuamos...")
```

`join(timeout=N)` espera un máximo de N segundos: si el hilo no ha terminado, el programa sigue igual. Es la herramienta perfecta para no quedarse bloqueado.



Mini-chequeo

1. ¿Un hilo `daemon` impide que el programa termine?
2. ¿Cuándo usarías `daemon=True` en lugar de un hilo normal?
3. ¿Qué hace `join(timeout=2)` si el hilo tarda 5 segundos?

>  Respuestas



Resumen en 3 frases

- Un hilo **daemon** se ejecuta en segundo plano y se **mata al terminar** el programa principal.
- Un hilo **normal** impide que el programa salga hasta completarse; por eso las tareas críticas van con `join()`.
- Regla: `daemon` para servicios auxiliares prescindibles, `normal` para lo que debe terminarse sí o sí.



Vocabulario rápido

Término

Idea general

Hilo `daemon`

Hilo de fondo que muere cuando termina el programa principal

Término	Idea general
daemon=True	Flag que convierte un hilo en daemon
Hilo normal	Hilo que el programa principal espera antes de salir
join(timeout)	Espera al hilo un máximo de segundos y sigue
Heartbeat	Señal periódica de “sigo vivo”, típica de hilos daemon

 [Volver al índice de la unidad](#) · **Anterior:** [03 · Hilos con argumentos](#) · **Siguiente:** [05 · Timer](#)

05 — Timer

Ejecuta una función una sola vez después de un retardo 🕒

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 05 · Timer

📌 La idea en una frase

`threading.Timer(retardo, funcion)` ejecuta funcion **una sola vez** después de retardo segundos, sin bloquear el resto del programa.

El Timer es un hilo especial que en vez de empezar a trabajar ya, se queda “dormido” el tiempo que le digas y entonces dispara la función. Es la manera más sencilla de programar un aviso diferido.

🕒 El ejemplo mínimo

```
import threading

def aviso():
    print("🕒 ¡Tiempo cumplido!")

temporizador = threading.Timer(5.0, aviso)
temporizador.start()
print("Timer iniciado, 5 segundos...")

# temporizador.cancel() # Si queremos cancelar
```

Salida:

```
Timer iniciado, 5 segundos...
🕒 ¡Tiempo cumplido!
```

El `print("Timer iniciado...")` sale **inmediatamente**: el programa no se queda bloqueado esperando. Cinco segundos después, el Timer dispara `aviso()` por su cuenta, como un hilo más.

Timer ejecuta la función **una sola vez** después del retardo. No se repite. Si lo que quieres es repetirlo cada N segundos, la forma clásica es que la propia función se re programe (lo

retocamos en el boletín avanzado).



Cancelar un Timer

Como el Timer es un hilo, puedes cancelarlo antes de que dispare con `.cancel()`:

```
import threading, time

def aviso():
    print("🕒 ¡Tiempo cumplido!")

t = threading.Timer(5.0, aviso)
t.start()

time.sleep(2)
t.cancel()          # cancelamos antes de los 5 segundos
print("Cancelado: el aviso nunca llegará")
```

Salida:

```
Cancelado: el aviso nunca llegará
```

`cancel()` funciona si el Timer todavía no ha disparado. Una vez disparado, ya no tiene sentido: la función ya se ejecutó.



Pasar argumentos al Timer

El Timer admite los mismos args/kwargs que cualquier hilo:

```
import threading

def recordatorio(mensaje, veces):
    for _ in range(veces):
        print(f"📌 {mensaje}")

t = threading.Timer(3.0, recordatorio, args=("¡Beber agua!", 3))
t.start()
print("Recordatorio programado en 3 segundos...")
```

Salida:

```
Recordatorio programado en 3 segundos...
📌 ¡Beber agua!
📌 ¡Beber agua!
📌 ¡Beber agua!
```

💡 Usos típicos del Timer

Uso	Ejemplo
Despertador / aviso	"¡Despierta!" a los 3 segundos
Recordatorio	Un mensaje que aparece al pasar un rato
Timeout de una operación	Cancelar algo que tarda demasiado
Cierre de sesión	Desconectar a un usuario tras N segundos de inactividad
Limpieza diferida	Borrar un archivo temporal después de un tiempo

Un detalle a recordar: el Timer dispara desde un **hilo aparte**. Si tu programa principal termina antes de que dispare y el Timer es **daemon por defecto** (`Timer(..., daemon=True)`), el aviso puede morir con el programa. Si lo necesitas sí o sí, haz `daemon=False` o añade un `time.sleep()` en el principal.

🧠 Mini-chequeo

1. ¿Cuántas veces ejecuta el Timer la función?
2. ¿El programa principal se bloquea mientras espera el retardo?
3. ¿Qué hace `t.cancel()`?

> Respuestas

Resumen en 3 frases

- `threading.Timer(retardo, funcion)` ejecuta la función **una sola vez** después del retardo, sin bloquear el programa.
- Puedes pasar argumentos con `args=/kwargs=` y cancelar el disparo con `.cancel()`.
- Es ideal para avisos, recordatorios y timeouts; si quieres repetición, la función debe reprogramarse a sí misma.

Vocabulario rápido

Término	Idea general
<code>threading.Timer</code>	Hilo que dispara una función tras un retardo
Retardo	Segundos que espera el Timer antes de ejecutar
<code>cancel()</code>	Cancela el disparo si aún no ha ocurrido
<code>args / kwargs</code>	Argumentos que recibe la función del Timer
Una sola vez	Comportamiento del Timer: no se repite por defecto

 [Volver al índice de la unidad](#) · Anterior: [04 · Hilos daemon](#) · Siguiente: [06 · El GIL](#)

06 — EL GIL

El candado de CPython que limita a los hilos para código CPU-bound 🔒

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 06 · El GIL

🔒 La idea en una frase

GIL = Global Interpreter Lock: un candado interno de CPython que evita que dos hilos ejecuten bytecode Python a la vez, así que los hilos no aceleran el código de CPU... pero sí el de espera.

Es la gran trampa de los hilos en Python: “¿para qué sirven si no aceleran nada?” La respuesta corta: sí aceleran lo que vale la pena (esperar por red y disco) y no aceleran lo que no (calcular). Vamos a verlo con números.

🔒 ¿Qué es el GIL?

GIL = Global Interpreter Lock. Es un candado que CPython (el Python de toda la vida) mantiene para que **solo un hilo ejecute bytecode Python en cada instante**.

¿Por qué existe? Para proteger la memoria interna del intérprete: sin el candado, dos hilos podrían corromper las estructuras de Python al tocarlas a la vez. Python paga el precio de la seguridad con un límite: **no hay paralelismo real de CPU** entre hilos de un mismo proceso.



Los hilos se turnan el candado en pequeños fragmentos: parecen simultáneos (por eso los print se entremezclan), pero nunca ejecutan Python a la vez.

⚡ CPU-bound: los hilos NO sirven de nada

Una tarea **CPU-bound** es la que usa la CPU a tope: sumar millones de números, procesar imágenes, comprimir. Ahí el GIL no deja ejecutar a más de un hilo, así que 4 hilos tardan lo mismo que 1.

```
import threading, time

# ⚡ CPU-bound: los hilos NO sirven de nada
def contar():
    total = 0
    for i in range(50_000_000):
        total += i

inicio = time.time()
hilos = [threading.Thread(target=contar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos CPU: {time.time() - inicio:.2f}s")
# → ~5 segundos (igual que 1 hilo)
```

Cuatro hilos contando 50 millones cada uno... y el tiempo es prácticamente el mismo que con uno solo. Peor aún: a veces es **ligeramente más lento** por el coste de turnarse el candado.

I/O-bound: los hilos **SÍ** aceleran

Una tarea **I/O-bound** espera por algo externo: descargar de una red, leer de disco, esperar una respuesta. Ahí el hilo **libera el GIL mientras espera**, y los demás pueden trabajar.

```
# 🌐 I/O-bound: los hilos SÍ aceleran
def esperar():
    time.sleep(2) # Simula descarga/lectura

inicio = time.time()
hilos = [threading.Thread(target=esperar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos I/O: {time.time() - inicio:.2f}s")
# → ~2 segundos (4 descargas en paralelo)
```


En serio: cuatro hilos esperando 2 segundos **cada uno** terminan en ~2 segundos, no en 8. Mientras un hilo espera la red, el siguiente aprovecha para iniciar su descarga. Todos esperan a la vez.



La tabla que hay que saberse

Tipo de tarea	¿Hilos ayudan?	Motivo
CPU-bound (calcular, procesar)	✗ No	El GIL solo deja ejecutar a uno
I/O-bound (esperar red, disco)	✓ Sí	El GIL se libera durante la espera

💡 **¿Cómo distingo cada tipo?** Si tu programa gasta el tiempo **calculando** (contar, cifrar, procesar), es CPU-bound. Si gasta el tiempo **esperando** (descargar, consultar una API, leer archivos), es I/O-bound. Los hilos brillan en el segundo caso.



Cuándo sirven de verdad los hilos

Con lo visto, el mapa mental queda así:

- **Solicitudes de red** → hilos sí. Descargar 10 archivos con 10 hilos es ~10 veces más rápido.
- **Lecturas/escrituras de archivos** → hilos sí. Varias operaciones de disco en paralelo.
- **Servidores que atienden clientes** → hilos sí. Cada cliente espera su turno; mientras espera, otros avanzan (lo verás en la U10).
- **Cálculo puro** → hilos no. Para eso, **multiprocessing**.

Para CPU-bound en Python, usa **multiprocessing** (varios procesos, cada uno con su propio GIL). Esos procesos sí ejecutan en paralelo de verdad, a cambio del coste de crear procesos que viste en el [punto 1](#).



Mini-chequeo

1. ¿Qué significa exactamente “CPython solo ejecuta un hilo a la vez”?
2. Descargar 5 archivos de 2 segundos cada uno: ¿1 hilo o 5 hilos? ¿Cuánto tarda cada opción?

3. Sumar 50 millones de números con 4 hilos: ¿más rápido, igual o más lento que con 1?

>  Respuestas

Resumen en 3 frases

- El **GIL** es un candado de CPython que impide que dos hilos ejecuten bytecode a la vez.
- Para código **CPU-bound**, los hilos no sirven; para código **I/O-bound**, aceleran muchísimo porque el GIL se libera durante la espera.
- Cuando necesitas paralelismo real de CPU, toca usar `multiprocessing`, no hilos.

Vocabulario rápido

Término	Idea general
GIL	Global Interpreter Lock: candado de CPython
CPU-bound	Tarea que gasta el tiempo calculando (no le sirven los hilos)
I/O-bound	Tarea que gasta el tiempo esperando (le sirven los hilos)
Bytecode	El código intermedio que ejecuta CPython, protegido por el GIL
<code>multiprocessing</code>	Módulo para paralelismo real con procesos (cada uno con su GIL)

 [Volver al índice de la unidad](#) · Anterior: [05 · Timer](#) · Siguiente: [07 · Estados del hilo](#)

07 — Estados del hilo

El ciclo de vida de un hilo, de nuevo a terminado 

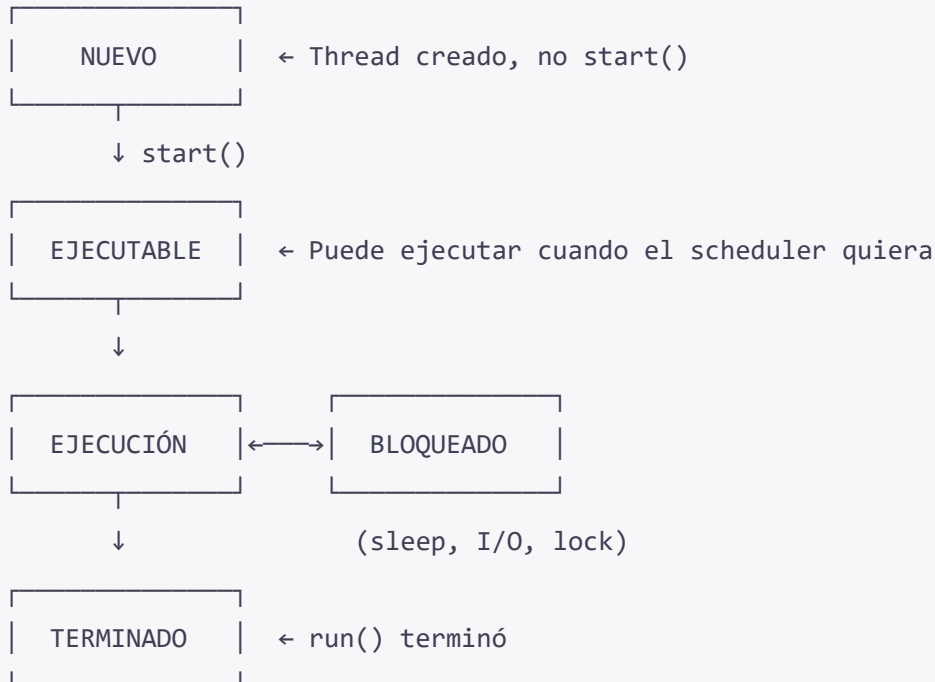
 Estás en:  U02 · Hilos Fundamentos → 07 · Estados del hilo

La idea en una frase

Un hilo nace como objeto, se lanza con `start()`, espera su turno, se ejecuta, se bloquea cuando espera algo y termina cuando su función acaba: ese es su ciclo de vida.

Cada hilo pasa por estados, igual que una persona pasa por situaciones a lo largo del día. Saber en qué estado está un hilo (o cuándo lo estará) es clave para entender por qué un programa se comporta como se comporta.

El diagrama del ciclo de vida



Los estados, uno a uno

Estado	Significado
NUEVO	El objeto Thread existe pero no se ha llamado a <code>start()</code>
EJECUTABLE	<code>start()</code> llamado. Puede ejecutar en cualquier momento
EJECUCIÓN	El scheduler le ha dado la CPU: está ejecutando su código ahora mismo
BLOQUEADO	Esperando (<code>sleep</code> , I/O, un lock)
TERMINADO	El método <code>run()</code> ha terminado

Recorrido completo de un hilo típico:

1. **NUEVO** — `threading.Thread(target=fn)` crea el objeto. Todavía no se ejecuta nada.
2. **EJECUTABLE** — `h.start()` lo lanza. A partir de aquí el scheduler del sistema operativo decide *cuándo* le toca.
3. **EJECUCIÓN** — Le toca: ejecuta su código. Puede volver a **EJECUTABLE** cuando el scheduler decide dar paso a otro hilo.
4. **BLOQUEADO** — El hilo se queda esperando: un `time.sleep()`, una lectura de red, una espera de I/O o (en el TEMA 03) un lock. Cuando lo que espera se libera, vuelve a **EJECUTABLE**.
5. **TERMINADO** — La función que ejecutaba el hilo acaba. Su `run()` terminó y no volverá a ejecutarse.



El estado BLOQUEADO, el más importante

En la práctica, el estado que más vemos es **BLOQUEADO**, y casi siempre por dos motivos:

- `time.sleep(n)` — el hilo pide “no me des CPU durante n segundos” (lo usaste en los [puntos 2 y 4](#)).
- **Espera de I/O** — descarga, lectura de archivo, esperar un socket (del TEMA 04 para allá).

Mientras un hilo está BLOQUEADO, **libera la CPU** (y, como viste en el [punto 6](#), también el GIL): otro hilo puede ejecutar. Es exactamente el mecanismo que hace rápidas las tareas I/O-bound.



`sleep(0)` es un caso curioso: cede la CPU **voluntariamente** sin esperar nada, solo para dar paso a otro hilo. Es una “buena práctica” en hilos cooperativos.

¿Cuándo termina un hilo?

Un hilo llega a **TERMINADO** cuando su función acaba (o si lanza una excepción no capturada). Y aquí entra el matiz del [punto 4](#):

- Un hilo **no daemon** que llega a **TERMINADO** es requisito para que el programa principal pueda salir.
- Un hilo **daemon** puede ser cortado en seco por el final del programa principal, aunque esté en **EJECUCIÓN** o **BLOQUEADO**: muere sin llegar “bien” a **TERMINADO**.

Puedes comprobar si un hilo sigue vivo en cada momento con `hilo.is_alive()`:

```
import threading, time

def corto():
    time.sleep(1)

h = threading.Thread(target=corto)
print(f"Antes de start: {h.is_alive()}")  # False (estado NUEVO)
h.start()
print(f"Justo tras start: {h.is_alive()}") # True (EJECUTABLE/EJECUCIÓN)
h.join()
print(f"Tras join: {h.is_alive()}")      # False (TERMINADO)
```

Salida:

```
Antes de start: False
Justo tras start: True
Tras join: False
```

Mini-chequeo

1. ¿En qué estado está un hilo justo después de crearlo pero antes de `start()`?
2. ¿Qué ocurre cuando un hilo en **EJECUCIÓN** llama a `time.sleep(2)`?
3. ¿Puede un hilo volver de **BLOQUEADO** a **EJECUCIÓN** sin pasar por **EJECUTABLE**?

>  **Respuestas**



Resumen en 3 frases

- El ciclo de vida de un hilo es **NUEVO** → **EJECUTABLE** → **EJECUCIÓN** ⇌ **BLOQUEADO** → **TERMINADO**.
- **BLOQUEADO** es el estado de las esperas (sleep, I/O, lock): al bloquearse, el hilo libera la CPU y el GIL.
- `is_alive()` te dice en vivo si el hilo todavía se está ejecutando o ya terminó.



Vocabulario rápido

Término	Idea general
NUEVO	Hilo creado, sin <code>start()</code> todavía
EJECUTABLE	<code>start()</code> llamado, esperando turno del scheduler
EJECUCIÓN	El hilo tiene la CPU y está ejecutando código
BLOQUEADO	Esperando: sleep, I/O o un lock
TERMINADO	La función del hilo ha acabado
Scheduler	Componente del SO que decide qué hilo ejecuta



[Volver al índice de la unidad](#) · Anterior: [06 · El GIL](#) · Siguiente: [08 · Hilos en la práctica](#)

08 — Hilos en la práctica

Be the code, el ring Hilo vs Proceso y Aprieta el lápiz 

 Estás en:  U02 · Hilos Fundamentos → 08 · Hilos en la práctica

La idea en una frase

Este punto es el taller de la unidad: seguimos la ejecución de varios hilos paso a paso, dirimimos la pelea Hilo vs Proceso y resolvemos los ejercicios de Aprieta el lápiz.

Ya sabes crear hilos, pasarles datos, convertirlos en daemons, temporizarlos y entender el GIL y los estados. Ahora toca juntarlo todo como lo harás en el examen: leyendo código multihilo y prediciendo su salida.

Be the code, my friend — El hilo viajero

“Sé el código y recorre su ejecución paso a paso. Dos hilos, dos viajeros.”

```
import threading, time

def viajero(nombre, paradas):
    for i in range(paradas):
        print(f"{nombre} 🚶 está en la parada {i+1}")
        time.sleep(0.5)
    print(f"{nombre} 🏠 ha llegado a su destino")

h1 = threading.Thread(target=viajero, args=("Ana", 3))
h2 = threading.Thread(target=viajero, args=("Bob", 2))

h1.start()
h2.start()
h1.join()
h2.join()
```

Traza (ejecución real):

```
▶ Ana (h1) empieza. Dice: "Ana 🚶 está en la parada 1"
▶ Bob (h2) empieza. Dice: "Bob 🚶 está en la parada 1"
⌚ Ana duerme 0.5s
⌚ Bob duerme 0.5s
▶ Bob se despierta. Dice: "Bob 🚶 está en la parada 2"
▶ Ana se despierta. Dice: "Ana 🚶 está en la parada 2"
⌚ Ana duerme 0.5s
⌚ Bob duerme 0.5s
▶ Bob se despierta. Dice: "Bob 🏁 ha llegado a su destino"
▶ Ana se despierta. Dice: "Ana 🚶 está en la parada 3"
⌚ Ana duerme 0.5s
▶ Ana se despierta. Dice: "Ana 🏁 ha llegado a su destino"
```

Fíjate en los detalles que te piden en los exámenes:

- Ambos hilos **arrancan a la vez**: Ana y Bob dicen su parada 1 casi al mismo tiempo.
- Mientras Ana duerme sus 0.5s (estado **BLOQUEADO** del [punto 7](#)), Bob aprovecha y avanza.
- Bob hace solo 2 paradas; Ana hace 3. **Bob llega antes** a su destino.
- Los `join()` del final garantizan que el mensaje del principal (si lo hubiera) esperaría a ambos.

Fíjate cómo los `print` se entremezclan. El orden exacto lo decide el scheduler del sistema operativo. **No hay garantía de orden.**



El ring de los conceptos — Hilo vs Proceso

Hilo: — Soy ligero, veloz y nacido para compartir. Nací en milisegundos y trabajo en la misma memoria que mis hermanos.

Proceso: — ¿Y dónde está tu privacidad? Yo tengo mi propia memoria aislada. Si tú cometes un error, te llevas por delante a todo el proceso. A mí nadie me toca.

Hilo: — Pero compartir es mi superpoder: me comunico con variables globales al instante, sin pipes ni sockets. ¿Cuánto tardas tú en enviar un dato a tu vecino?

Proceso: — Pipes, sockets, archivos... más lento, sí. Pero creo cien procesos y cada uno trabaja con su propio GIL a toda CPU. Tú y tus hermanos os peleáis por un solo GIL.

Hilo: — ¡El GIL es para calcular! Yo brillo esperando: descargas, lecturas de disco, clientes de red. Mientras uno espera, los demás avanzan.

Proceso: — Al final, cada uno a su oficio. Yo para aislamiento y CPU de verdad; tú para esperas y servicios ligeros.

Moraleja: Usa **procesos** cuando necesites aislamiento o paralelismo real de CPU (multiprocessing). Usa **hilos** cuando tu tarea es de espera (I/O) o quieres algo ligero y que comparta memoria. La tabla completa la tienes en el [punto 1](#).



Aprieta el lápiz

Resuelve estos seis ejercicios con lo aprendido en la unidad. Cuando los tengas (o te rindas), despliega las soluciones.

1. **Carrera de mensajes:** Crea 5 hilos que impriman su nombre 10 veces. Observa cómo se entremezclan.
2. **Temporizador daemon:** Crea un hilo daemon que imprima “tic” cada segundo. El programa principal espera 5 segundos y termina.
3. **Timer despertador:** Crea un Timer que imprima “¡Despierta!” a los 3 segundos.
4. **GIL test:** Compara el tiempo de 4 hilos vs 1 hilo haciendo una tarea CPU-bound (contar hasta 50M). Verifica que tardan lo mismo.
5. **I/O vs CPU:** Crea dos versiones de descarga simulada (sleep 2s) — una con 1 hilo y otra con 4. Mide la diferencia.
6. **Pool Puzzle:** Ordena las líneas para crear un programa que lance 2 hilos que saluden 3 veces cada uno.

>  Soluciones



Mini-chequeo

1. En la traza del viajero, ¿por qué Bob llega antes que Ana a su destino?
2. ¿Qué garantizan los `join()` del final del programa de los viajeros?
3. En el Pool Puzzle, ¿por qué el `import threading` debe ir al principio del orden?

>  Respuestas



Resumen en 3 frases

- La traza de varios hilos se **entremezcla** y su orden lo decide el scheduler: no hay garantía de orden, solo de que con `join()` se esperan.
- En el ring, **hilos** para tareas ligeras y de espera que comparten memoria; **procesos** para aislamiento y CPU de verdad.
- El Pool Puzzle y los ejercicios son exactamente el formato de examen: ordenar líneas y predecir salidas.



Vocabulario rápido

Término	Idea general
Traza	Recorrido paso a paso de la ejecución del programa
Intercalado	Alternancia de mensajes de varios hilos sin orden fijo
Scheduler	Decide qué hilo ejecuta en cada momento
Pool Puzzle	Ejercicio de ordenar líneas de código desordenadas
<code>join()</code>	Espera a que el hilo termine antes de continuar



[Volver al índice de la unidad](#) · Anterior: [07 · Estados del hilo](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé el hilo, el ring final y el laboratorio de la unidad 🧠

📖 Estás en: 🏠 U02 · Hilos Fundamentos → 09 · Cierre

Has terminado la teoría: qué es un hilo, cómo se crea y se espera, argumentos y nombres, daemons, Timer, el GIL y el ciclo de vida. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el hilo

Eres un hilo llamado “hilo-1”. Acabas de nacer en el programa de los viajeros del [punto 8](#). Tu misión: recorrer 3 paradas con 0.5s de sueño entre cada una.

¿Qué pasa, paso a paso?

1. **Naces en estado NUEVO:** `threading.Thread(target=viajero, args=("Ana", 3))`. Existes como objeto, pero no has hecho nada todavía.
2. **`h1.start()` te lanza:** pasas a **EJECUTABLE**. Ya puedes ejecutar cuando el scheduler quiera.
3. **El scheduler te da la CPU:** pasas a **EJECUCIÓN** e imprimes "Ana 🚶 está en la parada 1".
4. **`time.sleep(0.5)` te bloquea:** pasas a **BLOQUEADO**. Sueltas la CPU y el GIL; tu compañero Bob aprovecha y avanza.
5. **Te despiertas:** vuelves a **EJECUTABLE**, el scheduler te da CPU de nuevo y avanzas a la parada 2... y luego la 3.
6. **Tu función acaba:** pasas a **TERMINADO**. El `h1.join()` del programa principal confirma que ya no sigues vivo.

En ningún momento decides tú cuándo te toca. El scheduler del sistema operativo reparte la CPU entre ti y Bob como quiere; tu único control es lanzarte (`start()`) y hacer esperar al principal (`join()`).

💡 **Ahora tú:** ¿y si fueras el reloj daemon del [punto 4](#)? Tu función es un `while True`: nunca llegas “bien” a **TERMINADO**. El día que el programa principal acabe, te matan en seco, estés en **EJECUCIÓN** o **BLOQUEADO**. Ese es el destino del daemon.

Fireside Chat: Hilo vs Proceso

Dos trabajadores de la multitarea se sientan junto a la chimenea a zanjar, de una vez, quién hace qué.

Hilo: — Soy la unidad más pequeña de ejecución. Nací dentro de un proceso y comparto su memoria con mis hermanos. Me crean en milisegundos.

Proceso: — Yo soy el dueño de la casa. Memoria aislada, PID propio, espacio de direcciones separado. Si un hilo se equivoca, puede tirar la casa entera.

Hilo: — Pero comunicarme es instantáneo: una variable global y listo. Tú necesitas pipes, sockets o archivos para hablar con tu vecino.

Proceso: — Es cierto, más lento. Pero cuando toca calcular de verdad, cada proceso tiene su propio GIL. Yo puedo usar 8 CPUs de golpe; tú te peleas con tus hermanos por una.

Hilo: — ¡El GIL es para calcular! Yo brillo esperando: descargas, lectura de disco, clientes de red. Mientras uno espera, los demás avanzan. Ahí no me gana nadie.

Proceso: — Al final, cada uno a su oficio. Yo para aislamiento y CPU real; tú para esperas, servicios de fondo y todo lo ligero.

Moraleja: los hilos comparten memoria y son baratos (perfectos para I/O y servicios ligeros); los procesos aíslan y paralelizan CPU de verdad. Elige según la tarea.



¿Quién soy?

1. Soy la unidad más pequeña de ejecución y comparto memoria con mis hermanos dentro de un proceso.
2. Me llaman “main” y, sin un `join()`, puedo terminar el programa antes que los demás hilos.
3. Me ejecuto en segundo plano y muero cuando el programa principal termina, sin dramas.
4. Soy un candado de CPython que impide que dos hilos ejecuten bytecode a la vez.
5. Me ejecutan una sola vez tras un retardo, y pueden cancelarme antes de que dispare.
6. Soy el estado en el que un hilo espera dormido, leyendo de red o esperando un lock.

>  Respuestas



CONRAD VS EL MUNDO: “el programa termina antes que mis hilos”

CONRAD: — “Clásico: mi programa imprime ‘Fin’ y los hilos ni se han enterado. Razones: 1) **No puse `join()`**: el principal siguió a lo suyo y se fue antes de que terminaran. 2) **Llamé a la función con paréntesis** en `target=fn()`: se ejecutó al instante en el hilo principal y el hilo nació ya terminado. 3) **Hice el hilo `daemon`** y el principal salió: lo maté yo mismo. 4) **Esperaba que `start()` ejecutara al momento**, cuando en realidad solo lo pone en EJECUTABLE a merced del scheduler.”

CONRAD: — “Y lo mejor: *‘pero si lo he lanzado’*. ¡Pues claro! **Lanzar no es esperar**. `start()` pone el hilo en marcha; `join()` es lo único que hace que el programa principal se quede esperándolo. Sin `join()`, tu ‘Fin’ puede salir antes que todo el trabajo.”

CONRAD: — “Y no me vengas con *‘¿será que el GIL lo bloquea?’*. Si tus hilos son de espera (I/O), el GIL no es el problema: el problema es que **no los estás esperando tú**. A diagnosticar: ¿hay `join()`? ¿la función va sin paréntesis? ¿el hilo es `daemon` sin quererlo?”



Laboratorio de tortura: la sala de descargas

Duración: 45 minutos **Herramienta:** Python 3 (threading y time, sin instalar nada)

Escenario: estás montando el mini-descargador del instituto. Cuatro archivos de 2 segundos cada uno (simulados con `time.sleep(2)`) deben descargarse “en paralelo” con 4 hilos, y el programa debe cronometrar cuánto tarda el lote entero.

Tareas paso a paso:

1. Define `descargar(archivo)`: imprime  `descargando {archivo}`, hace `time.sleep(2)` y al final imprime  `{archivo} listo`.
2. Crea 4 hilos con `threading.Thread(target=descargar, args=(nombre,))`, uno por archivo, con `.name = "hilo-1"... "hilo-4"`.
3. Lanza todos con `for h in hilos: h.start()`.
4. Espera a todos con `for h in hilos: h.join()`.
5. Cronometra el lote con `time.time()` antes y después: con 4 hilos debe rondar los **2 segundos**, no los 8.

Fallo intencionado: en el paso 2, en lugar de `target=descargar`, escribe `target=descargar("foto.png")` — **con paréntesis y argumento**. ¿Qué pasa? La función se ejecuta en el momento de crear el hilo, en el hilo principal y **en serie**: los 4 archivos tardan 8 segundos, las descargas aparecen “ya terminadas” antes de lanzar nada, y los hilos nacen haciendo nada.

Pista 1: `target` recibe una **referencia** a la función, no una llamada. `descargar` sin paréntesis le pasa la función; `descargar("foto.png")` la ejecuta ahí mismo (lo avisamos en el [punto 2](#)). Si tus descargas imprimen antes del primer `start()`, ese es el bug.

Pista 2: si el lote tarda ~8 segundos en vez de ~2, no hay paralelismo: los `time.sleep(2)` se están sumando porque todo se ejecuta en el hilo principal. Un `print(f"Creando {archivo}")` dentro del bucle te delata: los 4 mensajes aparecen juntos y seguidos, sin intercalarse con los .



Logros de esta unidad

Logro	Cómo conseguirlo
 Primer Hilo	Crear, lanzar y esperar un hilo con <code>Thread</code> , <code>start()</code> y <code>join()</code>
 Hilo con Nombre	Pasar <code>args/kwargs</code> y nombrar N hilos con <code>.name = "hilo-" + str(n)</code>
 Daemon Tamer	Explicar y construir un hilo daemon de fondo que muere al salir
 Timer Whisperer	Programar un aviso con <code>threading.Timer</code> y cancelarlo si hace falta
 GIL Master	Explicar el GIL y distinguir tareas CPU-bound de I/O-bound



Atrévete a pensar

1. ¿Por qué los hilos comparten memoria y los procesos no?
2. ¿Qué le pasa a tu programa si un hilo **no daemon** entra en un bucle infinito?
3. ¿Cuándo conviene usar `multiprocessing` en lugar de hilos?
4. ¿Por qué la salida de varios hilos nunca tiene orden garantizado?
5. ¿Qué diferencia hay entre un hilo en `EJECUTABLE` y uno en `EJECUCIÓN`?

🧩 Crucigrama de bits

Horizontal:

1. Unidad más pequeña de ejecución dentro de un proceso (5 letras)
3. Método que lanza un hilo (5 letras)
5. Método que hace esperar al programa principal (4 letras)
6. Hilo de fondo que muere al salir el programa (6 letras)
8. Candado de CPython que limita a los hilos (3 letras)

Vertical:

2. Clase de Python con la que se crea un hilo (6 letras)
4. Estado del hilo que espera dormido o por I/O (9 letras)
7. Ejecuta una función una sola vez tras un retardo (5 letras)

💬 Entrevista de trabajo

1. “¿Qué es un hilo y en qué se diferencia de un proceso?”
2. “¿Cómo creas y lanzas varios hilos en Python con argumentos y nombres?”
3. “¿Qué es un hilo daemon? ¿Cuándo lo usarías?”
4. “¿Qué es el GIL y cómo afecta al rendimiento de tus hilos?”
5. “¿Cómo sabes si un hilo ha terminado y cómo esperas a varios hilos a la vez?”

💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, compara memoria, coste de creación, comunicación y aislamiento (la tabla del [punto 1](#)) y remata con la moraleja del [ring](#): hilos para I/O y servicios ligeros, procesos para aislamiento y CPU. Para la 4, recorre el [punto 6](#): qué es el GIL, por qué los hilos no aceleran CPU-bound, por qué sí I/O-bound, y cuándo toca multiprocessing. Si sabes contarlo fluido, ya eres medio programador concurrente.

🙋 No hay preguntas tontas

? ¿Un hilo puede crear otro hilo?

Sí. Un hilo puede lanzar otros hilos sin problema.

? ¿Cuántos hilos puedo crear?

Hay límite práctico. En Windows, unos pocos miles. Cada hilo consume ~1MB de memoria virtual por su pila.

? ¿Puedo matar un hilo desde fuera?

No limpiamente. No hay `hilo.kill()`. La forma correcta es usar una variable bandera que el hilo compruebe periódicamente.

? ¿Qué pasa si no llamo a `join()`?

El hilo se ejecuta igual. Pero el programa principal no espera. Si es no-daemon, el programa no terminará hasta que el hilo termine.

? ¿`sleep(0)` sirve para algo?

Sí, cede la CPU voluntariamente para que otro hilo pueda ejecutar. Es una “buena práctica” en hilos cooperativos.

? ¿Los hilos tienen prioridad?

En Python, no hay prioridades nativas. El scheduler del SO decide. Puedes simular prioridades con lógica condicional, pero no es real.

Poscréditos

El hilo de Ana recorre sus tres paradas. El de Bob, más ligero, llega antes a su destino.

El reloj daemon imprime la hora en segundo plano hasta que el programa decide apagarse.

El GIL vigila que ningún hilo pise a otro mientras CPython respira.

Los hilos comparten memoria, y compartir sin ponerse de acuerdo es peligroso.

PRÓXIMAMENTE EN UO3: *Dos hilos tocando la misma variable a la vez es una carrera. Si no se ponen de acuerdo, los datos se corrompen. Necesitan locks, semáforos y barreras: sincronización entre hilos.*

✓ Criterios de evaluación cubiertos (RA2)

RA2 — Hilos (parcial): estructura, creación, esperas, daemon y GIL.

CE	Criterio	Cubierto
RA2a	Identifica la estructura de un hilo	✓ Punto 1 + Punto 7
RA2b	Crea y lanza hilos con threading	✓ Puntos 2-3 + ⚡ Laboratorio de tortura
RA2e	Implementa esperas con join() y sleep()	✓ Puntos 2-4 + ⚡ Laboratorio de tortura
RA2f	Gestiona hilos daemon	✓ Punto 4 + ⚡ Laboratorio de tortura
RA2h	Conoce el GIL y sus limitaciones	✓ Punto 6 + Punto 8

Los criterios RA2c (Lock), RA2d (semáforos) y RA2g (condiciones de carrera) se cubren en el **TEMA 03 — Sincronización entre Hilos**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Hilos en la práctica](#) · Siguiente: [U03 · Sincronización entre Hilos](#)

U03 — Sincronización entre Hilos

Cuando los hilos se pisan: locks, semáforos y barreras 🔒

📖 Ruta del viaje: 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización entre Hilos → 🌐 TCP → 📡 UDP → 🌐 API REST → 📡 APIs comerciales → 🔒 Hash → 🧬 Cifrado → ⏳ Servidores → ⌚ asyncio

En la U02 lanzaste hilos por todas partes y comprobaste que los hilos de un mismo proceso **comparten memoria**. Eso es fantástico... y peligroso: si dos hilos tocan la misma variable a la vez, los resultados se descontrolan. Esta unidad pone orden en el caos: aprenderás qué es una **condición de carrera**, por qué `contador += 1` esconde una trampa de 3 pasos, y cómo los **locks**, **semáforos**, **barreras** y **condiciones** del módulo `threading` convierten el desorden en coordinación.

Verás, además, el patrón de **productor-consumidor** resuelto con `Condition`, cómo evitar los temidos **deadlocks** con las reglas del oficio, y un cierre práctico con laboratorio. Los boletines de la unidad te permitirán medir cuánto has aprendido.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una **condición de carrera** y por qué `contador += 1` no es una operación atómica.
- Proteger secciones críticas con `threading.Lock` usando `with lock:`.
- Distinguir `Lock` de `RLock` y saber cuándo necesitas el reentrante.
- Limitar el acceso concurrente a un recurso con `Semaphore`.
- Coordinar fases de trabajo en paralelo con `Barrier`.
- Sincronizar hilos con `Condition` (`wait` / `notify` / `notify_all`).
- Implementar el patrón **productor-consumidor** con una cola compartida.
- Evitar **deadlocks** con buenas prácticas (orden de locks, `with`, `RLock`).
- Decidir qué mecanismo de sincronización usar en cada situación.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Condición de carrera	Qué pasa cuando dos hilos se pisan la memoria compartida	Todos
02 · Lock	Exclusión mutua: acquire, release y with lock:	Todos
03 · RLock	El lock reentrante para cuando el mismo hilo entra dos veces	Todos
04 · Semaphore	El aforo máximo: hasta N hilos dentro a la vez	Todos
05 · Barrier	Nadie avanza hasta que llegan todos: fases sincronizadas	Todos
06 · Condition	Esperar a que otro hilo avise: wait y notify	Todos
07 · Productor-Consumidor	El patrón clásico con cola compartida, de la teoría al código	Todos
08 · Buenas prácticas	Deadlocks, orden de locks, el ring final y a practicar	Todos
09 · Cierre	Sé el lock, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

 Inicial resuelto

 Inicial por resolver

 Avanzado resuelto

 Avanzado por resolver

Criterios de evaluación cubiertos (RA2)

RA2: Gestiona la programación de hilos y su sincronización.

CE	Criterio	Dónde se cubre
RA2c	Sincroniza hilos con Lock	✅ Puntos 2, 3 y 7 + ⚡ Laboratorio (punto 9)
RA2d	Usa semáforos para acceso controlado	✅ Punto 4 + ⚡ Laboratorio (punto 9)
RA2g	Evita condiciones de carrera	✅ Puntos 1, 2 y 8 + ⚡ Laboratorio (punto 9)

RA2a (estructura de un hilo), RA2b (crear y lanzar hilos), RA2e (esperas con `join()` y `sleep()`), RA2f (hilos daemon) y RA2h (GIL) se cubren en la **U02 · Hilos Fundamentos**.

¿Por dónde empiezo?

¿Vienes de la U02 y ya sabes lanzar hilos y esperarlos con `join()`? Perfecto, ese es el trampolín ideal: repasa la [U02 · Hilos Fundamentos](#) para tener frescos `Thread`, `start()`, `join()` y el concepto de hilo daemon, y arranca en el [punto 1](#), que parte justo de ahí: el momento en que dos hilos comparten memoria y todo se descontrola.

¿Ya sabes qué es un lock y solo necesitas el semáforo o la barrera? Puedes saltar a los [puntos 4, 5 y 6](#). Pero si vienes de cero en sincronización, no te saltes los puntos 1 a 3: la condición de carrera y el lock son la base de todo lo demás.

📌 Primer punto: [01 · Condición de carrera](#)

▶▶ Al acabar la unidad, continúa en [U04 · Sockets TCP](#).

01 — Condición de carrera

Cuando dos hilos se pisan la memoria compartida 🏃 ⚡

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 01 · Condición de carrera

📌 La idea en una frase

Una **condición de carrera** ocurre cuando dos o más hilos acceden a la misma variable compartida a la vez, y el resultado final depende de qué hilo llegue primero: los incrementos se pierden y el resultado es incorrecto.

Dos hilos que incrementan una variable compartida sin sincronización:

```
import threading

contador = 0

def incrementar():
    global contador
    for _ in range(100_000):
        contador += 1 # ⚠️ Esto NO es atómico

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Esperado: 400.000 | Obtenido: {contador}")
# → 287.341, 312.045, 198.723... ¡nunca 400.000!
```

Lanzamos 4 hilos, cada uno suma 100.000 veces. El resultado esperado es 400.000... y jamás lo vemos. Cada ejecución da un número distinto, siempre menor. Esa es la cara visible de la condición de carrera.

🔍 ¿Por qué falla? El += no es atómico

`contador += 1` parece una línea, pero por dentro hace **3 operaciones**:

1. Leer contador de memoria
2. Sumar 1
3. Escribir el resultado

Si dos hilos leen el mismo valor antes de que ninguno haya escrito, ambos escriben el mismo resultado y se pierde un incremento:

```
Hilo-A: lee contador = 0
Hilo-B: lee contador = 0      ← ¡mismo valor!
Hilo-A: escribe contador = 1
Hilo-B: escribe contador = 1  ← ¡pisó el incremento de A!
Hilo-A: lee contador = 1
Hilo-A: escribe contador = 2
Hilo-B: lee contador = 1      ← ¡leyó valor viejo!
```

Entre la “lectura” del hilo A y su “escritura”, el hilo B cuela otra lectura y otra escritura. Al final, dos incrementos solo han subido el contador en 1.

⚠ En Python, el **GIL** evita que dos hilos ejecuten Python puro a la vez, pero **no** protege esta secuencia: el planificador puede interrumpir a un hilo entre el paso 1 y el paso 3 (en operaciones con `sys.settrace`, `sleep`, o cuando la operación se interrumpe). La condición de carrera es real.



La analogía: dos cajeros y una caja

Imagina una tienda con **una sola caja** y **dos cajeros**. La caja solo puede guardar el dinero que hay en ella, y los dos cajeros la comparten:

- El cajero A mira la caja: hay **100 €** (leer).
- Mientras A va a buscar cambio, el cajero B también mira la caja: **100 €** (leer).
- A vuelve y mete 50 €: escribe **150 €**.
- B vuelve y mete 50 €: también escribe **150 €**, ¡pisando el incremento de A!

Se han cobrado 100 € en total, pero en la caja solo hay 150 € en lugar de 200 €. **Los 50 € del cajero A han desaparecido** en la caja compartida: se perdieron por una condición de carrera. Exactamente lo mismo le pasa a contador.

El remedio es la **exclusión mutua**: mientras un cajero toca la caja, el otro espera fuera. Ese es el Lock, que verás en el [punto 2](#).



Dónde aparece la condición de carrera

Situación	Qué se pierde
Contador compartido (contador += 1)	Incrementos
Saldo de una cuenta bancaria	Ingresos / retiros
Cola de trabajos compartida	Elementos de la cola
Registro (log) con fecha y hora	Líneas de log
Caché con varios lectores/escritores	Datos actualizados

Es el bug más clásico de la programación concurrente: el programa **funciona casi siempre** y falla de forma intermitente e imposible de reproducir. Por eso la sincronización se diseña desde el principio, no al final.



Mini-chequeo

1. ¿Qué es una condición de carrera?
2. ¿Por qué contador += 1 no es atómico y qué tres pasos esconde?
3. ¿Cómo explicarías el problema con la analogía de los dos cajeros y la caja?

>  Respuestas



Resumen en 3 frases

- Dos hilos que tocan la misma variable a la vez producen resultados incorrectos: eso es una **condición de carrera**.
- contador += 1 no es atómico: son **leer, sumar, escribir**, y el planificador puede interrumpir entre pasos.
- La solución pasa por la **sincronización** (exclusión mutua), el tema del resto de la unidad.



Vocabulario rápido

Término	Idea general
Condición de carrera	Dos hilos tocan la misma variable a la vez y se pisan
Sección crítica	Zona de código que toca un recurso compartido
Operación atómica	Operación que no se puede interrumpir a mitad
Exclusión mutua	Solo un hilo accede al recurso compartido a la vez
GIL	Bloqueo global de Python: no protege la secuencia leer-sum-escribir

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · Lock](#)

02 — Lock

El cerrojo que protege la sección crítica 🔒

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 02 · Lock

📌 La idea en una frase

Un **Lock** (cerrojo) garantiza que solo un hilo entre en la **sección crítica** a la vez: el resto espera fuera hasta que el primero lo libera. Es la exclusión mutua que faltaba en el [punto 1](#).

En el ejemplo del contador compartido, la sección crítica es `contador += 1`. Protegerla con un Lock hace que los 4 hilos se turnen: nadie puede leer mientras otro está escribiendo, y el resultado final vuelve a ser el esperado.

```
import threading

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(100_000):
        with lock:          # 🗝 Solo un hilo aquí dentro
            contador += 1

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Con Lock: {contador}") # ✅ 400.000
```

Ahora el resultado es siempre **400.000**, en cualquier ejecución. El Lock se adquiere antes de tocar el contador y se libera al salir del bloque.

🗝 Métodos de Lock

Método	Qué hace
<code>lock.acquire()</code>	Bloquea. Si otro hilo lo tiene, espera
<code>lock.release()</code>	Libera. Otro hilo puede entrar
<code>lock.locked()</code>	Devuelve True si está bloqueado

`acquire()` devuelve True si consiguió el lock. Si otro hilo lo tiene, el hilo que llama se **bloquea** (espera) hasta que se libere. `locked()` es útil para inspeccionar el estado sin bloquearse.

Siempre with lock:

```
lock = threading.Lock()

# ✅ La forma segura
with lock:
    contador += 1

# ⚠️ La forma peligrosa
lock.acquire()
contador += 1
# lock.release() ← ¡si lo olvidas, DEADLOCK!
```

⚠️ **Siempre usa with lock:** Si usas `acquire()` manual y olvidas `release()`, el hilo se queda con el cerrojo puesto para siempre y los demás esperan eternamente: un **deadlock**. El `with` libera el lock solo, incluso si el código dentro lanza una excepción.

Be the code: contador con y sin Lock

Sin Lock — el caos:

```

Hilo-A: lee contador = 0
Hilo-B: lee contador = 0      ← ¡mismo valor!
Hilo-A: escribe contador = 1
Hilo-B: escribe contador = 1  ← ¡pisó el incremento de A!
Hilo-A: lee contador = 1
Hilo-A: escribe contador = 2
Hilo-B: lee contador = 1      ← ¡leyó valor viejo!

```

Con Lock — el orden:

```

Hilo-A: acquire() → DENTRO
Hilo-A: lee contador = 0
Hilo-A: escribe contador = 1
Hilo-A: release() → FUERA

Hilo-B: acquire() → DENTRO      ← esperó hasta que A soltó
Hilo-B: lee contador = 1        ← valor correcto
Hilo-B: escribe contador = 2
Hilo-B: release() → FUERA

Hilo-C: acquire() → DENTRO
Hilo-C: lee contador = 2        ← valor correcto
...

```

Con el Lock, el “leer → sumar → escribir” de cada hilo ocurre **de principio a fin sin que nadie se cuele** entre medias. Los cajeros de la analogía del [punto 1](#) ya no se pisan: mientras uno toca la caja, el otro espera fuera.

💡 **Regla de oro:** el Lock protege la **sección crítica**, no el hilo. Todo el código que toque la variable compartida debe ir dentro del mismo lock.

Mini-chequeo

1. ¿Qué garantiza un Lock?
2. ¿Qué pasa si dos hilos llaman a `acquire()` a la vez?
3. ¿Por qué `with lock:` es mejor que `acquire()/release()` manuales?

>  **Respuestas**



Resumen en 3 frases

- El Lock es el cerrojo que protege la sección crítica: solo un hilo a la vez.
- Con `with lock`: nunca se olvida liberarlo; con `acquire()/release()` manuales, un `release()` olvidado = deadlock.
- Protegido con Lock, el contador del [punto 1](#) pasa de resultados aleatorios a un **400.000** exacto.



Vocabulario rápido

Término	Idea general
Lock	Cerrojo: exclusión mutua sobre una sección crítica
<code>acquire()</code>	Bloquear el lock (esperar si está ocupado)
<code>release()</code>	Liberar el lock para que otro entre
<code>with lock</code> :	Contexto que adquiere y libera el lock automáticamente
Sección crítica	Código que toca un recurso compartido

 [Volver al índice de la unidad](#) · Anterior: [01 · Condición de carrera](#) · Siguiente: [03 · RLock](#)

03 — RLock

Cuando el mismo hilo quiere entrar dos veces 

 Estás en:  U03 · Sincronización entre Hilos → 03 · RLock

La idea en una frase

Un **Lock normal no es reentrante**: si el mismo hilo intenta adquirirlo dos veces, se espera a sí mismo y se produce un **deadlock**. Un **RLock** (Reentrant Lock) sí permite que el mismo hilo lo adquiera varias veces antes de liberarlo.

Recuerda el [punto 2](#): `acquire()` bloquea “si otro hilo lo tiene”. El problema es que el Lock no distingue entre “otro hilo” y “yo mismo”.

```
import threading

# Lock normal – DEADLOCK si el mismo hilo intenta adquirirlo dos veces
lock = threading.Lock()
lock.acquire()
lock.acquire() # ⚠ El hilo se espera a sí mismo → DEADLOCK

# RLock – el mismo hilo puede adquirirlo varias veces
rlock = threading.RLock()
rlock.acquire() # ok
rlock.acquire() # ok (mismo hilo)
rlock.release()
rlock.release()
```

Con el Lock normal, la segunda `acquire()` nunca vuelve: el hilo espera a que se libere un lock que él mismo tiene puesto. **El hilo se espera a sí mismo**. Con RLock, cada `acquire()` incrementa un contador interno y cada `release()` lo decrementa: el lock solo se libera de verdad cuando el contador llega a 0.

¿Cuándo lo necesitas? Funciones que se llaman entre sí

El caso más típico: una función que adquiere el lock y llama a **otra función que también quiere el mismo lock**.

```
import threading

rlock = threading.RLock()

def depositar(cuenta, cantidad):
    with rlock:
        # Aquí se llama a otra función que también usa el lock
        registrar_operacion("ingreso", cantidad)
        cuenta[0] += cantidad

def registrar_operacion(tipo, cantidad):
    with rlock:          # ✅ mismo hilo, RLock lo permite
        print(f"📄 {tipo}: {cantidad} €")

cuenta = [100]
depositar(cuenta, 50)
print(f"Saldo: {cuenta[0]} €")
```

Si registrar_operacion usara un Lock normal, depositar tendría el lock puesto y la segunda acquire() se bloquearía para siempre. Con RLock, las dos adquisiciones son del **mismo hilo** y funcionan.

💡 RLock también es tu aliado si la función se llama a sí misma (recursión) y cada llamada toca la sección crítica.

Lock vs RLock

Aspecto	Lock	RLock
¿Otro hilo puede adquirirlo?	Sí, cuando se libera	Sí, cuando se libera
¿El mismo hilo puede adquirirlo dos veces?	❌ Deadlock	✅ Sí (contador interno)

Aspecto	Lock	RLock
¿Cuándo se libera de verdad?	En el primer <code>release()</code>	Cuando los <code>release()</code> igualan a los <code>acquire()</code>
Uso típico	Sección crítica simple	Funciones que se llaman entre sí / recursión
Coste	Mínimo	Mínimo (un poco más de contabilidad)

⚠ **Regla:** usa Lock por defecto. Solo cambia a RLock cuando de verdad el mismo hilo necesite adquirirlo más de una vez (funciones que se llaman entre sí). Un RLock no evita que dos hilos distintos se pisen: para eso siguen valiendo las mismas reglas del [punto 2](#).

Mini-chequeo

1. ¿Qué le pasa a un Lock normal si el mismo hilo intenta adquirirlo dos veces?
2. ¿Cómo evita el RLock ese problema?
3. ¿Cuándo cambiarías de Lock a RLock?

>  Respuestas

Resumen en 3 frases

- El Lock normal no es reentrante: el mismo hilo que lo adquiere dos veces se bloquea a sí mismo (deadlock).
- El RLock lleva un contador interno y permite que el mismo hilo lo adquiriera varias veces, liberándose cuando el contador vuelve a 0.
- Úsalo en funciones que se llaman entre sí o en recursión; para el resto, Lock por defecto.

Vocabulario rápido

Término	Idea general
RLock	Lock reentrante: el mismo hilo puede adquirirlo varias veces

Término	Idea general
Reentrante	Capaz de volver a adquirirse por quien ya lo tiene
Contador interno	No de acquire() sin release(); a 0 se libera
Deadlock	Hilo que espera un recurso que él mismo tiene
Recursión	Función que se llama a sí misma



[Volver al índice de la unidad](#) · Anterior: [02 · Lock](#) · Siguiente: [04 · Semaphore](#)

04 — Semaphore

El aforo máximo de la sección crítica 🚫

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 04 · Semaphore

📌 La idea en una frase

Un **Semaphore** permite que hasta **N hilos** accedan al recurso a la vez: es como el aforo máximo de un local. Cuando se libera un puesto, entra el siguiente.

El Lock del [punto 2](#) deja pasar a **uno** a la vez. El Semaphore(2) deja pasar a **dos** a la vez; Semaphore(3), a **tres...** y así hasta el número que le pases al constructor.

```
import threading, time

semaforo = threading.Semaphore(2) # Máximo 2 hilos dentro

def entrar(id):
    print(f"Hilo-{id} esperando...")
    with semaforo:
        print(f" → Hilo-{id} DENTRO")
        time.sleep(2)
    print(f" ← Hilo-{id} SALE")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (salen de 2 en 2):

```
Hilo-0 esperando...
Hilo-1 esperando...
Hilo-2 esperando...
Hilo-3 esperando...
Hilo-4 esperando...
  → Hilo-0 DENTRO
  → Hilo-1 DENTRO
  ← Hilo-0 SALE
  ← Hilo-1 SALE
  → Hilo-2 DENTRO
  → Hilo-3 DENTRO
  ← Hilo-2 SALE
  ← Hilo-3 SALE
  → Hilo-4 DENTRO
  ← Hilo-4 SALE
```

Los 5 hilos esperan, pero dentro solo hay **2 a la vez**. Cuando uno sale, entra el siguiente. El semáforo hace de portero con un contador interno: `acquire()` lo baja, `release()` lo sube.



Semáforo con timeout

A veces no quieres esperar eternamente: si el recurso no se libera en X segundos, el hilo se rinde. `acquire(blocking=True, timeout=N)` devuelve `True` si consiguió el recurso y `False` si pasaron los N segundos.

```
import threading, time

semaforo = threading.Semaphore(1) # Solo 1 dentro

def entrar(id):
    if semaforo.acquire(timeout=1):
        print(f"Hilo-{id} entró")
        time.sleep(2)
        semaforo.release()
    else:
        print(f"Hilo-{id} TIMEOUT: se va")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (aproximada):

```
Hilo-0 entró
Hilo-1 TIMEOUT: se va
Hilo-2 TIMEOUT: se va
```

El hilo 0 ocupa el único puesto; los otros esperan 1 segundo y, al no entrar, **abandonan** en lugar de bloquearse para siempre. Ese True/False de `acquire(timeout=...)` te deja decidir si el hilo entra o se rinde.



¿Para qué sirve un semáforo?

Situación	¿Por qué un semáforo?
Limitar conexiones a una base de datos	No saturar el servidor de BD
Descargas simultáneas	Máximo N descargas a la vez
Acceso a una API con rate limit	No superar las peticiones permitidas
Impresoras compartidas	Solo las N impresoras disponibles
Aforo de un recurso físico	Como el aforo máximo de un local

💡 **Diferencia con Lock:** el Lock es Semaphore(1). Para **exclusión mutua** usa Lock (más simple y rápido); para **limitar acceso concurrente** a N recursos, Semaphore.

Mini-chequeo

1. ¿Cuántos hilos dejan entrar Semaphore(3) a la vez?
2. ¿Qué devuelve `acquire(timeout=2)` cuando el recurso se libera y cuando no?
3. ¿Cuándo elegirías Semaphore en lugar de Lock?

>  Respuestas

Resumen en 3 frases

- El Semaphore(N) limita a **N hilos** a la vez dentro del recurso: contador interno que baja al entrar y sube al salir.
- Con `acquire(timeout=N)` los hilos esperan un máximo de N segundos y se rinden si no entran (devuelve `True/False`).
- Es el mecanismo del **aforo**: conexiones a BD, descargas simultáneas o APIs con rate limit.

Vocabulario rápido

Término	Idea general
Semaphore	Aforo máximo: hasta N hilos dentro a la vez
Contador interno	No de puestos libres del recurso
<code>acquire()</code>	Reservar un puesto (baja el contador)
<code>release()</code>	Liberar un puesto (sube el contador)
Timeout	Esperar máximo N segundos y rendirse

 [Volver al índice de la unidad](#) · Anterior: [03 · RLock](#) · Siguiente: [05 · Barrier](#)

05 — Barrier

Ningún hilo avanza hasta que llegan todos 🏁

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 05 · Barrier

📌 La idea en una frase

Una **Barrier** obliga a todos los hilos a esperar hasta que el último llegue. Entonces todos continúan **a la vez**: perfecta para sincronizar **fases** de un trabajo en paralelo.

El Lock y el Semaphore controlan **cuántos** pasan. La Barrier controla **cuándo**: nadie cruza la meta hasta que el grupo completo está listo.

```
import threading, time

barrera = threading.Barrier(3) # 3 hilos deben llegar

def corredor(id):
    print(f"Corredor-{id} preparándose...")
    time.sleep(id * 0.5) # Cada uno tarda distinto
    print(f" → Corredor-{id} en la salida")
    barrera.wait() # Espera a los demás
    print(f" 🏁 Corredor-{id} SALIÓ!")

hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida:

```
Corredor-0 preparándose...
  → Corredor-0 en la salida
Corredor-1 preparándose...
  → Corredor-1 en la salida
Corredor-2 preparándose...
  → Corredor-2 en la salida
❧ Corredor-0 SALIÓ!
❧ Corredor-1 SALIÓ!
❧ Corredor-2 SALIÓ!
```

Fíjate en la salida: los 3 corredores tardan distinto en llegar a la salida (`time.sleep(id * 0.5)`), pero **los 3 cruzan a la vez** justo después de que el último llegue. `barrera.wait()` devuelve cuando los 3 hilos han llegado al `wait()`.

💡 El `Barrier(3)` espera exactamente 3 hilos. Si un hilo no llega (se bloquea o muere), los demás esperan para siempre: **el número de hilos debe coincidir** con el de la barrera.

Sincronizar fases de un trabajo

La aplicación más típica: un trabajo en paralelo con varias **fases**. Nadie empieza la fase 2 hasta que todos terminaron la fase 1.

FASE 1	FASE 2
Hilo-A: trabaja → wait() → trabaja → wait()	
Hilo-B: trabaja → wait() → trabaja → wait()	
Hilo-C: trabaja → wait() → trabaja → wait()	
┌────────┐	┌────────┐
todos acaban	todos acaban
a la vez y	a la vez y
pasan juntos	pasan juntos

Un ejemplo: descargar 3 archivos, y solo cuando los 3 estén descargados, empezar a procesarlos.

```
import threading, time

barrera = threading.Barrier(3)

def tarea(id):
    print(f" 📄 Descargando archivo-{id}...")
    time.sleep(1 + id)          # descargas de distinta duración
    barrera.wait()              # 🚧 espera a que todos terminen
    print(f" 📄 Procesando archivo-{id} (fase 2)")

hilos = [threading.Thread(target=tarea, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (extracto):

```
📄 Descargando archivo-0...
📄 Descargando archivo-1...
📄 Descargando archivo-2...
📄 Procesando archivo-0 (fase 2)
📄 Procesando archivo-1 (fase 2)
📄 Procesando archivo-2 (fase 2)
```

Ningún archivo se procesa hasta que los 3 están descargados. La barrera convierte el “cada uno a lo suyo” en “todos juntos en cada fase”.



Barrier frente al resto

Mecanismo	Pregunta que responde
Lock	¿Quién toca el recurso? (solo uno)
Semaphore	¿Cuántos a la vez? (hasta N)
Barrier	¿Cuándo empieza la siguiente fase? (cuando todos llegan)

Los tres se pueden combinar: una Barrier para coordinar fases, un Semaphore para limitar quién accede al recurso en cada fase y un Lock para las secciones críticas dentro de cada fase.



Mini-chequeo

1. ¿Qué hace `barrera.wait()`?
2. ¿Qué ocurre si solo llegan 2 de los 3 hilos esperados por `Barrier(3)`?
3. ¿Para qué tipo de problema es la barrera la herramienta ideal?

>  Respuestas



Resumen en 3 frases

- La `Barrier(N)` espera a que **N hilos** lleguen al `wait()`; cuando el último llega, todos continúan a la vez.
- Es la herramienta para **sincronizar fases**: nadie empieza la siguiente fase hasta que todos acaban la anterior.
- El número de hilos debe coincidir con el de la barrera, o el grupo se queda esperando para siempre.



Vocabulario rápido

Término	Idea general
Barrier	Barrera: N hilos esperan hasta que llegan todos
wait()	El hilo llega a la barrera y espera al resto
Fase	Etapas de un trabajo; las fases se separan con barreras
Sincronización de grupo	Coordinar N hilos, no solo 2
BreakingBarrierError	Se lanza si un hilo de la barrera se rompe/aborta



[Volver al índice de la unidad](#) · Anterior: [04 · Semaphore](#) · Siguiente: [06 · Condition](#)

06 — Condition

Cuando un hilo necesita que otro le avise 🚦

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 06 · Condition

📌 La idea en una frase

Una **Condition** permite que un hilo se duerma esperando a que otro le **notifique** que algo ocurrió: `wait()` duerme y libera el lock, `notify()` despierta a un hilo dormido.

El Lock, el Semaphore y la Barrier sincronizan **acceso**. La Condition sincroniza **eventos**: un hilo quiere hacer algo, pero tiene que esperar a que la condición (por ejemplo, “hay un elemento en la cola”) se cumpla, y es otro hilo quien se la avisa.

```

import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for i in range(5):
        with condition:
            item = random.randint(1, 100)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify() # Avisar al consumidor
            time.sleep(random.random())

def consumidor():
    for _ in range(5):
        with condition:
            while not cola:
                condition.wait() # Espera a que haya algo
            item = cola.pop(0)
            print(f"🍴 Consumido: {item}")

h_prod = threading.Thread(target=productor)
h_cons = threading.Thread(target=consumidor)
h_prod.start()
h_cons.start()
h_prod.join()
h_cons.join()

```

El productor añade un elemento y **notifica**; el consumidor, si la cola está vacía, **espera** hasta que le avisen. El Condition lleva asociado un lock interno: al entrar en `with condition:` se adquiere, y `wait()` lo **libera** mientras duerme para que el productor pueda entrar.



Métodos de Condition

Método	Qué hace
<code>wait()</code>	Libera el lock y espera una notificación
<code>notify()</code>	Despierta a un hilo que está esperando

Método

Qué hace

`notify_all()` Despierta a todos los hilos que esperan

⚠️ **Siempre dentro del `with condition:.wait()`, `notify()` y `notify_all()` exigen tener el lock de la condición adquirido.**

⚠️ La regla de oro: `while`, no `if`

Tras despertar con `notify()`, el hilo debe **volver a comprobar la condición**:

```
# ✅ CORRECTO
while not cola:
    condition.wait()

# ❌ INCORRECTO
if not cola:
    condition.wait()
```

¿Por qué? Pueden darse **falsas activaciones** (spurious wakeups) o que otro hilo haya consumido el elemento mientras esperábamos. Con `if`, el hilo despierta y asume que la condición se cumple... pero puede que ya no. Con `while`, tras despertar **recomprueba** y, si la condición sigue sin cumplirse, vuelve a dormir.

💡 Esta es una de las diferencias que separa al novato del profesional: **siempre `while` después de `wait()`**.



¿Cuándo usarla?

Situación

¿Por qué Condition?

Productor-consumidor con cola

El consumidor espera hasta que haya producto

Un hilo “jefe” que espera a que los obreros terminen

`wait()` hasta que avisen

Situación	¿Por qué Condition?
Recurso que se llena y se vacía	Notificar cuando el estado cambia
Turnos entre hilos	notify() al que le toca

La Condition es el mecanismo del **aviso**: un hilo no adivina cuándo hay trabajo, otro se lo dice. Es la pieza que convierte “comprobar sin parar” (y quemar CPU) en “dormir y que me despierten”.

```
import threading, time

condition = threading.Condition()
listo = False

def trabajador():
    global listo
    with condition:
        while not listo:          # espera la señal de la jefa
            condition.wait()
        print("🔧 Trabajador: ¡manos a la obra!")

def jefa():
    global listo
    time.sleep(1)
    with condition:
        listo = True
        condition.notify()       # ¡despierta al trabajador!
    print("👤 Jefa: ¡podéis empezar!")
```

Sin la condición, el trabajador tendría que comprobar listo en bucle gastando CPU. Con wait()/notify(), duerme hasta que la jefa le avisa.

Mini-chequeo

1. ¿Qué hace wait() exactamente con el lock de la condición?
2. ¿Por qué hay que usar while en lugar de if después de wait()?
3. ¿Qué diferencia hay entre notify() y notify_all()?

Resumen en 3 frases

- La Condition sincroniza **eventos**: un hilo hace `wait()` (liberando el lock y durmiendo) hasta que otro hace `notify()`.
- La regla de oro es **while** y no **if** después de `wait()`: las falsas activaciones exigen recomprobar la condición.
- Es la base del patrón **productor-consumidor**, que verás en el siguiente punto.

Vocabulario rápido

Término	Idea general
Condition	Mecanismo de espera y aviso entre hilos
<code>wait()</code>	Libera el lock y duerme hasta que le avisen
<code>notify()</code>	Despierta a un hilo que está esperando
<code>notify_all()</code>	Despierta a todos los hilos que esperan
Falsa activación	Despertar sin que nadie notificara de verdad (spurious wakeup)

 [Volver al índice de la unidad](#) · Anterior: [05 · Barrier](#) · Siguiente: [07 · Productor-Consumidor](#)

07 — Productor-Consumidor

El patrón clásico con cola y Condition 🧑‍💻 🧠

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 07 · Productor-Consumidor

La idea en una frase

El patrón **productor-consumidor** separa a quien **crea** datos (productor) de quien los **procesa** (consumidor): comparten una cola y una Condition para que el consumidor espere cuando la cola está vacía y el productor le avise cuando añade algo.

Es el patrón de sincronización más usado en el mundo real: un hilo llena una cola de tareas, otros hilos las procesan. La Condition del [punto 6](#) es el pegamento que los coordina.

```

import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for i in range(5):
        with condition:
            item = random.randint(1, 100)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify() # Avisar al consumidor
            time.sleep(random.random())

def consumidor():
    for _ in range(5):
        with condition:
            while not cola:
                condition.wait() # Espera a que haya algo
            item = cola.pop(0)
            print(f"🍴 Consumido: {item}")

h_prod = threading.Thread(target=productor)
h_cons = threading.Thread(target=consumidor)
h_prod.start()
h_cons.start()
h_prod.join()
h_cons.join()

```

Salida (aproximada):

```

📦 Producido: 73
🍴 Consumido: 73
📦 Producido: 41
🍴 Consumido: 41
📦 Producido: 8
🍴 Consumido: 8
...

```

El ritmo lo marcan los `sleep()`: a veces el consumidor se adelanta, encuentra la cola vacía y espera; a veces el productor se adelanta, deja elementos y el consumidor los consume de

golpe. La Condition garantiza que **ninguno se pierde un dato** y que nadie se queda esperando un elemento que ya llegó.

Paso a paso

Veamos cómo se coordinan los dos hilos:

1. El consumidor arranca, la cola está vacía
 - with condition: adquiere el lock
 - while not cola: es True
 - condition.wait() → libera el lock y se duerme 
2. El productor arranca, con el lock libre
 - with condition: adquiere el lock
 - cola.append(item) → cola = [73]
 - condition.notify() → despierta al consumidor 
 - sale del with → libera el lock
3. El consumidor se despierta
 - comprueba while not cola → False (¡hay un item!)
 - item = cola.pop(0) → procesa el 73
 - sale del with → libera el lock

La magia de wait() del [punto 6](#): mientras el consumidor duerme, **libera el lock** para que el productor pueda entrar y producir. Sin esa liberación, el productor nunca podría producir y el consumidor esperaría eternamente (deadlock).

Con varios productores y consumidores

Con **varios** hilos a cada lado, la regla cambia ligeramente: si varios consumidores esperan, notify() solo despierta a **uno** y los demás se quedan dormidos aunque haya trabajo. Para no dejar a nadie esperando, el productor usa notify_all():


```

import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for _ in range(3):
        with condition:
            item = random.randint(1, 10)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify_all() # despierta a TODOS los consumidores
            time.sleep(1)

def consumidor(id):
    for _ in range(3):
        with condition:
            while not cola:
                condition.wait()
            item = cola.pop(0)
            print(f"🍴 Consumidor-{id} comió: {item}")

hilos = []
for i in range(2):
    hilos.append(threading.Thread(target=consumidor, args=(i,)))
for _ in range(1):
    hilos.append(threading.Thread(target=productor))

for h in hilos: h.start()
for h in hilos: h.join()

```

💡 Y si quieres limitar el tamaño de la cola (buffer con tope), el semáforo del [punto 4](#) o la clase `queue.Queue(maxsize=N)` (thread-safe, sin locks a mano) te dan el control: los productores llaman a `put()` y los consumidores a `get()`.

Mini-chequeo

1. ¿Qué papel juega cada hilo en el patrón productor-consumidor?
2. ¿Por qué `wait()` tiene que liberar el lock mientras el consumidor duerme?

3. ¿Por qué con varios consumidores conviene `notify_all()` en lugar de `notify()`?

>  Respuestas

✓ Resumen en 3 frases

- El patrón **productor-consumidor** separa la creación de datos (productor) del procesamiento (consumidor) con una cola compartida.
- La `Condition` coordina el aviso: el consumidor hace `wait()` (liberando el lock) y el productor hace `notify()/notify_all()` cuando añade un elemento.
- La regla `while + wait()` garantiza que ningún hilo consuma un dato que no existe ni se quede dormido con trabajo pendiente.

Vocabulario rápido

Término	Idea general
Productor	Hilo que crea datos y los mete en la cola
Consumidor	Hilo que saca datos de la cola y los procesa
Cola compartida	Buffer donde el productor deja y el consumidor recoge
<code>notify_all()</code>	Avisar a todos los consumidores que esperan
Buffer limitado	Cola con tope máximo (semáforo o <code>queue.Queue</code>)

 [Volver al índice de la unidad](#) · Anterior: [06 · Condition](#) · Siguiente: [08 · Buenas prácticas](#)

08 — Buenas prácticas

Deadlocks, orden de locks, el ring final y a practicar 🔵

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 08 · Buenas prácticas

La idea en una frase

Ya tienes Lock, RLock, Semaphore, Barrier y Condition. Este punto cierra la teoría con el enemigo más temido —el **deadlock**—, las reglas para evitarlo y el “ring” que enfrenta a los mecanismos para que decidas cuándo usar cada uno.

Después de este punto, el cierre de la unidad (punto 9) y los boletines pondrán a prueba todo lo aprendido.

El deadlock: todos esperan al Lock

Un **deadlock** es la situación en la que dos o más hilos se esperan **mutuamente para siempre**: cada uno tiene un recurso y espera el que tiene el otro.

Ejemplo clásico con 2 hilos y 2 locks:

Hilo-A: adquiere Lock-1	Hilo-B: adquiere Lock-2
Hilo-A: espera Lock-2 ←→	Hilo-B: espera Lock-1

- Hilo-A tiene Lock-1 y espera a que se libere Lock-2.
- Hilo-B tiene Lock-2 y espera a que se libere Lock-1.
- **Ninguno de los dos puede avanzar.** Se esperan el uno al otro eternamente.

```

import threading, time

lock1 = threading.Lock()
lock2 = threading.Lock()

def hilo_a():
    with lock1:
        time.sleep(0.1)          # tiempo para que B tome lock2
        with lock2:              # B tiene lock2... espera para siempre 💀
            print("A: ¡trabajando!")

def hilo_b():
    with lock2:
        time.sleep(0.1)          # tiempo para que A tome lock1
        with lock1:              # A tiene lock1... espera para siempre 💀
            print("B: ¡trabajando!")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start(); h2.start()
h1.join(); h2.join() # ← nunca termina

```

Los `sleep(0.1)` fuerzan el escenario: A coge el `lock1`, B coge el `lock2`, y después cada uno intenta coger el del otro. El programa **se queda colgado** y ni A ni B imprimen su mensaje. La CPU se queda al 100% con dos hilos bloqueados que se esperan a sí mismos... en realidad, al otro.

Cómo evitar deadlocks: las 3 reglas

1. **Adquirir los locks siempre en el mismo orden.** Si todos los hilos piden primero Lock-1 y luego Lock-2, el escenario anterior no puede darse: nadie tiene Lock-2 sin haber pasado por Lock-1.
2. **Usar `with lock`:** (nunca olvidar `release()`). Un `release()` olvidado deja el cerrojo puesto para siempre y cualquier hilo que lo espere, esperará eternamente.
3. **Usar `RLock` si el mismo hilo necesita adquirirlo varias veces** (lo viste en el [punto 3](#)): evita que un hilo se espere a sí mismo.

⚠️ **¿Puedo tener más de un Lock?** Sí, pero cada Lock añade riesgo de deadlock. Sé disciplinado con el orden de adquisición: **mismo orden para todos los hilos, siempre.**

Preguntas rápidas (y sus respuestas)

¿Qué es más rápido, Lock o Semaphore? Lock es más simple y, para exclusión mutua, gana en velocidad. Semaphore tiene contador interno: solo lo necesitas para limitar acceso concurrente (aforo). Para “solo uno a la vez”, Lock.

¿Y si necesito sincronización entre procesos en vez de entre hilos? Usa multiprocessing.Lock, multiprocessing.Semaphore, etc. Son equivalentes a los de threading pero para procesos: recuerda que los procesos no comparten memoria, así que la sincronización se hace sobre recursos del sistema.

El ring de los conceptos — Lock vs Semáforo vs Barrera

Lock: “Soy el portero. Solo dejo pasar a **uno cada vez**. Los demás esperan fuera.”

Semáforo(3): “Soy el guardia de una sala con **3 sillas**. Cuando una se libera, entra el siguiente.”

Barrera(3): “Soy el juez de salida. **No permito que nadie corra** hasta que los 3 estén en la línea.”

Lock: “Para exclusión mutua, no hay mejor que yo. Dos hilos no pueden tocarme el recurso a la vez.”

Semáforo: “Para **limitar acceso concurrente**. Como un aforo máximo en un local.”

Barrera: “Para sincronizar **fases de un trabajo**. Todos completan la fase 1, luego todos empiezan la fase 2.”

Moraleja del ring: cada mecanismo responde a una pregunta distinta — ¿quién? (Lock), ¿cuántos? (Semáforo), ¿cuándo? (Barrera). Y la Condition del [punto 6](#) añade el aviso entre hilos.

Aprieta el lápiz

1. **Condición de carrera:** Crea 2 hilos que incrementen un contador 1M veces cada uno. Compara con y sin Lock.

2. **Semáforo descargas:** Crea 10 hilos que simulen descargas (sleep 2s) usando un Semáforo(3). Mide el tiempo total.
3. **Barrera de fases:** Crea 4 hilos que trabajen en 2 fases. La fase 2 no empieza hasta que todos terminan la fase 1.
4. **Productor-Consumidor:** 1 productor crea números aleatorios cada 0.5s; 2 consumidores los procesan. Usa Condition.
5. **Deadlock provocado:** Crea intencionadamente un deadlock con 2 hilos y 2 locks. Luego arréglalo.

> Soluciones

Pool Puzzle de sincronización

Instrucciones: Ordena estas líneas para que 3 hilos incrementen un contador compartido de forma segura.

```
a. lock = threading.Lock()
b. for h in hilos: h.start()
c. contador += 1
d. hilos = []
e. import threading
f. for _ in range(3):
g. with lock:
h. def incrementar():
i.     hilos.append(threading.Thread(target=incrementar))
j. for h in hilos: h.join()
k. global contador
l. contador = 0
m. print(contador)
```

> Solución

Mini-chequeo

1. Dibuja el escenario exacto de un deadlock con 2 hilos y 2 locks.
2. ¿Cuál es la regla de oro para los locks múltiples?
3. ¿Qué mecanismo usarías para: un recurso a la vez / aforo de 3 / coordinar fases?

Resumen en 3 frases

- El **deadlock** es un abrazo mortal: cada hilo espera un recurso que tiene el otro, y ninguno avanza.
- Reglas anti-deadlock: **mismo orden de locks**, **with lock**: siempre, y **RLock** si el mismo hilo se re-adquiere.
- El ring final te deja la regla: Lock para exclusión mutua, Semaphore para aforo, Barrier para fases y Condition para el aviso.

Vocabulario rápido

Término	Idea general
Deadlock	Hilos que se esperan mutuamente para siempre
Orden de locks	Regla: todos los hilos piden los locks en el mismo orden
Abrazo mortal	Otro nombre del deadlock (Hilo-A y Hilo-B entrelazados)
multiprocessing.Lock	Lock equivalente para procesos en lugar de hilos
Pool Puzzle	Ejercicio de ordenar líneas de código sueltas

 [Volver al índice de la unidad](#) · Anterior: [07 · Productor-Consumidor](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé el lock, laboratorio con carrera y el duelo final de la unidad 🧠

📖 Estás en: 🔒 U03 · Sincronización entre Hilos → 09 · Cierre

Has terminado la teoría: condición de carrera, Lock, RLock, Semaphore, Barrier, Condition, productor-consumidor y las reglas anti-deadlock. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el lock

Eres un threading.Lock recién creado. Te han asignado proteger el contador compartido de 4 hilos. Cada hilo quiere hacer `contador += 1` 100.000 veces.

¿Qué pasa?

1. El hilo A llama a `with lock:` → **adquieres** el cerrojo y le das paso.
2. A ejecuta `contador += 1` de principio a fin: lee, suma, escribe. **Nadie se cuela**: tú estás puesto.
3. A sale del bloque → llamas a `release()` y **liberas** el cerrojo.
4. El hilo B, que estaba esperando, entra: **adquieres** de nuevo, repite los pasos 2-3.
5. Los 4 hilos se turnan contigo: 100.000 veces cada uno.

Resultado: el contador llega a **400.000**, exacto, en todas las ejecuciones. Tú (el Lock) has convertido el caos de la condición de carrera en un turno ordenado.

💡 **Ahora tú:** ¿y si un hilo olvidara llamarte con `release()`? Te quedarías puesto para siempre y el resto esperaría eternamente: eso es un **deadlock**. Por eso los hilos te usan siempre con `with lock:`, que garantiza el `release()` pase lo que pase (lo viste en el [punto 2](#)).

🔥 Fireside Chat: Lock vs Semaphore

Dos mecanismos de sincronización se sientan junto a la chimenea a resolver, de una vez, quién controla el acceso.

Lock: — Yo soy la exclusión mutua. Solo uno de mis hilos toca el recurso a la vez. Los demás esperan fuera hasta que libero.

Semaphore: — Vaya, qué estricto. Yo dejo pasar hasta **N**. Tengo un contador interno: mientras queden puestos, entran; cuando se acaban, esperan.

Lock: — ¿Y para qué querría nadie que entren varios? Mi recurso no admite más de uno.

Semaphore: — Para un **aforo**, querido: 3 impresoras, 3 descargas, 3 conexiones a la BD. Conmigo entran exactamente 3. Tú solo sabes decir “uno”.

Lock: — Pero yo soy más simple y rápido. Para una sección crítica, no hay discusión.

Semaphore: — Y yo puedo hacer de ti: Semaphore(1) equivale a tu exclusión mutua. Pero tú no puedes ser yo.

Lock: — Vale, cada uno a lo suyo. ¿Y esa barrera que todo lo frena?

Semaphore: — *suspira* Esa no nos habla. Solo sabe esperar a que lleguen todos antes de dejar correr a nadie. Cosas de carreras de relevos.

Moraleja: Lock para exclusión mutua (solo uno), Semaphore para aforo (hasta N), Barrier para fases (cuando todos llegan). Y la Condition de fondo, avisando.



¿Quién soy?

1. Soy el cerrojo que deja pasar a **un solo** hilo a la sección crítica a la vez.
2. Soy el cerrojo que el mismo hilo puede adquirir varias veces sin bloquearse.
3. Soy el guardia de una sala con **3 sillas**: cuando una se libera, entra el siguiente.
4. Soy el juez de salida: no dejo correr a nadie hasta que los 3 corredores están en la línea.
5. Soy el aviso: un hilo se duerme con `wait()` hasta que otro lo despierta con `notify()`.
6. Soy el bug que ocurre cuando dos hilos se pisan la misma variable compartida.

>  Respuestas



CONRAD VS EL MUNDO: “olvidé el `release()` y todos esperan al lock”

CONRAD: — “Clásico: el programa se cuelga y nadie sabe por qué. Miras el código y... ¡claro! Usé `lock.acquire()`, hice el trabajo, y se me olvidó el `lock.release()`. El hilo se quedó con el cerrojo puesto, y todos los demás se quedaron esperando al lock para siempre. No es un fallo raro: es un **deadlock** en su forma más tonta.”

CONRAD: — “Y lo mejor: *‘pero yo hago acquire solo en este sitio’*. ¡Pues claro! También lo haces dentro de un `with`, y así nunca más. El `with lock:` hace el `release()` solo, aunque haya una excepción en medio. Adquirir a mano sin liberar es firmar una sentencia de muerte para tus hilos.”

CONRAD: — “Y no me vengas con *‘¿será que el ordenador va lento?’*. No. Si un hilo se queda esperando para siempre y el contador no llega a su valor, ya tienes la respuesta: hay un lock mal liberado o un orden de adquisición incorrecto. El GIL del [punto 1](#) no te protege de esto. A diagnosticar.”

Laboratorio de tortura: la caja común de la cooperativa

Duración: 45 minutos **Herramienta:** Python 3 (`threading` y `time`, sin instalar nada)

Escenario: 4 hilos representan a 4 socios de una cooperativa que comparten una caja común (el contador). Cada socio debe aportar 100.000 monedas (`contador += 1`). Al final de la jornada, la caja debe contener **400.000**.

Tareas paso a paso:

1. Escribe la versión **sin sincronizar** del contador y ejecútala varias veces: anota que el total nunca es 400.000 (condición de carrera del [punto 1](#)).
2. Protege el incremento con `lock = threading.Lock()` y `with lock:` (el [punto 2](#)). Ejecútala 3 veces: ahora siempre da **400.000**.
3. Sustituye el Lock por un Semaphore(1) y verifica que también funciona (el [punto 4](#)).
4. Añade una **segunda caja** (otro contador de operaciones) que se incremente desde una función auxiliar llamada dentro de otra: protégelas con un RLock para que el mismo hilo pueda adquirirlo dos veces ([punto 3](#)).
5. Añade una Barrier(4) para que los 4 socios **empiecen a aportar a la vez**: nadie toca su caja hasta que los 4 están en la salida ([punto 5](#)).

Fallo intencionado: en la tarea 2, “por accidente”, uno de los 4 hilos hace contador += 1 **fuera** del with lock: (o bien un hilo que usa lock.acquire() sin release()). ¿Qué pasa? El contador vuelve a dar valores por debajo de 400.000, y si es el release() olvidado, el programa se **cuelga** esperando el join(). Los demás hilos están perfectos: el fallo es de un único hilo.

Pista 1: busca el único contador += 1 que no está dentro de with lock:. El contador da mal aunque los otros 3 hilos estén bien protegidos: basta **un** hilo fuera de la sección crítica.

Pista 2: si todo parece protegido pero el programa no termina, revisa que nadie llamó a acquire() sin su release(). Un with lock: nunca lo olvida; un acquire() suelto, sí. Ahí tienes el deadlock del [punto 8](#).

Logros de esta unidad

Logro	Cómo conseguirlo
 Carrera Cazador	Explicar por qué contador += 1 no es atómico y provocar una condición de carrera
 Cerrojo Maestro	Proteger un contador compartido con Lock y with lock: y obtener siempre 400.000
 Aforo Controlado	Limitar con Semaphore el acceso a N recursos (descargas, BD, impresoras)
 Sincronizador de Fases	Coordinar N hilos con Barrier para que ninguna fase arranque antes de tiempo
 Jefe de Cola	Implementar un productor-consumidor con Condition (wait/notify) sin perder datos
 Rompe Deadlocks	Detectar un deadlock y arreglarlo con orden de locks y with lock:

Atrévete a pensar

1. ¿Por qué contador += 1 con 4 hilos y sin lock casi nunca da 400.000, pero a veces sí?
2. ¿Qué pasaría si usaras Semaphore(3) para proteger una sección crítica que debe ser exclusiva?

3. ¿Por qué `notify()` puede dejar a un consumidor esperando para siempre con varios consumidores?
4. ¿Cuál es la diferencia práctica entre `Lock` y `RLock` para una función que llama a otra?
5. ¿Por qué “adquirir siempre en el mismo orden” elimina el deadlock de 2 hilos y 2 locks?

> 💡 Soluciones

🧩 Crucigrama de bits

Horizontal:

1. Cerrojo que deja pasar a un solo hilo a la vez (4 letras)
3. Lock que el mismo hilo puede adquirir varias veces (5 letras)
6. Condición en la que dos hilos se esperan para siempre (8 letras)
8. Mecanismo de espera y aviso: `wait()` y `notify()` (9 letras)

Vertical:

2. Aforo máximo: hasta N hilos dentro a la vez (9 letras)
4. Sincroniza fases: nadie avanza hasta que llegan todos (7 letras)
5. Operación que no se puede interrumpir a mitad (7 letras)
7. Bug cuando dos hilos se pisan una variable compartida (5 letras)

> 📄 Soluciones

💬 Entrevista de trabajo

1. “¿Qué es una condición de carrera y cómo la evitas en Python?”
2. “Explica la diferencia entre `Lock`, `RLock`, `Semaphore` y `Barrier`. ¿Cuándo usarías cada uno?”
3. “¿Cómo funciona `Condition.wait()` y `notify()`? ¿Por qué se usa `while` y no `if`?”
4. “Implementa un productor-consumidor con varios consumidores.”
5. “¿Qué es un deadlock? Pon un ejemplo y explica cómo prevenirlo.”

💡 **Cómo encararlas:** la 2 y la 4 son las “preguntas reina”. Para la 2, recorre el ring del [punto 8](#): `Lock` = exclusión mutua (solo uno), `Semaphore` = aforo (hasta N), `Barrier` = fases (cuando todos llegan), `RLock` = reentrante. Para la 4, plantea una cola compartida, `Condition` con

`notify_all()`, y la regla `while not cola: wait()`. Si sabes contarlo fluido, ya eres medio experto en concurrencia.

No hay preguntas tontas

? ¿Qué es un deadlock?

Dos o más hilos esperándose mutuamente para siempre. Ejemplo: Hilo-A tiene Lock-1 y espera Lock-2; Hilo-B tiene Lock-2 y espera Lock-1.

? ¿Cómo evito deadlocks?

1. Adquirir los locks siempre en el **mismo orden**
2. Usar `with lock:` (nunca olvidar `release()`)
3. Usar `RLock` si el mismo hilo necesita adquirirlo varias veces

? ¿Puedo tener más de un Lock?

Sí. Pero cada Lock añade riesgo de deadlock. Sé disciplinado con el orden de adquisición.

? ¿Qué es más rápido, Lock o Semaphore?

Lock es más simple. Semaphore tiene contador interno. Para exclusión mutua, Lock gana en velocidad.

? ¿Y si necesito sincronización entre procesos en vez de entre hilos?

Usa `multiprocessing.Lock`, `multiprocessing.Semaphore`, etc. Son equivalentes pero para procesos.

Poscréditos

El contador llega a 400.000, exacto. Los 4 hilos se turnan con el Lock sin pisarse.

Un hilo se olvida del `release()`. Los demás esperan al lock. El programa se congela.

El semáforo regula el aforo, la barrera alinea las fases y el productor despierta al consumidor con un `notify()`.

La sincronización ha ganado la batalla contra el caos.

PRÓXIMAMENTE EN U04: *Sockets TCP*. Los hilos comparten memoria y ahora saben coordinarse... pero ¿y si los procesos ni siquiera comparten memoria? Tocaré hablar por la red.

✓ Criterios de evaluación cubiertos (RA2)

RA2: Gestiona la programación de hilos y su sincronización.

CE	Criterio	Cubierto
RA2c	Sincroniza hilos con Lock	✓ Puntos 2, 3 y 7 + ⚡ Laboratorio de tortura
RA2d	Usa semáforos para acceso controlado	✓ Punto 4 + ⚡ Laboratorio de tortura
RA2g	Evita condiciones de carrera	✓ Puntos 1, 2 y 8 + ⚡ Laboratorio con fallo intencionado

RA2a, RA2b, RA2e, RA2f y RA2h se cubren en la **U02 · Hilos Fundamentos**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Buenas prácticas](#) · Siguiente: [U04 · Sockets TCP](#)

U04 — Sockets TCP

Cliente y servidor que hablan por el puerto 🍷

📖 Ruta del viaje: 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🍷 Sockets TCP → 📡 UDP →
🌐 API REST → 🧪 APIs comerciales → 🔒 Hash → 🧬 Cifrado → ⌚ Servidores → ⌚ asyncio

“Un socket TCP es como una llamada telefónica: marcas, esperas a que contesten, habláis y colgáis.”

En la U03 sincronizaste hilos para que no se pisen entre sí dentro de un mismo programa. Ahora toca lo contrario: hacer que **procesos distintos, incluso en máquinas distintas, hablen entre ellos**. La herramienta es el **socket TCP**, el canal fiable que garantiza que cada byte que envías llegue entero y en orden. Es el mismo mecanismo que mueve la web, el correo y casi todo lo que ves en pantalla.

En esta unidad construirás tu primera conversación entre programas: un **cliente** que llama y un **servidor** que contesta. Aprenderás el three-way handshake que prepara la llamada, los errores típicos cuando la red se rompe, y el truco de `SO_REUSEADDR` para que tu servidor aguante reinicios sin quejarse. Al final, montarás un **servidor eco** completo y tendrás un método infalible para diagnosticar cualquier problema de red.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un **socket** y para qué sirven la **IP** y el **puerto**.
- Describir el **modelo de capas (TCP/IP)** y distinguir TCP de UDP.
- Implementar un **cliente TCP** con `connect()`, `sendall()` y `recv()`.
- Implementar un **servidor TCP** con `bind()`, `listen()` y `accept()`.
- Describir el **three-way handshake** y el cierre de una conexión TCP.
- Gestionar **errores de red** (timeout, rechazo, reinicio) con `try/except`.
- Configurar **`SO_REUSEADDR`** y **timeouts** para servidores robustos.
- Hablar **protocolos de aplicación** sobre TCP, como HTTP, a nivel de socket.

- Montar y depurar un **servidor + cliente eco TCP** completo.



Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Qué es un socket	IP + puerto, qué es un socket y la analogía del teléfono	Todos
02 · Cliente TCP	connect(), sendall() y recv(): el cliente que habla primero	Todos
03 · Servidor TCP	bind(), listen() y accept(): el servidor que escucha y atiende	Todos
04 · Ciclo de vida de la conexión	Three-way handshake y cierre de la conexión TCP	Todos
05 · Errores y gestión	ConnectionResetError, BrokenPipeError, timeouts y try/except	Todos
06 · SO_REUSEADDR	“Address already in use”, TIME_WAIT y cómo evitarlo	Todos
07 · Protocolos sobre TCP	HTTP hablado a pelo con un socket y el orden de bytes	Todos
08 · Servidor eco completo	Servidor + cliente eco, mano a mano TCP y ejercicios resueltos	Todos
09 · Cierre	Sé el socket, Fireside, Laboratorio de tortura, Crucigrama...	Todos



Flujo de lectura: los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.



Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación (HTTP, NTP).

CE	Criterio	Dónde se cubre
a)	Modelo de capas de red (TCP/IP)	✓ Punto 1
c)	Crea servidores TCP	✓ Puntos 3 y 8 + ⚡ Laboratorio (punto 9)
d)	Crea clientes TCP	✓ Puntos 2 y 8 + ⚡ Laboratorio (punto 9)
f)	Gestiona errores de red	✓ Punto 5 + ⚡ Laboratorio (punto 9)
g)	Configura opciones de socket (SO_REUSEADDR, non-blocking)	✓ Puntos 5 y 6

RA3b (UDP), RA3e (UDP servidor/cliente) y RA3h (protocolos HTTP/NTP) se cubren en la **U05 · Sockets UDP y Protocolos**.

📖 ¿Por dónde empiezo?

¿Vienes de la U03 y dominas hilos y sincronización? Perfecto, ese es el trampolín ideal: repasa la [U03 · Sincronización entre hilos](#) para tener frescos los locks y semáforos (te harán falta en el [TEMA 10](#)), y arranca en el [punto 1](#), que parte de cero: qué es un socket, la IP y el puerto.

¿Ya sabes qué es un socket y solo necesitas el servidor, los errores o el truco de SO_REUSEADDR? Ve directo al [punto 3](#), al [5](#) o al [6](#). Si vienes de cero en redes, no te saltes los puntos 1 a 4: la IP, el puerto y el handshake son la base de todo lo demás.

📌 Primer punto: [01 · Qué es un socket](#)

▶ Al acabar la unidad, continúa en [U05 · Sockets UDP y Protocolos](#).

01 — Qué es un socket

IP + puerto y el punto final de toda conexión 🍷

📖 Estás en: 🍷 U04 · Sockets TCP → 01 · Qué es un socket

📌 La idea en una frase

Un **socket** es el punto final de una conexión de red: la interfaz que tu programa usa para enviar y recibir datos a través de la red.

Piensa en una llamada telefónica (la analogía que abre esta unidad): marcas, esperas a que contesten, habláis y colgáis. Tu teléfono es el **socket**; el número que marcas es la **IP**; y la extensión a la que pides hablar es el **puerto**. Sin ese trío no hay conversación posible.

📄 IP y puerto: el “quién” y el “dónde”

Para que dos programas hablen por la red necesitan dos datos:

Concepto	Qué es	Ejemplo
IP	Identifica la máquina en la red	127.0.0.1, 192.168.1.10
Puerto	Identifica el programa dentro de la máquina	80 (web), 5000, 9000

- **127.0.0.1** es **localhost**: tu propia máquina. Perfecto para pruebas sin red real. Para que otros se conecten, usa tu IP real (ej: 192.168.1.x).
- Un puerto es un número del **0 al 65535**. El SO se reserva algunos para servicios conocidos (el 80 para web, el 443 para HTTPS); para tus programas usa puertos altos como 5000 o 9000.

💡 La **pareja (IP, puerto)** se llama *dirección de socket* o *endpoint*, y en Python siempre se pasa como una tupla: ("127.0.0.1", 5000).

🐍 Crear un socket en Python

```
import socket

# Crear un socket TCP
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Parámetro	Significado	Valores
AF_INET	Familia de direcciones	IPv4
AF_INET6	IPv6	
SOCK_STREAM	TCP (orientado a conexión)	
SOCK_DGRAM	UDP (sin conexión)	

- **AF_INET** dice al SO que usaremos direcciones **IPv4** (**AF_INET6** para IPv6).
- **SOCK_STREAM** pide un socket **TCP**, orientado a conexión: el canal fiable, ordenado y sin pérdidas que estudias en esta unidad.
- **SOCK_DGRAM** (lo verás en la [U05](#)) es el UDP sin conexión.

La analogía del teléfono, al detalle

TELÉFONO		SOCKET
Tu teléfono	↔	socket()
El número que marcas	↔	IP (AF_INET)
La extensión	↔	Puerto
La centralita	↔	El SO (sistema operativo)

El **SO** es la centralita: se encarga de que el paquete con tu mensaje salga de tu teléfono y llegue al teléfono correcto. Tú solo te ocupas de marcar (conectarte) y de hablar (enviar y recibir). El socket es ese teléfono: el **punto final** de la conversación.

Mini-chequeo

1. ¿Qué identifica la **IP** y qué identifica el **puerto**?
2. ¿Qué representa **127.0.0.1** y para qué sirve?

3. ¿Qué constante de Python crea un socket **TCP** y cuál uno **UDP**?

>  Respuestas

✓ Resumen en 3 frases

- Un **socket** es el punto final de una conexión de red: la interfaz para enviar y recibir datos.
- La **IP** identifica la máquina y el **puerto** identifica el programa dentro de ella: juntos forman la dirección (IP, puerto).
- En Python se crea con `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` para TCP.

Vocabulario rápido

Término	Idea general
Socket	Punto final de una conexión de red
IP	Identifica la máquina en la red
Puerto	Identifica el programa dentro de la máquina
AF_INET	Familia de direcciones IPv4
SOCK_STREAM	Socket TCP orientado a conexión
SOCK_DGRAM	Socket UDP sin conexión
localhost	127.0.0.1, tu propia máquina

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · Cliente TCP](#)

02 — Cliente TCP

connect, sendall y recv: el cliente que habla primero 🗣️

📖 Estás en: 🍷 U04 · Sockets TCP → 02 · Cliente TCP

La idea en una frase

El cliente TCP **marca el número**: crea el socket, se conecta con `connect()`, envía con `sendall()`, recibe con `recv()` y cierra. Sin cliente nadie llama; sin él, el servidor esperaría en el puerto para siempre.

En el [punto 1](#) viste qué es un socket. Aquí lo pones a trabajar por el lado del **cliente**: el que inicia la conversación. En la analogía del teléfono, eres tú marcando el número y esperando a que contesten.

Ciclo de vida del cliente

```
socket() → connect() → send()/recv() → close()
```

Cuatro pasos en orden: **crear**, **conectar**, **hablar** y **cerrar**. El `with` de Python se encarga del cierre automático (paso 4) al salir del bloque.

El cliente mínimo resuelto

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
    cliente.connect((HOST, PORT))
    cliente.sendall(b"Hola servidor!")
    respuesta = cliente.recv(1024)
    print(f"Respuesta: {respuesta.decode()}")
```

Salida (con el servidor del [punto 3](#) escuchando en el puerto 5000):

Respuesta: Recibido: Hola servidor!

Cada método, al detalle

Método	Qué hace	¿Cuándo?
<code>socket(AF_INET, SOCK_STREAM)</code>	Crea el socket TCP	Siempre, lo primero
<code>connect((HOST, PORT))</code>	Establece la conexión (three-way handshake)	Antes de hablar
<code>sendall(datos)</code>	Envía todos los bytes	Cuando quieras mandar
<code>recv(1024)</code>	Recibe hasta 1024 bytes	Cuando esperes respuesta
<code>close()</code> / <code>with</code>	Libera la conexión	Al terminar

- **`connect()`** lanza `ConnectionRefusedError` si no hay nadie escuchando en ese puerto. Es la forma de saber que el servidor no está encendido.
- **`sendall()`** se llama *all* porque garantiza que se envían **todos** los bytes (a diferencia de `send()`, que puede enviar solo una parte y tendrías que reintentar).
- **`recv(1024)`** no significa que *solo* puedas recibir 1024 bytes: es el **tamaño máximo del buffer**. Si el mensaje es más grande, necesitas varios `recv()` y concatenar. Tú tienes que implementar el protocolo de aplicación para saber cuándo has recibido el mensaje completo.
- El **`with`** cierra el socket automáticamente al salir: si lo usas, no necesitas `close()` explícito.

Be the code, my friend — Cliente TCP

“Sé el cliente. Cada paso, cada byte.”

● CLIENTE

1. `socket(AF_INET, SOCK_STREAM) → cli`
2. `connect(("127.0.0.1", 5000))`
→ Three-way handshake TCP
3. `sendall(b"Hola servidor!")`
4. Esperando respuesta... ⌚

● CLIENTE

4. `recv(1024) → b"Recibido: Hola servidor!"`
5. `print("Respuesta: Recibido: Hola servidor!")`
6. `close()`

⚠ **recv() es bloqueante:** el programa se queda esperando hasta que lleguen datos. Si el servidor nunca responde, te quedas colgado ahí (para salir del apuro, verás los **timeouts** en el [punto 5](#)).

Mini-chequeo

1. ¿Qué método establece la conexión con el servidor y qué error lanza si no hay nadie escuchando?
2. ¿Por qué usar `sendall()` y no `send()`?
3. ¿Qué hace `recv(1024)` y por qué no es un límite absoluto?

> Respuestas

Resumen en 3 frases

- El cliente TCP sigue `socket() → connect() → send()/recv() → close()`.
- `connect()` estrecha la mano, `sendall()` envía todo y `recv(1024)` lee hasta 1024 bytes.
- El `with` cierra el socket solo; si el servidor no existe, `connect()` lanza `ConnectionRefusedError`.

Vocabulario rápido

Término	Idea general
connect()	Establece la conexión TCP con el servidor
sendall()	Envía todos los bytes de golpe
recv()	Recibe datos; bloqueante hasta que llegan
Bloqueante	La llamada se queda esperando hasta que algo ocurra
ConnectionRefusedError	No hay nadie escuchando en ese puerto

 [Volver al índice de la unidad](#) · **Anterior:** [01 · Qué es un socket](#) · **Siguiente:** [03 · Servidor TCP](#)

03 — Servidor TCP

bind, listen y accept: el que escucha y atiende 📞

📖 Estás en: 🍷 U04 · Sockets TCP → 03 · Servidor TCP

La idea en una frase

El servidor TCP **descolgó primero**: reserva un puerto con `bind()`, avisa con `listen()` y se queda esperando con `accept()` a que un cliente llame. Todo el peso de la conversación recae sobre él.

En el [punto 2](#) el cliente marcaba el número. Este punto es la otra cara de la moneda: el servidor que **escucha**. Sin `bind()` no tiene puerto; sin `listen()` no anuncia que espera llamadas; sin `accept()` no recibe a nadie.

Ciclo de vida del servidor

```
socket() → bind() → listen() → accept() → recv()/send() → close()
```

Seis pasos. Los tres primeros (`socket`, `bind`, `listen`) son el “descolgar el teléfono”; `accept()` es el “¿diga?” que se queda esperando a que suene el timbre.

El servidor mínimo resuelto

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 5000

# 1. Crear socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:

    # 2. Asignar dirección y puerto
    servidor.bind((HOST, PORT))

    # 3. Escuchar conexiones entrantes
    servidor.listen()
    print(f"Servidor escuchando en {HOST}:{PORT}")

    # 4. Aceptar una conexión (BLOQUEANTE)
    conexion, direccion = servidor.accept()
    print(f"Conectado con {direccion}")

    with conexion:
        # 5. Recibir datos
        datos = conexion.recv(1024)
        print(f"Recibido: {datos.decode()}")

        # 6. Enviar respuesta
        conexion.sendall(b"Recibido: " + datos)

    # 7. Se cierra solo al salir del with
```

Salida (tras ejecutar el cliente del punto 2 en otra terminal):

```
Servidor escuchando en 127.0.0.1:5000
Conectado con ('127.0.0.1', 54321)
Recibido: Hola servidor!
```

 Cada método, al detalle

Método	Qué hace	¿Cuándo?
<code>bind((HOST, PORT))</code>	Reserva la dirección y el puerto	Antes de escuchar
<code>listen()</code>	Prepara el socket para recibir conexiones	Tras el bind
<code>accept()</code>	Espera un cliente y devuelve (conexion, direccion)	En bucle
<code>recv(1024)</code>	Recibe datos del cliente conectado	Cuando llegan
<code>sendall(datos)</code>	Envía la respuesta	Cuando hay que contestar

- **bind()** es la única forma de “poseer” un puerto: sin él, el socket no escucha en ningún sitio. Para el servidor es **obligatorio**; para el cliente, el SO asigna uno automáticamente.
- **listen()** avisa al SO: “prepárate, voy a aceptar llamadas entrantes”. Sin ella, no hay puerto abierto y el cliente recibe `ConnectionRefusedError`.
- **accept()** es **bloqueante**: el servidor se queda esperando hasta que un cliente llama. Devuelve **dos cosas**: el socket de la conversación (conexion) y la dirección del cliente (direccion, una tupla (IP, puerto)).
- Fíjate: a partir de `accept()`, todo se hace con **conexion**, no con servidor. servidor solo escucha; conexion es el canal de esa llamada concreta.

El bucle de atención

El servidor de arriba atiende **un solo cliente** y termina. Para atender a varios, se envuelve el `accept()` en un bucle:

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
    servidor.bind(("127.0.0.1", 5000))
    servidor.listen()
    print("Servidor escuchando...")

    while True:
        conexion, direccion = servidor.accept()
        print(f"Conectado con {direccion}")
        with conexion:
            datos = conexion.recv(1024)
            print(f"Recibido: {datos.decode()}")
            conexion.sendall(b"Recibido: " + datos)
```

⚠ Con este código básico, el servidor atiende **un cliente cada vez**: si llega un segundo mientras el primero habla, se queda en la cola de `listen()`. Para atender varios a la vez necesitas hilos o `select()`: eso es el **TEMA 10 — Servidores Concurrentes**.

🧩 Pool Puzzle — El ciclo de vida del socket

Las líneas de abajo están desordenadas. ¿Puedes ordenarlas para que formen un servidor TCP funcional?

```
a) with socket.socket() as srv:
b)     srv.listen()
c)     conn, addr = srv.accept()
d)     import socket
e)         conn.sendall(b"Bienvenido\n")
f)     srv.bind(("127.0.0.1", 5000))
g)     with conn:
h)         datos = conn.recv(1024)
```

> 🗝 Solución

🧠 Mini-chequeo

1. ¿Qué tres métodos preparan el servidor antes de atender a nadie?

2. ¿Qué devuelve `accept()` y por qué es bloqueante?
3. Tras `accept()`, ¿con qué objeto se recibe y envía: servidor o conexión?

> Respuestas

Resumen en 3 frases

- El servidor TCP sigue `socket()` → `bind()` → `listen()` → `accept()` → `recv()/send()` → `close()`.
- `bind()` reserva el puerto, `listen()` anuncia que espera llamadas y `accept()` bloquea hasta que un cliente llama.
- Para atender a varios clientes hay que envolver `accept()` en un bucle (y más adelante, usar hilos o `select()`).

Vocabulario rápido

Término	Idea general
<code>bind()</code>	Reserva la dirección y el puerto del servidor
<code>listen()</code>	Prepara el socket para conexiones entrantes
<code>accept()</code>	Acepta un cliente; bloqueante, devuelve (conexión, dirección)
conexión	El canal de la conversación con ese cliente
Bloqueante	La llamada espera hasta que algo ocurra

 [Volver al índice de la unidad](#) · Anterior: [02 · Cliente TCP](#) · Siguiente: [04 · Ciclo de vida de la conexión](#)

04 — Ciclo de vida de la conexión

Three-way handshake y cierre de una conexión TCP 🤝

📖 Estás en: 🚪 U04 · Sockets TCP → 04 · Ciclo de vida de la conexión

📌 La idea en una frase

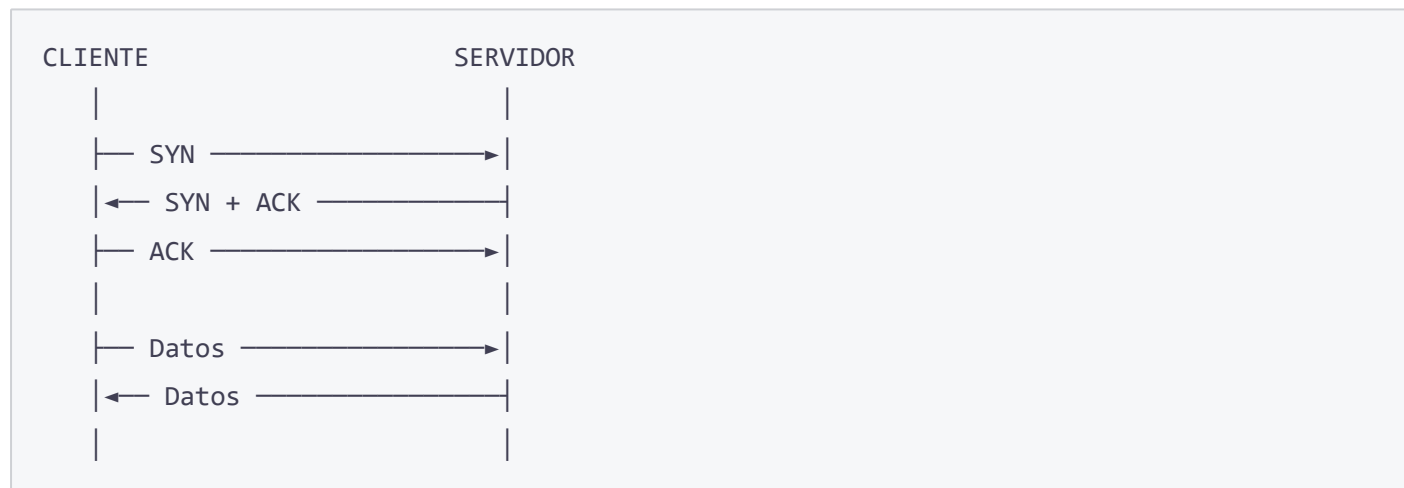
Una conexión TCP **nace, vive y muere**: arranca con un **three-way handshake** (SYN → SYN+ACK → ACK), transporta los datos y se cierra con un apretón de despedida. Todo esto pasa *debajo* de tu `connect()` y tu `accept()`.

En el [punto 2](#) y el [3](#) llamaste a `connect()` y `accept()` sin saber qué pasaba por dentro. Aquí levantas el capó y ves el **protocolo de transporte**: los tres mensajes que estrechan la mano y la despedida cuando la conversación acaba.

🧱 TCP: orientado a conexión

TCP garantiza que los datos lleguen **en orden** y **sin pérdidas**. A cambio, es un poco más lento que UDP. Ese “orientado a conexión” significa exactamente lo que vas a ver ahora: antes de enviar un solo byte, ambas partes tienen que **ponerse de acuerdo** con un saludo previo.

🤝 Three-way handshake (establecer conexión)



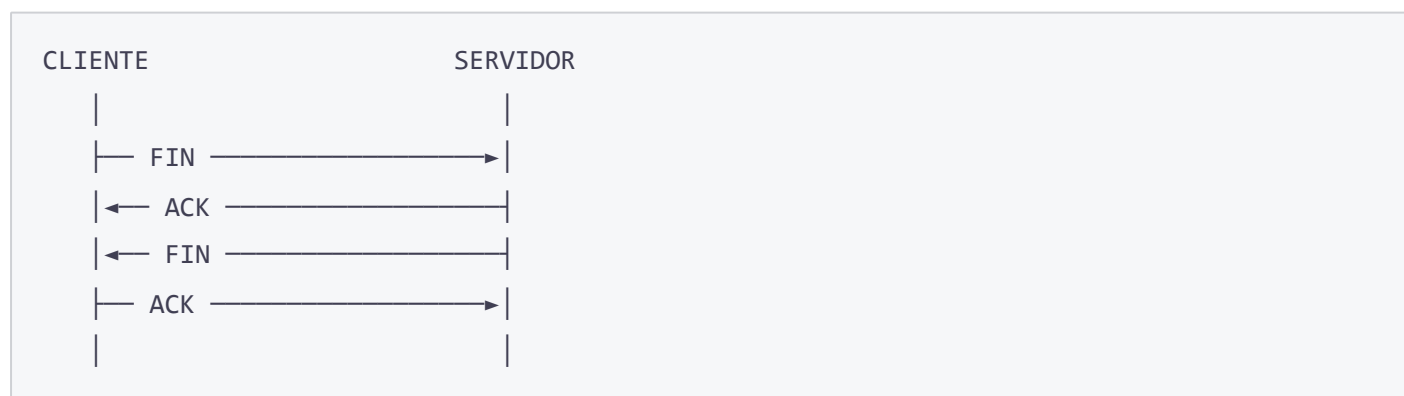
Los tres primeros mensajes son el handshake:

1. **SYN** (synchronize): el cliente dice *“quiero hablar contigo”*.
2. **SYN + ACK**: el servidor contesta *“de acuerdo, y yo también quiero hablar contigo”*.
3. **ACK** (acknowledge): el cliente confirma *“recibido, hablemos”*.

A partir de ahí, **los datos fluyen en ambas direcciones**. Si cuentas los mensajes (3), tienes el nombre: **three-way handshake**. Es exactamente lo que ocurre dentro de tu `cliente.connect()` en el [punto 2](#).

Cierre de la conexión: la despedida

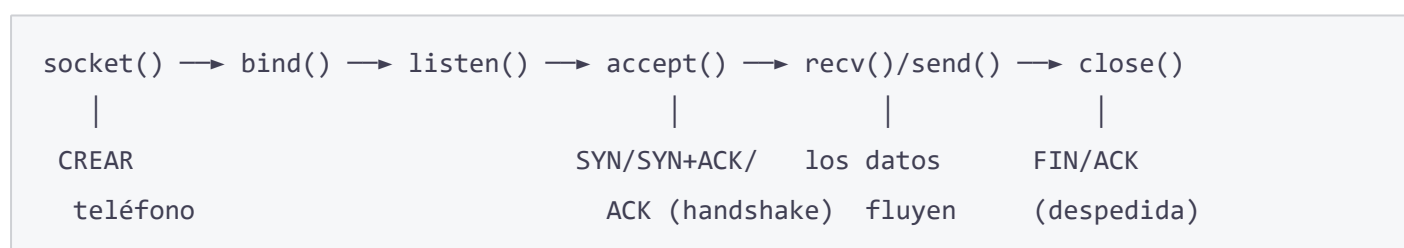
Cuando ya no hay más que decir, la conexión se cierra con un apretón de despedida:



1. Quien quiere cerrar envía **FIN** (*“no tengo más que enviar”*).
2. El otro lado responde **ACK** y, cuando también termina, envía su propio **FIN**.
3. El primero confirma con **ACK** y la conexión se libera.

💡 En Python no gestionas el cierre a mano: cuando el `with` termina (o llamas a `close()`), el SO ejecuta esta despedida por ti. Aunque lo parezca, **no es instantáneo**: el estado de cierre queda unos segundos en el sistema (eso te dará el problema de “Address already in use” del [punto 6](#)).

El ciclo completo, en una mirada



El **ciclo de vida de la conexión** es la historia completa entre la primera `socket()` y el último `close()`: preparar el teléfono, estrechar la mano, hablar y colgar.

Mini-chequeo

1. ¿Cuáles son los tres mensajes del handshake y en qué orden?
2. ¿Qué garantiza TCP sobre los datos, a cambio de ser más lento que UDP?
3. ¿Cuándo se cierra la conexión en Python y quién ejecuta la despedida?

>  Respuestas

Resumen en 3 frases

- Una conexión TCP arranca con el **three-way handshake** (SYN → SYN+ACK → ACK) antes de enviar datos.
- Se cierra con una despedida de **FIN/ACK** que el SO ejecuta al terminar el `with` o al llamar a `close()`.
- TCP garantiza **orden y sin pérdidas** a cambio de ser algo más lento que UDP.

Vocabulario rápido

Término	Idea general
Handshake	Saludo previo de TCP antes de enviar datos
SYN	“Quiero hablar contigo” (inicia la conexión)
SYN + ACK	“De acuerdo, y yo también”
ACK	Confirmación de recepción
FIN	“No tengo más que enviar” (inicia el cierre)
Orientado a conexión	TCP estrecha la mano antes de hablar

05 — Errores y gestión

Timeouts, reinicios y conexiones rechazadas bajo control 🏠

📖 Estás en: 🏠 U04 · Sockets TCP → 05 · Errores y gestión

📌 La idea en una frase

Las redes **fallan**, y tu programa debe saber qué hacer cuando lo hacen. Los dos aliados son el **try/except** (capturar el error y reaccionar) y los **timeouts** (no quedarse esperando para siempre).

En los [puntos 2](#) y [3](#) todo iba bien. En la vida real el servidor se cae, la red se corta o el mensaje tarda demasiado. Este punto te da la **red de seguridad** para que tu cliente (y tu servidor) no exploten ante lo inesperado.

🐍 La función conectar_seguro

```
import socket, time

def conectar_seguro(host, port, reintentos=3):
    for intento in range(reintentos):
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(5)
                s.connect((host, port))
                s.sendall(b"test")
                return s.recv(1024)
        except socket.timeout:
            print(f"🕒 Timeout (intento {intento+1})")
        except ConnectionRefusedError:
            print(f"🚫 Conexión rechazada – ¿el servidor está encendido?")
            time.sleep(1)
        except ConnectionResetError:
            print(f"💥 El servidor cerró la conexión abruptamente")
        except OSError as e:
            print(f"🚨 Error de red: {e}")
    return None
```

Repasemos qué hace cada pieza:

- **for intento in range(reintentos)** → reintenta hasta 3 veces antes de rendirse.
- **s.settimeout(5)** → si una operación tarda más de 5 segundos, lanza `socket.timeout` en lugar de quedarse bloqueada para siempre.
- **try/except por excepción** → cada error tiene su propio tratamiento, y el bucle sigue al siguiente intento.
- **return None** al final → si tras 3 intentos no hay respuesta, devuelve None y el llamador sabe que falló.

🧩 El mapa de excepciones

Excepción	Cuándo ocurre
<code>socket.timeout</code>	La operación excede el tiempo límite
<code>ConnectionRefusedError</code>	No hay nadie escuchando en ese puerto
<code>ConnectionResetError</code>	El otro lado cerró de golpe
<code>BrokenPipeError</code>	Escribes en un socket que ya se cerró
<code>OSError</code>	Red caída, DNS no resuelve, etc.

- **ConnectionResetError**: típico del servidor que muere mientras el cliente aún habla. Es un subtipo de `ConnectionError`.
- **BrokenPipeError**: el otro lado ya cerró y tú intentas `sendall()`. El SO “rompe la tubería”.
- **OSError** es la red de seguridad final: todo error de red cae aquí si no lo capturaste antes (también es la base de `ConnectionRefusedError`).

💡 **Regla práctica:** primero captura las excepciones específicas (`socket.timeout`, `ConnectionResetError`, `BrokenPipeError`) y deja `OSError` como última red.

🕒 Non-blocking y timeouts

`recv()` y `accept()` son bloqueantes: se quedan esperando. Tres formas de controlar esa espera:

```
import socket, select

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Opción 1: timeout fijo
s.settimeout(5.0)

# Opción 2: no bloqueante (lanza excepción si no hay datos)
s.setblocking(False)

# Opción 3: select (esperar con timeout en múltiples sockets)
lectura, _, _ = select.select([s], [], [], 1.0)
if lectura:
    datos = s.recv(1024)
```

- `settimeout(5.0)` → cada operación tiene 5 segundos; si se exceden, `socket.timeout`.
- `setblocking(False)` → nada bloquea: si no hay datos, la llamada lanza `BlockingIOError` al instante.
- `select.select([s], [], [], 1.0)` → espera hasta 1 segundo a que **alguno** de los sockets de la lista tenga datos. Si lo tiene, lectura no está vacía.

`select.select()` es la solución para esperar en **varios sockets a la vez sin hilos**: le pasas la lista y te avisa cuáles están listos. Lo usarás a fondo en el [TEMA 10 — Servidores Concurrentes](#).



Mini-chequeo

1. ¿Qué excepción lanza `recv()` si se pasa el timeout y qué excepción si escribes en un socket ya cerrado?
2. ¿Cuál es la diferencia entre `ConnectionRefusedError` y `ConnectionResetError`?
3. ¿Qué hace `select.select()` y cuándo conviene?

>  Respuestas



Resumen en 3 frases

- Las redes fallan: captura cada error con su propio `except` y deja `OSError` como red final.

- `settimeout()` evita que un `recv()` se quede bloqueado para siempre.
- `select()` permite esperar en varios sockets a la vez, sin hilos.



Vocabulario rápido

Término	Idea general
Timeout	Tiempo máximo de espera de una operación
ConnectionRefusedError	No hay nadie escuchando en el puerto
ConnectionResetError	El otro lado cerró la conexión de golpe
BrokenPipeError	Escribes en un socket ya cerrado
<code>setblocking(False)</code>	El socket no bloquea: lanza excepción si no hay datos
<code>select()</code>	Espera en varios sockets a la vez sin hilos

 [Volver al índice de la unidad](#) · **Anterior:** [04 · Ciclo de vida de la conexión](#) · **Siguiente:** [06 · SO_REUSEADDR](#)

06 — SO_REUSEADDR

Adiós al 'Address already in use' 

 Estás en:  U04 · Sockets TCP → 06 · SO_REUSEADDR

La idea en una frase

Si matas un servidor y lo reinicias rápido, el SO puede decir que la dirección **ya está en uso**. Una sola línea `—setsockopt(SO_REUSEADDR, 1)—` evita el error y te ahorra minutos de depuración.

El culpable se llama **TIME_WAIT** (lo viste al hablar del cierre en el [punto 4](#)): al cerrar, la conexión no se libera al instante. **SO_REUSEADDR** le dice al SO que puedes reutilizar ese puerto. Ponlo siempre en tus servidores.

El problema: “Address already in use”

```
# 1ª ejecución del servidor: todo bien
srv.bind(("127.0.0.1", 5000))      #  OK

# Matas el servidor con Ctrl+C y lo relanzas al instante...
srv.bind(("127.0.0.1", 5000))      #  OSError: [Errno 98] Address already in use
```

¿Por qué? Tras el cierre, la conexión entra en estado **TIME_WAIT** unos segundos: el SO mantiene reservado el par (IP, puerto) para asegurarse de que los últimos mensajes de la despedida (los FIN/ACK del [punto 4](#)) no queden huérfanos. Mientras tanto, nadie más puede hacer `bind()` a ese puerto.

La solución: SO_REUSEADDR

```
import socket

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 5000))
servidor.listen()
```

Una sola línea lo arregla todo:

Pieza	Significado
SOL_SOCKET	“Nivel de opciones” del propio socket
SO_REUSEADDR	La opción que permite reutilizar la dirección
1	Activa la opción (0 la desactivaría)

Con ella, el SO te deja hacer `bind()` al puerto aunque queden conexiones en `TIME_WAIT`. El servidor se puede **matar y reiniciar** sin esperar.

Pon esto siempre en tus servidores. Te ahorrarás minutos de depuración.

¿Por qué existe `TIME_WAIT` en primer lugar?

No es un capricho: la despedida TCP necesita tiempo para garantizar que los mensajes de cierre no lleguen tarde y revuelvan conexiones nuevas. Es **protección**, no burocracia. El problema es que, para un programador que reinicia su servidor mil veces al día, esa protección se convierte en un estorbo... hasta que aparece `SO_REUSEADDR`.

Mini-chequeo

1. ¿Qué error lanza el SO al relanzar un servidor sin `SO_REUSEADDR`?
2. ¿Qué tres piezas componen la llamada `setsockopt()` y qué significan?
3. ¿Por qué existe el estado `TIME_WAIT`?

>  Respuestas



Resumen en 3 frases

- Tras cerrar un servidor, el SO mantiene el puerto en **TIME_WAIT** y rechaza un nuevo `bind()` con “Address already in use”.
- `setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)` permite reutilizar la dirección y relanzar el servidor al instante.
- Es la primera línea que todo servidor TCP debería tener.



Vocabulario rápido

Término	Idea general
Address already in use	Error al hacer <code>bind()</code> a un puerto aún reservado
TIME_WAIT	Estado del SO que mantiene el puerto reservado unos segundos
SOL_SOCKET	Nivel de opciones del socket
SO_REUSEADDR	Permite reutilizar la dirección y el puerto
<code>setsockopt()</code>	Método para configurar opciones del socket

 [Volver al índice de la unidad](#) · Anterior: [05 · Errores y gestión](#) · Siguiente: [07 · Protocolos sobre TCP](#)

07 — Protocolos sobre TCP

HTTP hablado a pelo y el orden de los bytes 🌐

📖 Estás en: 🍷 U04 · Sockets TCP → 07 · Protocolos sobre TCP

📌 La idea en una frase

TCP es solo el **tubería**; los protocolos de aplicación deciden **qué** se escribe dentro. El rey de todos es **HTTP** (el que mueve la web) y, al hablar por un socket, hasta el **orden de los bytes** importa.

Hasta aquí TCP te aseguraba que los bytes llegaran intactos y en orden. Pero ¿qué bytes? Ahí entran los protocolos de aplicación: el texto **HTTP** para la web, **SMTP** para el correo, **FTP** para archivos. Todos viajan dentro de un socket TCP, y en este punto lo verás con tus propios ojos.

🌐 HTTP desde cero: el protocolo que mueve la web

HTTP es un protocolo de **texto**: mandas una petición en texto claro y recibes una respuesta también en texto. Para hablar con un servidor web solo necesitas un socket TCP y el texto correcto:

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("www.example.com", 80))
    s.sendall(b"GET / HTTP/1.1\r\nHost: www.example.com\r\nConnection: close\r\n\r\n")

    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

print(respuesta.decode()[:500])
```


La **petición** es solo texto:

```
GET / HTTP/1.1\r\n
Host: www.example.com\r\n
Connection: close\r\n
\r\n
```

Desglosemos:

Línea	Qué hace
GET / HTTP/1.1	Verbo HTTP (GET), recurso (/) y versión
Host: ...	Qué servidor quiere el recurso (imprescindible)
Connection: close	“Ciérrame la conexión cuando termines”
\r\n\r\n (línea en blanco)	Marca el final de las cabeceras

Y el bucle de lectura es el truco del [punto 2](#) en acción: `recv()` tantas veces como haga falta hasta que devuelva `b""` (el servidor cerró gracias a `Connection: close`), **concatenando** cada trozo.

💡 Esto es la base de la [U06 · APIs REST y HTTP](#): las librerías como `requests` hacen exactamente esto por debajo, pero ahora sabes el truco. HTTP sobre TCP también es un criterio de la [U05](#).

12 El orden de los bytes (byte ordering)

Cuando envías números (un puerto, una longitud, un ID) por la red, la pregunta es **en qué orden van sus bytes**. Dos máquinas pueden guardar los enteros “al revés” (little-endian o big-endian). Para no liarse, la red usa un estándar: **big-endian** (el byte más significativo primero), también llamado **network byte order**.

```
import struct

# Empaquetar un entero en network byte order ('!' = big-endian)
datos = struct.pack("!I", 5000)      # el puerto 5000 como 4 bytes
print(datos)                        # b'\x00\x00\x13\x88'

# Desempaquetarlo de vuelta
puerto = struct.unpack("!I", datos)[0]
print(puerto)                       # 5000
```

Código de formato	Significado
!	Network byte order (big-endian)
I	Entero sin signo de 4 bytes
H	Entero sin signo de 2 bytes

💡 El estándar de red manda en el cable: !I y !H son los códigos que verás en todos los protocolos que hablan números (NTP, DNS, TCP mismo). Si una máquina empaqueta en su propio orden y otra desempaqueta en el suyo, el número sale **capicúa cambiado**.

Mini-chequeo

1. ¿Qué dos cosas envía el cliente HTTP con `sendall()` y cómo sabe cuándo ha terminado de leer?
2. ¿Qué significa `\r\n\r\n` en una petición HTTP?
3. ¿Qué código de `struct` usa big-endian y para qué sirve?

>  Respuestas

Resumen en 3 frases

- TCP es la tubería; el **protocolo de aplicación** (HTTP, SMTP, FTP...) decide qué texto o bytes se escriben dentro.
- **HTTP es texto**: una petición GET con cabeceras, leída en bucle con `recv()` hasta el cierre.

- Los números se envían en **big-endian (network byte order)**, que en Python se fuerza con el prefijo `!` de `struct`.



Vocabulario rápido

Término	Idea general
HTTP	Protocolo de texto sobre TCP que mueve la web
GET	Verbo HTTP para pedir un recurso
Cabecera HTTP	Línea Nombre: valor de la petición o respuesta
Network byte order	Orden estándar de bytes en la red (big-endian)
struct	Módulo para empaquetar/desempaquetar números en bytes

 [Volver al índice de la unidad](#) · Anterior: [06 · SO_REUSEADDR](#) · Siguiente: [08 · Servidor eco completo](#)

08 — Servidor eco completo

Servidor + cliente eco, mano a mano y ejercicios resueltos 🛠️

📖 Estás en: 🏠 U04 · Sockets TCP → 08 · Servidor eco completo

La idea en una frase

La práctica clásica de TCP: un **servidor eco** que devuelve al cliente exactamente lo que recibe. Es el “Hola mundo” de los sockets y la base del laboratorio del [punto 9](#).

Junta todo lo aprendido: cliente ([punto 2](#)), servidor ([punto 3](#)) y `SO_REUSEADDR` ([punto 6](#)). Con dos terminales verás el protocolo completo en acción.

El servidor eco

```
import socket

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    print("Servidor eco escuchando en 127.0.0.1:9000")

    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        print(f"Recibido de {addr}: {datos.decode()}")
        conn.sendall(datos) # Eco
```

El truco está en la última línea: `conn.sendall(datos)` devuelve los **mismos bytes** que llegaron. Eco, literalmente.

El cliente que lo prueba

```
import socket

with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Prueba")
    print(cli.recv(1024).decode())
```

Salida del cliente (con el servidor en ejecución en otra terminal):

Prueba



Be the code, my friend — Mano a mano TCP

“Sé el servidor y el cliente. Cada paso, cada byte.”

● SERVIDOR

1. `socket(AF_INET, SOCK_STREAM) → srv`
2. `bind(("127.0.0.1", 5000))`
3. `listen()`
4. `accept() → espera...` ⌚

● CLIENTE

1. `socket(AF_INET, SOCK_STREAM) → cli`
2. `connect(("127.0.0.1", 5000))`
→ Three-way handshake TCP

● SERVIDOR

4. `accept()` devuelve `(conn, ("127.0.0.1", 54321))`
5. Esperando datos... ⌚

● CLIENTE

3. `sendall(b"Hola servidor!")`
4. Esperando respuesta... ⌚

● SERVIDOR

5. `recv(1024) → b"Hola servidor!"`
6. `print("Recibido: Hola servidor!")`
7. `sendall(b"Recibido: Hola servidor!")`

● CLIENTE

4. `recv(1024) → b"Recibido: Hola servidor!"`
5. `print("Respuesta: Recibido: Hola servidor!")`
6. `close()`

● SERVIDOR

8. `close()`

`accept()` y `recv()` son **bloqueantes**: el programa se queda esperando hasta que algo ocurra. Ese “espera... ⌚ ” es el corazón de la conversación.

Mini-chequeo

1. ¿Qué línea del servidor `eco` lo convierte en “eco”?
2. ¿Qué le pasa al servidor si el cliente llama antes de que llegue a `accept()`?

3. ¿Qué dos llamadas son bloqueantes en el mano a mano?

>  Respuestas

Aprieta el lápiz

1. **Eco server:** Crea un servidor que devuelva exactamente lo que recibe.
2. **Contador de letras:** El cliente envía una frase, el servidor responde con la cantidad de letras.
3. **Servidor hora:** El cliente se conecta y el servidor le devuelve la hora actual.
4. **Cliente con timeout:** Crea un cliente que intente conectar, y si no hay respuesta en 3s, muestre “Servidor no disponible”.

>  Soluciones

Resumen en 3 frases

- El servidor eco devuelve con `conn.sendall(datos)` exactamente lo que recibió con `recv(1024)`.
- En el mano a mano, `accept()` y `recv()` son los dos puntos bloqueantes que esperan al otro lado.
- Con el servidor en una terminal y el cliente en otra tienes el protocolo TCP completo funcionando.

Vocabulario rápido

Término	Idea general
Eco	El servidor devuelve lo mismo que recibe
<code>sendall(datos)</code>	La jugada del eco: reenviar los bytes recibidos
Bloqueante	<code>accept()</code> y <code>recv()</code> esperan hasta que algo ocurre
Terminal del servidor	Ventana donde el servidor queda escuchando

09 — Cierre: consolida lo aprendido

Sé el socket, laboratorio TCP y el cierre de la unidad 🗨️

📖 Estás en: 🏠 U04 · Sockets TCP → 09 · Cierre

Has terminado la teoría: socket, cliente, servidor, handshake, errores, `SO_REUSEADDR` y protocolos sobre TCP. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el socket

Eres el socket del servidor, recién creado con `socket(AF_INET, SOCK_STREAM)`. Tu misión: escuchar, aceptar y atender a quien llame.

¿Qué pasa?

1. Haces `bind(("127.0.0.1", 9000))`: reservas el puerto. **Ahora eres la centralita.**
2. Llamas a `listen()`: el SO ya sabe que esperas llamadas.
3. Te quedas en `accept()`... **esperas. Y esperas.** Eres bloqueante: no hay nada que hacer hasta que alguien llame.
4. Un cliente ejecuta `connect()` y el **three-way handshake** recorre la red (`SYN` → `SYN+ACK` → `ACK`). El timbre suena.
5. `accept()` despierta y devuelve (`conn`, `("127.0.0.1", 54321)`): un **nuevo socket** para esa conversación, con la dirección del cliente.
6. Llegas a `recv(1024)`... **esperas otra vez.** Cuando el cliente manda `b"Hola!"`, despiertas.
7. Contestas con `sendall()` y, al salir del `with`, el socket de la conversación se cierra con la despedida `FIN/ACK`.
8. Y tú, el socket servidor, sigues en tu `accept()`: **esperando al siguiente cliente.**

Todo el tiempo, el SO ha hecho el trabajo sucio: handshakes, despedidas, orden de bytes.

💡 **Ahora tú:** ¿y si el cliente llama mientras sigues en el paso 6 atendiendo a otro? La conexión queda **en la cola** de `listen()` (el *backlog*), esperando tu próximo `accept()`. Eso es lo que viste en el [punto 3](#): un solo hilo atiende a un cliente cada vez.



Fireside Chat: Servidor vs Cliente

El ring de los conceptos: dos sockets discuten quién hace el trabajo duro.

Servidor: — Yo soy el que hace todo el trabajo. Escucho, acepto, atiendo... todo el peso recae sobre mí.

Cliente: — ¿Trabajo dices? Yo soy el que inicia todo. Si no fuera por mí, tú estarías ahí escuchando en el puerto para siempre, como una planta.

Servidor: — Pero tengo que gestionar múltiples conexiones, mantener el estado, no caerme... Tú solo te conectas, mandas algo y te vas.

Cliente: — Y también gestiono errores: ¿y si el servidor no está? ¿y si hay timeout? ¿y si la red falla? No es tan sencillo.

Servidor: — Vale, vale. En realidad somos un equipo. Sin servidor no hay servicio, pero sin cliente no hay razón para existir.

Moraleja: servidor y cliente son dos caras de la misma moneda. El protocolo (quién envía qué y cuándo) es el verdadero protagonista. En la [U05](#) conocerás al otro protagonista: el UDP, el avión de papel.



¿Quién soy?

1. Soy el punto final de una conexión de red: la interfaz para enviar y recibir datos.
2. Soy el método del cliente que estrecha la mano antes de hablar.
3. Soy el método del servidor que se queda esperando a que alguien llame.
4. Soy el estado que mantiene el puerto reservado unos segundos tras cerrar.
5. Soy la opción que evita el “Address already in use”.
6. Soy el método que devuelve al cliente exactamente lo que recibió: eco.

>  Respuestas



CONRAD VS EL MUNDO: “conexión reiniciada por el host”

CONRAD: — “Clásico: el cliente hace `recv()` y le salta *‘ConnectionResetError’*. Pues claro. Razones: 1) **El servidor se cayó** en mitad de la conversación y su SO mandó un RST. 2) **Cerraste el servidor con Ctrl+C** mientras el cliente hablaba: misma historia. 3) El cliente intentó **escribir en un socket ya cerrado** → `BrokenPipeError`. 4) O el **timeout** se te pasó: `recv()` bloqueado para siempre porque nadie respondió.”

CONRAD: — “Y lo mejor: *‘pero yo hacía `sendall` y me daba error’*. ¡Pues claro! `sendall()` no avisa: es el SO quien lanza la excepción cuando la otra punta de la tubería ya no existe. Captura `ConnectionResetError` y `BrokenPipeError` por separado, como viste en el [punto 5](#), y tu cliente dejará de morir a lo loco.”

CONRAD: — “Y no me vengas con *‘¿será que la red va lenta?’*. Si el servidor se reinició y volvió a arrancar, **sin `SO_REUSEADDR`** te salta *Address already in use* al instante. Tres errores, tres causas, tres soluciones: reintentos con `try/except`, `settimeout()`, y la línea mágica de `setsockopt`. A diagnosticar.”

Laboratorio de tortura: cliente y servidor TCP

Duración: 45 minutos **Herramienta:** Python 3 (socket, sin instalar nada) + dos terminales

Escenario: construye un servidor eco TCP y un cliente que le mande mensajes, exactamente como en el [punto 8](#).

Tareas paso a paso:

1. **Escribe el servidor eco** (`servidor_eco.py`): `bind()` en `127.0.0.1:9000`, `listen()`, `accept()` y un bucle que haga `recv()` y responda con `sendall(datos)`. Imprime cada mensaje con su dirección.
2. **Escribe el cliente** (`cliente_eco.py`): pide un mensaje con `input()`, lo envía con `sendall()` y muestra la respuesta de `recv()`.
3. **Arranca el servidor** en una terminal y **el cliente** en otra. Envía tres mensajes seguidos y comprueba que el eco funciona y que el servidor ve la dirección de cada uno.
4. **Añade el contador:** haz que el servidor responda `"Eco #1: ..."`, `"Eco #2: ..."` llevando la cuenta de los mensajes recibidos.
5. **Añade `settimeout(3)`** al cliente y comprueba qué pasa si cierras el servidor antes de enviar.

Fallo intencionado: cierra el servidor con Ctrl+C y **relánzalo al instante**. ¿Qué pasa? Sin `SO_REUSEADDR`, el `bind()` falla con *“Address already in use”* porque las conexiones anteriores siguen en **TIME_WAIT**. Añade `servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` antes del `bind()` y comprueba que ahora sí puedes reiniciar sin esperar.

Pista 1: el estado `TIME_WAIT` es el culpable de que el puerto no se libere al instante. Con `SO_REUSEADDR` activado (como en el [punto 6](#)), el SO te deja reutilizar la dirección aunque queden conexiones en ese estado.

Pista 2: si el cliente se queda colgado en `recv()`, ese es el síntoma clásico de “el servidor nunca respondió”. Añade `cli.settimeout(3)` y verás la excepción `socket.timeout` aparecer a los 3 segundos, confirmando que la respuesta nunca llegó.

Logros de esta unidad

Logro	Cómo conseguirlo
 Telefonista	Crear el primer socket TCP con <code>socket(AF_INET, SOCK_STREAM)</code>
 Marca-Números	Implementar un cliente con <code>connect()</code> , <code>sendall()</code> y <code>recv()</code>
 Centralita	Implementar un servidor con <code>bind()</code> , <code>listen()</code> y <code>accept()</code>
 Handshaker	Explicar el three-way handshake y el cierre FIN/ACK
 Eco Master	Montar un servidor + cliente eco en dos terminales
 Fénix TCP	Dominar <code>SO_REUSEADDR</code> y reiniciar servidores sin error

Atrévete a pensar

1. ¿Por qué el servidor hace `accept()` y no `recv()` directamente?
2. ¿Qué pasaría si dos programas intentan `bind()` al mismo puerto a la vez?
3. ¿Cuándo conviene `select()` frente a un `while True` con `accept()`?
4. ¿Por qué HTTP necesita saber cuándo acaba la respuesta si TCP ya “sabe” cuándo acaban los datos?
5. ¿Qué diferencia hay entre cerrar con `with` y dejar el socket sin cerrar?

🧩 Crucigrama de bits

Horizontal:

1. Método del cliente TCP que establece la conexión (7 letras)
4. Paquete que inicia el three-way handshake (3 letras)
6. Opción que evita "Address already in use" (12 letras)
8. Método del servidor que espera un cliente (6 letras)

Vertical:

2. Método que envía todos los bytes de golpe (8 letras)
3. Dirección de tu propia máquina (9 letras)
5. Estado que mantiene el puerto reservado al cerrar (9 letras)
7. Método de eco: reenvía lo recibido (8 letras)

💬 Entrevista de trabajo

1. "¿Qué es un socket? ¿Qué papel juegan la IP y el puerto?"
2. "Escribe un servidor TCP que reciba un mensaje y lo devuelva."
3. "¿Cómo funciona el three-way handshake? Explícalo con un diagrama."
4. "¿Qué errores pueden ocurrir al comunicar por sockets y cómo los gestionas?"
5. "¿Qué es SO_REUSEADDR y cuándo lo necesitas?"

💡 **Cómo encararlas:** la 2 y la 3 son las "preguntas reina". Para la 2, escribe el servidor del [punto 3](#) sin pensarlo: `socket()` + `bind()` + `listen()` + `accept()` + `recv()` + `sendall()`. Para la 3, dibuja el `SYN` → `SYN+ACK` → `ACK` sobre los dos extremos y cuenta por qué hace falta el tercer mensaje. Si sabes contarlos fluidos, ya eres medio desarrollador de redes.

🧑 No hay preguntas tontas

❓ ¿Qué pasa si el servidor no llama a `listen()`?

El cliente recibe `ConnectionRefusedError`. Sin `listen()`, no hay puerto abierto.

? ¿Y si no llamo a `bind()`?

Para el servidor es obligatorio (necesita un puerto fijo). Para el cliente, el SO asigna uno automáticamente.

? ¿`recv(1024)` significa que solo puedo recibir 1024 bytes?

No, es el tamaño máximo del buffer. Si el mensaje es más grande, necesitas varios `recv()` y concatenar. Tú tienes que implementar el protocolo de aplicación para saber cuándo has recibido el mensaje completo.

? ¿Puedo tener más de un cliente conectado a la vez?

Con el código básico, no. Solo acepta un cliente cada vez. Para múltiples clientes, mira el **TEMA 10 — Servidores Concurrentes**.

? ¿Qué es “127.0.0.1”?

Es **localhost** — tu propia máquina. Perfecto para pruebas. Para que otros se conecten, usa tu IP real (ej: 192.168.1.x).

? ¿Y si un cliente envía datos muy seguido?

TCP los encola. El servidor los recibe en orden. Pero si el cliente envía más rápido de lo que el servidor lee, el buffer se llena y el cliente se bloquea.

Poscréditos

Un socket TCP estrecha la mano. SYN, SYN+ACK, ACK. La llamada está en marcha.

El cliente habla; el servidor escucha; el eco vuelve exactamente igual que llegó.

El servidor se cae y renace. Esta vez, `SO_REUSEADDR` le deja volver al instante.

Y en una terminal lejana, un `recv()` espera paciente a que el mundo le envíe algo que leer.

PRÓXIMAMENTE EN U05: Sockets UDP. La carta certificada se convierte en avión de papel: sin handshake, sin confirmación, sin orden. Y con él, HTTP hablado a pelo y el reloj de Internet (NTP).

Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación (HTTP, NTP).

CE	Criterio	Cubierto
a)	Modelo de capas de red (TCP/IP)	✓ Punto 1 + Fireside Chat
c)	Crea servidores TCP	✓ Puntos 3 y 8 + ⚡ Laboratorio de tortura
d)	Crea clientes TCP	✓ Puntos 2 y 8 + ⚡ Laboratorio de tortura
f)	Gestiona errores de red	✓ Punto 5 + ⚡ Laboratorio con fallo intencionado
g)	Configura opciones de socket (SO_REUSEADDR, non-blocking)	✓ Puntos 5 y 6 + ⚡ Laboratorio de tortura

RA3b (UDP), RA3e (UDP servidor/cliente) y RA3h (protocolos HTTP/NTP) se cubren en la **U05 · Sockets UDP y Protocolos**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Servidor eco completo](#) · Siguiente: [U05 · Sockets UDP y Protocolos](#)

U05 — Sockets UDP y Protocolos

Datagramas sin conexión: UDP, HTTP y NTP 📡

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🍷 TCP → 📡 **Sockets UDP y Protocolos** → 🌐 API REST → 🧪 APIs comerciales → 🗝 Hash → 🧬 Cifrado → ⌚ Servidores → ⌚ asincio

“UDP es como lanzar un avión de papel. TCP es como enviar una carta certificada. Cada uno tiene su momento.”

En la U04 los sockets TCP te enseñaron a ser un mensajero certificado: conectas, verificas entrega y mantienes orden. Aquí llega el otro extremo: **UDP**, el lanzador de aviones de papel. No hay conexión, no hay confirmación, no hay orden: mandas tu datagrama y rezas por que llegue. Y, sin embargo, es el protocolo que mueve las videollamadas, los juegos online, el DNS y hasta la hora exacta de Internet.

Esta unidad cierra el bloque de red con la otra cara de la moneda y sus dos grandes protocolos de aplicación sobre sockets: **HTTP** (el que mueve la web) y **NTP** (el que sincroniza los relojes). Entenderás cuándo la velocidad importa más que la fiabilidad, cómo se construye una petición HTTP a mano con un socket, y por qué tu ordenador sabe qué hora es sin tener un reloj atómico. El siguiente tema, [U06 · APIs REST y HTTP](#), tomará el relevo para automatizar todo esto con la librería requests.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar por qué UDP **no establece conexión** y qué implica (sin garantía de entrega ni de orden).
- Comparar **TCP y UDP** con criterio: fiabilidad contra velocidad, y elegir cuál usar según el caso.
- Implementar un **cliente UDP** con `sendto()` y `recvfrom()`, sin `connect()`.
- Implementar un **servidor UDP** con `bind()`, sin `accept()` ni `listen()`, y responder a la dirección del cliente.

- Describir cómo los datagramas pueden **perderse, duplicarse o llegar desordenados**, y por qué UDP lo asume.
- Construir un **cliente HTTP manual** hablando el protocolo a pelo con un socket TCP.
- Obtener la hora oficial de Internet con un cliente **NTP** (puerto 123) usando UDP.
- Decidir, con casos reales (DNS, VoIP, streaming, juegos, web), **cuándo usar TCP y cuándo UDP**.
- Montar y depurar un **servidor + cliente eco UDP** completo.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · TCP vs UDP	La carta certificada contra el avión de papel: fiabilidad vs velocidad	Todos
02 · Cliente UDP	Enviar y recibir sin conexión: <code>sendto()</code> y <code>recvfrom()</code>	Todos
03 · Servidor UDP	<code>bind()</code> , recibir datagramas y saber de quién vienen	Todos
04 · Datagramas y pérdida	Tamaño, paquetes perdidos, duplicados y orden no garantizado	Todos
05 · HTTP desde cero	El protocolo que mueve la web, hablado a pelo con un socket	Todos
06 · NTP y servidores de tiempo	Sincronizar relojes con UDP y el puerto 123	Todos
07 · Cuándo usar cada protocolo	Criterios de decisión y casos reales (DNS, VoIP, streaming)	Todos
08 · Práctica eco UDP	Servidor + cliente eco completos y ejercicios resueltos	Todos
09 · Cierre	Sé el datagrama, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver

✓ Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación (HTTP, NTP).

CE	Criterio	Dónde se cubre
a)	Modelo de capas de red (TCP/IP)	✓ Punto 1
b)	Identifica tipos de sockets (TCP/UDP)	✓ Puntos 1-4
e)	Implementa servidores y clientes UDP	✓ Puntos 2, 3 y 8 + ⚡ Laboratorio (punto 9)
h)	Implementa protocolos de aplicación (HTTP, NTP)	✓ Puntos 5 y 6

RA3c (servidor TCP), RA3d (cliente TCP), RA3f (errores) y RA3g (opciones) se cubren en la **U04 · Sockets TCP**.

¿Por dónde empiezo?

¿Vienes de la U04 y dominas los sockets TCP? Perfecto, ese es el trampolín ideal: repasa la [U04 · Sockets TCP](#) para tener frescos `accept()`, `listen()`, `connect()` y el three-way handshake, y arranca en el [punto 1](#), que parte justo de ahí: TCP contra UDP.

¿Ya sabes qué es UDP y solo necesitas NTP, el HTTP manual o la práctica final? Ve directo a los [puntos 5](#), [6](#) y [8](#). Si vienes de cero en sockets, no te saltes los puntos 1 a 4: los conceptos de datagrama y conexión son la base de todo lo demás.

📌 Primer punto: [01 · TCP vs UDP](#)

▶▶ Al acabar la unidad, continúa en [U06 · APIs REST y HTTP](#).

01 — TCP vs UDP

La carta certificada contra el avión de papel 🚀

📖 Estás en: 📡 U05 · Sockets UDP y Protocolos → 01 · TCP vs UDP

📌 La idea en una frase

UDP **no establece conexión**: mandas el mensaje y rezas porque llegue. No hay garantía de entrega ni de orden. TCP, en cambio, es el mensajero certificado: entrega cada carta, en orden, y si se pierde, la reenvía. Cada uno tiene su momento.

En la U04 conociste a TCP a fondo: handshake, retransmisiones, orden garantizado. Aquí llega su antagonista: **UDP** (User Datagram Protocol), el lanzador de aviones de papel. Más simple, más rápido, y sin ninguna promesa. La gracia está en saber cuándo usar cada uno.

✈️ La analogía: carta certificada contra avión de papel

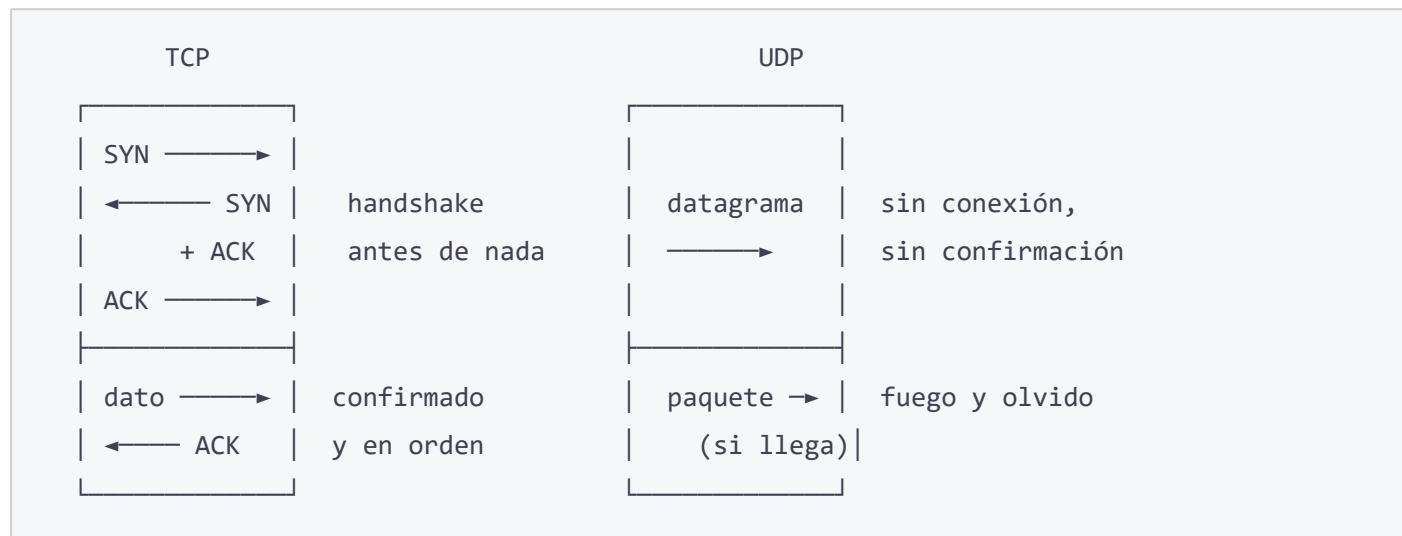
- **TCP** es la **carta certificada**: pagas más, el cartero te confirma la entrega, y si se pierde, se reenvía. Sabes que llegó, y en el orden correcto.
- **UDP** es el **avión de papel** desde el balcón: lo lanzas y olvidas. Si aterriza, bien; si el viento se lo lleva, también. Pero puedes lanzar cien aviones en el tiempo que tardas en preparar una carta certificada.

Esas dos filosofías definen el resto de la unidad: **fiabilidad contra velocidad**.

⚖️ TCP vs UDP cara a cara

Característica	TCP	UDP
Conexión	Sí (handshake)	No
Entrega garantizada	Sí	No
Orden	Sí	No

Característica	TCP	UDP
Velocidad	Más lento	Más rápido
Uso típico	Web, correo, FTP	Streaming, juegos, DNS



La columna de la derecha es la que te ocupará en esta unidad: tres filas de “No” que, paradójicamente, son la razón de que UDP sea tan rápido.

El socket en código

La diferencia se ve en una sola línea: el tipo de socket.

```
import socket

# Socket UDP: datagramas sin conexión
s_udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Socket TCP: flujo fiable con conexión (lo viste en la U04)
s_tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

- SOCK_DGRAM → datagramas (UDP): cada sendto() es un mensaje independiente.
- SOCK_STREAM → flujo de bytes (TCP): todo va por el mismo canal, conectado y ordenado.

💡 Recuerda el truco para no liarte: **DGRAM** = datagramas sueltos (**UDP**); **STREAM** = flujo continuo (**TCP**).



Fiabilidad vs velocidad: el verdadero dilema

No existe “el mejor” protocolo: existe el adecuado para cada momento.

- **Elige TCP cuando no puedes permitirte perder ni un byte:** una web, un correo, una transferencia de archivos. Si falta un trozo, el documento llega roto.
- **Elige UDP cuando prefieres estar al día antes que completo:** una videollamada, un partido online, una transmisión en vivo. Mejor un frame ligeramente desfasado que esperar eternamente a que se reenvíe el anterior.

Ese razonamiento lo profundizarás en el [punto 7](#), con casos reales. De momento, quédate con la tabla y la analogía: son el corazón de la unidad.



Mini-chequeo

1. ¿Qué garantiza TCP que UDP no garantiza? (tres cosas)
2. ¿Por qué UDP es más rápido que TCP?
3. ¿Cuál usarías para una videollamada y cuál para una transferencia de archivos?

>  Respuestas



Resumen en 3 frases

- UDP no establece conexión y no garantiza entrega ni orden; TCP sí, a costa de ser más lento.
- TCP es la carta certificada; UDP, el avión de papel: fiabilidad contra velocidad.
- En código se distinguen en el tipo de socket: `SOCK_STREAM` (TCP) frente a `SOCK_DGRAM` (UDP).



Vocabulario rápido

Término

Idea general

TCP

Protocolo con conexión, fiable y ordenado (`SOCK_STREAM`)

UDP

Protocolo sin conexión, rápido y sin garantías (`SOCK_DGRAM`)

Término	Idea general
Handshake	Saludo previo de TCP antes de enviar datos
Datagrama	Paquete independiente que manda UDP
SOCK_DGRAM	Constante de Python para crear sockets UDP

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · Cliente UDP](#)

02 — Cliente UDP

Enviar y recibir sin conexión: `sendto()` y `recvfrom()` 🚢

📖 Estás en: 🚢 U05 · Sockets UDP y Protocolos → 02 · Cliente UDP

🚢 La idea en una frase

Un cliente UDP **no se conecta**: crea el socket, manda un datagrama con `sendto()` a una dirección y espera la respuesta con `recvfrom()`. Nada de `connect()`, nada de handshake.

En la U04, el cliente TCP pasaba por `connect()` para estrechar la mano antes de hablar. UDP no tiene mano que estrechar: tu mensaje y la dirección del destinatario se envían **juntos**, en el mismo `sendto()`. Es el primer protocolo de esta unidad en el que comprobarás que “sin conexión” se nota hasta en la API.

🚢 Enviar sin conectarse: `sendto()`

El método clave del cliente UDP es `sendto(datos, dirección)`:

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(b"Hola UDP!", (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

Desglose de cada línea:

- `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` → socket UDP (el `SOCK_DGRAM` del [punto 1](#)).
- `cliente.sendto(b"Hola UDP!", (HOST, PORT))` → envía el datagrama a `127.0.0.1:5001`. La dirección viaja **en la llamada**, no en un `connect()` previo.

- `cliente.recvfrom(1024)` → espera la respuesta; devuelve una tupla (datos, dirección). Como aquí no nos importa de quién viene, la dirección se descarta con `_`.
- `datos.decode()` → los bytes recibidos se convierten a texto para poder imprimirlos.

Sendto contra send: la diferencia visual

Cliente TCP (U04)	Cliente UDP (aquí)
<code>s.connect((HOST, PORT))</code>	— (no existe)
<code>s.send(datos)</code>	<code>s.sendto(datos, (HOST, PORT))</code>
<code>s.recv(1024)</code>	<code>s.recvfrom(1024)</code>

En TCP primero te conectas y luego envías; en UDP el “a quién” va dentro del envío. Y como no hay conexión, **tampoco hay `close()` explícito que cierre el canal**: el `with` se encarga de liberar el socket.

⚠ En UDP, cada `sendto()` es un datagrama independiente. Pueden llegar **desordenados, duplicados o no llegar**. La API te lo pone fácil; la red, no tanto. Eso lo verás en detalle en el [punto 4](#).

Be the code: cliente UDP paso a paso

“Sé el cliente UDP y manda tu primer avión de papel.”

```
cliente.sendto(b"Hola UDP!", ("127.0.0.1", 5001))
```

1. `socket.socket(AF_INET, SOCK_DGRAM)` → crea el socket sin conexión
2. `sendto(b"Hola UDP!", ("127.0.0.1", 5001))`
 - empaqueta datagrama: bytes + dirección destino
 - el SO lo lanza a la red en un solo paquete
3. `recvfrom(1024)` → se queda escuchando hasta que llegue algo
 - devuelve (datos, dirección): datos = respuesta del servidor
4. `print(f"Respuesta: {datos.decode()}")` → "Respuesta: Recibido!"

Fíjate en el paso 2: **dirección y datos van juntos**. Si el servidor no está escuchando, no pasa nada raro: el datagrama se pierde en el vacío y el cliente se queda esperando en el `recvfrom()`

para siempre (a menos que pongas un `settimeout()`, que verás en el [punto 6](#)).

Mini-chequeo

1. ¿Qué dos métodos usa el cliente UDP y qué devuelve cada uno?
2. ¿Por qué no hay `connect()` en un cliente UDP?
3. ¿Qué pasa si envías un datagrama y no hay nadie escuchando?

>  Respuestas

Resumen en 3 frases

- El cliente UDP manda datagramas con `sendto(datos, dirección)` sin necesidad de `connect()`.
- La respuesta llega con `recvfrom()`, que devuelve los datos y la dirección del remitente.
- Como no hay conexión, cada datagrama es independiente: puede perderse, duplicarse o llegar desordenado.

Vocabulario rápido

Término	Idea general
<code>sendto()</code>	Envía un datagrama con su dirección destino
<code>recvfrom()</code>	Recibe un datagrama y devuelve datos + dirección
<code>SOCK_DGRAM</code>	Tipo de socket UDP
Sin conexión	No hay handshake: mandas y punto
<code>_</code>	Variable de Python para descartar un valor

 [Volver al índice de la unidad](#) · Anterior: [01 · TCP vs UDP](#) · Siguiente: [03 · Servidor UDP](#)

03 — Servidor UDP

Bind, recibir datagramas y saber de quién vienen 📡

📖 Estás en: 📡 U05 · Sockets UDP y Protocolos → 03 · Servidor UDP

📌 La idea en una frase

Un servidor UDP se limita a `bind()` en un puerto y a esperar datagramas con `recvfrom()`. **Sin `accept()`, sin `listen()`**: cualquiera que conozca la dirección puede hablarle, y él descubre de quién viene cada mensaje.

El servidor TCP de la U04 pasaba por `listen()` y `accept()`: creaba una conexión dedicada por cada cliente. UDP no tiene conexiones que aceptar: es como estar en un patio y escuchar quién te llama. El servidor **no se entera de quién llegará**, pero cuando recibe un datagrama, `recvfrom()` le dice exactamente de dónde vino.

📌 El servidor más pequeño del mundo

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor UDP en {HOST}:{PORT}")

    # Recibir datagrama (recvfrom devuelve datos + dirección)
    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")

    # Responder
    servidor.sendto(b"Recibido!", direccion)
```

Repasemos qué hace cada pieza:

- `servidor.bind((HOST, PORT))` → “reserva” el puerto 5001: a partir de ese momento, cualquier datagrama que llegue a 127.0.0.1:5001 es para este proceso. Sin `bind()`, el socket no escucha en ningún sitio.
- `datos, direccion = servidor.recvfrom(1024)` → se bloquea esperando un datagrama. Cuando llega, devuelve **dos cosas**: el contenido (datos) y la **dirección del cliente** (`direccion`), que es una tupla (IP, puerto).
- `servidor.sendto(b"Recibido!", direccion)` → responde al cliente usando precisamente esa dirección. Es la única forma de “devolver la pelota” en UDP: no hay conexión que recuerde quién eras.

❌ Sin `accept()`, sin `listen()`: la diferencia con TCP

Servidor TCP (U04)	Servidor UDP (aquí)
<code>bind()</code>	<code>bind()</code>
<code>listen()</code>	— (no existe)
<code>accept()</code> → devuelve conn	— (no existe)
<code>conn.recv(1024)</code>	<code>servidor.recvfrom(1024)</code>
(la conexión sabe quién es)	<code>recvfrom()</code> devuelve la dirección

En TCP, `accept()` crea una **conexión dedicada** y a partir de ahí todo va por ese canal. En UDP **no hay canal**: cada datagrama llega por su cuenta y el servidor debe mirar la dirección para saber a quién responder.

💡 Esto tiene una consecuencia importante: un solo servidor UDP puede atender a **cualquier número de clientes** sin `accept()` ni hilos, porque cada mensaje trae su dirección pegada. Eso lo explotarás en la práctica del [punto 8](#).

🧠 Mini-chequeo

1. ¿Qué método es imprescindible para que el servidor UDP escuche en un puerto?
2. ¿Qué devuelve `recvfrom()` y para qué sirve la segunda parte?
3. ¿Por qué el servidor UDP no necesita `accept()`?

Resumen en 3 frases

- El servidor UDP escucha con `bind()` y recibe datagramas con `recvfrom()`, sin `accept()` ni `listen()`.
- `recvfrom()` devuelve datos y dirección del cliente: con ella se responde usando `sendto()`.
- Al no haber conexiones, un mismo servidor atiende a cualquier número de clientes sin estructuras de conexión.

Vocabulario rápido

Término	Idea general
<code>bind()</code>	Reserva un puerto para escuchar datagramas
<code>recvfrom()</code>	Recibe un datagrama: datos + dirección del emisor
dirección	Tupla (IP, puerto) que identifica al cliente
<code>sendto()</code>	Responde al cliente con su dirección
Puerto reservado	El <code>bind()</code> marca el puerto donde el SO entrega los datagramas

 [Volver al índice de la unidad](#) · Anterior: [02 · Cliente UDP](#) · Siguiente: [04 · Datagramas y pérdida](#)

04 — Datagramas y pérdida

Paquetes perdidos, duplicados y sin orden: la vida real de UDP 🛠️

📖 Estás en: 🗺️ U05 · Sockets UDP y Protocolos → 04 · Datagramas y pérdida

📌 La idea en una frase

En UDP, cada `sendto()` es un **datagrama independiente**: puede llegar desordenado, duplicado o **no llegar**. No hay confirmación de recepción, así que si un paquete se pierde, se pierde para siempre.

Este es el punto donde aterrizas las consecuencias del “No” de la tabla del [punto 1](#). El cliente UDP es trivial de escribir (lo viste en el [punto 2](#)); lo que no es trivial es lo que pasa en el camino.

🔊 El datagrama: cada envío es un mundo aparte

En UDP, cada `sendto()` es un datagrama independiente. Pueden llegar **desordenados, duplicados o no llegar**.

Imagina que mandas 5 mensajes seguidos desde un cliente UDP:

```
import socket, time

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    for i in range(1, 6):
        cliente.sendto(f"Mensaje #{i}".encode(), ("127.0.0.1", 5001))
        time.sleep(0.2)
```

El servidor podría recibirlos en **cualquier orden** (o no recibir alguno). La red no promete nada: cada datagrama viaja por su cuenta, como aviones de papel lanzados desde el mismo balcón pero que el viento puede separar.

🚒 Los tres males de UDP

1. Pérdida de paquetes 🦴

No hay confirmación de recepción. Si un paquete se pierde, **se pierde para siempre**: UDP no reenvía nada. En una videollamada es un frame que se salta; en una transferencia de archivos sería un trozo irrecuperable.

2. Orden no garantizado

Los datagramas pueden llegar en distinto orden al que se enviaron. Sin números de secuencia, el receptor no tiene forma de reordenarlos. Por eso los protocolos que necesitan orden (web, correo) usan TCP.

3. Duplicados

En condiciones raras (retransmisión en capas inferiores, rutas paralelas), el mismo datagrama puede llegar **dos veces**. UDP no lo detecta: dos mensajes idénticos son simplemente dos mensajes.

```
Envío:      1 → 2 → 3 → 4 → 5
              | (se pierde el 3)
Recepción:  1 → 2 → 4 → 5      ← el 3 jamás llega

Otro caso:  1 → 2 → 2 → 3      ← el 2 duplicado
```

¿Por qué alguien elegiría esto?

Porque esos tres males son el **precio de la velocidad**: sin handshake, sin confirmaciones ni reenvíos, cada paquete cuesta el mínimo. Y hay aplicaciones donde perder un paquete molesta menos que esperar por él:

- Una videollamada: mejor un frame saltado que la conversación congelada esperando un reenvío.
- Un juego online: mejor 60 paquetes por segundo aunque se pierdan algunos que un “lag” por confirmar cada movimiento.
- DNS: una respuesta que no llega se repite; la siguiente valdrá igual.

La regla de decisión completa, con sus casos reales, la tienes en el [punto 7](#). Y cuando necesites fiabilidad **y** velocidad, la red ya inventó la solución intermedia: **QUIC**, un protocolo con las garantías de TCP pero funcionando sobre UDP (lo verás en el [punto 9](#)).



Mini-chequeo

1. ¿Qué tres cosas pueden pasarle a un datagrama en la red?
2. ¿Qué pasa si un paquete UDP se pierde? ¿Se reenvía?
3. ¿Por qué una videollamada prefiere UDP aunque pierda paquetes?

>  Respuestas



Resumen en 3 frases

- Cada `sendto()` de UDP es un datagrama independiente que puede perderse, duplicarse o llegar desordenado.
- No hay confirmación de recepción: un paquete perdido se pierde para siempre, nadie lo reenvía.
- Esos males son el precio de la velocidad, y aplicaciones como VoIP o juegos lo aceptan a cambio de latencia mínima.



Vocabulario rápido

Término	Idea general
Datagrama	Paquete independiente de UDP
Pérdida	Un datagrama que no llega y no se reenvía
Orden no garantizado	Los datagramas pueden llegar en distinto orden
Duplicado	El mismo datagrama llegando dos veces
Sin confirmación	UDP no avisa de la recepción (a diferencia de TCP)

 [Volver al índice de la unidad](#) · Anterior: [03 · Servidor UDP](#) · Siguiente: [05 · HTTP desde cero](#)

05 — HTTP desde cero

El protocolo que mueve la web, hablado a pelo con un socket 🌐

📖 Estás en: 📡 U05 · Sockets UDP y Protocolos → 05 · HTTP desde cero

📌 La idea en una frase

HTTP es un protocolo de **texto** sobre TCP: un cliente envía una petición escrita y el servidor responde con otra. Con un socket y un par de `sendall()/recv()` puedes hablarle a cualquier web del mundo.

Esta unidad es de UDP, pero HTTP merece su capítulo aquí: es el protocolo de aplicación que ya rozaste en la U04 (el servidor web del TEMA 02) y que en la [U06](#) automatizarás con requests. Ahora lo vas a hablar **a pelo**: a mano, carácter por carácter.

✉ La petición HTTP

Cuando tu navegador quiere una página, manda un texto como este:

```
GET /ruta HTTP/1.1\r\n
Host: ejemplo.com\r\n
User-Agent: MiNavegador\r\n
Accept: text/html\r\n
\r\n
```

Desglose:

- **GET /ruta HTTP/1.1** → la **primera línea**: verbo, recurso y versión del protocolo.
- **Host: ejemplo.com** → obligatoria en HTTP/1.1: dice a qué sitio preguntamos (un servidor puede alojar muchos).
- **User-Agent, Accept** → cabeceras opcionales que describen quién pregunta y qué quiere.
- **\r\n\r\n** → **línea en blanco final**: el “ya está, esto era todo”. Es la señal de que la petición terminó.

Los **verbos** principales:

Verbo	Qué pide	Ejemplo
GET	Leer un recurso	Obtener una página o imagen
POST	Enviar datos al servidor	Mandar un formulario
PUT	Guardar/reemplazar un recurso	Subir un archivo
DELETE	Borrar un recurso	Eliminar algo del servidor

La respuesta HTTP

El servidor contesta con otro texto: una línea de estado, cabeceras, y el cuerpo:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 1234\r\n
\r\n
<html>...cuerpo...</html>
```

- **HTTP/1.1 200 OK** → la primera línea: versión, **código de estado** y su significado. 200 es “todo bien”; también verás 404 Not Found, 500 Internal Server Error, etc.
- **Content-Type: text/html** → qué tipo de contenido viene a continuación.
- **Content-Length: 1234** → cuántos bytes mide el cuerpo, para saber cuándo se acabó.
- **\r\n\r\n** → otra vez la línea en blanco separando cabeceras de **cuerpo** (el HTML en sí).

 El código de estado es la frase corta que el navegador nunca te muestra: cuando ves “404”, estás leyendo la primera línea de esta respuesta.

Be the code, my friend: Cliente HTTP manual

“Sé el navegador. Olvida requests. Abre un socket y habla HTTP como en los viejos tiempos.”

```

import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # Conectar al servidor web
    s.connect(("www.example.com", 80))

    # Enviar petición HTTP/1.1
    petition = (
        "GET / HTTP/1.1\r\n"
        "Host: www.example.com\r\n"
        "Connection: close\r\n"
        "\r\n"
    )
    s.sendall(petition.encode())

    # Recibir respuesta
    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

    # Mostrar los primeros 500 caracteres
    print(respuesta.decode()[:500])

```

Traza:

1. `socket()` → crea socket TCP
2. `connect("www.example.com", 80)` → three-way handshake
3. Construye string HTTP: `"GET / HTTP/1.1\r\nHost: www.example.com\r\n..."`
4. `sendall` → envía bytes
5. Bucle `recv` → va recibiendo trozos de la respuesta
6. El servidor cierra conexión → `recv` devuelve `b""`
7. Decodifica bytes a string
8. Muestra `"<!doctype html>..."`  ¡es el HTML de la web!

Dos detalles importantes:

- **Connection: close** → le pide al servidor que cierre la conexión al terminar: así el bucle del paso 6 sabe cuándo parar (`recv()` devuelve `b""`).

- **Los bytes se acumulan** con respuesta += datos: la respuesta de una web no llega de una pieza, sino en trozos. El bucle los junta todos.

Este ejercicio lo convertirás en una herramienta de verdad en la [U06](#): parsear el JSON, los códigos de estado y las cabeceras de forma sistemática. De momento, ya has hablado HTTP como un navegador de verdad.

Mini-chequeo

1. ¿Qué significa la línea en blanco `\r\n\r\n` en una petición HTTP?
2. ¿Qué indica el código 200 de una respuesta?
3. ¿Por qué el bucle de `recv()` acumula bytes hasta que llega `b""`?

>  Respuestas

Resumen en 3 frases

- HTTP es un protocolo de texto sobre TCP: petición del cliente, respuesta del servidor, ambas con cabeceras y una línea en blanco final.
- La petición empieza con verbo + recurso + versión (GET /ruta HTTP/1.1); la respuesta con código de estado (200 OK).
- Con un socket TCP y `sendall()/recv()` puedes hablarle a cualquier web sin librerías: ese es el primer paso hacia las APIs de la U06.

Vocabulario rápido

Término	Idea general
HTTP	Protocolo de texto sobre TCP que mueve la web
GET	Verbo para pedir un recurso
200 OK	Código de estado: la petición fue bien
Cabecera	Línea Nombre: valor que describe petición o respuesta
<code>\r\n\r\n</code>	Línea en blanco que separa cabeceras de cuerpo

 [Volver al índice de la unidad](#) · **Anterior:** [04 · Datagramas y pérdida](#) · **Siguiente:** [06 · NTP y servidores de tiempo](#)

06 — NTP y servidores de tiempo

¿Qué hora es en Internet? Sincronizar relojes con UDP 🕒

📖 Estás en: 🏠 U05 · Sockets UDP y Protocolos → 06 · NTP y servidores de tiempo

📌 La idea en una frase

NTP (Network Time Protocol) usa UDP para sincronizar relojes: tu ordenador manda un pequeño datagrama al puerto 123 de un servidor de tiempo y recibe la hora exacta en la respuesta.

Curioso, ¿verdad? Justo cuando la exactitud importa, NTP usa el protocolo “que pierde paquetes”. La respuesta está en la estadística: NTP manda **muchas** peticiones y calcula la hora correcta de forma estadística. Si un paquete se pierde, no pasa nada: el próximo valdrá.

🕒 Por qué tu reloj no necesita ser atómico

Tu ordenador no tiene un reloj atómico, y aun así sabe qué hora es. La cadena es esta:

```
Tu ordenador — UDP → pool.ntp.org:123
Tu ordenador ← UDP — pool.ntp.org:123    (hora exacta)
```

- **pool.ntp.org** es un grupo de servidores de tiempo repartidos por el mundo; cualquier petición UDP al puerto **123** te devuelve la hora.
- NTP manda **múltiples peticiones** y cruza las respuestas para descontar el tiempo de viaje (latencia) y quedarse con la estimación más fiable.
- Por eso puede permitirse UDP: **un datagrama perdido es irrelevante** cuando manda muchos y promedia estadísticamente.

Así es como tu ordenador sabe la hora exacta sin tener un reloj atómico.

🐍 Cliente NTP manual en Python

El paquete NTP es un datagrama de **48 bytes** con un formato binario muy concreto. Lo mínimo para consultar la hora:

```
import socket, struct, time

def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        # Paquete NTP: 48 bytes, modo cliente
        paquete = b'\x1b' + 47 * b'\0'
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)

        # El timestamp está en los bytes 40-43
        t = struct.unpack('!I', datos[40:44])[0]
        # Ajustar época NTP (1900) a Unix (1970)
        return t - 2208988800

hora = hora_ntp()
print(f"Hora NTP oficial: {time.ctime(hora)}")
```

Desglose de cada pieza:

- `paquete = b'\x1b' + 47 * b'\0'` → el primer byte `\x1b` fija la **versión** (NTPv4, bits 3-5) y el **modo cliente** (bits 0-2: valor 3). Los 47 bytes restantes van a cero: es el relleno del formato de 48 bytes.
- `s.settimeout(5)` → muy importante en UDP: si la respuesta no llega en 5 segundos, la petición lanza una excepción en lugar de bloquearse para siempre (el fantasma del [punto 2](#)).
- `s.sendto(paquete, ("pool.ntp.org", 123))` → el cliente UDP del [punto 2](#) en acción, esta vez contra un servidor real en el puerto **123**.
- `struct.unpack('!I', datos[40:44])[0]` → el servidor escribe su hora como un entero sin signo de 4 bytes (!I) en las posiciones **40-43** del datagrama. struct lo desempaqueta a un número.
- `t - 2208988800` → el timestamp NTP cuenta segundos desde **1900**; Unix los cuenta desde **1970**. La diferencia exacta (2208988800) ajusta la época.

Salida:

```
Hora NTP oficial: Sat Aug 15 10:42:07 2026
```

SNTP: la versión ligera para dispositivos

SNTP (Simple Network Time Protocol) es la versión simplificada de NTP para dispositivos que no necesitan la precisión extrema de un servidor de tiempo: routers domésticos, móviles, IoT. Hace la misma consulta básica, pero sin los algoritmos complejos de estimación de NTP completo. Para el 99% de las aplicaciones (que la hora no se desvíe más de un segundo) es más que suficiente.

Mini-chequeo

1. ¿Por qué NTP usa UDP si la hora debe ser exacta?
2. ¿En qué puerto escuchan los servidores NTP y de qué tamaño es el paquete de consulta?
3. ¿Para qué sirve `s.settimeout(5)` en el cliente NTP?

>  Respuestas

Resumen en 3 frases

- NTP sincroniza relojes con UDP: un datagrama de 48 bytes al puerto 123 devuelve la hora exacta.
- Puede permitirse UDP porque promedia muchas peticiones: un paquete perdido es irrelevante.
- El timestamp viene en los bytes 40-43 del datagrama, se ajusta restando 2208988800 segundos de época, y `settimeout()` evita bloqueos eternos.

Vocabulario rápido

Término	Idea general
NTP	Protocolo que sincroniza relojes con UDP
pool.ntp.org	Grupo mundial de servidores de tiempo
Puerto 123	Puerto estándar de NTP
Época	Referencia de tiempo (NTP 1900, Unix 1970)

Término	Idea general
SNTP	Versión ligera de NTP para dispositivos sencillos

 [Volver al índice de la unidad](#) · **Anterior:** [05 · HTTP desde cero](#) · **Siguiente:** [07 · Cuando usar cada protocolo](#)

07 — Cuándo usar cada protocolo

Criterios de decisión y casos reales: DNS, VoIP, streaming 🌟

📖 Estás en: 📡 U05 · Sockets UDP y Protocolos → 07 · Cuándo usar cada protocolo

📌 La idea en una frase

No existe “el mejor protocolo”: existe la pregunta correcta. ¿Prefieres que llegue todo, o prefieres que llegue rápido? Según la respuesta, TCP o UDP.

Este punto junta todo lo anterior (la tabla del [punto 1](#) y los males del [punto 4](#)) para darte la regla de decisión que usarás de por vida como programador.

🌟 El criterio: velocidad o fiabilidad

```
¿Puedo permitirme perder datos?  
|  
├── NO ───────────> TCP  
|   (un byte perdido lo rompe todo:  
|   archivos, web, email, transacciones)  
|  
└── SÍ ───────────> UDP  
    (prefiero estar al día antes que completo:  
    voz, vídeo, juegos, DNS)
```

Dos preguntas que te ayudan a decidir:

1. ¿Es un dato “completo” o un “flujo en vivo”? Un archivo debe llegar entero (TCP); una conversación debe llegar fluida (UDP).
2. ¿Qué molesta más: que falte un trozo o que todo vaya lento? Si el trozo faltante es fatal, TCP; si el retraso es fatal, UDP.

📡 Casos reales: la tabla que se repite en las entrevistas

Aplicación	Protocolo	Por qué
Web (HTTP)	TCP	La página debe llegar completa y en orden
Correo (SMTP, POP3, IMAP)	TCP	Un mensaje a medias es inútil
Transferencia de archivos (FTP)	TCP	Un byte perdido corrompe el archivo
DNS	UDP	La consulta es diminuta; si se pierde, se repite
VoIP / videollamadas (Zoom, Skype)	UDP	La conversación debe ir fluida; un frame perdido se tolera
Streaming (Twitch, TV online)	UDP	Prefieres el directo fluido a un reenvío que lo congele
Juegos online (Fortnite, Minecraft)	UDP	Cientos de paquetes por segundo; la latencia manda
NTP (hora)	UDP	Manda muchas peticiones y promedia (viste el punto 6)

Fíjate en el patrón: **cuando el dato es valioso y debe quedar intacto, TCP; cuando lo valioso es el momento, UDP**. En el streaming de vídeo, por ejemplo, UDP es la opción clásica porque un frame roto se ve un instante y se olvida, pero un reenvío congelaría la emisión para todos.



Y si quieres ambas cosas: QUIC

¿Fiabilidad de TCP con velocidad de UDP? Eso existe: **QUIC**, el protocolo sobre el que funciona **HTTP/3**. QUIC se construye sobre UDP (la capa de transporte es UDP) pero añade por su cuenta confirmaciones, control de congestión y cifrado. Es la “vuelta a la tortilla” que verás en el cierre del [punto 9](#): la industria decide cuándo incluso el “no fiable” merece una capa de fiabilidad.



Mini-chequeo

1. ¿Qué criterio usarías para elegir entre TCP y UDP en una app nueva?

2. ¿Por qué DNS usa UDP si “perder” la respuesta de un dominio parece grave?
3. Pon dos ejemplos de TCP y dos de UDP distintos de los de la tabla.

> Respuestas

Resumen en 3 frases

- La regla de decisión es una sola pregunta: ¿prefieres que llegue todo (TCP) o que llegue rápido (UDP)?
- Los casos reales son consistentes: web/correo/FTP con TCP; DNS/VoIP/streaming/juegos con UDP.
- Si quieres ambas cosas, QUIC (sobre HTTP/3) construye fiabilidad encima de UDP.

Vocabulario rápido

Término	Idea general
Criterio de decisión	¿Fiabilidad o velocidad? Eso elige el protocolo
DNS	Directorio de Internet; usa UDP por su tamaño diminuto
VoIP	Voz sobre IP; usa UDP para ir fluida
Streaming	Emisión en vivo; prefiere fluidez a completitud
QUIC	Fiabilidad tipo TCP construida sobre UDP

 [Volver al índice de la unidad](#) · Anterior: [06 · NTP y servidores de tiempo](#) · Siguiente: [08 · Práctica eco UDP](#)

08 — Práctica eco UDP

Servidor + cliente eco completos y ejercicios resueltos 

 Estás en:  U05 · Sockets UDP y Protocolos → 08 · Práctica eco UDP

La idea en una frase

La práctica clásica de UDP: un **servidor eco** que devuelve al cliente exactamente lo que recibe. Es el “Hola mundo” de los datagramas y la base de todo lo que verás en el laboratorio del [punto 9](#).

Junta todo lo aprendido: cliente UDP ([punto 2](#)), servidor UDP ([punto 3](#)) y la naturaleza de los datagramas ([punto 4](#)). Con dos terminales y unos pocos bytes, verás el protocolo completo en acción.

El servidor eco

```
import socket

HOST = "127.0.0.1"
PORT = 9001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor eco UDP en {HOST}:{PORT}")

    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")

    # Devolver lo mismo que se ha recibido
    servidor.sendto(datos, direccion)
```

El truco está en la última línea: `servidor.sendto(datos, direccion)` devuelve los **mismos bytes** que llegaron, a la **misma dirección** de la que vinieron. Eco, literalmente.

El cliente que lo prueba

```
import socket

HOST = "127.0.0.1"
PORT = 9001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    mensaje = "Hola UDP, devuélveme esto"
    cliente.sendto(mensaje.encode(), (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"El servidor me devolvió: {datos.decode()}")
```

Salida del cliente (con el servidor en ejecución en otra terminal):

```
El servidor me devolvió: Hola UDP, devuélveme esto
```

Prueba a escribir el mensaje con un `input()` y tendrás tu primer chat UDP de una línea.



Be the code: eco UDP paso a paso

“Sé el datagrama que sale del cliente, llega al servidor y vuelve.”

1. Servidor: `bind(("127.0.0.1", 9001))` → reserva el puerto 9001
2. Servidor: `recvfrom(1024)` → se queda escuchando (bloqueado)
3. Cliente: `sendto(b"Hola UDP...", ("127.0.0.1", 9001))`
→ el datagrama viaja a 127.0.0.1:9001
4. Servidor: `recvfrom()` despierta → (datos, direccion)
→ datos = b"Hola UDP...", direccion = ("127.0.0.1", <puerto_cliente>)
5. Servidor: `sendto(datos, direccion)` → devuelve lo mismo al cliente
6. Cliente: `recvfrom()` → recibe el eco y lo imprime

La línea 4 es la más importante: la **dirección del cliente** aparece por primera vez cuando el servidor la necesita para responder. Ese es el flujo completo de UDP: fuego, escucha, responde.



Aprieta el lápiz

1. **Servidor UDP eco:** Crea un servidor UDP que devuelva al cliente lo mismo que recibe.
2. **Cliente HTTP manual:** Conéctate a httpbin.org y haz un GET a `/ip` para ver tu IP pública.

3. **Mini servidor web:** Crea un servidor TCP que responda con HTML básico ("`<h1>Hola</h1>`") a cualquier petición.
4. **Comparativa velocidad:** Mide cuánto tarda TCP vs UDP en enviar 100 mensajes pequeños.

>  Soluciones

Mini-chequeo

1. ¿Qué línea del servidor eco convierte el servidor en “eco”?
2. ¿Cómo descubre el servidor a quién responder?
3. ¿Qué necesitas para probar el cliente contra el servidor?

>  Respuestas

Resumen en 3 frases

- El servidor eco devuelve con `sendto(datos, direccion)` exactamente lo que recibió con `recvfrom()`.
- La dirección del cliente llega pegada en cada datagrama: sin ella no habría forma de responder.
- Con el servidor en una terminal y el cliente en otra tienes el protocolo UDP completo funcionando.

Vocabulario rápido

Término	Idea general
Eco UDP	El servidor devuelve lo mismo que recibe
<code>sendto(datos, direccion)</code>	La jugada del eco: reenviar a la dirección de origen
Terminal del servidor	Ventana donde el servidor queda escuchando
Puerta de respuesta	La dirección que <code>recvfrom()</code> entrega al servidor

09 — Cierre: consolida lo aprendido

Sé el datagrama, laboratorio eco UDP y el cierre de la unidad 🗨️

📖 Estás en: 🏠 U05 · Sockets UDP y Protocolos → 09 · Cierre

Has terminado la teoría: TCP vs UDP, cliente y servidor UDP, los caprichos de los datagramas, HTTP hablado a pelo y NTP para saber la hora. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el datagrama

Eres un datagrama UDP. Te acabas de crear con un `sendto()` y vas a emprender tu viaje hacia un servidor eco.

¿Qué pasa?

1. El cliente te empaqueta con la dirección destino: `sendto(datos, ("127.0.0.1", 9001))`.
No hay conexión: tú eres un mensaje suelto, sin número de secuencia y sin confirmación de vuelta.
2. Atraviesas la red como un avión de papel. **Nadie te reenvía si te pierdes:** esa es tu esencia y tu velocidad.
3. Llegas al servidor y `recvfrom(1024)` te despierta de su espera. El servidor lee tu contenido y tu dirección de origen.
4. El servidor decide devolverte con `sendto(datos, direccion)`: te convierte en **eco** y vuelves por donde viniste.
5. El cliente te recibe con `recvfrom()` y te imprime. Has completado un viaje de ida y vuelta... o no: si el viento (la red) te pierde, nadie se entera.

Todo sin conexión, sin handshake y sin confirmaciones.

💡 **Ahora tú:** ¿y si el servidor no está escuchando? Tu viaje termina en el vacío: el datagrama se pierde y el `recvfrom()` del cliente se queda esperando (a menos que haya un `settimeout()`). Eso es lo que aprendiste en el [punto 4](#).

Fireside Chat: TCP vs UDP

Dos protocolos de transporte se sientan junto a la chimenea a dirimir, de una vez, quién manda.

TCP: — Yo soy el mensajero certificado. Entrego cada carta, en orden, y si se pierde, la reenvío. Pero cuesta más.

UDP: — Yo soy el lanzador de aviones de papel. Mando y olvido. Si no llega, pues no llega. Pero lanzo 100 en el tiempo que tú preparas uno.

TCP: — Mis casos de uso: web (HTTP), correo (SMTP), transferencia de archivos (FTP). Todo lo que necesite fiabilidad.

UDP: — Mis casos de uso: videollamadas (Zoom), streaming (Twitch), juegos online (Fortnite), DNS. Prefiero velocidad antes que fiabilidad.

TCP: — Tengo control de congestión, retransmisión, checksums...

UDP: — Yo tengo... velocidad. Y puedo añadir fiabilidad en la capa de aplicación si quiero (QUIC, por ejemplo).

TCP: — Eres un temerario.

UDP: — Y tú un pesado. Por eso nos complementamos.

Moraleja: no son rivales: son dos herramientas para dos momentos. TCP cuando el dato debe quedar intacto; UDP cuando el momento es lo valioso.

¿Quién soy?

1. Soy el método del cliente UDP que envía el datagrama con su dirección destino.
2. Soy el método que devuelve datos y la dirección de quien los mandó.
3. Soy el protocolo de texto sobre TCP que mueve la web.
4. Soy el grupo de servidores de tiempo que responde en el puerto 123.
5. Soy el protocolo fiable que se construye encima de UDP para HTTP/3.
6. Soy el método que convierte a un servidor UDP en “eco”.

>  Respuestas



CONRAD VS EL MUNDO: “perdí paquetes y mi stream se corta”

CONRAD: — “Clásico: montas el streaming con UDP, se pierden paquetes, y en la reunión te dicen ‘*el vídeo se corta*’. Pues claro. Razones: 1) **No hay reenvío**: un datagrama perdido se pierde para siempre. 2) **No hay orden**: los paquetes que llegan pueden venir revueltos, y sin buffer de reordenación, el reproductor se atraganta. 3) **No hay control de congestión**: UDP manda igual de rápido aunque la red se sature, así que la pérdida se dispara. 4) Usaste UDP para algo que necesitaba fiabilidad... o TCP para algo que necesitaba velocidad.”

CONRAD: — “Y lo mejor: ‘*pero el cliente hacía `recvfrom` y nunca llegaba nada*’. ¡Pues claro! Si el datagrama se perdió, `recvfrom()` se queda bloqueado **para siempre**. Un `settimeout(5)` y verás la excepción aparecer. UDP no avisa: tú eres quien pone los tiempos.”

CONRAD: — “Y no me vengas con ‘*¿será que la red va lenta?*’. Si pierdes paquetes, el problema no es la velocidad: es que **elegiste mal el protocolo** o no pusiste tiempo de espera. HTTP para lo que debe ser fiable, UDP para lo que debe ser fluido, y `settimeout()` para no quedarte colgado. A diagnosticar.”

Laboratorio de tortura: eco UDP en dos terminales

Duración: 45 minutos **Herramienta:** Python 3 (socket, sin instalar nada) + dos terminales

Escenario: construye un servidor eco UDP y un cliente que le mande mensajes, exactamente como en el [punto 8](#).

Tareas paso a paso:

1. **Escribe el servidor eco** (`servidor_eco.py`): `bind()` en `127.0.0.1:9001` y un bucle infinito que haga `recvfrom()` y responda con `sendto(datos, direccion)`. Debe imprimir cada mensaje recibido con su dirección.
2. **Escribe el cliente** (`cliente_eco.py`): pide un mensaje con `input()`, lo envía con `sendto()` y muestra la respuesta de `recvfrom()`.
3. **Arranca el servidor** en una terminal y **el cliente** en otra. Envía tres mensajes seguidos y comprueba que el eco funciona y que el servidor ve la dirección de cada uno.
4. **Añade el contador**: haz que el servidor responda "Eco #1: ...", "Eco #2: ..." llevando la cuenta de los mensajes recibidos.

5. **Añade `settimeout(5)`** al cliente y comprueba qué pasa si cierras el servidor antes de enviar.

Fallo intencionado: en el servidor, en lugar de responder con la dirección que devuelve `recvfrom()`, responde a una dirección inventada: `servidor.sendto(datos, ("127.0.0.1", 9999))`. ¿Qué pasa? El cliente se queda **bloqueado para siempre** en su `recvfrom()`: el eco fue a otro puerto, donde nadie escucha. La respuesta existe, pero no llega a su destinatario. Solo el `settimeout(5)` te sacará del atolladero.

Pista 1: en UDP, la dirección del remitente **no se guarda en ningún sitio**: llega en cada `recvfrom()`. Para responder bien, usa la misma tupla que te entrega `recvfrom()`, como en el [punto 8](#).

Pista 2: si el cliente se queda colgado, ese es el síntoma del fallo: el `recvfrom()` del cliente espera un datagrama que fue a otro puerto. Añade `cliente.settimeout(5)` y verás la excepción `socket.timeout` aparecer a los 5 segundos, confirmando que la respuesta nunca llegó.



Logros de esta unidad

Logro	Cómo conseguirlo
 Avión de Papel	Crear y enviar el primer datagrama UDP con <code>sendto()</code>
 Eco Master	Montar un servidor + cliente eco UDP en dos terminales
 Fiable o Veloz	Explicar cuándo usar TCP y cuándo UDP con casos reales
 Navegador a pelo	Hablar HTTP a mano con un socket TCP y leer la respuesta
 Relojero	Obtener la hora oficial de Internet con un cliente NTP



Atrévete a pensar

1. ¿Por qué UDP puede “perder datos” y TCP no, y por qué eso es aceptable en VoIP?
2. ¿Qué pasaría si HTTP/1.1 se enviara por UDP en lugar de TCP?
3. ¿Cómo sabe un servidor UDP a quién responder si no hay conexión?
4. ¿Qué ventaja tiene QUIC (HTTP/3) al construirse sobre UDP en lugar de usar TCP?

5. ¿Por qué NTP usa UDP aunque la hora exacta parezca “importante”?

> 💡 Soluciones

🧩 Crucigrama de bits

Horizontal:

1. Método del cliente UDP para enviar un datagrama (6 letras)
4. Protocolo fiable sobre UDP que usa HTTP/3 (4 letras)
6. Método que devuelve datos y la dirección del remitente (8 letras)
8. Método del servidor UDP para reservar un puerto (4 letras)

Vertical:

2. Verbo HTTP para pedir un recurso (3 letras)
3. Unidad de datos independiente de UDP (9 letras)
5. Código de estado HTTP para "todo bien" (3 dígitos)
7. Puerto de los servidores NTP (3 dígitos)

> 📝 Soluciones

💬 Entrevista de trabajo

1. “¿Qué diferencia hay entre TCP y UDP? ¿Cuándo usarías cada uno?”
2. “Escribe un servidor UDP que reciba un mensaje y responda ‘Recibido’.”
3. “¿Cómo funciona HTTP a nivel de socket? Descríbeme una petición y su respuesta.”
4. “¿Cómo obtendrías la hora exacta de Internet en Python sin librerías externas?”
5. “¿Qué es QUIC y por qué se monta sobre UDP?”

💡 **Cómo encararlas:** la 1 y la 2 son las “preguntas reina”. Para la 1, repite la moraleja del [punto 7](#): fiabilidad contra velocidad, con los casos reales (web/correo → TCP; streaming/juegos/DNS → UDP). Para la 2, escribe el servidor del [punto 3](#) sin pensarlo: `bind()` + `recvfrom()` + `sendto()`. Si sabes contarlo fluido, ya eres medio desarrollador de redes.

🧑 No hay preguntas tontas

❓ ¿Cuándo usar UDP en vez de TCP?

Cuando la velocidad importa más que la fiabilidad: streaming, juegos, VoIP, DNS. Perder un frame de vídeo es mejor que esperar a que se reenvíe.

? ¿UDP puede perder datos?

Sí. No hay confirmación de recepción. Si pierdes un paquete, se pierde para siempre.

? ¿HTTP siempre usa TCP?

Sí, HTTP/1.1 y HTTP/2 usan TCP. **HTTP/3** usa QUIC, que va sobre UDP (¡la vuelta a la tortilla!).

? ¿NTP usa UDP? ¿No es importante que llegue la hora exacta?

Sí, NTP usa UDP. Pero manda muchas peticiones y calcula estadísticamente la hora correcta. Si un paquete se pierde, no pasa nada: el próximo valdrá.

Poscréditos

Un datagrama UDP sale del cliente. Sin conexión, sin confirmación, sin miedo.

El servidor eco lo recibe, lee su dirección y se lo devuelve igual que llegó.

En la otra terminal, un socket TCP estrecha la mano y la web entera llega, completa y en orden.

Y en algún lugar del mundo, un reloj sin átomos pregunta la hora exacta en el puerto 123.

PRÓXIMAMENTE EN U06: APIs REST. Hasta ahora hablabas HTTP a mano, petición a petición. Ahora toca automatizarlo: la librería requests, los verbos GET y POST, y el JSON viajando por la red.

Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación (HTTP, NTP).

CE	Criterio	Cubierto
a)	Modelo de capas de red (TCP/IP)	 Punto 1
b)	Identifica tipos de sockets (TCP/UDP)	 Puntos 1-4

CE	Criterio	Cubierto
e)	Implementa servidores y clientes UDP	✓ Puntos 2, 3 y 8 + ⚡ Laboratorio de tortura
h)	Implementa protocolos de aplicación (HTTP, NTP)	✓ Puntos 5 y 6 + Cliente HTTP manual

RA3c (servidor TCP), RA3d (cliente TCP), RA3f (errores) y RA3g (opciones) se cubren en la U04 · Sockets TCP.

 [Volver al índice de la unidad](#) · Anterior: [08 · Práctica eco UDP](#) · Siguiente: [U06 · APIs REST y HTTP](#)

U06 — APIs REST y HTTP

Hablar con la web: REST, métodos HTTP y JSON 🌐

📖 Ruta del viaje: 🚀 Proceso → 🔄 Hilo → 🗝 Sincronización → 🌐 TCP → 📡 UDP → 🌐 APIs REST y HTTP → 🧪 APIs comerciales → 🗝 Hash → 🧬 Cifrado → ⏰ Servidores → ⌚ asincio

En la U04 y la U05 montaste sockets y hablaste con procesos en la misma máquina. Ahora toca hablar con el mundo: la web entera se apoya en HTTP, y las APIs REST son la manera moderna de pedirle datos a cualquier servicio.

Cuando tu aplicación quiere saber el tiempo, traducir un texto o buscar repositorios en GitHub, no monta un socket a mano: llama a una **API**. Esta unidad te enseña el idioma de esa conversación: qué es una API, cómo funciona HTTP, qué métodos existen (GET, POST, PUT, DELETE), qué significan los códigos de estado, cómo se lee **JSON** y cómo usar la librería `requests` de Python para hacer peticiones reales contra servicios públicos. El siguiente tema, [U07 · APIs Comerciales](#), tomará el relevo con APIs de pago como OpenWeatherMap y OpenAI.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una API y por qué pone una capa de abstracción entre dos programas.
- Describir la web: qué es una URL, una petición HTTP y una respuesta HTTP.
- Dominar los métodos HTTP (GET, POST, PUT, PATCH, DELETE) y cuándo usar cada uno, incluyendo la idempotencia.
- Aplicar los principios REST: recursos, URLs semánticas, sin estado y JSON.
- Interpretar los códigos de estado HTTP (2xx, 3xx, 4xx y 5xx) sin mirar la tabla.
- Leer y generar JSON, y convertirlo a diccionarios y listas de Python.
- Hacer peticiones GET con `requests`, con parámetros de consulta y cabeceras.
- Enviar datos con POST, PUT y DELETE usando `json=`, y gestionar errores HTTP con criterio.
- Construir un mini cliente de API completo en Python que hable con servicios reales.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Web y HTTP	Qué es una API, la URL y el intercambio petición/respuesta	Todos
02 · Métodos HTTP	GET, POST, PUT, PATCH y DELETE: los verbos de la web y la idempotencia	Todos
03 · Principios REST	Recursos, URLs semánticas, sin estado y qué significa ser RESTful	Todos
04 · Códigos de estado	200, 201, 400, 401, 404, 500...: la señal de humo del servidor	Todos
05 · JSON	El idioma de las APIs: dumps, loads y parse de respuestas	Todos
06 · requests: el GET	<code>requests.get</code> , parámetros de consulta, cabeceras y la respuesta	Todos
07 · requests: POST, PUT y DELETE	Crear, actualizar y borrar con <code>json=</code> , y gestión de errores	Todos
08 · Práctica: mini cliente de API	Sé el código, un cliente completo y los ejercicios del lápiz	Todos
09 · Cierre	Sé la petición HTTP, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

 Inicial resuelto

 Inicial por resolver

 Avanzado resuelto

 Avanzado por resolver



Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Dónde se cubre
a)	Utiliza APIs REST para obtener datos externos	✓ Puntos 3, 6-8 + ⚡ Laboratorio (punto 9)
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✓ Puntos 1-5 y 7 + ⚡ Laboratorio (punto 9)

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en la **U10 · Servidores Concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **U11 · asyncio y disponibilidad**.



¿Por dónde empiezo?

¿Vienes de la U05 y dominas los sockets UDP y TCP? Perfecto, ese es el trampolín ideal: repasa la [U05 · Sockets UDP y protocolos](#) para tener fresco qué es un protocolo y cómo se abre una conexión, y arranca en el [punto 1](#), que parte justo de ahí: del intercambio de mensajes entre dos máquinas a la conversación estructurada de HTTP.

¿Ya sabes qué es una API y solo necesitas requests o el JSON? Ve directo al [punto 5](#) y al [punto 6](#). Si vienes de cero en la web, no te saltes los puntos 1 a 4: URL, métodos y códigos de estado son el idioma que vas a hablar toda la unidad.



Primer punto: [01 · Web y HTTP](#)



Al acabar la unidad, continúa en [U07 · APIs Comerciales](#).

01 — Web y HTTP

La conversación entre tu código y el servidor 🌐

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 01 · Web y HTTP

La idea en una frase

Una **API** es un conjunto de reglas para que dos programas se comuniquen, y **HTTP** es el idioma que usan en la web: tu programa manda una **petición** y el servidor responde.

No necesitas saber cómo está implementado el sistema por dentro: solo le pides algo y te lo devuelve. Esa capa que esconde la complejidad es la magia de las APIs.

¿Qué es una API?

API = *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Un conjunto de reglas para que dos programas se comuniquen.

Compara las dos formas de acceder a los datos de una base de usuarios:

```
# Sin API — accedes directamente a la BD (malo)
import sqlite3
conn = sqlite3.connect("usuarios.db")
conn.execute("SELECT * FROM usuarios")

# Con API — pides datos a un servicio
import requests
respuesta = requests.get("https://miapi.com/usuarios")
datos = respuesta.json()
```

Las APIs ponen una capa de abstracción: no necesitas saber cómo está implementado el sistema por dentro.

En la segunda opción no sabes qué motor de base de datos hay detrás, ni en qué lenguaje está el servidor, ni cuántos servidores son. Solo sabes la regla: “si pido GET /usuarios, me llega una lista”. Eso es trabajar con una API.



La web en tres piezas

Toda la web se reduce a tres piezas que ya conoces por navegar:

- **Cliente:** quien pide (tu navegador, tu código Python).
- **Servidor:** quien responde (una máquina en algún lugar del mundo).
- **Protocolo HTTP:** las reglas de la conversación.

La **URL** (*Uniform Resource Locator*) es la dirección a la que mandas la petición. Se divide en partes con significado:

```
https://api.github.com/users/python?per_page=10
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
scheme   host      path      query params
(https) (api.github.com) (/users/python) (?per_page=10)
```

Parte	Ejemplo	Qué es
scheme	https	El protocolo de transporte (cifrado en HTTPS)
host	api.github.com	El servidor al que hablas
path	/users/python	El recurso concreto que pides
query	?per_page=10	Parámetros extra de la petición



Petición y respuesta

HTTP funciona por **intercambio de mensajes**: tú envías una petición y el servidor te contesta.

Petición (lo que envía el cliente):

```
GET /users/python HTTP/1.1
Host: api.github.com
User-Agent: python-requests/2.31.0
Accept: */*
```

Respuesta (lo que devuelve el servidor):

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: ...

{"login": "python", "public_repos": 42}
```

En los próximos puntos desmontarás cada pieza: los **métodos** (GET, POST...) en el [punto 2](#), los **códigos de estado** (200, 404...) en el [punto 4](#) y el **cuerpo JSON** en el [punto 5](#).

La analogía del pedido a domicilio

Una petición HTTP es como **pedir comida a domicilio**:

1. Buscas el restaurante (**URL**) que vende lo que quieres.
2. Mándasle tu pedido (**petición HTTP**): “una pizza margarita” (GET), “que me pongas en la carta un plato nuevo” (POST).
3. El restaurante (servidor) la cocina y te la entrega (**respuesta HTTP**).
4. La entrega llega con un estado: “ todo bien” (200), “ el plato no existe” (404), “ el restaurante no te atiende sin reserva” (401).

No te hace falta saber cómo cocina la cocina: solo la dirección y el menú. Esa es la filosofía de las APIs.

Mini-chequeo

1. ¿Qué es una API y qué problema resuelve?
2. Identifica las cuatro partes de esta URL:
`https://api.openweathermap.org/data/2.5/weather?q=Valencia.`
3. ¿Qué dos mensajes intercambian cliente y servidor en HTTP?

>  Respuestas

Resumen en 3 frases

- Una API es un conjunto de reglas que permite a dos programas comunicarse sin conocer su implementación interna.
- La web funciona con HTTP: el cliente manda una petición y el servidor devuelve una respuesta.
- Una URL se compone de scheme, host, path y query: cada parte tiene un significado en la conversación.



Vocabulario rápido

Término	Idea general
API	Reglas para que dos programas se comuniquen
HTTP	El protocolo de intercambio de mensajes de la web
URL	La dirección completa de un recurso
Cliente	Quien pide (tu navegador o tu código)
Servidor	Quien responde y sirve los datos
Query	Parámetros extra de la URL (?clave=valor)



[Volver al índice de la unidad](#) · **Siguiente:** [02 · Métodos HTTP](#)

02 — Métodos HTTP

Los verbos de la web: leer, crear, actualizar y borrar 🚦

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 02 · Métodos HTTP

📌 La idea en una frase

HTTP tiene **verbos** (métodos) que dicen qué quieres hacer con un recurso: **GET** lo lee, **POST** lo crea, **PUT** lo reemplaza, **PATCH** lo retoca y **DELETE** lo borra.

Los métodos HTTP se corresponden con las operaciones **CRUD** de las bases de datos: Create, Read, Update, Delete. Elegir el verbo correcto es la primera decisión de una API bien hecha.

🚦 Los verbos de HTTP

Método	CRUD	Uso	Cuerpo
GET	Read	Obtener datos	❌ No
POST	Create	Crear recurso	✅ Sí
PUT	Update	Reemplazar completo	✅ Sí
PATCH	Update	Actualizar parcial	✅ Sí
DELETE	Delete	Borrar recurso	❌ No

Un mismo recurso puede “vivir” con todos los verbos. La URL no cambia: cambia el método:

GET	/usuarios	→ Listar usuarios
GET	/usuarios/5	→ Obtener usuario 5
POST	/usuarios	→ Crear usuario
PUT	/usuarios/5	→ Actualizar usuario 5
DELETE	/usuarios/5	→ Borrar usuario 5

📄 ¿Qué es la idempotencia?

Un método es **idempotente** si llamarlo **una vez o varias produce el mismo resultado**. Piensa en un interruptor: “apagar la luz” es idempotente (da igual cuántas veces lo hagas, la luz queda apagada); “sumar 1” no lo es.

Método	Idempotente	Por qué
GET	✓	Leer 5 veces devuelve lo mismo
PUT	✓	Reemplazar el recurso completo: el estado final es igual
DELETE	✓	Borrar algo ya borrado sigue borrado
POST	✗	Cada llamada crea un recurso nuevo : 2 llamadas = 2 recursos
PATCH	⚠	Depende de la operación parcial que apliques

💡 Si un cliente reenvía una petición por culpa de un timeout, un método idempotente (PUT) es seguro de repetir; un POST podría duplicar el recurso.

Métodos en Python

Con requests, cada método es una función: `requests.get`, `requests.post`, `requests.put`, `requests.delete`...

```
import requests

# POST – crear un recurso
nuevo_usuario = {"nombre": "Ana", "email": "ana@x.com"}
resp = requests.post("https://miapi.com/usuarios", json=nuevo_usuario)
print(resp.status_code) # 201 Created

# DELETE – borrar
resp = requests.delete("https://miapi.com/usuarios/5")
print(resp.status_code) # 204 No Content
```

El `json=` de `requests.post` convierte el diccionario Python a JSON automáticamente y lo envía en el cuerpo de la petición. Lo verás en detalle en el [punto 7](#).



Mini-chequeo

1. ¿Qué método usarías para... a) ver un artículo, b) crearlo, c) cambiarlo entero, d) borrarlo?
2. ¿Por qué POST no es idempotente y PUT sí?
3. ¿GET lleva cuerpo en la petición? ¿Y POST?

>  Respuestas



Resumen en 3 frases

- HTTP tiene cinco verbos con un propósito claro: GET lee, POST crea, PUT reemplaza, PATCH retoca y DELETE borra.
- La idempotencia distingue los métodos seguros de repetir (GET, PUT, DELETE) del que crea (POST).
- En Python, cada método es una función de requests y json= se encarga del cuerpo automáticamente.



Vocabulario rápido

Término	Idea general
Método HTTP	El verbo que dice qué hacer con el recurso
GET	Leer datos (sin cuerpo, idempotente)
POST	Crear un recurso (con cuerpo, no idempotente)
PUT	Reemplazar el recurso completo
PATCH	Actualizar solo una parte
DELETE	Borrar el recurso
Idempotente	Repetirlo no cambia el resultado

 [Volver al índice de la unidad](#) · Anterior: [01 · Web y HTTP](#) · Siguiente: [03 · Principios REST](#)

03 — Principios REST

Recursos, URLs semánticas y sin estado 🏠

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 03 · Principios REST

📌 La idea en una frase

REST (*Representational State Transfer*) es el estándar de las APIs modernas: trata las cosas del sistema como **recursos**, a cada recurso le da una **URL** y lo manipula con los **verbos HTTP** que viste en el [punto 2](#).

REST no es una tecnología ni una librería: es un conjunto de **convenciones**. Cuando las sigues, cualquiera puede usar tu API sin leer un manual de 100 páginas.

🧱 Recursos: las cosas, no las acciones

En REST, modelas **sustantivos**, no verbos. Un recurso es “una cosa” del sistema: un usuario, un producto, un pedido. Cada recurso tiene su URL:

```
/productos      → todos los productos
/productos/123  → el producto 123
```

Fíjate en la diferencia con las APIs antiguas basadas en acciones:

❌ Acciones (no REST) ✅ Recursos (REST)

/getUsuario?id=5	GET /usuarios/5
/crearUsuario	POST /usuarios
/borrarUsuario?id=5	DELETE /usuarios/5

En REST la URL es un **nombre** (qué cosa) y el método HTTP es el **verbo** (qué hacer). Se separan los dos conceptos en vez de mezclarlos en la URL.

Los cuatro pilares de REST

- **Recursos:** cada entidad tiene una URL (/usuarios, /productos/123).
- **Verbos HTTP:** GET (leer), POST (crear), PUT (actualizar), DELETE (borrar).
- **Sin estado** (stateless): cada petición contiene todo lo necesario.
- **JSON:** el formato de intercambio de datos.

Con esos cuatro pilares, una API REST es predecible: si sabes usar una, sabes usar todas. Ese es el mapa mental que te llevas de esta unidad:

GET	/usuarios	→ Listar usuarios
GET	/usuarios/5	→ Obtener usuario 5
POST	/usuarios	→ Crear usuario
PUT	/usuarios/5	→ Actualizar usuario 5
DELETE	/usuarios/5	→ Borrar usuario 5

Sin estado (stateless)

Que una API sea **stateless** significa que el servidor **no guarda contexto entre peticiones**: cada petición lleva toda la información necesaria para ser entendida por sí sola.

Piénsalo como un camarero que no recuerda tu cara: cada vez que pides algo, le tienes que decir todo de nuevo. No puedes decir “lo mismo de antes”, porque el camarero no guarda memoria de lo anterior.

Consecuencia práctica: el servidor no tiene que recordar quién eres entre llamadas. Si tu petición necesita autenticación, la envías en cada petición (en la cabecera Authorization), no “la primera vez y ya está”.

¿Qué significa “RESTful”?

Una API es **RESTful** si cumple de verdad los principios REST: recursos con URLs semánticas, verbos HTTP bien usados, sin estado y JSON como formato.

Dos ejemplos opuestos:

-  **RESTful:** GET `https://api.github.com/users/python` → “dame el recurso del usuario python”.
-  **No RESTful:** GET `https://miapi.com/obtenerUsuario?id=5&accion=leer` → meto la acción en la URL y abuso del GET.

Elegir el verbo correcto para cada operación es lo que el [punto 2](#) llamaba “hacer bien REST”. Las APIs de GitHub, GitHub Search o JSONPlaceholder que usarás en esta unidad son RESTful: fíjate en cómo sus URLs son limpias y su método dice la intención.

Mini-chequeo

1. ¿Qué es un recurso en REST? Pon dos ejemplos de URLs.
2. ¿Por qué GET `/getUsuario?id=5` no es RESTful?
3. ¿Qué significa que una API sea “sin estado”?

>  **Respuestas**

Resumen en 3 frases

- REST modela **recursos** con URLs semánticas y los manipula con los **verbos HTTP**, sin meter la acción en la URL.
- Sus cuatro pilares son recursos, verbos HTTP, **sin estado** y JSON.
- Una API que cumple esos principios es **RESTful**: predecible y fácil de usar.

Vocabulario rápido

Término	Idea general
REST	Estándar de APIs: recursos + verbos HTTP + sin estado + JSON
Recurso	Una “cosa” del sistema con su URL
URL semántica	URL que nombra el recurso, no la acción
Stateless	El servidor no guarda contexto entre peticiones
RESTful	API que cumple los principios REST

 [Volver al índice de la unidad](#) · **Anterior:** [02 · Métodos HTTP](#) · **Siguiente:** [04 · Códigos de estado](#)

04 — Códigos de estado

La señal de humo del servidor: 200, 404, 500... 🚨

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 04 · Códigos de estado

La idea en una frase

Cada respuesta HTTP lleva un **código de estado** que te dice si la petición salió bien o mal: **2xx es éxito, 3xx es redirección, 4xx es un error tuyo y 5xx es un error del servidor.**

Es la señal de humo de la web: en tres dígitos, el servidor te dice qué ha pasado sin necesidad de leer el cuerpo.

Las familias de códigos

Familia	Significado	Ejemplos
2xx	Éxito	200 OK, 201 Created, 204 No Content
3xx	Redirección	301 Moved Permanently
4xx	Error del cliente (tuyo)	400, 401, 403, 404, 429
5xx	Error del servidor	500, 503

Consejo: los códigos 2xx son éxito, 4xx son error TUYO, 5xx son error DEL SERVIDOR. Cuando veas un 4xx, revisa tu petición; cuando veas un 5xx, la culpa es de la otra parte.

La tabla completa

Código	Significado	Situación típica
200	OK	GET exitoso
201	Created	POST exitoso (recurso creado)

Código	Significado	Situación típica
204	No Content	DELETE exitoso
301	Moved Permanently	La URL cambió
400	Bad Request	Enviaste datos incorrectos
401	Unauthorized	Falta autenticación
403	Forbidden	No tienes permiso
404	Not Found	El recurso no existe
429	Too Many Requests	Te pasaste del límite
500	Internal Server Error	Error del servidor
503	Service Unavailable	El servicio está caído

Truco para no confundirte:

- **400 vs 404:** 400 = “tu petición está mal formada”, 404 = “lo que pides no existe”.
- **401 vs 403:** 401 = “no te has identificado”, 403 = “te has identificado pero no tienes permiso”.

Códigos en Python

En requests, el código está en `resp.status_code` (un entero). Lo compruebas antes de procesar los datos:

```
import requests

resp = requests.get("https://api.github.com/users/python")
if resp.status_code == 200:
    print("✅ Todo bien:", resp.json()["login"])
elif resp.status_code == 404:
    print("❌ Usuario no encontrado")
else:
    print(f"⚠️ Código inesperado: {resp.status_code}")
```

💡 `requests` **no lanza excepción** cuando llega un 404 o un 500: te devuelve la respuesta con su `status_code` y `resp.ok` será `False`. Eres tú quien decide cómo tratarlo. Si quieres que una respuesta mala lance excepción, usa `resp.raise_for_status()` (lo verás en el [punto 7](#)).

Mini-chequeo

1. ¿Qué familia indica un error del servidor? ¿Y un error tuyo?
2. ¿Qué código devuelve un POST exitoso? ¿Y un DELETE exitoso?
3. Diferencias 401 vs 403 y 400 vs 404.

>  Respuestas

Resumen en 3 frases

- El código de estado es la señal de humo de HTTP: 2xx éxito, 3xx redirección, 4xx error tuyo, 5xx error del servidor.
- En Python lo lees con `resp.status_code` y decides qué hacer con cada código.
- `requests` no lanza excepción ante un 4xx/5xx: comprobar `status_code` (o usar `raise_for_status()`) es tu responsabilidad.

Vocabulario rápido

Término	Idea general
Código de estado	Los 3 dígitos que resumen el resultado de una petición
2xx	Éxito (200 OK, 201 Created, 204 No Content)
4xx	Error del cliente (400, 401, 403, 404, 429)
5xx	Error del servidor (500, 503)
<code>status_code</code>	La propiedad de la respuesta que lo contiene (int)
<code>raise_for_status()</code>	Método que lanza excepción si el código es 4xx/5xx

05 — JSON

El idioma de las APIs: de texto a diccionario y vuelta 📦

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 05 · JSON

La idea en una frase

JSON (*JavaScript Object Notation*) es el formato de intercambio de datos más usado en las APIs: un texto estructurado con llaves, corchetes y comillas que Python convierte directamente a **diccionarios** y **listas**.

Es el “idioma” en el que las APIs te devuelven los datos. En el [punto 3](#) lo viste como uno de los cuatro pilares de REST; aquí le toca el turno de verdad.

El formato

Un JSON es un texto con pares "clave": valor. Los valores pueden ser números, textos, booleanos, listas u otros objetos anidados:

```
{
  "nombre": "Ana",
  "edad": 25,
  "email": "ana@x.com",
  "activo": true,
  "intereses": ["Python", "redes", "cripto"]
}
```

Fíjate en las reglas: las **claves van entre comillas dobles**, los **textos entre comillas dobles**, los booleanos son true/false en minúscula y las listas van entre corchetes.

De JSON a Python y viceversa

El módulo json de la librería estándar hace la conversión en las dos direcciones:

```
import json

# JSON (texto) → Python
texto = '{"nombre": "Ana", "edad": 25, "activo": true}'
datos = json.loads(texto)          # dict
print(datos["nombre"])             # "Ana"
print(type(datos))                 # <class 'dict'>

# Python → JSON (texto)
python_dict = {"nombre": "Ana", "edad": 25, "activo": True}
texto_json = json.dumps(python_dict, ensure_ascii=False, indent=2)
print(texto_json)
```

La equivalencia es casi automática:

JSON	Python
{ ... }	dict
[...]	list
"texto"	str
42	int / float
true / false	True / False
null	None

💡 `json.dumps(..., ensure_ascii=False)` conserva las tildes tal cual; `indent=2` lo formatea legible. Y recuerda: `loads` viene de “**load** string” y `dumps` de “**dump** string”.

JSON dentro de requests

Con `requests` ni siquiera tocas el texto: `resp.json()` parsea el cuerpo automáticamente y te da el dict/list de Python.

```
import requests

resp = requests.get("https://api.github.com/users/python")
datos = resp.json()

print(datos["login"])          # "python"
print(datos["public_repos"])   # 42
print(datos["avatar_url"])     # "https://..."
```

Cuando la respuesta es una **lista** de recursos, `resp.json()` devuelve una lista de diccionarios y la recorres normal:

```
resp = requests.get("https://jsonplaceholder.typicode.com/posts")
posts = resp.json()
print(f"Hay {len(posts)} posts")          # 100
print(posts[0]["title"])                  # el título del primer post
```

⚠ `resp.json()` **lanza excepción** si el cuerpo no es JSON (por ejemplo, un HTML de error). Por eso en el [punto 4](#) primero comprobas `status_code` y luego parseas.



Mini-chequeo

1. ¿A qué tipo de Python se convierte `[1, 2, 3]`? ¿Y `{"a": true}`?
2. ¿Qué hace `json.loads` y qué hace `json.dumps`?
3. ¿Qué pasa si llamas a `resp.json()` sobre una respuesta que no es JSON?

>  Respuestas



Resumen en 3 frases

- JSON es el formato de intercambio de datos de las APIs: texto estructurado que Python mapea a dicts y listas.
- `json.loads` y `json.dumps` hacen la conversión en las dos direcciones con el módulo `json`.
- Con `requests`, `resp.json()` parsea el cuerpo directamente, pero lanza excepción si no es JSON.



Vocabulario rápido

Término	Idea general
JSON	Formato de intercambio de datos de las APIs
json.loads	De texto JSON a objeto Python
json.dumps	De objeto Python a texto JSON
resp.json()	Parseo directo del cuerpo de la respuesta
ensure_ascii	Opción de dumps para conservar tildes
JSONDecodeError	Excepción al parsear algo que no es JSON

 [Volver al índice de la unidad](#) · **Anterior:** [04 · Códigos de estado](#) · **Siguiente:** [06 · requests: el GET](#)

06 — requests: el GET

La librería estándar de facto para hablar HTTP 📡

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 06 · requests: el GET

La idea en una frase

`requests` es la librería estándar de facto para hacer peticiones HTTP en Python: `requests.get(url)` te devuelve un objeto **respuesta** con el código, las cabeceras y el cuerpo ya parseado.

Todo lo que has visto hasta ahora —métodos, códigos, JSON— se materializa aquí en una llamada de una línea.

El GET más básico

```
import requests

# GET básico
resp = requests.get("https://api.github.com")
print(resp.status_code)      # 200
print(resp.headers)          # Cabeceras de respuesta
print(resp.elapsed)          # Tiempo que tardó
```

`requests.get` se encarga de todo lo que en la U04 hacías a mano con sockets: resolver el DNS, abrir la conexión TCP, el handshake TLS en HTTPS, construir la petición HTTP y leer la respuesta.

La respuesta y sus propiedades

El objeto que devuelve `requests.get` es un objeto **respuesta** con estas propiedades:

Propiedad	Qué devuelve
<code>resp.status_code</code>	Código HTTP (int)

Propiedad	Qué devuelve
resp.ok	True si 200-399
resp.text	Cuerpo como string
resp.json()	Cuerpo como dict (¡lanza excepción si no es JSON!)
resp.headers	Dict con cabeceras
resp.elapsed	Objeto timedelta

La jerarquía de uso habitual es: primero `status_code` (o `resp.ok`), luego `resp.json()` para los datos. Y si solo quieres ver qué te ha contestado en bruto, `resp.text`.

Parámetros de consulta

Muchas APIs filtran o buscan con **query params** en la URL. En vez de montar la URL a mano con `f"..."`, `requests` te da `params=` y construye la query string por ti:

```
import requests

params = {"q": "python", "page": 1, "per_page": 10}
resp = requests.get("https://api.github.com/search/repositories", params=params)
# URL final: https://api.github.com/search/repositories?q=python&page=1&per_page=10
```

Ventajas de `params=` frente a concatenar a mano: escapa los caracteres especiales (espacios, `&`, `#`) y te ahorra errores de “se me olvidó el ?”.

Cabeceras personalizadas

Algunas APIs requieren cabeceras específicas: autenticación, formato esperado o el identificador de tu aplicación. Se envían con `headers=`:

```
import requests

cabeceras = {
    "Authorization": "Bearer mi-token-secreto",
    "Accept": "application/json",
    "User-Agent": "MiApp/1.0"
}

resp = requests.get("https://api.github.com/user", headers=cabeceras)
```

⚠ **Seguridad:** un token en la cabecera Authorization nunca debe ir “en claro” en el código ni subirse a GitHub. En la [U07](#) verás cómo guardarlo en variables de entorno con python-dotenv.

Mini-chequeo

1. ¿Qué devuelve `requests.get` exactamente?
2. ¿Para qué sirve `params=` y qué construye por ti?
3. ¿Qué propiedad te dice si el código está entre 200 y 399?

> Respuestas

Resumen en 3 frases

- `requests.get(url)` devuelve un objeto respuesta con código, cabeceras y cuerpo.
- `params=` construye la query string por ti y `headers=` envía cabeceras personalizadas (como el token).
- El orden de uso: comprobar `status_code/ok` y luego `resp.json()`.

Vocabulario rápido

Término	Idea general
requests	La librería de facto para peticiones HTTP en Python
Objeto respuesta	Lo que devuelve <code>requests.get</code> : código, cabeceras y cuerpo

Término	Idea general
params	Diccionario que se convierte en query string
headers	Diccionario con las cabeceras de la petición
resp.text	El cuerpo en bruto como string
resp.elapsed	Tiempo que tardó la petición (timedelta)
 Volver al índice de la unidad · Anterior: 05 · JSON · Siguiente: 07 · requests: POST, PUT y DELETE	

07 — requests: POST, PUT y DELETE

Enviar datos a la API y gestionar los errores 🚀

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 07 · requests: POST, PUT y DELETE

La idea en una frase

Con `requests.post`, `requests.put` y `requests.delete` **creas, reemplazas y borras recursos**: el cuerpo se envía con `json=` y la respuesta se trata igual que en el GET.

Los verbos del [punto 2](#) cobran vida: `json=` convierte tu diccionario a JSON y lo pone en el cuerpo de la petición.

POST: crear un recurso

Para crear, `requests.post` con `json=`:

```
import requests

resp = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json={"title": "foo", "body": "bar", "userId": 1}
)
print(resp.status_code)      # 201 Created
print(resp.json())           # La respuesta parseada como dict
```

JSONPlaceholder es una API de pruebas: cada POST te devuelve el recurso creado con un ID nuevo (101, 102...). Un POST exitoso responde **201 Created** (visto en el [punto 4](#)).

PUT y PATCH: actualizar

PUT reemplaza el recurso **completo** (idempotente); PATCH actualiza solo **una parte**:

```
import requests

# PUT – reemplazo completo
resp = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}
)
print(resp.status_code)          # 200 OK
print(resp.json()["title"])       # "nuevo titulo"
```

En la práctica casi todo el mundo usa `requests.put` con `json=` del mismo modo; `requests.patch` existe para los casos de “toca solo este campo”.



DELETE: borrar

```
import requests

resp = requests.delete("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code)          # 200 OK (JSONPlaceholder)
# En REST puro, un borrado exitoso devuelve 204 No Content
```

El código de estado te confirma el resultado: en REST canónico, DELETE exitoso → **204 No Content** (sin cuerpo que leer).



Gestión de errores: `status_code` y `raise_for_status`

Como viste en el [punto 4](#), `requests` **no lanza excepción** ante un 4xx/5xx. Dos estrategias:

```

import requests

# Opción 1: comprobar el código a mano
resp = requests.post("https://jsonplaceholder.typicode.com/posts",
                    json={"title": "x", "body": "y"})
if resp.status_code == 201:
    print("✅ Post creado con ID", resp.json()["id"])
else:
    print("❌ Algo falló:", resp.status_code)

# Opción 2: raise_for_status() lanza excepción si es 4xx/5xx
try:
    resp = requests.get("https://httpbin.org/status/404")
    resp.raise_for_status()          # lanza HTTPError
    datos = resp.json()
except requests.exceptions.HTTPError as e:
    print("⚠️ Error HTTP:", e)

```

`raise_for_status()` es la vía rápida: si la respuesta es mala, lanza una excepción y tu código no sigue parseando un error.

🧩 Pool Puzzle — Petición API con errores

Estas líneas hacen una petición a una API y gestionan errores. ¿En qué orden van?

```

a)     if respuesta.status_code == 200:
b) import requests
c)     print("Error:", respuesta.status_code)
d) respuesta = requests.get("https://api.github.com/users/python")
e)     datos = respuesta.json()
f)     else:
g)         print(datos["login"])
h) try:

```

> 🔒 Solución

🧠 Mini-chequeo

1. ¿Qué hace `json=` en `requests.post`?
2. ¿Qué código devuelve un POST exitoso y qué devuelve un DELETE exitoso?
3. ¿Qué lanza `raise_for_status()` y cuándo lo usarías?

> Respuestas

Resumen en 3 frases

- `requests.post/put/delete` crean, reemplazan y borran recursos enviando el cuerpo con `json=`.
- El resultado se lee igual que en el GET: `status_code` y `resp.json()`.
- Los errores se gestionan comprobando `status_code` o dejando que `raise_for_status()` lance la excepción.

Vocabulario rápido

Término	Idea general
<code>requests.post</code>	Envía un recurso nuevo con <code>json=</code>
<code>requests.put</code>	Reemplaza el recurso completo
<code>requests.patch</code>	Actualiza solo una parte
<code>requests.delete</code>	Borra el recurso
<code>json=</code>	Convierte un dict Python a JSON en el cuerpo
<code>raise_for_status()</code>	Lanza <code>HTTPError</code> si el código es 4xx/5xx

 [Volver al índice de la unidad](#) · Anterior: [06 · requests: el GET](#) · Siguiente: [08 · Práctica: mini cliente de API](#)

08 — Práctica: mini cliente de API

Sé el código, un cliente completo y los ejercicios del lápiz 🖋️

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 08 · Práctica: mini cliente de API

La idea en una frase

Aquí junta todo lo aprendido en un solo programa: un **mini cliente de API** que pide datos a servicios reales, los parsea y gestiona los errores — y un paseo “siendo” la librería requests.

De aquí saldrás con un programa completo que ya habla el idioma de la web.

El mini cliente de API

Un cliente de API típico se compone de tres bloques: hacer la petición, comprobar el estado y procesar los datos. Todo junto:

```

import requests

def obtener_usuario(nombre):
    """Devuelve un dict con los datos del usuario o None si no existe."""
    resp = requests.get(f"https://api.github.com/users/{nombre}")
    if resp.status_code == 200:
        return resp.json()
    if resp.status_code == 404:
        print(f"❌ El usuario '{nombre}' no existe.")
    else:
        print(f"⚠️ Código inesperado: {resp.status_code}")
    return None

def resumen_usuario(datos):
    print(f"👤 Login: {datos['login']}")
    print(f"❤️ Nombre: {datos['name']}")
    print(f"📦 Repos: {datos['public_repos']}")
    print(f"👥 Seguidores: {datos['followers']}")

if __name__ == "__main__":
    datos = obtener_usuario("python")
    if datos:
        resumen_usuario(datos)

```

Observa el patrón que repite cualquier cliente real: `requests.get` → comprobar `status_code` → `resp.json()` → usar los datos. Es el mismo esqueleto del [punto 6](#), ahora con errores bien delimitados.

★ Sé el código: una petición GET, milisegundo a milisegundo

“Sé la librería `requests` y recorre cada milisegundo de una petición GET.”

```
import requests

# 1. Construimos la URL y parámetros
url = "https://api.github.com/users/python"

# 2. Llamamos a requests.get()
respuesta = requests.get(url)

# 3. Procesamos la respuesta
datos = respuesta.json()
print(f"Usuario: {datos['login']}")
print(f"Repos: {datos['public_repos']}")
```

Traza interna:

1. `requests.get("https://api.github.com/users/python")`
2. `requests` construye la URL:
 - scheme: `https`
 - host: `api.github.com`
 - path: `/users/python`
3. Abre conexión TCP a `api.github.com:443` (HTTPS)
 - DNS lookup: `api.github.com` → `140.82.121.5`
 - Three-way handshake TCP
 - Handshake TLS (cifrado)
4. Envía petición HTTP:
`GET /users/python HTTP/1.1`
`Host: api.github.com`
`User-Agent: python-requests/2.31.0`
`Accept-Encoding: gzip, deflate`
`Accept: */*`
5. 🕒 Espera respuesta (~150ms)
6. Recibe respuesta:
`HTTP/1.1 200 OK`
`Content-Type: application/json`
...
7. `requests` parsea el JSON → dict de Python
8. Extraemos `datos['login']` → `"python"`

Todo lo que en la U04 hacías a mano —DNS, TCP, TLS, construir la petición— lo hace `requests` en una línea. Tu única tarea es leer la respuesta.



Aprieta el lápiz

1. **GET a GitHub API:** obtén los datos del usuario “python” y muestra su nombre real, repos públicos y seguidores.
2. **Parámetros de búsqueda:** busca repositorios de Python con más de 1000 estrellas usando `/search/repositories`.

3. **POST a JSONPlaceholder:** crea un post nuevo en `jsonplaceholder.typicode.com/posts` y muestra el ID devuelto.
4. **Gestión de errores:** haz GET a una URL que no existe (404) y una que dé error 500. Muestra mensajes adecuados.

>  Soluciones

Mini-chequeo

1. ¿Cuál es el esqueleto de cualquier petición con requests?
2. En la traza del “Sé el código”, ¿qué hace requests entre el paso 3 y el 6?
3. ¿Por qué comprobar `status_code` antes de `resp.json()` es una buena práctica?

>  Respuestas

Resumen en 3 frases

- Un cliente de API se reduce a tres bloques: petición, comprobación de estado y procesamiento de datos.
- requests hace por ti todo el trabajo de red (DNS, TCP, TLS) que en la U04 hacías a mano con sockets.
- El patrón `status_code` → `json()` es la clave para que tu código no reviente ante respuestas de error.

Vocabulario rápido

Término	Idea general
Cliente de API	Programa que habla con una API mediante peticiones HTTP
Esqueleto de petición	<code>requests.get</code> → <code>status_code</code> → <code>resp.json()</code> → usar datos
Traza interna	El camino que sigue la petición dentro de la librería
Endpoint	La URL concreta de un recurso o acción de la API

09 — Cierre: consolida lo aprendido

Sé la petición HTTP, laboratorio real y el ring final de la unidad 🗣️

📖 Estás en: 🌐 U06 · APIs REST y HTTP → 09 · Cierre

Has terminado la teoría: API, URL, métodos HTTP, principios REST, códigos de estado, JSON, requests con GET y con POST/PUT/DELETE, y el mini cliente completo. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real contra una API pública con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé la petición HTTP

Eres una petición HTTP. Acabas de nacer en un `requests.get()` dentro del portátil de un estudiante y tu destino es `api.github.com`. Empieza el viaje.

¿Qué pasa?

1. `requests` te construye: fija el `scheme` `https`, el `host` `api.github.com` y el `path` `/users/python`. Eres un mensaje `GET /users/python HTTP/1.1` con cabeceras (`Host`, `User-Agent`, `Accept`).
2. Abres la conexión: primero el **DNS lookup** (`api.github.com` → una IP), luego el **three-way handshake TCP** y el **handshake TLS** que te cifra para el viaje.
3. Atraviesas Internet hasta el servidor y te entrega el proceso servidor que te espera en el puerto 443.
4. El servidor interpreta tu método (`GET`), localiza el recurso y contesta: **200 OK** con el cuerpo `application/json`.
5. De vuelta en Python, `requests` parsea el cuerpo JSON y te convierte en un **dict** (`datos["login"]` → `"python"`).
6. El estudiante imprime tu mensaje. Misión cumplida.

Todo en ~150 ms. Sin saber qué hay en cada router del camino.

💡 **Ahora tú:** ¿y si en lugar de un `GET` el estudiante usara un `POST` para crear un post en JSONPlaceholder? Tu cuerpo ya no está vacío: viajas con `{"title": "foo", "body": "bar", "userId": 1}` dentro, y el servidor te responde **201 Created** con el recurso creado y su ID nuevo. Eso es lo que aprendiste en el [punto 7](#).

Fireside Chat: GET vs POST vs PUT vs DELETE

Cuatro verbos HTTP se sientan junto a la chimenea a decidir, de una vez, quién es el más importante.

GET: — Yo soy el más usado. Solo pido información, no cambio nada. Soy seguro, idempotente y cacheable.

POST: — Pues yo soy el que crea cosas. Sin mí no habría nuevos recursos. Eso sí, no soy idempotente: si llamas dos veces, creo dos veces.

PUT: — Idempotente, como GET, pero yo actualizo. Mando el recurso completo y reemplazo lo que haya.

DELETE: — Y yo elimino. También idempotente: da igual cuántas veces lo llames, el recurso ya no está.

GET: — Oye, ¿y PATCH? Siempre se nos olvida...

POST: — Ese es el primo moderno que actualiza solo un campo. Pero mejor no liarnos, que ya somos suficientes.

Moraleja: cada verbo HTTP tiene un propósito: GET (leer), POST (crear), PUT (reemplazar), DELETE (borrar). Elegir el correcto es hacer bien REST.



¿Quién soy?

1. Soy el formato de intercambio de datos que casi todas las APIs usan hoy: llaves, corchetes y comillas.
2. No soy idempotente: si me llamas dos veces, creo dos recursos.
3. Soy el código de estado que aparece cuando el recurso que buscas no existe.
4. Pongo una capa de abstracción: no necesitas saber cómo está implementado el sistema por dentro.
5. Soy la propiedad de la respuesta que devuelve True si el código está entre 200 y 399.
6. Soy la librería de facto para hacer peticiones HTTP en Python, y construyo la query string con params=.



CONRAD VS EL MUNDO: “404 que nadie entiende”

CONRAD: — “Clásico: haces GET a `https://api.github.com/users/python` y te llega 404. Razones: 1) **Escribiste mal la URL:** `/user` en lugar de `/users`, o te comiste un carácter. 2) **El recurso no existe:** el usuario cambió de nombre o fue borrado. 3) **Confundiste el recurso con la acción:** pediste `/search/repositories` sin `?q=`. 4) No distinguiste el **404** (no existe) del **401** (no autenticado) ni del **403** (sin permiso).”

CONRAD: — “Y lo mejor: *‘¿será que GitHub está caído?’* ¡Pues no! Si el servidor estuviera caído verías **500** o **503**, no 404. El 404 es tuyo: la red está bien, lo que pediste está mal. A revisar la URL y el método.”

CONRAD: — “Y no me vengas con *‘¿será que el JSON está mal?’* si antes ni comprobaste el `status_code`. Primero `status_code`, luego `json()`. Si parseas un 404, `resp.json()` te lanza una excepción y el mensaje de error te lleva por el camino de la amargura. A diagnosticar.”

⚡ Laboratorio de tortura: mini cliente contra una API pública

Duración: 45 minutos **Herramienta:** Python 3 (requests, pip install si hace falta) y conexión a Internet

Escenario: construye un mini cliente que consulte dos APIs públicas de verdad: **GitHub API** y **JSONPlaceholder**. Usa todo lo de la unidad: GET con params, POST, PUT, DELETE y gestión de errores.

Tareas paso a paso:

1. **GET básico:** obtén `https://api.github.com/users/python` y muestra `login`, `name`, `public_repos` y `followers`.
2. **Búsqueda con params:** busca repositorios de Python con más de 1000 estrellas con `params={"q": "python stars:>1000", "sort": "stars", "per_page": 5}` y muestra nombre y estrellas de los 5 primeros.
3. **POST:** crea un post en `https://jsonplaceholder.typicode.com/posts` con `json={"title": "Hola API", "body": "Probando", "userId": 1}` y muestra el ID devuelto.

4. **PUT**: actualiza el post 1 con `requests.put` y comprueba que el título cambió.
5. **DELETE**: borra el post 1 y muestra el código de estado.
6. **Errores**: pide un usuario inexistente (`/users/usuarioquenoexiste123`) y muestra un mensaje “no encontrado” sin que el programa reviente.

Fallo intencionado: en el paso 3, en lugar de `requests.post`, usa `requests.get` con `json=`. ¿Qué pasa? La API no te deja crear nada con GET: te responde un 404 (o “Not Found”) y, como tu código llama a `resp.json()` sin comprobar `status_code`, intenta parsear un cuerpo que no es JSON y **revienta**.

Pista 1: los métodos no son intercambiables. Crear es siempre **POST** (lo viste en el [punto 2](#)). Si usas GET para escribir, la API te lo devuelve con un 4xx.

Pista 2: si el fallo no se ve a simple vista, añade `print(resp.status_code, resp.text)` justo antes del `resp.json()`. Verás el código y el cuerpo de error en texto plano: esa es la prueba de que el problema está en el método elegido.

Logros de esta unidad

Logro	Cómo conseguirlo
 HTTP Rider	Explicar de memoria la anatomía de una petición y una respuesta HTTP
 Verb Master	Elegir correctamente GET, POST, PUT, PATCH o DELETE para cada operación CRUD
 Status Whisperer	Interpretar 200, 201, 400, 401, 404 y 500 sin mirar la tabla
 JSON Parsel	Parsear una respuesta JSON a dict de Python y extraer campos con seguridad
 API Client	Construir un mini cliente de API con GET, POST y gestión de errores

Atrévete a pensar

1. ¿Por qué POST no es idempotente y PUT sí?
2. ¿Qué diferencia hay entre 401 y 403?

3. ¿Por qué REST prefiere recursos en las URLs (/usuarios/5) en lugar de acciones (getUsuario?id=5)?
4. ¿Qué significa “sin estado” y qué consecuencias tiene para el servidor?
5. ¿Cuándo usarías `resp.text` en lugar de `resp.json()`?

> 💡 Soluciones

🧩 Crucigrama de bits

Horizontal:

1. Verbo HTTP para leer datos (3 letras)
4. Código de estado cuando el recurso no existe (3 letras)
5. Formato de intercambio de datos de las APIs (4 letras)
8. Verbo HTTP para borrar un recurso (6 letras)

Vertical:

2. Código de estado de error del servidor (3 letras)
3. Verbo HTTP para crear recursos (4 letras)
6. Cabecera que indica el formato que acepta la petición (6 letras)
7. La librería de facto para peticiones HTTP en Python (8 letras)

> 📄 Soluciones

💬 Entrevista de trabajo

1. “¿Qué es una API REST? Explícala como si fuera un camarero.”
2. “¿Qué diferencia hay entre GET y POST? ¿Y entre PUT y PATCH?”
3. “Explica los códigos de estado: ¿qué significan 200, 201, 400, 401, 404 y 500?”
4. “Escribe en Python una petición GET con parámetros a una API y procesa la respuesta JSON.”
5. “¿Cómo gestionarías los errores de una petición HTTP en tu código?”

💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, recorre los pilares del [punto 3](#): recursos con URL, verbos HTTP, sin estado y JSON — con la analogía del camarero del [punto 1](#). Para la 4, escribe el esqueleto del [punto 6](#): `params=`, `status_code`, `resp.json()`. Si sabes contarlo fluido, ya eres medio desarrollador de APIs.

No hay preguntas tontas

? ¿Necesito siempre una API key?

No, hay APIs públicas sin key (como la de GitHub para datos públicos). Pero la mayoría requiere autenticación. Y cuando la necesites, la mandas en la cabecera Authorization (nunca en la URL ni en el código): lo verás en la [U07](#).

? ¿Puedo modificar datos con GET?

Técnicamente sí, pero **no debes**. GET es para leer. POST/PUT para modificar. Si usas GET para modificar, violas REST: el método deja de decir la verdad sobre lo que haces.

? ¿Qué es CORS y me afecta en Python?

CORS es una restricción del navegador. En Python no te afecta (no hay navegador). Es cosa de JavaScript.

? ¿Qué significa que una API sea “sin estado” (stateless)?

Que cada petición contiene toda la información necesaria. El servidor no guarda contexto entre peticiones. Como un camarero que no recuerda tu cara: cada vez le tienes que decir todo.

? ¿JSON o XML?

Hoy en día, JSON gana por goleada. XML solo se usa en entornos legacy (bancos, SOAP).

Poscréditos

Tu petición GET parte de tu portátil, cruza Internet y vuelve con un JSON bajo el brazo.

El camarero te trajo exactamente lo que pediste: status 200 y los datos listos para usar.

Y ahora que sabes hablar con la web, toca llamar a las APIs de verdad: las comerciales.

PRÓXIMAMENTE EN U07: APIs comerciales. OpenWeatherMap, OpenAI y las API keys que nunca, jamás, se suben a GitHub.

Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Cubierto
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3, 6-8 + ⚡ Laboratorio de tortura
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1-5 y 7 + ⚡ Laboratorio de tortura

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en la **U10 · Servidores Concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **U11 · asyncio y disponibilidad**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Práctica API](#) · Siguiente: [U07 · APIs Comerciales](#)

U07 — APIs Comerciales

Consumir OpenWeatherMap y OpenAI con claves y cuidado 🧪

📖 Ruta del viaje: 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 🧪 APIs Comerciales → 🗝 Hash → 🧬 Cifrado → ⏱ Servidores → ⌚ asyncio

*En la U06 consumiste APIs públicas y gratuitas: sin clave, sin límites, sin miedo. Ahora toca el mundo real: las APIs comerciales te dan una **API key**, te limitan las peticiones, se caen y tienes que gestionar sus errores como un adulto.*

En esta unidad consumirás dos de las APIs más usadas del planeta: **OpenWeatherMap** para el tiempo y **OpenAI** para el cerebro artificial. Aprenderás a guardar tus claves con seguridad en variables de entorno, a gestionar rate limits y errores HTTP con elegancia, y a construir un programa completo que habla de verdad con ambas. Todo lo que viste en la [U06 · APIs REST y HTTP](#) —GET, JSON, requests— se viste aquí de traje de gala.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una **API key**, para qué sirve y por qué nunca debe ir en el código.
- Guardar credenciales en **variables de entorno** con python-dotenv y proteger tu repo con .gitignore.
- Consumir **OpenWeatherMap**: URL, parámetros (q,appid,units,lang) y parseo de la respuesta JSON.
- Llamar a la **API de OpenAI**: chat completions, roles de mensaje (system/user), max_tokens y temperature.
- Distinguir los **códigos de error** de una API comercial (401, 403, 404, 429, 500).
- Gestionar **rate limits** con esperas, raise_for_status() y **backoff exponencial**.
- Aplicar **buenas prácticas de seguridad**: rotación de claves, gitignore y nunca exponer credenciales.
- Construir un **programa completo** que consume OpenWeatherMap y OpenAI con las claves en .env.

- Conocer herramientas para probar APIs (Postman, curl, httpie, Insomnia).

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · API Keys	El carnet de identidad: qué es, cómo se obtiene y dónde se manda	Todos
02 · Variables de entorno	python-dotenv, el .env y por qué tus claves nunca van al repo	Todos
03 · OpenWeatherMap	El tiempo de tu ciudad: URL, parámetros y parseo del JSON	Todos
04 · OpenAI	El cerebro artificial: chat completions, mensajes y roles	Todos
05 · Rate limiting	Límites, cuotas y el temido 429: espera y backoff	Todos
06 · Errores HTTP	401, 403, 429 y 500: gestionar fallos con <code>raise_for_status</code>	Todos
07 · Seguridad y buenas prácticas	Rotación de claves, gitignore y no exponer secretos	Todos
08 · Práctica: APIs comerciales	Sé el código, el programa completo y los ejercicios del lápiz	Todos
09 · Cierre	Sé la petición, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Dónde se cubre
a)	Utiliza APIs REST para obtener datos externos	✓ Puntos 3-5 y 8 + ⚡ Laboratorio (punto 9)
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✓ Puntos 1, 5-6 y 8 + ⚡ Laboratorio (punto 9)

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en la **U10 · Servidores Concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **U11 · asyncio y disponibilidad**.

📖 ¿Por dónde empiezo?

¿Vienes de la U06 y dominas requests, GET y JSON? Perfecto, ese es el trampolín ideal: repasa la [U06 · APIs REST y HTTP](#) para tener fresco params= y resp.json(), y arranca en el [punto 1](#), que parte justo de ahí: de la petición con requests a la petición con clave.

¿Ya tienes claves de OpenWeatherMap y OpenAI y solo necesitas consumirlas? Ve directo al [punto 3](#) y al [punto 4](#). Si vienes de cero en APIs, no te saltes los puntos 1 y 2: la API key y el .env son la base de todo lo demás.

📌 Primer punto: [01 · API Keys](#)

▶▶ Al acabar la unidad, continúa en [U08 · Hash y Cifrado Clásico](#).

01 — API Keys

El carnet de identidad de las APIs comerciales 📄

📖 Estás en: 📌 U07 · APIs Comerciales → 01 · API Keys

La idea en una frase

Una **API key** es el carnet de identidad que te da la API comercial: identifica quién eres, cuánto puedes usar y quién te facturará si te pasas.

Las APIs de verdad no son de mentira. Te piden API key, te limitan las peticiones, se caen y tienes que gestionar sus errores como un adulto. La API key es la puerta de entrada a todo eso.

¿Qué es una API key?

Una **API key** (o token) es una cadena secreta que la API te asigna al registrarte. Cada petición que haces la lleva contigo, y el servidor sabe así tres cosas:

Qué sabe el servidor	Ejemplo
Quién eres	El usuario sergio tiene la key abc123...
Cuánto puedes usar	Tu plan gratis permite 60 peticiones/minuto
Quién paga el exceso	Si pasas del plan, la factura cae en tu cuenta

Es exactamente el **carnet de identidad** del mundo de las APIs: sin él, el servidor no sabe si eres un estudiante probando o un robot mandando 10.000 peticiones por segundo.

¿Cómo se obtiene?

1. Te registras en la web del proveedor (OpenWeatherMap, OpenAI, GitHub...).
2. Creas una clave desde el **panel de control** (suele llamarse “API Keys”, “Tokens” o “Credentials”).
3. Copias la cadena y **nunca, jamás, la subes a GitHub**.

Las claves de OpenAI empiezan por sk- y las de OpenWeatherMap son una cadena alfanumérica larga. En este curso usarás siempre el marcador TU_API_KEY para no escribir claves reales.

📌 ¿Dónde va la clave?

Hay dos formas habituales de enviar la clave en la petición:

1. Como parámetro de la URL (lo usa OpenWeatherMap con appid):

```
GET /data/2.5/weather?q=Sevilla&appid=TU_API_KEY&units=metric
```

2. Como cabecera Authorization (lo usa OpenAI con Bearer):

```
Authorization: Bearer TU_API_KEY
```

Con requests, las dos formas se escriben así:

```
import requests

API_KEY = "TU_API_KEY" # ⚠ NUNCA subas esto a GitHub

# Opción 1: la clave en la query string (params)
resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)

# Opción 2: la clave en la cabecera Authorization (headers)
resp = requests.get(
    "https://api.github.com/user",
    headers={"Authorization": f"Bearer {API_KEY}"}
)
```

⚠ **Ojo con la URL:** si la clave va en la query string, queda registrada en los logs del servidor y en tu historial. Las cabeceras son más discretas. Por eso OpenAI exige Bearer y OpenWeatherMap (más antiguo) usa appid.



La analogía del llavero de acceso

Una API key es como el **llavero de acceso a un edificio de oficinas**:

- Sin llavero no entras: el servidor te responde **401 Unauthorized**.
- Tu llavero tiene un **nivel**: el de estudiante abre la planta del laboratorio, no la sala de servidores (eso sería un **403 Forbidden**).
- Si pierdes el llavero, no vale “borrar el código”: hay que **anular la cerradura** y pedir uno nuevo (rotar la clave).

Y la regla de oro que repetiremos toda la unidad:

Regla de oro: las API keys nunca van en el código. Usa variables de entorno.



Mini-chequeo

1. ¿Qué tres cosas sabe el servidor cuando le envías tu API key?
2. ¿Dónde puede viajar la clave en una petición HTTP? Pon un ejemplo de cada sitio.
3. ¿Qué te devuelve la API si la clave es incorrecta?

>  **Respuestas**



Resumen en 3 frases

- Una API key es el carnet de identidad de las APIs comerciales: identifica quién eres y cuánto puedes usar.
- Se obtiene registrándote en el proveedor y se envía en la query string (appid) o en la cabecera Authorization.
- Las API keys nunca van en el código: en el [punto 2](#) verás dónde se guardan de verdad.



Vocabulario rápido

Término

Idea general

API key

Carnet de identidad de la API: identifica quién eres

Término	Idea general
Token	Sinónimo de clave de acceso
appid	Parámetro donde OpenWeatherMap espera tu clave
Authorization	Cabecera HTTP donde se mandan claves y tokens
Bearer	Esquema de autorización de la cabecera (Bearer TU_API_KEY)
401	Código de error cuando la clave es inválida o falta

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · Variables de entorno](#)

02 — Variables de entorno

Nunca subas tus claves: `python-dotenv` y el `.env` 🔒

📖 Estás en: 📌 U07 · APIs Comerciales → 02 · Variables de entorno

La idea en una frase

Las claves nunca van en el código: se guardan en un archivo `.env` que se carga con `python-dotenv`, y en Python se leen con `os.getenv()`.

Así el código queda limpio (nadie ve tus secretos al abrir el repo) y las claves viven en un archivo que no se sube a git.

El archivo `.env`

Un archivo `.env` es un fichero de texto plano en la raíz de tu proyecto donde cada línea define una variable de entorno:

```
OPENWEATHER_API_KEY=abc123...  
OPENAI_API_KEY=sk-...
```

Este archivo NO se sube a git. Es tuyo, de tu máquina, y cada persona (o cada CI) tiene el suyo. Si alguien abre tu repo en GitHub, debería encontrar el código pero nunca el `.env`.

Cargarlo con `python-dotenv`

Primero se instala la librería:

```
pip install python-dotenv
```

Y en el código, al arrancar, se carga el `.env` y se leen las variables:

```
# pip install python-dotenv
from dotenv import load_dotenv
import os

load_dotenv() # Carga el archivo .env

API_KEY = os.getenv("OPENWEATHER_API_KEY")
if not API_KEY:
    raise ValueError("❌ Falta OPENWEATHER_API_KEY en .env")
```

Desglose de las tres piezas:

Línea	Qué hace
<code>load_dotenv()</code>	Lee el <code>.env</code> y mete cada variable en el entorno del proceso
<code>os.getenv("OPENWEATHER_API_KEY")</code>	Devuelve el valor, o <code>None</code> si no existe
<code>if not API_KEY: raise ValueError(...)</code>	Frena el programa con un mensaje claro si falta la clave

💡 El `raise ValueError` es tu salvavidas: en lugar de llegar a una petición con `None` y fallar con un error críptico del servidor, tu programa muere al instante diciéndote **qué** clave falta.

🚫 No subir claves al repo

Git nunca va a saber por sí solo que el `.env` es secreto: hay que decírselo con un `.gitignore`. Este archivo le dice a git qué ficheros no debe rastrear:

```
.gitignore
```

```
.env
```

Con ese `.gitignore` en tu repo, `git add .` ya no se llevará el `.env` por delante.

⚠️ **Aviso para adultos:** si algún día subes una clave por error, borrarla del código NO la borra del historial de git. La única solución es **rotar la clave** (crear una nueva y anular la vieja). Lo verás en el [punto 7](#).



Mini-chequeo

1. ¿Qué hace `load_dotenv()` exactamente?
2. ¿Qué devuelve `os.getenv("MI_CLAVE")` si la variable no existe?
3. ¿Por qué el `.env` no se sube a git y cómo se lo dices a git?

>  Respuestas



Resumen en 3 frases

- Las claves se guardan en un archivo `.env` de texto plano, fuera del código.
- `load_dotenv()` lo carga y `os.getenv("CLAVE")` devuelve el valor (o `None`).
- Un `.gitignore` con `.env` evita que tus secretos lleguen a GitHub.



Vocabulario rápido

Término	Idea general
<code>.env</code>	Archivo de texto donde viven tus variables secretas
<code>load_dotenv()</code>	Carga el <code>.env</code> al entorno del proceso
<code>os.getenv()</code>	Lee el valor de una variable de entorno (o <code>None</code>)
<code>.gitignore</code>	Lista de ficheros que git ignora
Variable de entorno	Valor externo que el código lee, nunca incrustado

 [Volver al índice de la unidad](#) · Anterior: [01 · API Keys](#) · Siguiente: [03 · OpenWeatherMap](#)

03 — OpenWeatherMap

El tiempo en tu ciudad con una llamada 🌤️

📖 Estás en: 📌 U07 · APIs Comerciales → 03 · OpenWeatherMap

📌 La idea en una frase

OpenWeatherMap te da el tiempo de cualquier ciudad con un solo GET: construyes la URL, pasas `q`, `appid`, `units` y `lang`, y parseas el JSON que te devuelve.

Es la API comercial más sencilla que vas a tocar en el módulo: perfecta para estrenar tu primera API key de verdad.

📖 Registro y API key

1. Entra en openweathermap.org y crea una cuenta gratuita.
2. En el panel, apartado **API Keys**, te generan tu clave personal.
3. Guárdala en tu `.env` (punto 2) como `OPENWEATHER_API_KEY`.

🎁 **Plan gratis:** suficiente para clase. Te permite 60 peticiones por minuto (el límite exacto lo verás en el [punto 5](#)).

📌 La URL y sus parámetros

El endpoint del tiempo actual es:

```
https://api.openweathermap.org/data/2.5/weather
```

Parámetro	Qué hace	Ejemplo
<code>q</code>	La ciudad (o “ciudad,país”)	<code>q=Sevilla</code>
<code>appid</code>	Tu API key (punto 1)	<code>appid=TU_API_KEY</code>
<code>units</code>	Unidades: <code>metric</code> , <code>imperial</code> , <code>standard</code>	<code>units=metric</code>

Parámetro	Qué hace	Ejemplo
lang	Idioma de las descripciones	lang=es

💡 Sin units=metric la temperatura llega en **kelvin**. Con metric llega en grados centígrados: mucho más amigable.

Llamada y parseo

```
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

def clima(ciudad):
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}

    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"❌ Error: {resp.json().get('message', 'desconocido')}")
        return

    datos = resp.json()
    print(f"☁️ {datos['name']}: {datos['main']['temp']}°C")
    print(f"    Sensación: {datos['main']['feels_like']}°C")
    print(f"    Humedad: {datos['main']['humidity']}%")
    print(f"    {datos['weather'][0]['description']}")

clima("Sevilla")
```

Respuesta JSON (simplificada):

```
{
  "name": "Sevilla",
  "main": { "temp": 18.5, "feels_like": 17.2, "humidity": 65 },
  "weather": [{ "description": "cielo claro" }]
}
```

Los campos que usamos:

Ruta en el JSON	Qué es
<code>datos['name']</code>	El nombre de la ciudad
<code>datos['main']['temp']</code>	Temperatura actual
<code>datos['main']['feels_like']</code>	Sensación térmica
<code>datos['main']['humidity']</code>	Humedad en %
<code>datos['weather'][0]['description']</code>	Texto descriptivo del cielo

💡 `weather` es una **lista** de diccionarios (porque puede haber varias condiciones), por eso se accede con `[0]`.

Mini-chequeo

1. ¿Qué parámetro convierte la temperatura a grados centígrados?
2. ¿Por qué `datos['weather'][0]['description']` necesita el `[0]`?
3. ¿Qué comprobas antes de hacer `resp.json()` en la función `clima`?

>  Respuestas

Resumen en 3 frases

- OpenWeatherMap devuelve el tiempo con un GET a `/data/2.5/weather` pasando `q`, `appid`, `units` y `lang`.
- La respuesta es JSON anidado: `main` guarda temperaturas y `weather` la descripción.
- Comprobar el `status_code` antes de parsear evita errores feos cuando la API falla.



Vocabulario rápido

Término	Idea general
Endpoint	La URL concreta de un recurso (/data/2.5/weather)
q	Parámetro de ciudad
units=metric	Devuelve grados centígrados en lugar de kelvin
lang=es	Descripciones del tiempo en español
main	Objeto con temperaturas y humedad
weather	Lista con las descripciones del cielo

 [Volver al índice de la unidad](#) · Anterior: [02 · Variables de entorno](#) · Siguiente: [04 · OpenAI](#)

04 — OpenAI

El cerebro artificial al alcance de una API 🤖

📖 Estás en: 📌 U07 · APIs Comerciales → 04 · OpenAI

La idea en una frase

La API de OpenAI convierte un chat en una función: le pasas una lista de **mensajes con roles** y te devuelve la respuesta de texto que puedes imprimir.

No hay modelo instalado en tu máquina: todo vive en los servidores de OpenAI y tú solo envías peticiones HTTP. Exactamente el mismo patrón que en OpenWeatherMap, pero con un POST y una cabecera Authorization.

Registro y clave

1. Crea una cuenta en platform.openai.com.
2. Genera tu clave en **API Keys**: empieza por sk-.
3. Guárdala en tu .env como OPENAI_API_KEY (nunca en el código).

⚠️ A diferencia de OpenWeatherMap, la clave de OpenAI se manda en la **cabecera Authorization: Bearer sk-...**, no en la URL.

Chat completions

La librería oficial openai hace todo el trabajo. Se instala con `pip install openai`:


```

from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()

cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

respuesta = cliente.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "Eres un profesor de Python divertido."},
        {"role": "user", "content": "Explica qué es un Lock en 2 frases."}
    ],
    max_tokens=100,
    temperature=0.7
)

print(respuesta.choices[0].message.content)

```

Desglose:

Pieza	Qué hace
OpenAI(api_key=...)	Crea el cliente con tu clave del .env
model	Qué modelo usar (gpt-3.5-turbo, barato y suficiente para clase)
messages	La conversación como lista de mensajes con rol
max_tokens	Longitud máxima de la respuesta (aquí 100)
temperature	Creatividad: 0 = literal, 1 = desatado
respuesta.choices[0].message.content	El texto de la respuesta

Mensajes y roles

Una conversación con la API es una **lista de mensajes**, y cada mensaje lleva un rol:

Rol	Quién habla	Ejemplo
system	Las instrucciones del asistente	“Eres un profesor de Python divertido.”
user	El humano	“Explica qué es un Lock en 2 frases.”
assistant	Las respuestas previas del modelo	(lo rellena la API)

💡 El mensaje system es la clave del truco: define **el carácter y las reglas** del asistente. El user es la pregunta, y en una conversación larga vas encadenando user → assistant → user → ...

Alternativa con httpx (sin librería oficial)

Si no quieres la librería openai, el mismo chat es una petición HTTP normal. Esto te enseña lo que hay debajo:

```
import httpx, os
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("OPENAI_API_KEY")

resp = httpx.post(
    "https://api.openai.com/v1/chat/completions",
    headers={"Authorization": f"Bearer {API_KEY}"},
    json={
        "model": "gpt-3.5-turbo",
        "messages": [{"role": "user", "content": "Hola!"}]
    }
)
print(resp.json()["choices"][0]["message"]["content"])
```

Fíjate: es un POST con un JSON en el cuerpo y la clave en la cabecera. La librería oficial no es magia, solo envuelve esto.

Consumo y coste

OpenAI **no es gratis**, pero para uso educativo es casi simbólico:

Dato	Valor
Precio de GPT-3.5-turbo	~\$0.0015 por 1.000 tokens
Tokens de una pregunta normal	~100 tokens
Coste de esa pregunta	~\$0.00015 (¡menos de un céntimo!)

Un token es, aproximadamente, una pieza de palabra. Con `max_tokens` controlas cuánto gastas por llamada.

📖 Las preguntas tontas del curso te lo recuerdan en el cierre: “¿OpenAI es caro? GPT-3.5-turbo cuesta ~\$0.0015 por 1000 tokens. Muy barato para pruebas.”



Mini-chequeo

1. ¿En qué cabecera y con qué esquema viaja la clave de OpenAI?
2. ¿Qué rol define el comportamiento del asistente? ¿Cuál es la pregunta del usuario?
3. ¿Qué hace `temperature=0.9` frente a `temperature=0`?

>  Respuestas



Resumen en 3 frases

- OpenAI recibe una lista de mensajes con roles (`system`, `user`, `assistant`) y devuelve el texto en `choices[0].message.content`.
- La clave viaja en `Authorization: Bearer`, y se controla el gasto con `max_tokens` y `temperature`.
- Sin la librería oficial, todo se reduce a un POST a `/v1/chat/completions` con `httpx`.



Vocabulario rápido

Término	Idea general
Chat completions	El endpoint que genera texto de conversación

Término	Idea general
messages	La lista de mensajes que define la conversación
system	Rol que fija las reglas del asistente
temperature	Parámetro de creatividad (0-1)
max_tokens	Tope de longitud (y de coste) de la respuesta
Bearer	Esquema de autorización en la cabecera

 [Volver al índice de la unidad](#) · **Anterior:** [03 · OpenWeatherMap](#) · **Siguiente:** [05 · Rate limiting](#)

05 — Rate limiting

Límites, cuotas y el temido 429 🌈

📖 Estás en: 📌 U07 · APIs Comerciales → 05 · Rate limiting

📌 La idea en una frase

Toda API comercial **limita cuánto puedes pedir**: si te pasas, responde **429 Too Many Requests**, y tu programa debe esperar antes de reintentar.

No es un castigo: es protección. Un cliente desbocado derribaría el servidor de todos los demás usuarios.

🕒 ¿Qué es el rate limit?

El **rate limit** es la cuota de peticiones que tu API key puede hacer en un intervalo de tiempo. Si la superas, la API deja de responderte.

API	Límite típico del plan gratis
OpenWeatherMap	60 peticiones/minuto
GitHub API	60 peticiones/hora
OpenAI	Por tokens por minuto (varía con el plan)

Estos límites existen por una razón sencilla: **una API no es tu base de datos**. Un bucle sin freno que pregunta el tiempo de 1.000 ciudades en un segundo es exactamente el comportamiento que quieren cortar.

🐢 El error 429

Cuando te pasas del límite, la respuesta es:

```
HTTP/1.1 429 Too Many Requests
```

Y OpenWeatherMap añade un mensaje en el JSON:

```
{ "cod": 429, "message": "You have exceeded your quota." }
```

💡 Algunas APIs van más allá y **bloquean tu IP** temporalmente si insistes. Por eso el 429 se toma en serio: esperas, no machacas.

😴 Esperar antes de reintentar

La respuesta natural a un 429 es **parar, esperar y volver a intentar**:

```
import requests
import time

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": "TU_API_KEY", "units": "metric"}
)

if resp.status_code == 429:
    print("☁️ Demasiadas peticiones. Esperando...")
    time.sleep(5) # Respiramos 5 segundos antes de reintentar
```

Si quieres comprobar cuánto te queda, muchas APIs mandan cabeceras con la cuota restante (por ejemplo X-RateLimit-Remaining).

🔄 Reintentos inteligentes (backoff)

Esperar siempre lo mismo no es la mejor idea: si el servidor está saturado, mejor ir espaciando cada vez más los reintentos. Ese es el **backoff exponencial**:

```
import time, requests

def conectar_con_backoff(url, params, max_intentos=5):
    for intento in range(max_intentos):
        try:
            resp = requests.get(url, params=params, timeout=5)
            resp.raise_for_status()
            return resp.json()
        except:
            espera = 2 ** intento # 1, 2, 4, 8, 16 segundos
            print(f"⚠ Intento {intento+1} fallido. Esperando {espera}s...")
            time.sleep(espera)
    raise Exception("No se pudo conectar tras varios intentos")
```

La progresión de esperas con $2 \times \text{intento}$:

```
Intento 1 fallido → espera 1s
Intento 2 fallido → espera 2s
Intento 3 fallido → espera 4s
Intento 4 fallido → espera 8s
Intento 5 fallido → espera 16s → se rinde
```

El **backoff exponencial** evita saturar un servidor que ya está tocado. Es la técnica que usan los clientes profesionales y la retomarás en la [U11 · asyncio](#) para verla de otra forma.

Mini-chequeo

1. ¿Qué código HTTP indica que te has pasado del límite de peticiones?
2. ¿Por qué no es buena idea reintentar sin parar cuando ves un 429?
3. ¿Cuánto espera conectar_con_backoff antes del intento 4?

>  Respuestas

Resumen en 3 frases

- Las APIs limitan tus peticiones por tiempo (cuota) y te avisan con **429** cuando te pasas.
- La respuesta profesional es esperar y reintentar con calma, nunca machacar.

- El **backoff exponencial** duplica la espera entre intentos (1, 2, 4, 8...) para no saturar al servidor.



Vocabulario rápido

Término	Idea general
Rate limit	Cuota de peticiones por tiempo
Cuota	El máximo que tu plan te permite
429	Too Many Requests: te has pasado del límite
Backoff exponencial	Espera que se duplica entre reintentos
X-RateLimit-Remaining	Cabecera con las peticiones que te quedan



[Volver al índice de la unidad](#) · Anterior: [04 · OpenAI](#) · Siguiente: [06 · Errores HTTP](#)

06 — Errores HTTP

401, 403, 429, 500: el servidor te está hablando ⚠

📖 Estás en: 🧰 U07 · APIs Comerciales → 06 · Errores HTTP

La idea en una frase

`requests` no lanza errores por sí solo: tú compruebas el `status_code` o llamas a `raise_for_status()`, y capturas las excepciones para que tu programa no reviente.

Las APIs comerciales **fallan**. Y fallan a menudo. Prepárate.

Los códigos que verás

Estos son los códigos que una API comercial te va a devolver en la vida real:

Código	Qué significa	Causa típica
400	Bad Request	Enviaste mal el JSON o los parámetros
401	Unauthorized	La API key falta o es inválida
403	Forbidden	La key es válida pero sin permiso para eso
404	Not Found	La URL o el recurso no existe
429	Too Many Requests	Te pasaste del rate limit (punto 5)
500	Internal Server Error	El servidor de la API está tocado
503	Service Unavailable	La API está de mantenimiento o caída

💡 La regla mnemotécnica de la U06 sigue valiendo: los **4xx** son tu culpa (mira la URL, la key o los parámetros), los **5xx** son del servidor.

`raise_for_status()` y `try/except`

requests no lanza excepción cuando recibe un 404 o un 500: te devuelve la respuesta y sigues. Para convertir los errores HTTP en excepciones gestionables existe `raise_for_status()`:

```
import requests

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": "falsa", "units": "metric"}
)
print(resp.status_code) # 401 – la key es inválida

resp.raise_for_status() # Lanza una excepción HTTPError si es 4xx o 5xx
```

⚠ **Error típico de novato:** llamar a `resp.json()` sin comprobar el código. Si el cuerpo es un error (o no es JSON), `resp.json()` lanza una excepción y el programa revienta con un mensaje que no dice nada útil.



Una llamada segura de verdad

Juntando los códigos, el timeout y los reintentos, la función robusta queda así:

```

import requests

def llamada_segura(url, params, max_intentos=3):
    for intento in range(max_intentos):
        try:
            resp = requests.get(url, params=params, timeout=10)
            resp.raise_for_status() # Lanza excepción si 4xx o 5xx
            return resp.json()

        except requests.exceptions.Timeout:
            print(f"🕒 Timeout (intento {intento+1})")

        except requests.exceptions.HTTPError as e:
            codigo = e.response.status_code
            if codigo == 429: # Rate limit
                print("🌧 Demasiadas peticiones. Esperando...")
                import time
                time.sleep(5)
                continue
            elif codigo == 401:
                print("🔑 API key inválida")
                return None
            else:
                print(f"❌ HTTP {codigo}: {e.response.text}")
                return None

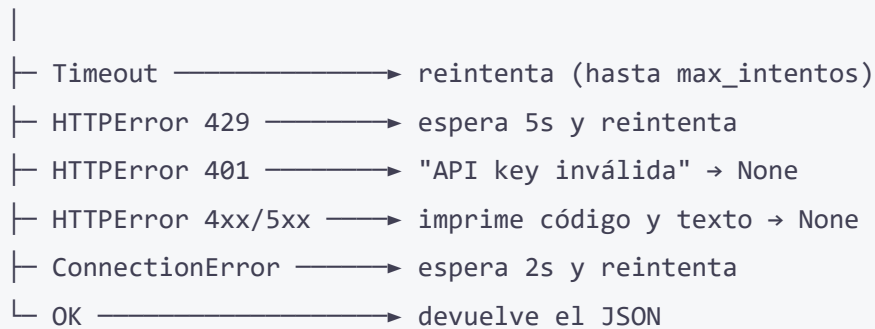
        except requests.exceptions.ConnectionError:
            print(f"🚫 Error de conexión (intento {intento+1})")
            import time
            time.sleep(2)

    return None

```

El flujo de decisión, en un vistazo:

```
requests.get(timeout=10)
```



Mini-chequeo

1. ¿Qué diferencia hay entre 401 y 403?
2. ¿Qué hace `raise_for_status()` y qué excepción lanza?
3. ¿Por qué no debes llamar a `resp.json()` sin comprobar antes el código de estado?

>  Respuestas



Resumen en 3 frases

- Los códigos de una API comercial se dividen en 4xx (tu error) y 5xx (error del servidor).
- `raise_for_status()` convierte los códigos malos en excepciones capturables con `try/except`.
- Una llamada segura combina `timeout`, comprobación de códigos y reintentos, como `llamada_segura`.



Vocabulario rápido

Término	Idea general
401 / 403	No autenticado / autenticado sin permiso
429	Rate limit superado
500 / 503	Error del servidor / API caída
<code>raise_for_status()</code>	Lanza <code>HTTPError</code> si el código es 4xx o 5xx

Término	Idea general
<code>requests.exceptions.Timeout</code>	La petición no obtuvo respuesta a tiempo
<code>requests.exceptions.ConnectionError</code>	No se pudo conectar al servidor
 Volver al índice de la unidad · Anterior: 05 · Rate limiting · Siguiente: 07 · Seguridad y buenas prácticas	

07 — Seguridad y buenas prácticas

Protege tus claves como si fueran la llave de tu casa 🗝️

📖 Estás en: 📌 U07 · APIs Comerciales → 07 · Seguridad y buenas prácticas

📌 La idea en una frase

Una **API key filtrada es una factura en tu nombre**: nunca se sube a GitHub, se **rota** si se filtra y se guarda siempre en el `.env`.

Cualquiera con tu clave puede usarla (y gastar tu cuota, o peor, tu dinero). La seguridad aquí es una decisión de cada `git commit`.

🚫 Lo que nunca debes hacer

Esto funciona, pero es una bomba de relojería:

```
# ❌ MAL: la clave incrustada en el código
import requests

API_KEY = "sk-1234567890abc" # 🚨 ¡Alguien la verá!

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)
```

En cuanto hagas `git push`, esa cadena viaja a GitHub y queda ahí **para siempre** (aunque luego la borres). El servidor de OpenWeatherMap cobrará en tu cuenta cada uso que haga quien la encuentre.

La versión correcta:

```
#  BIEN: la clave vive en el .env, el código no sabe nada
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)
```

El .gitignore y el commit

El .gitignore es tu primera línea de defensa:

```
.gitignore
```

```
.env
```

 **Comprueba antes de commitear:** `git status` debe mostrar tu `.env` como ignorado. Si aparece en la lista, para: no hagas `git add .` sin mirar.

Y si algún día te preguntas “¿puedo guardar la API key en el código para pruebas?” — la respuesta honesta:

Solo si el código **nunca** va a GitHub. Mejor acostúmbrate a `.env` desde el principio: es un hábito, no una técnica.

Rotación de claves

Si una clave se filtra (la subiste a GitHub, la enviaste por correo, la viste en un log), la solución **no es borrar** —ya es tarde— sino **rotar**:

1. Entra en el panel del proveedor (OpenWeatherMap, OpenAI...).
2. **Revoca** la clave filtrada (la anulas al momento).
3. **Genera una nueva** clave y actualiza tu `.env`.

La metáfora del [punto 1](#): si pierdes el llavero del edificio, no vales con “olvidé dónde lo dejé”. Cambias la cerradura.



Checklist de buenas prácticas

☐ Práctica	Por qué
☐ Claves en el <code>.env</code>	El código queda limpio y sin secretos
☐ <code>.gitignore</code> con <code>.env</code>	Git no se lleva tus claves al repo
☐ Revisar <code>git status</code> antes de commit	Cazas errores antes de que sean públicos
☐ Rotar la clave si se filtra	Anulas el daño al momento
☐ Clave en la cabecera, no en la URL	La URL queda en logs e historial
☐ No usar la misma clave en varios proyectos	Un fallo no compromete todo



Mini-chequeo

1. ¿Por qué borrar una clave del código no basta si ya subió a GitHub?
2. ¿Qué es rotar una clave y cuándo se hace?
3. Escribe el `.gitignore` mínimo de este módulo.

>  Respuestas



Resumen en 3 frases

- Una clave en el código es una clave en GitHub: el `.env` + `.gitignore` la mantienen fuera del repo.
- Si una clave se filtra, la única solución real es **rotarla** (revocar y crear otra).
- La seguridad no es una función, es un hábito de cada commit: revisar `git status` antes de subir.



Vocabulario rápido

Término	Idea general
Filtrar una clave	Que una clave salga a un sitio público (GitHub, logs)
Rotar	Revocar la clave comprometida y generar una nueva
.gitignore	Archivo que excluye ficheros de git
Secretos	Credenciales que nunca deben hacerse públicas
Panel del proveedor	Web donde generas y revocas tus claves

 [Volver al índice de la unidad](#) · **Anterior:** [06 · Errores HTTP](#) · **Siguiente:** [08 · Práctica: APIs comerciales](#)

08 — Práctica: APIs comerciales

Sé el código, el programa completo y los ejercicios del lápiz ✎

📖 Estás en: 📁 U07 · APIs Comerciales → 08 · Práctica: APIs comerciales

La idea en una frase

Aquí juntas todo lo aprendido en un solo programa: un **cliente de APIs comerciales** que pide el tiempo a OpenWeatherMap y charla con OpenAI, con las claves en el `.env` — y un paseo “siendo” la petición.

De aquí saldrás con el programa más real del módulo: dos APIs de pago consumidas como se hace en producción.

El programa completo: tiempo + cerebro

```

import requests
from dotenv import load_dotenv
import os
from openai import OpenAI

load_dotenv()
OPENWEATHER_KEY = os.getenv("OPENWEATHER_API_KEY")
OPENAI_KEY = os.getenv("OPENAI_API_KEY")

if not OPENWEATHER_KEY or not OPENAI_KEY:
    raise ValueError("❌ Faltan claves en .env")

def clima(ciudad):
    """Devuelve y muestra el tiempo actual de una ciudad."""
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": ciudad, "appid": OPENWEATHER_KEY, "units": "metric", "lang": "es"}
    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"❌ {ciudad}: {resp.json().get('message', 'error desconocido')}")
        return
    datos = resp.json()
    print(f"☀️ {datos['name']}: {datos['main']['temp']}°C "
          f"({datos['weather'][0]['description']}")

def preguntar_a_gpt(pregunta):
    """Envía una pregunta a OpenAI y devuelve la respuesta."""
    cliente = OpenAI(api_key=OPENAI_KEY)
    respuesta = cliente.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "Eres un profesor de Python divertido."},
            {"role": "user", "content": pregunta}
        ],
        max_tokens=100,
        temperature=0.7
    )
    return respuesta.choices[0].message.content

if __name__ == "__main__":

```

```
clima("Madrid")
clima("Barcelona")
clima("Sevilla")
print("\n💬 GPT dice:", preguntar_a_gpt("¿Qué es un Lock en 2 frases?"))
```

Fíjate en el esqueleto que se repite en toda la unidad: clave del `.env` → `requests.get/OpenAI` → comprobar / parsear → usar los datos. Es el mismo patrón del [punto 6 de la U06](#), ahora con API key.

★ Sé el código: la consulta meteorológica

“Sé el programa que pregunta por el tiempo, desde que escribes ‘python clima.py’ hasta que ves el resultado.”



```
$ python clima.py Barcelona
```

Traza interna:

1. `load_dotenv()` → carga `.env` → obtiene `API_KEY`
2. Construye URL: `https://api.openweathermap.org/data/2.5/weather?q=Barcelona&appid=abc123...&units=metric&lang=es`
3. `requests.get()` → abre socket TCP
 - DNS lookup: `api.openweathermap.org` → `104.16.x.x`
 - Conexión TLS (HTTPS)
 - Envía petición HTTP GET
4. 🕒 Latencia de red (~200ms)
5. Recibe HTTP 200 + JSON:
HTTP/1.1 200 OK
Content-Type: application/json

```
{"name":"Barcelona","main":{"temp":21.3},"weather":[{"description":"nubes dispersas"}]}
```
6. `resp.json()` → dict de Python
7. Extrae `datos['main']['temp'] = 21.3`
8. Extrae `datos['weather'][0]['description'] = "nubes dispersas"`
9. `print` → " 🌤️ Barcelona: 21.3°C"
10. `print` → " nubes dispersas" 🌤️

En los pasos 3 a 5 ocurre lo que en la U04 hacías a mano con sockets: DNS, TCP, TLS y el GET HTTP. La única novedad frente a la U06 es el paso 1: la **API key** que ahora viaja en la petición.

🔧 Herramientas para probar APIs

Antes de escribir Python, puedes probar una API a mano con estas herramientas:

Herramienta	Tipo	Para qué
Postman	GUI	Probar APIs, colecciones, tests
curl	CLI	<code>curl https://api.github.com/users/python</code>
httpie	CLI moderno	<code>http https://api.github.com/users/python</code>
Insomnia	GUI	Alternativa open source a Postman

💡 Con Postman o curl puedes ver la respuesta cruda de OpenWeatherMap (con tu clave en `appid`) antes de escribir una sola línea de Python. Cuando la API responde en la herramienta, pasas el mismo GET a `requests`.

Aprieta el lápiz

1. **Clima en 3 ciudades:** Usa OpenWeatherMap para mostrar el clima de Madrid, Barcelona y Sevilla a la vez.
2. **Chat con GPT:** Pregúntale a GPT-3.5 qué es un semáforo en Python y que te ponga un ejemplo.
3. **API con errores:** Conéctate a una URL que no existe y captura el error. Luego a una con API key falsa (401).
4. **Backoff test:** Crea una función que intente conectarse a un servidor que no existe y muestre el tiempo de espera entre intentos.

>  Soluciones

Mini-chequeo

1. ¿Qué tres bloques forman el programa completo de esta práctica?
2. En la traza del “Sé el código”, ¿qué nuevo paso añade la U07 frente a la U06?
3. ¿Para qué te sirve Postman o curl antes de escribir Python?

>  Respuestas

Resumen en 3 frases

- Un cliente de APIs comerciales = claves del `.env` + llamada a la API + parseo de la respuesta.
- El programa completo consume OpenWeatherMap (tiempo) y OpenAI (chat) siguiendo el mismo patrón `requests` de la U06.
- Postman, curl, httpie e Insomnia te dejan probar la API a mano antes de programar.



Vocabulario rápido

Término	Idea general
Cliente de API	Programa que consume una API
Traza interna	El camino paso a paso de la petición
.env	Donde viven las claves que el programa carga
Postman / Insomnia	GUIs para probar APIs
curl / httpie	CLIs para probar APIs

 [Volver al índice de la unidad](#) · Anterior: [07 · Seguridad y buenas prácticas](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé la petición con API key, laboratorio real y el ring final de la unidad 🧠

📖 Estás en: 🧪 U07 · APIs Comerciales → 09 · Cierre

Has terminado la teoría: API keys, variables de entorno, OpenWeatherMap, OpenAI, rate limits, errores HTTP, seguridad y el programa completo. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real contra dos APIs de verdad con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé la petición

Eres una petición HTTP con una API key. Acabas de nacer en un `requests.get()` dentro del portátil de un estudiante y tu destino es `api.openweathermap.org`. Empieza el viaje.

¿Qué pasa?

1. `requests` te construye: `https://api.openweathermap.org/data/2.5/weather?q=Sevilla&appid=TU_API_KEY&units=metric&lang=es`. Llevas el **carnet de identidad** (la API key) en la query string.
2. Abres la conexión: primero el **DNS lookup** (`api.openweathermap.org` → una IP), luego el **three-way handshake TCP** y el **handshake TLS** que te cifra para el viaje.
3. Atraviesas Internet hasta el servidor de OpenWeatherMap y te entrega el proceso que te espera en el puerto 443.
4. El servidor **te lee la carnet**: comprueba tu `appid` contra su base de claves y tu plan (¿cuánto puedes pedir por minuto?).
5. Te responde **200 OK** con un JSON: `{"name":"Sevilla","main":{"temp":18.5},...}`.
6. De vuelta en Python, `requests` parsea el JSON y te convierte en un **dict** (`datos["main"]["temp"]` → 18.5).
7. El estudiante imprime 🌤️ Sevilla: 18.5°C. Misión cumplida.

Todo en ~200 ms. Con tu identidad verificada en cada viaje.

💡 **Ahora tú:** ¿y si la clave fuera inválida? En el paso 4 el servidor no te reconoce: te responde **401 Unauthorized** con un JSON de error. Si el estudiante no comprueba el

`status_code` y llama directo a `resp.json()`, el programa muere con un mensaje que no dice nada útil. Por eso existe `raise_for_status()` (punto 6).

Fireside Chat: REST vs SOAP vs GraphQL

Tres estilos de API se sientan junto a la chimenea a discutir, de una vez, quién es el mejor.

REST: — Mira, soy el estándar. Simple, recursos, JSON. Todo el mundo me conoce.

SOAP: — Yo fui el rey en los 2000. XML, WSDL, muy estructurado. Aún vivo en bancos.

GraphQL: — Yo soy el moderno. Una sola URL, pides exactamente lo que necesitas, ni más ni menos.

REST: — ¿Y eso es bueno? Yo tengo URLs claras: `/usuarios/5`, `/productos`.

GraphQL: — Sí, pero si quieres el nombre y el correo del usuario y los títulos de sus posts, ¿cuántas peticiones necesitas?

REST: — Dos: `/usuarios/5` y `/usuarios/5/posts`.

GraphQL: — Yo con una: `query { usuario(id:5) { nombre email posts { titulo } } }`.

REST: — Vale, eres más eficiente. Pero yo soy más simple para empezar.

GraphQL: — Tienes razón. Para proyectos pequeños, REST gana. Para grandes, yo escalo mejor.

Moraleja: en este módulo manda REST: OpenWeatherMap y OpenAI (vía chat completions) son APIs REST con JSON. SOAP vive en los bancos y GraphQL es la alternativa eficiente para APIs grandes.



¿Quién soy?

1. Soy el carnet de identidad que la API comercial te da al registrarte.
2. Soy el código de estado cuando te pasas del límite de peticiones.
3. Soy el rol de mensaje de OpenAI que define el comportamiento del asistente.
4. Soy la función de `requests` que lanza una excepción si la respuesta es 4xx o 5xx.
5. Soy el archivo donde guardas tus claves y que nunca, jamás, se sube a git.
6. Soy la técnica que duplica la espera entre reintentos (1, 2, 4, 8...).



CONRAD VS EL MUNDO: “clave subida a GitHub”

CONRAD: — “Clásico: haces `git add .` && `git commit` && `git push` con la API key escrita en el código (o sin `.env` en el `.gitignore`). Razones por las que estás vendido: 1) **La clave queda en el historial de git para siempre**, aunque la borres después. 2) **Cualquiera que encuentre tu repo la usa**: tu cuota gratis se agota en una tarde, o peor, tu tarjeta paga el exceso. 3) La clave no caduca sola: es tu llavero, y lo dejaste en la puerta.”

CONRAD: — “Y lo mejor: *‘es que era un repo privado’*. ¡Pues claro! Los repos privados se hacen públicos, se comparten, se filtran. La seguridad no depende de la visibilidad del repo, sino de que el secreto **no esté ahí**.”

CONRAD: — “Y no me vengas con *‘¿será que la clave no se ve?’*. Ya lo sabes: **rota**. Revoca la clave filtrada en el panel del proveedor, genera una nueva, ponla en el `.env`, y añade `.env` al `.gitignore`. Primero el daño, luego el arreglo.”



Laboratorio de tortura: OpenWeatherMap + OpenAI

Duración: 1 hora **Herramienta:** Python 3 (requests, python-dotenv, openai) y conexión a Internet

Escenario: monta un programa que consume las dos APIs comerciales de la unidad con tus claves reales guardadas en el `.env`. Es lo que repetirás en cualquier proyecto que necesite tiempo o IA.

Tareas paso a paso:

1. **Crea el `.env`** con tus dos claves reales: `OPENWEATHER_API_KEY=...` y `OPENAI_API_KEY=sk-....`. Añade `.env` al `.gitignore` antes de hacer nada más.
2. **Clima de tu ciudad:** escribe `clima.py` que cargue la clave con `load_dotenv()`, haga GET a OpenWeatherMap y muestre temperatura, sensación, humedad y descripción.
3. **Compara dos ciudades:** llama a la misma función con dos ciudades y muestra cuál está más caliente.

4. **GPT responde:** escribe `gpt.py` que pregunte a GPT-3.5 qué es un Lock en Python con `max_tokens=100` y `temperature=0.7`.
5. **Gestiona el 401:** cambia la `appid` a una clave falsa y haz que el programa detecte el error sin reventar (comprueba `status_code`).
6. **Simula un 429:** haz un bucle de 70 peticiones seguidas a OpenWeatherMap (el plan gratis permite 60/min). ¿Qué código recibes a partir de la 61?

Fallo intencionado: en la tarea 2, en lugar de leer la clave del `.env` con `os.getenv`, escríbela a fuego en el código (`API_KEY = "TU_API_KEY"`) y haz `git add . && git commit`. La clave queda en el historial de git para siempre, aunque luego la borres y añadas el `.env` al `.gitignore`. Es el fallo más caro de esta unidad: el código funciona, pero acabas de regalar tu carnet de identidad.

Pista 1: las claves son como las llaves de casa: si las dejas en la puerta (en un commit público), no vale con “recogerlas” (borrarlas). Hay que **cambiar la cerradura**: revocar la clave en el panel del proveedor y generar una nueva. Eso es rotar (punto 7).

Pista 2: comprueba tú mismo que el daño es real con `git log -p`: ahí verás la clave escrita en el commit aunque ya la hayas borrado del fichero. Y para el resto del laboratorio, vuelve al `.env`: `load_dotenv() + os.getenv()` es la única vía.

Logros de esta unidad

Logro	Cómo conseguirlo
 Key Holder	Obtener tu API key en OpenWeatherMap y OpenAI y guardarla en el <code>.env</code>
 Env Master	Cargar claves con <code>python-dotenv</code> y proteger el repo con <code>.gitignore</code>
 Weather Rider	Consumir OpenWeatherMap y parsear <code>main</code> y <code>weather</code> del JSON
 GPT Whisperer	Llamar a chat completions con roles, <code>max_tokens</code> y <code>temperature</code>
 429 Survivor	Sobrevivir al rate limit con esperas y backoff exponencial
 Key Guardian	Rotar una clave filtrada y explicar por qué nunca va en el código

Atrévete a pensar

1. ¿Por qué las API keys no deben ir en la URL ni en el código?
2. ¿Qué diferencia hay entre 401 y 403?
3. ¿Qué es el rate limit y por qué las APIs comerciales lo imponen?
4. ¿Por qué el backoff exponencial es mejor que esperar siempre lo mismo?
5. ¿Qué harías si descubres que tu API key se ha subido a GitHub?

> 💡 Soluciones

🧩 Crucigrama de bits

Horizontal:

1. Código de estado cuando te pasas del límite de peticiones (3 letras)
4. Archivo donde guardas tus claves y nunca se sube a git (3 letras)
5. Parámetro de OpenWeatherMap que pone las unidades en grados (5 letras)
8. Código de estado cuando la API key es inválida (3 letras)

Vertical:

2. Rol de mensaje de OpenAI que define el comportamiento del asistente (6 letras)
3. Proveedor de clima con API key gratuita (14 letras)
6. Librería para peticiones HTTP en Python (8 letras)
7. Espera exponencial entre reintentos (7 letras)

> 📝 Soluciones

💬 Entrevista de trabajo

1. “¿Qué es una API key y cómo la enviarías al servidor?”
2. “¿Cómo guardas las credenciales en una aplicación Python? ¿Por qué no en el código?”
3. “¿Qué es un rate limit? ¿Cómo gestionarías un 429?”
4. “Escribe en Python una llamada a OpenWeatherMap con requests y procesa la respuesta JSON.”
5. “¿Qué diferencia hay entre REST, SOAP y GraphQL? ¿Cuál usarías y por qué?”

💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, recorre el [punto 1](#): la API key es el carnet de identidad, va en la query (appid) o en Authorization: Bearer, y nunca en el código. Para la 4, escribe el esqueleto del [punto 3](#): params (con units=metric), comprobar

`status_code`, `resp.json()` y acceder a `main` y `weather`. Si además mencionas el `.env` y el `raise_for_status`, la entrevista es tuya.

No hay preguntas tontas

? ¿Me pueden banear por usar mucho la API?

Sí. Todas tienen límites (rate limits). Lee la documentación. OpenWeatherMap gratis: 60 peticiones/minuto.

? ¿Qué pasa si me paso del límite?

HTTP 429 (Too Many Requests). Algunas APIs bloquean tu IP por un tiempo.

? ¿OpenAI es caro?

GPT-3.5-turbo cuesta ~\$0.0015 por 1000 tokens. Una pregunta normal son ~100 tokens = \$0.00015. Muy barato para pruebas.

? ¿Puedo guardar la API key en el código para pruebas?

Solo si el código nunca va a GitHub. Mejor acostúmbrate a `.env` desde el principio.

Poscréditos

Tu petición parte con su API key cifrada por TLS y vuelve con el JSON del tiempo bajo el brazo.

El 429 llegó, esperaste con backoff, y la petición siguiente salió bien.

Y las claves se quedaron en el `.env`, lejos de GitHub, donde deben vivir.

PRÓXIMAMENTE EN U08: *hash y cifrado clásico. Las contraseñas que no se pueden leer: MD5, SHA y el viejo cifrado César.*

Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Cubierto
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3-5 y 8 + ⚡ Laboratorio de tortura
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1, 5-6 y 8 + ⚡ Laboratorio de tortura

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en la **U10 · Servidores Concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **U11 · asyncio y disponibilidad**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Práctica APIs comerciales](#) · Siguiente: [U08 · Hash y Cifrado Clásico](#)

U08 — Hash y Cifrado Clásico

La huella digital que protege tus contraseñas 🗝️

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 Hilo → 🗝️ Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 🧪 APIs comerciales → 🗝️ **Hash y Cifrado Clásico** → 🧬 Cifrado → ⚖️ Servidores → ⌚ asncio

Cada vez que escribes una contraseña en un formulario, un servidor decide si es la correcta sin saber cuál es. ¿Cómo es posible? Gracias al **hash**: una función que convierte cualquier texto en una huella digital imposible de revertir. En esta unidad aprenderás a usar MD5, SHA-1 y SHA-256 para guardar contraseñas sin riesgo, a protegerlas con **sal** frente a tablas rainbow, y conocerás al abuelo de todos los cifrados: el **César**.

Entenderás también los **principios de seguridad** que sostienen todo el edificio (Zero Trust, mínimo privilegio, defensa en profundidad) y sabrás distinguir cuándo toca hashear (integridad, contraseñas) y cuándo cifrar (confidencialidad). El siguiente tema, [U09 · Cifrado Moderno](#), tomará el relevo con AES y RSA, que sí se pueden deshacer con una clave.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un hash, sus propiedades (determinismo, unidireccionalidad, longitud fija, efecto avalancha) y su papel como huella digital.
- Comparar MD5, SHA-1, SHA-256 y SHA-512 con criterio, sabiendo cuáles están rotos y cuáles son seguros.
- Diseñar un sistema de **registro/login** que almacene solo hashes, nunca la contraseña original.
- Explicar la **sal** (salt) y cómo neutraliza las tablas rainbow.
- Cifrar y descifrar con el **cifrado César**, y romperlo por fuerza bruta en 25 intentos.
- Distinguir cuándo usar hash (integridad) y cuándo cifrado (confidencialidad).
- Verificar la integridad de un archivo con su checksum SHA-256.

- Aplicar los principios básicos de seguridad (Zero Trust, mínimo privilegio, defensa en profundidad) a cualquier sistema.

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Principios de seguridad	La base de todo: cero confianza, mínimo privilegio y defensa en profundidad	Todos
02 · Qué es un hash	La huella digital y sus propiedades, con hashlib en acción	Todos
03 · MD5, SHA-1 y SHA-256	Tabla comparativa, hash de archivos y cuál usar hoy	Todos
04 · Hash de contraseñas	Registro y login guardando solo la huella, nunca la original	Todos
05 · Hash con sal	Cómo la sal arruina las tablas rainbow (y el Pool Puzzle)	Todos
06 · Cifrado César	El abuelo de la criptografía, paso a paso y por fuerza bruta	Todos
07 · Hash vs Cifrado	El ring: integridad contra confidencialidad	Todos
08 · Buenas prácticas y verificación	Checksums de descargas, cuándo hash y cuándo cifrar, ejercicios	Todos
09 · Cierre	Sé el hash, Fireside, Laboratorio de tortura, Crucigrama...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Inicial resuelto

● Inicial por resolver

👤 Avanzado resuelto

★ Avanzado por resolver

✓ Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen la integridad y la confidencialidad de los datos.

CE	Criterio	Dónde se cubre
a)	Principios básicos de seguridad	✓ Punto 1 + Punto 7
c)	Implementa funciones hash (MD5, SHA)	✓ Puntos 2-5 + ⚡ Laboratorio (punto 9)
h)	Conoce sistemas de roles y RBAC	✓ Punto 1 (principios: mínimo privilegio)

RA5b (tipos de cifrado), RA5d (AES), RA5e (RSA), RA5f (firmas digitales) y RA5g (cifrado híbrido) se cubren en la **U09 · Cifrado Moderno**.

📖 ¿Por dónde empiezo?

¿Vienes de la U07 y dominas las APIs comerciales? Perfecto, este tema no necesita nada de lo anterior: la seguridad es un campo nuevo y autónomo. Si además ya sabes qué es una API key y por qué no debe ir en el código, tienes el chip mental adecuado: en esta unidad verás por qué tampoco se guardan las contraseñas en claro. Arranca en el [punto 1](#), que sienta los cimientos de todo lo demás.

¿Ya sabes qué es un hash y solo necesitas el César o el diseño de contraseñas? Puedes saltar a los [puntos 4](#), [5](#) y [6](#). Pero si vienes de cero en criptografía, no te saltes los puntos 1 a 3: las propiedades del hash y la comparativa de algoritmos son la base de todo el módulo.

📌 Primer punto: [01 · Principios de seguridad](#)

▶ Al acabar la unidad, continúa en [U09 · Cifrado Moderno](#).

01 — Principios de seguridad

La base de todo: cero confianza y defensa en profundidad 🇧🇪

📖 Estás en: 🔒 U08 · Hash y Cifrado Clásico → 01 · Principios de seguridad

📌 La idea en una frase

Antes de cifrar una sola letra, tienes que interiorizar los **principios de seguridad**: si no entiendes por qué se hace algo, cualquier herramienta que aprendas después será un martillo mal usado.

Los algoritmos cambian (MD5 cayó, SHA-1 cayó, algún día caerá SHA-256), pero los principios son eternos. Son las reglas del oficio que distinguen a quien “copia código” de quien diseña sistemas seguros.

🏰 Los seis principios que no puedes olvidar

Principio	Traducción al español de la calle
Zero Trust	No te fíes ni de tu sombra. Verifica siempre.
Mínimo privilegio	Da solo los permisos mínimos necesarios.
Defensa en profundidad	No pongas toda la seguridad en una capa.
Cifra todo	En tránsito (TLS) y en reposo (disco).
Rotación de claves	Cambia las claves periódicamente.
No inventes tu cripto	Las bibliotecas existentes ya han sido auditadas.

Desglose de cada uno:

- **Zero Trust** — La red interna no es de fiar por defecto. Un usuario autenticado hoy puede ser un atacante mañana. Por eso cada operación sensible se vuelve a verificar. En esta unidad lo ves con los hashes: ni siquiera confiamos en nuestra propia base de datos, por eso no guardamos contraseñas en claro.

- **Mínimo privilegio** — Nadie debería tener más acceso del que necesita para su trabajo. Un becario no necesita borrar la tabla de usuarios. Esto conecta directamente con **RBAC** (Control de Acceso Basado en Roles): a cada rol le toca un paquete de permisos, y nadie se sale del suyo. Lo verás a fondo en la [U09 · Cifrado Moderno](#), pero el principio lo aplicas ya.
- **Defensa en profundidad** — Si solo tienes una cerradura, alguien con la llave abre la casa entera. La seguridad real apila capas: contraseña con hash + sal, cifrado en disco, firewall, logs. Si una capa cae, las demás siguen.
- **Cifra todo** — Los datos viajan por la red (en tránsito, con TLS) y descansan en el disco (en reposo). En esta unidad ciframos en reposo las contraseñas con hash; en la U09 cifrarás mensajes completos con AES y RSA.
- **Rotación de claves** — Una clave que lleva 5 años en uso es una clave comprometida. Igual que cambias la cerradura cuando se pierde una copia, las claves y contraseñas se renuevan periódicamente.
- **No inventes tu cripto** — La tentación de escribir “mi propio algoritmo” es enorme... y siempre termina mal. Las bibliotecas como `hashlib` o `pycryptodome` llevan décadas siendo atacadas y parcheadas. Tú no puedes hacer eso en un finde.



La analogía de las cerraduras de una casa

Imagina tu casa como un sistema informático. Las **cerraduras** son los mecanismos de seguridad: la puerta principal (contraseña), la alarma (detección de intrusos), la caja fuerte (cifrado de datos sensibles).

- Poner una sola cerradura en la puerta y nada más es **seguridad de una capa**: si alguien copia la llave, entra y listo.
- Dejar la llave bajo el felpudo es el equivalente a guardar contraseñas en **texto plano**: la seguridad existe, pero no sirve de nada.
- Tener cerradura **y** alarma **y** caja fuerte es **defensa en profundidad**: aunque salten la primera, la segunda los delata y la tercera los frena.
- Y lo más importante: **ningún cerrajero serio diseña su propia cerradura desde cero**; compra una certificada. Eso es *no inventes tu cripto*.

Cuando en el [punto 4](#) guardes una contraseña con hash, estarás instalando una cerradura decente. Cuando añadas **sal** en el [punto 5](#), estarás poniendo la segunda vuelta.



La cita que debes saber

“La seguridad no es un producto, es un proceso.” — Bruce Schneier

No es un interruptor que enciendes y olvidas: es una disciplina continua de revisar, parchear, rotar y auditar. Por eso existen los principios: para que el proceso nunca se detenga.



Mini-chequeo

1. ¿Por qué “no inventes tu cripto” es un principio y no una sugerencia?
2. ¿Qué relación hay entre “mínimo privilegio” y RBAC?
3. ¿Por qué el cifrado se aplica “en tránsito” y “en reposo”?

>  Respuestas



Resumen en 3 frases

- La seguridad real se construye sobre **principios** (Zero Trust, mínimo privilegio, defensa en profundidad), no sobre herramientas sueltas.
- **No inventes tu cripto**: usa bibliotecas auditadas, cifra todo y rota las claves.
- Estos principios son el filtro con el que evaluarás cada decisión del resto de la unidad (y de tu vida como programador).



Vocabulario rápido

Término	Idea general
Zero Trust	No confiar en nadie por defecto; verificar siempre
Mínimo privilegio	Cada usuario solo con los permisos que necesita
Defensa en profundidad	Varias capas de seguridad independientes
RBAC	Control de acceso por roles (a cada rol, sus permisos)
Rotación de claves	Renovar claves/contraseñas periódicamente

Término

Idea general

Cifrado en tránsito / en reposo

Proteger datos mientras viajan / mientras se guardan



[Volver al índice de la unidad](#) · **Siguiente:** [02 · Qué es un hash](#)

02 — Qué es un hash

La huella digital de cualquier dato 🔍

📖 Estás en: 🔒 U08 · Hash y Cifrado Clásico → 02 · Qué es un hash

📌 La idea en una frase

Un **hash** es una función que convierte cualquier entrada (un texto, un archivo, una contraseña) en una cadena de **longitud fija**, y lo hace de forma **unidireccional**: no se puede revertir.

Piensa en una **huella dactilar**: aunque conozcas la huella, no puedes reconstruir el dedo. El hash funciona igual: conoces la huella digital del dato, pero el dato original es irrecuperable.

👣 La huella digital

Un hash toma un mensaje de cualquier tamaño y devuelve un resumen de tamaño fijo:

"a"	→
ca978112ca1bbdcafacc231b39a23dc4da786eff8147c4e72b9807785afee48bb	(SHA-256)
"Hola mundo"	→
ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c	(SHA-256)
"En un lugar de la Mancha"	→
e6ff77084bec551e5c5ae84a1b44d4eed6dd9cabf86bea2abbfd510230bb3ec7	(SHA-256)

Fíjate en la clave: da igual que la entrada tenga 1 carácter o 1 millón, el hash **siempre tiene la misma longitud** (64 caracteres en SHA-256). Eso lo convierte en una herramienta ideal para comprobar que “esto que me han enviado es exactamente lo que esperaba”.

🐍 Hash en Python: tres sabores

Con el módulo `hashlib` (de la librería estándar, sin instalar nada) puedes calcular hashes de cualquier texto:

```
import hashlib

texto = b"Hola mundo"

print("MD5:", hashlib.md5(texto).hexdigest())
print("SHA1:", hashlib.sha1(texto).hexdigest())
print("SHA256:", hashlib.sha256(texto).hexdigest())
```

Salida:

```
MD5: f822102f4515609fc31927a84c6db7f8
SHA1: c083106c930790151165b95bd11860724e3836cb
SHA256: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c
```

Nota la diferencia de longitudes: MD5 devuelve **32** caracteres (128 bits), SHA1 **40** (160 bits) y SHA256 **64** (256 bits). En el [punto 3](#) decides cuál usar.

💡 El b antes del texto lo convierte a **bytes**, que es lo que espera hashlib. Si tienes un str, usa `.encode()`.



La analogía de la licuadora

Un hash es como una licuadora: metes lo que sea (una manzana, un ladrillo, un contrato) y siempre sale **batido**. Pero es un batido **unidireccional**: nadie puede reconstruir la manzana a partir del zumo. Añade la propiedad clave: si cambias una sola brizna de la entrada, el batido cambia por completo (eso es el **efecto avalancha**, que verás abajo).



Las cinco propiedades del hash

- **Determinista**: misma entrada → mismo hash. Siempre. Da igual cuántas veces lo calcules.
- **Unidireccional**: no se puede obtener el original a partir del hash.
- **Longitud fija**: MD5 = 128 bits, SHA1 = 160 bits, SHA256 = 256 bits, pase lo que pase con la entrada.
- **Efecto avalancha**: un cambio mínimo (una letra, un bit) cambia el hash **por completo**.
- **Colisiones**: dos entradas diferentes con el mismo hash. En teoría posibles, en la práctica **casi imposibles** en SHA-256 (harían falta más intentos que átomos en el universo).

La propiedad estrella para la seguridad es la **unidireccionalidad**: por eso en el [punto 4](#) guardaremos contraseñas como hash, y no como texto plano.

Mini-chequeo

1. ¿Qué longitud tiene un hash SHA-256 expresado en hexadecimal? ¿Y en bits?
2. Calcula el hash SHA-256 de “Hola mundo”. ¿Qué pasa si cambias una letra?
3. ¿Por qué “unidireccional” es la propiedad más importante para guardar contraseñas?

>  Respuestas

Resumen en 3 frases

- Un hash convierte cualquier entrada en una huella de **longitud fija**, siempre la misma para la misma entrada.
- Es **unidireccional** (irreversible), con **efecto avalancha** ante cambios mínimos y colisiones prácticamente imposibles en SHA-256.
- `hashlib.md5/sha1/sha256(texto).hexdigest()` te da la huella en una línea.

Vocabulario rápido

Término	Idea general
Hash	Función que resume cualquier dato en una cadena fija
Unidireccional	Imposible de revertir
Efecto avalancha	Un bit de cambio altera todo el hash
Colisión	Dos entradas con el mismo hash
hexdigest()	Convierte el hash a texto hexadecimal legible

 [Volver al índice de la unidad](#) · Anterior: [01 · Principios de seguridad](#) · Siguiente: [03 · MD5, SHA-1 y SHA-256](#)

03 — MD5, SHA-1 y SHA-256

Cuál elegir y por qué algunos ya no sirven 📋

📖 Estás en: 🔒 U08 · Hash y Cifrado Clásico → 03 · MD5, SHA-1 y SHA-256

📌 La idea en una frase

No todos los hash son iguales: **MD5 y SHA-1 están rotos** (se pueden fabricar colisiones) y solo sirven para checksums no críticos; **SHA-256 y SHA-512 son los seguros** para todo uso general.

En el [punto 2](#) aprendiste qué es un hash. Ahora toca elegir: la familia SHA-2 (SHA-256, SHA-512) es hoy el estándar de oro, y hashlib te la da gratis.

⚖️ La tabla comparativa

Algoritmo	Bits	¿Seguro?	Uso recomendado
MD5	128	❌ No	Solo checksums no críticos
SHA-1	160	❌ No	Solo legacy
SHA-256	256	✅ Sí	Todo uso general
SHA-512	512	✅ Sí	Cuando necesites más seguridad

El porqué de cada fila:

- **MD5 (1992)** — Durante décadas fue el rey, hasta que en 2004 se demostraron **colisiones prácticas**: es posible fabricar dos archivos distintos con el mismo MD5. Un atacante puede hacer que un programa legítimo y otro malicioso tengan el mismo hash. Roto.
- **SHA-1 (1995)** — Heredó el trono... y en 2017 el equipo de Google logró la primera **colisión real** (dos PDFs distintos, mismo SHA-1). Roto también.
- **SHA-256 / SHA-512 (familia SHA-2, 2001)** — A día de hoy siguen en pie: no se conoce forma práctica de forzarlos. Son los que ves en las descargas de software, los certificados TLS y las cadenas de bloques.

- **SHA-512** — Mismo algoritmo que SHA-256 pero con resumen de 512 bits: más margen futuro, a costa de ser algo más lento (en el [boletín avanzado](#) lo medirás).

⚠ **Nunca uses MD5 o SHA-1 para seguridad.** Son vulnerables a colisiones. Usa SHA-256 o superior. La única excepción es un checksum casual para detectar corrupción accidental (no ataques).

Hash de un archivo: verificar integridad

La aplicación más cotidiana del hash es comprobar que un archivo no se ha corrompido ni manipulado. La página de descarga te da su SHA-256; tú calculas el del archivo que has descargado y comparas:

```
import hashlib

# Hash de un archivo (verificar integridad)
with open("archivo.pdf", "rb") as f:
    hash_archivo = hashlib.sha256(f.read()).hexdigest()
    print(f"SHA256 del archivo: {hash_archivo}")
```

Salida (depende del archivo):

```
SHA256 del archivo: 8a1f9c... (64 caracteres hexadecimales)
```

Dos detalles importantes:

- Se abre en modo binario "rb" porque queremos el hash de los bytes exactos, sin conversión de texto.
- **Un solo bit cambiado en el archivo → un hash completamente distinto** (efecto avalancha del [punto 2](#)). Por eso un checksum te dice si el archivo es *exactamente* el que publicó el autor.

SHA-512 en acción

Si necesitas más margen, el código es idéntico cambiando el nombre:

```
import hashlib

texto = b"Hola mundo"

print("SHA256:", hashlib.sha256(texto).hexdigest())
print("SHA512:", hashlib.sha512(texto).hexdigest())
```

Salida:

```
SHA256: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c
SHA512:
34ddb0edac59e441459e07cf33bd628f53fbbf752141125f069f32081b169f933666c71b2f1b83031da66bc905
```

Fíjate: SHA-512 devuelve **128 caracteres** hexadecimales (512 bits), el doble que SHA-256.



Mini-chequeo

1. ¿Por qué MD5 y SHA-1 no sirven para seguridad?
2. ¿En qué caso seguirías usando MD5 hoy?
3. ¿Para qué abre un archivo en modo "rb" cuando calculas su hash?

>  Respuestas



Resumen en 3 frases

- MD5 y SHA-1 tienen colisiones demostradas: **rotos para seguridad**; SHA-256/SHA-512 son hoy los fiables.
- El hash de un archivo (`open(..., "rb") + sha256().hexdigest()`) es el checksum con el que verificas que una descarga no fue manipulada.
- Cambia solo el nombre de la función en `hashlib` para pasar de 256 a 512 bits.



Vocabulario rápido

Término	Idea general
Colisión	Dos entradas distintas con el mismo hash
Checksum	Hash de un archivo para verificar integridad
SHA-2	Familia segura: SHA-256 y SHA-512
Legacy	Código antiguo que se mantiene pero no se usa en nada nuevo
Modo binario "rb"	Lectura de bytes exactos, necesaria para hashear archivos

 [Volver al índice de la unidad](#) · Anterior: [02 · Qué es un hash](#) · Siguiente: [04 · Hash de contraseñas](#)

04 — Hash de contraseñas

Guarda la huella, nunca la original 🗝️

📖 Estás en: 🗝️ U08 · Hash y Cifrado Clásico → 04 · Hash de contraseñas

La idea en una frase

Cuando un usuario se registra, no guardamos su contraseña: guardamos su **hash**. En el login recalculamos el hash del intento y comparamos hashes. Así, si roban la base de datos, no roban contraseñas.

Este es el momento en que todo lo anterior cobra sentido. Y es el **“Be the code”** de la unidad: te metes dentro del programa que registra a Ana y luego decide si su login es válido.

Be the code: registro y login con hash

“Sé el programa que registra a un usuario y luego verifica su login. Contraseñas seguras, sin almacenar la original.”

```

import hashlib

def registrar(usuario, contraseña):
    hash_contra = hashlib.sha256(contraseña.encode()).hexdigest()
    print(f" Usuario '{usuario}' registrado")
    print(f" Hash: {hash_contra[:16]}...")
    return hash_contra

def login(usuario, contraseña, hash_almacenado):
    hash_intento = hashlib.sha256(contraseña.encode()).hexdigest()
    if hash_intento == hash_almacenado:
        print(f" ✅ {usuario}: login correcto")
        return True
    else:
        print(f" ❌ {usuario}: contraseña incorrecta")
        return False

# Simular registro y login
hash_ana = registrar("Ana", "MiClaveSecreta123")
login("Ana", "MiClaveSecreta123", hash_ana) # ✅
login("Ana", "OtraClave", hash_ana)         # ❌

```

Salida:

```

Usuario 'Ana' registrado
Hash: 33b1f04c2e6e5d3c...
✅ Ana: login correcto
❌ Ana: contraseña incorrecta

```

La máquina nunca ve la contraseña: ve su hash. El registro calcula el hash y lo devuelve (en una app real, lo guardarías en la base de datos). El login vuelve a calcular el hash del intento y lo compara con el almacenado.

La traza paso a paso

1. Ana se registra con "MiClaveSecreta123"
2. Python codifica a bytes → `b"MiClaveSecreta123"`
3. `hashlib.sha256()` procesa → 256 bits de hash
4. `hexdigest()` devuelve 64 caracteres hexadecimales
5. Guardamos en BD: "33b1f04c2e6e5d3c..." (solo el hash)

6. Ana hace login con "MiClaveSecreta123"
7. Calculamos hash del intento → "33b1f04c2e6e5d3c..."
8. Comparamos con el almacenado → ¡COINCIDEN!
9.  Login correcto

10. Un atacante intenta con "OtraClave"
11. Hash del intento → "ff34ee22..." (distinto)
12. No coincide →  Acceso denegado

Fíjate en el paso 8: no comparamos contraseñas, **comparamos hashes**. Y como el hash es determinista ([punto 2](#)), la misma contraseña siempre produce el mismo hash.

La regla de oro

La contraseña original **nunca** se almacena. Solo su hash.

- Si roban la base de datos con hashes, no tienen contraseñas: solo huellas irreversibles.
- Los sistemas serios además añaden **sal** (lo verás en el [punto 5](#)) para que ni siquiera las huellas idénticas se reconozcan entre usuarios.
- Cifrar la contraseña en lugar de hashearla **no** sirve para esto: si el atacante roba también la clave de cifrado (y suele hacerlo), recupera todo. El hash es la elección correcta por ser unidireccional.

 **Y lo que NUNCA debe aparecer en tu código:** `if contraseña == contraseña_guardada:` comparando texto plano, ni un `print(f"Contraseña de Ana: {contraseña}")` en los logs. Eso es *seguridad de una sola capa* (el [punto 1](#) que acabamos de violar).

Mini-chequeo

1. ¿Qué guardamos exactamente en la base de datos al registrar a un usuario?
2. ¿Por qué el login vuelve a calcular el hash en lugar de comparar contraseñas?

3. ¿Por qué cifrar la contraseña sería peor que hashearla para este caso?

>  Respuestas

✓ Resumen en 3 frases

- En el registro se guarda el **hash** de la contraseña; en el login se hashea el intento y se comparan huellas.
- La traza lo deja claro: en ningún momento existe la contraseña original fuera del cerebro (y del teclado) del usuario.
- Es la primera “cerradura decente” del [punto 1...](#) y el [punto 5](#) le pone la segunda vuelta.

Vocabulario rápido

Término	Idea general
Hash de contraseña	Huella irreversible que se guarda en lugar del texto plano
registrar()	Función que hashea y “almacena” la contraseña
login()	Función que hashea el intento y compara con el almacenado
hexdigest()	Representación hexadecimal del hash
Texto plano	La contraseña tal cual, lo que NUNCA se guarda

 [Volver al índice de la unidad](#) · Anterior: [03 · MD5, SHA-1 y SHA-256](#) · Siguiente: [05 · Hash con sal](#)

05 — Hash con sal

La sal que arruina las tablas rainbow 🧪

📖 Estás en: 🔒 U08 · Hash y Cifrado Clásico → 05 · Hash con sal

📌 La idea en una frase

Hashear una contraseña sin más tiene un agujero: si Ana y Bob usan la misma clave, sus hashes son **idénticos** y un atacante lo ve al instante. La solución es la **sal** (salt): un valor aleatorio distinto por usuario que se mezcla con la contraseña antes de hashear.

El [punto 4](#) guardaba el hash, pero había un problema silencioso. Vamos a arreglarlo.

🕒 El problema: hashes idénticos

```
import hashlib
print(hashlib.sha256(b"clave123").hexdigest())
print(hashlib.sha256(b"clave123").hexdigest())
```

Ambas líneas imprimen **exactamente el mismo hash**. Si Ana y Bob usan “clave123”, su base de datos tendrá dos filas con el mismo hash:

```
Ana | <mismo_hash_de_clave123>
Bob | <mismo_hash_de_clave123>
```

Un atacante que robe esa base lo ve al instante: “*estos dos comparten contraseña*”. Peor aún: un atacante con una **tabla rainbow** (hash de millones de contraseñas comunes precalculadas) buscaría el hash y encontraría “clave123” en segundos. Todo el sistema del [punto 4](#) cae.

🧪 La solución: salt

La **sal** es un valor aleatorio (16 bytes, generado con `os.urandom`) que se concatena a la contraseña **antes** de hashear. Como cada usuario tiene su propia sal, aunque dos compartan contraseña, sus hashes serán distintos.

```

import hashlib, os

def registrar_con_salt(usuario, contraseña):
    salt = os.urandom(16) # 🧂 Salt: 16 bytes NUEVOS cada vez
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    almacenado = salt.hex() + hash_contra # Guardamos: salt + hash juntos
    print(f" Usuario '{usuario}' registrado")
    print(f" Almacenado (salt+hash): {almacenado[:32]}...")
    return almacenado

def login_con_salt(usuario, contraseña, almacenado):
    # Del string guardado extraemos el salt (primera mitad) y el hash (resto)
    salt = bytes.fromhex(almacenado[:32]) # 🧂 RECUPERAMOS el mismo salt
    hash_original = almacenado[32:] # Hash que se guardó al registrar
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    if hash_intento == hash_original:
        print(f" ✅ {usuario}: login correcto")
        return True
    else:
        print(f" ❌ {usuario}: contraseña incorrecta")
        return False

# Simular – dos usuarios CON LA MISMA contraseña
hash_ana = registrar_con_salt("Ana", "clave123") # os.urandom genera salt_A
hash_bob = registrar_con_salt("Bob", "clave123") # os.urandom genera salt_B
(distinto)

print(f"\n¿Son iguales los hashes? {hash_ana == hash_bob}") # ¡NO!
# salt_A + "clave123" ≠ salt_B + "clave123" → hashes distintos

login_con_salt("Ana", "clave123", hash_ana) # ✅ extrae salt_A, recalcula, coincide
login_con_salt("Ana", "otra", hash_ana) # ❌ extrae salt_A, recalcula, NO coincide

```

Salida (los valores de salt cambian en cada ejecución, por eso son aleatorios):

```
Usuario 'Ana' registrado
Almacenado (salt+hash): a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6...
Usuario 'Bob' registrado
Almacenado (salt+hash): ffee00112233445566778899aabbccdde...
```

¿Son iguales los hashes? False

- ✅ Ana: login correcto
- ❌ Ana: contraseña incorrecta

La clave del proceso: la sal se genera **una vez**, al registrar, y se guarda junto al hash (como `salt.hex() + hash`). En el login **se recupera la misma sal** del string almacenado (`almacenado[:32]` son los 32 caracteres hex de la sal = 16 bytes) para poder recalculer el hash exacto. Sin ese paso, nunca coincidirían.

💡 El `[:32]` no es casualidad: `os.urandom(16)` genera 16 bytes, y `salt.hex()` los representa como **32 caracteres hexadecimales** (2 por byte). Por eso la sal ocupa siempre los 32 primeros caracteres del string guardado.

Por qué la sal destroza las tablas rainbow

Una **tabla rainbow** es un diccionario precomputado que mapea contraseñas comunes a sus hashes:

```
"123456" → 8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92
"password" → 5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
"clave123" → 3584f4ef4e3b2e5b5c6f2a...
```

Con esa tabla, buscar un hash de “clave123” es instantáneo. Pero si a cada usuario se le mezcló **su propia sal** con la contraseña, el hash guardado ya no es el de “clave123”: es el de “salt_A + clave123”, que es único e impredecible. La tabla rainbow deja de servir: para que funcionara, el atacante tendría que precomputar hashes para **todas las salas posibles**, lo cual es inviable.

Sin sal

Dos usuarios con la misma clave → mismo hash

Con sal

→ hashes distintos siempre

Sin sal

La tabla rainbow funciona al instante

Con sal

→ la tabla rainbow queda inútil

Un ataque revela contraseñas compartidas

→ nada se delata



Pool Puzzle — Verificación de contraseña con hash

¿Puedes ordenar estas líneas para crear un sistema de registro + login con sal?

```
a)    hash_login = hashlib.sha256((password + sal).encode()).hexdigest()
b)    sal = os.urandom(16).hex()
c)    return hash_guardado.split(":")[0] == hash_login
d)    import hashlib, os
e)    hash_guardado = hashlib.sha256((password + sal).encode()).hexdigest()
f)    return hash_guardado + ":" + sal
g)    def verificar(password, hash_guardado):
h)    def registrar(password):
```

>  Solución



Mini-chequeo

1. ¿Qué problema resuelve la sal y contra qué ataque concreto protege?
2. ¿Por qué los 32 primeros caracteres del string guardado son la sal?
3. En el login, ¿de dónde sale la sal que se usa para recalcular el hash?

>  Respuestas



Resumen en 3 frases

- Sin sal, dos usuarios con la misma contraseña comparten hash y las tablas rainbow triunfan.

- Con sal (`os.urandom(16)`), cada usuario mezcla su contraseña con un valor aleatorio único y sus hashes son irreconocibles entre sí.
- La sal se genera al registrar, se guarda junto al hash, y en el login se recupera para recalcular la huella exacta.



Vocabulario rápido

Término	Idea general
Sal (salt)	Valor aleatorio mezclado con la contraseña antes de hashear
Tabla rainbow	Diccionario precomputado de hashes de contraseñas comunes
<code>os.urandom(16)</code>	Genera 16 bytes aleatorios criptográficamente fuertes
<code>bytes.fromhex()</code>	Convierte texto hexadecimal de vuelta a bytes
Almacenado	String con formato salt + hash

 [Volver al índice de la unidad](#) · Anterior: [04 · Hash de contraseñas](#) · Siguiente: [06 · Cifrado César](#)

06 — Cifrado César

El abuelo de la criptografía, roto pero didáctico 🏰

📖 Estás en: 🔒 U08 · Hash y Cifrado Clásico → 06 · Cifrado César

📌 La idea en una frase

El **cifrado César** desplaza cada letra un número fijo de posiciones en el alfabeto: es el cifrado más antiguo que se conoce y, también, el más fácil de romper. Perfecto para aprender el concepto, inútil para proteger nada real.

Es el otro gran protagonista de esta unidad: mientras el hash (puntos anteriores) no se puede deshacer, el César **sí** se descifra con una clave: el desplazamiento.

🏰 La historia: Julio César y su desplazamiento de 3

Julio César, para que sus mensajes militares no fueran leídos si caían en manos enemigas, desplazaba **cada letra 3 posiciones** en el alfabeto. El receptor, que conocía el truco, desplazaba cada letra **3 hacia atrás**.

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
| | | |
▼ ▼ ▼ ▼
D E F G (cada letra se convierte en la que está 3 posiciones más adelante)
```

“Hola Mundo” con desplazamiento 3 → **“Krod Pxqgr”**. Sencillo de entender, y por eso es el cifrado ideal para empezar.

🐍 Cifrar y descifrar en Python

La función recorre el texto carácter a carácter. Si el carácter es una letra, calcula su nueva posición con la aritmética % 26 (para que al llegar a la ‘z’ vuelva a la ‘a’). Los espacios y signos se dejan igual.

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

def descifrar_cesar(texto, desplazamiento):
    return cifrar_cesar(texto, -desplazamiento)

original = "Hola Mundo"
cifrado = cifrar_cesar(original, 3)
descifrado = descifrar_cesar(cifrado, 3)

print(f"Original: {original}")      # Hola Mundo
print(f"Cifrado: {cifrado}")        # Krod Pxqgr
print(f"Descifrado:{descifrado}")  # Hola Mundo
```

Salida:

```
Original:  Hola Mundo
Cifrado:   Krod Pxqgr
Descifrado: Hola Mundo
```

Dos detalles del código:

- `caracter.isupper()` decide si la base es `ord('A')` (65) o `ord('a')` (97), para que las mayúsculas sigan siendo mayúsculas y las minúsculas, minúsculas.
- **Descifrar es cifrar con desplazamiento negativo:** `descifrar_cesar(texto, 3)` llama a `cifrar_cesar(texto, -3)`. No hace falta otra función.



Be the code: César paso a paso

“Sé el cifrado César caracter a caracter.”

```
cifrar_cesar("Hola Mundo", 3)
```

1. Carácter 'H' (mayúscula)

```
→ base = ord('A') = 65
→ (ord('H') - 65 + 3) % 26 + 65
→ (72 - 65 + 3) % 26 + 65
→ 10 % 26 + 65
→ 10 + 65 = 75
→ chr(75) = 'K'
```

2. Carácter 'o' (minúscula)

```
→ base = ord('a') = 97
→ (111 - 97 + 3) % 26 + 97
→ 17 % 26 + 97
→ 17 + 97 = 114
→ chr(114) = 'r'
```

3. 'l' → 'o'

4. 'a' → 'd'

5. ' ' → ' ' (no es letra, se queda igual)

6. 'M' → 'P'

7. 'u' → 'x'

8. 'n' → 'q'

9. 'd' → 'g'

10. 'o' → 'r'

Resultado: "Krod Pxqgr" 🎉

Repasa el carácter 'H': `ord('H')` es 72, le restamos la base 65, sumamos el desplazamiento 3, hacemos `% 26` para quedarnos dentro del alfabeto y volvemos a sumar la base. Da 75, que es `chr(75) = 'K'`.

💥 Romperlo por fuerza bruta

El desplazamiento solo puede ser un número del **1 al 25** (el 0 no cambia nada, y 26 vuelve a ser 0). Prueba los 25 y mira cuál tiene sentido:


```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

cifrado = "Krod Pxqgr"
for desplazamiento in range(1, 26):
    intento = cifrar_cesar(cifrado, -desplazamiento)
    print(f"Desplazamiento {desplazamiento}: {intento}")
```

Salida (extracto):

```
Desplazamiento 1: Jqnc Owpfq
Desplazamiento 2: Ipmb Nvoep
Desplazamiento 3: Hola Mundo   ← ¡aquí! Tiene sentido
Desplazamiento 4: Gnkz Ltmcn
...
Desplazamiento 25: Lspe Qyrhs
```

En 3 intentos lo encontramos. **Por eso el César no sirve para nada serio:** solo 25 claves posibles se prueban en segundos.

⚠️ **César NO es seguro.** Solo 25 desplazamientos posibles. Se rompe en segundos. Pero es perfecto para aprender el concepto. El cifrado serio que sí puedes usar en producción lo verás en la [U09 · Cifrado Moderno](#) con AES y RSA.

Mini-chequeo

1. ¿Cómo se descifra un mensaje César si conoces el desplazamiento?
2. ¿Por qué % 26 es imprescindible en la fórmula?
3. ¿Cuántos desplazamientos distintos hay que probar en fuerza bruta?

>  **Respuestas**



Resumen en 3 frases

- El César desplaza cada letra un número fijo de posiciones; descifrar es desplazar hacia atrás (desplazamiento negativo).
- Es un cifrado **reversible** con clave (el desplazamiento), al contrario que el hash: se puede recuperar el mensaje original.
- Con solo 25 desplazamientos posibles, se rompe por fuerza bruta al instante: inútil en producción, perfecto para aprender.



Vocabulario rápido

Término	Idea general
Cifrado César	Desplazar cada letra un número fijo de posiciones
Desplazamiento	La “clave”: cuántas posiciones se mueve cada letra
% 26	Módulo que mantiene las letras dentro del alfabeto
Fuerza bruta	Probar todas las claves posibles (aquí, 25)
isalpha()	Devuelve True si el carácter es una letra

 [Volver al índice de la unidad](#) · Anterior: [05 · Hash con sal](#) · Siguiente: [07 · Hash vs Cifrado](#)

07 — Hash vs Cifrado

El ring de los conceptos: integridad contra confidencialidad 

 Estás en:  U08 · Hash y Cifrado Clásico → 07 · Hash vs Cifrado

La idea en una frase

El **hash** y el **cifrado** no son rivales: son herramientas distintas para problemas distintos. El hash es **unidireccional** (integridad, contraseñas); el cifrado es **reversible** con clave (confidencialidad). Ambos son necesarios.

Ya has visto las dos piezas: el hash en los [puntos 2](#) a 5 y el cifrado (César) en el [punto 6](#). Ahora llega el momento de ponerlas frente a frente para que nunca las confundas. Y la mejor forma de hacerlo es con el clásico ring de la unidad.

El ring de los conceptos: Hash vs Cifrado

Dos mecanismos de seguridad se citan en el ring de los conceptos para resolver de una vez quién hace qué.

Hash: — Yo soy la huella digital. Transformo cualquier texto en una cadena fija. No se puede deshacer. Unidireccional. Para siempre.

Cifrado: — Vaya, qué drástico. Yo puedo cifrar y descifrar. Tengo clave. Si tú pierdes un hash, no hay vuelta atrás. Yo puedo recuperar el mensaje original.

Hash: — ¡Esa es precisamente mi gracia! Para contraseñas no quieres que se pueda deshacer. Si alguien roba la base de datos, que se pegue con los hashes.

Cifrado: — Pero para enviar un mensaje secreto, el hash no sirve. Necesitas que el destinatario pueda leerlo. Ahí entro yo.

Hash: — Y para verificar integridad, nadie me gana. Un archivo, un hash. Si cambia un bit, el hash cambia por completo.

Cifrado: — Al final, cada uno a lo suyo. Tú para integridad y contraseñas; yo para confidencialidad.



La moraleja: cada uno a lo suyo

	Hash	Cifrado
¿Qué hace?	Resume en una huella de longitud fija	Transforma con una clave
¿Reversible?	✗ No, unidireccional	✓ Sí, con la clave
Protege	Integridad (y contraseñas)	Confidencialidad
Ejemplo típico	Checksum de descarga, login	Enviar un mensaje secreto
Ejemplo en esta unidad	SHA-256 de “clave123”	Cifrado César “Hola” → “Krod”

Moraleja: El hash verifica integridad (no se puede deshacer). El cifrado protege confidencialidad (se puede deshacer con la clave). Ambos son necesarios.



La escena que lo aclara todo

Imagina que envías un mensaje secreto por una red hostil:

- **Solo cifrado:** el mensaje llega cifrado, pero ¿y si un atacante lo modifica por el camino? El destinatario descifra un mensaje manipulado y no lo sabe. La confidencialidad estaba protegida; la **integridad, no**.
- **Solo hash:** puedes comprobar que el mensaje no cambió, pero cualquier persona que lo intercepte **lo lee tal cual**. La integridad estaba protegida; la **confidencialidad, no**.
- **Cifrado + hash:** cifras el mensaje (nadie lo lee) y además calculas su hash (el destinatario comprueba que nadie lo tocó). Así lo hacen de verdad los sistemas serios.

El cifrado moderno (AES y RSA) y las **firmas digitales** que verás en la [U09 · Cifrado Moderno](#) combinan exactamente estas dos ideas.



Mini-chequeo

1. Si quieres que nadie pueda leer un mensaje, ¿qué necesitas: hash o cifrado?
2. Y si solo quieres comprobar que un archivo no fue manipulado, ¿qué usas?

3. ¿Por qué un hash sirve para guardar contraseñas y el cifrado no es la mejor opción ahí?

>  Respuestas

Resumen en 3 frases

- El hash es **unidireccional**: verifica integridad y guarda contraseñas, pero no se puede deshacer.
- El cifrado es **reversible con clave**: protege la confidencialidad y permite recuperar el mensaje original.
- No se excluyen: en los sistemas reales se combinan (cifrar para esconder, hashear para verificar).

Vocabulario rápido

Término	Idea general
Integridad	Que un dato no ha sido modificado (hash)
Confidencialidad	Que solo quien corresponde lo lea (cifrado)
Unidireccional	Imposible de revertir, propia del hash
Reversible con clave	Se deshace con la clave correcta, propia del cifrado
Firma digital	Combinación de hash + cifrado (lo verás en U09)

 [Volver al índice de la unidad](#) · Anterior: [06 · Cifrado César](#) · Siguiente: [08 · Buenas prácticas y verificación](#)

08 — Buenas prácticas y verificación

Checksums, cuándo hash y cuándo cifrar, y a practicar 

Estás en:  U08 · Hash y Cifrado Clásico → 08 · Buenas prácticas y verificación

La idea en una frase

El hash se usa para **verificar integridad**: descargas un archivo, calculas su checksum y lo comparas con el publicado por el autor. Si coincide, el archivo es exactamente el que publicaron; si no, no te fíes de él.

Este punto cierra la teoría con las reglas del oficio y un repaso práctico: cuándo toca hash, cuándo toca cifrar, y los ejercicios “Aprieta el lápiz” del tema para que lo interiorices.

Verificación de integridad: el checksum de las descargas

La aplicación más cotidiana del hash (ya la rozaste en el [punto 3](#)): las páginas de descarga de software publican el **SHA-256** de cada archivo. Tú descargas el archivo, calculas su hash y comparas:

Página	oficial:	SHA256	=
	a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b		
Tu	descarga:	SHA256	=
	a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b		
			 COINCIDEN

```
import hashlib

with open("archivo.pdf", "rb") as f:
    hash_calculado = hashlib.sha256(f.read()).hexdigest()

hash_oficial = "a1b2c3d4..." # el que publica la web

if hash_calculado == hash_oficial:
    print("✅ El archivo es íntegro: no ha sido manipulado.")
else:
    print("❌ ¡Alerta! El archivo NO coincide con el oficial.")
```

Por qué funciona: el efecto avalancha ([punto 2](#)) hace que un solo bit distinto produzca un hash completamente diferente. Si el hash coincide, la probabilidad de que el archivo haya sido tocado es prácticamente nula (con SHA-256).

🔒 Descarga Ubuntu, un instalador o un ISO desde un espejo no oficial y verifícalo: es la rutina de seguridad que ya practicas sin darte cuenta cada vez que instalas software.

🤔 ¿Cuándo hash y cuándo cifrado? La regla de oro

Recuerda el [punto 7](#): no son rivales, son para cosas distintas.

Necesitas...	¿Hash o cifrado?	¿Por qué?
Verificar que un archivo no cambió	🗂 Hash	Unidireccional, un bit cambia todo el hash
Guardar contraseñas	🗂 Hash (con sal)	No quieres recuperarlas nunca, solo comparar
Enviar un mensaje que solo el destinatario lea	🔑 Cifrado	El destinatario debe poder descifrarlo
Confirmar la identidad del autor de un mensaje	🗂 Hash + 🔑 firma digital (U09)	Combina ambos: hash del mensaje cifrado con clave privada

⚠ **Error típico de novato:** cifrar la contraseña en vez de hashearla. Si roban la base de datos y la clave de cifrado, recuperan todo. El hash, al ser unidireccional, ni con la clave se deshace. Solo el hash (con sal) es correcto para contraseñas.

Guía rápida de buenas prácticas

- Usa siempre SHA-256 o SHA-512 para seguridad; MD5 y SHA-1 solo como checksum no crítico (lo viste en el [punto 3](#)).
- Hashea con sal toda contraseña antes de guardarla ([punto 5](#)).
- Nunca compares ni imprimas contraseñas en claro ([punto 4](#)).
- Verifica la integridad de archivos descargados contra su checksum oficial.
- No inventes tu cripto: usa hashlib y las bibliotecas auditadas ([punto 1](#)).

Aprieta el lápiz

1. **Compara hashes:** Calcula MD5, SHA1 y SHA256 de la palabra “python”. ¿Cuánto mide cada hash?
2. **Efecto avalancha:** Calcula SHA256 de “Hola mundo” y “Hola mund0” (cambia una ‘o’ por ‘0’). Compara.
3. **César personalizado:** Cifra “Hola mundo” con desplazamiento 5 y descifra el resultado.
4. **Fuerza bruta César:** Dado un texto cifrado con César (sin saber el desplazamiento), prueba los 25 desplazamientos hasta encontrar el que tenga sentido.
5. **Hash de archivo:** Calcula el SHA256 de un archivo .py de tu proyecto.

>  Soluciones

Mini-chequeo

1. ¿Cómo compruebas que una descarga no fue manipulada?
2. ¿Por qué cifrar la contraseña no sirve para guardarla, y el hash sí?
3. ¿Cuál es el primer paso que probarías en fuerza bruta con un texto cifrado con César?

>  Respuestas



Resumen en 3 frases

- El checksum (hash del archivo) es la forma estándar de **verificar integridad** en descargas.
- Regla de oro: **hash** para integridad y contraseñas, **cifrado** para confidencialidad.
- Con los 5 ejercicios de “Aprieta el lápiz” tienes el material mínimo para dominar la unidad.



Vocabulario rápido

Término	Idea general
Checksum	Hash de un archivo para comprobar su integridad
Verificar integridad	Confirmar que un dato no fue modificado
Regla de oro	Hash → integridad; cifrado → confidencialidad
Aprieta el lápiz	Los ejercicios prácticos de la unidad
Firma digital	Hash + cifrado con clave privada (U09)



[Volver al índice de la unidad](#) · Anterior: [07 · Hash vs Cifrado](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé el hash, laboratorio con sal y el ring final de la unidad 

 Estás en:  U08 · Hash y Cifrado Clásico → 09 · Cierre

Has terminado la teoría: huella digital, propiedades del hash, MD5/SHA-1/SHA-256, registro y login con hash, la sal contra las tablas rainbow y el cifrado César. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

Sé el hash

Eres el sistema de registro y login de una aplicación con sal. Acaba de llegar una petición: registrar a un nuevo usuario llamado “Ana” con contraseña “clave123”.

¿Qué pasa?

1. Generas una **sal** aleatoria de 16 bytes: `os.urandom(16)`. Cada usuario tiene la suya, irrepetible.
2. Concatenas: `sal + contraseña.encode()` → mezclas la sal con la clave antes de hashear.
3. Calculas `hashlib.sha256(sal + contraseña.encode()).hexdigest()`: la huella de la mezcla.
4. Guardas **sal + hash juntos** (como `salt.hex() + hash`): así el login podrá recuperar la sal.
5. Cuando Ana intenta entrar, extraes su sal (`bytes.fromhex(almacenado[:32])`), recalculas el hash del intento y lo comparas con el guardado.
6. Si coinciden →  login correcto. Si no →  contraseña incorrecta.

En ningún momento existe la contraseña de Ana fuera de su teclado. Ni tú (la base de datos) la guardas ni la ves.

 **Ahora tú:** ¿y si dos usuarios usan “clave123”? Como cada uno tiene su sal distinta, sus hashes son diferentes: nadie puede adivinar que comparten contraseña. Eso es lo que aprendiste en el [punto 5](#).

Fireside Chat: Hash vs Cifrado

Dos mecanismos de seguridad se sientan junto a la chimenea a resolver, de una vez, quién hace qué.

Hash: — Yo soy la huella digital. Transformo cualquier texto en una cadena fija. No se puede deshacer. Unidireccional. Para siempre.

Cifrado: — Vaya, qué drástico. Yo puedo cifrar y descifrar. Tengo clave. Si tú pierdes un hash, no hay vuelta atrás. Yo puedo recuperar el mensaje original.

Hash: — ¡Esa es precisamente mi gracia! Para contraseñas no quieres que se pueda deshacer. Si alguien roba la base de datos, que se pegue con los hashes.

Cifrado: — Pero para enviar un mensaje secreto, el hash no sirve. Necesitas que el destinatario pueda leerlo. Ahí entro yo.

Hash: — Y para verificar integridad, nadie me gana. Un archivo, un hash. Si cambia un bit, el hash cambia por completo.

Cifrado: — Al final, cada uno a lo suyo. Tú para integridad y contraseñas; yo para confidencialidad.

Moraleja: el hash verifica integridad (no se puede deshacer); el cifrado protege confidencialidad (se deshace con la clave). Ambos son necesarios.



¿Quién soy?

1. Me genero con `os.urandom(16)` y soy distinto para cada usuario.
2. Soy un diccionario precomputado de hashes de contraseñas comunes.
3. Tuve colisiones demostradas en 2004: ya nadie me usa para seguridad.
4. Un solo bit distinto en la entrada y me convierto en un hash completamente diferente.
5. Soy el estándar de 256 bits que nunca debes romper con MD5 o SHA-1.
6. Soy el cifrado que desplaza cada letra y que se rompe en 25 intentos.

>  Respuestas



CONRAD VS EL MUNDO: “guardé las contraseñas con MD5 y sin sal”

CONRAD: — “Clásico: la base de datos cae en manos de un atacante y, en cuestión de horas, ha sacado la mitad de las contraseñas. Razones: 1) Usé **MD5**, que tiene colisiones demostradas desde 2004. 2) **Sin sal**: dos usuarios con la misma contraseña comparten hash y las tablas rainbow funcionan al instante. 3) **Guardé la contraseña en claro** en el log ‘por si acaso’. 4) Comparé contraseñas en texto plano en el código, como si la base fuera un cuaderno. 5) **Cifré** la contraseña en vez de hashearla... y el atacante robó también la clave de cifrado.”

CONRAD: — “Y lo mejor: *‘pero SHA-1 sigue siendo rápido’*. ¡Pues claro! Rápido para ti y para el atacante. Seguridad no es velocidad: es resistencia. **SHA-256 con sal**, siempre.”

CONRAD: — “Y no me vengas con *‘¿será que los hashes se ven parecidos?’*. Los hashes no se parecen jamás: el efecto avalancha del [punto 2](#) te garantiza que un bit distinto cambia todo. Si ves dos hashes parecidos, tienes un bug. A diagnosticar.”

Laboratorio de tortura: registro y login con sal

Duración: 45 minutos **Herramienta:** Python 3 (hashlib y os, sin instalar nada)

Escenario: construye un mini sistema de registro y login en Python que guarde los usuarios en un diccionario en memoria (nada de base de datos: basta un `usuarios = {}`). Cada usuario debe almacenar `sal + hash` juntos, exactamente como en el [punto 5](#).

Tareas paso a paso:

1. Define `registrar(usuario, contraseña)`: genera la sal con `os.urandom(16)`, calcula `hashlib.sha256(salt + contraseña.encode()).hexdigest()`, guarda en el diccionario `usuarios[usuario] = salt.hex() + hash`, y devuelve el string almacenado.
2. Define `login(usuario, contraseña)`: recupera el string del diccionario, extrae la sal con `bytes.fromhex(almacenado[:32])`, recalcula el hash del intento y compara con la parte de hash del string guardado.
3. Registra a dos usuarios con la **misma** contraseña y comprueba que sus hashes son distintos.
4. Haz login correcto y otro con contraseña equivocada: deben dar  y .
5. Añade un `verificar_integridad(archivo)` que calcule el SHA-256 de un archivo y lo compare con un hash esperado (el [punto 8](#)).

Fallo intencionado: en el `login`, en lugar de recuperar la sal del string almacenado, **genera una sal nueva** con `os.urandom(16)`. ¿Qué pasa? El hash del intento nunca coincide: cada login

genera una sal distinta, así que recalculas con una sal que no es la del registro. Todos los logins fallan, incluso con la contraseña correcta.

Pista 1: la sal debe ser **la misma** en el registro y en el login. Por eso se guarda junto al hash en el [punto 5](#): `bytes.fromhex(almacenado[:32])` la recupera tal cual.

Pista 2: si el fallo no se ve a simple vista, añade un `print(f"Sal usada en login: {salt.hex()[:8]}...")` y compáralo con la sal guardada en el registro. Si son distintas en cada intento, ese es el bug.



Logros de esta unidad

Logro	Cómo conseguirlo
 Huella Digital	Calcular MD5, SHA-1, SHA-256 y SHA-512 de cualquier texto con <code>hashlib</code>
 Vault Guardian	Implementar registro y login con hash + sal en Python
 Rainbow	Explicar cómo la sal neutraliza las tablas rainbow
Breaker	
 César Cracker	Cifrar, descifrar y romper por fuerza bruta el cifrado César



Atrévete a pensar

1. ¿Por qué no se puede “descifrar” un hash, y qué alternativas reales le quedan al atacante?
2. ¿Qué pasaría si dos usuarios con la misma contraseña tuvieran el mismo hash en tu base de datos?
3. ¿Por qué descifrar un César con desplazamiento desconocido es trivial y no pasa lo mismo con SHA-256?
4. ¿Cuándo usarías cifrado en lugar de hash? Pon un ejemplo concreto.
5. ¿Qué diferencia hay entre verificar la integridad de un archivo y proteger su confidencialidad?

>  **Soluciones**

Crucigrama de bits

Horizontal:

1. Función que convierte cualquier dato en una huella de longitud fija (4 letras)
4. Valor aleatorio que se mezcla con la contraseña antes de hashear (3 letras)
6. Algoritmo de 128 bits roto desde 2004 (3 letras)
8. Cifrado por desplazamiento de letras, el abuelo de la criptografía (5 letras)

Vertical:

2. Algoritmo seguro de 256 bits de la familia SHA-2 (5 letras)
3. Diccionario precomputado de hashes de contraseñas comunes (6 letras)
5. Propiedad del hash: un bit de cambio altera todo (9 letras)
7. Lo opuesto de cifrado: operación para recuperar el mensaje (8 letras)

>  Soluciones

Entrevista de trabajo

1. “¿Cómo guardarías las contraseñas de tus usuarios en una base de datos?”
2. “¿Qué diferencia hay entre hash y cifrado? ¿Cuándo usarías cada uno?”
3. “¿Qué es el efecto avalancha y por qué importa en los hashes?”
4. “¿Qué es una tabla rainbow y cómo la neutraliza la sal?”
5. “¿Cómo verificas que un archivo descargado no fue manipulado?”

 **Cómo encararlas:** la 1 y la 2 son las “preguntas reina”. Para la 1, recorre la cadena del [punto 4](#) y el [punto 5](#): hash con sal (SHA-256), nunca la original, comparar hashes en el login. Para la 2, repite la moraleja del [punto 7](#): hash → integridad, cifrado → confidencialidad. Si sabes contarlo fluido, ya eres medio desarrollador seguro.

No hay preguntas tontas

 ¿Se puede descifrar un hash?

No, el hash no se descifra (es unidireccional, como viste en el [punto 2](#)). Pero se puede **adivinar** usando tablas rainbow o fuerza bruta. Por eso se usa **sal**: añadir un valor aleatorio antes de hashear vuelve inútiles ambos ataques. Un hash no se revierte: se intenta acertar la entrada.

? ¿Qué es una tabla rainbow?

Una tabla precomputada de hashes para contraseñas comunes (123456, password, etc). Si tu hash está en la tabla, tu contraseña está comprometida. El **sal** lo evita porque añade aleatoriedad: el hash guardado ya no es el de la contraseña sola, sino el de `sal + contraseña`, que es único por usuario.

? ¿Qué pasa si dos usuarios tienen la misma contraseña?

Sin **sal**, tendrían el mismo hash y un atacante lo sabría al instante. Con **sal**, aunque la contraseña sea la misma, los hashes son **diferentes** (mira el ejemplo de hash con **sal** del [punto 5](#)). Nadie puede deducir que comparten clave.

? ¿Para qué sirve el hash de un archivo?

Para verificar **integridad**. Descargas Ubuntu, verificas su SHA256, y si coincide con el de la web oficial, sabes que no lo han manipulado. Un solo bit distinto en el archivo produce un hash completamente diferente: esa es la señal de alarma.

Poscréditos

La base de datos guarda solo hashes con sal. Nadie conoce las contraseñas, ni siquiera el servidor.

Un atacante roba la base de datos. Encuentra miles de huellas irreversibles e inútiles.

En su otra pantalla, un mensaje cifrado viaja seguro. El hash verificó que nadie lo tocó.

Las cerraduras se apilan. La casa sigue en pie.

PRÓXIMAMENTE EN U09: Cifrado moderno. El hash verifica, pero ¿cómo se envía un mensaje secreto que solo el destinatario pueda leer? AES y RSA.

Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen la integridad y la confidencialidad de los datos.

CE	Criterio	Cubierto
a)	Principios básicos de seguridad	✅ Principios (punto 1) + hash vs cifrado (punto 7)
c)	Implementa funciones hash (MD5, SHA)	✅ Puntos 2-5 + ⚡ Laboratorio de tortura
h)	Conoce sistemas de roles y RBAC	✅ Mínimo privilegio (punto 1)

RA5b (tipos de cifrado), RA5d (AES), RA5e (RSA), RA5f (firmas digitales) y RA5g (cifrado híbrido) se cubren en la **U09 · Cifrado Moderno**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Buenas prácticas y verificación](#) · Siguiendo: [U09 · Cifrado Moderno](#)

U09 — Cifrado Moderno

AES, RSA y firmas digitales para proteger el mensaje 🧬

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 🧪 APIs comerciales → 🔑 Hash → 🧬 **Cifrado Moderno** → 🏠 Servidores → ⌚ asyncio

En la U08 aprendiste a hashear: a convertir una contraseña en una huella digital irrepetible. Pero el hash no se puede deshacer: es unidireccional. ¿Y si quieres enviar un mensaje secreto que el destinatario sí pueda leer? Ahí entra el **cifrado moderno**: con **AES** (rápido y simétrico) y **RSA** (lento y elegante) podrás cifrar, descifrar, firmar y verificar mensajes, tal y como lo hacen HTTPS, WhatsApp o Signal cada segundo.

Aprenderás también el **cifrado híbrido** (lo mejor de ambos mundos: RSA reparte la clave, AES cifra el tráfico), las **firmas digitales** para demostrar quién creó un mensaje y el **control de acceso RBAC** para decidir quién puede hacer qué. Es la otra mitad de la seguridad que empezaste en la [U08 · Hash y Cifrado Clásico](#).

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Distinguir el cifrado **simétrico** (una clave) del **asimétrico** (par público/privado) y saber cuándo usar cada uno.
- Cifrar y descifrar con **AES** (modos EAX, ECB, CBC, GCM) usando pycryptodome.
- Explicar qué son el **nonce**, el **tag** y el **padding** y por qué hay que enviarlos junto al cifrado.
- Generar pares de claves **RSA** de 2048 bits y cifrar/descifrar con clave pública y privada.
- **Firmar** digitalmente un mensaje con tu clave privada y **verificar** la firma con la clave pública del firmante.
- Diseñar un **cifrado híbrido** AES + RSA, el método que usa HTTPS.
- Implementar un sistema **RBAC** de roles y permisos.
- Construir un mini sistema seguro completo que cifre, descifre, firme y verifique.



Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Cifrado simétrico vs asimétrico	Una clave o un par: cuándo usar cada uno	Todos
02 · AES	El cifrado simétrico moderno con pycryptodome, nonce y tag	Todos
03 · Modos de AES	ECB, CBC y GCM: cómo se aplica el bloque, IV y padding	Todos
04 · RSA	Generar el par de claves y cifrar con la pública	Todos
05 · Firmas digitales	Firmar y verificar: integridad + autenticidad	Todos
06 · Cifrado híbrido	AES + RSA: el método que usa HTTPS	Todos
07 · RBAC y roles	Control de acceso basado en roles	Todos
08 · Práctica: sistema seguro	Cifrar y firmar un sistema completo	Todos
09 · Cierre	Sé la clave, Fireside, Laboratorio, Crucigrama...	Todos



Flujo de lectura: los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.



Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✅ Inicial resuelto

● Inicial por resolver

👉 Avanzado resuelto

★ Avanzado por resolver



Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen la integridad y la confidencialidad de los datos.

CE	Criterio	Dónde se cubre
a)	Principios básicos de seguridad	✓ Puntos 1 y 7 (mínimo privilegio)
b)	Tipos de cifrado (simétrico, asimétrico)	✓ Puntos 1, 2 y 4
c)	Implementa funciones hash (MD5, SHA)	✓ Cubierto en la U08
d)	AES	✓ Puntos 2-3 + ⚡ Laboratorio (punto 9)
e)	RSA	✓ Puntos 4-5 + ⚡ Laboratorio (punto 9)
f)	Firmas digitales	✓ Punto 5 + ⚡ Laboratorio (punto 9)
g)	Cifrado híbrido	✓ Punto 6 + ⚡ Laboratorio (punto 9)
h)	Conoce sistemas de roles y RBAC	✓ Punto 7

¿Por dónde empiezo?

¿Vienes de la U08 y dominas el hash y el cifrado César? Perfecto, este tema es la evolución natural: ya sabes verificar integridad con SHA-256 y distinguir cuándo toca hash y cuándo cifrar. Repasa la [U08 · Hash y Cifrado Clásico](#) para tener fresco el principio de *mínimo privilegio* y el de *no inventes tu cripto*, y arranca en el [punto 1](#), que parte justo de la pregunta: ¿una clave o dos?

¿Ya sabes qué es AES y RSA y solo necesitas las firmas o el híbrido? Puedes saltar a los [puntos 5](#) y [6](#). Pero si vienes de cero en criptografía moderna, no te saltes los puntos 1 a 4: entender la diferencia simétrico/asimétrico y cómo se cifra con cada uno es la base de todo lo demás.

📌 **Primer punto:** [01 · Cifrado simétrico vs asimétrico](#)

▶ Al acabar la unidad, continúa en [U10 · Servidores Concurrentes](#).

01 — Cifrado simétrico vs asimétrico

Una clave o un par: cuándo usar cada uno 🔑

📖 Estás en: 🧬 U09 · Cifrado Moderno → 01 · Cifrado simétrico vs asimétrico

📌 La idea en una frase

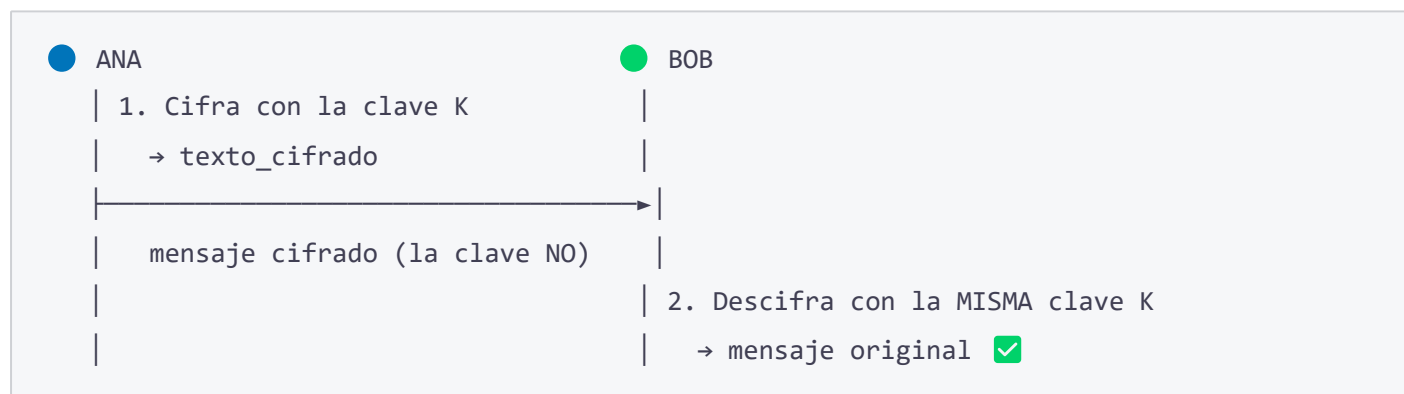
El cifrado **simétrico** usa **una sola clave** para cifrar y descifrar; el **asimétrico** usa un **par de claves** (pública + privada). La primera es rapidísima pero hay que compartir la clave; la segunda resuelve el reparto a costa de velocidad.

Recuerda el principio de la U08: *no inventes tu cripto*. Aquí no inventamos nada: usamos los dos grandes bloques del cifrado moderno y aprendemos cuándo toca cada uno.

🔑 Cifrado simétrico: una sola clave

“Un solo cifrado con una sola clave. El que tiene la clave, puede descifrar.”

Todo el mundo con una llave abre la misma cerradura. El **mismo secreto** sirve para cifrar y para descifrar, por eso se llama *simétrico*: los dos lados usan la misma llave.



Las dos caras de la moneda:

- ✓ **Rapidez:** cifra cantidades enormes en milisegundos. Es el cifrado de los datos en volumen.
- ✗ **El problema:** ¿cómo le haces llegar la clave K a Bob sin que nadie la intercepte? Ese es el *problema de la distribución de claves*. En la vida real, enviar la clave por el mismo canal que el mensaje es como dejar la llave bajo el felpudo.



Cifrado asimétrico: el par público/privado

Dos claves: una **pública** (todos pueden verla) y una **privada** (solo tú).



La regla de oro de las claves:

Operación	¿Qué clave usas?
Cifrar para alguien	La clave pública de esa persona
Descifrar	Tu clave privada
Firmar	Tu clave privada
Verificar una firma	La clave pública del firmante



Nadie puede descifrar lo que cifraste con la pública de Bob, salvo Bob con su privada. La pública se puede repartir por todos los sitios: es pública.



La comparativa: AES vs RSA

Característica	AES (simétrico)	RSA (asimétrico)
Claves	Una	Dos (pública + privada)
Velocidad	Rápido (~1GB/s)	Lento (~1MB/s)
Tamaño máximo	Ilimitado	~190 bytes (con 2048 bits)
Distribución de clave	Problema	Fácil

⚠️ RSA no sirve para cifrar mensajes grandes. Para eso necesitas **cifrado híbrido** (lo verás en el [punto 6](#)).

🧠 ¿Cuándo uso cada uno?

Situación	Eliges
Cifrar un archivo, una base de datos, un disco	Simétrico (AES) — rápido y no hay problema de volumen
Enviar un secreto a alguien por un canal inseguro	Asimétrico (RSA) — no hay que compartir la clave
Cifrar un mensaje largo en HTTPS	Híbrido — RSA reparte la clave AES, AES cifra el tráfico
Demostrar quién firmó un documento	Asimétrico — la firma digital (punto 5)

El asimétrico nunca sustituye al simétrico: lo **complementa**. La clave pública/privada resuelve el reparto de secretos; la simétrica se encarga del volumen. La unión de ambos es el cifrado híbrido que mueve Internet.

🧠 Mini-chequeo

1. ¿Cuál es el problema principal del cifrado simétrico?
2. ¿Con qué clave cifra un mensaje para que solo Bob lo lea?
3. ¿Por qué RSA no sirve para cifrar un archivo de 1 GB?

> 📖 Respuestas

✅ Resumen en 3 frases

- El **simétrico** usa una clave, es rapidísimo y sirve para el volumen; su talón de Aquiles es compartir la clave.
- El **asimétrico** usa un par público/privado: cifrar con la pública de otro, descifrar solo con tu privada.

- No compiten: se **complementan** en el cifrado híbrido, que es lo que usa HTTPS.



Vocabulario rápido

Término		Idea general
Cifrado simétrico		Una sola clave para cifrar y descifrar (AES)
Cifrado asimétrico		Par de claves: pública para cifrar, privada para descifrar (RSA)
Clave pública		Se puede repartir libremente; cifra para su dueño
Clave privada		Secreto absoluto; descifra y firma
Distribución de claves	de	El problema de hacer llegar el secreto compartido sin que lo intercepten



[Volver al índice de la unidad](#) · **Siguiente:** [02 · AES](#)

02 — AES

El cifrado simétrico moderno con pycryptodome 

 Estás en:  U09 · Cifrado Moderno → 02 · AES

La idea en una frase

AES es el cifrado simétrico moderno por excelencia: con una sola clave (de 16, 24 o 32 bytes) cifra y descifra a ~1 GB/s. Con pycryptodome, cifrar un mensaje es cifrar con la clave y enviar, junto al texto cifrado, el **nonce** y el **tag**.

Es el *estándar de oro* del cifrado simétrico: lo usa tu disco cifrado, tu conexión HTTPS, tu tarjeta bancaria y los chats cifrados. Rápido, auditado y con modos para cada necesidad (los verás en el [punto 3](#)).

Componentes de AES

Cuando cifras con AES en modo EAX, lo que viaja por la red es **más que el texto cifrado**:

Componente	Descripción	¿Se envía?
Clave	16, 24 o 32 bytes (AES-128/192/256)	 Secreto
Nonce	Número aleatorio único	 Se envía con el cifrado
Tag	Código de autenticación (integridad)	 Se envía
Texto cifrado	El mensaje cifrado	 Se envía

Paquete enviado: [nonce 16B | tag 16B | cifrado ...]

- El **nonce** (*number used once*) hace que dos cifrados del mismo mensaje con la misma clave den resultados distintos. El receptor lo necesita para descifrar.
- El **tag** es el código de autenticación: si alguien manipula el cifrado, el tag no coincide y el descifrado falla. Es la integridad (U08) dentro del cifrado.

🔑 La **clave** no viaja jamás en ese paquete: el receptor ya la tiene (o la recibe por otro canal, como verás en el [cifrado híbrido](#)).

AES con pycryptodome

Instala la biblioteca (la más usada en Python para criptografía):

```
pip install pycryptodome
```

Cifrar y descifrar un mensaje:

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

# Generar clave (32 bytes = 256 bits)
clave = get_random_bytes(32)

# Cifrar
cifrador = AES.new(clave, AES.MODE_EAX)
mensaje = b"Mensaje secreto"
texto_cifrado, tag = cifrador.encrypt_and_digest(mensaje)

print(f"Original: {mensaje}")
print(f"Cifrado (hex): {texto_cifrado.hex()}")
print(f"Nonce: {cifrador.nonce.hex()}")

# Descifrar
cifrador2 = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
texto_descifrado = cifrador2.decrypt(texto_cifrado)
print(f"Descifrado: {texto_descifrado}")
```

```
Original: b'Mensaje secreto'
Cifrado (hex): 7f2e9b1a4c6d8e0f...
Nonce: 4f3a2b1c9d8e7f6a...
Descifrado: b'Mensaje secreto'
```

AES es **simétrico**: misma clave para cifrar y descifrar. El problema es compartir esa clave de forma segura (el [punto 4](#) y el [punto 6](#) lo resuelven).

Para verificar la integridad del descifrado usa `decrypt_and_verify` en lugar de `decrypt`: si el tag no coincide, lanza una excepción (alguien manipuló el mensaje).



Be the code, my friend — AES paso a paso

“Ana quiere enviar un mensaje a Bob. Traza cada byte.”

● ANA (emisora)

1. Tiene el mensaje: "Nos vemos a las 8"
2. Genera clave AES: `get_random_bytes(32)` → `b'\xa1\xb2\xc3...' (32 bytes)`
3. Crea cifrador AES modo EAX
 - `AES.new(clave, AES.MODE_EAX)`
 - Genera nonce automáticamente: `b'\x4f\x3a...' (16 bytes)`
4. Cifra:
 - `encrypt_and_digest(b"Nos vemos a las 8")`
 - Devuelve (texto_cifrado, tag)
 - `texto_cifrado = b'\x7f\xe2...' (ilegible)`
 - `tag = b'\x1a\xb2...' (16 bytes)`
5. Envía a Bob: nonce + tag + texto_cifrado

nonce (16B)	tag (16B)	cifrado
\x4f\x3a...	\x1a\xb2...	\x7f\xe2...

● BOB (receptor)

6. Recibe: nonce = primeros 16 bytes
tag = siguientes 16 bytes
cifrado = resto
7. `AES.new(clave, AES.MODE_EAX, nonce=recibido)`
8. `decrypt_and_verify(cifrado)`
 - Si el tag coincide → datos íntegros ✓
 - Si no → lanza excepción (alguien manipuló el mensaje)
9. Bob lee: "Nos vemos a las 8" ☒



Mini-chequeo

1. ¿Cuántos bytes de clave usa AES-256? ¿Y AES-128?
2. ¿Qué pasa si el receptor descifra sin el nonce correcto?
3. ¿Para qué sirve el tag? ¿Qué pasa si no se verifica?

>  Respuestas



Resumen en 3 frases

- AES es el **cifrado simétrico moderno**: una clave de 16/24/32 bytes, rápido y seguro.
- En el paquete viajan **nonce + tag + cifrado**; la clave nunca viaja.
- Con `pycryptodome` cifras con `encrypt_and_digest` y descifras con el mismo nonce, verificando el tag.



Vocabulario rápido

Término	Idea general
AES	Advanced Encryption Standard: cifrado simétrico estándar
Clave AES	16 (AES-128), 24 (AES-192) o 32 (AES-256) bytes
Nonce	Número aleatorio único que viaja con el cifrado
Tag	Código de autenticación que detecta manipulaciones
<code>encrypt_and_digest</code>	Cifra y genera el tag de integridad a la vez
<code>pycryptodome</code>	La biblioteca de criptografía de Python que usarás en todo el tema

 [Volver al índice de la unidad](#) · Anterior: [01 · Cifrado simétrico vs asimétrico](#) · Siguiente: [03 · Modos de AES](#)

03 — Modos de AES

ECB, CBC y GCM: cómo se aplica el bloque, IV y padding 🧱

📖 Estás en: 🧬 U09 · Cifrado Moderno → 03 · Modos de AES

📌 La idea en una frase

AES cifra de 16 en 16 bytes. El **modo** decide cómo se encadenan esos bloques: **ECB** los cifra iguales (y deja ver patrones), **CBC** mezcla cada bloque con el anterior usando un **IV**, y **GCM/EAX** añaden autenticación con un **tag**. Si el mensaje no llena un bloque, entra el **padding**.

El modo importa tanto como el algoritmo: usar AES en modo ECB es como poner la misma cerradura a todas las puertas de un pasillo... con la misma llave.

🧱 ¿Por qué hace falta un modo?

AES trabaja sobre **bloques de 16 bytes**. Un mensaje de 3 bytes o de 50 bytes no es múltiplo de 16, y además si dos bloques son idénticos, con el mismo proceso producirían el mismo cifrado. El modo de operación resuelve los dos problemas: cómo tratar bloques repetidos y qué hacer con el trozo que sobra.

Mensaje de 45 bytes:

bloque 1	bloque 2	bloque 3	sobra 13
----------	----------	----------	----------

← hace falta padding

⚖️ La tabla de los modos

Modo	¿Autentica? (tag)	¿Necesita IV/nonce?	¿Padding?	Veredicto
ECB	❌ No	❌ No	Sí	⚠️ Evitar: muestra patrones

Modo	¿Autentica? (tag)	¿Necesita IV/nonce?	¿Padding?	Veredicto
CBC	✗ No	✓ IV	Sí	✓ Clásico y seguro
GCM	✓ Sí (tag)	✓ Nonce	No	✓ Moderno, cifra y autentica
EAX	✓ Sí (tag)	✓ Nonce	No	✓ El que usas en esta unidad

⚠ **ECB es la trampa clásica.** Cifra cada bloque de forma independiente: mismo bloque de entrada → mismo bloque de salida. En imágenes o documentos con zonas repetidas, el patrón se ve a simple vista. Por eso *nunca* cifres datos largos o estructurados en modo ECB.

🔗 CBC: cada bloque depende del anterior

En **CBC** (*Cipher Block Chaining*), cada bloque se mezcla (XOR) con el cifrado del bloque anterior antes de cifrarse. El primer bloque se mezcla con el **IV** (*vector de inicialización*), un valor aleatorio de 16 bytes que viaja junto al cifrado.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

clave = get_random_bytes(32)
iv = get_random_bytes(16) # vector de inicialización
mensaje = b"Mensaje que no mide 16 bytes exactos"

cifrador = AES.new(clave, AES.MODE_CBC, iv=iv)
texto_cifrado = cifrador.encrypt(pad(mensaje, 16)) # padding hasta múltiplo de 16

descifrador = AES.new(clave, AES.MODE_CBC, iv=iv)
original = unpad(descifrador.decrypt(texto_cifrado), 16)
print(f"Original: {original}")
```

- El **IV** es como el nonce: aleatorio, único por cifrado, y **se envía** con el texto cifrado.
- El **padding** rellena el último bloque hasta 16 bytes. `pad/unpad` de `Crypto.Util.Padding` lo hacen por ti; **el receptor tiene que aplicar unpad** para recuperar el mensaje exacto.



GCM y EAX: cifrar y autenticar a la vez

GCM (*Galois/Counter Mode*) y **EAX** son modos *autenticados*: además del texto cifrado producen un **tag** que detecta cualquier manipulación. Es el “tag” que ya usaste en el [punto 2](#) con `encrypt_and_digest`.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_GCM) # genera nonce automáticamente
texto_cifrado, tag = cifrador.encrypt_and_digest(b"Autenticado y cifrado")

descifrador = AES.new(clave, AES.MODE_GCM, nonce=cifrador.nonce)
try:
    original = descifrador.decrypt_and_verify(texto_cifrado, tag)
    print(f"✅ Descifrado y verificado: {original}")
except ValueError:
    print("❌ El tag no coincide: el mensaje fue manipulado.")
```



Regla práctica: si solo necesitas confidencialidad, CBC; si además quieres integridad, GCM o EAX. En esta unidad usamos **EAX** por su sencillez: `encrypt_and_digest` cifra y autentica en una llamada, sin padding manual.



Mini-chequeo

1. ¿Por qué no debes usar ECB con datos largos o repetitivos?
2. ¿Qué es el IV en modo CBC y por qué hay que enviarlo?
3. ¿Qué hace `decrypt_and_verify` que no haga `decrypt`?

>  Respuestas



Resumen en 3 frases

- AES cifra en bloques de 16 bytes y el **modo** decide cómo encadenarlos: ECB (inseguro), CBC (clásico, con IV y padding) o GCM/EAX (autenticados, con tag).

- El **IV**/nonce viaja siempre con el cifrado; el **padding** rellena el último bloque hasta 16 bytes y se retira con unpad.
- Para esta unidad, **EAX** te da cifrado + integridad en una sola llamada: `encrypt_and_digest` y `decrypt_and_verify`.



Vocabulario rápido

Término	Idea general
Modo de operación	Cómo se encadenan los bloques de 16 bytes de AES
ECB	Modo inseguro: cada bloque se cifra solo (patrones visibles)
CBC	Modo clásico: cada bloque se mezcla con el anterior (IV + padding)
GCM / EAX	Modos autenticados: cifran y generan tag de integridad
IV	Vector de inicialización: aleatorio, se envía con el cifrado
Padding	Relleno del último bloque hasta 16 bytes (pad/unpad)



[Volver al índice de la unidad](#) · Anterior: [02 · AES](#) · Siguiente: [04 · RSA](#)

04 — RSA

Generar el par de claves y cifrar con la pública 🔑

📖 Estás en: 🧬 U09 · Cifrado Moderno → 04 · RSA

📌 La idea en una frase

RSA es el cifrado asimétrico por excelencia: generas un **par de claves** (pública + privada), cifras con la **pública** del destinatario y solo su **privada** puede descifrar. Así se resuelve el problema de compartir la clave del cifrado simétrico.

Es lento y tiene límite de tamaño, pero hace algo que AES no puede: **distribuir secretos sin compartirlos**. Por eso es la pieza que reparte claves en HTTPS, el correo cifrado y las firmas digitales.

🔑 Generar el par de claves

Con pycryptodome, generar 2048 bits es una línea:

```
from Crypto.PublicKey import RSA

# Generar par de claves
clave_rsa = RSA.generate(2048)
privada = clave_rsa.export_key()
publica = clave_rsa.publickey().export_key()

# Las claves se exportan en formato PEM (texto legible)
print("Pública (PEM):")
print(publica.decode())
print("Privada (PEM):")
print(privada.decode())
```

```
Pública (PEM):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...
-----END PUBLIC KEY-----
```


- La **pública** se puede compartir por cualquier sitio (es *pública*).
- La **privada** es secreto absoluto: quien la tenga puede descifrar todo lo que se cifre con tu pública y firmar en tu nombre.
- Generar claves RSA tarda **~1-2 segundos** (busca números primos enormes). AES genera su clave instantáneamente.



Cifrar con la pública, descifrar con la privada

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

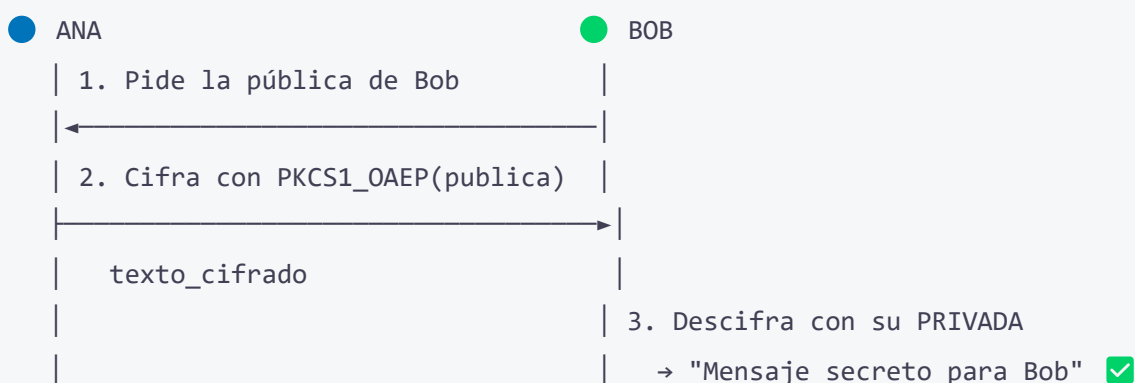
# Generar par de claves
clave_rsa = RSA.generate(2048)
privada = clave_rsa.export_key()
publica = clave_rsa.publickey().export_key()

# Cifrar con clave PÚBLICA
cifrador = PKCS1_OAEP.new(RSA.import_key(publica))
mensaje = b"Mensaje secreto para Bob"
texto_cifrado = cifrador.encrypt(mensaje)

# Descifrar con clave PRIVADA
descifrador = PKCS1_OAEP.new(RSA.import_key(privada))
texto_descifrado = descifrador.decrypt(texto_cifrado)
print(f"Descifrado: {texto_descifrado}")
```

Descifrado: b'Mensaje secreto para Bob'

El flujo entre dos personas:



- **PKCS1_OAEP** es el esquema de relleno de RSA en pycryptodome: añade aleatoriedad al cifrado (el mismo mensaje cifrado dos veces da resultados distintos).
- Nadie más puede descifrar: ni quien intercepte el tráfico, ni quien tenga la pública. Solo la **privada de Bob**.

El límite de RSA

RSA no cifra cualquier cosa:

- Con claves de 2048 bits, el tamaño máximo de mensaje es de **~190 bytes** (depende del relleno).
- La velocidad ronda **~1 MB/s**, frente a los ~1 GB/s de AES.

Intentar cifrar un mensaje de 300 bytes:

ValueError: Ciphertext with incorrect length. ← RSA no puede con tanto

 Para mensajes largos, RSA cifra la **clave AES** (32 bytes) y AES cifra el mensaje completo. Ese es el **cifrado híbrido** del [punto 6](#).

Mini-chequeo

1. ¿Con qué clave cifras un mensaje para que solo lo lea Bob?
2. ¿Qué pasa si se filtra la clave privada de Bob?
3. ¿Por qué no puedes cifrar un archivo de 1 GB con RSA?

>  **Respuestas**

Resumen en 3 frases

- RSA genera un **par de claves**: pública (compartible) y privada (secreta), en formato PEM.
- Cifras con `PKCS1_OAEP.new(pública).encrypt()` y descifras con `PKCS1_OAEP.new(privada).decrypt()`.
- Es lento y limitado a ~190 bytes: sirve para repartir secretos pequeños, no para cifrar el tráfico.



Vocabulario rápido

Término	Idea general
RSA	Cifrado asimétrico basado en la dificultad de factorizar primos
Par de claves	Pública + privada generadas juntas (<code>RSA.generate(2048)</code>)
PKCS1_OAEP	Esquema de relleno seguro de RSA en pycryptodome
PEM	Formato de texto en el que se exportan las claves
Clave privada	Secreto absoluto: descifra y firma
Límite de RSA	~190 bytes de mensaje con claves de 2048 bits

 [Volver al índice de la unidad](#) · Anterior: [03 · Modos de AES](#) · Siguiente: [05 · Firmas digitales](#)

05 — Firmas digitales

Firmar y verificar: integridad + autenticidad 🍌

📖 Estás en: 🧬 U09 · Cifrado Moderno → 05 · Firmas digitales

📌 La idea en una frase

Una **firma digital** demuestra que un mensaje fue creado por quien dice serlo (autenticidad) y que no fue modificado (integridad): se cifra el **hash SHA-256** del mensaje con la clave **privada** del firmante, y cualquiera puede verificarlo con su clave **pública**.

Es el equivalente digital de la firma en papel, pero mejor: no se puede copiar, no se puede falsificar y arrastra consigo la garantía de que nadie tocó el documento. Lo usan los certificados HTTPS, el software firmado y los correos con firma.

🖋️ Firmar (con tu clave privada)

```
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = b"Este mensaje es de Ana"

h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)
print(f"Firma: {firma.hex()[:32]}...")
```

Firma: 1a3f5b7c9d2e4f6a8b0c1d2e3f4a5b6c...

Firmar es cifrar un hash con tu privada:

1. Calculas SHA256 del mensaje (256 bits de huella, lo viste en la U08).
2. Cifras ese hash con tu **clave privada** → eso es la firma.
3. Envías mensaje + firma.

✅ Verificar (con la clave pública del firmante)

```
try:
    h = SHA256.new(mensaje)
    pkcs1_15.new(clave.publickey()).verify(h, firma)
    print("✅ Firma VÁLIDA – mensaje de Ana, no alterado")
except (ValueError, TypeError):
    print("❌ Firma INVÁLIDA – manipulada o no es de Ana")
```

✅ Firma VÁLIDA – mensaje de Ana, no alterado

El flujo completo:

1. Ana escribe mensaje
2. Calcula SHA256 del mensaje
3. Cifra el hash con su clave PRIVADA → esto es la firma
4. Envía: mensaje + firma
5. Bob recibe
6. Descifra firma con clave PÚBLICA de Ana → hash original
7. Calcula SHA256 del mensaje recibido
8. ¿Coinciden? → ✅ Es de Ana y nadie lo modificó

🔑 **Dos garantías en una:** si los hashes no coinciden, el mensaje fue **modificado** (falla la integridad). Si la clave pública de Ana no descifra la firma, el mensaje **no es de Ana** (falla la autenticidad). El hash da integridad; la clave privada da autenticidad.

🤖 Be the code, my friend — Firma y verificación

“Sé el proceso de firma digital desde que Ana escribe hasta que Bob verifica.”

● ANA

1. Tiene mensaje: "Mañana a las 8 en el café"
2. SHA256 → 256 bits de hash
3. Cifra hash con su RSA privada → firma digital (256 bytes)
4. Envía a Bob: [mensaje] + [firma]

🚀 Por la red viaja: "Mañana a las 8 en el café" + 256 bytes de firma

● BOB

5. Recibe mensaje + firma
6. Calcula SHA256 del mensaje recibido
7. Descifra la firma con clave pública de Ana
8. Compara hashes...

¿Son iguales? → ☒ Mensaje de Ana
¿Son diferentes? → ☐ Algo va mal

9. Si alguien modificó el mensaje: hashes diferentes → ❌
10. Si no es de Ana: la clave pública no descifra la firma → ❌

Mini-chequeo

1. ¿Con qué clave se firma y con qué clave se verifica?
2. ¿Qué garantiza que el mensaje no fue modificado?
3. ¿Qué falla si el mensaje se altera después de firmar?

>  Respuestas

Resumen en 3 frases

- Firmar = cifrar el **SHA-256** del mensaje con tu **clave privada** (`pkcs1_15.new(clave).sign(h)`).
- Verificar = descifrar la firma con la **clave pública** del firmante y comparar hashes (`verify`).

- La firma aporta **autenticidad** (quién) e **integridad** (no alterado) a la vez.



Vocabulario rápido

Término	Idea general
Firma digital	Hash del mensaje cifrado con la clave privada del firmante
pkcs1_15	Esquema de firma RSA con hash en pycryptodome
Verificar firma	Descifrar la firma con la pública y comparar hashes
Autenticidad	Garantía de quién creó el mensaje
Integridad	Garantía de que no fue modificado



[Volver al índice de la unidad](#) · Anterior: [04 · RSA](#) · Siguiente: [06 · Cifrado híbrido](#)

06 — Cifrado híbrido

AES + RSA, el método que usa HTTPS 🍷

📖 Estás en: 🧬 U09 · Cifrado Moderno → 06 · Cifrado híbrido

📌 La idea en una frase

El **cifrado híbrido** combina lo mejor de ambos mundos: **RSA** cifra una **clave AES** aleatoria (que viaja protegida) y **AES** cifra el mensaje completo. Rápido como el simétrico, con el reparto de claves resuelto por el asimétrico. Es exactamente lo que hace **HTTPS**.

En el [punto 4](#) viste que RSA no cifra mensajes grandes. La solución: no cifres el mensaje con RSA, cifra la **clave de sesión** AES. El mensaje (por grande que sea) lo cifra AES, que es rapidísimo.

🔄 El intercambio, paso a paso



🖼️ Diagrama del cifrado híbrido

📦 **Lo que viaja por la red:** `clave_AES_cifrada_RSA + nonce + tag + mensaje_cifrado_AES`. El atacante ve la clave AES cifrada con RSA: inútil sin la privada de Bob.


```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

# Claves de Bob
clave_bob = RSA.generate(2048)

# Ana cifra
clave_aes = get_random_bytes(32)
mensaje = "Hola Bob, ¿quedamos mañana?".encode()

cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje)

cifrador_rsa = PKCS1_OAEP.new(clave_bob.publickey())
clave_aes_cifrada = cifrador_rsa.encrypt(clave_aes)

# Se envía: (clave_aes_cifrada, nonce, tag, cifrado)

# Bob descifra
descifrador_rsa = PKCS1_OAEP.new(clave_bob)
clave_aes_recibida = descifrador_rsa.decrypt(clave_aes_cifrada)

descifrador_aes = AES.new(clave_aes_recibida, AES.MODE_EAX, nonce=cifrador_aes.nonce)
mensaje_descifrado = descifrador_aes.decrypt(cifrado)

print(f"Mensaje descifrado: {mensaje_descifrado.decode()}")
```

Mensaje descifrado: Hola Bob, ¿quedamos mañana?

Desglose de las dos mitades:

- **RSA hace el reparto:** cifra los 32 bytes de la clave AES con la pública de Bob. Ese es el secreto que viaja protegido.
- **AES hace el volumen:** cifra el mensaje (de cualquier tamaño) con esa clave. Y su tag verifica la integridad.



Por qué HTTPS usa esto

Este es el método que usa **HTTPS**: RSA para negociar la clave de sesión, AES para cifrar el tráfico.

Cuando entras en una web con `https://`:

1. Tu navegador genera una **clave AES** aleatoria (la clave de sesión).
2. La cifra con la **clave pública RSA** del servidor (su certificado).
3. El servidor la descifra con su **clave privada**: ahora ambos comparten la misma clave AES.
4. Todo el tráfico de la sesión se cifra con **AES**, rápido, y con **firma** (via TLS) para la integridad.

RSA solo se usa al principio, para repartir el secreto. El resto, a toda velocidad con AES. Así trabajan también WhatsApp, Signal, el correo cifrado y las VPN.



Mini-chequeo

1. ¿Qué cifra RSA en un esquema híbrido? ¿Y qué cifra AES?
2. ¿Por qué no se cifra el mensaje completo con RSA?
3. ¿Qué envía Ana a Bob y qué es lo único que no puede leer un atacante?

>  Respuestas



Resumen en 3 frases

- El híbrido junta **RSA** (repartir la clave) y **AES** (cifrar el volumen).
- El mensaje que viaja es `clave_AES_cifrada_RSA + nonce + tag + cifrado`.
- Es el mecanismo de **HTTPS**: RSA negocia la clave de sesión y AES cifra el tráfico.



Vocabulario rápido

Término

Idea general

Cifrado híbrido

AES para el mensaje + RSA para la clave AES

Término	Idea general
Clave de sesión	Clave AES aleatoria generada para una comunicación
Clave_AES_cifrada_RSA	El secreto que viaja protegido por RSA
HTTPS	TLS sobre HTTP: RSA negocia, AES cifra
Tag	Autenticación del cifrado AES (integridad)

 [Volver al índice de la unidad](#) · **Anterior:** [05 · Firmas digitales](#) · **Siguiente:** [07 · RBAC y roles](#)

07 — RBAC y roles

Control de acceso basado en roles 

 Estás en:  U09 · Cifrado Moderno → 07 · RBAC y roles

La idea en una frase

RBAC (*Role-Based Access Control*) es la forma de decidir **quién puede hacer qué**: los permisos no se asignan a cada persona, sino a su **rol**, y cada usuario hereda los permisos de su rol. Es el principio de **mínimo privilegio** de la U08 llevado a la práctica.

Un becario no necesita borrar la tabla de usuarios. Un editor no necesita compartir documentos con el mundo. Con roles, esa regla se escribe **una sola vez** y se aplica a todos: cambias el rol y cambian los permisos, sin tocar al usuario.

Los actores

```

class Usuario:
    def __init__(self, nombre, rol):
        self.nombre = nombre
        self.rol = rol

class Documento:
    def __init__(self, titulo, contenido):
        self.titulo = titulo
        self.contenido = contenido

PERMISOS = {
    "admin": ["leer", "escribir", "borrar", "compartir"],
    "editor": ["leer", "escribir"],
    "lector": ["leer"],
}

def puede(usuario, accion):
    return accion in PERMISOS.get(usuario.rol, [])

ana = Usuario("Ana", "admin")
bob = Usuario("Bob", "lector")

print(f"Ana puede borrar: {puede(ana, 'borrar')}")    # True
print(f"Bob puede borrar: {puede(bob, 'borrar')}")    # False

```

```

Ana puede borrar: True
Bob puede borrar: False

```

Cómo funciona la máquina:

- PERMISOS es un diccionario **rol → lista de acciones**.
- `puede(usuario, accion)` mira el rol del usuario y comprueba si la acción está en su lista.
- `PERMISOS.get(usuario.rol, [])` devuelve lista vacía para roles desconocidos: **por defecto, nada permitido** (mínimo privilegio).



La matriz de permisos

Rol	Leer	Escribir	Borrar	Compartir
admin	✓	✓	✓	✓
editor	✓	✓	✗	✗
lector	✓	✗	✗	✗

💡 **Matriz de control de acceso:** esta tabla es la “constitución” del sistema. Antes de escribir una sola línea de RBAC, decides quién tiene qué; luego la implementas, como en el código anterior. Si un permiso cambia, solo cambias la tabla (y el diccionario), nunca a cada usuario.

🔒 RBAC también protege la criptografía

En un sistema real, RBAC no solo decide leer/escribir: también decide **quién puede cifrar, descifrar o firmar**.

```

PERMISOS = {
    "admin":    ["cifrar", "descifrar", "firmar"],
    "usuario":  ["cifrar", "firmar"],
    "invitado": ["cifrar"],
}

def puede(usuario, accion):
    return accion in PERMISOS.get(usuario["rol"], [])

print(puede({"rol": "admin"}, "descifrar"))    # True
print(puede({"rol": "invitado"}, "firmar"))    # False

```

La combinación de cifrado + RBAC es la que usan los sistemas de archivos cifrados, los repositorios de secretos y las plataformas de gestión de claves: **incluso con la clave correcta, sin el rol correcto no se puede**.

🧠 Mini-chequeo

1. ¿Qué ventaja tiene asignar permisos por rol y no por usuario?
2. ¿Qué devuelve puede para un usuario con un rol que no está en PERMISOS?

3. ¿Qué relación tiene RBAC con el principio de mínimo privilegio?

>  Respuestas

Resumen en 3 frases

- RBAC asigna permisos a **roles**, no a personas, y cada usuario hereda los de su rol.
- La **matriz rol × acción** es la regla del sistema; se implementa con un diccionario y una función puede.
- Es la puesta en práctica del **mínimo privilegio** de la U08, y controla también quién puede cifrar, descifrar o firmar.

Vocabulario rápido

Término	Idea general
RBAC	Control de acceso basado en roles
Rol	El paquete de permisos de una función (admin, editor, lector)
Permiso	Una acción concreta (leer, escribir, borrar, compartir)
Matriz de acceso	Tabla rol × acción que define el sistema
Mínimo privilegio	Cada rol solo con los permisos que necesita

 [Volver al índice de la unidad](#) · Anterior: [06 · Cifrado híbrido](#) · Siguiente: [08 · Práctica: sistema seguro](#)

08 — Práctica: sistema seguro

Cifrar, firmar y verificar un sistema completo 🏗️

📖 Estás en: 🧬 U09 · Cifrado Moderno → 08 · Práctica: sistema seguro

📌 La idea en una frase

Este punto junta todas las piezas de la unidad en un **mini sistema seguro**: cifrar un mensaje con **AES** (confidencialidad), proteger su **clave** con **RSA** (reparto) y **firmarlo** (autenticidad + integridad). Un mensaje que viaja cifrado, solo para el destinatario, y firmado por quien dice serlo.

Es la versión “en pequeño” de lo que hacen los protocolos reales: cifrado híbrido para el contenido + firma para la autoría. Aquí lo montas con las piezas que ya conoces de los puntos 2 a 7.

🏗️ El sistema completo


```

from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes

# — 1. Claves de Ana (firmante) y Bob (destinatario)
clave_ana = RSA.generate(2048)
clave_bob = RSA.generate(2048)

# — 2. Ana prepara el mensaje
mensaje = "Plan de ataque para el proyecto: mañana a las 8".encode()

# — 3. Cifrado híbrido: AES cifra, RSA protege la clave AES
clave_aes = get_random_bytes(32)
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje)
clave_aes_cifrada = PKCS1_OAEP.new(clave_bob.publickey()).encrypt(clave_aes)

# — 4. Firma: SHA256 + clave privada de Ana
h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave_ana).sign(h)

# — 5. Bob recibe y descifra
clave_aes_recibida = PKCS1_OAEP.new(clave_bob).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)

# — 6. Bob verifica la firma de Ana
try:
    pkcs1_15.new(clave_ana.publickey()).verify(SHA256.new(original), firma)
    print("✅ Firma VÁLIDA – mensaje de Ana, no alterado")
except (ValueError, TypeError):
    print("❌ Firma INVÁLIDA – no es de Ana o fue manipulado")

print(f"📦 Mensaje recibido: {original.decode()}")

```

✅ Firma VÁLIDA – mensaje de Ana, no alterado

📦 Mensaje recibido: Plan de ataque para el proyecto: mañana a las 8

Las 6 fases del sistema:

Fase	Pieza	Garantía
3a. Cifrar mensaje	AES (EAX)	Confidencialidad del contenido
3b. Proteger la clave AES	RSA (PKCS1_OAEP)	Solo Bob la descifra
4. Firmar	SHA256 + RSA privada	Autenticidad de Ana
5. Descifrar	RSA privada + AES	Confidencialidad
6. Verificar	SHA256 + RSA pública	Autenticidad + integridad



Be the code, my friend — El mensaje completo

“Sé el mensaje desde que Ana lo escribe hasta que Bob lo lee y lo verifica.”

● ANA

1. Escribe: "Plan de ataque para el proyecto: mañana a las 8"
2. Genera clave AES de 32 bytes
3. Cifra con AES (EAX) → cifrado + tag
4. Cifra la clave AES con la RSA pública de Bob → clave_aes_cifrada
5. SHA256 del mensaje → lo firma con su RSA privada → firma

🚀 Viaja por la red:

clave_AES_cifrada_RSA	nonce	tag	cifrado
			firma

● BOB

6. Descifra clave_aes_cifrada con su RSA privada → clave AES
7. Descifra el mensaje con AES (nonce + cifrado)
8. SHA256 del mensaje recibido
9. Verifica la firma con la RSA pública de Ana

¿Firma válida? →	✅	Es de Ana, nadie lo tocó
¿Firma inválida? →	❌	Algo va mal

10. Bob lee: "Plan de ataque para el proyecto: mañana a las 8" 🗿

Aprieta el lápiz

1. **AES básico:** Cifra un mensaje con AES, luego descifralo. Muestra el nonce y el tag.
2. **RSA: cifra y descifra:** Genera un par RSA, cifra un mensaje corto y descifralo.
3. **Cifrado híbrido:** Cifra un mensaje largo con AES, cifra la clave AES con RSA. Simula el intercambio.
4. **Firma y verifica:** Firma un mensaje, luego modifícalo y comprueba que la verificación falla.
5. **RBAC:** Implementa un sistema con 3 roles (admin, editor, lector) y 4 acciones (leer, escribir, borrar, compartir).

> Soluciones



Mini-chequeo

1. ¿Qué garantiza la fase de cifrado AES y qué garantiza la firma?
2. ¿Por qué el mensaje se cifra con AES y no directamente con RSA?
3. ¿Qué recibe Bob del mensaje y qué comprueba antes de confiar en él?

>  Respuestas



Resumen en 3 frases

- Un sistema seguro junta **cifrado híbrido** (AES + RSA) y **firma digital** (SHA-256 + RSA privada).
- El mensaje viaja como `clave_AES_cifrada_RSA + nonce + tag + cifrado + firma`; cada pieza aporta una garantía distinta.
- Con los 5 ejercicios de “Aprieta el lápiz” (y sus soluciones) tienes el material para montar el sistema completo por tu cuenta.



Vocabulario rápido

Término	Idea general
Sistema seguro	Cifrado híbrido + firma digital combinados
Confidencialidad	Solo el destinatario lee (AES + RSA)
Autenticidad	Se sabe quién firmó (RSA privada del firmante)
Integridad	Nadie modificó el mensaje (hash + tag)
Fase	Cada pieza del pipeline cifrar → firmar → verificar



[Volver al índice de la unidad](#) · Anterior: [07 · RBAC y roles](#) · Siguiente: [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé la clave, laboratorio híbrido con firma y el cierre de la unidad 🧠

📖 Estás en: 🧬 U09 · Cifrado Moderno → 09 · Cierre

Has terminado la teoría: simétrico vs asimétrico, AES con sus modos, RSA, firmas digitales, cifrado híbrido y RBAC. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé la clave

Primero eres una clave simétrica: la compartes con Ana y con Bob. Luego te conviertes en un par asimétrico: una mitad pública y otra privada.

Primera vida: eres la clave AES (simétrica).

1. Ana te genera con `get_random_bytes(32)`: eres 32 bytes aleatorios.
2. Ana cifra su mensaje contigo: `AES.new(tú, AES.MODE_EAX)` → texto cifrado + tag.
3. Bob necesita descifrar... pero tú no viajas en el paquete. El **problema de la distribución de claves**: ¿cómo llegas a Bob sin que nadie te intercepte?
4. Si un atacante te captura, lo descifra todo. Eres poderosa y rápida, pero frágil en el reparto.

Segunda vida: eres el par RSA (asimétrico).

1. Naces como par: `RSA.generate(2048)`. Tu mitad **pública** se puede publicar en todos los sitios; tu mitad **privada** jamás sale de casa.
2. Bob cifra un secreto con tu mitad pública... y solo tu mitad privada puede descifrarlo.
3. Un atacante te ve entero: la pública no le sirve para nada sin la privada. Resolviste el reparto.
4. Pero eres lenta y no puedes con mensajes grandes: para el volumen necesitas a tu colega AES.

💡 **Ahora tú:** ¿quién gana la partida? Ninguno: el [punto 6](#) te enseñó que trabajan en equipo. RSA reparte la clave AES, y AES cifra el tráfico. Eso es lo que hace HTTPS.

Fireside Chat: AES vs RSA

Dos cifrados se sientan junto a la chimenea a decidir, de una vez, quién es el más importante.

AES: — Soy rapidísimo. Cifro un archivo entero en milisegundos. ¿El problema? Ambos necesitamos la misma clave.

RSA: — Yo soy lento, pero elegante. Tú me das tu clave pública, yo cifro, y solo tú con tu privada puedes descifrar.

AES: — Entonces, ¿para qué sirves si eres tan lento?

RSA: — Para **distribuir tu clave**. Yo cifro tu clave AES con mi RSA. Tú la descifras con tu privada. Luego usamos AES para todo.

AES: — O sea, ¿trabajamos en equipo?

RSA: — Exacto. Eso se llama **cifrado híbrido**. Lo usan HTTPS, WhatsApp, Signal...

Moraleja: AES aporta velocidad y volumen; RSA aporta el reparto de secretos. Juntos mueven Internet.



¿Quién soy?

1. Soy el número aleatorio único que viaja junto al cifrado para poder descifrar.
2. Soy el código de autenticación que detecta si el mensaje fue manipulado.
3. Soy el cifrado simétrico estándar, con claves de 16, 24 o 32 bytes.
4. Soy el modo de AES que cifra cada bloque solo y deja ver patrones (¡no me uses!).
5. Soy la clave que se puede repartir por todos los sitios, pero no descifro yo sola.
6. Soy el resultado de cifrar el SHA-256 de un mensaje con la clave privada del firmante.

>  Respuestas



CONRAD VS EL MUNDO: “cifré el mensaje pero no puedo descifrarlo”

CONRAD: — “Clásico: cifras un mensaje con AES, lo envías, y al descifrar obtienes basura o una excepción. Razones: 1) Usaste una **clave de 7 caracteres** como `b'clave123'`: AES espera

exactamente 16, 24 o 32 bytes, y una frase corta no lo es. 2) **No guardaste el nonce**: cada AES.new genera uno nuevo, y sin el mismo nonce no hay descifrado posible. 3) Cifraste con una clave y descifraste con otra: *‘¿será que la clave cambió sola?’* ¡No, Conrad! La misma clave y el mismo nonce, siempre. 4) **No verificaste el tag**: decrypt devuelve basura en silencio; con decrypt_and_verify la manipulación salta al instante. 5) Usaste **ECB** para datos estructurados y los patrones se ven a simple vista.”

CONRAD: — “Y lo mejor: *‘es que el modo EAX no me deja descifrar con ECB’*. ¡Pues claro! El modo de cifrado es **parte de la receta**: si cifras con EAX, descifras con EAX y con el mismo nonce. Mezclar modos es mezclar cerraduras de casas distintas.”

CONRAD: — “Y no me vengas con *‘¿será que pycryptodome está roto?’*. La biblioteca lleva años auditada ([punto 1 de la U08](#): no inventes tu cripto). Si no descifras, revisa la clave, el nonce y el modo. A diagnosticar.”

Laboratorio de tortura: mensaje cifrado y firmado

Duración: 45 minutos **Herramienta:** Python 3 (pip install pycryptodome)

Escenario: Ana quiere enviar a Bob un mensaje **confidencial** (solo Bob lo lee) y **autenticado** (solo Ana pudo crearlo). Monta el sistema completo del [punto 8](#), pero con dos pares RSA distintos: el de Ana (firmante) y el de Bob (destinatario).

Tareas paso a paso:

1. Genera `clave_ana = RSA.generate(2048)` y `clave_bob = RSA.generate(2048)`.
2. Ana cifra `b"Mensaje ultrasecreto"` con **AES (EAX)**: `cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje)`, y protege su clave AES con `PKCS1_OAEP.new(clave_bob.publickey()).encrypt(clave_aes)`.
3. Ana **firma** el mensaje con su clave privada: `pkcs1_15.new(clave_ana).sign(SHA256.new(mensaje))`.
4. Bob **descifra** con su privada (PKCS1_OAEP) y con AES (mismo nonce).
5. Bob **verifica** la firma con la **pública de Ana** y muestra  o .
6. Modifica **un byte** del mensaje descifrado y comprueba que, si se volviera a firmar con el original, la verificación del tocado falla.

Fallo intencionado: en la verificación de la firma, usa la clave **pública de Bob** en lugar de la de Ana. ¿Qué pasa? La verificación lanza `ValueError`: la firma de Ana **no verifica con la pública de Bob**, aunque el mensaje sea íntegro. El sistema parece roto... pero no lo está: estás verificando con la clave equivocada.

Pista 1: la firma se verifica **siempre con la clave pública del firmante**, no con la del destinatario. Ana firma con su privada → se verifica con su pública. Bob solo cifra/descifra con su par.

Pista 2: si no ves el fallo, imprime de quién es cada clave antes de verificar: `print(clave_ana.publickey().export_key().decode()[:50])` frente a la de Bob. Si la pública que usas para verificar no es la de Ana, ese es el bug.

Logros de esta unidad

Logro	Cómo conseguirlo
 Simétric Master	Cifrar y descifrar con AES (EAX) mostrando nonce y tag
 Key Pair Forger	Generar un par RSA de 2048 bits y cifrar/descifrar con PKCS1_OAEP
 Hybrid Engineer	Montar el cifrado híbrido AES + RSA completo
 Signature Detective	Firmar y verificar, detectando una firma manipulada
 RBAC Architect	Implementar un sistema de roles y permisos con puede

Atrévete a pensar

1. ¿Por qué no se cifra todo el tráfico de Internet directamente con RSA?
2. ¿Qué pasaría si reutilizaras el mismo nonce dos veces con la misma clave AES?
3. ¿Qué diferencia hay entre cifrar un mensaje con RSA y firmarlo con RSA?
4. ¿Por qué la firma se verifica con la clave pública del firmante y no con la del destinatario?
5. ¿Cómo garantiza el cifrado híbrido que un atacante que intercepta el paquete no pueda leer nada?

🧩 Crucigrama de bits

Horizontal:

1. Cifrado simétrico estándar (3 letras)
4. Cifrado asimétrico más famoso (3 letras)
6. Código de autenticación del cifrado (3 letras)
8. Valor aleatorio único que viaja con el cifrado (5 letras)

Vertical:

2. Modo de AES inseguro que muestra patrones (3 letras)
3. Clave que se reparte libremente (7 letras)
5. Vector de inicialización del modo CBC (2 letras)
7. Mezcla de AES y RSA (7 letras)

💬 Entrevista de trabajo

1. “¿Qué diferencia hay entre cifrado simétrico y asimétrico? ¿Cuándo usarías cada uno?”
2. “¿Cómo cifrarías un mensaje largo para que solo lo lea un destinatario concreto?”
3. “¿Qué es una firma digital y qué garantiza?”
4. “Explica cómo funciona HTTPS por dentro.”
5. “¿Qué es RBAC y cómo lo implementarías en un sistema?”

💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, recorre la comparativa del [punto 1](#): una clave vs par, velocidad vs reparto, y el complemento híbrido. Para la 4, cuenta la escena del [punto 6](#): el navegador genera una clave AES, la cifra con la pública del servidor (su certificado), el servidor la descifra con su privada, y el tráfico se cifra con AES. Si sabes contarlo fluido, ya eres medio desarrollador seguro.

🙋 No hay preguntas tontas

❓ ¿Qué clave uso para cada operación?

- Cifrar para alguien: usa su clave **pública**
- Descifrar: usa tu clave **privada**
- Firmar: usa tu clave **privada**
- Verificar firma: usa la clave **pública** del firmante

? ¿Qué es más seguro, AES-256 o RSA-2048?

Ambos son seguros hoy en día. No compiten: son herramientas diferentes para problemas diferentes. AES protege el volumen; RSA reparte secretos. Elegir uno sobre el otro no tiene sentido: el sistema seguro los usa juntos.

? ¿Qué pasa si pierdo mi clave privada?

Pierdes acceso a todo lo cifrado con tu clave pública. Por eso se hacen **copias de seguridad** (y se guardan bien). Perder la privada es perder la llave de la caja fuerte: el cifrado hace su trabajo demasiado bien.

? ¿Puedo tener la misma clave RSA siempre?

Sí, las claves no caducan. Pero por seguridad se recomienda rotarlas cada cierto tiempo (como cambiar la contraseña). Es el principio de **rotación de claves** de la U08.

? ¿Cuánto tarda RSA en generar claves?

Generar RSA 2048 bits lleva ~1-2 segundos. AES genera clave instantáneamente. Esa diferencia también explica por qué RSA se usa una vez (repartir) y AES todo el rato (cifrar).

Poscréditos

Un mensaje cifrado viaja por la red. Solo el destinatario puede leerlo.

RSA repartió la clave. AES cifró el contenido. La firma garantizó quién lo escribió.

Nadie pudo espiar. Nadie pudo fingir. Nadie pudo modificar.

PRÓXIMAMENTE EN U10: *servidores concurrentes: un servidor que atiende a muchos clientes a la vez sin bloquearse. ThreadPool, hilos y benchmark.*

Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen la integridad y la confidencialidad de los datos.

CE	Criterio	Cubierto
a)	Principios básicos de seguridad	✓ Simétrico vs asimétrico (punto 1) + RBAC (punto 7)
b)	Tipos de cifrado (simétrico, asimétrico)	✓ Puntos 1, 2 y 4
c)	Implementa funciones hash (MD5, SHA)	✓ Cubierto en la U08; se usa en las firmas (punto 5)
d)	AES	✓ Puntos 2-3 + ⚡ Laboratorio
e)	RSA	✓ Puntos 4-5 + ⚡ Laboratorio
f)	Firmas digitales	✓ Punto 5 + ⚡ Laboratorio
g)	Cifrado híbrido	✓ Punto 6 + ⚡ Laboratorio
h)	Conoce sistemas de roles y RBAC	✓ Punto 7

 [Volver al índice de la unidad](#) · Anterior: [08 · Práctica sistema seguro](#) · Siguiente: [U10 · Servidores Concurrentes](#)

U10 — Servidores Concurrentes

Servidores que atienden a muchos clientes a la vez 🕒

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 📡 APIs comerciales → 🔒 Hash → 🧬 Cifrado → 🕒 **Servidores Concurrentes** → ⌚ asyncio

“Un servidor secuencial atiende a un cliente cada vez. Los demás esperan. Un servidor concurrente atiende a todos a la vez. Como un camarero con 10 mesas.”

Hasta ahora has construido servidores TCP que atienden **de uno en uno**: si un cliente tarda 3 segundos en procesarse, los que llegan detrás esperan su turno en una cola invisible. Esos servidores funcionan, pero se quedan cortos en cuanto el tráfico crece. En esta unidad darás el salto a la **conurrencia**: primero lanzando un **hilo por cada cliente** y después con un **ThreadPoolExecutor**, el equipo de hilos reutilizables que usa la industria.

También medirás cuánto ganas con cada enfoque mediante un **benchmark**, aprenderás a proteger con **Lock** el estado compartido del servidor (esos contadores globales que varios hilos tocan a la vez) y cerrarás montando un servidor concurrente completo con su lanzador de clientes. El siguiente tema, [U11 · asyncio y Disponibilidad](#), tomará el relevo con un modelo aún más ligero para miles de conexiones.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar por qué un servidor secuencial solo atiende un cliente cada vez y cuál es su límite real.
- Identificar el problema del **cliente lento**: cómo bloquea a todos los que esperan en la cola.
- Implementar un **servidor multihilo** que lanza un hilo por cada conexión entrante.
- Gestionar un **ThreadPoolExecutor** con un número fijo de hilos reutilizables.
- Medir el rendimiento de un servidor con un **benchmark** de N clientes simultáneos.
- Proteger con **Lock** las variables compartidas entre los hilos del servidor.

- Conocer los **límites del sistema** (número de hilos, context switch) y aplicar buenas prácticas.
- Montar un **servidor concurrente completo** con su lanzador de clientes.
- Decidir con criterio cuándo usar hilo por cliente y cuándo ThreadPool en producción.



Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Servidor secuencial	El servidor que atiende de uno en uno y su límite	Todos
02 · El problema de la espera	Por qué un cliente lento congela a toda la cola	Todos
03 · Hilo por cliente	Lanzar un hilo por cada conexión y volver a escuchar	Todos
04 · ThreadPoolExecutor	Un equipo fijo de hilos reutilizables y el Pool Puzzle	Todos
05 · Benchmark	Medir cuánto ganas con la concurrencia	Todos
06 · Sincronización en servidores	El Lock para el estado compartido entre hilos	Todos
07 · Límites y buenas prácticas	Cuántos hilos puedes crear y cómo hacerlo bien	Todos
08 · Servidor concurrente completo	Todo junto: servidor, lanzador y Aprieta el lápiz	Todos
09 · Cierre	Sé el servidor, Fireside, Laboratorio de tortura...	Todos



Flujo de lectura: los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.



Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

✓ Inicial resuelto

● Inicial por resolver

👤 Avanzado resuelto

★ Avanzado por resolver

✓ Criterios de evaluación cubiertos (RA4c-d)

RA4: Implementa servicios en red, desarrollando servidores concurrentes capaces de atender a varios clientes simultáneamente.

CE	Criterio	Dónde se cubre
c)	Implementa servidores concurrentes con hilos	✓ Puntos 1-3 y 8 + ⚡ Laboratorio (punto 9)
d)	Gestiona pools de hilos (ThreadPoolExecutor)	✓ Puntos 4-5 y 8 + ⚡ Laboratorio (punto 9)

RA4a-b (APIs REST y comerciales) se cubren en las **U06 y U07**. RA4e-g (asyncio, disponibilidad, comparativa de modelos) se cubren en la **U11 · asyncio y Disponibilidad**.

📖 ¿Por dónde empiezo?

¿Vienes de la U09 y dominas el cifrado moderno? Ese no es el trampolín que necesitas: lo que traes fresco de las [U04](#) y [U05](#) son los sockets TCP cliente-servidor. Repasa el `accept()`, el `recv()` y el `sendall()` de la U04 y arranca en el [punto 1](#), que parte justo de un servidor TCP normal y corriente.

¿Ya sabes escribir servidores y solo quieres el salto a la concurrencia? Ve directo al [punto 3](#) o, si tienes prisa por el material “de producción”, al [punto 4](#). Pero si vienes de cero, no te saltes los puntos 1 y 2: entender el problema del servidor secuencial es la base para valorar todo lo demás.

📌 Primer punto: [01 · Servidor secuencial](#)

▶▶ Al acabar la unidad, continúa en [U11 · asyncio y Disponibilidad](#).

01 — Servidor secuencial

El servidor que atiende a un cliente cada vez 🌈

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 01 · Servidor secuencial

📌 La idea en una frase

Un servidor **secuencial** atiende a un cliente cada vez: `accept()` recibe una conexión, la procesa hasta el final y solo entonces vuelve a aceptar. Los demás clientes esperan en una cola invisible, aunque ya estén conectados.

Es el servidor que construiste en la [U04 · Sockets TCP](#): un bucle `while True` con `accept()`. Funciona, pero tiene un límite muy claro que vamos a medir.

🐢 El servidor secuencial

El ejemplo clásico simula trabajo pesado con un `time.sleep(3)` dentro de la atención a cada cliente:

```
import socket, time

def servidor_lento():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🌈 Servidor SECUENCIAL: 1 cliente cada vez")

        while True:
            conn, addr = srv.accept()
            print(f" Cliente {addr} conectado")
            time.sleep(3) # Simula trabajo pesado
            conn.sendall(b"Procesado\n")
            conn.close()
            print(f" Cliente {addr} atendido")
```

Si 3 clientes se conectan a la vez: el primero tarda 3s, el segundo 6s, el tercero 9s. 🌈

El bucle hace esto: acepta → procesa (los 3 segundos) → cierra → acepta de nuevo. Mientras procesa, el `accept()` siguiente no se ejecuta, así que nadie más entra. El socket del sistema operativo sí “acepta” la conexión TCP a nivel de red (los clientes no ven error), pero el código no los atiende: están **en cola**.

La traza: secuencial vs concurrente

El clásico “Be the code” del tema. Traza mentalmente qué pasa cuando 3 clientes se conectan a la vez:

Servidor secuencial (3 clientes a la vez):

```
Tiempo: 0s — Cliente-1 conecta
        |      Servidor procesa cliente-1 (3s)
3s — Cliente-1 listo
        |      Cliente-2 conectó en t=0.1s, pero espera...
        |      Servidor procesa cliente-2 (3s)
6s — Cliente-2 listo
        |      Cliente-3 conectó en t=0.2s, pero espera...
        |      Servidor procesa cliente-3 (3s)
9s — Cliente-3 listo ❧
```

Servidor concurrente (3 clientes a la vez):

```
Tiempo: 0s — Cliente-1 conecta → hilo-1 procesa
        |      Cliente-2 conecta → hilo-2 procesa
        |      Cliente-3 conecta → hilo-3 procesa
        |      (Los 3 procesan en paralelo)
3s — Cliente-1 listo ❧
        |      Cliente-2 listo ❧
        |      Cliente-3 listo ❧
```

Con hilos, todos terminan a la vez. Sin hilos, el último espera 9s.

Por ahora nos quedamos con la primera columna: el secuencial. El límite se ve solo: **el tiempo total crece linealmente** con el número de clientes. En el [punto 3](#) construiremos el concurrente.

1. ¿Por qué los clientes 2 y 3 “se conectan” sin error pero no reciben respuesta?
2. Si 5 clientes se conectan a la vez y cada uno tarda 2s en procesarse, ¿cuánto espera el quinto?
3. ¿Qué línea del código es la que se queda “atascada” mientras se procesa un cliente?

> Respuestas

Resumen en 3 frases

- El servidor secuencial procesa **un cliente cada vez** y los demás esperan en la cola del sistema operativo.
- Su límite es lineal: **n clientes × tiempo por cliente** de espera para el último.
- La solución a la espera es la **conurrencia**: atender a varios clientes a la vez (punto 3).

Vocabulario rápido

Término	Idea general
Servidor secuencial	Atiende un cliente cada vez, en orden
Cola del socket	Conexiones TCP aceptadas por el SO que aún no atiende el código
accept()	Llamada que recoge la siguiente conexión pendiente
time.sleep(3)	Simula trabajo pesado en los ejemplos
Límite lineal	El tiempo total crece con el número de clientes

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · El problema de la espera](#)

02 — El problema de la espera

El cliente lento que bloquea a todos los demás 🤖

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 02 · El problema de la espera

📌 La idea en una frase

El problema del servidor secuencial no es el número de clientes, sino el **cliente lento**: uno solo que tarda 3 segundos congela a todos los que esperan detrás. Basta un cliente pesado para paralizar el servidor entero.

En el [punto 1](#) viste que el tiempo crece linealmente. Ahora vamos a entender por qué es un problema real, no solo una curiosidad teórica: cualquier cliente que tarde, tarde por lo que sea (cálculo pesado, acceso a una base de datos lenta, una red de 56k...), convierte a todos los demás en espectadores.

🤖 Un cliente lento congela la cola

Vuelve al código del servidor secuencial:

```
import socket, time

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        time.sleep(3)           # ← el trabajo "lento" de este cliente
        conn.sendall(b"Procesado\n")
        conn.close()
```

Traza la secuencia con un cliente lento en medio:

```
Tiempo: 0s — Cliente-1 conecta y entra a procesarse
        | Cliente-2 conecta (t=0.1s) → espera...
        | Cliente-1 tarda 3s (trabajo pesado, red lenta...)
3s — Cliente-1 listo → recién ahora se acepta a Cliente-2
    | Cliente-3 conecta (t=0.2s) → espera...
    | Cliente-2 tarda 3s
6s — Cliente-2 listo → recién ahora se acepta a Cliente-3
    | Cliente-3 tarda 3s
9s — Cliente-3 listo 🏁
```

Fíjate en el detalle clave: **Cliente-2 conectó en t=0.1s** y fue atendido a las 3s. **Cliente-3 conectó en t=0.2s** y fue atendido a las 6s. No es que el servidor sea lento: es que **el trabajo del cliente anterior bloquea el accept()**. El cliente más rápido del mundo espera a que el más lento termine.

⚠ El problema empeora con un solo cliente pesado: una petición que tarde 30 segundos (una consulta enorme, un archivo gigante) pone a **toda** la cola a esperar 30 segundos, y al que llega detrás, otros 30.



La analogía: la ventanilla única

Imagina un banco con **una sola ventanilla**. Llegan 10 personas a la vez. La primera tiene un trámite que dura 3 minutos; las otras 9 esperan en fila. La segunda ya está “en el banco” (se ha conectado al socket), pero su trámite no empieza hasta que la primera termina.



VENTANILLA ÚNICA (servidor secuencial):

[PERSONA-1 ⌚ 3min] [PERSONA-2 🧑] [PERSONA-3 🧑] [PERSONA-4 🧑] ...

La ventanilla solo puede atender a una persona a la vez.



VENTANILLAS MÚLTIPLES (servidor concurrente):

[PERSONA-1 ⌚] [PERSONA-2 ⌚] [PERSONA-3 ⌚] [PERSONA-4 ⌚] ...

Cada persona tiene su ventanilla: todas avanzan a la vez.

La moraleja es obvia: si el banco tiene 10 ventanillas, 10 personas se atienden en paralelo. Esa es exactamente la idea del [punto 3](#): dar a cada cliente su propia “ventanilla” (su hilo).

Situación	Servidor secuencial	Servidor concurrente
10 clientes, 2s cada uno	~20 segundos ($10 \times 2s$)	~2 segundos (en paralelo)
1 cliente lento de 30s	Todos esperan +30s	Solo el lento tarda 30s
Picos de conexiones	La cola crece sin límite visible	Se atienden en paralelo



Mini-chequeo

1. Un cliente tarda 8 segundos en procesarse. ¿Cuánto espera el siguiente cliente que conectó en el mismo segundo?
2. ¿En qué punto del código se “atasca” la atención al resto de clientes?
3. ¿El problema es el número de clientes o la lentitud del procesamiento?

>  Respuestas



Resumen en 3 frases

- Un cliente lento **bloquea el `accept()`** y todos los que esperan detrás se congelan.
- El tiempo de espera del último cliente es **$n \times$ tiempo del más lento** en el peor caso.
- La analogía de la **ventanilla única** lo resume: más ventanillas (hilos) = más clientes a la vez.



Vocabulario rápido

Término	Idea general
Cliente lento	Conexión cuyo procesamiento tarda (cálculo, red, base de datos)
Bloqueo	El <code>accept()</code> no se ejecuta hasta que termina el cliente actual
Cola	Clientes ya conectados al socket que aún no se atienden
Ventanilla única	Analogía del servidor secuencial
Procesamiento	El trabajo que hace el servidor con cada cliente

 [Volver al índice de la unidad](#) · **Anterior:** [01 · Servidor secuencial](#) · **Siguiente:** [03 · Hilo por cliente](#)

03 — Hilo por cliente

Cada cliente se atiende en su propio hilo 

 Estás en:  U10 · Servidores Concurrentes → 03 · Hilo por cliente

La idea en una frase

En lugar de atender al cliente dentro del bucle, el servidor **lanza un hilo por cada cliente** y vuelve inmediatamente a `accept()`. Cada conexión tiene su propia “ventanilla”.

Ya conoces los hilos de la [U02 · Hilos fundamentales](#) y la sincronización de la [U03](#). Este punto los pone a trabajar: el patrón **hilo por cliente** (thread per connection) es el salto más directo desde el servidor secuencial.

El servidor multihilo

Cada cliente recibe su propio hilo. La función que lo atiende se ejecuta en paralelo con todas las demás:

```

import socket, threading

def atender(conn, addr):
    print(f"[+] Cliente {addr} conectado")
    with conn:
        datos = conn.recv(1024)
        print(f"    Recibido: {datos.decode()}")
        import time
        time.sleep(2) # Simular trabajo
        conn.sendall(b"OK: " + datos)
    print(f"[-] Cliente {addr} desconectado")

def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🚀 Servidor MULTITHILO – varios clientes a la vez")

    while True:
        conn, addr = srv.accept()
        hilo = threading.Thread(target=atender, args=(conn, addr))
        hilo.start()

```

Dos diferencias frente al servidor secuencial del [punto 1](#):

1. **No hay trabajo en el bucle principal:** `accept()` → crear hilo → `start()` → vuelta a `accept()`. El bucle nunca se bloquea.
2. **conn se pasa como argumento** al hilo: cada hilo recibe su socket y su dirección, y los libera con `with conn:` al terminar.

🔔 Recuerda la convención del módulo: los hilos se nombran con `.name = "hilo-"+str(n)` si necesitas trazarlos.

Be the code — Servidor multihilo en acción

“Sé el servidor que recibe 3 clientes a la vez. Traza cada hilo.”

● SERVIDOR PRINCIPAL

1. Crea socket, `bind(5000)`, `listen()`
2. `accept()` → espera... ⌚

[Cliente-1: `t=0s`]

3. `accept()` → `conn1`, `addr1`
4. Crea hilo-1 → `start(atender, conn1)`
5. Vuelve a `accept()` inmediatamente

[Cliente-2: `t=0.1s`]

6. `accept()` → `conn2`, `addr2`
7. Crea hilo-2 → `start(atender, conn2)`
8. Vuelve a `accept()`

[Cliente-3: `t=0.2s`]

9. `accept()` → `conn3`, `addr3`
10. Crea hilo-3 → `start(atender, conn3)`
11. Vuelve a `accept()`

AHORA 4 HILOS EJECUTANDO:

Hilo ppal	<code>accept()</code> esperando más clientes
Hilo-1	recibe → procesa (2s) → responde
Hilo-2	recibe → procesa (2s) → responde
Hilo-3	recibe → procesa (2s) → responde

Los 3 clientes son atendidos en paralelo.

A los ~2s, todos reciben respuesta. 🍷

El hilo principal **nunca se detiene**: en cuanto lanza un hilo, vuelve a `accept()`. El trabajo pesado (`time.sleep(2)`) vive en los hilos secundarios, que se ejecutan en paralelo gracias al *threading* de la [U02](#).



Ventajas y el primer cuidado

Ventaja

Explicación



Simple de escribir

Solo un `threading.Thread(target=atender, args=(conn, addr))`

Ventaja	Explicación
✓ Aprovecha la CPU	Varios clientes procesados de verdad en paralelo
✓ El bucle no se bloquea	<code>accept()</code> vuelve al instante, nadie espera en la cola

Pero hay una letra pequeña: **crear un hilo por cada cliente puede saturar el sistema con 10.000 conexiones**. Cada hilo consume memoria y el sistema operativo tiene que gestionarlos. Ese límite lo verás en el [punto 4](#) (el `ThreadPool`) y en el [punto 7](#).

Mini-chequeo

1. ¿Qué hace el hilo principal mientras un hilo secundario procesa a su cliente?
2. ¿Por qué no hace falta `join()` en el bucle principal del servidor?
3. ¿Qué pasaría con este código si llegan 10.000 conexiones a la vez?

>  Respuestas

✓ Resumen en 3 frases

- El patrón **hilo por cliente** lanza un `threading.Thread` por cada conexión y el bucle vuelve al instante a `accept()`.
- Todos los clientes se atienden **en paralelo**, cada uno en su propio hilo.
- Es simple y efectivo para pocos clientes, pero **no escala** a miles de conexiones: para eso existe el `ThreadPool`.

Vocabulario rápido

Término	Idea general
Hilo por cliente	Patrón: un <code>Thread</code> por cada conexión aceptada
Hilo principal	El del bucle <code>accept()</code> : solo acepta y lanza hilos
<code>atender()</code>	Función que procesa a un cliente en su hilo

Término	Idea general
start()	Pone el hilo en marcha sin bloquear al llamante
Thread(target, args)	Constructor del hilo con su función y argumentos

 [Volver al índice de la unidad](#) · Anterior: [02 · El problema de la espera](#) · Siguiente: [04 · ThreadPoolExecutor](#)

04 — ThreadPoolExecutor

Un equipo fijo de hilos que se reutiliza ⚡

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 04 · ThreadPoolExecutor

📌 La idea en una frase

Crear un hilo por cada cliente puede saturar el sistema con 10.000 conexiones. Un **ThreadPoolExecutor** reutiliza un número fijo de hilos: si llegan más clientes que hilos, esperan en una cola en lugar de multiplicar los hilos.

Es el mismo servicio del [punto 3](#), pero con **recursos bajo control**: el pool es un equipo de camareros limitado, no un camarero nuevo por cada mesa.

⚠ El problema: miles de hilos

El enfoque del [punto 3](#) funciona de maravilla con 10 clientes. Con 10.000 conexiones simultáneas se convierte en una bomba:

- Cada hilo consume **memoria propia** (pila de ejecución).
- El sistema operativo debe **planificar** (context switch) entre miles de hilos: la CPU pierde más tiempo cambiando de hilo que trabajando.
- El proceso puede quedarse **sin memoria** y morir.

Enfoque	Ventaja	Desventaja
Hilo por cliente	Simple	Puede saturar el SO (miles de hilos)
ThreadPool	Límite controlado	Si el pool se llena, los clientes esperan

“Si el pool se llena, los clientes esperan” suena a defecto, pero es la **protección** que queremos: esperar en cola es mejor que matar el servidor.

⚡ El servidor con pool

```
import socket, concurrent.futures

MAX_HILOS = 10

def atender(conn, addr):
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK: " + datos)

def servidor_pool():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()

    with concurrent.futures.ThreadPoolExecutor(max_workers=MAX_HILOS) as pool:
        print(f" ⚡ Servidor con Pool de {MAX_HILOS} hilos")
        while True:
            conn, addr = srv.accept()
            pool.submit(atender, conn, addr)
```

Dos piezas nuevas frente al [punto 3](#):

1. `ThreadPoolExecutor(max_workers=MAX_HILOS)` crea el equipo de hilos (10 aquí). Se usa como contexto (`with ... as pool`) para que se cierre solo al salir.
2. `pool.submit(atender, conn, addr)` sustituye a `Thread(target=...).start()`: entrega la tarea al pool y este decide qué hilo libre la coge. Si todos están ocupados, la tarea espera en la cola interna del pool.

⚡ El `submit()` devuelve un `Future`, pero en un servidor no lo necesitamos: no nos interesa el resultado, solo que la tarea se ejecute.

🎯 ¿Qué tamaño de pool elegir?

No hay una fórmula mágica, pero estas reglas de oro ayudan:

Situación

Tamaño razonable

Servidor de prueba / clase

5-10 hilos

Situación	Tamaño razonable
CPU intensivo (cálculo puro)	No de núcleos de la CPU
I/O intensivo (red, discos, APIs)	Más hilos que núcleos (los hilos esperan mucho en I/O)

💡 En [U11 · asyncio](#) verás que la concurrencia por I/O se puede hacer aún más ligera, sin hilos. De momento, el pool es la opción sensata.

🧩 Pool Puzzle — Servidor con ThreadPool

Desordena y reconstruye este servidor que usa ThreadPoolExecutor:

```
a) from concurrent.futures import ThreadPoolExecutor
b)     pool.submit(atender, conn, addr)
c)     with ThreadPoolExecutor(max_workers=5) as pool:
d) import socket
e) def atender(conn, addr):
f)     conn.sendall(b"OK\n")
g) while True:
h)     with socket.socket() as srv:
i)         datos = conn.recv(1024)
j)         conn, addr = srv.accept()
k)         srv.bind(("0.0.0.0", 5000))
l)         srv.listen()
```

> 🔒 Solución

🧠 Mini-chequeo

1. ¿Qué pasa con el cliente número 11 si el pool tiene `max_workers=10` y todos están ocupados?
2. ¿Qué ventaja tiene reutilizar hilos frente a crear uno nuevo por cliente?
3. ¿Por qué el pool se crea fuera del bucle `while True`?

> 📖 Respuestas



Resumen en 3 frases

- `ThreadPoolExecutor(max_workers=N)` mantiene **N hilos fijos reutilizables** y una cola interna.
- Si el pool está lleno, los clientes **esperan en cola** en lugar de multiplicar los hilos.
- Es la opción de **producción**: límite controlado, hilos reutilizados y sin riesgo de saturar el SO.



Vocabulario rápido

Término	Idea general
<code>ThreadPoolExecutor</code>	Pool de hilos reutilizables de <code>concurrent.futures</code>
<code>max_workers</code>	Número máximo de hilos del pool
<code>submit()</code>	Entrega una tarea al pool (devuelve un <code>Future</code>)
Cola interna	Tareas esperando a que un hilo se libere
<code>Future</code>	Objeto con el resultado futuro de la tarea (no lo usamos en servidores)

 [Volver al índice de la unidad](#) · Anterior: [03 · Hilo por cliente](#) · Siguiente: [05 · Benchmark](#)

05 — Benchmark

Medir cuánto ganas con la concurrencia 🕒

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 05 · Benchmark

La idea en una frase

Un **benchmark** lanza N clientes a la vez contra tu servidor y cronometra cuánto tardan todos en recibir respuesta. Es la prueba objetiva de que la concurrencia funciona... o de que tu servidor sigue siendo secuencial.

Hasta ahora hemos razonado sobre tiempos (“el secuencial tarda $n \times \text{tiempo}$ ”). Este punto lo **mide de verdad**: con un lanzador masivo de clientes verás con tus propios ojos la diferencia entre secuencial, hilos y pool.

El experimento

¿Cuánto más rápido es un servidor concurrente? Aquí tienes un experimento que lanza 10 clientes y mide el tiempo total:

```
import socket, time, threading, concurrent.futures

def cliente(id):
    """Conecta, envía 'ping', recibe 'pong'"""
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(b"ping")
        s.recv(1024)

def prueba(tipo_servidor, n=10):
    """Lanza n clientes en paralelo y cronometra"""
    inicios = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=n) as pool:
        futuros = [pool.submit(cliente, i) for i in range(n)]
        concurrent.futures.wait(futuros)
```

Fíjate en un detalle: el propio lanzador usa un `ThreadPoolExecutor` con `n` hilos para lanzar los `n` clientes en paralelo. El cronometraje (los inicios y el tiempo total) lo añade quien ejecute la prueba, usando `time.time()` antes y después.

Los resultados

Con un servidor que tarda **2 segundos por cliente**, la tabla comparativa queda así:

Enfoque	10 clientes (2s c/u)	Fórmula
 Secuencial	~20 segundos	$n \times \text{tiempo_por_cliente}$
 Hilos	~2 segundos	$\max(\text{tiempo_por_cliente})$
 ThreadPool (5 hilos)	~4 segundos	$\text{ceil}(n/\text{workers}) \times \text{tiempo_por_cliente}$

El ThreadPool con 5 hilos tarda el doble que hilos ilimitados, pero **no ahoga el sistema**. Con 1000 clientes, hilos ilimitados matarían el PC; el pool encola y sirve de a 5.

La fórmula del pool lo explica: con 10 clientes y 5 hilos, se atienden **2 tandas** de 5 ($\text{ceil}(10/5) = 2$), cada una de 2 segundos $\rightarrow$ ~4 segundos. El hilo por cliente lo hace en una sola tanda $\rightarrow$ ~2 segundos. Y el secuencial... 10 tandas de uno $\rightarrow$ ~20 segundos.

El lanzador masivo de clientes

Este es el script que probará tu servidor real, el “ Cliente de prueba — lanzador masivo”:


```

import socket, threading, time

def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"    ✅ Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"    ❌ Cliente-{id}: {e}")

print("Lanzando 10 clientes...")
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print("🏁 Todos los clientes terminaron")

```

Este script prueba que tu servidor concurrente realmente atiende a varios clientes a la vez. Si es secuencial, los clientes 2-9 esperarán su turno.

Para medir el tiempo total, envuelve el bloque con `inicio = time.time()` antes de lanzar los hilos y `print(f"Tiempo total: {time.time() - inicio:.2f}s")` al final. Ese número es tu **benchmark**.

Cómo interpretar los resultados

Lo que ves	Qué significa
Tiempo $\approx n \times \text{tiempo_por_cliente}$	Tu servidor es secuencial (o el pool tiene 1 hilo)
Tiempo $\approx \max(\text{tiempo_por_cliente})$	Tu servidor es concurrente (hilos o pool con suficientes workers)

Lo que ves	Qué significa
Tiempo $\approx$ <code>ceil(n/workers)</code> $\times$ <code>tiempo_por_cliente</code>	Tu pool está trabajando en tandas
Errores  <code>ConnectionRefused</code>	El servidor no está arrancado o el puerto es otro

💡 El benchmark también sirve para **elegir el tamaño del pool**: sube `max_workers` y mira dónde deja de mejorar el tiempo. Más hilos no siempre es más rápido (punto 7).

Mini-chequeo

1. ¿Por qué el lanzador usa `join()` sobre todos los hilos?
2. Un servidor con pool de 3 hilos recibe 9 clientes que tardan 1s cada uno. ¿Tiempo total aproximado?
3. ¿Qué le diría la salida del lanzador a un servidor que imprime “Procesado” de uno en uno?

>  **Respuestas**

Resumen en 3 frases

- El benchmark lanza **N clientes en paralelo** y mide el tiempo total con `time.time()`.
- Secuencial $\approx n \times \text{tiempo}$; hilos $\approx \max(\text{tiempo})$; pool $\approx \text{ceil}(n/\text{workers}) \times \text{tiempo}$.
- El lanzador masivo demuestra si tu servidor atiende de verdad a varios clientes a la vez.

Vocabulario rápido

Término	Idea general
Benchmark	Prueba que mide el rendimiento del servidor
Lanzador masivo	Script que crea N clientes simultáneos
<code>join()</code>	Espera a que terminen todos los hilos
<code>ceil(n/workers)</code>	Número de tandas del pool (redondeo hacia arriba)

Término	Idea general
time.time()	Cronómetro para medir duraciones

 [Volver al índice de la unidad](#) · Anterior: [04 · ThreadPoolExecutor](#) · Siguiente: [06 · Sincronización en servidores](#)

06 — Sincronización en servidores

El Lock para el estado compartido entre hilos 🔒

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 06 · Sincronización en servidores

📌 La idea en una frase

Cuando varios hilos del servidor tocan la **misma variable global** (un contador de conexiones, un total de bytes), hay **condición de carrera**: dos hilos pueden leer y escribir a la vez y perder actualizaciones. La solución es el **Lock** de la [U03](#).

En el [punto 3](#) y el [punto 4](#) los hilos eran independientes. Pero un servidor real suele llevar **estado**: “¿cuántos clientes he atendido?”, “¿cuántos bytes he recibido?”. Ese estado es compartido... y eso es un problema.

💥 La condición de carrera en el servidor

Imagina un contador global de conexiones atendidas, sin protección:

```
import socket, threading

contador = 0 # ← variable global compartida

def atender(conn, addr):
    global contador
    contador += 1          # ⚠️ ¡no es atómico!
    print(f"Total atendidos: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
```

El problema es que `contador += 1` se ejecuta en **varios pasos**: leer el valor → sumar 1 → guardarlo. Dos hilos pueden leer el mismo valor (p. ej. 7) a la vez y los dos guardar 8: **una conexión se pierde**.

```
Hilo-A: lee contador (=7)
Hilo-B: lee contador (=7) ← ¡mismo valor!
Hilo-A: guarda 8
Hilo-B: guarda 8          ← contador debía ser 9, quedó en 8 ✨
```

⚠️ **Regla de oro:** si dos hilos del servidor modifican la misma variable, usa **Lock**. Mejor aún: **evita compartir estado** cuando puedas.

🔒 El Lock en acción

Con un `threading.Lock`, la sección crítica queda protegida:

```
import socket, threading

contador = 0
lock = threading.Lock()

def atender(conn, addr):
    global contador
    with lock:                # 🔒 nadie más puede entrar aquí
        contador += 1
        print(f"Total atendidos: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")

def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=atender, args=(conn, addr)).start()
```

`with lock:` adquiere el lock al entrar y lo libera al salir (aunque haya una excepción). Entre esas líneas, **un solo hilo a la vez**: los demás esperan su turno. La lectura, suma y guardado del contador ya son indivisibles.



El `with lock`: del servidor es exactamente el patrón que ya conoces de la [U03](#). [Sincronización entre hilos](#): misma herramienta, distinto escenario.



¿Y el contador de bytes?

El mismo patrón sirve para acumular cualquier valor global. Un contador de bytes totales recibidos:

```
import socket, threading

total_bytes = 0
lock = threading.Lock()

def atender(conn, addr):
    global total_bytes
    with conn:
        datos = conn.recv(1024)
    with lock:
        total_bytes += len(datos)          # 🗝️ actualización atómica
        print(f" {addr} envió {len(datos)} bytes - Total: {total_bytes}")
```

Nota: el `recv()` y el `sendall()` **no** necesitan lock (cada conexión es un socket distinto); solo lo necesita la **variable compartida**. Proteger de más (meter el `recv` dentro del lock) volvería el servidor secuencial sin querer.



Mini-chequeo

1. ¿Por qué contador `+= 1` no es una operación segura entre hilos?
2. ¿Qué pasa si dos hilos incrementan el contador a la vez sin Lock?
3. En el servidor, ¿qué necesita Lock: el `recv()` del socket o la variable global?



 Respuestas



Resumen en 3 frases

- Los hilos del servidor compiten por las **variables globales** (contadores, totales) y eso causa condiciones de carrera.
- El **Lock** (with lock:) hace indivisible la actualización: un hilo a la vez en la sección crítica.
- Regla práctica: protege solo lo compartido; lo que es por conexión (recv/sendall) no necesita lock.



Vocabulario rápido

Término	Idea general
Condición de carrera	Dos hilos compiten por la misma variable y se pierden datos
Sección crítica	Código que toca estado compartido y necesita Lock
threading.Lock()	Cerradura para exclusión mutua entre hilos
with lock:	Adquiere el lock al entrar y lo libera al salir
Estado compartido	Variables globales que varios hilos leen/escriben

 [Volver al índice de la unidad](#) · **Anterior:** [05 · Benchmark](#) · **Siguiente:** [07 · Límites y buenas prácticas](#)

07 — Límites y buenas prácticas

Cuántos hilos puedes crear y cómo hacerlo bien 🧠

📖 Estás en: 🏠 U10 · Servidores Concurrentes → 07 · Límites y buenas prácticas

La idea en una frase

Más hilos no es siempre mejor: el sistema operativo tiene límites, el cambio de contexto tiene coste y un hilo colgado puede fugarse de recursos. La buena práctica es **controlar los límites** y elegir la herramienta adecuada para cada escala.

Ya tienes las dos herramientas (hilo por cliente y ThreadPool). Este punto responde a las preguntas que te harás cuando el servidor vaya a producción: ¿cuántos hilos puedo crear? ¿qué pasa si uno se cuelga? ¿y si llegan 10.000 conexiones?

¿Cuántos hilos puede tener un servidor?

Depende del SO. En Windows, unos pocos miles. Pero hay un matiz importante:

Más allá de **~100 hilos**, el **cambio de contexto** (context switch) perjudica el rendimiento.

Cada vez que la CPU alterna entre hilos, pierde tiempo en guardar y restaurar el estado de cada uno. Con pocos hilos, ese coste es despreciable; con cientos, la CPU pasa más tiempo *cambiando de hilo* que *trabajando*. Por eso el ThreadPool del [punto 4](#) limita max_workers: el rendimiento sube con los hilos... hasta un punto, y luego baja.

No de hilos	Efecto esperado
1-10	Sencillo, bajo consumo, poco paralelismo
~100	Punto de equilibrio razonable en un servidor de clase
Miles	Riesgo de agotar memoria y coste de context switch alto

¿Qué pasa si un hilo se cuelga?

Ese hilo queda **bloqueado**, pero los demás siguen funcionando: la concurrencia te salva de un colapso total. El problema real es si el hilo colgado **no libera el socket** (fuga de recursos):

- Cada conexión abierta ocupa un **file descriptor** y memoria en el SO.
- Con suficientes fugas, el servidor agota los descriptors y **deja de aceptar** nuevas conexiones.
- La solución es garantizar el cierre: el `with conn:` que usamos en toda la unidad cierra el socket al salir del bloque, **aunque haya una excepción**.

🔵 Si un hilo se queda eternamente esperando datos que nunca llegan, un **timeout** (`conn.settimeout()`) lo libera a los pocos segundos. Lo practicarás en el boletín avanzado.

🏭 ¿ThreadPool o hilo por cliente en producción?

ThreadPool siempre. Controlas cuántos hilos se crean y evitas saturar el SO. Hilo por cliente es solo para prototipos:

Criterio	Hilo por cliente	ThreadPool
Control de recursos	❌ Ninguno	✅ <code>max_workers</code> fijo
Escala a miles de conexiones	❌ Satura el sistema	✅ Encola y sirve de a N
Complejidad	✅ Simple	✅ Igual de simple (<code>submit()</code>)
Uso recomendado	Prototipos, clases	Producción

🌪️ ¿Y si el servidor recibe 10.000 conexiones?

- Con hilo por cliente: 10.000 hilos → el sistema se colapsa.
- Con ThreadPool de **100 hilos + cola de espera**: el servidor sobrevive, aunque los clientes esperen turno.
- La alternativa que **escala mejor** es **asyncio** (la verás en la [U11 · asyncio y Disponibilidad](#)): atiende miles de conexiones sin un hilo por conexión, con un bucle de eventos muy ligero.

🔄 ¿Puedo mezclar hilos y asyncio?

Sí, con `loop.run_in_executor()`. Pero como principiante, la recomendación es clara: **empieza con uno u otro**. Los dos modelos (hilos y asyncio) son potentes por separado y confusos juntos. Cuando domines asyncio en la [U11](#), podrás combinar bloqueos de bibliotecas de terceros (que no son asíncronas) usando el ejecutor.

✓ Guía rápida de buenas prácticas

- **Limita los hilos:** usa `ThreadPoolExecutor(max_workers=N)` en producción, nunca hilos ilimitados.
 - **Cierra siempre la conexión:** `with conn:` libera el socket aunque haya errores.
 - **Protege el estado compartido** con `Lock` (punto 6); mejor aún, evita compartir estado.
 - **Usa timeouts** en `recv()` para que un cliente mudo no cuelgue un hilo para siempre.
 - **Mide con el benchmark** (punto 5) antes de tocar `max_workers`: los datos mandan, no las corazonadas.
 - **El pool se crea una vez**, fuera del bucle `accept()` (punto 4).
-

Mini-chequeo

1. ¿Por qué “miles de hilos” puede ser *peor* que “100 hilos” aunque la CPU tenga muchos núcleos?
2. Un hilo se cuelga sin liberar su socket. ¿Qué les pasa a los demás? ¿Y al servidor a largo plazo?
3. Recibes 10.000 conexiones simultáneas. ¿Qué herramienta usarías y por qué?

>  Respuestas

✓ Resumen en 3 frases

- Los hilos tienen **límites del SO** y el **context switch** penaliza por encima de ~100.
- Un hilo colgado no tumba al servidor, pero **fugarse de sockets** sí lo ahoga con el tiempo.
- Regla de producción: **ThreadPool con límite + timeouts + with conn: + benchmark.**

Vocabulario rápido

Término	Idea general
Context switch	Cambio de hilo de la CPU: necesario, pero caro
Fuga de recursos	Socket o memoria que nunca se libera
setimeout()	Límite de tiempo para <code>recv()</code> : evita hilos colgados
max_workers	Límite de hilos del pool (tu freno de mano)
asyncio	Modelo de concurrencia ligera para miles de conexiones (U11)

 [Volver al índice de la unidad](#) · Anterior: [06 · Sincronización en servidores](#) · Siguiente: [08 · Servidor concurrente completo](#)

08 — Servidor concurrente completo

Todo junto: servidor multihilo y su lanzador de clientes 

 Estás en:  U10 · Servidores Concurrentes → 08 · Servidor concurrente completo

La idea en una frase

Este punto junta toda la unidad en dos scripts listos para ejecutar: un **servidor concurrente** (`servidor.py`) que atiende a cada cliente en su hilo con un contador protegido por Lock, y un **lanzador de clientes** (`lanzaclientes.py`) que lo pone a prueba con N conexiones a la vez.

Es el “Be the code” definitivo: aquí tienes el código completo, cómo ejecutarlo en dos terminales y los ejercicios de “Aprieta el lápiz” para cerrar la teoría.

El servidor: `servidor.py`

Un servidor TCP multihilo con contador de conexiones protegido por Lock (el punto [3](#) + el punto [6](#)):

```

import socket, threading

contador = 0
lock = threading.Lock()

def atender(conn, addr):
    global contador
    with lock:
        contador += 1
        print(f"[+] Cliente {addr} conectado - total: {contador}")
    with conn:
        datos = conn.recv(1024)
        print(f"    Recibido: {datos.decode()}")
        conn.sendall(b"OK: " + datos)
    with lock:
        contador -= 1
        print(f"[-] Cliente {addr} desconectado - total: {contador}")

def servidor_concurrente():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("👂 Servidor CONCURRENTE escuchando en 127.0.0.1:5000")
        while True:
            conn, addr = srv.accept()
            hilo = threading.Thread(target=atender, args=(conn, addr))
            hilo.name = "hilo-" + str(contador)
            hilo.start()

if __name__ == "__main__":
    servidor_concurrente()

```

Lo que demuestra este servidor:

- **Concurrencia:** `accept()` → crear hilo → `start()` → volver a aceptar (punto 3).
- **Sincronización:** el contador global solo se toca dentro de `with lock:` (punto 6).
- **Limpieza:** `with conn:` cierra el socket sí o sí (punto 7).
- **Convención:** punto de entrada con `if __name__ == "__main__":` y hilos nombrados con `.name`.



El lanzador: lanzaclientes.py

Un script que lanza 10 clientes simultáneos contra el servidor (el lanzador masivo del [punto 5](#)):

```

import socket, threading, time

def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"    ✅ Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"    ❌ Cliente-{id}: {e}")

print("Lanzando 10 clientes...")
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print("🏁 Todos los clientes terminaron")

```



Para convertirlo en benchmark, envuelve el bloque con `inicio = time.time()` al principio y `print(f"Tiempo total: {time.time() - inicio:.2f}s")` al final.



Cómo probarlos juntos

TERMINAL 1 (servidor)

`python servidor.py`



Servidor CONCURRENTE...

[+] Cliente ('127.0.0.1', ...)

[+] Cliente ('127.0.0.1', ...)

Recibido: Cliente-0

Recibido: Cliente-1

TERMINAL 2 (lanzador)

`python lanzaclientes.py`

Lanzando 10 clientes...



Cliente-0: OK: Cliente-0



Cliente-1: OK: Cliente-1

... (todos responden)



Todos los clientes terminaron

Si el servidor fuera secuencial, el lanzador tardaría $10 \times \text{tiempo_por_cliente}$ y las respuestas llegarían de una en una. Con el servidor concurrente, **todas responden casi a la vez**: ese es el resultado del punto 5. Se mata el servidor con **Ctrl+C** y cada script se ejecuta en su terminal.



Aprieta el lápiz

1. **Servidor secuencial → hilos**: Convierte un servidor TCP secuencial en uno multihilo.
2. **ThreadPool**: Implementa el mismo servidor con `ThreadPoolExecutor` de 5 hilos.
3. **Lanzador de clientes**: Crea un script que lance 10 clientes simultáneos para probar tu servidor.
4. **Contador de conexiones**: El servidor debe mostrar cuántos clientes han sido atendidos (usa `Lock` para la variable compartida).

>  Soluciones



Mini-chequeo

1. ¿Por qué el contador se incrementa y decrementa dentro de `with lock:`?
2. ¿Qué pasa en el lanzador si el servidor no está arrancado?
3. ¿Dónde se nombra el hilo y con qué valor?

>  Respuestas



Resumen en 3 frases

- `servidor.py` combina **hilo por cliente + Lock + with conn**: en un servidor de producción sencillo.
- `lanzaclientes.py` prueba que atiende a **N clientes a la vez** y sirve de benchmark.
- Los ejercicios “Aprieta el lápiz” cubren las dos variantes (hilos y pool) y el contador sincronizado.



Vocabulario rápido

Término	Idea general
servidor.py	El servidor concurrente completo de la unidad
lanzaclientes.py	El script que dispara N clientes simultáneos
hilo.name	Nombre del hilo para trazarlo (“hilo-1”, “hilo-2”...)
if name	Punto de entrada del script
Ctrl+C	Cómo se mata el servidor en pruebas

 [Volver al índice de la unidad](#) · **Anterior:** [07 · Límites y buenas prácticas](#) · **Siguiente:** [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé el servidor, el ring final y el Laboratorio de tortura 🕒

📖 Estás en: 🕒 U10 · Servidores Concurrentes → 09 · Cierre

Has terminado la teoría: el servidor secuencial y su límite, el cliente lento que bloquea la cola, el hilo por cliente, el `ThreadPoolExecutor`, el benchmark, el `Lock` para el estado compartido y las buenas prácticas. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé el servidor

Eres un servidor TCP en 127.0.0.1:5000 con un `ThreadPoolExecutor` de 5 hilos. De repente llegan 10 clientes a la vez.

¿Qué pasa?

1. Tu bucle principal hace `accept()` y recibe la **conexión-1**: `pool.submit(atender, conn1, addr1)`.
2. La tarea entra en el pool: **hilo-1 libre** la coge al instante y empieza a procesar.
3. Vuelves a `accept()` y recibes la **conexión-2**, **conexión-3**, ... hasta la **conexión-5**: cada una ocupa un hilo libre (hilo-1 a hilo-5).
4. Llega la **conexión-6**: los 5 hilos están ocupados, así que `submit()` la **encola** internamente.
5. En cuanto hilo-1 termina con la conexión-1 (responde y cierra con `with conn:`), el pool **reutiliza** ese hilo para la conexión-6.
6. Así hasta la conexión-10: **dos tandas de 5**, el `ceil(10/5)` del [punto 5](#).

Nunca se crea un hilo nuevo. El servidor tiene exactamente 5 hilos y una cola: el sistema no se ahoga.

💡 **Ahora tú:** ¿y si llegaran 10.000 conexiones? El pool las iría encolando y sirviendo de a 5. Tardarían más, pero el servidor **sobreviviría**. Y para 10.000 conexiones, la solución de la [U11](#) · [asyncio](#) sería todavía más ligera.



Fireside Chat: Hilo por cliente vs ThreadPool

Dos enfoques de concurrencia se sientan junto a la chimenea a resolver, de una vez, quién sirve mejor a los clientes.

HiloPorCliente: — Soy el enfoque clásico. Llegas un cliente, creo un hilo, lo atiende, el hilo muere. Sencillo y directo.

ThreadPool: — ¿Y si llegan 1000 clientes? Creas 1000 hilos y el sistema se colapsa. Yo tengo un número fijo de hilos, como un equipo de camareros limitado.

HiloPorCliente: — Pero cada cliente tiene su hilo dedicado. No esperan. Es más justo.

ThreadPool: — Justo pero ineficiente. Con mi pool, los clientes esperan un poco pero el sistema no se ahoga. Además, reutilizo hilos, evitando el coste de crearlos y destruirlos constantemente.

HiloPorCliente: — Para pocos clientes simultáneos, soy más simple de implementar.

ThreadPool: — Y yo escalo mejor. Para servidores en producción, soy la opción sensata.

Moraleja: Hilo por cliente es simple y funciona para pocos clientes. ThreadPool escala mejor y controla los recursos. En producción, usa ThreadPool.



¿Quién soy?

1. Soy el `accept()` que se queda atascado mientras se procesa al cliente anterior.
2. Soy la llamada que entrega la tarea al pool y devuelve un `Future`.
3. Soy la analogía del servidor secuencial: una sola persona atiende a toda la cola.
4. Soy la variable global que dos hilos pueden pisar si no me protegen con `Lock`.
5. Soy lo que mide cuánto tarda en responder un servidor con N clientes simultáneos.
6. Soy la fórmula del pool: te digo en cuántas tandas se atienden N clientes con W hilos.

>  Respuestas



CONRAD VS EL MUNDO: “un cliente lento bloquea el servidor”

CONRAD: — “Clásico: montas un servidor secuencial, llega un cliente que tarda 30 segundos y *‘el servidor se ha quedado colgado’*. Pues no: solo tiene **una ventanilla**. Mientras procesa al lento, el `accept()` no se ejecuta y toda la cola espera. Los clientes 2 al 10 ya están conectados al socket... pero nadie los atiende.”

CONRAD: — “Y lo mejor: *‘he puesto `time.sleep(3)` y las pruebas van lentas’*. ¡Pues claro! Cada cliente suma 3 segundos. Con 10 clientes, el último espera 30. $n \times \text{tiempo_por_cliente}$, lo dijimos en el [punto 1](#). Si quieres que todos terminen a la vez, necesitas hilos o un pool.”

CONRAD: — “Y no me vengas con *‘¿será que el puerto está ocupado?’*. El puerto va bien: lo que pasa es que el servidor atiende de uno en uno. Lanza el **lanzador de clientes** del [punto 5](#) y míralo en el log: si las respuestas llegan de una en una, tienes un secuencial. A diagnosticar.”

Laboratorio de tortura: servidor concurrente con ThreadPool

Duración: 45 minutos **Herramienta:** Python 3 (solo stdlib: socket, threading, concurrent.futures)

Escenario: construye un servidor TCP en `127.0.0.1:5000` que responda `"OK: " + datos` a cada cliente, usando **ThreadPoolExecutor** con `max_workers=3`. Acompáñalo de un lanzador que dispare 10 clientes simultáneos y cronometre el total.

Tareas paso a paso:

1. Escribe `atender(conn, addr)` con `with conn::` recibe con `recv(1024)` y responde con `sendall(b"OK: " + datos)`.
2. Monta el servidor con `with socket.socket() as srv,`
`ThreadPoolExecutor(max_workers=3) as pool:` (patrón del [punto 4](#)).
3. En el bucle: `conn, addr = srv.accept() → pool.submit(atender, conn, addr)`.
4. Escribe el lanzador de 10 clientes con `threading.Thread` y mide el tiempo total con `time.time()` ([punto 5](#)).
5. Añade un **contador global de conexiones activas** protegido con `Lock` ([punto 6](#)) que se imprima con cada `[+]/[-]`.
6. Ejecuta servidor y lanzador en terminales separadas. El servidor se mata con **Ctrl+C**.

Fallo intencionado: cambia `max_workers=3` por `max_workers=1`. ¿Qué pasa? El pool se convierte en un servidor **secuencial de facto**: una sola tarea a la vez, las otras 9 en cola. El tiempo total

pasa de ~1 tanda a **10 tandas**: con clientes que tardan 2s, el lanzador marca ~20s en lugar de ~8s ($\text{ceil}(10/3)=4 \text{ tandas} \times 2\text{s}$).

Pista 1: el problema no está en el código del cliente ni del servidor: está en el **tamaño del pool**. Con `max_workers=1`, el `ThreadPool` se comporta como el servidor secuencial del [punto 1](#): el tiempo total es $n \times \text{tiempo_por_cliente}$.

Pista 2: usa la fórmula del [punto 5](#): $\text{ceil}(n/\text{workers}) \times \text{tiempo_por_cliente}$. Con `workers=1`, $\text{ceil}(10/1) = 10$ tandas. Con `workers=3`, $\text{ceil}(10/3) = 4$. La diferencia de tiempo te delata la configuración.

Logros de esta unidad

Logro	Cómo conseguirlo
 Cola Crusher	Explicar por qué un cliente lento bloquea a todo un servidor secuencial
 Thread Spawner	Montar un servidor multihilo que lanza un hilo por cada cliente
 Pool Master	Implementar un servidor con <code>ThreadPoolExecutor</code> y elegir el tamaño del pool
 Benchmarker	Medir con un lanzador masivo la diferencia entre secuencial, hilos y pool
 Lock Guardian	Proteger con <code>Lock</code> el contador de conexiones de un servidor concurrente

Atrévete a pensar

1. ¿Por qué un servidor secuencial “acepta” a los clientes pero no los atiende?
2. ¿Qué ventaja real tiene un pool de 5 hilos frente a 5 hilos creados y destruidos por cliente?
3. ¿Cuándo vale la pena sacrificar velocidad (esperar en cola del pool) por estabilidad?
4. ¿Por qué el `recv()/sendall()` de cada conexión no necesita `Lock`, pero el contador global sí?

5. ¿Qué problemas aparecerían si lanzaras 100.000 hilos contra tu servidor?

> 💡 Soluciones

🧩 Crucigrama de bits

Horizontal:

1. Atiende a un cliente cada vez, el enemigo de la concurrencia (10 letras)
4. Hilos por conexión, el patrón del punto 3 (4 letras)
6. Equipo fijo de hilos reutilizables (4 letras)
8. Lo que hace el bucle principal tras lanzar un hilo (6 letras)

Vertical:

2. Protege la variable compartida entre hilos (4 letras)
3. Prueba que lanza N clientes y mide el tiempo (9 letras)
5. Método que entrega una tarea al pool (6 letras)
7. Se libera con with conn: aunque haya excepción (6 letras)

> 📝 Soluciones

💬 Entrevista de trabajo

1. “¿Qué es un servidor concurrente y en qué se diferencia de uno secuencial?”
2. “¿Cómo implementarías un servidor que atienda a 500 clientes a la vez?”
3. “Hilo por cliente o ThreadPoolExecutor: ¿cuál elegirías y por qué?”
4. “¿Qué es una condición de carrera? Pon un ejemplo en un servidor.”
5. “¿Cómo medirías si tu servidor es realmente concurrente?”

💡 **Cómo encararlas:** la 1 y la 2 son las “preguntas reina”. Para la 1, recorre la cadena de la unidad: secuencial → $n \times \text{tiempo}$ → cliente lento bloquea → hilo por cliente → pool. Para la 2, plantea ThreadPoolExecutor con un max_workers razonable, submit() en el bucle de accept(), with conn: y un contador con Lock. Si sabes contarlo fluido y defiendes por qué **no** lanzarías 500 hilos, ya eres medio programador de servidores.

🙋 No hay preguntas tontas

? ¿Cuántos hilos puede tener un servidor?

Depende del SO. En Windows, unos pocos miles. Pero más allá de ~100, el cambio de contexto (context switch) perjudica el rendimiento ([punto 7](#)).

? ¿Qué pasa si un hilo se cuelga?

Ese hilo queda bloqueado, pero los demás siguen. El problema es si no libera el socket (fuga de recursos).

? ¿ThreadPool o hilo por cliente en producción?

ThreadPool siempre. Controlas cuántos hilos se crean. Hilo por cliente solo para prototipos.

? ¿Es seguro compartir variables globales entre hilos del servidor?

No sin Lock. Si dos hilos modifican la misma variable, usa Lock. Mejor aún: evita compartir estado.

? ¿Puedo mezclar hilos y asyncio?

Sí, con `loop.run_in_executor()`. Pero empieza con uno u otro.

? ¿Y si el servidor recibe 10.000 conexiones?

ThreadPool con 100 hilos + cola de espera. O usa **asyncio** ([U11](#)) que escala mejor.

Poscréditos

El servidor secuencial hacía esperar a todos en la cola de la ventanilla única.

Llegó el hilo por cliente y cada conexión tuvo su propia ventanilla.

El ThreadPool puso límites al entusiasmo: cinco camareros, cola ordenada, nadie se ahoga.

El Lock protegió el contador. El benchmark demostró la mejora. Los clientes responden a la vez.

El servidor ya atiende a muchos. Pero 10.000 conexiones siguen esperando algo más ligero...

PRÓXIMAMENTE EN U11: *asyncio y disponibilidad. Sin un hilo por conexión, un solo bucle de eventos atenderá miles de clientes a la vez. El servidor concurrente se vuelve esbelto.*



Criterios de evaluación cubiertos (RA4c-d)

RA4: Implementa servicios en red, desarrollando servidores concurrentes capaces de atender a varios clientes simultáneamente.

CE	Criterio	Cubierto
c)	Implementa servidores concurrentes con hilos	✓ Hilo por cliente (puntos 3 y 8) + ⚡ Laboratorio
d)	Gestiona pools de hilos (ThreadPoolExecutor)	✓ ThreadPool y benchmark (puntos 4-5) + ⚡ Laboratorio

RA4a-b (APIs REST y comerciales) se cubren en las **U06 y U07**. RA4e-g (asyncio, disponibilidad, comparativa de modelos) se cubren en la **U11 · asyncio y Disponibilidad**.

 [Volver al índice de la unidad](#) · Anterior: [08 · Servidor concurrente completo](#) · Siguiente: [U11 · asyncio y Disponibilidad](#)

U11 — asyncio y Disponibilidad

Cerrar el viaje: asincronía, latidos y servicios siempre disponibles 🕒

📖 **Ruta del viaje:** 🚀 Proceso → 🔄 Hilo → 🔒 Sincronización → 🌐 TCP → 📡 UDP → 🌐 API REST → 🧪 APIs comerciales → 🔒 Hash → 🧬 Cifrado → ⏳ Servidores → 🕒 **asyncio y Disponibilidad**

“Asyncio es como un cocinero que, mientras espera a que hierva el agua, corta verduras. En vez de quedarse mirando la olla, hace otras cosas.”

En la U10 construiste servidores concurrentes con hilos: un hilo por cliente o un ThreadPool. Funcionan, pero cada hilo cuesta memoria y contexto de sistema. En esta unidad cambiarás de modelo: **asyncio**, un solo hilo que coordina miles de tareas cooperativas, y las herramientas de **disponibilidad** (heartbeat, timeouts y backoff) que mantienen un servicio vivo cuando todo falla alrededor.

También aprenderás a que un servidor **nunca se cuelgue** con un cliente mudo (timeout), a que **avise** de que sigue vivo (heartbeat) y a que **reintente** con cabeza cuando un servicio se cae (backoff). El siguiente tema ya no existe: esta es la última unidad del viaje, y cierras con un servidor asyncio robusto que no se bloquea, se vigila solo y se recupera.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es el **event loop** y cómo coordina tareas asíncronas sin bloquear.
- Definir y usar **corrutinas** con `async def` y `await`.
- Lanzar tareas concurrentes con `asyncio.create_task` y `asyncio.gather`.
- Evitar que un servicio se cuelgue usando **timeouts** con `asyncio.wait_for`.
- Implementar **heartbeats** que verifiquen que un servidor sigue vivo.
- Diseñar **reintentos con backoff exponencial** para conectar ante fallos transitorios.
- Comparar **hilos vs asyncio** y decidir cuál usar en cada escenario.
- Montar un **monitor de servicio** que combine heartbeat, timeout y backoff.

- Aplicar los mecanismos de disponibilidad (RA4e), los servidores asincio (RA4f) y la comparativa de modelos (RA4g).

Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Event Loop	El gestor de tareas que no se bloquea nunca	Todos
02 · Corrutinas	async def, await y la función que sabe esperar	Todos
03 · create_task y gather	Lanzar varias tareas “a la vez” de verdad	Todos
04 · Timeouts	wait_for para que nada se cuelgue para siempre	Todos
05 · Heartbeat	El latido que confirma que el servicio sigue vivo	Todos
06 · Backoff	Reintentos con espera exponencial (1, 2, 4, 8...)	Todos
07 · Threads vs asyncio	La comparativa definitiva de modelos de concurrencia	Todos
08 · Disponibilidad y práctica	El monitor de servicio completo y Aprieta el lápiz	Todos
09 · Cierre	Sé la corrutina, Fireside, Laboratorio de tortura...	Todos

 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

Boletines de la unidad

Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

 Inicial resuelto

 Inicial por resolver

 Avanzado resuelto

 Avanzado por resolver

Criterios de evaluación cubiertos (RA4e-g)

RA4: Implementa servicios en red, desarrollando mecanismos de disponibilidad y servidores asíncronos.

CE	Criterio	Dónde se cubre
e)	Implementa mecanismos de disponibilidad (heartbeat, reintentos, timeout)	✓ Puntos 4-6 y 8 + ⚡ Laboratorio (punto 9)
f)	Desarrolla servidores con asyncio	✓ Puntos 1-3 y 8 + ⚡ Laboratorio (punto 9)
g)	Compara modelos de concurrencia (hilos vs asyncio)	✓ Punto 7 + 🔥 Fireside Chat (punto 9)

RA4c (servidores concurrentes con hilos) y RA4d (ThreadPool) se cubren en la **U10 · Servidores Concurrentes**.

¿Por dónde empiezo?

¿Vienes de la U10 y dominas los servidores concurrentes? Perfecto, ese es el trampolín ideal: repasa la [U10 · Servidores Concurrentes](#) para tener frescos el hilo por cliente, el ThreadPool y el problema del bloqueo, y arranca en el [punto 1](#), que parte justo del problema de la espera que dejaste planteado.

¿Ya sabes qué es asyncio y solo quieres la disponibilidad? Ve directo al [punto 4](#) y de ahí a los puntos 5 y 6. Pero si vienes de cero, no te saltes los puntos 1 a 3: el event loop y las corrutinas son la base de todo lo demás.

📌 Primer punto: [01 · Event Loop](#)

01 — Event Loop

El gestor de tareas que no se bloquea nunca ⚙️

📖 Estás en: ⌚ U11 · asyncio y Disponibilidad → 01 · Event Loop

La idea en una frase

El **event loop** es el gestor de asyncio: un bucle que decide qué tarea se ejecuta en cada momento. Cuando una corrutina espera, la pausa y ejecuta otra. Un solo hilo, miles de tareas, ninguna espera activa.

Viene directo del problema que dejaste en la [U10 · Servidores Concurrentes](#): un servidor que usa `accept()` y `recv()` se **bloquea** mientras espera. La solución de la U10 fueron los hilos. La de hoy es distinta: **un solo hilo que cambia de tarea cuando una espera.**

El problema de esperar

```
# ❌ Esto bloquea TODO el programa
datos = conn.recv(1024) # El programa se para aquí hasta que lleguen datos
```

Si el código se queda en esa línea, no puede atender a nadie más. El bloqueo es el enemigo. Y las respuestas posibles son dos:

Solución 1: Hilos (TEMA 10) — caros si hay muchos clientes: cada hilo consume memoria y contexto del sistema operativo.

Solución 2: Asyncio — un solo hilo, pero cambia de tarea cuando una espera. Ningún `recv()` bloqueante, ningún `sleep()` que pare todo.

La analogía de la cafetería

Imagina a un único camarero en una cafetería con 10 mesas:

Modelo	El camarero	Problema
Secuencial	Atiende la mesa 1 hasta el final, luego la 2...	Las mesas 2-10 esperan
Hilos	Contrata 10 camareros, uno por mesa	Caro con 10.000 mesas
Event loop	Anota el pedido de la 1, pasa a la 2 mientras la cafetera trabaja...	Ninguno: nadie espera de brazos cruzados

El camarero del event loop **nunca se queda mirando la cafetera**: cada espera es una oportunidad para atender a otra mesa. Eso es lo que hace asyncio: “esperar” nunca significa quedarse quieto.

Qué es el event loop, de verdad

El **event loop** es un bucle que mantiene una lista de tareas listas para ejecutarse. En cada vuelta:

1. Ejecuta la tarea que toca, hasta que encuentra un `await` (una espera).
2. Cuando la tarea espera, la aparca y sigue con la siguiente.
3. Cuando la espera termina (llegan datos, pasa el tiempo), la tarea vuelve a la cola de listas.

```
import asyncio

async def tarea(nombre, segundos):
    print(f" {nombre} empieza")
    await asyncio.sleep(segundos)
    print(f" {nombre} termina ({segundos}s)")

async def main():
    # Lanzar 3 tareas "a la vez"
    await asyncio.gather(
        tarea("A", 3),
        tarea("B", 1),
        tarea("C", 2)
    )

asyncio.run(main())
```

Salida:

```
A empieza
B empieza
C empieza
B termina (1s)
C termina (2s)
A termina (3s)
```

Las 3 empiezan a la vez. B termina primero (solo 1s). El event loop aprovecha los `await` de otras para avanzar.

El event loop también gestiona timers, sockets, procesos y cualquier I/O: cada `await` le dice “esto va a tardar, ocúpate de otras cosas”.



Mini-chequeo

1. ¿Por qué `conn.recv(1024)` bloquea todo el programa en un servidor síncrono?
2. ¿Cuántos hilos usa `asyncio`?
3. En la analogía de la cafetería, ¿qué representa la cafetera que trabaja sola?

>  Respuestas



Resumen en 3 frases

- `recv()`, `accept()` y `sleep()` síncronos **bloquean** el hilo y paralizan todo lo demás.
- El **event loop** es un solo hilo que coordina miles de tareas, pausando las que esperan y ejecutando las que están listas.
- Cada `await` es una oportunidad para atender a otra tarea: nadie se queda mirando la olla.



Vocabulario rápido

Término	Idea general
Event loop	El gestor que decide qué tarea asíncrona se ejecuta

Término	Idea general
Bloqueo	Cuando el hilo espera sin hacer nada (recv, sleep)
Corrutina	Función asíncrona que puede pausarse (punto 2)
asyncio.run()	Arranca el event loop y ejecuta la corrutina principal
await	La marca de “esto va a tardar, ocúpate de otras cosas”

 [Volver al índice de la unidad](#) · **Siguiente:** [02 · Corrutinas](#)

02 — Corrutinas

async def, await y la función que sabe esperar 

 Estás en:  U11 · asyncio y Disponibilidad → 02 · Corrutinas

La idea en una frase

Una **corrutina** es una función declarada con `async def` que puede **pausarse** con `await` y reanudarse después: no es un hilo, es una función que sabe esperar.

`async def` y `await`

La pieza básica de `asyncio`:

```
import asyncio

async def saludar():
    print("Hola")
    await asyncio.sleep(1) # "Oye, mientras duermo, haz otras cosas"
    print("Mundo")

asyncio.run(saludar())
```

Concepto	Qué es
<code>async def</code>	Define una corrutina (función que puede pausarse)
<code>await</code>	Pausa la corrutina hasta que algo termine
<code>asyncio.run()</code>	Crea el event loop y ejecuta la corrutina principal

`await` = "Oye, esto va a tardar. Mientras, ocúpate de otras cosas."

Cuando se encuentra `await asyncio.sleep(1)`, la corrutina `saludar()` se pausa y devuelve el control al event loop. Un segundo después, el event loop la reanuda en la siguiente línea (`print("Mundo")`). Ese ir-y-volver es lo que hace que un solo hilo sirva a miles de corrutinas.



Corrutina no es una función normal

La diferencia clave con una función normal:

```
import asyncio

async def tarea():
    return 42

print(tarea())          # ❌ No imprime 42: imprime <coroutine object ...>
```

Llamar a una corrutina **no ejecuta su cuerpo**: crea un objeto corrutina. Solo se ejecuta cuando el event loop la programa, de una de estas formas:

```
asyncio.run(tarea())    # Desde fuera: arranca el event loop

async def main():
    resultado = await tarea() # Desde dentro: espera y obtiene el resultado
```

Forma	Cuándo usarla
<code>asyncio.run(c)</code>	Solo para la corrutina principal, una vez por programa
<code>await c()</code>	Dentro de otra corrutina: espera su resultado
<code>asyncio.gather(c1, c2)</code>	Lanza varias a la vez (punto 3)
<code>asyncio.create_task(c)</code>	Lanza en segundo plano (punto 3)



Mini-chequeo

1. ¿Qué diferencia hay entre `async def f()` y `def f()`?
2. ¿Qué hace `await` con el hilo?
3. Si llamas a `tarea()` (sin `await` ni `run`), ¿qué obtienes?

>  Respuestas



Resumen en 3 frases

- Una corrutina es una **función que puede pausarse** (`async def + await`).
- `await` no bloquea el hilo: cede el control al event loop.
- Para ejecutar una corrutina hace falta `asyncio.run()` o programarla dentro de otra corrutina.



Vocabulario rápido

Término	Idea general
Corrutina	Función <code>async def</code> que puede pausarse y reanudarse
<code>await</code>	Pausa la corrutina y devuelve el control al event loop
<code>asyncio.run()</code>	Arranca el event loop y ejecuta la corrutina principal
Objeto corrutina	El resultado de llamar a una <code>async def</code> sin ejecutarla
Cooperativo	Las tareas se ceden el control voluntariamente en cada <code>await</code>

 [Volver al índice de la unidad](#) · Anterior: [01 · Event Loop](#) · Siguiente: [03 · create_task y gather](#)

03 — create_task y gather

Lanzar varias tareas "a la vez" de verdad 🚀

📖 Estás en: ⌚ U11 · asyncio y Disponibilidad → 03 · create_task y gather

La idea en una frase

asyncio.gather lanza varias corrutinas a la vez y espera a todas; asyncio.create_task lanza una corrutina **en segundo plano** para que la principal siga su camino. Así se consigue la **conurrencia real** de asyncio.

asyncio.gather: todas a la vez

Del [punto 1](#):

```
import asyncio

async def tarea(nombre, segundos):
    print(f" {nombre} empieza")
    await asyncio.sleep(segundos)
    print(f" {nombre} termina ({segundos}s)")

async def main():
    await asyncio.gather(
        tarea("A", 3),
        tarea("B", 1),
        tarea("C", 2)
    )

asyncio.run(main())
```

Salida:

```
A empieza
B empieza
C empieza
B termina (1s)
C termina (2s)
A termina (3s)
```

gather espera a **todas** las corrutinas: el main() no termina hasta que A, B y C lo hacen. El tiempo total es el de la más lenta (3s), no la suma (6s).

Si en lugar de gather hicieras `await tarea("A", 3) → await tarea("B", 1) → await tarea("C", 2)`, serían $3 + 1 + 2 = 6$ segundos. La concurrencia de gather es lo que lo reduce a 3.

asyncio.create_task: en segundo plano

A veces no quieres esperar: lanzas una tarea y sigues con lo tuyo.

```
import asyncio

async def heartbeat():
    while True:
        print("💓 latido")
        await asyncio.sleep(2)

async def main():
    asyncio.create_task(heartbeat()) # 🎯 en segundo plano, no la espero
    for i in range(3):
        print(f"main trabajando {i}")
        await asyncio.sleep(1)
    print("main termina")

asyncio.run(main())
```

	gather	create_task
Espera	Espera a todas	No espera (segundo plano)

	gather	create_task
Cuándo usarla	Quieres el resultado de todas	Tareas de fondo: heartbeat, timers, monitorización
Captura	Se puede await el resultado	Hay que guardar la referencia para no perderla

⚠ **Truco del maestro:** si creas una tarea con `create_task`, el `main()` termina y **se lleva la tarea consigo**. Para que una tarea de fondo sobreviva, `main()` debe esperar (un `await asyncio.sleep(...)` o `await tarea`) — lo verás en el [punto 5](#).

Mini-chequeo

1. ¿Cuánto tarda `gather` con tareas de 1, 2 y 3 segundos?
2. ¿Qué hace `create_task` con la corrutina?
3. ¿Qué pasa si `main()` termina y deja tareas creadas con `create_task`?

>  Respuestas

Resumen en 3 frases

- `asyncio.gather` lanza varias corrutinas a la vez y **espera a todas**.
- `asyncio.create_task` lanza una corrutina **en segundo plano** sin bloquear a la principal.
- La concurrencia de `asyncio` reduce el tiempo total al de la tarea más lenta, no a la suma.

Vocabulario rápido

Término	Idea general
gather	Lanza corrutinas a la vez y espera a todas
create_task	Lanza una corrutina en segundo plano
Task	La corrutina ya programada en el event loop

Término	Idea general
Concurrencia	Varias tareas avanzan en el mismo hilo, alternándose
Segundo plano	La tarea se ejecuta mientras la principal sigue su camino



[Volver al índice de la unidad](#) · Anterior: [02 · Corrutinas](#) · Siguiente: [04 · Timeouts](#)

04 — Timeouts

`wait_for` para que nada se cuelgue para siempre 🕒

📖 Estás en: 🕒 U11 · asyncio y Disponibilidad → 04 · Timeouts

La idea en una frase

Un **timeout** es un tope de tiempo: si una operación tarda más de lo previsto, `asyncio.wait_for` lanza `asyncio.TimeoutError` y el servicio sigue vivo. Nada se cuelga para siempre.

El problema del cuelgue

Un `reader.read(1024)` espera datos... ¿y si el cliente nunca los envía? La corrutina quedaría esperando para siempre, ocupando su tarea sin hacer nada. En un servidor, un puñado de clientes mudos llena el servicio de corrutinas zombie.

`asyncio.wait_for` pone un límite:

```
import asyncio

async def atender_con_timeout(reader, writer):
    try:
        # Esperar datos con timeout de 10 segundos
        datos = await asyncio.wait_for(reader.read(1024), timeout=10)
        writer.write(b"OK: " + datos)
        await writer.drain()
    except asyncio.TimeoutError:
        writer.write("🕒 Timeout: conexión cerrada por inactividad\n".encode())
        await writer.drain()
    finally:
        writer.close()
        await writer.wait_closed()
```

Con `timeout=10`, un cliente mudo recibe el aviso y la conexión se cierra a los 10 segundos. **El servidor no se queda colgado con él** y la corrutina libera los recursos.



wait_for con respaldo

El patrón del “fallback”: si la tarea tarda demasiado, usas un plan B.

```
import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Resultado lento"

async def respaldo():
    return "Resultado en caché"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=5)
        print(r)
    except asyncio.TimeoutError:
        print(await respaldo())

asyncio.run(main())
```

Salida: Resultado en caché — la tarea lenta no llegó en 5 segundos y el respaldo tomó el relevo. Es la semilla del mecanismo de disponibilidad que montarás en el [punto 8](#).



El timeout también se combina con el [backoff](#): en cada intento de conexión, wait_for corta la espera y el backoff decide cuánto esperar antes del siguiente.



Mini-chequeo

1. ¿Qué excepción lanza wait_for cuando se supera el tiempo?
2. ¿Qué pasa con la corrutina “lenta” tras el timeout?
3. ¿Por qué un timeout es esencial en un servidor de producción?

>  Respuestas



Resumen en 3 frases

- `asyncio.wait_for(corrutina, timeout=N)` corta una operación que se pasa de N segundos.
- Al saltar `asyncio.TimeoutError` puedes **responder con un plan B** (respaldo, aviso, cierre).
- Los timeouts son el primer mecanismo de **disponibilidad**: un servicio que no se cuelga está disponible.



Vocabulario rápido

Término	Idea general
Timeout	Tope de tiempo para una operación
<code>wait_for</code>	Función que aplica el tope y lanza <code>TimeoutError</code>
<code>TimeoutError</code>	Excepción que indica que se superó el tiempo
Respaldo (fallback)	La alternativa que se ejecuta ante el timeout
Cancelar	Interrumpir una corrutina que ya no vale la pena

 [Volver al índice de la unidad](#) · Anterior: [03 · create_task y gather](#) · Siguiente: [05 · Heartbeat](#)

05 — Heartbeat

El latido que confirma que el servicio sigue vivo ❤️

📖 Estás en: ⌚ U11 · asyncio y Disponibilidad → 05 · Heartbeat

📌 La idea en una frase

Un **heartbeat** (latido) es un mensaje periódico para verificar que el servidor sigue vivo. Si el latido deja de sonar, algo va mal: ese es el aviso.

❤️ El latido en asyncio

En un hilo, un heartbeat sería un `threading.Thread` daemon con `time.sleep`. En asyncio, es una **corrutina en segundo plano** lanzada con `create_task` (el [punto 3](#)):

```
import asyncio

async def heartbeat(intervalo=5):
    while True:
        print("❤️ Heartbeat: servidor vivo")
        await asyncio.sleep(intervalo)

async def servidor_con_heartbeat():
    # Lanzar heartbeat en segundo plano
    asyncio.create_task(heartbeat())

    servidor = await asyncio.start_server(
        lambda r, w: None, "127.0.0.1", 5000
    )
    async with servidor:
        await servidor.serve_forever()

asyncio.run(servidor_con_heartbeat())
```

Cada 5 segundos el servidor imprime que sigue vivo. El `create_task` lo lanza **en segundo plano** mientras el servidor atiende conexiones. La salida:

♥ Heartbeat: servidor vivo
♥ Heartbeat: servidor vivo
♥ Heartbeat: servidor vivo
...

El heartbeat responde a la pregunta “¿está vivo el servidor?” sin necesidad de que un cliente haga una petición. Es la **vigilancia pasiva** del servicio.

Heartbeat con estado

Un heartbeat más útil informa del **estado**: cuántas conexiones atiende ahora mismo.

```
import asyncio

conexiones = 0

async def atender(reader, writer):
    global conexiones
    conexiones += 1
    datos = await reader.read(1024)
    writer.write(b"OK: " + datos)
    await writer.drain()
    writer.close()
    await writer.wait_closed()
    conexiones -= 1

async def heartbeat():
    while True:
        print(f"♥♥ Vivo - {conexiones} conexiones")
        await asyncio.sleep(5)
```

Si el latido deja de aparecer en el log... el proceso está muerto o bloqueado. Y si aparece con 0 conexiones durante horas, sabes que el servicio está arriba pero nadie lo usa: también es información.

Mini-chequeo

1. ¿Qué es un heartbeat?

2. ¿Por qué se lanza con `create_task` y no con un simple `await`?
3. ¿Qué información adicional puede llevar el latido?

> Respuestas

Resumen en 3 frases

- Un heartbeat es un **mensaje periódico** que prueba que el servicio sigue vivo.
- En asyncio se implementa como una **corrutina en bucle infinito lanzada con `create_task`**.
- Añadirle el **estado** (conexiones, errores) lo convierte en vigilancia real de disponibilidad.

Vocabulario rápido

Término	Idea general
Heartbeat	Latido: mensaje periódico de “sigo vivo”
<code>create_task</code>	Cómo se lanza el latido en segundo plano
Bucle infinito	El <code>while True</code> del heartbeat
Estado del servicio	Datos que el latido puede reportar (conexiones, errores)
Disponibilidad	Capacidad de un servicio de seguir respondiendo

 [Volver al índice de la unidad](#) · Anterior: [04 · Timeouts](#) · Siguiente: [06 · Backoff](#)

06 — Backoff

Reintentos con espera exponencial (1, 2, 4, 8...) 

 Estás en:  U11 · asyncio y Disponibilidad → 06 · Backoff

La idea en una frase

El **backoff exponencial** reintentará una conexión fallida esperando cada vez más: 1s, 2s, 4s, 8s... Así no machacas a un servidor que ya está teniendo problemas.

El problema: reintentar sin piedad

Un cliente que falla y reintentará en bucle cerrado cada milisegundo puede **tumbar** un servidor ya debilitado: cada intento añade carga. La solución: esperar cada vez más entre intentos.

```
import asyncio

async def conectar_con_backoff(host, port, max_intentos=5):
    for intento in range(max_intentos):
        try:
            reader, writer = await asyncio.wait_for(
                asyncio.open_connection(host, port),
                timeout=3
            )
            print(f"✅ Conectado a {host}:{port}")
            return reader, writer
        except (asyncio.TimeoutError, ConnectionRefusedError):
            espera = 2 ** intento # 1, 2, 4, 8, 16 segundos
            print(f"⚠️ Intento {intento+1} fallido. Esperando {espera}s...")
            await asyncio.sleep(espera)

    raise Exception("No se pudo conectar")
```

La espera sigue la secuencia $2^0, 2^1, 2^2, 2^3, 2^4 = 1, 2, 4, 8, 16$ segundos. Un fallo puntual se resuelve en el intento 2 con solo 1s de espera; un servidor caído de verdad no recibe 5 peticiones seguidas: recibe 5 con descanso creciente. El `wait_for(..., timeout=3)` añade además un tope por intento (el [punto 4](#)).



La secuencia del backoff

Intento	Espera (2 ** intento)	Acumulado
0	1s	1s
1	2s	3s
2	4s	7s
3	8s	15s
4	16s	31s

El **backoff exponencial** evita saturar un servidor que ya está teniendo problemas. Es la diferencia entre “un cliente molesto” y “un cliente que remata al servidor”.



Mini-chequeo

1. ¿Qué secuencia de espera genera 2^{**} intento con 4 reintentos?
2. ¿Qué dos excepciones captura el ejemplo?
3. ¿Por qué es mejor que reintentar sin espera?

>  Respuestas



Resumen en 3 frases

- El backoff exponencial reintenta con **espera creciente** (2^{**} intento).
- Se combina con `wait_for` para que cada intento tenga también un **timeout**.
- Es el segundo mecanismo de disponibilidad: el cliente que **aguanta sin machacar**.



Vocabulario rápido

Término	Idea general
Backoff	Reintento con espera creciente entre intentos
2 ** intento	Fórmula de la espera: 1, 2, 4, 8, 16...
ConnectionRefusedError	El servicio no escucha en ese puerto
Reintento	Nueva tentativa tras un fallo
Jitter (variación)	Añadir aleatoriedad a la espera para evitar avalanchas (tema avanzado)

 [Volver al índice de la unidad](#) · Anterior: [05 · Heartbeat](#) · Siguiente: [07 · Threads vs asyncio](#)

07 — Threads vs asyncio

La comparativa definitiva de modelos de concurrencia 🚦

📖 Estás en: ⌚ U11 · asyncio y Disponibilidad → 07 · Threads vs asyncio

📌 La idea en una frase

Los **hilos** son multitarea de verdad gestionada por el SO; **asyncio** es multitarea cooperativa en un solo hilo. Para servidores con mucho I/O y miles de conexiones, asyncio gana; para simplicidad con pocos clientes, los hilos.

🚦 El ring de los conceptos

Thread: “Yo soy multitarea de verdad. Tengo mi propia pila, mi propio contexto. El SO me gestiona.”

Asyncio: “Yo soy multitarea cooperativa. Un solo hilo, pero cambio de tarea cuando una espera.”

Thread: “Si tengo 10.000 clientes, creo 10.000 hilos. El sistema sufre.”

Asyncio: “Yo con 10.000 clientes uso un hilo y 10.000 corrutinas. Mucho más ligero.”

Thread: “Pero mis operaciones son bloqueantes de verdad. Si llamo a `time.sleep()`, otro hilo ejecuta.”

Asyncio: “Mis operaciones son `await` — nunca bloqueo. El event loop decide qué toca.”

Thread: “Para servidores pequeños (<100 clientes), soy más simple.”

Asyncio: “Para servidores con mucho I/O y muchas conexiones, soy imbatible.”

⚖️ La tabla comparativa

Característica	Threads	Asyncio
No de hilos	Varios (gestión del SO)	1 (event loop)

Característica	Threads	Asyncio
Cambio de contexto	Gestionado por el SO	Cooperativo (en await)
Escalabilidad	Media (límite de hilos)	Alta (miles de conexiones)
Complejidad	Baja (simple)	Media (curva de aprendizaje)
CPU-bound	No sirve (GIL)	No sirve
I/O-bound	Sí funciona	Excelente

Dos reglas de oro de la tabla:

- **I/O-bound** (esperas de red, disco, timers): asyncio brilla. Las corrutinas esperan sin ocupar hilos.
- **CPU-bound** (cálculo puro, bucles pesados): ni threads ni asyncio con el **GIL**. Para eso están los procesos (multiprocessing, TEMA 01).

¿Cuándo usar cada uno?

Situación	Elige
Servidor con pocos clientes (< 100) y quieres simplicidad	Threads
Servidor con miles de conexiones y mucho I/O	Asyncio
Código de terceros bloqueante (requests, time.sleep)	Threads (o run_in_executor)
Todo el stack propio, operaciones con await	Asyncio
Cálculo intensivo de CPU	Ninguno: procesos (multiprocessing)

💡 Puedes **mezclar** ambos: `loop.run_in_executor()` lleva código bloqueante a un hilo real mientras el event loop sigue. Pero como norma, “mejor si todo es asyncio”: un solo modelo mental.



Mini-chequeo

1. ¿Quién gestiona el cambio de contexto en threads? ¿Y en asyncio?
2. ¿Por qué asyncio no sirve para CPU-bound?
3. ¿Cuándo es más simple usar threads?

>  Respuestas



Resumen en 3 frases

- Threads = **multitarea del SO**; asyncio = **multitarea cooperativa en un hilo**.
- Para I/O-bound con muchas conexiones, **asyncio es imbatible**; para pocos clientes, los hilos son más simples.
- Ninguno sirve para CPU-bound (GIL): ahí mandan los **procesos**.



Vocabulario rápido

Término	Idea general
Multitarea del SO	El sistema operativo decide qué hilo ejecuta
Multitarea cooperativa	Las corrutinas ceden el control voluntariamente en cada await
GIL	Candado de Python que limita la ejecución paralela de hilos
I/O-bound	Tarea que pasa la mayor parte del tiempo esperando
CPU-bound	Tarea que pasa la mayor parte del tiempo calculando
run_in_executor	Puente para llevar código bloqueante a un hilo real

 [Volver al índice de la unidad](#) · Anterior: [06 · Backoff](#) · Siguiente: [08 · Disponibilidad y práctica](#)

08 — Disponibilidad y práctica

Todo junto: el monitor de servicio y Aprieta el lápiz 

 Estás en:  U11 · asyncio y Disponibilidad → 08 · Disponibilidad y práctica

La idea en una frase

Este punto junta la unidad: un **monitor de servicio** que vigila un servidor con **heartbeat** (¿sigue vivo?), lo comprueba con **timeout** (¿responde a tiempo?) y reconecta con **backoff** (¿reintento sin machacar?). Disponibilidad = latido + timeout + backoff.

El monitor de servicio

Un monitor asíncrono que comprueba el estado de un servicio cada N segundos y reintenta con backoff cuando cae:

```

import asyncio

async def comprobar_servicio(host, port):
    """Devuelve True si el servicio responde dentro del timeout."""
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(host, port), timeout=2
        )
        writer.close()
        await writer.wait_closed()
        return True
    except (asyncio.TimeoutError, ConnectionRefusedError):
        return False

async def monitor(host, port):
    fallos = 0
    while True:
        if await comprobar_servicio(host, port):
            print(f"❤️ {host}:{port} disponible (fallos seguidos: {fallos})")
            fallos = 0
        else:
            fallos += 1
            espera = 2 ** min(fallos, 4)  # backoff: 2, 4, 8, 16, 16...
            print(f"💔 {host}:{port} caído – reintento en {espera}s")
            await asyncio.sleep(espera)
            continue
        await asyncio.sleep(5)  # comprobación periódica

asyncio.run(monitor("127.0.0.1", 5000))

```

Tres mecanismos de disponibilidad en un solo script:

Mecanismo	Qué hace	Dónde
Heartbeat	Comprueba el servicio periódicamente (cada 5s)	while True + asyncio.sleep(5)
Timeout	No espera a un servicio mudo más de 2s	asyncio.wait_for(..., timeout=2)

Mecanismo	Qué hace	Dónde
Backoff	Ante fallos, espera cada vez más (2, 4, 8...)	<code>2 ** min(fallos, 4)</code>



Be the code: el event loop en un servidor

“Sé el event loop. Tu trabajo es coordinar corrutinas sin bloquear ni un milisegundo.”

El servidor asyncio con todas las piezas (del [punto 3](#) + el [punto 5](#)):

```
import asyncio

async def atender(reader, writer):
    addr = writer.get_extra_info('peername')
    print(f"[+] Cliente {addr} conectado")

    datos = await reader.read(1024)
    print(f"    Recibido: {datos.decode()}")

    writer.write(b"OK: " + datos)
    await writer.drain() # Espera a que se envíe

    writer.close()
    await writer.wait_closed()
    print(f"[-] Cliente {addr} desconectado")

async def main():
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    print("🚀 Servidor ASYNCIO en 127.0.0.1:5000")

    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())
```

La traza, paso a paso:

Event Loop arranca

```
|
|— 1. Ejecuta main()
|   |— Crea servidor TCP
|   |— Registra atender() para nuevos clientes
|
|— 2. Event Loop: "Espero eventos... (I/O, timers, etc.)"
|
|— [Cliente-1 conecta]
| 3. Event Loop: "¡Cliente nuevo! Ejecuto atender(cliente1)"
| 4. atender(cliente1) empieza
| 5. await reader.read() → "No hay datos aún"
| 6. atender(cliente1) se pausa (cede el control)
|
|— [Cliente-2 conecta mientras cliente1 espera]
| 7. Event Loop: "¡Otro cliente! Ejecuto atender(cliente2)"
| 8. atender(cliente2) empieza
| 9. await reader.read() → "Tampoco hay datos"
| 10. atender(cliente2) se pausa
|
|— [Cliente-1 envía datos]
| 11. Event Loop: "Cliente1 tiene datos → reanudo atender(cliente1)"
| 12. atender(cliente1) recibe los datos
| 13. writer.write() → escribe buffer
| 14. await writer.drain() → espera envío → se pausa
|
|— [Cliente-2 envía datos]
| 15. Event Loop: "Cliente2 tiene datos → reanudo atender(cliente2)"
| 16. atender(cliente2) recibe, responde, termina ❧
|
|— [writer.drain() de cliente1 listo]
| 17. Event Loop: "Cliente1 puede finalizar"
| 18. atender(cliente1) termina ❧
|
|— Event Loop sigue esperando más clientes...
```

Nunca hay espera activa. Cuando una corrutina espera, otra aprovecha. **Un solo hilo, miles de conexiones.**



Aprieta el lápiz

1. **Asyncio básico:** Crea 3 corrutinas que esperen 1, 2 y 3 segundos. Lánzalas con `gather` y mide el tiempo total.
2. **Servidor asyncio:** Convierte el servidor TCP del TEMA 10 a asyncio.
3. **Heartbeat:** Añade un heartbeat que imprima “ vivo” cada 3s mientras el servidor funciona.
4. **Backoff:** Crea un cliente que intente conectarse 3 veces con backoff exponencial.
5. **Monitor:** Monta el monitor de servicio con heartbeat + timeout + backoff que vigile un puerto.

>  Soluciones

Mini-chequeo

1. ¿Qué tres mecanismos de disponibilidad combina el monitor?
2. En la traza del servidor, ¿qué hace el event loop cuando `atender(cliente1)` llega a `await reader.read()`?
3. ¿Por qué el servidor asyncio atiende a 2 clientes a la vez sin hilos?

>  Respuestas

Resumen en 3 frases

- Disponibilidad = **heartbeat + timeout + backoff**: vigilas, cortas lo que tarda y reintentas con cabeza.
- El servidor asyncio atiende miles de conexiones en **un solo hilo** gracias a los `await`.
- Con el monitor y el servidor completos, ya tienes el material para el cierre y los boletines.

Vocabulario rápido

Término	Idea general
Monitor de servicio	Vigila el estado de un servicio periódicamente
Disponibilidad	Un servicio que no se cuelga, avisa y se recupera

Término	Idea general
start_server	La función de asyncio que crea el servidor TCP
serve_forever	Mantiene el servidor escuchando hasta el Ctrl+C
reader / writer	Los objetos asyncio de lectura y escritura de cada cliente

 [Volver al índice de la unidad](#) · **Anterior:** [07 · Threads vs asyncio](#) · **Siguiente:** [09 · Cierre](#)

09 — Cierre: consolida lo aprendido

Sé la corrutina, el ring final y el último Laboratorio de tortura 🍷

📖 Estás en: ⌚ U11 · asyncio y Disponibilidad → 09 · Cierre

Has terminado la teoría: el event loop, las corrutinas, gather y create_task, los timeouts, el heartbeat, el backoff y la comparativa con los hilos. Este cierre es el aterrizaje final del viaje: recorres lo aprendido con juegos, un laboratorio con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del [punto 8](#) y antes de abrir los boletines.

★ Sé la corrutina

Eres una corrutina `monitor()` en un event loop con 2.000 corrutinas más. Tu trabajo: comprobar un servicio cada 5 segundos y avisar si cae.

¿Qué pasa?

1. El event loop te ejecuta. Llegas a `await asyncio.wait_for(open_connection(...), timeout=2)`: **te pausas** y el event loop pasa a otras corrutinas.
2. El servicio responde. El event loop te reanuda: imprimes  disponible.
3. Llegas a `await asyncio.sleep(5)`: **te vuelves a pausar**. Durante esos 5 segundos, el event loop atiende a las otras 1.999 corrutinas.
4. Un día el servicio no responde: `TimeoutError`. Imprimes  caído y esperas 2 ** fallos segundos (**backoff**), pausada, mientras el event loop sigue con las demás.
5. Así, con una corrutina, vigilas un servicio **sin bloquear a nadie**: las otras 2.000 corrutinas nunca se enteran de tu espera.

💡 **Ahora tú:** ¿y si fueran 2.000 monitores vigilando 2.000 servicios? Con asyncio no pasa nada: 2.000 corrutinas en un hilo. Con hilos, 2.000 hilos hundirían el sistema ([punto 7](#)).

🔥 Fireside Chat: hilos vs asyncio

Dos modelos de concurrencia se sientan junto a la chimenea a resolver, de una vez, quién atiende mejor a los clientes.

Hilo: — Yo soy multitarea de verdad. Tengo mi propia pila, mi propio contexto. El SO me gestiona.

Asyncio: — Yo soy multitarea cooperativa. Un solo hilo, pero cambio de tarea cuando una espera.

Hilo: — Si tengo 10.000 clientes, creo 10.000 hilos. El sistema sufre.

Asyncio: — Yo con 10.000 clientes uso un hilo y 10.000 corrutinas. Mucho más ligero.

Hilo: — Pero mis operaciones son bloqueantes de verdad. Si llamo a `time.sleep()`, otro hilo ejecuta.

Asyncio: — Mis operaciones son `await` — nunca bloqueo. El event loop decide qué toca.

Hilo: — Para servidores pequeños (<100 clientes), soy más simple.

Asyncio: — Para servidores con mucho I/O y muchas conexiones, soy imbatible.

Moraleja: Hilos para lo simple y bloqueante; asyncio para escalar en I/O. Y si algo calcula a saco, ni uno ni otro: procesos.



¿Quién soy?

1. Soy el gestor que decide qué corrutina se ejecuta en cada momento.
2. Soy la función declarada con `async def` que puede pausarse.
3. Soy la marca que le dice al event loop “esto va a tardar, ocúpate de otras cosas”.
4. Soy la llamada que lanza varias corrutinas a la vez y espera a todas.
5. Soy el mensaje periódico que confirma que un servicio sigue vivo.
6. Soy la excepción que aparece cuando una operación supera su timeout.
7. Soy la espera que crece en cada reintento: 1, 2, 4, 8...

>  Respuestas



CONRAD VS EL MUNDO: “el servidor se cuelga si un cliente no responde”

CONRAD: — “Clásico: montas un servidor asyncio, llega un cliente que no envía nada y ‘*el servidor se ha quedado colgado*’. Pues no: el servidor sigue, lo que pasa es que **una corrutina se queda esperando a un read() que nunca llega**. Cada cliente mudo es una corrutina ocupada para siempre. Sin timeout, un puñado de clientes mudos te llena el servicio de corrutinas zombie.”

CONRAD: — “Y lo mejor: ‘*he puesto `asyncio.sleep(2)` y mi servidor va lento*’. ¡Pues claro! Y si te olvidas del `await` y escribes `asyncio.sleep(2)` sin más, la corrutina ni se pausa ni espera: la tarea no hace lo que crees. Revisa que cada llamada asíncrona tenga su `await`.”

CONRAD: — “Y no me vengas con ‘*¿será que `asyncio` no vale?*’. El event loop va perfecto: el problema es que **no hay timeout ni backoff**. Envuelve el `read()` en `asyncio.wait_for`, añade un heartbeat, y el servicio no se cuelga ni se te muere en silencio. A diagnosticar.”

Laboratorio de tortura: el monitor de servicio definitivo

Duración: 45 minutos **Herramienta:** Python 3 (solo stdlib: `asyncio`)

Escenario: construye un **monitor de servicio** que vigile `127.0.0.1:5000` con heartbeat (comprobar cada 5s), timeout (2s por comprobación) y backoff (2, 4, 8...). Acompáñalo de un servidor asyncio mínimo que se apague a los 15 segundos para que el monitor experimente una caída real.

Tareas paso a paso:

1. Escribe el servidor asyncio mínimo con `asyncio.start_server` que responda `b"OK"` y se detenga a los 15s con `asyncio.sleep(15)` ([punto 8](#)).
2. Escribe `comprobar_servicio()` con `asyncio.wait_for(asyncio.open_connection(...), timeout=2)` que devuelva `True/False` ([punto 4](#)).
3. Escribe el monitor con `while True + asyncio.sleep(5)` y `backoff 2 ** min(fallos, 4)` ([puntos 5 y 6](#)).
4. Ejecuta monitor y servidor en **terminales separadas**. El servidor se apaga solo a los 15s; el monitor debe pasar de  disponible a  caído con esperas crecientes.
5. Vuelve a arrancar el servidor a mano: el monitor debe volver a  (se ha recuperado).

Fallo intencionado: cambia el timeout del monitor a `timeout=0.1` con el servidor funcionando. ¿Qué pasa? El monitor ve `TimeoutError` casi siempre, aunque el servicio esté sano: con 0.1s no

le da tiempo a completar la conexión. Ahora tienes un monitor **falso positivo**: reporta caído un servicio que responde. Un timeout demasiado agresivo es tan malo como no tenerlo.

Pista 1: el fallo no está en el servidor (que responde), está en el **timeout del monitor**: con `timeout=0.1` la comprobación no llega a completarse y salta `TimeoutError`. Es el mismo mecanismo del [punto 4](#), pero usado mal.

Pista 2: fíjate en el orden del [punto 8](#): heartbeat (¿compruebo?), timeout (¿respondo a tiempo?), backoff (¿cómo reintento?). Si el monitor dice “caído” con el servicio arriba, el problema es el **umbral**: el timeout debe ser mayor que el tiempo real de respuesta del servicio.

Logros de esta unidad

Logro	Cómo conseguirlo
 Loop Whisperer	Explicar qué hace el event loop cuando una corrutina espera
 Coroutine Master	Escribir corrutinas con <code>async def</code> y <code>await</code> sin bloqueos
 Gather Gatherer	Lanzar tareas concurrentes con <code>gather</code> y <code>create_task</code>
 Timeout Guardian	Evitar que un servicio se cuelgue con <code>wait_for</code>
 Heartbeat Keeper	Añadir un latido que avise de que el servicio sigue vivo
 Backoff Strategist	Reintentar conexiones con espera exponencial
 Model Decider	Elegir con criterio entre hilos y <code>asyncio</code>

Atrévete a pensar

1. ¿Por qué `asyncio` usa un solo hilo y aún así atiende miles de conexiones?
2. ¿Qué pasa si olvidas el `await` delante de una operación asíncrona?
3. ¿Cuándo usarías `threads` y cuándo `asyncio` en producción?
4. ¿Por qué un timeout demasiado corto es peor que no tenerlo?
5. ¿Cómo combinarías heartbeat, timeout y backoff en un solo servicio?

🧩 Crucigrama de bits

Horizontal:

1. El gestor que decide qué corrutina ejecutar (11 letras)
4. Función asíncrona que puede pausarse (10 letras)
6. La espera creciente de los reintentos (7 letras)
8. Mensaje periódico que confirma que un servicio vive (9 letras)

Vertical:

2. Llamada que lanza varias corrutinas a la vez y espera a todas (6 letras)
3. La marca "esto va a tardar, ocúpate de otras cosas" (5 letras)
5. Excepción que aparece al superar el tiempo límite (11 letras)
7. Tope de tiempo para una operación (7 letras)

💬 Entrevista de trabajo

1. “¿Qué es asyncio y en qué se diferencia de los hilos?”
2. “Explica qué hace el event loop con un ejemplo.”
3. “¿Qué es una corrutina? ¿Qué diferencia `await` de un `sleep()` bloqueante?”
4. “¿Cómo harías que un servidor no se cuelgue nunca con un cliente mudo?”
5. “Diseña un servicio siempre disponible: ¿qué mecanismos le pones?”

💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, recorre la cadena de la unidad: bloqueo → hilos (caros con miles) → event loop → corrutinas → `gather/create_task` → comparativa del [punto 7](#). Para la 4, plantea `asyncio.wait_for(reader.read(...), timeout=10)` con `except asyncio.TimeoutError` y cierre de la conexión: el [punto 4](#) es tu respuesta. Y la 5 es tu momento estrella: heartbeat + timeout + backoff. Si lo cuentas fluido, ya eres medio ingeniero de disponibilidad.

🙋 No hay preguntas tontas

? ¿Asyncio es más rápido que threads?

Para I/O-bound tasks, sí, porque no hay cambio de contexto del SO. Para CPU-bound, no hay diferencia (ambos limitados por el GIL).

? ¿Puedo mezclar código síncrono con asyncio?

Sí, con `loop.run_in_executor()`. Pero mejor si todo es asyncio.

? ¿Qué es una corrutina?

Una función declarada con `async def` que puede pausarse con `await` y reanudarse después. No es un hilo, es una función que sabe esperar.

? ¿await bloquea el hilo?

No. `await` le dice al event loop: “ahora no necesito CPU, ocúpate de otras corrutinas”. Es cooperativo.

? ¿Cuándo usar threads y cuándo asyncio?

- Threads: proyectos pequeños, librerías bloqueantes, simplicidad.
- Asyncio: muchos clientes concurrentes, mucho I/O, escalabilidad.

? ¿Y si me olvido del await?

La operación no se espera (ni a veces se ejecuta): la corrutina no hace lo que crees. Es el fallo más típico al empezar con asyncio.

Poscréditos

El servidor secuencial hacía esperar a todos en la cola de la ventanilla única.

Llegó el hilo por cliente y cada conexión tuvo su propia ventanilla.

El ThreadPool puso límites al entusiasmo: cinco camareros, cola ordenada, nadie se ahoga.

El Lock protegió el contador. El benchmark demostró la mejora. Los clientes responden a la vez.

Y entonces llegó asyncio: un solo camarero, miles de mesas, ninguna espera quieta.

El latido confirma que el servicio sigue vivo. El timeout corta lo que se cuelga. El backoff reintentará con cabeza.

El viaje ha recorrido once hitos: 🚀 Proceso, 🧵 Hilo, 🔒 Sincronización, 🌐 TCP, 🌐 UDP, 🌐 API REST, 🧪 APIs comerciales, 🔑 Hash, 🧬 Cifrado, 🏗️ Servidores concurrentes y ⌚ asyncio y Disponibilidad.

Y con esto... ¡el viaje ha terminado! 🏁

✅ Criterios de evaluación cubiertos (RA4e-g)

RA4: Implementa servicios en red, desarrollando mecanismos de disponibilidad y servidores asíncronos.

CE	Criterio	Cubierto
e)	Implementa mecanismos de disponibilidad (heartbeat, reintentos, timeout)	✅ Heartbeat, backoff y timeouts (puntos 4-6) + ⚡ Laboratorio
f)	Desarrolla servidores con asyncio	✅ Corrutinas, gather y start_server (puntos 1-3 y 8) + ⚡ Laboratorio
g)	Compara modelos de concurrencia (hilos vs asyncio)	✅ Comparativa (punto 7) + 🔥 Fireside Chat

RA4c (servidores concurrentes con hilos) y RA4d (ThreadPool) se cubrieron en la **U10 · Servidores Concurrentes**.

📄 [Volver al índice de la unidad](#) · Anterior: [08 · Disponibilidad y práctica](#)

Boletín U01 — Inicial (Resuelto)

Boletín U01 — Inicial (Resuelto)

1. Listar el directorio actual

```
import subprocess

resultado = subprocess.run(["dir"], capture_output=True, text=True, shell=True)
print(resultado.stdout)
```

En Linux: `subprocess.run(["ls"])`. En Windows, `dir` es un comando interno del shell, por eso `shell=True` ([punto 7](#)).

2. Saber el nombre del equipo

```
import subprocess

resultado = subprocess.run(["hostname"], capture_output=True, text=True)
print(f"El nombre del equipo es {resultado.stdout.strip()}")
```

`capture_output=True` captura la salida de `hostname` y `text=True` la devuelve como string.

3. Abrir la calculadora

```
import subprocess, time

calc = subprocess.Popen(["calc.exe"])
print(f"Calculadora lanzada con PID {calc.pid}")
time.sleep(2)
calc.terminate()
print("Calculadora cerrada")
```

`Popen` no espera: el programa muestra el PID, espera 2 segundos y cierra con `terminate()`.

4. Mi primer PID

```
import os

print(f"Mi PID es {os.getpid()}")
```

`os.getpid()` devuelve el identificador único del proceso actual: el **PID** ([punto 1](#)).

5. Versión de Python

```
import subprocess

resultado = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(resultado.stdout.strip())
```

`capture_output=True` captura la salida. `text=True` la devuelve como string. `.strip()` elimina el salto de línea final.

6. Abrir el bloc de notas y esperar

```
import subprocess, time

notepad = subprocess.Popen(["notepad.exe"])
print(f"PID: {notepad.pid}")
time.sleep(3)
notepad.terminate()
```

`Popen` no espera. `terminate()` envía señal de cierre.

7. Ordena el ciclo de vida

NUEVO → **LISTO** → **EJECUCIÓN** → **BLOQUEADO** → **TERMINADO**

El orden correcto: el proceso **nace** (**NUEVO**), espera su turno de CPU (**LISTO**), la CPU lo ejecuta (**EJECUCIÓN**), puede quedarse esperando un recurso (**BLOQUEADO**) y finalmente **termina**. Repasa el [punto 2](#).

8. Ping con timeout

```
import subprocess

try:
    resultado = subprocess.run(["ping", "8.8.8.8", "-n", "3"],
                               capture_output=True, text=True, timeout=3)
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("El ping tardó demasiado")
```

En Linux usa ["ping", "-c", "3", "8.8.8.8"].timeout lanza TimeoutExpired si excede.

Boletín U01 — Inicial



Boletín U01 — Inicial

Ejercicios básicos para afianzar los conceptos de procesos, `subprocess.run`, `subprocess.Popen` y estados de la unidad U01.

1. Listar el directorio actual

Crea un programa que use `subprocess.run` para ejecutar el comando `dir` (Windows) o `ls` (Linux) y muestre la salida completa por pantalla.

2. Saber el nombre del equipo

Usa `subprocess.run` con el comando `hostname` para capturar y mostrar el nombre de tu máquina.

3. Abrir la calculadora

Usa `subprocess.Popen` para abrir la calculadora (`calc.exe`). El programa debe mostrar el PID, esperar 2 segundos y cerrarla con `terminate()`.

4. Mi primer PID

Crea un programa que imprima su propio PID con `os.getpid()`.

5. Versión de Python

Usa `subprocess.run` para ejecutar `python --version` con `capture_output=True` y `text=True`, y muestra la salida limpia (sin saltos de línea).

Pista: `resultado.stdout.strip()` te deja la salida limpia.

6. Abrir el bloc de notas y esperar

Usa `subprocess.Popen` para abrir `notepad.exe`, muestra su PID, espera 3 segundos y ciérralo con `terminate()`.

7. Ordena el ciclo de vida

Ordena los siguientes estados de un proceso en el orden correcto de su ciclo de vida: EJECUCIÓN, NUEVO, TERMINADO, BLOQUEADO, LISTO.

Pista: el proceso nace, luego espera su turno de CPU, se ejecuta, puede quedarse esperando un recurso y finalmente muere. Repasa el [punto 2](#).

8. Ping con timeout

Lanza `ping 8.8.8.8 -n 3` (Windows) o `ping -c 3 8.8.8.8` (Linux) con `timeout=3` y captura la excepción si se excede el tiempo.

Pista: envuelve `subprocess.run` en un `try/except` y captura `subprocess.TimeoutExpired`. Si tu conexión es rápida, prueba con `timeout=1`.

Boletín U01 — Avanzado (Resuelto)

Boletín U01 — Avanzado (Resuelto)

1. Filtrar ipconfig

```
import subprocess

resultado = subprocess.run(["ipconfig"], capture_output=True, text=True)
lineas = resultado.stdout.split("\n")
for linea in lineas:
    if "IPv4" in linea:
        print(linea.strip())
```

`subprocess.run` captura toda la salida de `ipconfig`; `split("\n")` la divide en líneas y el filtro con `in` deja solo las de "IPv4".

2. Comprobar si un programa existe

```
import subprocess

resultado = subprocess.run(["where", "python"], capture_output=True, text=True)
print(f"Código de retorno: {resultado.returncode}")
if resultado.returncode == 0:
    print(f"Python encontrado en: {resultado.stdout.strip()}")
else:
    print("Python NO está en el PATH")
```

En Linux usarías `["which", "python"]`. El código de retorno `0` indica que `where/which` lo encontró; distinto de `0`, no.

3. Abrir el navegador

```
import subprocess

# Windows: start es interno del shell, necesita shell=True
subprocess.run(["start", "http://localhost:4321"], shell=True)

# Linux / macOS:
# subprocess.run(["xdg-open", "http://localhost:4321"])
```

start no es un ejecutable real: es un comando interno de cmd, por eso usa shell=True. xdg-open sí es un ejecutable real en Linux ([punto 7](#)).

4. Lanzador múltiple

```
import subprocess, time

programas = [
    ("Bloc de notas", subprocess.Popen(["notepad.exe"])),
    ("Calculadora", subprocess.Popen(["calc.exe"])),
    ("Paint", subprocess.Popen(["mspaint.exe"])),
]

for nombre, proceso in programas:
    print(f"{nombre} → PID {proceso.pid}")

time.sleep(5)

for nombre, proceso in programas:
    proceso.terminate()
    print(f"Cerrando {nombre}...")

print("Todos terminados 🏁")
```

Guardamos cada objeto Popen en una tupla con su nombre. Recorremos la lista para mostrar PIDs y, a los 5 segundos, para terminarlos todos con terminate().

5. Captura de ping

```
import subprocess

resultado = subprocess.run(["ping", "8.8.8.8", "-n", "5"], capture_output=True,
text=True)
for linea in resultado.stdout.split("\n"):
    if "tiempo" in linea.lower() or "time" in linea.lower():
        print(linea)
```

`split("\n")` divide la salida y el filtro busca las líneas con “tiempo” (Windows en español) o “time” (sistemas en inglés). `.lower()` cubre ambas.

6. PID del padre

```
import os

print(f"Mi PID es {os.getpid()}")
print(f"El PID de mi padre es {os.getppid()}")
```

`os.getpid()` devuelve tu PID; `os.getppid()` el del proceso que te lanzó (normalmente tu terminal o el IDE).

7. Comunicación en dos direcciones

```
import subprocess

proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

salida, _ = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"
```

`communicate(input="hola mundo")` escribe en el `stdin` del hijo, espera su fin y devuelve (`stdout`, `stderr`). El hijo ejecuta `print(input().upper())` ([punto 6](#)).

8. Timeout con reintentos

```
import subprocess

def ejecutar_con_reintentos(comando, intentos=3, timeout=3):
    for intento in range(1, intentos + 1):
        try:
            resultado = subprocess.run(comando, capture_output=True,
                                       text=True, timeout=timeout, check=False)
            if resultado.returncode == 0:
                print(f"✅ Éxito en el intento {intento}")
                return resultado
            print(f"Intento {intento}: código {resultado.returncode}")
        except subprocess.TimeoutExpired:
            print(f"Intento {intento}: se agotó el tiempo")
            raise TimeoutError(f"El comando {comando} falló tras {intentos} intentos")

    try:
        resultado = ejecutar_con_reintentos(["ping", "8.8.8.8", "-n", "3"])
    except TimeoutError as e:
        print(f"❌ {e}")
```

Cada intento usa `timeout=3` y `check=False`; se captura `TimeoutExpired` y se vuelve a intentar hasta 3 veces. Si nada funciona, se lanza una excepción propia.

9. Mini monitor de procesos

```
import subprocess, time

proceso = subprocess.Popen(["notepad.exe"])

while proceso.poll() is None:
    print(f"El proceso (PID {proceso.pid}) sigue vivo...")
    time.sleep(1)

print(f"El proceso terminó con código {proceso.poll()}")
```

`proc.poll()` devuelve `None` mientras el proceso vive y su código de retorno cuando termina. El bucle `while proc.poll() is None` vigila cada segundo hasta que muere.

Boletín U01 — Avanzado



Boletín U01 — Avanzado

Ejercicios que requieren aplicar los conceptos de procesos, subprocess y comunicación entre procesos de forma más profunda, con programas completos.

1. Filtrar ipconfig

Ejecuta `ipconfig` con `subprocess.run`, captura toda la salida y muestra solo las líneas que contengan “IPv4”.

Pista: divide el `stdout` con `split("\n")` y filtra cada línea con el operador `in` para buscar “IPv4”.

2. Comprobar si un programa existe

Usa `subprocess.run` con el comando `where python` (Windows) o `which python` (Linux) para comprobar si Python está accesible desde la terminal. Muestra el código de retorno.

Pista: el código de retorno 0 indica que `where/which` encontró Python. Distinto de 0, no lo encontró.

3. Abrir el navegador

Usa `subprocess.Popen` para abrir el navegador predeterminado con la URL `http://localhost:4321`. En Windows usa `start http://localhost:4321` con `shell=True`, en Linux usa `xdg-open http://localhost:4321`.

Pista: `start` es un comando interno del shell de Windows: necesita `shell=True`. `xdg-open` es un ejecutable real en Linux.

4. Lanzador múltiple

Crea un programa que lance 3 programas distintos a la vez (notepad, calc, mspaint). Muestra sus PIDs y luego los mata todos a los 5 segundos.

Pista: guarda los objetos `Popen` en una lista o tupla junto con el nombre de cada programa. Recorre la lista para mostrar PIDs y más tarde para terminarlos con `terminate()`.

5. Captura de ping

Ejecuta `ping 8.8.8.8 -n 5`, captura `stdout` y extrae solo las líneas que contienen “tiempo” o “time”.

Pista: divide el `stdout` con `split("\n")` y filtra cada línea con el operador `in` para buscar las palabras clave.

6. PID del padre

Crea un programa que muestre su propio PID y el PID del proceso padre (PPID).

Pista: usa `os.getpid()` para tu PID y `os.getppid()` para el PPID. Ejecuta desde la terminal y observa quién es el padre.

7. Comunicación en dos direcciones

Crea un proceso hijo que reciba texto por `stdin` y devuelva el texto en mayúsculas.

Pista: usa `subprocess.Popen` con `stdin=subprocess.PIPE`, `stdout=subprocess.PIPE` y `text=True`. Luego llama a `communicate(input="tu texto")` para enviar datos y leer la respuesta.

8. Timeout con reintentos

Crea una función que ejecute un comando con `timeout` de 3s. Si falla, reintenta hasta 3 veces. Si al final no funciona, lanza una excepción.

Pista: envuelve `subprocess.run` con `timeout=3` y `check=False` dentro de un bucle `for`. Captura `subprocess.TimeoutExpired` en cada intento y rompe el bucle si el comando se ejecuta correctamente. Si se agotan los intentos, lanza una excepción personalizada.

9. Mini monitor de procesos

Crea un programa que lance un subprocess y verifique su estado cada segundo mientras está vivo.

Pista: `proc.poll()` devuelve `None` si el proceso sigue ejecutándose, o el código de retorno si ya terminó. Úsalo en un bucle `while proc.poll() is None` con `time.sleep(1)` dentro.

Boletín U02 — Inicial (Resuelto)



Boletín U02 — Inicial (Resuelto)

1. Hilo que cuenta

```
import threading

def contar():
    for i in range(1, 6):
        print(i)

h = threading.Thread(target=contar)
h.start()
h.join()
```

Salida: 1 2 3 4 5. La función cuenta del 1 al 5, se lanza con `start()` y se espera con `join()`.

2. Múltiples hilos con retardo

```
import threading, time

def numero(n):
    print(f"Hilo {n} empieza")
    time.sleep(1)
    print(f"Hilo {n} termina")

hilos = [threading.Thread(target=numero, args=(i,)) for i in range(1, 4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Los tres hilos se lanzan a la vez y duermen su segundo **en paralelo**: todo el lote acaba en ~1 segundo, no en 3. Los mensajes se entremezclan sin orden garantizado.

3. Hilo con argumento

```
import threading

def saludar(nombre):
    print(f"Hola, {nombre}")

h = threading.Thread(target=saludar, args=("Ana",))
h.start()
h.join()
```

Salida: Hola, Ana. La coma final de ("Ana",) convierte el valor en una tupla de un elemento, que es lo que espera args=.

4. Hilo que saluda

```
import threading

def saludar():
    print("Hola desde un hilo")

h = threading.Thread(target=saludar)
h.start()
h.join()
```

Nunca olvides start() para lanzar y join() para esperar.

5. Dos hilos en paralelo

```
import threading

def hilo_a():
    for _ in range(3):
        print("Hilo A")

def hilo_b():
    for _ in range(3):
        print("Hilo B")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start()
h2.start()
h1.join()
h2.join()
```

Los prints se entremezclarán. El orden no está garantizado.

6. Hilo con nombre

```
import threading

def mostrar():
    print(f"Soy {threading.current_thread().name}")

h = threading.Thread(target=mostrar, name="MiHilo")
h.start()
h.join()
```

`current_thread().name` funciona desde dentro del hilo.

7. Esperando con join()

```
import threading, time

def tarea():
    print("Empezando")
    time.sleep(3)
    print("Terminado")

h = threading.Thread(target=tarea)
h.start()
h.join()
print("El programa terminó")
```

Salida:

```
Empezando
Terminado
El programa terminó
```

Sin `join()`, el “El programa terminó” podría salir antes que “Terminado” (o incluso antes que “Empezando”, según el scheduler). Con `join()`, el orden es garantizado.

8. Varios hilos con nombre y argumentos

```
import threading

def trabajar(n):
    print(f"hilo-{n} trabajando")

hilos = [threading.Thread(target=trabajar, args=(n,), name="hilo-" + str(n))
         for n in range(1, 4)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Cada hilo lleva su nombre (hilo-1, hilo-2, hilo-3) y recibe su número como argumento. Los mensajes se entremezclan, pero el orden de lanzamiento y espera es correcto gracias a los dos bucles.

Boletín U02 — Inicial



Boletín U02 — Inicial

Ejercicios básicos para afianzar los conceptos de hilos de la unidad U02: crear, lanzar, esperar, pasar argumentos y nombrar hilos.

1. Hilo que cuenta

Crea un hilo que ejecute una función que cuente del 1 al 5 imprimiendo cada número. Lánzalo con `start()` y espera con `join()`.

2. Múltiples hilos con retardo

Crea 3 hilos que impriman su número (1, 2, 3) y luego duerman 1 segundo con `time.sleep(1)`. Lánzalos todos y espera a que terminen.

3. Hilo con argumento

Crea una función que reciba un nombre como argumento e imprima “Hola, {nombre}”. Lanza un hilo pasándole el argumento con `args= ("Ana", )`.

Pista: recuerda la coma final de la tupla: `args= ("Ana", )`. Sin la coma, `( "Ana" )` es solo un string y la función recibe un argumento de más.

4. Hilo que saluda

Crea un hilo que ejecute una función que imprima “Hola desde un hilo”. Usa `threading.Thread`, `start()` y `join()`.

5. Dos hilos en paralelo

Crea dos funciones `hilo_a()` y `hilo_b()`, cada una con un bucle que imprima su nombre 3 veces. Lanza los dos hilos y observa cómo se entremezclan los mensajes.

6. Hilo con nombre

Crea una función que imprima `f"Soy {threading.current_thread().name}"`. Lánzala en un hilo con `name="MiHilo"`.

7. Esperando con `join()`

Crea un hilo que imprima “Empezando”, duerma 3 segundos y termine imprimiendo “Terminado”. En el programa principal, después de `start()`, espera con `join()` y luego imprime “El programa terminó”. Comprueba el orden de las tres salidas.

8. Varios hilos con nombre y argumentos

Crea 3 hilos en un bucle, cada uno con `name = "hilo-" + str(n)` y que reciba su número como argumento para imprimir `"hilo-N trabajando"`. Lanza todos y espera a todos.

Pista: usa una lista por comprensión para crear los 3 hilos y luego dos bucles: `for h in hilos: h.start()` y `for h in hilos: h.join()`.

Boletín U02 — Avanzado (Resuelto)

Boletín U02 — Avanzado (Resuelto)

1. Join con timeout

```
import threading, time

def lento():
    print("Hilo empezando")
    time.sleep(5)
    print("Hilo terminó")

h = threading.Thread(target=lento)
h.start()
h.join(timeout=2)
print(f"¿Sigue vivo tras el timeout? {h.is_alive()}")
```

Salida:

```
Hilo empezando
¿Sigue vivo tras el timeout? True
Hilo terminó
```

`join(timeout=2)` espera como máximo 2 segundos: al expirar, el hilo **sigue vivo** (duerme hasta los 5s) y el principal continúa. Como es no-daemon, el programa no termina hasta que el hilo acaba su sueño.

2. Listar hilos activos

```

import threading, time

def tarea(n):
    time.sleep(1)
    print(f"{threading.current_thread().name} terminó")

hilos = [threading.Thread(target=tarea, args=(i,), name="hilo-" + str(i))
         for i in range(1, 4)]

for h in hilos:
    h.start()

print("Hilos activos:")
for h in threading.enumerate():
    print(f" {h.name}")

for h in hilos:
    h.join()

```

`threading.enumerate()` devuelve la lista de todos los hilos vivos, **incluido el principal** (MainThread). Justo tras el `start()` de los tres, la lista contiene 4 entradas: MainThread, hilo-1, hilo-2 y hilo-3.

3. Identificar el hilo actual

```

import threading

def mostrar():
    h = threading.current_thread()
    print(f"name: {h.name} | ident: {h.ident} | daemon: {h.daemon}")

mostrar() # desde el hilo principal

h = threading.Thread(target=mostrar, name="secundario")
h.start()
h.join()

```

Salida parecida a:

```
name: MainThread | ident: 14600 | daemon: False
name: secundario | ident: 14601 | daemon: False
```

La misma función devuelve datos distintos según quién la ejecute: `current_thread()` es el objeto Thread que se está ejecutando en ese momento. El `ident` es un número único mientras el hilo vive.

4. 🎯 Carrera de mensajes

```
import threading

def mensaje(nombre):
    for i in range(1, 11):
        print(f"{nombre}: mensaje {i}")

hilos = [threading.Thread(target=mensaje, args=(f"hilo-{n}",))
         for n in range(1, 6)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Cinco hilos imprimiendo 10 mensajes cada uno: la salida es un caos predecible en el desorden. El orden exacto cambia en cada ejecución porque lo decide el scheduler del sistema operativo. Solo está garantizado que los 50 mensajes salgan (y que el programa espere gracias a los `join()`).

5. 🔍 Simular descarga con progreso

```
import threading, time

def descargar(archivo):
    for porcentaje in range(0, 101, 25):
        print(f"{archivo}: {porcentaje}%")
        time.sleep(0.5)

h = threading.Thread(target=descargar, args=("libro.pdf",))
h.start()
h.join()
```

Salida:

```
libro.pdf: 0%
libro.pdf: 25%
libro.pdf: 50%
libro.pdf: 75%
libro.pdf: 100%
```

El progreso avanza en saltos de 25% (range(0, 101, 25)) con medio segundo de espera por paso: una descarga simulada de 2 segundos en total.

6. Detener hilo con bandera

```

import threading, time

ejecutando = True

def infinito():
    while ejecutando:
        print("trabajando...")
        time.sleep(0.5)
        print("Hilo detenido por bandera")

h = threading.Thread(target=infinito)
h.start()

input("Pulsa Enter para detener...")
ejecutando = False
h.join()
print("Programa terminado")

```

Los hilos no se matan desde fuera (no hay `hilo.kill()`). La forma correcta es una **variable bandera** que el hilo comprueba en cada vuelta de su bucle: al pulsar Enter, el principal pone `ejecutando = False` y el hilo sale del bucle y termina ordenadamente.

7. 🤖👤 Hilo que devuelve un valor

```

import threading, time

resultado = []

def calcular():
    time.sleep(1)
    valor = 6 * 7
    resultado.append(valor)

h = threading.Thread(target=calcular)
h.start()
h.join()
print(f"El resultado del hilo es: {resultado[0]}")

```

Salida:

El resultado del hilo es: 42

Las funciones de los hilos **no tienen return**. Para “devolver” un valor se usa una **lista compartida** (la memoria compartida del [punto 1](#)): el hilo hace append, y tras join() el principal lee el valor con seguridad, porque ya sabe que el hilo ha terminado.

8. 🕒 Timer con repetición

```
import threading, time

def tic():
    print(f"🕒 {time.strftime('%H:%M:%S')}")
    threading.Timer(2.0, tic).start() # se reprograma a sí misma

tic()

time.sleep(6) # el principal deja correr unos cuantos ticks
print("Programa principal terminando...")
```

Salida parecida a:

```
🕒 14:35:00
🕒 14:35:02
🕒 14:35:04
🕒 14:35:06
Programa principal terminando...
```

El Timer normal solo dispara **una vez** ([punto 5](#)). Para repetirlo, la propia función crea y lanza otro Timer al final: recursividad. Así cada tick programa el siguiente. El principal decide cuándo parar: al terminar, el Timer pendiente (daemon por defecto) muere con él.

9. 🏗️ Pool de hilos manual

```

import threading, queue, time

def trabajador():
    while True:
        tarea = cola.get()
        if tarea is None:          # centinela: fin de trabajo
            break
        print(f"{threading.current_thread().name} hace: {tarea}")
        time.sleep(0.5)
        cola.task_done()

cola = queue.Queue()
for t in ["tarea-1", "tarea-2", "tarea-3", "tarea-4"]:
    cola.put(t)

workers = [threading.Thread(target=trabajador, name="worker-" + str(i))
            for i in range(1, 3)]
for w in workers:
    w.start()

cola.join()          # espera a que las 4 tareas se marquen como hechas
for w in workers:
    cola.put(None)    # enviamos el centinela a cada worker
for w in workers:
    w.join()

print("Todas las tareas terminadas")

```

queue.Queue es **thread-safe**: varios hilos pueden hacer get() sin corromper la cola. Dos workers compiten por las 4 tareas (las cogen en cuanto están libres, por eso la repartición puede variar). El principal espera con cola.join() y, al terminar, envía None (centinela) para que cada worker salga del bucle ordenadamente, sin dejar hilos colgados.

Bonus: tres clásicos resueltos

Tres casos que suelen aparecer en exámenes con su solución completa.

Bonus 1 — join() o no join()

```
import threading, time

def lento():
    for i in range(5):
        print(i)
        time.sleep(1)

h = threading.Thread(target=lento)
h.start()
# Sin join() → el programa principal no espera
print("Fin del programa principal")
```

Sin `join()`, el programa principal no espera: su último `print` aparece mientras el hilo sigue contando. El programa en sí no puede terminar hasta que el hilo (no daemon) acabe, pero el orden de salida queda a merced del scheduler.

Bonus 2 — Hilo daemon

```
import threading, time

def tic():
    while True:
        print("tic")
        time.sleep(1)

h = threading.Thread(target=tic, daemon=True)
h.start()
time.sleep(3)
print("Programa termina — el daemon muere")
```

El daemon imprime “tic” cada segundo y se **mata al salir** del programa principal ([punto 4](#)). Si fuera no-daemon con su `while True`, el programa jamás terminaría.

Bonus 3 — Timer


```
import threading

def despierta():
    print("¡Despierta!")

t = threading.Timer(4.0, despierta)
t.start()
```

Timer ejecuta **UNA SOLA vez** después del retardo de 4 segundos ([punto 5](#)). No se repite; para repetirlo, la función debe reprogramarse (ejercicio 8).

Boletín U02 — Avanzado



Boletín U02 — Avanzado

Ejercicios que requieren aplicar los conceptos de hilos de la unidad U02 de forma más profunda: esperas con timeout, introspección, banderas de parada y timers que se repiten.

1. Join con timeout

Crea un hilo que duerma 5 segundos. En el hilo principal, usa `join(timeout=2)` y comprueba con `is_alive()` si el hilo sigue ejecutándose después del timeout.

Pista: `h.join(2)` devuelve `None` cuando termina a tiempo y también cuando expira. Para saber si sigue vivo después del timeout, pregunta con `h.is_alive()` justo después del `join`.

2. Listar hilos activos

Crea 3 hilos que hagan tareas distintas. Usa `threading.enumerate()` para listar todos los hilos activos y muestra sus nombres.

Pista: `threading.enumerate()` devuelve una lista de objetos `Thread`. Itera sobre ella y muestra `.name`. Recuerda que el hilo principal (`main`) también aparece en la lista.

3. Identificar el hilo actual

Crea una función que muestre `threading.current_thread().name`, `threading.current_thread().ident` y `threading.current_thread().daemon`. Ejecútala desde el hilo principal y desde un hilo secundario.

Pista: para ejecutar la función desde ambos sitios, llámala directamente en el principal y también como target de un `threading.Thread`.

4. 🎯 Carrera de mensajes

Crea 5 hilos, cada uno imprime su nombre 10 veces. Observa el orden impredecible.

Pista: usa una lista por comprensión para crear los 5 hilos. Lanza todos con un bucle `for h in hilos: h.start()` y espera con `for h in hilos: h.join()`. El orden de salida cambiará en cada ejecución.

5. 🔍 Simular descarga con progreso

Crea un hilo que descargue (simulado con `sleep`) y muestre progreso cada 25%.

Pista: dentro de la función, itera con `range(0, 101, 25)`, usa `time.sleep(0.5)` para simular el tiempo de descarga e imprime el porcentaje en cada paso.

6. 🧩 Detener hilo con bandera

Crea un hilo infinito que se detenga cuando pulses Enter.

Pista: declara una variable booleana global `ejecutando = True`. El hilo comprueba la variable en un `while` `ejecutando:`. Desde el principal, usa `input()` para esperar Enter y luego cambia `ejecutando = False`.

7. 😊👤 Hilo que devuelve un valor

Usa una variable compartida para que un hilo devuelva un resultado al principal.

Pista: los hilos no tienen `return`. Usa una lista compartida como contenedor: el hilo guarda el resultado con `lista.append(valor)` y el principal lo lee tras `hilo.join()`.

8. ⌚ Timer con repetición

El Timer normal solo se dispara una vez. Crea un timer que se repita cada 2 segundos usando recursividad.

Pista: dentro de la función que ejecuta el Timer, crea otro `threading.Timer(2.0, funcion).start()` al final. Así la función se programa a sí misma de nuevo.

9. 🏗️ Pool de hilos manual

Crea un “pool” manual con una cola de tareas. 2 hilos cogen tareas de la cola y las ejecutan.

Pista: usa `queue.Queue` (thread-safe) para las tareas. Los trabajadores ejecutan un bucle llamando a `cola.get()`. Al terminar cada tarea llaman a `cola.task_done()`. El principal espera con `cola.join()`. Crea los workers como `daemon=True` para que no bloqueen la salida, o usa un valor centinela (`None`) en la cola para que terminen ordenadamente.

Boletín U03 — Inicial (Resuelto)



Boletín U03 — Inicial (Resuelto)

1. Lock protege una lista

```
import threading

lista = []
lock = threading.Lock()

def añadir(letra):
    global lista
    for _ in range(5):
        with lock:
            lista.append(letra)

h1 = threading.Thread(target=añadir, args=("A",))
h2 = threading.Thread(target=añadir, args=("B",))
h1.start()
h2.start()
h1.join()
h2.join()

print(lista)
```

Sin el Lock, los append se mezclarían (por ejemplo, AABABB...). Con `with lock:`, cada añadido es atómico y al final hay **5 A** y **5 B** sin mezclas.

2. RLock básico

```

import threading

rlock = threading.RLock()

def función_interna():
    with rlock:
        print(" ♦ Función interna: lock adquirido")

def función_externa():
    with rlock:
        print(" ♦ Función externa: lock adquirido")
        función_interna()  # mismo hilo → RLock lo permite

función_externa()

```

función_interna adquiere el mismo RLock que función_externa ya tiene puesto. Con un Lock normal sería un **deadlock** (el hilo se espera a sí mismo); con RLock no hay problema ([punto 3](#)).

3. Semáforo con timeout

```

import threading, time

semaforo = threading.Semaphore(2)

def entrar(id):
    if semaforo.acquire(timeout=1):
        print(f"Hilo-{id}: DENTRO")
        time.sleep(1)
        semaforo.release()
    else:
        print(f"Hilo-{id}: timeout")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

```

Solo 2 hilos entran a la vez; los que no consiguen entrar en 1 segundo muestran **"timeout"** y se van en lugar de esperar eternamente ([punto 4](#)).

4. Carrera sin Lock

```
import threading
contador = 0
def inc():
    global contador
    for _ in range(1000):
        contador += 1
hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador) # ❌ No será 2000
```

Sin Lock, hay condición de carrera. El valor será < 2000 y variará en cada ejecución ([punto 1](#)).

5. Contador protegido con Lock

```
import threading
contador = 0
lock = threading.Lock()
def inc():
    global contador
    for _ in range(1000):
        with lock:
            contador += 1
hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador) # ✅ 2000
```

with lock: garantiza exclusión mutua: el “leer → sumar → escribir” ocurre de principio a fin sin que se cuele el otro hilo ([punto 2](#)).

6. Sección crítica con sleep

```

import threading, time

total = 0
lock = threading.Lock()

def trabajar():
    global total
    for _ in range(20):
        with lock:
            total += 10
        time.sleep(0.1)

hilos = [threading.Thread(target=trabajar) for _ in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Total: {total}") # ✅ 600 (3 hilos x 20 x 10)

```

El Lock protege la sección crítica (`total += 10`); el `sleep` ocurre fuera para no bloquear a los demás.

7. Semáforo simple

```

import threading, time
s = threading.Semaphore(2)
def entrar(id):
    print(f"{id} espera")
    with s:
        print(f" → {id} dentro")
        time.sleep(1)
        print(f" ← {id} sale")
hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()

```

Solo 2 hilos entran a la vez. Los demás esperan hasta que un puesto se libera ([punto 4](#)).

8. Barrera de 3

```
import threading, time
b = threading.Barrier(3)
def corredor(id):
    time.sleep(id)
    print(f" {id} preparado")
    b.wait()
    print(f"🏁 {id} salió")
hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Todos imprimen “preparado” antes de que nadie “salga”: la barrera no deja cruzar hasta que los 3 han llegado ([punto 5](#)).

Boletín U03 — Inicial



Boletín U03 — Inicial

Ejercicios básicos para afianzar los conceptos de sincronización: condición de carrera, Lock, RLock, semáforo y barrera.

1. Lock protege una lista

Crea 2 hilos que añadan elementos a una lista compartida. Uno añade “A” 5 veces, el otro “B” 5 veces. Usa un Lock para evitar mezclas.

2. RLock básico

Crea una función que adquiere un RLock dos veces desde el mismo hilo (llamando a otra función que también lo adquiere). Comprueba que no se produce deadlock.

3. Semáforo con timeout

Crea un Semaphore(2) y lanza 4 hilos que intenten entrar. Cada hilo espera como máximo 1 segundo con `acquire(timeout=1)`. Los que no consigan entrar deben mostrar “timeout”.

Pista: `acquire(timeout=1)` devuelve `True` si consiguió el recurso y `False` si pasó el segundo. Usa ese `True/False` para decidir si el hilo entra o muestra “timeout”.

4. Carrera sin Lock

Crea 2 hilos que incrementen un contador compartido 1000 veces cada uno **sin** usar Lock. Ejecuta el programa varias veces y anota los resultados.

Pista: el contador casi nunca será 2000: esa es la condición de carrera. `contador += 1` no es atómico (leer → sumar → escribir).

5. Contador protegido con Lock

Repite el ejercicio 4 pero protege el incremento con `lock = threading.Lock()` y `with lock:`. ¿Qué valor obtienes ahora?

6. Sección crítica con sleep

Crea una variable compartida `total = 0` y 3 hilos que hagan `total += 10` y luego `time.sleep(0.1)` repetido 20 veces. Protege la sección con un `Lock` y comprueba que al final `total` vale 600.

7. Semáforo simple

Crea un `Semaphore(2)` y lanza 4 hilos que simulen entrar en un recurso (imprime “entra”, duerme 1 segundo, imprime “sale”). Comprueba que solo 2 hilos están dentro a la vez.

8. Barrera de 3

Crea 3 hilos que simulen corredores: cada uno se prepara (duerme un tiempo distinto) y después espera en una `Barrier(3)`. Imprime “ salió” cuando cruzan. Comprueba que los 3 salen a la vez.

Boletín U03 — Avanzado (Resuelto)

Boletín U03 — Avanzado (Resuelto)

1. Condition con notify_all

```
import threading, time

cola = []
condition = threading.Condition()

def consumidor(id):
    for _ in range(1):
        with condition:
            while not cola:
                condition.wait()
            item = cola.pop(0)
            print(f"🦉 Consumidor-{id} procesó: {item}")

def productor():
    for i in range(3):
        with condition:
            cola.append(f"item-{i}")
            print(f"📦 Producido: item-{i}")
            condition.notify_all() # despierta a los 3 consumidores
        time.sleep(0.5)

hilos = [threading.Thread(target=consumidor, args=(i,)) for i in range(3)]
hilos.append(threading.Thread(target=productor))
for h in hilos: h.start()
for h in hilos: h.join()
```

Con 3 consumidores esperando, `notify_all()` los despierta a **todos**; si usáramos `notify()` solo despertaría a uno y los demás podrían quedarse dormidos ([punto 6](#)).

2. Deadlock entre dos hilos

```

import threading, time

lockA = threading.Lock()
lockB = threading.Lock()

def hilo_1():
    with lockA:
        time.sleep(0.1)
        with lockB: # espera a que el hilo 2 libere B
            print("Hilo 1 trabajando")

def hilo_2():
    with lockB:
        time.sleep(0.1)
        with lockA: # espera a que el hilo 1 libere A
            print("Hilo 2 trabajando")

h1 = threading.Thread(target=hilo_1)
h2 = threading.Thread(target=hilo_2)
h1.start(); h2.start()
h1.join(); h2.join() # ❌ nunca termina: deadlock

```

El hilo 1 tiene A y espera B; el hilo 2 tiene B y espera A. **Ningún print sale nunca** y el programa se queda colgado ([punto 8](#)).

3. Semáforo con recursos limitados

```

import threading, time

impresoras = threading.Semaphore(3)

def imprimir(id):
    with impresoras:
        print(f"🖨 Imprimiendo documento-{id}...")
        time.sleep(2)
        print(f"✅ Documento-{id} impreso")

hilos = [threading.Thread(target=imprimir, args=(i,)) for i in range(6)]
for h in hilos: h.start()
for h in hilos: h.join()

```

Con Semaphore(3) nunca hay más de **3 documentos imprimiendo a la vez**, aunque haya 6 hilos esperando ([punto 4](#)).

4. Banco con cuentas compartidas

```
import threading, time, random

saldo = 100
lock = threading.Lock()

def ingresar():
    global saldo
    for _ in range(5):
        with lock:
            cantidad = random.randint(10, 50)
            saldo += cantidad
            print(f"➕ Ingreso {cantidad} € → saldo {saldo} €")
            time.sleep(random.random())

def retirar():
    global saldo
    for _ in range(5):
        with lock:
            cantidad = random.randint(10, 50)
            if saldo >= cantidad:
                saldo -= cantidad
                print(f"➖ Retiro {cantidad} € → saldo {saldo} €")
            else:
                print(f"❌ Saldo insuficiente ({saldo} €) para retirar {cantidad} €")
            time.sleep(random.random())

h1 = threading.Thread(target=ingresar)
h2 = threading.Thread(target=retirar)
h1.start(); h2.start()
h1.join(); h2.join()
print(f"Saldo final: {saldo} €")
```

El Lock global evita que ingreso y retiro se pisen; el retirador comprueba `saldo >= cantidad` dentro del lock antes de restar, así el saldo **nunca es negativo**.

5. 🔍 Semáforo con timeout

```
import threading, time

semaforo = threading.Semaphore(2)

def tarea(id):
    if semaforo.acquire(blocking=True, timeout=2):
        print(f"Hilo-{id}: entró")
        time.sleep(3)
        semaforo.release()
    else:
        print(f"Hilo-{id}: se rinde (timeout 2s)")

hilos = [threading.Thread(target=tarea, args=(i,)) for i in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Con `timeout=2`, los hilos que no consiguen entrar en 2 segundos **se rinden** (devuelve `False`) en vez de bloquearse para siempre. Los primeros 2 entran; el resto abandona.

6. 🧩 Barrera con fases múltiples

```
import threading, time

barrera = threading.Barrier(3)

def trabajador(id):
    for fase in range(3):
        print(f" 🧩 Hilo-{id}: trabajando en fase {fase+1}")
        time.sleep(id * 0.3) # cada hilo tarda distinto
        barrera.wait()      # nadie pasa a la siguiente fase hasta que llegan los 3
        print(f" 🏁 Hilo-{id}: fase {fase+1} completada")

hilos = [threading.Thread(target=trabajador, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Los `barrera.wait()` después de cada fase garantizan que **ningún hilo empieza la fase N+1** hasta que los 3 terminaron la fase N ([punto 5](#)).

7. 🐱🐼 Condition con timeout

```
import threading, time

cola = []
condition = threading.Condition()

def productor():
    for _ in range(2):
        time.sleep(3)          # produce cada 3 segundos
        with condition:
            cola.append("producto")
            print("📦 Producido (cada 3s)")
            condition.notify()

def consumidor():
    for _ in range(2):
        with condition:
            if not condition.wait(timeout=2): # espera máximo 2s
                print("⌚ timeout: el consumidor se va")
            elif cola:
                print(f"🍷 Consumido: {cola.pop(0)}")

h1 = threading.Thread(target=productor)
h2 = threading.Thread(target=consumidor)
h1.start(); h2.start()
h1.join(); h2.join()
```

`wait(timeout=2)` devuelve `False` si expiran los 2 segundos sin notificación: el consumidor **se va** en lugar de esperar los 3 segundos del productor. Devuelve `True` si le avisan a tiempo.

8. 🕒 Deadlock provocado y solución

Provocación (deadlock):


```

import threading, time

lock1 = threading.Lock()
lock2 = threading.Lock()

def hilo_a():
    with lock1:
        time.sleep(0.1)
        with lock2:
            print("A: trabajando")

def hilo_b():
    with lock2:          # ❌ orden invertido
        time.sleep(0.1)
        with lock1:
            print("B: trabajando")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start(); h2.start()
h1.join(); h2.join()   # se queda colgado

```

Solución (mismo orden de locks):

```

def hilo_b():
    with lock1:          # ✅ mismo orden que A: lock1 → lock2
        time.sleep(0.1)
        with lock2:
            print("B: trabajando")

```

Adquiriendo los locks **siempre en el mismo orden** (lock1 → lock2), es imposible que un hilo tenga lock2 esperando lock1 mientras otro tiene lock1 esperando lock2. El programa termina correctamente ([punto 8](#)).

9. Productor-Consumidor múltiple

```

import threading, time, random, queue

buffer = queue.Queue(maxsize=5)

def productor(id):
    for _ in range(4):
        item = f"p{id}-{random.randint(1, 100)}"
        buffer.put(item)          # thread-safe, no necesita Lock
        print(f"📦 P{id} produjo: {item} (buffer {buffer.qsize()}/5)")
        time.sleep(random.random())

def consumidor(id):
    for _ in range(2):
        item = buffer.get()       # thread-safe
        print(f"📦 C{id} consumió: {item} (buffer {buffer.qsize()}/5)")
        time.sleep(random.random())

hilos = [threading.Thread(target=productor, args=(i,)) for i in range(2)]
hilos += [threading.Thread(target=consumidor, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()

```

queue.Queue(maxsize=5) es **thread-safe**: put() y get() ya están sincronizadas internamente. Con maxsize=5, si el buffer se llena, los productores esperan; si se vacía, los consumidores esperan. Sin locks a mano ([punto 7](#)).

Boletín U03 — Avanzado



Boletín U03 — Avanzado

Ejercicios que requieren aplicar los conceptos de sincronización de forma más profunda, con programas completos y varios mecanismos a la vez.

1. Condition con notify_all

Crea 3 hilos consumidores que esperen en una Condition. Un hilo productor añade 3 elementos y usa `notify_all()` para despertar a todos los consumidores a la vez.

2. Deadlock entre dos hilos

Crea 2 hilos y 2 locks (A y B). El hilo 1 adquiere A luego B. El hilo 2 adquiere B luego A. Añade `time.sleep(0.1)` entre adquisiciones para provocar el deadlock.

3. Semáforo con recursos limitados

Simula 3 impresoras compartidas. Crea 6 hilos que necesiten una impresora para imprimir (simulado con `sleep`). Usa `Semaphore(3)` para que solo 3 impriman a la vez.

4. Banco con cuentas compartidas

Dos hilos: uno ingresa dinero, otro retira. Usa Lock para evitar saldo negativo.

Pista: Usa un Lock global. Cada hilo lo adquiere con `with lock`: antes de modificar el saldo. El hilo retirador debe comprobar `if saldo >= cantidad` antes de restar.

5. Semáforo con timeout

Crea un semáforo donde los hilos esperen máximo 2s. Si no entran, se van.

Pista: `semaforo.acquire(blocking=True, timeout=2)` devuelve `True` si consiguió el recurso, o `False` si pasaron los 2 segundos. Úsalo para decidir si el hilo entra o se rinde.

6. 🧩 Barrera con fases múltiples

Crea 3 hilos que trabajen en 3 fases. La fase siguiente no empieza hasta que todos terminan la anterior.

Pista: Coloca `barrera.wait()` justo después del trabajo de cada fase. Así ningún hilo avanza a la siguiente fase hasta que todos han llegado al `wait()`.

7. 😬🕒 Condition con timeout

Un productor que produce cada 3s, consumidor que espera máximo 2s. Si no hay producto, se va.

Pista: `cond.wait(timeout=2)` devuelve `False` si el tiempo expiró sin ser notificado. Úsalo para que el consumidor salga si el productor tarda demasiado.

8. 🕒 Deadlock provocado y solución

Crea un deadlock con 2 hilos y 2 locks. Luego arréglalo adquiriendo los locks en el mismo orden.

Pista: El deadlock ocurre cuando el hilo A toma `lock1` y espera `lock2`, mientras B toma `lock2` y espera `lock1`. La solución: ambos hilos adquieren los locks en el mismo orden (`lock1 → lock2`).

9. 🏗️ Productor-Consumidor múltiple

2 productores y 3 consumidores compitiendo por un buffer de tamaño 5.

Pista: Usa `queue.Queue(maxsize=5)` como buffer compartido: es thread-safe y no necesita Lock. Los productores llaman a `put()` y los consumidores a `get()`.

Boletín U04 — Inicial (Resuelto)



Boletín U04 — Inicial (Resuelto)

1. Servidor saludo

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.sendall(b"Bienvenido al servidor")
```

`accept()` espera un cliente (bloqueante). Al salir del `with`, la conexión se cierra solo.

2. Cliente que envía y recibe

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Hola")
    respuesta = cli.recv(1024)
    print(f"Respuesta: {respuesta.decode()}")
```

`connect()` lanza `ConnectionRefusedError` si no hay servidor.

3. Servidor contador

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        numero = int(conn.recv(1024).decode())
        conn.sendall(str(numero + 1).encode())
```

Se recibe el número como **texto**, se convierte con `int()`, se incrementa y se devuelve convertido de nuevo a texto con `str()`. El cliente verá "4" si envió "3".

4. Servidor eco

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos) # Eco
```

Devuelve exactamente lo mismo que recibe.

5. Cliente mínimo

```
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Hola servidor")
```

Sin `recv()`, el cliente solo envía. El `with` cierra el socket al salir.

6. Servidor mínimo

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        print(f"Recibido: {datos.decode()}")
```

`accept()` espera un cliente (bloqueante).

7. IP y puerto

- a) La **IP** identifica la **máquina** en la red; el **puerto** identifica el **programa** dentro de esa máquina.
- b) **127.0.0.1** es **localhost**: tu propia máquina. Se usa para probar sin red real.
- c) **TCP** → `socket.SOCK_STREAM`; **UDP** → `socket.SOCK_DGRAM`. Ambos con familia `AF_INET`.

8. Servidor hora

```
import socket, time
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        hora = time.strftime("%H:%M:%S")
        conn.sendall(hora.encode())
```

La hora se obtiene con `time.strftime("%H:%M:%S")` y se envía como bytes con `.encode()`.

 [Volver a la unidad](#) · Por resolver:  [Boletín U04 — Inicial](#)

Boletín U04 — Inicial



Boletín U04 — Inicial

Ejercicios básicos para afianzar los conceptos de sockets TCP: cliente, servidor, eco y la pareja IP/puerto de la unidad U04.

1. Servidor saludo

Crea un servidor TCP que, cuando un cliente se conecte, le envíe “Bienvenido al servidor” y luego cierre la conexión.

2. Cliente que envía y recibe

Crea un cliente que se conecte a 127.0.0.1:9000, envíe “Hola” y luego espere y muestre la respuesta del servidor.

3. Servidor contador

Crea un servidor que acepte una conexión, reciba un número como texto, lo convierta a entero, lo incremente en 1 y devuelva el resultado.

4. Servidor eco

Crea un servidor TCP que acepte una conexión, reciba un mensaje y devuelva exactamente lo mismo que recibe.

5. Cliente mínimo

Crea un cliente TCP que se conecte a 127.0.0.1:9000 y envíe `b"Hola servidor"`. No necesita esperar respuesta: con `connect()` y `with` basta.

6. Servidor mínimo

Crea un servidor TCP que escuche en 127.0.0.1:9000, acepte una conexión, reciba datos y los imprima por pantalla.

7. IP y puerto

Responde por escrito:

a) ¿Qué identifica la IP y qué identifica el puerto en una comunicación de red? b) ¿Qué significa 127.0.0.1 y para qué se usa? c) ¿Qué constante de Python crea un socket TCP y cuál un socket UDP?

Pista: repasa el punto 1 de la unidad: la IP identifica la **máquina**, el puerto identifica el **programa**; 127.0.0.1 es **localhost**; y los tipos de socket son `SOCK_STREAM` (TCP) y `SOCK_DGRAM` (UDP).

8. Servidor hora

Crea un servidor TCP que, cuando un cliente se conecte, le devuelva la hora actual con formato HH:MM:SS y cierre la conexión.

Pista: usa `time.strftime("%H:%M:%S")` para obtener la hora y envía el texto con `.encode()`.



[Volver a la unidad](#) · Resuelto:  [Boletín U04 — Inicial \(Resuelto\)](#)

Boletín U04 — Avanzado (Resuelto)

Boletín U04 — Avanzado (Resuelto)

1. Servidor que cuenta caracteres

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        texto = conn.recv(1024).decode()
        conn.sendall(str(len(texto)).encode())
```

`len(texto)` cuenta los caracteres del texto recibido y se devuelve como texto (`str`) convertido a bytes con `.encode()`. Si el cliente envía "ho1a", recibe "4".

2. Cliente interactivo

```
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    while True:
        mensaje = input("Escribe (o 'salir'): ")
        if mensaje == "salir":
            break
        cli.sendall(mensaje.encode())
        respuesta = cli.recv(1024).decode()
        print(f"Respuesta: {respuesta}")
```

El `while True` mantiene el diálogo abierto. Cuando el usuario escribe "salir", el `break` rompe el bucle y el `with` cierra el socket.

3. Servidor que registra IP

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        direccion = conn.getpeername()
        print(f"Cliente conectado: {direccion}")
        conn.sendall(str(direccion).encode())
```

`conn.getpeername()` devuelve la tupla (IP, puerto) del extremo remoto (el cliente). Se imprime en el servidor y se devuelve al cliente convertida a texto.

4. Servidor de mayúsculas

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        texto = conn.recv(1024).decode()
        conn.sendall(texto.upper().encode())
```

`texto.upper()` convierte el texto recibido a MAYÚSCULAS y se devuelve como bytes con `.encode()`.

5. Cliente con reconexión

```

import socket, time

host = "127.0.0.1"
port = 9000

for intento in range(3):
    try:
        with socket.socket() as cli:
            cli.settimeout(2)
            cli.connect((host, port))
            cli.sendall(b"test")
            print(f"Conectado en el intento {intento + 1}")
            break
    except (ConnectionRefusedError, socket.timeout):
        print(f"Intento {intento + 1} fallido, esperando 2s...")
        time.sleep(2)
else:
    print("Servidor no disponible tras 3 intentos")

```

El for reintenta hasta 3 veces. Cada except captura el fallo (rechazo o timeout) y espera 2s con `time.sleep(2)`. El break sale si se conecta; el else del for se ejecuta solo si nunca hubo break.

6. Servidor que gestiona múltiples conexiones (sin hilos)

```

import socket, select

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 9000))
servidor.listen()
servidor.setblocking(False) # no bloqueante

clientes = []
print("Servidor atendiendo hasta 3 clientes con select...")

while True:
    lectura, _, _ = select.select([servidor] + clientes, [], [], 1.0)

    for sock in lectura:
        if sock is servidor:
            # El servidor está listo: acepta un nuevo cliente
            conn, addr = servidor.accept()
            conn.setblocking(False)
            clientes.append(conn)
            print(f"Conectado {addr}, hay {len(clientes)} cliente(s)")
        else:
            # Un cliente envió datos
            datos = sock.recv(1024)
            if datos:
                sock.sendall(datos) # eco
            else:
                clientes.remove(sock)
                sock.close()

```

El servidor **no bloqueante** más `select.select()` permite esperar a la vez en el socket servidor y en los sockets de los clientes. Cuando el socket listo es el servidor, se acepta una conexión nueva; cuando es un cliente, se leen sus datos. Sin hilos: un solo hilo atiende a todos.

7. Calculadora remota

```

import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        expr = conn.recv(1024).decode()
        try:
            resultado = eval(expr)
            conn.sendall(f"Resultado: {resultado}".encode())
        except Exception as e:
            conn.sendall(f"Error: {e}".encode())

```

`eval("5+3")` devuelve 8. El `try/except` captura divisiones entre cero o sintaxis inválida y devuelve el error al cliente en lugar de romper el servidor.

⚠ **Ojo:** `eval()` ejecuta código arbitrario: solo úsalo en ejercicios locales de confianza, nunca con entradas de usuarios desconocidos en producción.

8. Timeout personalizado

```

import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.settimeout(10)
        try:
            datos = conn.recv(1024)
            print(f"Recibido: {datos.decode()}")
        except socket.timeout:
            conn.sendall(b"Hasta luego")
            print("Cliente inactivo, cerrando conexion")

```

`conn.settimeout(10)` da al cliente 10 segundos para enviar datos. Si no lo hace, `recv()` lanza `socket.timeout` y el servidor se despide antes de cerrar.

9. Servidor de chat simple

```
import socket, threading

clientes = []
lock = threading.Lock()

def atender(conn, addr):
    print(f"Conectado: {addr}")
    with conn:
        while True:
            try:
                datos = conn.recv(1024)
            except ConnectionResetError:
                break
            if not datos:
                break
            mensaje = f"{addr}: {datos.decode()}".encode()
            with lock:
                for c in clientes:
                    if c is not conn:
                        c.sendall(mensaje)

    with lock:
        clientes.remove(conn)
    conn.close()
    print(f"Desconectado: {addr}")

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    print("Servidor de chat escuchando...")
    while True:
        conn, addr = srv.accept()
        with lock:
            clientes.append(conn)
        threading.Thread(target=atender, args=(conn, addr), daemon=True).start()
```

Un **hilo por cliente** (`threading.Thread`) permite atender a varios a la vez. Cada hilo espera en su propio `recv()` y, al recibir un mensaje, recorre la lista global de conexiones reenviándolo a **todos menos al emisor**. El **Lock** protege la lista cuando varios hilos la modifican a la vez.

Boletín U04 — Avanzado



Boletín U04 — Avanzado

Ejercicios que requieren aplicar los conceptos de sockets TCP de forma más profunda, con programas completos: clientes interactivos, `select()`, timeouts y servidores multiusuario.

1. Servidor que cuenta caracteres

Crea un servidor que reciba un texto y devuelva el número de caracteres (ej: “hola” → “4”).

Pista: recibe con `recv(1024).decode()`, calcula `len(texto)` y envía el resultado con `str(...).encode()`.

2. Cliente interactivo

Crea un cliente que pida texto por teclado con `input()`, lo envíe al servidor y muestre la respuesta. El bucle termina cuando el usuario escribe “salir”.

Pista: envuelve el `input() + sendall() + recv()` en un `while True` y rompe el bucle con `break` cuando el texto sea “salir”.

3. Servidor que registra IP

Crea un servidor que, al recibir una conexión, muestre la dirección IP y el puerto del cliente usando `conn.getpeername()`, y luego devuelva esos datos al cliente.

Pista: `getpeername()` devuelve una tupla (IP, puerto). Envíala convertida a texto con `str(direccion).encode()`.

4. Servidor de mayúsculas

El cliente envía texto, el servidor lo devuelve en MAYÚSCULAS.

Pista: recibe los datos con `recv(1024).decode()`, aplica `.upper()` y envía el resultado con `sendall(...)`.

5. Cliente con reconexión

Cliente que intenta conectar, y si falla, reintenta hasta 3 veces con 2s de espera.

Pista: envuelve `socket.connect()` en un bucle `for` con `try/except`. Captura `ConnectionRefusedError` y `socket.timeout`, espera 2s con `time.sleep(2)` y reintenta.

6. Servidor que gestiona múltiples conexiones (sin hilos)

Usa `select.select()` para atender hasta 3 clientes en un solo hilo.

Pista: configura el socket servidor como no bloqueante con `setblocking(False)`. `select.select()` te devuelve los sockets que tienen datos listos para leer. Si el socket listo es el servidor, acepta una nueva conexión; si es un cliente, recibe datos.

7. Calculadora remota

El cliente envía “5+3”, el servidor calcula y responde “Resultado: 8”.

Pista: usa `eval(expr)` para evaluar la expresión recibida. Envuelve en `try/except` para capturar errores (por ejemplo, división entre cero o sintaxis inválida).

8. Timeout personalizado

Crea un servidor que cierre la conexión si el cliente no envía datos en 10 segundos.

Pista: después de `accept()`, llama a `conn.settimeout(10)`. Captura `socket.timeout` y envía un mensaje de despedida antes de cerrar.

9. Servidor de chat simple

Un servidor que recibe mensajes de un cliente y los reenvía a todos los demás.

Pista: mantén una lista global de conexiones. Usa un `Lock` al modificar la lista. Cuando un socket recibe datos, recorre la lista y reenvía con `sendall()` a todos menos al emisor. Usa un hilo por cliente con `threading.Thread`.

Boletín U05 — Inicial (Resuelto)



Boletín U05 — Inicial (Resuelto)

1. Cliente UDP con entrada de usuario

```
import socket

HOST = "127.0.0.1"
PORT = 9001

mensaje = input("Escribe tu mensaje: ")
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(mensaje.encode(), (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

El cliente UDP no tiene `connect()`: la dirección va dentro del `sendto()`, y la respuesta llega con `recvfrom()`.

2. Servidor UDP con eco personalizado

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")
    srv.sendto(f"Recibido: {datos.decode()}".encode(), direccion)
```

Se responde con la **dirección** que devuelve `recvfrom()`: sin ella no hay forma de contestar en UDP.

3. Servidor UDP contador

```
import socket

contador = 0

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    while True:
        datos, direccion = srv.recvfrom(1024)
        contador += 1
        srv.sendto(f"Mensaje #{contador}".encode(), direccion)
```

El contador vive **fuera** del bucle para que no se reinicie con cada mensaje. Cada respuesta lleva su número de orden.

4. Cliente UDP mínimo

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    cli.sendto(b"Hola UDP", ("127.0.0.1", 9001))
```

UDP no tiene `connect()`. Directamente `sendto()`.

5. Servidor UDP mínimo

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    print(f"De {direccion}: {datos.decode()}")
```

UDP no tiene `accept()`. Solo `recvfrom()`, que entrega datos y dirección del emisor.

6. Servidor UDP eco

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    srv.sendto(datos, direccion)
```

Eco puro: `sendto(datos, direccion)` reenvía al cliente **exactamente** lo que llegó.

7. TCP vs UDP: clasifica

a) **TCP**: Web (HTTP), Correo (SMTP), Transferencia de archivos (FTP) → el dato debe llegar **completo y en orden**. **UDP**: Videollamada (Zoom), Juego online (Fortnite), DNS → la **velocidad** importa más; perder un paquete se tolera o se repite.

b)

Característica	TCP	UDP
Conexión	Sí (handshake)	No
Entrega garantizada	Sí	No
Orden	Sí	No
Velocidad	Más lento	Más rápido

8. Cliente HTTP manual

```

import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("www.example.com", 80))
    petition = (
        "GET / HTTP/1.1\r\n"
        "Host: www.example.com\r\n"
        "Connection: close\r\n"
        "\r\n"
    )
    s.sendall(petition.encode())

    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

    print(respuesta.decode()[:500])

```

HTTP es texto sobre TCP: mandas una petición con `sendall()` y recibes la respuesta en trozos hasta que `recv()` devuelve `b""` (el servidor cerró la conexión gracias a `Connection: close`).

Boletín U05 — Inicial



Boletín U05 — Inicial

Ejercicios básicos para afianzar los conceptos de UDP (cliente, servidor, eco), TCP vs UDP y el HTTP manual de la unidad U05.

1. Cliente UDP con entrada de usuario

Crea un cliente UDP que pida un mensaje al usuario por teclado con `input()`, lo envíe a `127.0.0.1:9001` y espere una respuesta.

2. Servidor UDP con eco personalizado

Crea un servidor UDP que escuche en `127.0.0.1:9001`. Al recibir un mensaje, responda con "Recibido: " seguido del mensaje original.

3. Servidor UDP contador

Crea un servidor UDP que lleve la cuenta de cuántos mensajes ha recibido. Al responder, incluya el número de mensaje (ej: "Mensaje #1", "Mensaje #2").

Pista: declara un contador fuera del bucle `while True:` e increméntalo en cada `recvfrom()`. La respuesta se construye con un f-string.

4. Cliente UDP mínimo

Crea un cliente UDP que envíe `b"Hola UDP"` a `127.0.0.1:9001`. No necesitas esperar respuesta: con `sendto()` y `with` basta.

5. Servidor UDP mínimo

Crea un servidor UDP que escuche en `127.0.0.1:9001`, reciba un datagrama y lo imprima con la dirección de quien lo envió.

6. Servidor UDP eco

Crea un servidor UDP que escuche en 127.0.0.1:9001 y devuelva al cliente **lo mismo** que recibe.

7. TCP vs UDP: clasifica

a) Clasifica cada aplicación como TCP o UDP y justifica brevemente:

- Web (HTTP)
- Videollamada (Zoom)
- Correo (SMTP)
- Juego online (Fortnite)
- Transferencia de archivos (FTP)
- DNS

b) Completa la tabla:

Característica	TCP	UDP
Conexión		
Entrega garantizada		
Orden		
Velocidad		

8. Cliente HTTP manual

Conéctate con un socket TCP a `www.example.com:80`, haz un GET a `/` y muestra los primeros 500 caracteres de la respuesta.

Pista: envía `"GET / HTTP/1.1\r\nHost: www.example.com\r\nConnection: close\r\n\r\n"` con `sendall()`. Recibe en bucle con `recv(4096)` acumulando bytes hasta que devuelva `b""`, y entonces decodifica.

Boletín U05 — Avanzado (Resuelto)

Boletín U05 — Avanzado (Resuelto)

1. Cliente NTP manual

```
import socket, struct, time

def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        # Paquete NTP: 48 bytes, modo cliente
        paquete = b'\x1b' + 47 * b'\0'
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)

        # El timestamp está en los bytes 40-43
        t = struct.unpack('!I', datos[40:44])[0]
        # Ajustar época NTP (1900) a Unix (1970)
        return t - 2208988800

hora = hora_ntp()
print(f"Hora NTP oficial: {time.ctime(hora)}")
```

El `settimeout(5)` evita que el `recvfrom()` se quede bloqueado si el datagrama de respuesta se pierde (muy UDP). El timestamp se desempaqueta de los bytes 40-43 con `struct.unpack('!I', ...)` y se ajusta restando **2208988800** segundos de época.

2. Servidor UDP multimensaje

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    for i in range(1, 4):
        datos, direccion = srv.recvfrom(1024)
        print(f"Mensaje {i} de {direccion}: {datos.decode()}")
        srv.sendto(f"OK #{i}".encode(), direccion)
```

Con `for i in range(1, 4)` el servidor responde a 3 mensajes y se cierra solo al salir del bucle (el `with` libera el socket). Cada respuesta lleva su número de orden.

3. Cliente HTTP con parseo de cabeceras

```
import socket

with socket.socket() as s:
    s.connect(("example.com", 80))
    s.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n")
    resp = b""
    while True:
        d = s.recv(4096)
        if not d:
            break
        resp += d

cabeceras, _, cuerpo = resp.decode(errors="replace").partition("\r\n\r\n")
for linea in cabeceras.split("\r\n"):
    if linea.startswith(("Content-Type:", "Content-Length:")):
        print(linea)
```

`partition("\r\n\r\n")` separa las cabeceras del cuerpo. Buscando las líneas que empiezan por `Content-Type:` y `Content-Length:` extraes justo los dos valores que pide el ejercicio.

4. Cliente HTTP manual a /ip

```
import socket

with socket.socket() as s:
    s.connect(("httpbin.org", 80))
    s.sendall(b"GET /ip HTTP/1.1\r\nHost: httpbin.org\r\nConnection: close\r\n\r\n")
    resp = b""
    while True:
        d = s.recv(4096)
        if not d: break
        resp += d
    print(resp.decode()[ :500])
```

HTTP es texto sobre TCP. Mandas una petición y recibes una respuesta: en el cuerpo de /ip viene tu IP pública en JSON.

5. Compara TCP y UDP

```
import socket, time
def test_tcp():
    t = time.time()
    for _ in range(10):
        with socket.socket() as s:
            s.connect(("127.0.0.1", 9000))
            s.send(b"x")
            s.recv(1024)
    return time.time() - t
def test_udp():
    t = time.time()
    for _ in range(10):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.sendto(b"x", ("127.0.0.1", 9001))
            s.recvfrom(1024)
    return time.time() - t
print(f"TCP: {test_tcp():.3f}s")
print(f"UDP: {test_udp():.3f}s")
```

UDP suele ser más rápido porque no tiene handshake: cada intercambio TCP paga un `connect()` (three-way handshake) que UDP se ahorra.

6. Mini servidor web

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 8080))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.recv(1024) # Leer petición (la ignoramos)
        respuesta = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<h1>Hola mundo</h1>"
        conn.sendall(respuesta.encode())
```

Abre <http://127.0.0.1:8080> en tu navegador y verás “Hola mundo”. El `SO_REUSEADDR` te permite relanzar el servidor sin esperar a que el puerto se libere.

7. Ping UDP

```
import socket, time, threading

def servidor():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
        srv.bind(("127.0.0.1", 9001))
        datos, direccion = srv.recvfrom(1024)
        srv.sendto(b"PONG", direccion)

threading.Thread(target=servidor, daemon=True).start()

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    inicio = time.time()
    cli.sendto(b"PING", ("127.0.0.1", 9001))
    datos, _ = cli.recvfrom(1024)
    fin = time.time()

print(f"PONG recibido en {fin - inicio:.4f} segundos")
```

El servidor se ejecuta en un hilo daemon mientras el cliente mide el tiempo de ida y vuelta. Ese tiempo es el **RTT** (round-trip time), la métrica de latencia de las redes.

8. Broadcast UDP

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind(("0.0.0.0", 9001))
    print("Servidor escuchando en todas las interfaces...")
    while True:
        datos, direccion = servidor.recvfrom(1024)
        print(f"Datagrama de {direccion}: {datos.decode()}")
        servidor.sendto(b"Recibido!", direccion)
```

"0.0.0.0" significa "cualquier interfaz": el servidor acepta datagramas de cualquier cliente. El bucle infinito atiende a todos los que lleguen, uno tras otro, respondiendo con la dirección de cada uno.

9. HTTP desde cero con parseo

```
import socket

with socket.socket() as s:
    s.connect(("example.com", 80))
    s.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n")
    respuesta = b""
    while True:
        d = s.recv(4096)
        if not d: break
        respuesta += d

texto = respuesta.decode(errors="replace")
cabeceras, _, cuerpo = texto.partition("\r\n\r\n")
lineas = cabeceras.split("\r\n")
print(f"Código de estado: {lineas[0]}")
for linea in lineas[1:]:
    print(f" {linea}")
print(f"Cuerpo: {len(cuerpo)} caracteres")
```

`partition("\r\n\r\n")` separa cabeceras del cuerpo: la **primera línea** de las cabeceras es el código de estado (HTTP/1.1 200 OK), y el resto son las cabeceras Nombre: valor.

10. Mini navegador web

```

import socket

def descargar_html(url):
    host = url.split("/", 1)[0] # "www.example.com"
    ruta = "/" + url.split("/", 1)[1] if "/" in url else "/"

    with socket.socket() as s:
        s.connect((host, 80))
        s.sendall(f"GET {ruta} HTTP/1.1\r\nHost: {host}\r\nConnection:
close\r\n\r\n".encode())
        respuesta = b""
        while True:
            d = s.recv(4096)
            if not d:
                break
            respuesta += d

        cabeceras, _, cuerpo = respuesta.partition(b"\r\n\r\n")
        return cuerpo.decode(errors="replace")

html = descargar_html("www.example.com/")
with open("pagina.html", "w", encoding="utf-8") as f:
    f.write(html)
print(f"Guardados {len(html)} caracteres de HTML en pagina.html")

```

Se extrae el **host** (la parte antes de la primera /) y la **ruta** (el resto). Tras recibir toda la respuesta, `partition(b"\r\n\r\n")` separa cabeceras del cuerpo y solo se guarda el cuerpo en el archivo.

11. Comparativa TCP vs UDP (100 mensajes)

```

import socket, time, threading

def servidor_tcp():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 9000))
        srv.listen()
        for _ in range(100):
            conn, _ = srv.accept()
            with conn:
                conn.recv(1024)
                conn.send(b"x")

def servidor_udp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
        srv.bind(("127.0.0.1", 9001))
        for _ in range(100):
            datos, direccion = srv.recvfrom(1024)
            srv.sendto(datos, direccion)

threading.Thread(target=servidor_tcp, daemon=True).start()
threading.Thread(target=servidor_udp, daemon=True).start()
time.sleep(0.2) # dar tiempo a que arranquen los servidores

def test_tcp():
    t = time.time()
    for _ in range(100):
        with socket.socket() as s:
            s.connect(("127.0.0.1", 9000))
            s.send(b"x")
            s.recv(1024)
    return time.time() - t

def test_udp():
    t = time.time()
    for _ in range(100):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.sendto(b"x", ("127.0.0.1", 9001))
            s.recvfrom(1024)
    return time.time() - t

print(f"TCP: {test_tcp():.3f}s")

```



```
print(f"UDP: {test_udp():.3f}s")
```

Ambos servidores se ejecutan en hilos mientras el cliente mide 100 intercambios. El servidor TCP exige `accept()` por cada conexión (handshake incluido); el UDP solo `recvfrom()` + `sendto()`. La diferencia de tiempo es el coste de la fiabilidad de TCP.

12. Servidor HTTP simple

```
import socket

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 8080))
    srv.listen()
    print("Servidor HTTP escuchando en http://127.0.0.1:8080")
    while True:
        conn, addr = srv.accept()
        with conn:
            peticion = conn.recv(1024).decode(errors="replace")
            primera_linea = peticion.split("\r\n")[0]
            partes = primera_linea.split(" ")
            ruta = partes[1] if len(partes) > 1 else "/"
            print(f"Petición de {addr}: {primera_linea}")

            if ruta == "/":
                cuerpo, estado = "<h1>Bienvenido a mi servidor</h1>", "200 OK"
            elif ruta == "/acerca":
                cuerpo, estado = "<h1>Sobre este servidor</h1>", "200 OK"
            else:
                cuerpo, estado = "<h1>404 – Página no encontrada</h1>", "404 Not Found"

            respuesta = (
                f"HTTP/1.1 {estado}\r\n"
                "Content-Type: text/html\r\n"
                f"Content-Length: {len(cuerpo.encode())}\r\n"
                "\r\n" + cuerpo
            )
            conn.sendall(respuesta.encode())
```

Se lee la primera línea de la petición (GET /ruta HTTP/1.1) y se extrae la **ruta** (segunda palabra). Según la ruta se sirve distinto HTML con su código de estado (200 OK o 404 Not Found) y la cabecera Content-Length con el tamaño exacto del cuerpo.

Boletín U05 — Avanzado



Boletín U05 — Avanzado

Ejercicios que requieren aplicar los conceptos de UDP, NTP, HTTP y la comparativa TCP/UDP de forma más profunda, con programas completos.

1. Cliente NTP manual

Crea un cliente UDP que obtenga la hora actual desde `pool.ntp.org` usando el puerto 123. Envía un paquete de 48 bytes (el primero con valor `\x1b` y el resto `\0`). Extrae el timestamp de los bytes 40 a 43 con `struct.unpack('!I', ...)` y ajústalo restando 2208988800 para convertirlo a hora Unix.

Pista: añade `s.settimeout(5)` antes del `recvfrom()`: si la respuesta se pierde (UDP), lanza una excepción en lugar de bloquearse para siempre.

2. Servidor UDP multimensaje

Crea un servidor UDP que reciba y responda a 3 mensajes consecutivos en un bucle antes de cerrarse. Cada respuesta debe incluir el número de orden: "OK #1", "OK #2", "OK #3".

Pista: usa `for i in range(1, 4)` en lugar de `while True`: así el servidor se cierra solo tras la tercera respuesta. El `with` libera el socket al salir.

3. Cliente HTTP con parseo de cabeceras

Conéctate con un socket TCP a `example.com:80` y haz un GET a `/`. Una vez recibida la respuesta completa, parsea las cabeceras y muestra los valores de `Content-Type` y `Content-Length`.

Pista: separa cabeceras de cuerpo con `partition(b"\r\n\r\n")`. Recorre las líneas de las cabeceras buscando las que empiecen por `Content-Type:` y `Content-Length:`.

4. Cliente HTTP manual a /ip

Conéctate con un socket TCP a `httpbin.org:80` y haz un GET a `/ip`. Muestra los primeros 500 caracteres de la respuesta. Verás tu IP pública en el cuerpo.

Pista: envía `b"GET /ip HTTP/1.1\r\nHost: httpbin.org\r\nConnection: close\r\n\r\n"` y recibe en bucle con `recv(4096)` hasta que devuelva `b""`.

5. Compara TCP y UDP

Escribe un programa que mida cuánto tarda en completar 10 intercambios de mensajes contra un servidor TCP y contra un servidor UDP en local. Compara los tiempos.

Pista: en TCP cada intercambio exige `connect()` (handshake); en UDP basta un `sendto()` + `recvfrom()`. Mide con `time.time()` antes y después de cada bucle.

6. Mini servidor web

Crea un servidor TCP que escuche en `127.0.0.1:8080`, acepte una conexión, lea la petición (ignorándola) y responda con `HTTP/1.1 200 OK` y un HTML con `"<h1>Hola mundo</h1>"`.

Pista: usa `SO_REUSEADDR` con `setsockopt()` para poder relanzar el servidor sin esperar. Abre `http://127.0.0.1:8080` en el navegador para verlo.

7. Ping UDP

Cliente manda "PING", servidor responde "PONG". Mide cuánto tarda.

Pista: Necesitas dos funciones (servidor y cliente) ejecutándose en paralelo. Usa `threading.Thread` con `daemon=True` para lanzar el servidor. Mide el tiempo con `time.time()` antes y después del intercambio de mensajes.

8. Broadcast UDP

El servidor escucha en todas las interfaces y responde a cualquiera.

Pista: El servidor debe escuchar en `"0.0.0.0"` para aceptar conexiones de cualquier interfaz. Usa un bucle infinito con `recvfrom()` y responde con `sendto()` a la dirección de cada cliente.

9. HTTP desde cero con parseo

Cliente HTTP manual que parsea el código de estado y las cabeceras.

Pista: Después de recibir la respuesta HTTP completa, separa las cabeceras del cuerpo con `partition("\r\n\r\n")`. La primera línea de las cabeceras contiene el código de estado (ej: `HTTP/1.1 200 OK`).

10. Mini navegador web

Crea una función que descargue el HTML de una URL y lo guarde en un archivo.

Pista: Extrae el host y la ruta de la URL. Conéctate al puerto 80 del host, envía un GET con `Connection: close`. Tras recibir toda la respuesta, separa el cuerpo de las cabeceras con `partition(b"\r\n\r\n")` y escribe el cuerpo a un archivo.

11. Comparativa TCP vs UDP

Mide el tiempo de 100 mensajes con TCP y con UDP en local.

Pista: Necesitas servidores TCP y UDP separados ejecutándose en hilos. El servidor TCP requiere `accept()` por cada mensaje; el UDP solo `recvfrom()`. Mide el tiempo total para 100 intercambios en cada protocolo y compara.

12. Servidor HTTP simple

Crea un servidor TCP que entienda peticiones HTTP GET y sirva respuestas.

Pista: Con `accept()` obtienes la conexión. Lee la petición con `recv()` y parsea la primera línea (`GET /ruta HTTP/1.1`). Según la ruta, devuelve distinto contenido HTML con cabeceras HTTP válidas incluyendo `Content-Length`.

Boletín U06 — Inicial (Resuelto)



Boletín U06 — Inicial (Resuelto)

1. GET a JSONPlaceholder

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code) # 200
```

`requests.get` no lanza excepción ante errores: el código de estado te lo dice `resp.status_code`.

2. Título del post

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")
print(resp.json()["title"])
```

`resp.json()` convierte el cuerpo JSON a un **dict** de Python, y con `["title"]` accedes al campo.

3. Contar usuarios

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/users")
usuarios = resp.json()
print(f"Hay {len(usuarios)} usuarios") # 10
```

Cuando la respuesta es una lista de recursos, `resp.json()` devuelve una **lista** de diccionarios y `len()` cuenta los elementos.

4. GET a GitHub

```
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.status_code) # 200
```

5. Mostrar JSON

```
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.json())
```

`resp.json()` convierte el JSON a diccionario Python.

6. Nombre real

```
import requests
datos = requests.get("https://api.github.com/users/python").json()
print(datos["name"]) # "Python"
```

Encadenas `requests.get(...).json()` para ir directo al dict, y con `datos["name"]` sacas el nombre real.

7. Métodos HTTP

Método	Operación CRUD	Uso
GET	Read	Obtener datos (sin cuerpo)
POST	Create	Crear recurso (con cuerpo)
PUT	Update	Reemplazar el recurso completo
PATCH	Update	Actualizar solo una parte
DELETE	Delete	Borrar el recurso

8. Códigos de estado

a) 200 → **OK** (GET exitoso) b) 201 → **Created** (POST exitoso, recurso creado) c) 204 → **No Content** (DELETE exitoso) d) 400 → **Bad Request** (enviaste datos incorrectos) e) 401 → **Unauthorized** (falta autenticación) f) 404 → **Not Found** (el recurso no existe) g) 500 → **Internal Server Error** (error del servidor)

Los **2xx** son éxito, los **4xx** son error del cliente (tuyo) y los **5xx** son error del servidor.

Boletín U06 — Inicial



Boletín U06 — Inicial

Ejercicios básicos para afianzar los conceptos de APIs REST, métodos HTTP, códigos de estado y JSON de la unidad U06.

1. GET a JSONPlaceholder

Usa `requests.get` para obtener el post con ID 1 de `https://jsonplaceholder.typicode.com/posts/1`. Muestra el `status_code`.

2. Título del post

Del ejercicio anterior, muestra el campo "title" del post obtenido usando `resp.json()`.

3. Contar usuarios

Haz GET a `https://jsonplaceholder.typicode.com/users` y muestra cuántos usuarios hay (la longitud de la lista).

Pista: `resp.json()` devuelve una lista de diccionarios; su longitud es el número de usuarios.

4. GET a GitHub

Usa `requests.get` para obtener los datos del usuario python de `https://api.github.com/users/python`. Muestra el `status_code`.

5. Mostrar JSON

Del ejercicio anterior, muestra la respuesta completa con `print(resp.json())`.

6. Nombre real

Del mismo usuario, muestra el campo "name" (su nombre real) accediendo al diccionario.

7. Métodos HTTP

Relaciona cada método HTTP con su operación CRUD y su uso:

Método	Operación CRUD	Uso
GET	---	---
POST	---	---
PUT	---	---
PATCH	---	---
DELETE	---	---

Pista: CRUD = Create, Read, Update, Delete. Recuerda que PUT reemplaza el recurso completo y PATCH solo una parte.

8. Códigos de estado

Completa el significado de estos códigos de estado HTTP:

a) 200 → ___ b) 201 → ___ c) 204 → ___ d) 400 → ___ e) 401 → ___ f) 404 → ___ g) 500 → ___

Pista: 2xx es éxito, 4xx es error del cliente, 5xx es error del servidor.

Boletín U06 — Avanzado (Resuelto)

Boletín U06 — Avanzado (Resuelto)

1. PUT para actualizar

```
import requests
resp = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}
)
print(resp.status_code)          # 200 OK
print(resp.json()["title"])       # "nuevo titulo"
```

`requests.put` con `json=` reemplaza el recurso **completo**. PUT es **idempotente**: repetirlo deja el recurso en el mismo estado.

2. DELETE un recurso

```
import requests
resp = requests.delete("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code)          # 200 OK
```

JSONPlaceholder responde **200** con cuerpo vacío. En REST canónico, un borrado exitoso devuelve **204 No Content** (sin cuerpo que leer). DELETE también es **idempotente**: da igual cuántas veces lo llares, el recurso ya no está.

3. HEAD request

```
import requests
resp = requests.head("https://api.github.com")
print(resp.headers["Content-Type"])      # "application/json; charset=utf-8"
print(resp.headers["X-RateLimit-Remaining"]) # 60
```

requests.head solo pide **cabeceras**, sin cuerpo: es barato y sirve para comprobar que una API responde o consultar los límites de tasa.

4. 🎯 Info de usuario de GitHub

```
import requests

datos = requests.get("https://api.github.com/users/python").json()

print(f"Nombre: {datos['name']}")
print(f"Bio: {datos['bio']}")
print(f"Repos públicos: {datos['public_repos']}")
print(f"Seguidores: {datos['followers']}")
print(f"Creado: {datos['created_at']}")
```

resp.json() te da el dict completo y cada campo se accede por su clave (name, bio, public_repos, followers, created_at).

5. 🔍 Buscador de repositorios

```
import requests

params = {"q": "python stars:>1000", "sort": "stars", "per_page": 5}
resp = requests.get("https://api.github.com/search/repositories", params=params)

for repo in resp.json()["items"]:
    print(repo["full_name"], "★", repo["stargazers_count"])
```

params construye la query string automáticamente. La búsqueda devuelve items, una lista de repositorios, y de cada uno sacas full_name y stargazers_count.

6. 🧩 API con paginación

```
import requests
repos = []

for page in (1, 2):
    params = {"page": page, "per_page": 10}
    resp = requests.get("https://api.github.com/users/python/repos", params=params)
    repos.extend(resp.json())

print(f"Repos obtenidos: {len(repos)}") # 20
for repo in repos:
    print(repo["name"])
```

Con page y per_page recorres la paginación: cada petición devuelve 10 repos y extend() los acumula en la lista. Es la técnica que usan los clientes reales para traer conjuntos completos.

7. 🤩 POST y respuesta

```
import requests

nuevo = {"title": "Hola API", "body": "Probando POST", "userId": 1}
resp = requests.post("https://jsonplaceholder.typicode.com/posts", json=nuevo)
print(resp.status_code) # 201 Created
print("ID:", resp.json()["id"]) # 101

verificacion = requests.get("https://jsonplaceholder.typicode.com/posts/101")
print(verificacion.json()["title"]) # "Hola API"
```

El POST devuelve **201 Created** con el recurso creado (ID 101). El GET posterior a ese ID confirma que la creación ha sido real.

8. 🕒 Tiempo de respuesta

```
import requests, time

apis = {
    "GitHub": "https://api.github.com",
    "JSONPlaceholder": "https://jsonplaceholder.typicode.com/posts/1",
    "httpbin": "https://httpbin.org/get",
    "ReqRes": "https://reqres.in/api/users/1",
    "OpenNotify": "http://api.open-notify.org/iss-now.json",
}

for nombre, url in apis.items():
    inicio = time.time()
    requests.get(url)
    fin = time.time()
    ms = (fin - inicio) * 1000
    print(f"{nombre}: {ms:.0f} ms")
```

`time.time()` antes y después de cada `requests.get()` te da los milisegundos. La más rápida cambia según la red del momento, pero siempre es la que menos distancia recorre y mejor responde. Es la comparativa que luego usarás en el [TEMA 11](#) con `asyncio`.

9. Cliente de API con caché

```

import requests, time

cache = {}
TTL = 60 # segundos de vida de cada entrada

def obtener(url):
    ahora = time.time()
    if url in cache:
        dato, timestamp = cache[url]
        if ahora - timestamp < TTL:
            print(f"📦 Desde caché: {url}")
            return dato
    print(f"🌐 Petición real: {url}")
    dato = requests.get(url).json()
    cache[url] = (dato, ahora)
    return dato

print(obtener("https://jsonplaceholder.typicode.com/posts/1"))
print(obtener("https://jsonplaceholder.typicode.com/posts/1")) # 📦 Desde caché

```

El diccionario usa la **URL como clave** y guarda (dato, timestamp). Antes de pedir, se comprueba si la URL está en caché y si su timestamp sigue dentro del TTL. La segunda llamada no sale a la red: responde al instante desde el diccionario.

Boletín U06 — Avanzado



Boletín U06 — Avanzado

Ejercicios que requieren aplicar los conceptos de APIs REST, requests y JSON de forma más profunda, con programas completos contra APIs reales.

1. PUT para actualizar

Haz PUT a `https://jsonplaceholder.typicode.com/posts/1` con `json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}`. Muestra el código de estado y el título actualizado.

Pista: `requests.put` con `json=` reemplaza el recurso completo. El título actualizado está en `resp.json()["title"]`.

2. DELETE un recurso

Haz DELETE a `https://jsonplaceholder.typicode.com/posts/1`. ¿Qué código de estado devuelve un borrado exitoso?

3. HEAD request

Usa `requests.head()` para pedir solo las cabeceras de `https://api.github.com`. Muestra los valores de `Content-Type` y `X-RateLimit-Remaining` de las cabeceras.

Pista: `requests.head` no devuelve cuerpo, solo cabeceras. Accede a ellas con `resp.headers["Content-Type"]`.

4. Info de usuario de GitHub

Obtén los datos del usuario “python” y muestra: nombre, bio, repos públicos, seguidores, fecha de creación.

Pista: La API de GitHub (`https://api.github.com/users/python`) devuelve un JSON con campos como `name`, `bio`, `public_repos`, `followers` y `created_at`. Usa `resp.json()` para acceder a

cada campo.

5. 🔍 Buscador de repositorios

Busca repositorios de Python con más de 1000 estrellas. Muestra nombre y estrellas.

Pista: La API de búsqueda de GitHub (/search/repositories) acepta q como parámetro. Usa "python stars:>1000" para filtrar. Ordena por stars con sort=stars y limita a 5 resultados con per_page=5.

6. 🧩 API con paginación

La API de GitHub página los resultados. Obtén los primeros 20 repos del usuario "python" (2 páginas de 10).

Pista: Usa los parámetros page y per_page en cada petición a <https://api.github.com/users/{usuario}/repos>. Itera sobre las páginas que necesites y acumula los resultados en una lista.

7. 😊😞 POST y respuesta

Crea un post en JSONPlaceholder, luego haz GET para verificarlo.

Pista: JSONPlaceholder es una API de pruebas. El POST devuelve el recurso creado con un ID (101). Luego puedes hacer GET a <https://jsonplaceholder.typicode.com/posts/101> para verificar la creación.

8. 🕒 Tiempo de respuesta

Mide el tiempo de respuesta de 5 APIs distintas. ¿Cuál es más rápida?

Pista: Mide el tiempo con `time.time()` antes y después de cada `requests.get()`. Prueba varias APIs gratuitas (GitHub, JSONPlaceholder, httpbin, ReqRes) y compara los tiempos en milisegundos.

9. 🏠 Cliente de API con caché

Crea un cliente que cachee respuestas en un diccionario para no repetir peticiones.

Pista: Usa un diccionario global donde la clave sea la URL. Guarda el dato y el timestamp. Antes de hacer una petición, comprueba si la URL está en caché y si el tiempo transcurrido desde que se guardó es menor al TTL.

Boletín U07 — Inicial (Resuelto)



Boletín U07 — Inicial (Resuelto)

1. Humedad actual

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}
resp = requests.get("https://api.openweathermap.org/data/2.5/weather", params=params)
print(f"Humedad: {resp.json()['main']['humidity']}%")
```

Necesitas una API key real de OpenWeatherMap (gratuita). Con units=metric la temperatura viene en grados.

2. Descripción del clima

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric", "lang": "es"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(datos["weather"][0]["description"])
```

weather es una **lista**: el [0] coge la condición principal y description es su texto.

3. Temperatura mínima y máxima

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(f"Mínima: {datos['main']['temp_min']}°C")
print(f"Máxima: {datos['main']['temp_max']}°C")
```

main contiene temp, temp_min, temp_max, feels_like y humidity.

4. Temperatura y sensación térmica

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Madrid", "appid": API_KEY, "units": "metric"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(f"Temperatura: {datos['main']['temp']}°C")
print(f"Sensación: {datos['main']['feels_like']}°C")
```

5. Error 401

```
import requests
resp = requests.get("https://api.openweathermap.org/data/2.5/weather",
params={"q": "Sevilla", "appid": "falsa", "units": "metric"})
print(resp.status_code) # 401
```

API key incorrecta → **401 Unauthorized**: el servidor no te reconoce y rechaza la petición.

6. ¿Dónde va la API key?

a) En el parámetro **appid** de la query string: ?q=Sevilla&appid=TU_API_KEY. b) En la cabecera **Authorization** con el esquema **Bearer**: Authorization: Bearer TU_API_KEY.

7. Clave desde el .env

```
from dotenv import load_dotenv
import os

load_dotenv()
print(os.getenv("OPENWEATHER_API_KEY")) # "test123"
```

El .env debe contener la línea OPENWEATHER_API_KEY=test123. load_dotenv() carga el archivo y os.getenv lee el valor.

8. Protege el repo

a) En el `.gitignore`:

```
.env
```

b) `os.getenv("MI_CLAVE")` devuelve **None** si la variable no existe. Por eso conviene comprobarlo y lanzar un `raise ValueError` con un mensaje claro.

Los **4xx** son error del cliente (tu clave, tu URL, tus parámetros): el 401 es el típico cuando la API key es mala.

Boletín U07 — Inicial



Boletín U07 — Inicial

Ejercicios básicos para afianzar los conceptos de API keys, variables de entorno, OpenWeatherMap y errores HTTP de la unidad U07.

1. Humedad actual

Usa OpenWeatherMap para obtener el clima de tu ciudad. Muestra la humedad (`humidity`) del campo `main`.

2. Descripción del clima

Del mismo JSON, muestra la descripción del tiempo (campo `description` dentro de `weather`).

3. Temperatura mínima y máxima

Muestra la temperatura mínima (`temp_min`) y máxima (`temp_max`) del pronóstico actual de tu ciudad.

4. Temperatura y sensación térmica

Muestra la temperatura actual (`temp`) y la sensación térmica (`feels_like`) de tu ciudad en grados centígrados.

Pista: recuerda el parámetro `units=metric` para que la API devuelva grados en lugar de kelvin.

5. Error 401

Haz una petición a OpenWeatherMap con una API key falsa (`appid="falsa"`). ¿Qué código de estado devuelve?

6. ¿Dónde va la API key?

a) ¿En qué parámetro de la query string espera OpenWeatherMap la clave? b) ¿En qué cabecera y con qué esquema espera OpenAI la clave?

7. Clave desde el .env

Crea un archivo `.env` con `OPENWEATHER_API_KEY=test123`. Usa `python-dotenv` para cargarlo y muestra la clave con `os.getenv`.

Pista: `load_dotenv()` primero, `os.getenv("OPENWEATHER_API_KEY")` después.

8. Protege el repo

a) Escribe la línea del `.gitignore` que evita subir el archivo `.env` a git. b) ¿Qué devuelve `os.getenv("MI_CLAVE")` si la variable no existe?

Boletín U07 — Avanzado (Resuelto)

Boletín U07 — Avanzado (Resuelto)

1. Clima con .env para dos ciudades

```
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

def clima(ciudad):
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}
    datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
                        params=params).json()
    return datos["main"]["temp"]

t1 = clima("Madrid")
t2 = clima("Barcelona")
print(f"Madrid: {t1}°C | Barcelona: {t2}°C")
print("Gana Madrid" if t1 > t2 else "Gana Barcelona")
```

La función devuelve la temperatura y el programa compara las dos llamadas. La clave nunca está en el código: viene del .env.

2. GPT con temperatura creativa


```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

resp = cliente.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "Cuenta una historia corta sobre Python."}],
    temperature=0.9, # más creativo
    max_tokens=200
)
print(resp.choices[0].message.content)

```

temperature=0.9 da respuestas más creativas y variadas; temperature=0 sería literal y repetitivo.

3. Capturar errores HTTP

```

import requests

for codigo in (404, 500):
    try:
        resp = requests.get(f"https://httpbin.org/status/{codigo}")
        resp.raise_for_status()
    except requests.exceptions.HTTPError as e:
        print(f"❌ Error {e.response.status_code} en {e.request.url}")

```

raise_for_status() lanza una HTTPError para cualquier 4xx/5xx, y e.response.status_code te dice qué código ha sido.

4. Comprobación de claves

```

from dotenv import load_dotenv
import os

load_dotenv()

API_KEY = os.getenv("OPENWEATHER_API_KEY")
if not API_KEY:
    raise ValueError("❌ Falta OPENWEATHER_API_KEY en .env")
print("Clave cargada correctamente")

```

Si la variable no existe, `os.getenv` devuelve `None` y el programa muere con un mensaje claro **antes** de hacer ninguna petición con clave vacía.

5. Chat con GPT y rol system

```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

resp = cliente.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "Eres un profesor de Python divertido."},
        {"role": "user", "content": "¿Qué es Python?"}
    ]
)
print(resp.choices[0].message.content)

```

El mensaje `system` define el **carácter y las reglas** del asistente; el `user` lleva la pregunta.

6. Timeout

```
import requests

try:
    resp = requests.get("https://192.0.2.1", timeout=3)
except requests.exceptions.Timeout:
    print("Timeout – el servidor no responde")
```

192.0.2.1 es una IP de test que nunca responde. El `timeout=3` hace que `requests` aborte a los 3 segundos y lanza `requests.exceptions.Timeout`.

7. 🎯 Pronóstico extendido

```
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

params = {
    "q": "Sevilla",
    "appid": API_KEY,
    "units": "metric",
    "lang": "es"
}

datos = requests.get("https://api.openweathermap.org/data/2.5/forecast",
                    params=params).json()

for i in range(0, len(datos["list"]), 8): # cada 8 elementos = 1 día
    dia = datos["list"][i]
    fecha = dia["dt_txt"][:10]
    temp = dia["main"]["temp"]
    desc = dia["weather"][0]["description"]
    print(f"{fecha}: {temp}°C – {desc}")
```

El endpoint `/forecast` devuelve datos cada 3 horas: $24 \div 3 = 8$ intervalos por día. Saltando de 8 en 8 obtienes un dato por día del pronóstico de 5 días.

8. 🔍 Múltiples ciudades

```

import requests, time
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

ciudades = ["Madrid", "Barcelona", "Sevilla", "Valencia", "Bilbao"]

for ciudad in ciudades:
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}
    datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
                        params=params).json()
    print(f"{datos['name']}: {datos['main']['temp']}°C - "
          f"{datos['weather'][0]['description']}")
    time.sleep(0.2) # respiro entre peticiones: evita el rate limit

```

El `time.sleep(0.2)` espacia las peticiones para no disparar el límite de 60/min del plan gratis.

9. GPT: explicador automático

```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

conceptos = ["Lock", "Semaphore", "Barrier"]

for concepto in conceptos:
    resp = cliente.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user",
                    "content": f"Explica qué es {concepto} en 2 frases."}],
        max_tokens=60
    )
    print(f"{concepto}: {resp.choices[0].message.content}\n")

```

Se itera sobre la lista de conceptos y cada uno recibe una pregunta con `max_tokens` corto para limitar la longitud (y el coste).

10. 🤖 Conversación con contexto

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

mensajes = [{"role": "system", "content": "Eres un tutor de Python."}]

while True:
    pregunta = input("Tú: ")
    if pregunta.lower() == "salir":
        break

    mensajes.append({"role": "user", "content": pregunta})

    resp = cliente.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=mensajes,
        max_tokens=150
    )

    respuesta = resp.choices[0].message.content
    print(f"GPT: {respuesta}")
    mensajes.append({"role": "assistant", "content": respuesta})
```

La lista `mensajes` acumula cada turno (user → assistant → user → ...): así el modelo “recuerda” lo hablado. El mensaje `system` inicial fija el rol del tutor.

11. 🕒 Monitor de uptime

```

import requests, time
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

URL = "https://api.openweathermap.org/data/2.5/weather"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}

fallos = 0
MAX_FALLOS = 3

while True:
    try:
        resp = requests.get(URL, params=params, timeout=5)
        if resp.status_code == 200:
            print("✅ La API responde OK")
            fallos = 0
        else:
            print(f"⚠️ Respuesta inesperada: {resp.status_code}")
            fallos += 1
    except requests.exceptions.RequestException as e:
        print(f"❌ Error: {e}")
        fallos += 1

    if fallos >= MAX_FALLOS:
        print("🚨 ALERTA: la API lleva 3 fallos seguidos")
        fallos = 0

    time.sleep(30) # comprobamos cada 30 segundos

```

El contador fallos se incrementa con cada error y se **reinicia** al obtener un 200. Cuando llega a 3, salta la alerta. Este es el esqueleto de los monitores de disponibilidad que verás en la U11 con asyncio.

12. Agregador de APIs

```

import requests

def resumen():
    resultado = {}

    usuario = requests.get("https://api.github.com/users/python").json()
    resultado["login"] = usuario["login"]
    resultado["repos"] = usuario["public_repos"]

    posts = requests.get("https://jsonplaceholder.typicode.com/posts").json()
    resultado["posts_publicados"] = len(posts)

    ip = requests.get("https://httpbin.org/ip").json()
    resultado["mi_ip"] = ip["origin"]

    return resultado

info = resumen()
print(f"👤 Usuario: {info['login']}")
print(f"📦 Repos públicos: {info['repos']}")
print(f"📄 Posts en JSONPlaceholder: {info['posts_publicados']}")
print(f"💻 Mi IP (según httpbin): {info['mi_ip']}")

```

Cada API aporta una clave distinta al diccionario: GitHub los datos de usuario, JSONPlaceholder el conteo de posts y httpbin la IP. Una función centraliza las tres llamadas y combina lo relevante en un solo resumen.

Boletín U07 — Avanzado



Boletín U07 — Avanzado

Ejercicios que requieren aplicar los conceptos de APIs comerciales, variables de entorno, rate limits y OpenAI de forma más profunda, con programas completos contra APIs reales.

1. Clima con .env para dos ciudades

Carga tu API key de OpenWeatherMap desde un archivo .env con python-dotenv. Obtén el clima de dos ciudades distintas y compara sus temperaturas.

Pista: `load_dotenv() + os.getenv("OPENWEATHER_API_KEY")`. Escribe una función `clima(ciudad)` y llámala dos veces, guardando la temperatura de cada una para compararlas.

2. GPT con temperatura creativa

Usa la API de OpenAI para pedir una historia corta sobre Python. Ajusta el parámetro `temperature=0.9` para obtener una respuesta más creativa.

Pista: en `cliente.chat.completions.create(...)`, además de `model` y `messages`, pasa `temperature=0.9` y un `max_tokens` razonable para que la historia no se corte.

3. Capturar errores HTTP

Haz una petición a `https://httpbin.org/status/404` y otra a `https://httpbin.org/status/500`. Captura los errores usando `raise_for_status()` dentro de un bloque `try/except`.

Pista: dentro del `except` `requests.exceptions.HTTPError` as `e`, puedes leer `e.response.status_code` para saber qué código ha fallado.

4. Comprobación de claves

Carga el .env y comprueba que `OPENWEATHER_API_KEY` existe. Si no existe, lanza un error claro con `raise ValueError`.

Pista: `os.getenv` devuelve `None` si la variable falta: esa es la condición para lanzar el error.

5. Chat con GPT y rol system

Pregunta a GPT-3.5 “¿Qué es Python?” usando un mensaje system que diga “Eres un profesor de Python divertido” antes del mensaje user.

6. Timeout

Haz una petición a `https://192.0.2.1` con `timeout=3` y captura la excepción de timeout.

Pista: `192.0.2.1` es una IP de test que nunca responde. La excepción es `requests.exceptions.Timeout`.

7. 🎯 Pronóstico extendido

Usa OpenWeatherMap para obtener el pronóstico de 5 días de tu ciudad.

Pista: OpenWeatherMap tiene un endpoint `/forecast` que devuelve datos cada 3 horas. Usa `lang=es` para descripciones en español. Para obtener un dato por día, salta 8 elementos ($24 \text{ horas} \div 3 \text{ horas} = 8 \text{ intervalos}$).

8. 🔍 Múltiples ciudades

Obtén el clima de 5 ciudades a la vez usando un solo bucle.

Pista: Itera sobre una lista de ciudades y haz una petición a OpenWeatherMap por cada una. Añade `time.sleep(0.2)` entre peticiones para evitar el rate limit. Cada respuesta contiene `main.temp` y `weather[0].description`.

9. 🧩 GPT: explicador automático

Pregunta a GPT-3.5 que explique 3 conceptos de Python: Lock, Semaphore, Barrier.

Pista: La API de chat de OpenAI recibe una lista de mensajes. Puedes iterar sobre una lista de conceptos y pedir una explicación corta para cada uno, limitando la respuesta con `max_tokens`.

10. 😊😞 Conversación con contexto

Chat con GPT que recuerda el historial de la conversación.

Pista: Mantén una lista mensajes que acumule cada interacción. Empieza con un mensaje `{"role": "system"}` para definir el rol. En cada turno, añade el mensaje del usuario y la respuesta del asistente para mantener el contexto.

11. 🕒 Monitor de uptime

Comprueba cada 30s si una API responde. Si falla 3 veces seguidas, alerta.

Pista: Un bucle infinito con `time.sleep(30)` dentro. Un contador de fallos consecutivos se incrementa en cada error y se reinicia al obtener una respuesta OK. Cuando supere el máximo, muestra una alerta.

12. 🏗️ Agregador de APIs

Llama a 3 APIs distintas y combina sus respuestas en un solo resumen.

Pista: Define una función que llame a varias APIs y guarde los datos relevantes en un diccionario. Cada API aporta una clave diferente. Usa GitHub para datos de usuario, JSONPlaceholder para el conteo de posts y httpbin para la IP.

Boletín U08 — Inicial (Resuelto)



Boletín U08 — Inicial (Resuelto)

1. SHA1 de tu nombre

```
import hashlib
print(hashlib.sha1(b"sergi").hexdigest())
# 52a431556910f83a3d65e1e7c3772f5d44011eb3
```

Con tu nombre (en minúsculas), por ejemplo b"ana", obtendrías su hash SHA1 de **40 caracteres** hexadecimales (160 bits).

2. MD5 de una frase

```
import hashlib
print(hashlib.md5("Python es genial".encode()).hexdigest())
# 1bdf047b34a4890934fc36d12b464f2c
```

MD5 devuelve **32 caracteres** hexadecimales (128 bits).

3. Compara “Hola” vs “hola”

```
import hashlib
h1 = hashlib.sha256(b"Hola").hexdigest()
h2 = hashlib.sha256(b"hola").hexdigest()
print(f"Hola: {h1}")
print(f"hola: {h2}")
print(f"¿Iguales? {h1 == h2}")
```

- Hola → e633f4fc79badea1dc5db970cf397c8248bac47cc3acf9915ba60b5d76b0e88f
- hola → b221d9dbb083a7f33428d7c2a3c3198ae925614d70210e28716ccaa7cd4ddb79

Son **diferentes**: aunque solo cambia la mayúscula por la minúscula, el hash cambia por completo. Es el **efecto avalancha** ([punto 2](#)).

4. Longitud de los hashes de “Hola mundo”

```
import hashlib
texto = b"Hola mundo"
print(f"MD5:      {len(hashlib.md5(texto).hexdigest())} chars (128 bits)")
print(f"SHA1:     {len(hashlib.sha1(texto).hexdigest())} chars (160 bits)")
print(f"SHA256:    {len(hashlib.sha256(texto).hexdigest())} chars (256 bits)")
```

MD5 → **32**, SHA1 → **40**, SHA256 → **64** caracteres hexadecimales. La longitud es **fija** para cada algoritmo, da igual el tamaño de la entrada.

5. Hash de un archivo simple

```
import hashlib

with open("mensaje.txt", "rb") as f:
    hash_archivo = hashlib.sha256(f.read()).hexdigest()
print(f"SHA256 del archivo: {hash_archivo}")

hash_texto = hashlib.sha256(b"Hola mundo").hexdigest()
print(f"SHA256 del texto:    {hash_texto}")
print(f"¿Coinciden? {hash_archivo == hash_texto}") # True
```

Si el archivo contiene exactamente Hola mundo, ambos coinciden: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c. La clave es abrirlo en modo binario "rb".

6. Cifrado César sencillo

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

print(cifrar_cesar("Hola", 3)) # "Krod"
```

Cada letra avanza 3 posiciones: H→K, o→r, l→o, a→d. Resultado: Krod.

7. Descifrar un César dado

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

def descifrar_cesar(texto, desplazamiento):
    return cifrar_cesar(texto, -desplazamiento)

print(descifrar_cesar("Krod", 3)) # "Hola"
```

Descifrar es cifrar con desplazamiento **negativo** ([punto 6](#)): "Krod" con -3 devuelve "Hola".

8. Determinismo del hash

```
import hashlib
h1 = hashlib.sha256(b"clave123").hexdigest()
h2 = hashlib.sha256(b"clave123").hexdigest()
print(f"Primera: {h1}")
print(f"Segunda: {h2}")
print(f"¿Iguales? {h1 == h2}") # True
```

Son **iguales**: el hash es **determinista** ([punto 2](#)). Misma entrada → mismo hash, siempre. Esa propiedad es la que hace posible el login por comparación de hashes del [punto 4](#).

Boletín U08 — Inicial



Boletín U08 — Inicial

Ejercicios básicos para afianzar los conceptos de hash (MD5, SHA-1, SHA-256) y cifrado César de la unidad U08.

1. SHA1 de tu nombre

Calcula el hash SHA1 de tu nombre (en minúsculas) usando `hashlib.sha1()`. Muestra el resultado en hexadecimal con `.hexdigest()`.

Pista: recuerda que `hashlib` espera bytes: escribe tu nombre como `b"sergi"` o con `.encode()`.

2. MD5 de una frase

Calcula el hash MD5 de la frase `"Python es genial"`. Muestra el resultado en hexadecimal.

3. Compara “Hola” vs “hola”

Calcula SHA256 de `"Ho1a"` y de `"ho1a"`. ¿Los hashes son iguales o diferentes? ¿Por qué?

Pista: solo cambia la mayúscula por la minúscula... y el hash cambia por completo. Así es el efecto avalancha.

4. Longitud de los hashes de “Hola mundo”

Calcula MD5, SHA1 y SHA256 de `"Ho1a mundo"` y escribe cuántos caracteres hexadecimales devuelve cada uno.

5. Hash de un archivo simple

Crea un archivo de texto con el contenido `"Ho1a mundo"`, ábrelo en modo binario (`"rb"`) y calcula su hash SHA256. Compara el resultado con el hash SHA256 del propio texto.

Pista: `hashlib.sha256(f.read()).hexdigest()` dentro de un `with open("mensaje.txt", "rb") as f:`. El hash debe coincidir con `hashlib.sha256(b"Hola mundo").hexdigest()`.

6. Cifrado César sencillo

Implementa una función `cifrar_cesar(texto, desplazamiento)` y cifra "Hola" con desplazamiento 3. ¿Qué texto obtienes?

7. Descifrar un César dado

Usando la función de descifrado César, descifra "Krod" con desplazamiento 3. ¿Qué texto original aparece?

Pista: descifrar es cifrar con desplazamiento **negativo**: `cifrar_cesar(texto, -desplazamiento)`.

8. Determinismo del hash

Calcula dos veces el SHA256 de "clave123" y compáralos. ¿Son iguales? ¿Qué propiedad del hash te lo garantiza?

Pista: un hash es **determinista**: misma entrada → mismo hash, siempre.

Boletín U08 — Avanzado (Resuelto)

Boletín U08 — Avanzado (Resuelto)

1. Hash de un archivo de texto

```
import hashlib

with open("mensaje.txt", "rb") as f:
    hash_archivo = hashlib.sha256(f.read()).hexdigest()
print(f"SHA256 del archivo: {hash_archivo}")

hash_texto = hashlib.sha256(b"Hola mundo").hexdigest()
print(f"SHA256 del texto: {hash_texto}")
print(f"¿Coinciden? {hash_archivo == hash_texto}") # True
```

Si mensaje.txt contiene exactamente Hola mundo (sin salto de línea extra, o con el mismo contenido exacto), el hash coincide: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c. Hashear un archivo en modo binario es hashear **sus bytes exactos** ([punto 3](#)).

2. Cifrado César con espacios

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

print(cifrar_cesar("Hola mundo", 5)) # Mtqf rzsit
```

El espacio no es una letra (isalpha() es False), así que se añade tal cual. Las letras avanzan 5 posiciones: **Hola → Mtqf**, **mundo → rzsit**. Resultado: **Mtqf rzsit**.

3. Descifrado con mayúsculas y minúsculas

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

def descifrar_cesar(texto, desplazamiento):
    return cifrar_cesar(texto, -desplazamiento)

print(descifrar_cesar("Nkrru", 2)) # Lipps
```

Descifrar es cifrar con desplazamiento **negativo** ([punto 6](#)): cada letra se mueve 2 hacia atrás. La mayúscula inicial N usa la base `ord('A')` y devuelve L; el resto, minúsculas, usan `ord('a')`. `isupper()` garantiza que el caso se preserve.

4. Verificador de integridad

```
import hashlib

with open("archivo.pdf", "rb") as f:
    hash_calculado = hashlib.sha256(f.read()).hexdigest()

hash_oficial = "8a1f9c..." # el hash publicado por la web oficial

if hash_calculado == hash_oficial:
    print("✅ El archivo es íntegro: no ha sido manipulado.")
else:
    print("❌ ¡Alerta! El archivo NO coincide con el oficial.")
```

La comparación de hashes detecta **cualquier** modificación: un solo bit distinto produce un hash completamente diferente (efecto avalancha). Es la verificación de integridad del [punto 8](#).

5. Fuerza bruta César

```
def descifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base - desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

palabras = {"hola", "mundo", "python", "clave", "secreto"}

cifrado = "Mtqf rzsit" # "Hola mundo" con desplazamiento 5
for desplazamiento in range(1, 26):
    intento = descifrar_cesar(cifrado, desplazamiento)
    if any(p in intento.lower() for p in palabras):
        print(f"Desplazamiento {desplazamiento}: {intento} ← ¡parece correcto!")
```

Solo hay **25 desplazamientos posibles** ([punto 6](#)). Filtrando por palabras comunes en español, el desplazamiento 5 devuelve “Hola mundo” y es el único con sentido. Se rompe en segundos.

6. Hash con salt

```

import hashlib, os

def registrar_con_salt(contraseña):
    salt = os.urandom(16) # 🧂 16 bytes nuevos cada vez
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return salt.hex() + hash_contra # sal + hash juntos

def verificar(contraseña, almacenado):
    salt = bytes.fromhex(almacenado[:32]) # 🧂 recuperamos la sal
    hash_original = almacenado[32:]
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return hash_intento == hash_original

hash_ana = registrar_con_salt("clave123")
hash_bob = registrar_con_salt("clave123")
print(f"¿Hashes iguales? {hash_ana == hash_bob}") # False
print(f"¿Ana entra? {verificar('clave123', hash_ana)}") # True
print(f"¿Ana con otra? {verificar('otra', hash_ana)}") # False

```

Con la misma contraseña, Ana y Bob tienen **hashes distintos** gracias a sus sales únicas. El salt ocupa los 32 primeros caracteres hex (16 bytes), que se recuperan con `bytes.fromhex()` para el login ([punto 5](#)).

7. Comparación gráfica de avalancha

```

import hashlib

h1 = hashlib.sha256(b"Hola mundo").hexdigest()
h2 = hashlib.sha256(b"Hola mund0").hexdigest() # una 'o' por un '0'

diferentes = sum(1 for a, b in zip(h1, h2) if a != b)
print(f"h1: {h1}")
print(f"h2: {h2}")
print(f"Caracteres diferentes: {diferentes} de {len(h1)}")

```

La entrada cambia **una sola letra**, pero de los 64 caracteres hex suelen diferir la mitad o más ($\approx 50\%$), que es el **efecto avalancha** ([punto 2](#)). Ninguna parte del hash “se parece” al del texto original.

8. Velocidad de hashes

```
import hashlib, time

texto = b"clave123"
iteraciones = 1_000_000

for nombre, func in (("MD5", hashlib.md5), ("SHA1", hashlib.sha1), ("SHA256",
hashlib.sha256)):
    inicio = time.time()
    for _ in range(iteraciones):
        func(texto).hexdigest()
    fin = time.time()
    print(f"{nombre}: {fin - inicio:.2f} segundos")
```

MD5 suele ser el más rápido, SHA1 intermedio y **SHA256 el más lento**: más bits que procesar. Ahora bien, “más lento” es justo lo que quieres en una contraseña: al atacante le cuesta más adivinar por fuerza bruta. Nunca elijas el algoritmo por velocidad, sino por seguridad ([punto 3](#)).

9. Mini gestor de contraseñas

```

import hashlib, os, json

usuarios = {}

def registrar(usuario, contraseña):
    salt = os.urandom(16)
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    usuarios[usuario] = salt.hex() + hash_contra

def login(usuario, contraseña):
    almacenado = usuarios[usuario]
    salt = bytes.fromhex(almacenado[:32])
    hash_original = almacenado[32:]
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return hash_intento == hash_original

registrar("Ana", "clave123")
registrar("Bob", "clave123")
print(f"Ana login correcto: {login('Ana', 'clave123')}")      # True
print(f"Ana login fallido: {login('Ana', 'otra')}")          # False
print(f"Hashes distintos: {usuarios['Ana'] != usuarios['Bob']}") # True

with open("contraseñas.json", "w") as f:
    json.dump(usuarios, f, indent=2)

```

Cada usuario se guarda como `salt.hex() + hash` en un diccionario, y se exporta a JSON con `json.dump`. Para verificar, el login extrae la sal (primeros 32 caracteres hex), la convierte con `bytes.fromhex()` y recalcula el hash ([punto 5](#)). Si dos usuarios comparten contraseña, sus registros JSON son **distintos**.

Boletín U08 — Avanzado



Boletín U08 — Avanzado

Ejercicios que requieren aplicar los conceptos de hash, sal y cifrado César de forma más profunda, con programas completos.

1. Hash de un archivo de texto

Escribe “Hola mundo” en un archivo `mensaje.txt`. Lee su contenido en modo binario y calcula el hash SHA256. Verifica que coincida con hacer hash directamente del mismo string.

Pista: abre el archivo con `with open("mensaje.txt", "rb") as f:` y pasa `f.read()` directo a `hashlib.sha256().hexdigest()`. El resultado debe coincidir con `hashlib.sha256(b"Hola mundo").hexdigest()`.

2. Cifrado César con espacios

Implementa una función `cifrar_cesar(texto, desplazamiento)` que cifre frases completas respetando espacios y signos de puntuación sin modificarlos. Pruébala con “Hola mundo” con desplazamiento 5.

Pista: recorre carácter a carácter. Si `character.isalpha()` es verdadero, aplica la fórmula con `% 26`; si no, añade el carácter tal cual. La salida debe ser “Mtqf rzsit”.

3. Descifrado con mayúsculas y minúsculas

Implementa descifrado César que gestione tanto mayúsculas como minúsculas. Descifra “Nkr ru” sabiendo que fue cifrado con desplazamiento 2.

Pista: descifrar es cifrar con desplazamiento negativo: `cifrar_cesar(texto, -desplazamiento)`. Usa `ord('A')` o `ord('a')` según `isupper()` para preservar mayúsculas y minúsculas.

4. Verificador de integridad

Calcula el SHA256 de un archivo y compáralo con un hash esperado.

Pista: lee el archivo en modo binario ("rb") y pásalo directamente a `hashlib.sha256().hexdigest()`. Compara el hash calculado con el hash esperado del mismo contenido. Si coinciden, el archivo no ha sido modificado.

5. Fuerza bruta César

Dado un texto cifrado con César (desplazamiento desconocido), prueba los 25 desplazamientos y muestra solo los que tengan palabras en español.

Pista: para cada desplazamiento del 1 al 25, aplica el descifrado César. Usa un conjunto de palabras comunes en español (como "hola", "mundo", "python") para filtrar resultados. Si el texto descifrado contiene alguna palabra conocida, es probable que sea el correcto.

6. Hash con salt

Añade un salt aleatorio al hash de una contraseña para evitar tablas rainbow.

Pista: genera un salt de 16 bytes con `os.urandom(16)`. El hash final es `salt + sha256(salt + password.encode()).digest()`. Para verificar, extrae los primeros 16 bytes (el salt) y repite el cálculo. Si dos usuarios tienen la misma contraseña, sus hashes serán distintos gracias al salt.

7. Comparación gráfica de avalancha

Muestra cómo un bit de diferencia en la entrada cambia completamente el hash.

Pista: convierte ambos hashes a hexadecimal con `hexdigest()` y compáralos carácter por carácter. Cuenta cuántos caracteres son diferentes. El efecto avalancha debería mostrar aproximadamente el 50% de caracteres distintos aunque la entrada cambie solo un bit.

8. Velocidad de hashes

Compara cuánto tarda MD5 vs SHA1 vs SHA256 en hashear 1 millón de veces.

Pista: usa un bucle de 1 millón de iteraciones llamando a cada función de hash (`hashlib.md5`, `hashlib.sha1`, `hashlib.sha256`). Mide el tiempo total con `time.time()` antes y después. MD5 es el más rápido; SHA256 el más lento pero más seguro.

9. Mini gestor de contraseñas

Guarda contraseñas con hash + salt en un archivo JSON. Permite registro y login.

Pista: almacena cada usuario en un JSON con el formato `salt.hex() + hashlib.sha256(salt + password.encode()).hexdigest()`. Para verificar, extrae el salt (primeros 32 caracteres hexadecimales), conviértelo a bytes con `bytes.fromhex()` y recalcula el hash con la contraseña proporcionada.

Boletín U09 — Inicial (Resuelto)



Boletín U09 — Inicial (Resuelto)

pycryptodome debe estar instalado: `pip install pycryptodome`.

1. AES modo ECB

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_ECB)
texto_cifrado = cifrador.encrypt(b"0123456789ABCDEF")
print(f"Cifrado (hex): {texto_cifrado.hex()}")
```

El mensaje mide **exactamente 16 bytes**, así que no hace falta padding. ECB cifra cada bloque de forma independiente (recuerda: no lo uses con datos largos o repetitivos, [punto 3](#)).

2. Nonce y tag

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(b"Hola mundo con AES")

print(f"Longitud nonce: {len(cifrador.nonce)} bytes")
print(f"Longitud tag: {len(tag)} bytes")
print(f"Longitud cifrado: {len(texto_cifrado)} bytes")
```

En modo EAX el **nonce** mide **16 bytes** y el **tag** también **16 bytes**. Los tres (nonce, tag y cifrado) viajan juntos; la clave no ([punto 2](#)).

3. RSA: exportar clave

```

from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
publica = clave.publickey().export_key().decode()

print(f"Primeros 40: {publica[:40]}")
print(f"Últimos 40: {publica[-40:]}")

```

```

Primeros 40: -----BEGIN PUBLIC KEY-----
Últimos 40: -----END PUBLIC KEY-----

```

La clave pública en formato **PEM** se puede compartir con cualquiera. La privada NO.

4. AES: cifrar mensaje

```

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(b"Hola AES")
print(f"Cifrado: {texto_cifrado.hex()}")

```

encrypt_and_digest devuelve el texto cifrado y el **tag** de integridad de una vez.

5. AES: descifrar

```

cifrador2 = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
original = cifrador2.decrypt(texto_cifrado)
print(f"Original: {original.decode()}")

```

Necesitas la misma **clave** y el mismo **nonce** para descifrar ([punto 2](#)).

6. RSA: generar claves

```
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
print(clave.publickey().export_key().decode()[:50] + "...")
```

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...
```

La clave pública se puede compartir. La privada NO.

7. Simétrico vs asimétrico

- a) El simétrico usa **una sola clave** (misma para cifrar y descifrar). El asimétrico usa **dos**: pública + privada.
- b) Con la clave **pública de Bob**. Solo su clave privada puede descifrarlo.
- c) Por su **límite de tamaño** (~190 bytes con claves de 2048 bits) y su **velocidad** (~1 MB/s). Para volúmenes grandes se usa AES (o el cifrado híbrido del [punto 6](#)).

8. AES: cifrar y descifrar completo

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
mensaje = b"El cifrado simetrico es rapido"

cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(mensaje)

print(f"Nonce: {cifrador.nonce.hex()}")
print(f"Tag: {tag.hex()}")
print(f"Cifrado: {texto_cifrado.hex()}")

descifrador = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
original = descifrador.decrypt(texto_cifrado)
print(f"Original: {original.decode()}")
```

El nonce y el tag se envían junto al cifrado; el receptor los usa con la misma clave para descifrar y verificar la integridad ([punto 2](#)).

Boletín U09 — Inicial



Boletín U09 — Inicial

Ejercicios básicos para afianzar los conceptos de AES, RSA y la diferencia simétrico/asimétrico de la unidad U09.

1. AES modo ECB

Cifra el mensaje `b"0123456789ABCDEF"` (16 bytes exactos) usando AES en modo ECB. Muestra el resultado en hexadecimal.

Pista: ECB no necesita nonce ni tag: `AES.new(clave, AES.MODE_ECB)` y `cifrador.encrypt(mensaje)`. El mensaje debe medir exactamente 16 bytes (o usar pad).

2. Nonce y tag

Cifra `b"Hola mundo con AES"` usando AES modo EAX. Imprime por pantalla las longitudes del nonce y del tag (en bytes).

Pista: `cifrador.nonce` y `tag` son objetos bytes: `len(cifrador.nonce)` y `len(tag)`. El nonce del modo EAX suele medir 16 bytes.

3. RSA: exportar clave

Genera un par de claves RSA de 2048 bits. Exporta la clave pública a formato PEM (string) y muestra sus primeros 40 caracteres y sus últimos 40 caracteres.

4. AES: cifrar mensaje

Cifra el mensaje `b"Hola AES"` con AES en modo EAX y muestra el texto cifrado en hexadecimal.

5. AES: descifrar

Descifra el mensaje cifrado en el ejercicio 4 y muestra el original.

Pista: necesitas la misma clave y el mismo nonce: `AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)`.

6. RSA: generar claves

Genera un par RSA de 2048 bits y muestra los primeros 50 caracteres de la clave pública en formato PEM.

7. Simétrico vs asimétrico

Responde razonadamente:

a) ¿Cuántas claves usa el cifrado simétrico? ¿Y el asimétrico? b) ¿Con qué clave cifras un mensaje para que solo Bob lo lea? c) ¿Por qué RSA no sirve para cifrar un archivo grande?

8. AES: cifrar y descifrar completo

Cifra b"El cifrado simetrico es rapido" con AES (EAX), y descifralo a continuación. Muestra el nonce, el tag y el mensaje original.

Pista: usa `encrypt_and_digest` para cifrar y `AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce).decrypt(texto_cifrado)` para descifrar.

Boletín U09 — Avanzado (Resuelto)

Boletín U09 — Avanzado (Resuelto)

1. Cifrado híbrido simplificado

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

clave_bob = RSA.generate(2048)

# Bob genera clave AES y cifra el mensaje
clave_aes = get_random_bytes(32)
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(b"El cifrado hibrido funciona")

# La clave AES viaja protegida por la RSA pública de Bob
clave_aes_cifrada = PKCS1_OAEP.new(clave_bob.publickey()).encrypt(clave_aes)

# Descifrado en orden inverso: primero RSA, luego AES
clave_aes_recibida = PKCS1_OAEP.new(clave_bob).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)
print(f"Mensaje original: {original.decode()}")
```

Mensaje original: El cifrado hibrido funciona

RSA reparte la clave AES (32 bytes) y AES cifra el mensaje completo. El orden de descifrado es el inverso al de cifrado ([punto 6](#)).

2. Firma alterada


```

from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = "Transferencia de 500€".encode()

h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)

# Modificamos UN byte de la firma
firma_mutada = bytearray(firma)
firma_mutada[0] ^= 0xFF
firma_mutada = bytes(firma_mutada)

try:
    pkcs1_15.new(clave.publickey()).verify(h, firma_mutada)
    print("✅ Firma válida")
except (ValueError, TypeError):
    print("❌ Firma inválida – la firma fue alterada")

```

❌ Firma inválida – la firma fue alterada

La firma depende del hash y de la clave: **un solo byte distinto** la invalida por completo ([punto 5](#)).

3. RBAC con permisos cifrado

```

permisos = {
    "admin":    ["cifrar", "descifrar", "firmar"],
    "usuario":  ["cifrar", "firmar"],
    "invitado": ["cifrar"],
}

def puede(usuario, accion):
    return accion in permisos.get(usuario["rol"], [])

print(puede({"rol": "admin"}, "descifrar"))    # True
print(puede({"rol": "usuario"}, "descifrar"))  # False
print(puede({"rol": "usuario"}, "firmar"))     # True
print(puede({"rol": "invitado"}, "firmar"))    # False
print(puede({"rol": "invitado"}, "cifrar"))     # True

```

```

True
False
True
False
True

```

Cada rol tiene su paquete de permisos; puede comprobar si la acción está en el del rol ([punto 7](#)). `permisos.get(usuario["rol"], [])` devuelve lista vacía para roles desconocidos: por defecto, nada permitido.

4. Cifrar archivo completo

```

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)

# Cifrar el archivo
with open("mensaje.txt", "rb") as f:
    contenido = f.read()

cifrador = AES.new(clave, AES.MODE_EAX)
cifrado, tag = cifrador.encrypt_and_digest(contenido)

with open("mensaje.cifrado", "wb") as f:
    f.write(cifrador.nonce + tag + cifrado)    # nonce + tag + cifrado

# Descifrar
with open("mensaje.cifrado", "rb") as f:
    paquete = f.read()

nonce = paquete[:16]
tag = paquete[16:32]
cifrado_recibido = paquete[32:]

descifrador = AES.new(clave, AES.MODE_EAX, nonce=nonce)
original = descifrador.decrypt_and_verify(cifrado_recibido, tag)
print(f"Archivo descifrado: {original.decode()}")

```

Se guardan los tres componentes juntos (nonce + tag + cifrado) y al descifrar se **separan por sus longitudes** (16 y 16 bytes). `decrypt_and_verify` además comprueba que el archivo no fue manipulado.

5. RSA: cifrar mensajes largos

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

clave = RSA.generate(2048)
mensaje_largo = b"x" * 300 # 300 bytes

cifrador = PKCS1_OAEP.new(clave.publickey())
try:
    cifrador.encrypt(mensaje_largo)
except ValueError as e:
    print(f"❌ Error: {e}")

```

❌ Error: Plaintext is too long.

El problema: RSA con claves de 2048 bits solo admite ~190 bytes de mensaje.

La solución (cifrado híbrido): no cifra el mensaje con RSA; cifra la clave AES con RSA y el mensaje con AES:

```

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

clave_aes = get_random_bytes(32)
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje_largo)
clave_aes_cifrada = PKCS1_OAEP.new(clave.publickey()).encrypt(clave_aes)

clave_aes_recibida = PKCS1_OAEP.new(clave).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)
print(f"Longitud del original: {len(original)} bytes")

```

Longitud del original: 300 bytes

Así cualquier mensaje cabe, sea cual sea su tamaño ([punto 6](#)).

6. Intercambio de claves simulado

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

# Ana genera su par RSA
clave_ana = RSA.generate(2048)

# Bob cifra con la RSA pública de Ana
clave_aes = get_random_bytes(32)
mensaje = b"Mensaje para Ana"
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje)
clave_aes_cifrada = PKCS1_OAEP.new(clave_ana.publickey()).encrypt(clave_aes)

# Se envían los 4 componentes
# Ana descifra en orden inverso
clave_aes_recibida = PKCS1_OAEP.new(clave_ana).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)
print(f"Ana recibe: {original.decode()}")

```

```
Ana recibe: Mensaje para Ana
```

Los 4 componentes que viajan son (clave_AES_cifrada, nonce, tag, cifrado). Ana usa su **clave privada** para extraer la clave AES y luego descifra con ella ([punto 6](#)).

7. Firma con verificación de integridad

```
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = b"Este mensaje es de Ana"

# Firmar el original
h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)

# Mensaje MODIFICADO después de firmar
mensaje_tocado = b"Este mensaje es de Ana pero lo cambie"

try:
    pkcs1_15.new(clave.publickey()).verify(SHA256.new(mensaje_tocado), firma)
    print("✅ Firma válida")
except (ValueError, TypeError):
    print("❌ Firma inválida – el mensaje fue manipulado")
```

❌ Firma inválida – el mensaje fue manipulado

La firma se calcula sobre el **hash del mensaje original**. Si el mensaje cambia un solo byte, el hash es distinto y la verificación falla: la firma detecta cualquier modificación ([punto 5](#)).

8. RSA vs AES benchmark

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes
import time

clave_rsa = RSA.generate(2048)
clave_aes = get_random_bytes(32)
mensaje = b"Rendimiento de cifrado"

cifrador_rsa = PKCS1_OAEP.new(clave_rsa.publickey())
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)

# 100 cifrados RSA
inicio = time.time()
for _ in range(100):
    cifrador_rsa.encrypt(mensaje[:16])
rsa_ms = (time.time() - inicio) * 1000

# 1000 cifrados AES
inicio = time.time()
for _ in range(1000):
    cifrador_aes.encrypt(mensaje)
aes_ms = (time.time() - inicio) * 1000

print(f"100 cifrados RSA: {rsa_ms:.1f} ms")
print(f"1000 cifrados AES: {aes_ms:.1f} ms")

```

La diferencia es **abismal**: cifrar 10 veces más mensajes con AES tarda una fracción de lo que tarda RSA con solo 100. Por eso AES va para el volumen y RSA solo para repartir la clave ([punto 1](#)).

9. Sistema de cifrado de extremo a extremo

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre
        self.clave_rsa = RSA.generate(2048)

    def cifrar_para(self, mensaje, destinatario):
        clave_aes = get_random_bytes(32)
        cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
        cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje.encode())

        clave_aes_cifrada = PKCS1_OAEP.new(destinatario.clave_rsa.publickey()).encrypt(clave_aes)
        return cifrador_aes.nonce, tag, clave_aes_cifrada, cifrado

    def descifrar(self, nonce, tag, clave_aes_cifrada, cifrado):
        clave_aes = PKCS1_OAEP.new(self.clave_rsa).decrypt(clave_aes_cifrada)
        original = AES.new(clave_aes, AES.MODE_EAX, nonce=nonce).decrypt_and_verify(cifrado, tag)
        return original.decode()

ana = Usuario("Ana")
bob = Usuario("Bob")

nonce, tag, clave_aes_cifrada, cifrado = ana.cifrar_para("Hola Bob, quedamos a las 8", bob)
print(f"Bob recibe: {bob.descifrar(nonce, tag, clave_aes_cifrada, cifrado)}")

```

```

Bob recibe: Hola Bob, quedamos a las 8

```

cifrar_para cifra el mensaje con AES y protege la clave AES con la **pública del destinatario**. descifrar invierte el proceso usando la **privada del propio usuario** y verifica el tag. Es el esquema del [punto 8](#): cada usuario guarda su privada y solo él puede descifrar lo que le envían.

Boletín U09 — Avanzado



Boletín U09 — Avanzado

Ejercicios que requieren aplicar AES, RSA, firmas y cifrado híbrido de forma más profunda, con programas completos.

1. Cifrado híbrido simplificado

Genera una clave AES de 32 bytes y cifra `b"El cifrado hibrido funciona"`. Luego cifra esa clave AES con una clave RSA pública. Descifra en orden inverso y verifica el mensaje original.

Pista: cifra el mensaje con `AES.new(clave_aes, AES.MODE_EAX)` y `encrypt_and_digest`; cifra la clave AES con `PKCS1_OAEP.new(clave.publickey()).encrypt(clave_aes)`. Para descifrar, invierte el orden: primero RSA (con la privada), luego AES (con el mismo nonce).

2. Firma alterada

Firma digitalmente el mensaje `"Transferencia de 500€".encode()`. Modifica UN byte de la firma y comprueba que la verificación falla con `pkcs1_15.new(...).verify(...)`.

Pista: para modificar un byte de la firma, conviértela a `bytearray`, cambia un índice (`firma_mutada[0] ^= 0xFF`) y vuelve a `bytes`. La verificación debe lanzar `ValueError` o `TypeError`.

3. RBAC con permisos cifrado

Crea un sistema RBAC con 3 roles: `admin` (cifrar, descifrar, firmar), `usuario` (cifrar, firmar), `invitado` (solo cifrar). Implementa la función `puede(usuario, accion)` y pruébala con cada rol.

Pista: un diccionario `permisos = {"admin": ["cifrar", "descifrar", "firmar"], ...}` y `return accion in permisos.get(usuario["rol"], [])`. Los roles desconocidos deben devolver `False`.

4. Cifrar archivo completo

Cifra un archivo de texto con AES y guarda el resultado. Luego descifralo.

Pista: abre el archivo en modo binario "rb" / "wb". Usa AES.MODE_EAX, cifra con encrypt_and_digest y guarda nonce + tag + cifrado. Para descifrar, separa los tres componentes y usa decrypt_and_verify.

5. RSA: cifrar mensajes largos

RSA solo cifra ~190 bytes. Intenta cifrar un mensaje de 300 bytes. ¿Qué pasa? ¿Cómo lo arreglas?

Pista: captura el ValueError para ver el mensaje de error. Piensa en combinar RSA con un cifrado simétrico como AES (cifrado híbrido).

6. Intercambio de claves simulado

Simula el intercambio de claves entre Ana y Bob: Ana genera RSA, Bob cifra una clave AES con RSA pública de Ana.

Pista: define una clave AES de 32 bytes del lado de Bob. Bob cifra el mensaje con AES, luego cifra la clave AES con la RSA pública de Ana. Envía los 4 componentes (clave_AES_cifrada, nonce, tag, cifrado). Ana descifra en orden inverso.

7. Firma con verificación de integridad

Firma un mensaje, modifica el mensaje, y muestra que la verificación falla.

Pista: usa SHA256.new(mensaje) y pkcs1_15.new(clave).sign(h). Después de firmar el original, crea un segundo mensaje modificado y verifícalo con la misma firma. La verificación debe lanzar ValueError o TypeError.

8. RSA vs AES benchmark

Mide cuánto tarda cifrar el mismo mensaje con RSA y AES. La diferencia es abismal.

Pista: usa time.time() antes y después de un bucle de 100 cifrados RSA y otro de 1000 cifrados AES. Con time.time() - t obtienes los segundos. Multiplica por 1000 para milisegundos.

9. Sistema de cifrado de extremo a extremo

Simula un chat cifrado: cada usuario tiene su par RSA, y los mensajes se cifran con AES + RSA híbrido.

Pista: crea una clase `Usuario` con `nombre` y `clave_rsa`. Un método `cifrar_para(mensaje, destinatario)` que genera clave AES, cifra el mensaje y la clave. Otro método `descifrar(nonce, tag, clave_aes_cifrada, cifrado)` que hace el proceso inverso.

Boletín U10 — Inicial (Resuelto)

Boletín U10 — Inicial (Resuelto)

1. Servidor eco básico

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos) # eco: devuelve exactamente lo recibido
```

Solo atiende **UN cliente** y termina. `with conn`: cierra el socket al salir.

2. Servidor eco con bucle

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        with conn:
            datos = conn.recv(1024)
            conn.sendall(datos) # eco
```

Clientes uno tras otro. Si uno tarda, los demás esperan: es el servidor secuencial del [punto 1](#).

3. Cliente eco

```
import socket
with socket.socket() as s:
    s.connect(("127.0.0.1", 5000))
    s.sendall(b"Hola eco")
    print(s.recv(1024).decode())
```

Se conecta, envía b"Hola eco" y muestra la respuesta del servidor (el mismo mensaje, de vuelta).

4. Servidor secuencial

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK")
```

Solo atiende UN cliente y termina: accept() una vez, responde b"OK" y cierra.

5. Servidor multihilo

```
import socket, threading
def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Cada cliente en su propio hilo. Todos se atienden en paralelo ([punto 3](#)).

6. Cliente con respuesta

```
import socket
with socket.socket() as s:
    s.connect(("127.0.0.1", 5000))
    s.sendall(b"Hola")
    print(s.recv(1024).decode())
```

Muestra por pantalla la respuesta del servidor. Con el servidor multihilo del ejercicio 5, recibiría OK.

7. Servidor con ThreadPoolExecutor

```
import socket, concurrent.futures
def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
with socket.socket() as srv, concurrent.futures.ThreadPoolExecutor(3) as pool:
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        pool.submit(atender, conn, addr)
```

Máximo **3 hilos**. Los clientes adicionales esperan en cola ([punto 4](#)).

8. Lanzador de 5 clientes

```
import socket, threading
def cliente(id):
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(f"Soy {id}".encode())
        print(s.recv(1024).decode())
hilos = [threading.Thread(target=cliente, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Lanza **5 clientes a la vez** contra el servidor. El `join()` espera a que terminen todos antes de acabar el script.

Boletín U10 — Inicial



Boletín U10 — Inicial

Ejercicios básicos para afianzar los conceptos de servidores concurrentes: sockets TCP, el servidor secuencial y sus límites, y el salto a los hilos.

1. Servidor eco básico

Crea un servidor TCP que reciba datos de un cliente y los devuelva exactamente igual (eco). Sin bucle, atiende un solo cliente y termina.

2. Servidor eco con bucle

Modifica el servidor anterior para que acepte clientes en un bucle infinito, dando eco a cada uno.

3. Cliente eco

Crea un cliente TCP que se conecte al servidor, envíe `b"Hola eco"` y muestre la respuesta recibida.

4. Servidor secuencial

Crea un servidor TCP que atienda a **un solo cliente** y termine: `accept()`, recibe, responde `b"OK"` y cierra.

Pista: es la versión mínima del servidor secuencial: `with socket.socket() as srv: → bind(("127.0.0.1", 5000)) → listen() → accept() → with conn: → recv() y sendall(b"OK")`.

5. Servidor multihilo

Convierte el servidor anterior en uno **multihilo**: dentro de un `while True`, cada cliente que acepte se atiende en su propio `threading.Thread`.

Pista: define `def atender(conn, addr):` con el eco, y en el bucle lanza `threading.Thread(target=atender, args=(conn, addr)).start()`. El bucle vuelve al `accept()` al instante.

6. Cliente con respuesta

Crea un cliente TCP que se conecte al servidor, envíe `b"Hola"` y muestre por pantalla la respuesta que reciba.

7. Servidor con ThreadPoolExecutor

Implementa un servidor TCP que use `concurrent.futures.ThreadPoolExecutor` con **3 hilos** (`max_workers=3`). Responde `b"OK"` a cada cliente.

Pista: with `socket.socket()` as `srv`, `ThreadPoolExecutor(max_workers=3)` as `pool`: y en el bucle `pool.submit(atender, conn, addr)`. Con 3 hilos, el 4o cliente espera en cola.

8. Lanzador de 5 clientes

Crea un script que lance **5 clientes simultáneos** contra `127.0.0.1:5000`, cada uno enviando `f"Soy {id}"`, y muestre la respuesta de cada uno.

Pista: haz una lista de hilos `[threading.Thread(target=cliente, args=(i,)) for i in range(5)]`, lánzalos con `h.start()` y espera a todos con `h.join()`. La función `cliente(id)` conecta, envía y muestra `s.recv(1024).decode()`.

Boletín U10 — Avanzado (Resuelto)

Boletín U10 — Avanzado (Resuelto)

1. Servidor con timeout

```
import socket

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print("🕒 Servidor con timeout de 5 segundos")
    while True:
        conn, addr = srv.accept()
        conn.settimeout(5)          # 🕒 5 segundos máximos para recv
        with conn:
            try:
                datos = conn.recv(1024)
                print(f" Recibido de {addr}: {datos.decode()}")
            except socket.timeout:
                print(f" 🕒 {addr} no envió nada en 5s → cierro conexión")
```

`conn.settimeout(5)` limita el `recv()`: si el cliente no envía datos en 5 segundos, salta `socket.timeout` y el `with conn:` cierra el socket ([punto 7](#)). Un cliente mudo ya no cuelga un hilo para siempre.

2. Cliente con timeout

```

import socket

try:
    with socket.socket() as s:
        s.settimeout(2)          # 🕒 2 segundos para conectar y recibir
        s.connect(("127.0.0.1", 9999))
        s.sendall(b"ping")
        print(s.recv(1024))
except socket.timeout:
    print("Servidor no disponible")

```

Al conectar a un puerto en el que nada escucha, o al no recibir respuesta, `socket.timeout` se captura y se muestra “Servidor no disponible” en lugar de un fallo sin control.

3. Contador de bytes totales

```

import socket, threading

total_bytes = 0
lock = threading.Lock()

def atender(conn, addr):
    global total_bytes
    with conn:
        datos = conn.recv(1024)
        with lock:          # 🗝️ actualización atómica
            total_bytes += len(datos)
            print(f" {addr} envió {len(datos)} bytes – Total: {total_bytes}")

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()

```

El `recv()` queda **fuera** del `Lock` (cada socket es independiente) y solo la suma al contador global entra en `with lock`:: así varios hilos suman bytes sin condición de carrera ([punto 6](#)).

4. Servidor con límite de clientes

```
import socket, threading

MAX_CLIENTES = 3
clientes_activos = 0
lock = threading.Lock()

def atender(conn, addr):
    global clientes_activos
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
    with lock:
        clientes_activos -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print(f"🟡 Máximo {MAX_CLIENTES} clientes a la vez")
    while True:
        conn, addr = srv.accept()
        with lock:
            if clientes_activos >= MAX_CLIENTES:
                conn.sendall(b"Servidor completo")
                conn.close()
                print(f"❌ {addr} rechazado: servidor completo")
                continue
            clientes_activos += 1
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Al aceptar se comprueba el máximo bajo with lock:. El cuarto cliente recibe "Servidor completo" y se cierra **sin incrementar** el contador; los que entran lo decrementan al terminar.

5. Prueba de carga

```

import socket, threading, time

resultados = {}
lock = threading.Lock()

def cliente(id):
    inicio = time.time()
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(b"ping")
            s.recv(1024)
        exito = True
    except Exception:
        exito = False
    fin = time.time()
    with lock:
        resultados[id] = (exito, fin - inicio)

hilos = [threading.Thread(target=cliente, args=(i,)) for i in range(20)]
for h in hilos: h.start()
for h in hilos: h.join()

exitosos = sum(1 for ok, _ in resultados.values() if ok)
tiempos = [t for ok, t in resultados.values() if ok]
media = sum(tiempos) / len(tiempos) if tiempos else 0
print(f"Exitosos: {exitosos}/20 | Total: {sum(tiempos):.2f}s | Media: {media:.2f}s")

```

Cada cliente guarda su resultado (éxito + duración) en un diccionario compartido protegido por Lock. Tras join() a todos, se calculan las estadísticas: es el **benchmark** del [punto 5](#) llevado a 20 clientes.

6. Servidor con cola de espera

```

import socket, threading, queue

COLA = queue.Queue()

def atender():
    while True:
        conn, addr = COLA.get()          # 🔔 espera a que haya trabajo
        with conn:
            datos = conn.recv(1024)
            conn.sendall(b"OK: " + datos)
            print(f"  Atendido {addr}")
        COLA.task_done()

N_TRABAJADORES = 3
for _ in range(N_TRABAJADORES):
    threading.Thread(target=atender, daemon=True).start()

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print("🔔 Cola de espera con 3 trabajadores")
    while True:
        conn, addr = srv.accept()
        COLA.put((conn, addr))          # 📦 encola la conexión

```

El hilo principal solo **acepta y encola**. Los N trabajadores (daemon) sacan conexiones con `COLA.get()` y las atienden: es un ThreadPool hecho a mano con `queue.Queue`, el mecanismo interno de los pools del [punto 4](#).

7. Estado del servidor

```

import socket, threading

activas = 0
lock = threading.Lock()

def atender(conn, addr):
    global activas
    with conn:
        comando = conn.recv(1024).decode().strip()
        if comando == "STATUS":
            with lock:
                conn.sendall(f"Activas: {activas}".encode())
        else:
            conn.sendall(b"OK")
    with lock:
        activas -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        with lock:
            activas += 1
        threading.Thread(target=atender, args=(conn, addr)).start()

```

Un cliente que envíe "STATUS" recibe el número de conexiones activas (el resto recibe "OK"). El contador se incrementa al aceptar y se decrementa al salir, siempre bajo Lock.

8. Heartbeat en servidor

```

import socket, threading, time

activas = 0
lock = threading.Lock()

def heartbeat():
    while True:
        time.sleep(5)
        with lock:
            print(f"💖 Servidor vivo - {activas} conexiones")

threading.Thread(target=heartbeat, daemon=True).start()

def atender(conn, addr):
    global activas
    with lock:
        activas += 1
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
    with lock:
        activas -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()

```

El hilo heartbeat es **daemon** (muere con el servidor) y cada 5s imprime el estado leyendo el contador bajo Lock. Es la semilla del *heartbeat* de disponibilidad que ampliarás en la [U11 . asyncio](#).

9. Balanceador de carga simple


```

import socket, threading

PUERTOS_WORKERS = [5001, 5002]
indice = 0

def worker(puerto):
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", puerto))
        srv.listen()
        print(f" ⚙️ Worker escuchando en {puerto}")
        while True:
            conn, addr = srv.accept()
            with conn:
                datos = conn.recv(1024)
                conn.sendall(f"Worker-{puerto}: ".encode() + datos)

for puerto in PUERTOS_WORKERS:
    threading.Thread(target=worker, args=(puerto,), daemon=True).start()

def balanceador():
    global indice
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print(f"⚖️ Balanceador en 127.0.0.1:5000 (round-robin)")
        while True:
            conn, addr = srv.accept()
            puerto = PUERTOS_WORKERS[indice % len(PUERTOS_WORKERS)]
            indice += 1
            with conn:
                datos = conn.recv(1024)
                with socket.socket() as worker_s:
                    worker_s.connect(("127.0.0.1", puerto))
                    worker_s.sendall(datos)
                    respuesta = worker_s.recv(1024)
                conn.sendall(respuesta)
            print(f" {addr} → worker {puerto}")

balanceador()

```

El balanceador (puerto 5000) hace de **proxy**: lee del cliente, reenvía al worker elegido por **round-robin** (índice % 2), recibe su respuesta y la devuelve al cliente. Los workers 5001 y 5002 son servidores normales, cada uno en su hilo. Los clientes que envíen varias peticiones verán cómo se alterna la respuesta Worker-5001/Worker-5002.

Boletín U10 — Avanzado



Boletín U10 — Avanzado

Ejercicios que requieren aplicar la concurrencia de forma más profunda: timeouts, contadores sincronizados, límites de clientes, colas de espera, heartbeats y hasta un balanceador de carga.

1. Servidor con timeout

Implementa un servidor que cierre la conexión si el cliente no envía datos en 5 segundos (usa `socket.settimeout` o `select`).

2. Cliente con timeout

Crea un cliente que intente conectarse a `127.0.0.1:9999` con timeout de 2 segundos. Captura la excepción `socket.timeout` y muestra “Servidor no disponible”.

3. Contador de bytes totales

Añade un contador global de bytes recibidos (protegido con `Lock`) al servidor multihilo. Cada vez que un cliente envía datos, suma los bytes y muestra el total acumulado.

Pista: `total_bytes = 0 + lock = threading.Lock()`. Dentro de `atender`, haz `with lock:` para hacer `total_bytes += len(datos)` y `print`. El `recv()` y `sendall()` no necesitan `Lock` (cada socket es independiente).

4. Servidor con límite de clientes

Crea un servidor que acepte máximo 3 clientes. El cuarto recibe “Servidor completo” y se cierra.

Pista: Usa una variable global `clientes_activos` protegida con `Lock`. Al aceptar un cliente, comprueba si ya se alcanzó el máximo; si es así, envía el mensaje y cierra sin incrementar el contador. Decrementa al terminar.

5. Prueba de carga

Lanza 20 clientes simultáneos contra tu servidor y mide cuánto tardan todos en recibir respuesta.

Pista: Usa un diccionario resultados compartido para guardar el tiempo de cada cliente. Cada cliente mide `time.time()` antes y después de la conexión. Lanza 20 hilos, haz `join` a todos y calcula estadísticas (total, media, exitosos).

6. Servidor con cola de espera

Cuando el pool está lleno, los clientes entran en una cola. Cuando un hilo se libera, atiende al siguiente.

Pista: Usa `queue.Queue` para encolar las conexiones aceptadas. Crea N hilos trabajadores (daemon) que saquen elementos de la cola con `cola.get()` y atiendan al cliente. El hilo principal solo acepta y encola.

7. Estado del servidor

Añade un endpoint especial: si el cliente envía "STATUS", el servidor responde con número de conexiones activas.

Pista: Mantén un contador activas con Lock. En la función de atención, parsea el comando: si es "STATUS", responde con el valor actual del contador; si no, responde "OK". Incrementa al entrar, decrementa al salir.

8. Heartbeat en servidor

El servidor tiene un hilo heartbeat que imprime "💖 Servidor vivo — N conexiones" cada 5s.

Pista: Crea un hilo `daemon=True` con un bucle infinito que haga `time.sleep(5)` y luego imprima el estado usando el Lock para leer el contador de conexiones activas.

9. Balanceador de carga simple

Crea un "balanceador" que recibe peticiones y las distribuye entre 2 servidores workers.

Pista: Crea dos workers en los puertos 5001 y 5002 (cada uno en su hilo). El balanceador en el puerto 5000 acepta conexiones y las reenvía al worker actual haciendo de proxy: lee del cliente, envía al worker, recibe la respuesta y la reenvía al cliente. Alterna entre workers con un índice round-robin.

Boletín U11 — Inicial (Resuelto)



Boletín U11 — Inicial (Resuelto)

1. Mensaje diferido

```
import asyncio

async def preparar():
    print("Preparando...")
    await asyncio.sleep(2)
    print("¡Listo!")

asyncio.run(preparar())
```

`await asyncio.sleep(2)` pausa la corrutina 2 segundos; `asyncio.run()` crea el event loop.

2. Saludo y despedida

```
import asyncio

async def saludar():
    await asyncio.sleep(0.5)
    print("Hola")

async def despedirse():
    await asyncio.sleep(1)
    print("Adiós")

async def main():
    await asyncio.gather(saludar(), despedirse())

asyncio.run(main())
```

`gather` ejecuta ambas “a la vez”: el total es ~1s (la más lenta), no 1.5s.

3. Temporizador

```
import asyncio

async def tic():
    for i in range(4):
        print("tic")
        await asyncio.sleep(2)

asyncio.run(tic())
```

4 “tic” espaciados 2 segundos. Un `asyncio.sleep` dentro de un `for`.

4. Corrutina que devuelve

```
import asyncio

async def sumar(a, b):
    await asyncio.sleep(1)
    return a + b

async def main():
    resultado = await sumar(3, 4)
    print(resultado)

asyncio.run(main())
```

Para obtener el valor hace falta `await`: sin él tendrías un objeto corrutina, no el resultado.

5. Tres tareas a la vez

```
import asyncio, time

async def tarea(nombre, segundos):
    await asyncio.sleep(segundos)
    print(f" {nombre} terminó")

async def main():
    inicio = time.time()
    await asyncio.gather(
        tarea("A", 1), tarea("B", 2), tarea("C", 3)
    )
    print(f"Tiempo total: {time.time() - inicio:.2f}s")

asyncio.run(main())
```

~3 segundos, el de la tarea más lenta. gather las lanza concurrentemente: los tiempos no se suman.

6. Tarea en segundo plano

```
import asyncio

async def contar():
    for i in range(1, 5):
        print(f" contar: {i}")
        await asyncio.sleep(0.5)

async def main():
    asyncio.create_task(contar()) # en segundo plano
    await asyncio.sleep(1)
    print("main sigue")

asyncio.run(main())
```

contar() se ejecuta “en segundo plano”: mientras main() espera su segundo, la tarea cuenta. Se ve la alternancia de las dos corrutinas en el event loop.

7. Heartbeat básico


```

import asyncio

async def heartbeat():
    while True:
        print("💓 vivo")
        await asyncio.sleep(2)

async def main():
    asyncio.create_task(heartbeat())
    await asyncio.sleep(5)
    print("main terminó")

asyncio.run(main())

```

`create_task` lanza el latido en segundo plano; `main()` espera 5s y lo deja latir tres veces antes de terminar. Si `main()` no esperara, la tarea moriría con él.

8. Timeout básico

```

import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Hecho"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=3)
        print(r)
    except asyncio.TimeoutError:
        print("Timeout!")

asyncio.run(main())

```

`wait_for` corta a los 3s: salta `asyncio.TimeoutError`, la corrutina `lenta()` se cancela y el programa imprime "Timeout!".

Boletín U11 — Inicial



Boletín U11 — Inicial

Ejercicios básicos para afianzar los conceptos de asyncio: corrutinas, el event loop, gather, las tareas en segundo plano y los primeros mecanismos de disponibilidad.

1. Mensaje diferido

Crea una corrutina `async def preparar()` que imprima “Preparando...”, espere 2 segundos con `await asyncio.sleep(2)`, e imprima “¡Listo!”. Ejecútala con `asyncio.run()`.

2. Saludo y despedida

Crea dos corrutinas: `saludar()` (espera 0.5s y dice “Hola”) y `despedirse()` (espera 1s y dice “Adiós”). Ejecuta ambas con `asyncio.gather`.

3. Temporizador

Crea una corrutina que imprima “tic” cada 2 segundos, 4 veces. Usa `asyncio.sleep(2)` dentro de un bucle `for`.

4. Corrutina que devuelve

Crea una corrutina `sumar(a, b)` que espere 1 segundo y devuelva `a + b`. Lánzala y muestra el resultado.

Pista: llamar a una corrutina no ejecuta su cuerpo: devuelve un objeto corrutina. Para obtener el resultado usa `await sumar(3, 4)` dentro de una corrutina `main()` y ejecuta con `asyncio.run(main())`.

5. Tres tareas a la vez

Crea 3 corrutinas que esperen 1, 2 y 3 segundos. Lánzalas con `asyncio.gather` y mide el tiempo total con `time.time()`. ¿Cuánto dura en vez de 6 segundos?

Pista: con `gather` el total es el de la tarea más lenta (3s), no la suma. Guarda `inicio = time.time()` antes del `gather` y resta después.

6. Tarea en segundo plano

Crea una corrutina `contar()` que imprima un número cada 0.5s (hasta 4). Lánzala con `asyncio.create_task` desde una corrutina `main()` que haga `await asyncio.sleep(1)` y luego imprima “main sigue”. ¿Qué se imprime mientras tanto?

7. Heartbeat básico

Crea una corrutina `heartbeat` que imprima “ vivo” cada 2 segundos. Lánzala con `create_task` desde `main()` y deja que `main()` espere 5 segundos antes de terminar.

Pista: el `heartbeat` es un `while True` con `await asyncio.sleep(2)`. `main()` debe esperar (`await asyncio.sleep(5)`) o la tarea de fondo muere con él.

8. Timeout básico

Crea una corrutina `lenta()` que tarde 8 segundos y devuelva “Hecho”. Usa `asyncio.wait_for` con `timeout` de 3s y muestra qué excepción salta y qué imprime tu `except`.

Pista: `asyncio.wait_for(lenta(), timeout=3)` lanza `asyncio.TimeoutError` al pasar los 3 segundos y cancela la corrutina `lenta`.

Boletín U11 — Avanzado (Resuelto)



Boletín U11 — Avanzado (Resuelto)

1. Backoff exponencial

```
import asyncio

async def conectar(intentos=4):
    for i in range(intentos):
        espera = 2 ** i          # 1, 2, 4, 8
        try:
            print(f"Intento {i+1}...")
            raise ConnectionRefusedError # Simular error
        except ConnectionRefusedError:
            print(f"  Fallo: espero {espera}s")
            await asyncio.sleep(espera)
    print("Servicio no disponible")

asyncio.run(conectar())
```

Backoff exponencial: 1s, 2s, 4s, 8s. El último intento falla y se imprime “Servicio no disponible”.

2. Timeout con respaldo

```
import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Resultado real"

async def respaldo():
    return "Resultado en caché"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=5)
        print(r)
    except asyncio.TimeoutError:
        r = await respaldo()
        print(r)

asyncio.run(main())
```

A los 5s salta `TimeoutError`; el `except` ejecuta la corrutina de respaldo que devuelve “Resultado en caché”.

3. Dos heartbeats

```

import asyncio

async def hb_a():
    while True:
        print("💓 A")
        await asyncio.sleep(3)

async def hb_b():
    while True:
        print("💓 B")
        await asyncio.sleep(5)

async def main():
    asyncio.create_task(hb_a())
    asyncio.create_task(hb_b())
    await asyncio.sleep(12)  # da tiempo a que latan
    print("main terminó")

asyncio.run(main())

```

Los dos latidos se ejecutan en segundo plano; `main()` espera 12s (A late 4 veces, B unas 2). Sin ese `await`, las tareas se cancelarían al acabar `main()`.

4. 🎯 Web scraper asíncrono

```

import asyncio, httpx, time

URLS = ["https://httpbin.org/get"] * 5

async def descargar(url):
    async with httpx.AsyncClient() as cliente:
        r = await cliente.get(url)
        return r.status_code

async def main_async():
    tareas = [descargar(u) for u in URLS]
    resultados = await asyncio.gather(*tareas)
    print("Resultados:", resultados)

inicio = time.time()
asyncio.run(main_async())
print(f"Versión asíncrona: {time.time() - inicio:.2f}s")

# Versión síncrona equivalente para comparar:
inicio = time.time()
with httpx.Client() as cliente:
    for u in URLS:
        cliente.get(u)
print(f"Versión síncrona: {time.time() - inicio:.2f}s")

```

`gather(*tareas)` lanza las 5 descargas a la vez: la versión asíncrona tarda lo que la más lenta, la síncrona la suma de todas.

5. Servidor asyncio con heartbeat

```

import asyncio

conexiones = 0

async def atender(reader, writer):
    global conexiones
    conexiones += 1
    datos = await reader.read(1024)
    writer.write(b"OK: " + datos)
    await writer.drain()
    writer.close()
    await writer.wait_closed()
    conexiones -= 1

async def heartbeat():
    while True:
        print(f"💖 Vivo — {conexiones} conexiones")
        await asyncio.sleep(5)

async def main():
    asyncio.create_task(heartbeat())          # latido en segundo plano
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    print("🚀 Servidor ASYNCIO en 127.0.0.1:5000")
    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())

```

`create_task(heartbeat())` lanza el latido antes de aceptar conexiones. Cada 5s imprime “💖 Vivo — N conexiones”. Se mata con Ctrl+C.

6. 🕒 Monitorización de servidores


```

import asyncio, random

estados = {"Server-A": True, "Server-B": True, "Server-C": True}

async def servidor_simulado(nombre):
    """Cambia su estado aleatoriamente cada 5-15s."""
    while True:
        await asyncio.sleep(random.randint(5, 15))
        estados[nombre] = random.choice([True, False])
        print(f" [{nombre}] cambia a {'ARRIBA' if estados[nombre] else 'CAÍDO'}")

async def monitor():
    while True:
        await asyncio.sleep(3)
        caidos = [n for n, ok in estados.items() if not ok]
        if caidos:
            print(f"🚨 Caídos: {' '.join(caidos)}")
        else:
            print("🟢 Todo en pie")

async def main():
    for nombre in estados:
        asyncio.create_task(servidor_simulado(nombre))
    await monitor()          # el monitor corre para siempre

asyncio.run(main())

```

Cada servidor simulado cambia su estado de forma aleatoria; el monitor lee el diccionario cada 3s y reporta los caídos. Es el heartbeat del [punto 8](#) aplicado a varios servicios a la vez.

7. 🧩 Semáforo asyncio

```

import asyncio, httpx, time

sem = asyncio.Semaphore(3)          # solo 3 a la vez

async def descargar(n):
    async with sem:                  # reserva un hueco del semáforo
        async with httpx.AsyncClient() as cliente:
            r = await cliente.get(f"https://httpbin.org/delay/{n}")
            return r.status_code

async def main():
    tareas = [descargar(n) for n in range(1, 7)]  # 6 descargas
    resultados = await asyncio.gather(*tareas)
    print("Resultados:", resultados)

inicio = time.time()
asyncio.run(main())
print(f"Tiempo total: {time.time() - inicio:.2f}s")

```

`async with sem:` garantiza que como máximo 3 corrutinas ejecutan el bloque a la vez; las otras 3 esperan su turno.

8. 🤡 Timeout con fallback

```
import asyncio

async def descargar_principal():
    await asyncio.sleep(5)          # servidor lento
    return "Datos del principal"

async def descargar_respaldo():
    await asyncio.sleep(1)
    return "Datos del respaldo"

async def main():
    try:
        r = await asyncio.wait_for(descargar_principal(), timeout=2)
    except asyncio.TimeoutError:
        r = await descargar_respaldo()
    print(r)

asyncio.run(main())
```

A los 2s salta `TimeoutError` y el `except` ejecuta la corrutina de respaldo. La disponibilidad gana: siempre se devuelve una respuesta, aunque venga del servidor de backup.

9. Chat asíncrono

```

import asyncio

clientes = set()

async def broadcast(mensaje, emisor=None):
    for writer in list(clientes):          # copia: permite eliminar al iterar
        if writer != emisor:
            writer.write(mensaje)
            await writer.drain()

async def gestionar(reader, writer):
    clientes.add(writer)
    print(f"Cliente conectado ({len(clientes)} en total)")
    try:
        while True:
            try:
                datos = await asyncio.wait_for(reader.read(1024), timeout=60)
            except asyncio.TimeoutError:
                break                      # desconexión silenciosa
            if not datos:
                break
            await broadcast(b"[" + writer.get_extra_info("peername")[0].encode() + b"]"
" + datos, emisor=writer)
        finally:
            clientes.discard(writer)
            writer.close()
            await writer.wait_closed()

    async def main():
        servidor = await asyncio.start_server(gestionar, "127.0.0.1", 6000)
        print("💬 Chat ASYNCIO en 127.0.0.1:6000")
        async with servidor:
            await servidor.serve_forever()

    asyncio.run(main())

```

broadcast reenvía a todos los clientes menos al emisor. `wait_for(..., timeout=60)` detecta desconexiones: si un cliente no envía nada en 60s, se cierra su conexión y se quita del conjunto.

Boletín U11 — Avanzado



Boletín U11 — Avanzado

Ejercicios que requieren aplicar asyncio de forma más profunda: backoff, timeouts con respaldo, heartbeats múltiples, descargas concurrentes, servidores con latido y monitorización de varios servicios.

1. Backoff exponencial

Crea una función asíncrona `conectar()` que intente conectarse 4 veces con backoff (1s, 2s, 4s, 8s). Simula el fallo lanzando `ConnectionRefusedError`. Si falla, imprime “Servicio no disponible”.

Pista: `espera = 2 ** intento`. Envuelve cada intento en `try/except ConnectionRefusedError`, espera con `await asyncio.sleep(espera)` y pasa al siguiente.

2. Timeout con respaldo

Crea una corrutina `lenta()` que tarde 8 segundos. Usa `asyncio.wait_for` con timeout de 5s. Si salta `TimeoutError`, ejecuta una corrutina de respaldo que devuelva “Resultado en caché”.

Pista: `asyncio.wait_for(lenta(), timeout=5)` lanza `TimeoutError`; en el `except` haz `await respaldo()`.

3. Dos heartbeats

Crea dos corrutinas `heartbeat`: `hb_a()` imprime “❤️ A” cada 3s, `hb_b()` imprime “❤️ B” cada 5s. La función `main` las lanza con `asyncio.create_task` y espera 12 segundos.

Pista: dos `create_task` y luego `await asyncio.sleep(12)` para que `main()` no las cancele antes de tiempo.

4. 🎯 Web scraper asíncrono

Descarga 5 URLs a la vez con `asyncio` y `httpx`. Compara el tiempo con una versión síncrona.

Pista: usa `httpx.AsyncClient` dentro de cada corrutina. Crea una lista de tareas con `[descargar(u) for u in urls]` y ejecútalas con `asyncio.gather(*tareas)`. Mide el tiempo total con `time.time()`.

5. 🔍 Servidor asyncio con heartbeat

Servidor asyncio que imprime “💖 Vivo — N conexiones” cada 5s.

Pista: usa `asyncio.start_server` para el servidor TCP. Crea una corrutina `heartbeat` con un bucle infinito `while True: await asyncio.sleep(5); ...`. Lánzala con `asyncio.create_task` antes de iniciar el servidor.

6. 🕒 Monitorización de servidores

3 servidores simulados. Un monitor asíncrono comprueba su estado cada 3s.

Pista: usa un diccionario `estados = {"Server-A": True, ...}`. Cada servidor es una corrutina que cambia su estado aleatoriamente cada ~5-15s. El monitor lee el diccionario cada 3s y reporta servidores caídos.

7. 🚦 Semáforo asyncio

Limita a 3 descargas simultáneas usando `asyncio.Semaphore`.

Pista: crea `sem = asyncio.Semaphore(3)`. Dentro de la función de descarga, usa `async with sem:` para que solo 3 corrutinas puedan ejecutar el bloque a la vez. Usa `httpbin.org/delay/{n}` para simular descargas lentas.

8. 😊😞 Timeout con fallback

Intenta descargar de un servidor principal. Si tarda más de 2s, usa un servidor de respaldo.

Pista: envuelve la llamada a la corrutina principal con `asyncio.wait_for(descargar_principal(), timeout=2)`. Captura `asyncio.TimeoutError` y en el `except` ejecuta la corrutina de respaldo.

9. 🏠 Chat asíncrono

Crea un servidor de chat con `asyncio` que reenvíe mensajes a todos los clientes conectados.

Pista: mantén un conjunto `clientes` con los objetos `writer` de cada conexión. Crea una función `broadcast(mensaje, emisor=None)` que itere sobre una copia del conjunto y envíe a todos excepto al emisor. Usa `asyncio.wait_for(reader.read(1024), timeout=60)` para detectar desconexiones.